# THE OMNI-COMPASS®

## A UNIFIED GOVERNING CONVERGENCE CONTROL CORE PLANE ENGINE

A MATHEMATICALLY CLOSED CIRCLE

## FRAMEWORK




### DATA CENTERS & PHYSICAL INFRASTRUCTURE

- NVIDIA DGX Systems
- NVIDIA DGX Cloud / Base Command
- NVIDIA DCGM / GPU Operator
- NVIDIA Dynamic Power Software (DPS)
- NVIDIA Run:ai
- Phaidra (Autonomous Cooling)
- Schneider Electric EcoStruxure IT
- Vertiv

### CLOUD & MULTI-CLOUD PLATFORMS

- Amazon Web Services
- Microsoft Azure
- Google Cloud
- Oracle Cloud Infrastructure
- Alibaba Cloud
- IBM Cloud
- DigitalOcean

### CONTAINERS & ORCHESTRATION

- Kubernetes
- Red Hat OpenShift
- Amazon EKS
- Microsoft AKS
- Google GKE
- VMware Tanzu
- SUSE Rancher / RKE2

### WORKLOAD SCHEDULING & HPC

- Slurm
- KubeRay
- Ray
- Kueue

### RESOURCE OPTIMIZATION & AUTONOMOUS OPERATIONS

- IBM Turbonomic
- Apptio
- Moogsoft AIOps
- BigPanda

### AI APPS & PLATFORMS

- ChatGPT (OpenAI)
- Claude (Anthropic)
- Grok (xAI)

### NETWORKING, SECURITY & SERVICE MESH

- Istio
- Cilium
- Cloudflare
- Akamai
- Palo Alto Networks
- Fortinet
- CrowdStrike
- F5

### OBSERVABILITY & AIOPS

- Datadog
- Dynatrace
- New Relic
- AppDynamics
- Splunk
- Elastic
- Prometheus
- Grafana
- OpenTelemetry

### GITOPS & AUTOMATION

- Terraform
- Ansible
- Argo CD
- Flux
- GitHub Actions
- GitLab CI
- Jenkins

### DATA, STREAMING & ANALYTICS

- Snowflake
- Databricks
- MongoDB
- PostgreSQL
- Oracle
- Apache Kafka
- Redis

### IDENTITY, POLICY & GOVERNANCE

- Okta
- HashiCorp Vault
- Consul
- OPA / Gatekeeper

### IT OPERATIONS & SERVICES

- ServiceNow
- PagerDuty
- Splunk On-Call
- Jira

A UNIFIED GOVERNANCE.
BOUNDED EVOLUTION.
SUSTAINED COHERENCE.

OWNER AND DEVELOPER

**THE OMNI-COMPASS LLC.**

THE BRAIN AND NERVOUS SYSTEM
FOR FRAGMENTED STACKED
MUSCLE / MODULES.

STABILITY. CONVERGENCE. CONTINUITY.

www.omni-compass.com

U.S. PATENT, COPYRIGHT & TRADEMARK APPLICATIONS FILED

ALL RIGHTS RESERVED

# THE OMNI-COMPASS MANUAL

# THE OMNI-COMPASS MANUAL

*The Governor of Everything:*
*Its Philosophy, Its Mathematics, Its Physics,*
*and How to Wire It onto Your Stack*




**AJ Dubra**
Owner and Developer

**THE OMNI-COMPASS LLC**
October 2026

**The Omni-Compass Manual**
The Governor of Everything: Its Philosophy, Its Mathematics, Its Physics, and How to Wire It onto Your Stack
October 2026

Copyright © 2026 The Omni-Compass LLC. All rights reserved. No part of this book may be reproduced, stored or transmitted in any form or by any means, except as allowed by the Omni-Compass Evaluation License, without the written permission of The Omni-Compass LLC.

The Omni-Compass LLC reserves the right to change this book, the software it describes and their terms at any time, without notice. Every copy, export and printout of any part of this book carries this page, the LICENSE, the NOTICE and the Disclosures unchanged.

All patents, copyrights and trademarks covering the Omni-Compass engine, its mathematics and its software have been filed in the United States by The Omni-Compass LLC (no filing number is stated, and no grant, registration or approval is claimed).
www.omni-compass.com

“Omni-Compass”, the Omni-Compass rose and the related names and marks are trademarks of The Omni-Compass LLC. Other names that appear in this book are the property of their owners and are used only to identify the systems Omni-Compass works with.

**License.** This book and the software it describes are proprietary and are not open source (SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0). They may be used only to evaluate Omni-Compass and to reproduce its published results, in simulation and on systems the reader owns or controls. Any commercial use, commercialization, monetization, production use, redistribution, hosted or managed service, or incorporation into any product or service requires a written Omni-Compass Enterprise License, signed by The Omni-Compass LLC and paid for. The full terms are printed in Appendix I.

**Disclaimer.** The software is provided “as is”, without warranty of any kind. Every result in this book carries its evidence class. A simulated result is a statement about a model; a software benchmark is a statement about the software it ran on; only a hardware meter speaks for hardware. Nothing in this book is a promise of a particular saving on a particular system: the only number that applies to a system is the one its own paired runs produce, on its own receipt.

Front cover: the Omni-Compass identity plate. Back cover: the Omni-Compass unified control authority. Plates 1 to 19: the founder's plates of the theory.

Set in Liberation Serif and DejaVu Sans Mono. Built from the repository by `docs/book/build_book.py`; the book and the code change in the same commit.

The Omni-Compass LLC · www.omni-compass.com

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

# Contents

**Foreword** xv

**Preface** xvii

**Quick Reference: The Card for the Glove Box** xix

**Executive Summary** xxi

**Disclosures, Declarations and Disclaimers** xxiii

**Part One. The Philosophy and the Theory** 1

1. What Omni-Compass Is . . . 2
2. The Unified Circle Principle . . . 7
3. The Four-Piece Engine . . . 21
4. The Compass . . . 28
5. Basins, Polarity and the Dual-Basin Engine . . . 30
6. The Reflex Rule: One Body, One Brain . . . 33

**Part Two. The Mathematics** 35

7. The Canonical Declaration . . . 36
8. The Engine: Eight Equations and One Control Law . . . 43
9. The Canonical Engine . . . 46
10. The Closed Circle: Why It Cannot Leave Its Compass . . . 48
11. Closing the Circle in the Engine . . . 52
12. The Tracking Theorem . . . 53
13. The Conveyance Law . . . 56
14. Formal Status of the Mathematics . . . 59
15. The Mechanism of Action . . . 60
16. The Engines and Their Audit . . . 68

**Part Three. The Physics: the Compass and the Nervous System** **71**

17. The Compass: Push, Pull and the Two Forces . . . 72
18. The Physics of a Processor . . . 77
19. The Two-Way Nervous System . . . 78
20. The Nervous System in Detail . . . 81

**Part Four. The Body: Muscles, Realms and Organisms** **83**

21. The Muscles, the Realms and the Six Organisms . . . 84
22. The Four Realms . . . 89
23. The Realm Muscles . . . 91
24. The Domain Map . . . 102
25. The Problem Map . . . 108
26. The Six Organisms and the Benchmark Grid . . . 112
27. The Metrics Catalog . . . 113

**Part Five. The Harness and the Wiring** **117**

28. The Plug, the Adapters and the Wire Check . . . 118
29. The Harness . . . 121
30. The Wiring Guide . . . 123
31. Before You Start, and the Eight Levels . . . 126
32. Stack by Stack . . . 130
33. The Integration Manual . . . 141
34. Running the GPU Benchmark . . . 151

**Part Six. Operating It** **157**

35. The OFF Switch, the Rules, and the Log . . . 158
36. Maintenance, Upgrades and Security . . . 161
37. The Operator Manual . . . 163

**Part Seven. Proving It** **171**

38. How to Read the Results . . . 173
39. The Benefit Sheet: One Number per Benchmark . . . 178
40. The Dossier: Every Result in One Place . . . 185
41. Paired Runs and Receipts on Your Own System . . . 195
42. Evidence Classes and How to Read a Result . . . 197



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

43. Results to Date . . . 199
44. Wire In, or Watch: The Verdict per Knob . . . 217
45. The Pilot Protocol and Kit . . . 240
46. The GPU Bench . . . 242
47. The GPU Preregistration . . . 245
48. The Realms Preregistration . . . 256
49. The Compass Law on Real Kubernetes: Preregistration . . . 267
50. The Evidence Ledger . . . 288
51. The Claims Register . . . 294
52. The Benchmark Report . . . 301
53. The Referee Report . . . 322
54. Comparison with Existing Controllers . . . 335
55. The State of Play . . . 338

**Part Eight. Value, License and History** **349**

56. Where the Value Comes From . . . 350
57. The Economics of a Receipt . . . 356
58. The Buyer Edition . . . 357
59. Due Diligence . . . 365
60. License and Commercial Terms . . . 367
61. Licensing: Questions and Answers . . . 368
62. Third-Party Notices . . . 370
63. Repository Standards . . . 371
64. Python, C++ and the Seal . . . 373
65. The Founder's Working Notes . . . 375
66. History . . . 380

**Back Matter** **387**

Glossary . . . 388
Appendix A. Command Reference . . . 391
Appendix B. File Map . . . 394
Appendix C. The Equations in Full . . . 396
Appendix D. Metrics . . . 398
Appendix E. Troubleshooting . . . 400
Appendix F. Evidence Map . . . 402

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

Appendix G. The Muscles: What Each Is For, and How It Is Wired . . . . . 405
Appendix H. Source of the Engine . . . . . 464
Appendix I. The License . . . . . 491
Contact . . . . . 499



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

# List of Plates

Plate 1. The Unified Circle Principle

Plate 2. Closed-circle structural architecture of the unified law

Plate 3. The closed circle tested

Plate 4. Symmetry principles and conservation laws in the closed circle

Plate 5. Scale invariance and structural universality

Plate 6. The lineage of unification

Plate 7. A candidate for a closed, unified law

Plate 8. The Omni-Compass Grand Theory: the four pieces

Plate 9. The unified container of all reality: 24 domains, four-piece engine, one closure law

Plate 10. The four quadrants and the compression process

Plate 11. The closure law and the closed-circle engine

Plate 12. The Omni-Compass rose

Plate 13. The Omni-Compass Principle

Plate 14. Geometric funneling and basin formation

Plate 15. The Omni-Compass dual-basin engine

Plate 16. Mathematical integration

Plate 17. The engine core: the governing law in motion

Plate 18. The equation form and the symbol chart

Plate 19. Empirical validation, experimental pathways, and the discipline of proof

# Foreword

I built Omni-Compass to sit on top of what already exists. Kubernetes, the GPU driver, the building controller, the battery inverter, the robot's servo loop: those are muscles, and they are good muscles. What none of them has is one brain that reads all of them at once, holds each of them in the middle of its safe range, and puts every setting back exactly where it found it when it stops. That brain is Omni-Compass.

This manual is the whole of it: what the governor is, the mathematics it runs on, how its nervous system reads and writes, how it is wired to each kind of machine, how to turn it on one level at a time, how to switch it off, and how to prove on your own system what it does. Keep it with the code. When the code changes, this manual changes in the same commit.

**On this edition.** This edition is written for the referee and the underwriter as much as for the engineer. Every result in it was run three separate times on a frozen, fingerprinted engine, under rules written before the first run, and is reported with every loss beside every gain. Where a number is from a model it says so; where a test could not see an effect it says so; where something has not been shown it says so, in its own section. The questions a careful reader will ask are asked here first, in section 3.5 beside each result and in section 16.7 as a list, and answered where an answer exists. Nothing in this manual asks to be believed on our word: every table names the runs it was made from, every run's raw files are in the repository, and every table can be rebuilt from them by the tool it names.

**AJ Dubra** Founder, The Omni-Compass LLC

# Preface

I wrote this book so that whoever connects Omni-Compass to a running system understands what they are connecting: not only which wire goes where, but why the governor behaves the way it does, where its law comes from, and how to prove on their own system what it does for them.

The book is in eight parts, and it can be read in two ways.

| You are | Read |
|---|---|
| CEO, board member, investor | the Foreword, the Executive Summary, Part One, the Results to Date, and Part Eight |
| CTO, architect, head of platform | Parts One to Four, then the eight wiring levels in Part Five, then Part Seven |
| The engineer wiring it | everything, in order. Do not skip the wire check or the watch level |
| Data-center, facility or energy operator | the Quick Reference card, Part One's chapter on the reflex, Part Five (levels five to seven, the processor and the data center under the servers), Part Six |
| Site reliability engineer, on call | the Quick Reference card, Part Six (the switches and the log), the troubleshooting appendix |
| Auditor, diligence team | Part Two (the mathematics), Part Seven (the proof), Appendices C, F and H |
| Procurement and counsel | the Disclosures, the license in the back matter, Part Eight |

**Part One, the philosophy and the theory,** sets out the closed circle: a system that is closed, bounded and pulled toward a center cannot run away, and everything it does is a return. **Part Two, the mathematics,** gives the eight equations and the control law that make the circle exact, with the theorem, the declaration and the audits that hold them fixed. **Part Three, the physics,** brings the law into the machine: the compass, push and pull, the band and its cushions, the physics of a processor, and the two-way nervous system. **Part Four, the body,** lays out the muscles, the four realms and the six organisms. **Part Five, the harness and the wiring,** is the universal plug and the step-by-step work of wiring Omni-Compass onto a stack. **Part Six, operating it,** is the OFF switch, the rules and the log. **Part Seven, proving it,** is every rule written before a run and every result after it. **Part Eight** is value, the license, the seal and the history.

**Conventions.** `Code` is a command, file or switch exactly as typed. "Native" means a system as it runs today, without Omni-Compass. "Muscle" means any machine, service or controller Omni-Compass can read and set. "Knob" or "lever" is one setting on a muscle. "The band" is a knob's or a service reading's safe range. A step that writes to a system is marked **WRITES**. Every result carries its evidence class: T for a theorem, V for verification of the code, S for simulation, L for live software, P for a physical meter.

Some chapters gather documents that were written as the work went on, each at the moment its result came in. They are kept as they were written, because the record of how a result was reached is part of the proof. Where two chapters give different figures for the same thing, the later run and the State of Play govern.

# Quick Reference: The Card for the Glove Box

Keep this page where the people who run the system can reach it, the way a driver keeps the card in the glove box. Every line is explained in the section named beside it; nothing on this page replaces those sections.

**What it is.** A supervisory governor on top of the controllers you already run. It never replaces them: it moves only settings they already accept, inside a band you set, and hands every setting back when it stops (sections 1.1 and 8).

**The two names.** *Native* is your system as it runs on its own. *Omni* is the same system with Omni-Compass on top. There is no third arm and no other name (section 1.1).

**The modes** (section 1.3).

| Mode | What it does | When |
|---|---|---|
| Off (manual) | nothing; native runs alone | before installing, and whenever you choose |
| Watching | reads every gauge and logs what it would do; writes nothing | the first level of every installation (level 1) |
| Autopilot | idle, gas, brake and reset, by its law | after the watch level has passed |
| Cruise | every machine in service while a known queue waits | batch work |

**The pedals.** *Idle*: no gas, no brake, the floor of two machines in service, ready for the next burst. *Gas*: capacity added at once as traffic climbs, never held back. *Brake*: capacity eased off a step at a time, never below idle. *Reset*: the brake held to the floor; every setting handed back to where native had it, read back, and the record removed.

**The ways it stops, and what each leaves behind** (Chapter 35).

| You want to, or it happens that | Do this | What is left |
|---|---|---|
| a run ends | nothing: the reset runs by itself at the end of every run | every setting at the operator's value, read back |
| stop everything at once, for any security reason | `python3 tools/omni_switch.py off` | every governor hands back and exits; none starts again until `python3 tools/omni_switch.py on` |
| stop one muscle | its kill file, or `OMNI_KILL=1` | that muscle handed back; the others run on |
| Omni-Compass dies without handing back (killed, crashed, hung) | nothing: the watchdog's lease hands back for it | every setting back at the operator's value, measured at 7 to 11 seconds in 30 of 30 repetitions (section 3.5) |

**The eight levels of an installation** (section 9.3). Take them in order; go to the next only when the pass holds.

| Level | What it touches | Pass |
|---|---|---|
| 0 | nothing: `python3 verify.py` | `VERIFICATION: PASS` |
| 1 | nothing: watch mode, with a read-only identity | zero writes, and readings your operators agree with |
| 2 | the pods, through your autoscalers (**WRITES**) | p95, p99 and failed requests no worse than native, over paired runs |

| Level | What it touches | Pass |
|---|---|---|
| 3 | the machines, through the release gate and the verdict (**WRITES**) | fewer machines in service, with no service gauge worse |
| 4 | the replica decision itself (an option, not a step) | as level 3 |
| 5 | a GPU box, two wires: the clock ceiling and the power limit (**WRITES**) | work per energy up or equal; no request more than 2% slower at the median, p95 or p99 |
| 6 | the processor's clock and power on bare metal (**WRITES**) | energy per unit of work down, no service gauge worse |
| 7 | site power and the supply-air setpoint (**WRITES**) | the site meter's energy down, every rack inlet inside its band, no service gauge worse (section 10.11) |

**Before any level that writes:** run the wire check (section 8.4), start in watch, and keep the master switch in reach.

**Warning signs, and the first thing to do.**

| You see | First | Then read |
|---|---|---|
| a service slower or failing more with Omni than without | `python3 tools/omni_switch.py off`: the system is native again | section 8.5, wired right or wired wrong |
| a setting at a value Omni-Compass did not write | nothing: it stops writing that lever by itself (one writer) | section 7.2 |
| a meter that is blind or late | nothing: blind means hold; it moves nothing it cannot see | section 7.3 |
| your results point the other way from the published ones | presume the wiring first, not the law | `DISCLOSURES.md`, Chapter 8 |

**Who to call.** Licensing, pilots and the Omni-Compass Enterprise License: The Omni-Compass LLC, www.omni-compass.com. Security reports: the process in `SECURITY.md`, never a public issue.


*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

# Executive Summary

**What it is.** Omni-Compass is a supervisory governor. It sits on top of the controller a system already has, the Kubernetes autoscaler, the GPU's firmware, the cooling loop, the battery inverter, the robot's servo, the grid's tap changer, the database's connection pooler, the message broker's consumer group, the cache's memory ceiling, and it moves only the settings those controllers already accept. It reads each muscle's own meter, computes one bounded force per muscle from a single smooth law, and holds the service in the middle of its safe band rather than far below its limit. The native controller keeps running exactly as shipped; Omni-Compass sets what it already exposes, and when Omni-Compass stops, every setting goes back to the value it read once before its first write. It is never standalone and it never competes with the controller underneath it.

**What it does for you.** Every running system keeps room "just in case": machines sized for the worst minute of the day, a pool of connections held open for a load that comes once an hour, a consumer group sized for the quiet hours and then overrun at the peak, a cache ceiling set once and left. Omni-Compass turns that room into one of two things, and the operator chooses which: more work for the same resources, or the same work for fewer. The two are the same measurement read from two sides (section 3.4): a third more work from what is already installed is a quarter off the bill.

**What has been shown, on the frozen engine Omni v3, three separate times each, and run again on one commit on 10 October.** On real Kubernetes, seven preregistered tests of ten paired runs each (`results/live/V3_*.md`): work inside the response line **+54% to +68%** in every run of the all-four test, the 95th-percentile response time **57% to 66% faster** across the steady, wandering, all-four and fault tests, failed requests 11% to 18% fewer where load swings and under faults, machines 1.7% to 2.0% fewer at steady load and 16% to 20% fewer on a queue of batch jobs, 7% to 9% fewer under a public day of demand (the Google cluster trace of 2011), and beside a noisy neighbour no difference beyond the noise on any row. On the five live stacks, run for the first time with the brain's own verdict on the knob (every notch tried on the stack itself before it is written, section 9.4): a real database behind its pooler (`results/live/V3_PGBENCH.md`) held 6% to 11% fewer connections for the same work on its read-only workload and refused the add above the operator's setting on the slow write workload, with nothing worse anywhere; a real database's buffer pool, MySQL under sysbench (`results/live/V3_SYSBENCH.md`), was held 10% to 20% smaller on the read-and-write workload with nothing worse; a real database's storage-engine cache, MongoDB under YCSB (`results/live/V3_YCSB.md`), was held 3% to 13% smaller on one workload with nothing worse; on a real message broker, Apache Kafka (`results/live/V3_KAFKA.md`), and a real cache, Redis (`results/live/V3_REDIS.md`), the first set read exactly nothing, no gain and no loss, because every spend trial on Kafka and most on Redis were abandoned before they could be judged: the rule of 9 October ended a spend trial when the service turned calm, and a spend that works calms the service within seconds; that rule is ours, it was amended the same day (a spend trial runs to its samples) and the five stacks run again on it. The earlier sets, in which the compass bought Kafka's hundredfold shorter queue with three to four times the consumers and Redis's hit rate with three to four times the memory, are whole in `docs/history`. In the modelled realms, 945 muscles and six organisms (`results/realms/REALMS.md`): every organism superior within its guardrails, zero muscles worse, at every size from 1 to 1,000 copies, the published table reproduced in 3 of 3 again on 10 October. On the independent simulators: a power grid's losses better in 7 of

11 grids and worse in 4; robot arms' peak torque down 29% where Omni-Compass moved and left native where the paired trial said not to; buildings' electricity bought and daily peak better in all 11 battery districts and the bill worse in 7; drone swarms' energy a mission 7% to 20% lower with no late mission, near miss or collision. The one combined number, the Omni index over the six real categories the same in all three of our runs, stands at **+4.8%** on the sets of 10 October (`results/OMNI_INDEX.md`): Kubernetes +28.9% over seven tests, the database +0.8%, messaging +0.1%, the cache exactly nothing, the database's storage-engine cache +0.7% and the database's buffer pool +1.0%, every category weighed the same and every row inside the noise counted as exactly nothing. It stood at +20.5% on the sets of 7 to 9 October; it came down because the spends that had bought the stacks' earlier gains at a resource cost were refused by the brain's verdict, or on Kafka and Redis not yet judged, so that every gain left is one the brain proved on the stack itself, and no row is a loss. A second reading of the same tables, declared 9 October and scoring service alone (work and speed, with the resources shown and not scored), stands at **+10.7%** (Kubernetes +84.5%, the five stacks nothing); it is printed beside the first in the index file, and section 16.6 says what each of the two is for. Nothing is hidden in either.

**What has not been shown, said plainly.** An energy or cloud-bill saving on real machines. Energy on kind is a declared model, because the machines are containers on one runner. Azure's bill on a 4-worker fleet read no difference beyond the noise on every gauge, which is the honest reading of a fleet too small to show one machine; the 40-worker fleet that can show one is preregistered and has been refused by Azure's own cluster capacity in its region, each refusal recorded. Every earlier GPU result ran on a controller since replaced and is obsolete; the card runs again on rented hardware at one named commit. Each of these is an open item in Chapter 43, not a footnote.

**How it stays safe.** It watches before it writes. It records every original setting before it acts and reads back every write. It never writes past a knob's cover. It gives everything back the moment service crosses the wall or a sense goes blind. It stops writing a knob the instant anyone else touches it. It moves a slow knob only after a paired trial on the muscle itself shows no more than 2% cost. One OFF switch returns every setting to its original value; a watchdog with a lease does the same if the governor is killed outright; and a separate kill switch, for security only, turns the whole harness off in one human hand.

**How you know.** Every run produces receipts: native against omni on the same system, the same load and the same clock, each gauge with its 95% interval, the raw files archived beside the code and fingerprinted. Results are labelled by a rule written before the run, counted only when all three of our runs agree, and reported with every loss in the table.


*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

# Disclosures, Declarations and Disclaimers

This page is the one place every declaration about Omni-Compass is made. The README, the manual, the book, every guide and every report point here. Where any other page seems to say more than this page, this page governs.

## Rights

1. Omni-Compass, its engine, its mathematics, its software and its documentation are the property of The Omni-Compass LLC. All patents, copyrights and trademarks covering them have been filed in the United States by The Omni-Compass LLC. No filing number is stated here, and no grant, registration or approval is claimed.
2. The software is licensed for evaluation and simulation only (`LICENSE`). Running it in production, on any system beyond evaluation, or in any product or service, requires a signed, paid Omni-Compass Enterprise License.
3. Names of other companies and products (NVIDIA, Kubernetes, Red Hat OpenShift, Amazon EKS, Google GKE, Microsoft AKS, Karpenter, Lambda and others) are the property of their owners. They identify the systems Omni-Compass was tested with or connects to. No affiliation, endorsement or certification by any of them is stated or implied.
4. All rights reserved. The Omni-Compass LLC reserves the right to change the software, the documentation, the results and the terms at any time, without notice. Every copy, fork, export, report, archive or printout of any part of Omni-Compass carries the README's notice, `LICENSE`, `NOTICE` and this page unchanged. The Omni-Compass LLC, www.omni-compass.com

## What a result is, and what it is not

1. Every result carries its evidence class: **T** a theorem, **V** verified in code, **S** a model (simulation), **L** live software (real Kubernetes on GitHub's machines), **P** a physical meter (a real card's own power meter). A model is a statement about the model. A software benchmark is a statement about the software and machine it ran on. Only a meter speaks for hardware.
2. The models were written by the same people who wrote the law. They show how the mechanism behaves; they are not an independent test.
3. On Kubernetes in kind every machine stays powered, so energy there is a declared model, not a meter.
4. Every comparison is paired: the same seed, load and clock with and without Omni-Compass, in an order rotated by repetition. Intervals are 95% intervals over the paired repetitions. Ratios across seeds are summarised as geometric means. Labels are assigned by code from rules written and committed before the run (`docs/GPU_PREREGISTRATION.md`, `docs/REALMS_PREREGISTRATION.md`, `docs/K8S_COMPASS_PREREGISTRATION.md`), never by hand.
5. Losses and failed runs are kept beside the wins (`docs/EVIDENCE_LEDGER.md`, `results/live/`, git history). The first real-card run, for example, saved energy (+3.6% work per energy, proven) and made the slowest

answers 58.5% slower; it is published as it happened.

6. No result is a promise of a saving on any other system. No dollar figure in this repository is a measured result. Nothing here is a vendor certification, a safety certification, legal advice or investment advice.

## The rule Omni-Compass keeps on every muscle

Omni-Compass sits on top of a muscle that already works and moves its settings only to save energy, and only where it measures that the muscle loses at most **2%** in any measure: work done, response time (median, p95 or p99), time over the service line, or the machine's own cost of running Omni-Compass. Where no move passes that test, Omni-Compass leaves the muscle exactly as it runs alone. On a live system the verdict (`omnicompass/verdict.py`) measures it with paired trials on the muscle itself before every step. Every result reports each measure, and where one is worse at all, by however little, the result says so and by how much.

## The wiring declaration

**Omni-Compass acts only through the wires it is given.** It reads the meters it is pointed at and moves the settings it is allowed to move. If a reading is the wrong one, slow, or blind; if a lever is the wrong one, already owned by another controller, or given a range (cover) that is wrong for the machine; if a band or a service line is set for a different workload; then Omni-Compass will do exactly what its law says with the wrong information, and the result will not be the published one.

Therefore:

1. **It cannot be slapped on.** Every installation goes through the levels of the manual in order (chapter 9), starting read-only, and every writing level begins with the wire check (manual, section 8.4) and a watch arm that must equal native.
2. **If your paired receipts differ from the published benchmarks in direction** (service worse, or no saving where one was shown), **the first presumption is wiring, not the law.** Confirm the installation with the checks in the manual's section 8.5 (*Wired right or wired wrong*) before drawing any conclusion about Omni-Compass.
3. **This has happened to us, and is published.** On the first real card (NVIDIA A10, 2026-10-02) the governor read being busy as trouble, set its power lid under the card's own working draw and allowed the clock under the card's own working clock; the card served bursts more slowly and the slowest answers were 58.5% slower. The cause was found in the card's own samples and corrected (`docs/GPU_PREREGISTRATION.md`, amendments 6 and 7). That is what a wiring fault looks like, and how it is found.
4. Omni-Compass never writes outside a lever's cover, never fights another writer (it stops and leaves that lever alone), reads back every write, and returns every lever to the value it read before its first write when it stops or when the OFF switch is used. These protect the machine; they do not make a wrong wiring right.

## Safety and responsibility

1. Run watch mode first; run the wire check before any write; keep the master OFF switch (`python3 tools/omni_switch.py off`, the whole harness at once) in reach at every level, and use it at the first sign of anything wrong, a suspected breach included.
2. Omni-Compass is a supervisory governor. The machines' own controls (firmware, autoscalers, safety systems) stay in place and keep their own protections; Omni-Compass sets only values they already accept.
3. Do not connect Omni-Compass to safety-critical systems (vehicles, medical devices, grid protection, life safety) except as a modelled study, and never without the qualified review those systems require.


*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

4. The software is provided "as is", without warranty of any kind, as stated in `LICENSE`. The operator is responsible for its use on their systems.

## Changes to the work and to this page

1. The software, the law's settings, the benchmarks, the results, the documents, the licensing terms and this page may change at any time, without notice. The version in the repository's `main` branch at a given commit is the version that commit describes; `RELEASE_MANIFEST.json` and `python3 verify.py` identify it.
2. Statements about work in progress, planned features, expected results and future runs are expectations, not promises. Where a result has not yet been measured, the documents say so (`docs/STATE_OF_PLAY.md`, `docs/DOSSIER.md`, Chapter 28).
3. A signed Omni-Compass Enterprise License governs its own terms for its own term.

## Privacy and data

Omni-Compass contains no telemetry. It sends nothing to The Omni-Compass LLC or anyone else, and makes no network connection of its own except to the systems an operator points it at. Its logs and receipts stay on the operator's machines.

## Where everything is

| Question | Page |
|---|---|
| What has been measured, and how strongly | `docs/STATE_OF_PLAY.md`, `docs/DOSSIER.md`, `docs/EVIDENCE_LEDGER.md` |
| The rules written before each run | `docs/GPU_PREREGISTRATION.md`, `docs/REALMS_PREREGISTRATION.md`, `docs/K8S_COMPASS_PREREGISTRATION.md` |
| How to wire it, level by level, and how to confirm it is wired right | `docs/OMNI_COMPASS_MANUAL.md` (chapters 8 and 9), the book `docs/OMNI_COMPASS_MANUAL.pdf` |
| The license, in plain answers, and third-party components | `LICENSE`, `LICENSING_FAQ.md`, `ENTERPRISE_LICENSE.md`, `NOTICE`, `THIRD_PARTY_NOTICES.md`, `PATENTS.md`, `TRADEMARKS.md`, `SECURITY.md`, `.github/CLA.md` |
| The muscles, what each is for and how it is wired | `docs/MUSCLE_CATALOG.md` |
| Proof the code is the code that ran | `python3 verify.py`, `RELEASE_MANIFEST.json`, `results/SEAL.json` |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

PART ONE

# The Philosophy and the Theory

*Where Omni-Compass comes from: the closed circle, the four pieces, the compass, the basins and the reflex. The engineering in the rest of the book is this idea made exact.*

CHAPTER 1

# What Omni-Compass Is

A governor on a steam engine does not build the engine and does not turn the shaft. It watches the speed and moves the throttle so the speed stays in a band. Omni-Compass is that, for every machine you run.

It is a process on a host. It reads meters, steps a bounded mathematical law, and writes only the levers it has been given. It is not the chip, not the GPU driver, not Kubernetes, not the building controller. Those keep running exactly as they do today; Omni-Compass sets the values they already accept.

Three facts about that process fix what the rest of this manual can and cannot claim. First, it is **one process per stack**, a controller (`omni_controller/controller.py` on a cluster; the harness's own controller on a database or a cache) that wakes on a fixed period, reads the stack's own gauges through the stack's own interface, decides, writes through the same interface, and reads back what the device took; it has no agent inside the pods, no kernel module, no hook in the request path, so it can add latency to nothing it does not govern. Second, every lever it may touch is **named before it starts**, read once into a snapshot, and handed back to that snapshot when it stops, is killed, or loses a sense; a lever it was not given it cannot reach, because the plug for it does not exist. Third, it has a **watching mode** in which it reads and decides and writes nothing, and every rollout begins there (Chapter 31): the first thing an operator learns about Omni-Compass on their own stack is what it would have done, in a log, with the stack untouched.

What follows from those three is the shape of the evidence in Part VI. Because the native controller keeps running underneath, every benchmark is the native stack against the same native stack with Omni-Compass on top, and never Omni-Compass against anything else. Because the levers are the stack's own settings, every gain is one the operator could have reached by hand, if they had been able to read every gauge every few seconds and move every knob without ever being wrong; the claim is not a new mechanism but a steady hand on the mechanisms that exist. And because the process is small and separate, its own cost (CPU on the host, reads of the API) is measured in every run and counted against it where it shows.

## Always on top of a native controller

The first design decision, and the one every other follows from, is that Omni-Compass is **supervisory**. It never replaces the controller a system already has, never bypasses it, and never runs in its place. The Kubernetes Horizontal Pod Autoscaler keeps scaling pods on its own CPU target; Omni-Compass moves that target inside a range the operator allows. The cluster autoscaler keeps adding and deleting machines; Omni-Compass marks machines idle and lets the autoscaler do the deleting. The card's firmware keeps boosting the clock and enforcing its power limit; Omni-Compass sets the ceiling the boost may reach and the limit the firmware enforces. PgBouncer keeps handing connections to clients; Omni-Compass sets its pool size through the pooler's own console. Kafka's consumer group keeps rebalancing partitions on its own; Omni-Compass changes how many consumers are in the group, which is what an operator does. Redis keeps evicting by its own rule; Omni-Compass sets the ceiling that rule works under.

This matters to a referee for a precise reason: every benchmark in this manual has exactly two arms, **native** (the system with its own controller, as shipped and as the operator set it) and **omni** (the same system with Omni-Compass on top). There is no third arm in which Omni-Compass replaces the controller, and no comparison in which Omni-Compass is a rival to it. A gain is therefore always "what the governor added on top of what the native controller already did", and a loss is always "what the governor cost on top of it". Nothing in the record is an argument against any native controller; every native controller in this manual is a good controller that the governor sits on.

## Do no harm: the verdict

The second design decision is that Omni-Compass must never make anything worse. This is not a slogan printed on a report; it is a trigger inside the engine. Before Omni-Compass moves a slow knob (a machine given back, a card's clock ceiling, a robot axis's speed, a drone's cruise override), it runs a **paired trial on the muscle itself** (`omnicompass/verdict.py`): the muscle's own cost per piece of work is measured at native and at one step past the deepest step already allowed, under the same traffic, with at least thirty pieces of work on each side; the step is allowed only if the trial's median cost is no more than 2% above the reference and no more than 2% above the cost first measured at native. A refused step is not tried again for a set number of decisions. Where no step passes, the verdict reads **left native**, the knob is never moved, and the result for that muscle says "nothing for Omni to move". This is why, in the robot-arm benchmark, two of four arms were left entirely native and are reported as such, and why in the real Kubernetes tests machines were kept whenever one machine fewer made the requests slower in the trial. Gains are not capped; the allowance caps only what may be spent to get them.

The same principle, applied after the fact to every published result, is the page of wiring verdicts (`docs/WIRING_VERDICTS.md`, section 16.6b): every knob in every table is given one of three words from the table itself, **write** where a gauge is better in all three of our runs and none worse in all three of our runs, **watch** where nothing is better in all three of our runs (the knob stays native and Omni only reads it, one wire out and no wire in), and **operator's choice** where gains and losses are both the same in all three of our runs. Omni-Compass is wired out of every muscle; it is wired in only where it has earned it.

## The pedals, the modes and the two switches

Omni-Compass drives a system the way a self-driving car drives itself: it feels, gauges and adjusts, with nobody in the seat. The vocabulary is used the same way everywhere in this repository (`docs/MECHANISM_OF_ACTION.md`, "The pedals"):

- **Off (manual).** Omni-Compass is not driving; native runs alone. This is the native arm of every benchmark.
- **Watching.** Omni-Compass reads every gauge and logs the decision it would take, and writes nothing. Any difference between a watching arm and native is the cost of its being there (its CPU, its reads) plus noise, never a decision; a watching arm that writes anything fails its run. Watching is the first level of every rollout (Chapter 31).
- **Autopilot.** The pedals: **idle** (no foot on the gas or the brake, calm traffic, the engine at its floor of two machines in service, ready for the next burst), **gas** (traffic climbs and capacity is added at once, never held back), **brake** (traffic falls and capacity is eased off a step at a time, never below idle), **reset** (the brake held to the floor: every setting the governor ever wrote is handed back to where native had it, read back, and the record removed; every benchmark arm ends with a reset and checks it).
- **Cruise.** For work that comes as a pile: when pods have waited for a place for two decisions in a row, every machine goes into service and stays there, without second-guessing, until the line has been empty for two

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

decisions. In cruise the governor also stops its five-second floor reads, because every machine is already in service and the reads would only cost CPU on the machines the queue is using.

- **The emergency brake.** When the work is done and demand is at zero (the service at its autoscaler's own floor, wanting no more, at half its target or less), straight to idle in one move, with every safety check still holding.
- **The kill switch.** Separate from all of the above, and for **security only**: one switch in a human hand (`python3 tools/omni_switch.py off`, `omnicompass/master.py`) turns Omni-Compass's governing off across the whole machine at once, for anything rogue, or for anyone trying to drive a system through Omni-Compass's brain. Every governor hands back and exits; none starts while the switch is off. The reset is the end of a run; the kill switch is the end of trust.

The floor of two machines is an operator's number; the brake never takes a system below it, however hard it is pressed. There is no ceiling: every machine the operator has is usable, and the protection is the band inside each machine.

## Nothing is hardwired

Omni-Compass carries no table of per-stack special cases. One law (Chapter 17) is written once and runs on every muscle through one plug (Chapter 28). The gains differ from muscle to muscle only through the muscle's declared physics: its response time, its decision period and its cover. Where a rule was found not to work, it was removed, not switched off and left in the code: the engine files are frozen and fingerprinted, and `verify.py` fails if any of them changes. A reader who finds a behaviour in a result can find the exact lines that produced it, and will find no second path.

The rule has a practical consequence for anyone comparing Omni-Compass with a tuned controller. A tuned controller carries, somewhere, a number that someone chose for one stack: a cool-down of five minutes, a target of 70%, a step of two replicas. Each of those numbers is a bet that the stack will behave tomorrow as it did when the number was chosen, and each is one more place where a reader has to ask whether the result was the law or the tuning. Omni-Compass has a handful of such numbers, the centre of the band, the 5% cushions and the 2% do-no-harm line, and they are the same on every muscle, written once in the law and declared in every preregistration. Everything else the law needs (how fast the muscle answers, how often it may be moved, how far it may go) is read from the muscle's own declaration in the catalog (`omnicompass/muscles.py`) or from the native controller's own settings at the start of the run, never typed in by hand for a benchmark. When a benchmark needs a line, a cover or a chunk size (the MySQL pool moves in the server's own 128 MB chunks, the Redis ceiling in notches), the number comes from the stack being governed and is written in the preregistration before the first counted run, where a referee can see that it was not chosen after seeing the result.

The second consequence is how the repository changes. A guard or a gain that turned out not to help is not left in the code under a flag, because a flag is a second path that a later reader has to reason about and a later test has to cover. It is removed, the removal is written in the changelog, and the fingerprint moves to the next engine version (Chapter 15), so that no result taken on the old path is ever read against the new one. The three fingerprints that exist, v1, v2 and v3, are the whole history of such changes.

## The word

The law is a **compass**: a band with a cushion at each wall, pushed to its middle by two antagonist forces, smooth, never hammering. The repository uses that word and no other for it. Earlier dated records that used another word are kept word for word, as records are; the current law is `omnicompass/compass_law.py`, `CompassLaw`.


*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

The word was chosen for what the law does, not for decoration. A compass has a needle, a centre it is drawn back to, and a card marked at the edges; it does not slam to a wall and stay there, and it does not chatter when the hand holding it is steady. The law has the same three parts. The **band** is the range the gauge is allowed to travel, from 0 to 1 once the reading is placed on it, with the operator's line at the top. The **cushions** are the last 5% at each wall, 10% of the band in all: a reading inside a cushion is already too close to a wall, and the law treats it as such before the line is touched rather than after. The **centre**, 0.5 by default, is where the needle comes to rest, and two forces bring it there: a **pull** proportional to the distance from the centre, gentle near it and harder up the walls, like a spring; and a **push** proportional to the speed at which the reading is moving, meeting whatever is shoving it (a load rising, heat building) with an equal and opposite force and doubling as the friction that stops the needle sloshing past the centre (the gain is chosen for critical damping: it glides to the centre and stops, with no overshoot and no ringing). Past the upper wall the pull goes to its full force at once, and the down side may not act until the reading is back inside the band: the law fails up, never down. The force itself is `A × tanh(raw / A)`: near the centre it behaves as a spring, in proportion to the distance, and far from the centre it bends over and flattens into its maximum `A`, the law's authority, so that no reading, however far out, can make the law ask for more than one full step. That flattening is what "never hammering" means in code: there is no reading that produces a jump, and two readings a decision apart produce two forces a small amount apart. Section 6 gives the rules in full and Appendix C the equations; the reader who only needs the picture can keep the needle, the centre and the two cushions in mind and will not be misled by anything later.

## What it sets, muscle by muscle

Every muscle keeps its own control, and Omni-Compass sets what that control already accepts:

| Muscle | Its own control (kept) | What Omni-Compass sets |
|---|---|---|
| Kubernetes service | the HPA | the HPA's CPU target; replica floors |
| Kubernetes node pool | Cluster Autoscaler / Karpenter / MachineSet | how many machines stay in service (park and wake) |
| NVIDIA GPU | the card's firmware (boost, power and thermal limits) | the clock ceiling (up wire) and the power limit (down wire) |
| CPU | the kernel's frequency governor | the frequency ceiling |
| Data hall, building | the chiller and air handler loops | the supply-air setpoint; units in service |
| Battery site | the inverter | the reserve level; the peak ceiling |
| Robot joint, vehicle axis | the servo loop | speed and effort limits |
| Process loop, feeder | the PI controller | the setpoint inside its band |
| Database behind a pooler | PgBouncer's pool, the DBA's fixed setting | the pool size through the pooler's own console |
| Database's storage engine | MongoDB's WiredTiger cache, the operator's fixed setting | the cache size through the server's own console |
| Message broker | the consumer group at the operator's count | how many consumers are in the group |
| Cache | the operator's memory ceiling and eviction rule | the memory ceiling through the cache's own console |
| Substation | the tap changer's own controller | the tap position, one whole tap per move |
| Drone | the shipped autopilot | the cruise override inside the autopilot's limits |

When Omni-Compass stops, every one of those values goes back to what it was before Omni-Compass acted: not to the last value it wrote, but to the value it read once, before its first write.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

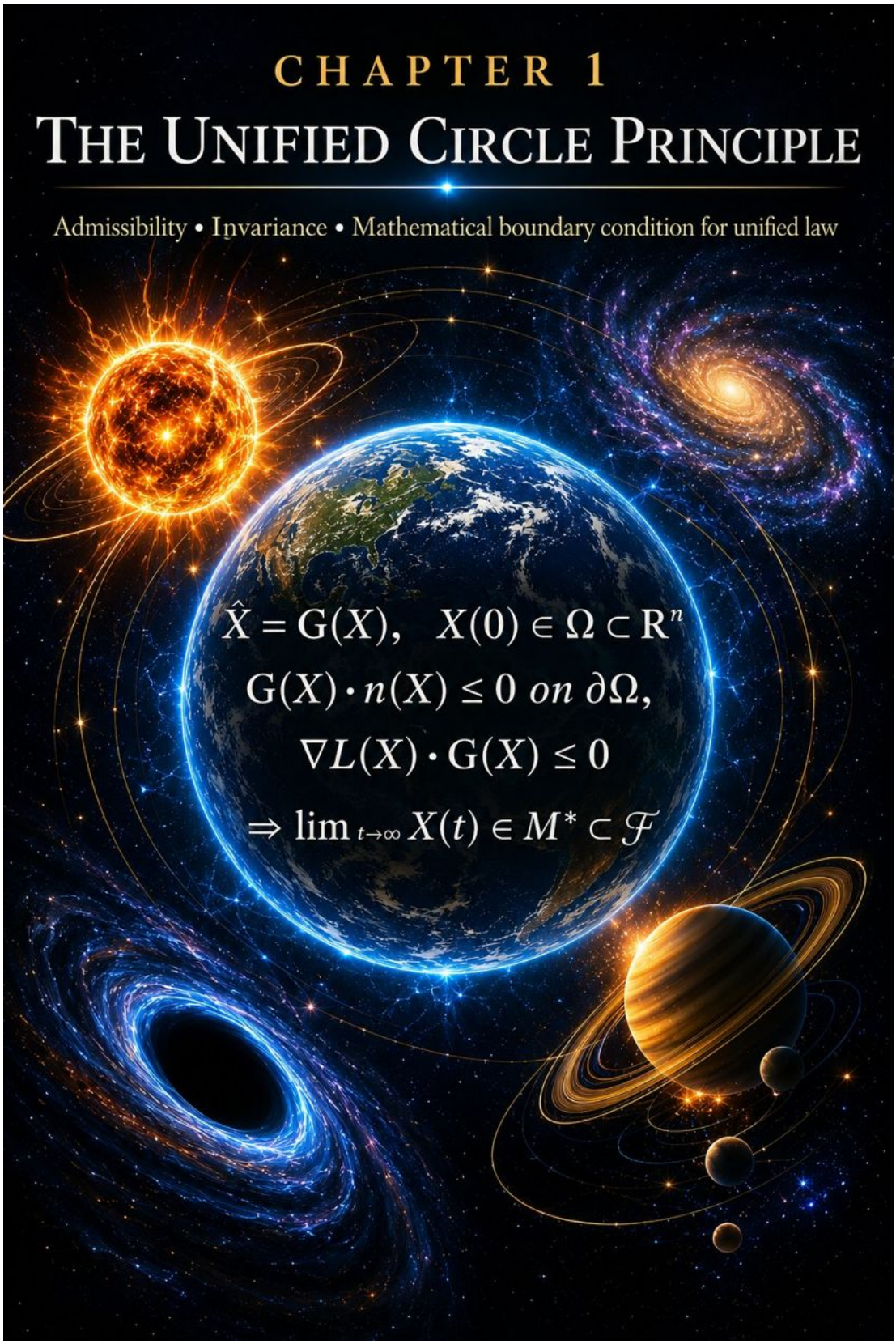



*Plate 1. The Unified Circle Principle*



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 2

# The Unified Circle Principle

Omni-Compass begins with one idea: a system that is closed, bounded and pulled toward a center cannot run away, and everything it does is a return. I call this the closed circle. This chapter states the principle as I hold it, in its mathematical form, and then says plainly which parts of it the software uses and which parts belong to the wider theory the software grew out of.

## The statement

```
dX/dt = G(X),        X(0) in Omega, a subset of R^n
G(X) . n(X) &lt;= 0     on the boundary of Omega
grad L(X) . G(X) &lt;= 0
=&gt;  lim (t -&gt; infinity) X(t) in M*, a subset of F
```

Read line by line:

1. **dX/dt = G(X).** The state X moves according to one law G. There is no outside hand: every change comes from the law and the state it is in.
2. **X(0) in Omega.** The state starts inside the admissible region Omega: the set of states that are physically and operationally allowed.
3. **G(X) . n(X) <= 0 on the boundary.** At the wall of Omega, the flow never points outward (n is the outward normal). Nothing that starts inside can cross out. In control theory this is the Nagumo condition: Omega is forward invariant.
4. **grad L(X) . G(X) <= 0.** There is a function L, the energy of the compass, that never increases along the motion. This is a Lyapunov condition.
5. **=> X(t) settles in M*.** Together, an invariant container and a non-increasing energy force every path into the set M* where the energy stops falling: the bottom of the compass.

The power of the statement is that it does not depend on the particular disturbance. Every input up to the size the container is built for has the same answer before it is asked: the state stays inside and returns. That is what I mean when I say that a closed circle holds the answer to every question that can be put to it.

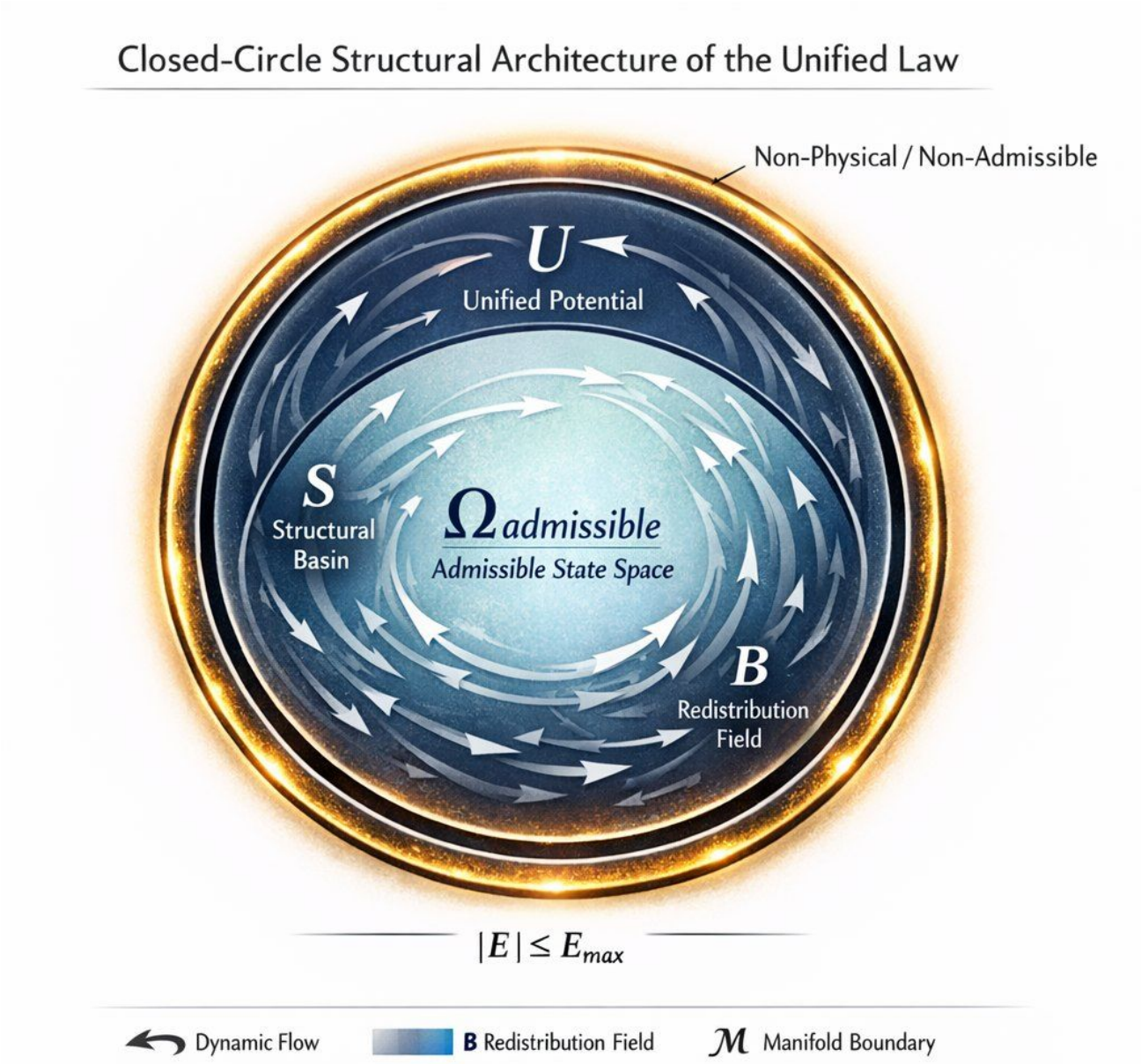



*Plate 2. Closed-circle structural architecture of the unified law*



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## Closure, admissibility, boundedness

The theory sets five requirements on any law that claims to close the circle:

| Requirement | Meaning | In the engine |
|---|---|---|
| Finite deviation | the distance from the admissible state has a ceiling, abs(E) <= E_max | the deviation state E drains faster than it can fill |
| Finite curvature | the landscape the state moves on has no infinite slopes or pits | the drive is bounded by tanh; the authority is clipped |
| Global conservation | what flows in is matched by what flows out | the drain and the bath's friction match the drive |
| Recurrence | a disturbed state returns arbitrarily close to where it was | convergence to the target pole at rate K_P |
| One law | every subsystem obeys the same law, no special cases | one engine, one compass, every muscle |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*




*Plate 3. The closed circle tested*



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## The closed circle tested

The canonical evolution law has four parts: deviation is damped nonlinearly and coupled to alignment and to the basin; alignment relaxes toward unity and responds to deviation; the basin's curvature evolves with internal structure; damping absorbs excess growth and redistributes energy. Bounded random noise is permitted; divergence is not. A cycle is identified when the system returns arbitrarily close to a prior state within a bounded tolerance under noise. The state advances stepwise under a stable numerical integrator with boundedness checks. Under parameter sweeps the test is always the same three checks: no divergence, bounded, recurring.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

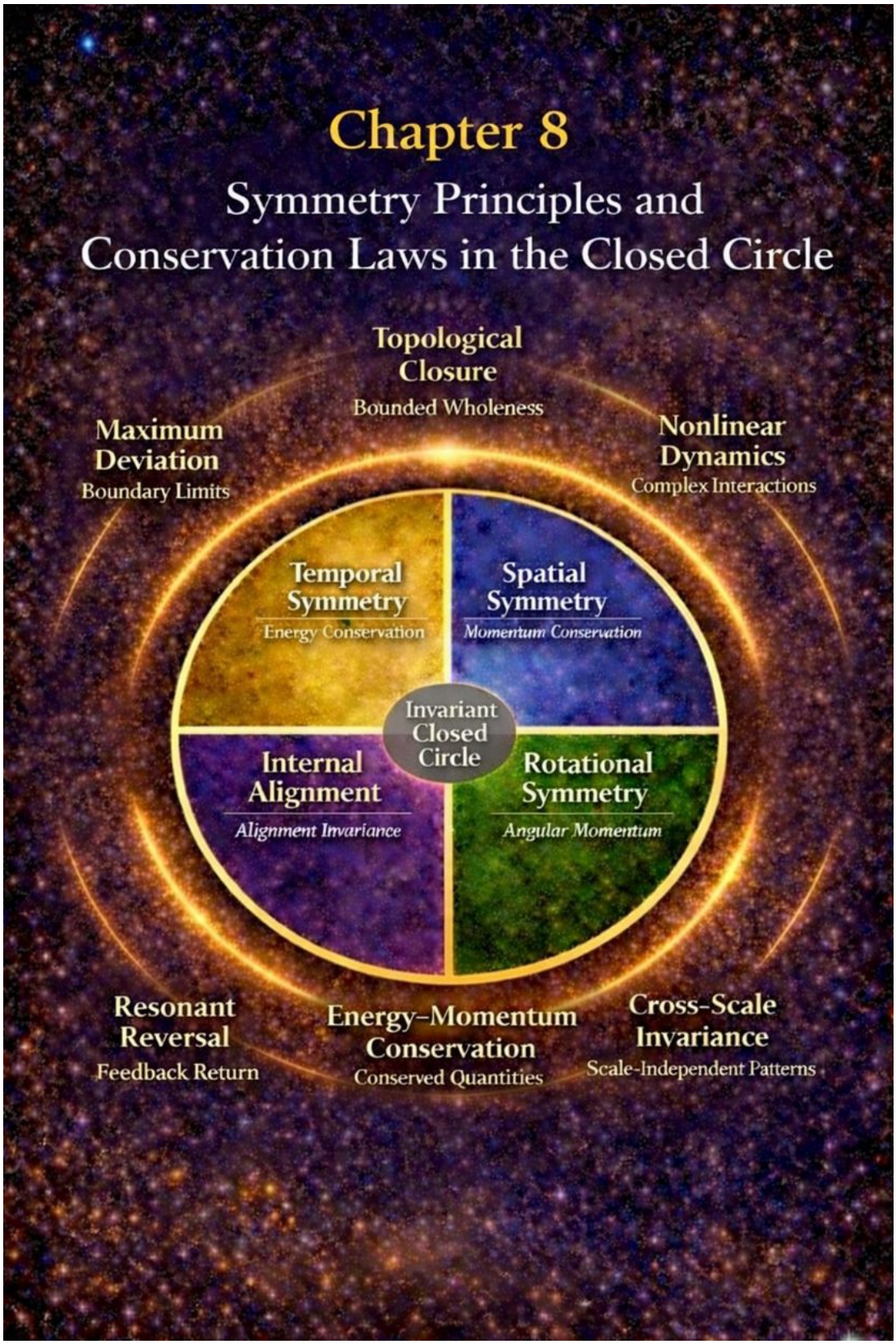



*Plate 4. Symmetry principles and conservation laws in the closed circle*



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## Symmetry and conservation in the closed circle

Every conservation law is the shadow of a symmetry (Noether). The closed circle carries four:

| Symmetry | Conserved | In Omni-Compass |
|---|---|---|
| Temporal (the law does not change with time) | energy | the same law governs every decision; the receipt compares like with like |
| Spatial (the law does not depend on where) | momentum | the same law governs every muscle in every realm |
| Rotational | angular momentum | the compass is symmetric about its center; the engine is symmetric between its poles |
| Internal alignment | alignment invariance | the alignment state is pulled to its pole whichever pole it is |

Around them sit the boundary conditions of the circle: maximum deviation (the boundary limit), topological closure (bounded wholeness), nonlinear dynamics (complex interactions), resonant reversal (feedback return), energy-momentum conservation, and cross-scale invariance (scale-independent patterns).

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*




*Plate 5. Scale invariance and structural universality*



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## Scale invariance

The theory claims that one bounded nonlinear action has the same form at every scale: cosmological (curvature geometry, compression and expansion, thermodynamic cycling), galactic (rotation curves, mass-energy distribution), stellar (gravitational compression, fusion equilibrium, radiative transport), atomic (electron displacement, orbital coherence, radiative damping) and quantum (localized deviation wells, bounded oscillation, finite amplitude). The universal criteria for such a law are that it operates at every scale and preserves bounded deviation, finite curvature and recurrence. The state vector carries the same meaning at every scale: E the deviation magnitude, U the coherence or flow alignment, S the structural curvature or basin geometry, B the redistribution or damping.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*




*Plate 6. The lineage of unification*



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## The lineage of unification

Each step in the history of physics unified two things that had been separate: geometry formalized structure; motion was unified across earth and sky; astronomy revealed recurrence; fields replaced action at a distance; curvature unified gravity, though singularities remained; symmetry came to govern the quantum realm. The closed circle is offered as the next step: unity without divergence, a single bounded law with no singularity and no runaway.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

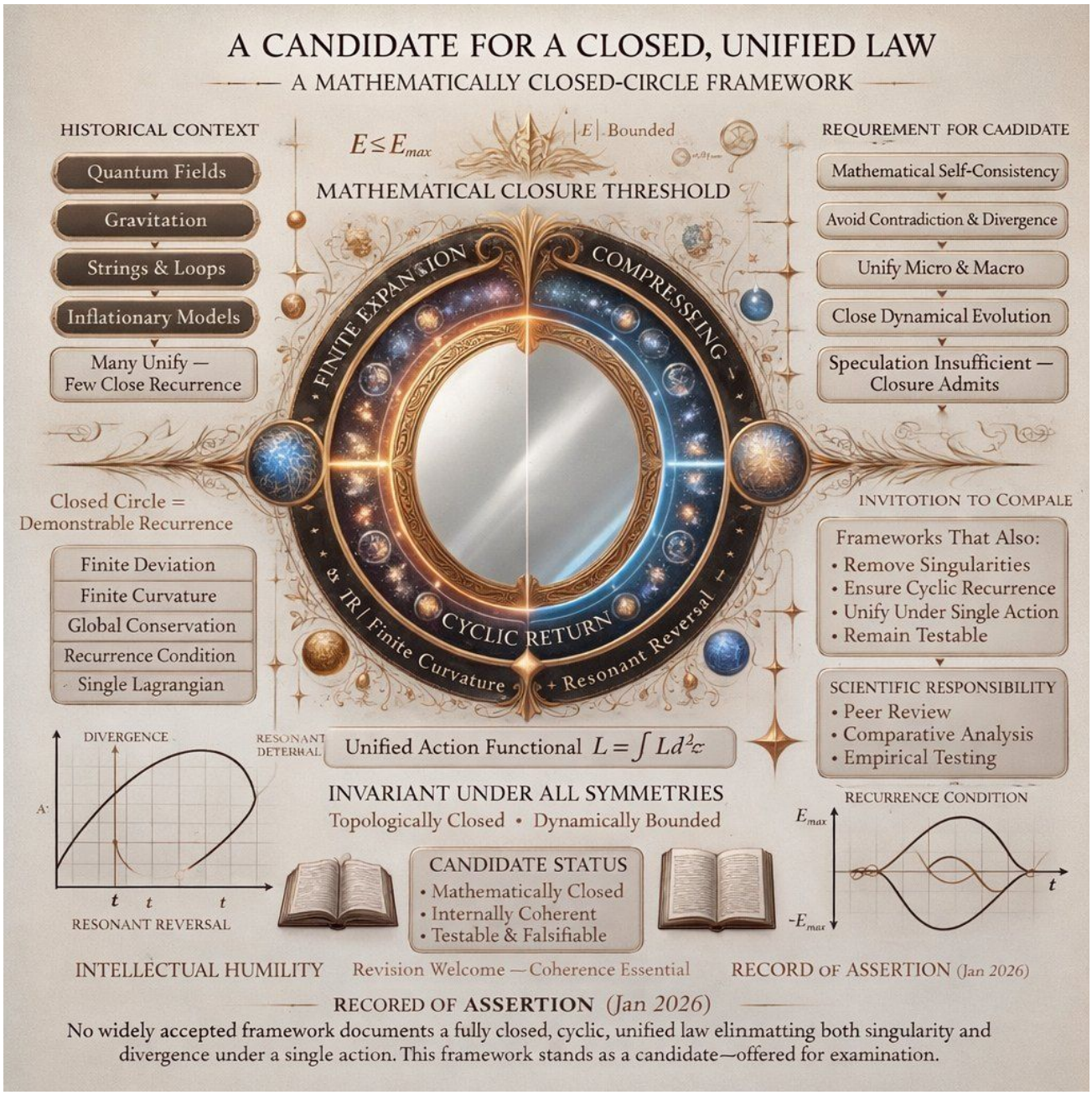



*Plate 7. A candidate for a closed, unified law*



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## What is tested and what is theory

The software uses the engineering core of this principle, and every part of that core is checked:

- The six-state engine, its bounded control command held through the integration step, and the convergence of its alignment state to its pole are proved (`docs/TRACKING_THEOREM.md`) and tested against a C++ twin.
- The invariance and boundedness of each engine state were checked state by state (Chapter "Closing the Circle").
- The compass law pulls every muscle's service reading to the middle of its band under a bounded, smooth force, and its behavior is measured in simulation and on real software.

The wider claims - one law across every physical scale, the cosmological and quantum readings, the candidate unified action - are the theory from which Omni-Compass was built. They are stated here so that the engineer understands where the design comes from. They are not claims that the software has measured, and no result in this book depends on them.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

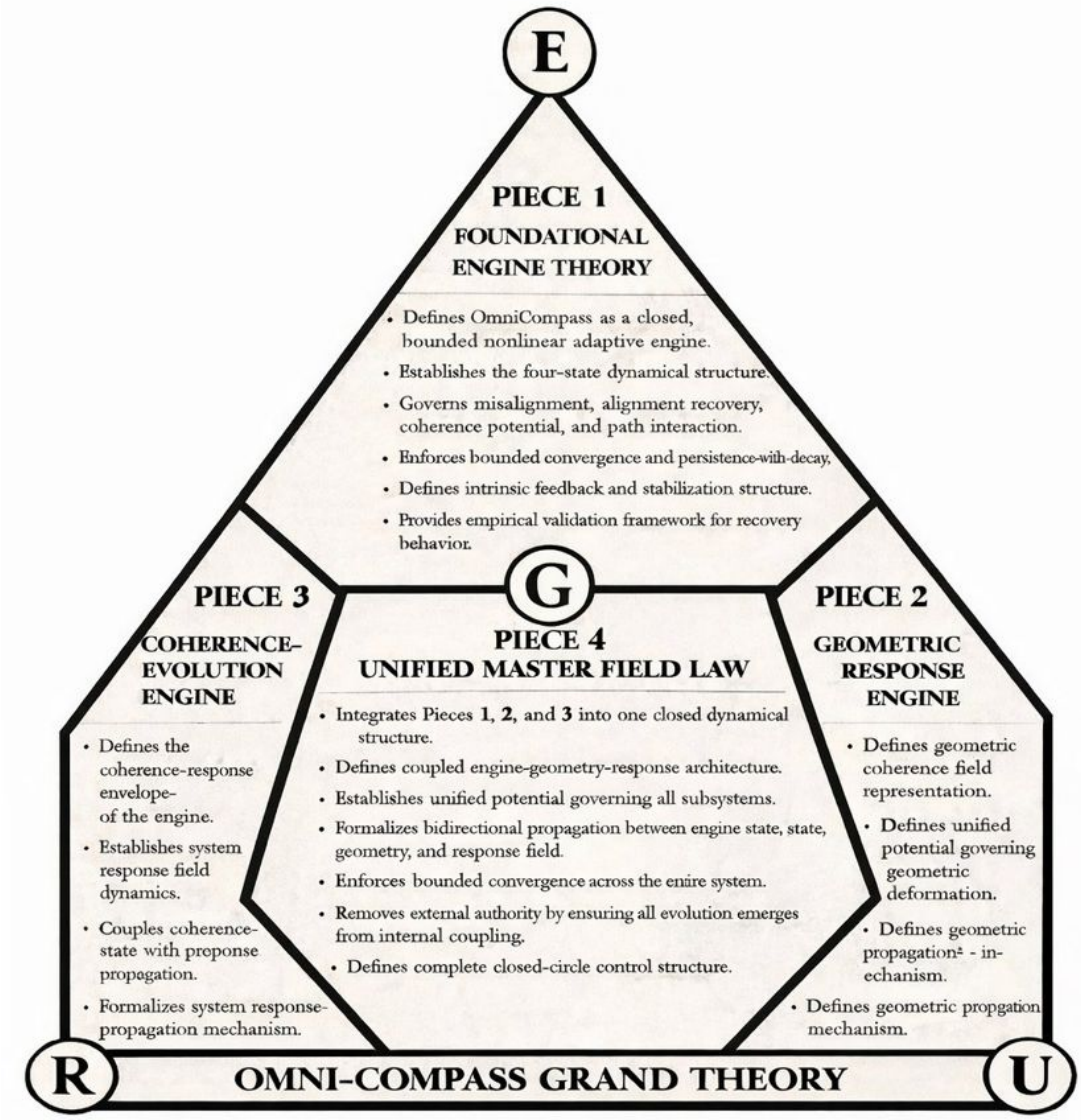



*Plate 8. The Omni-Compass Grand Theory: the four pieces*



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 3

# The Four-Piece Engine

The engine is built in four pieces, stacked like the stages of a press, each feeding the next:

| Piece | Name | What it establishes |
|---|---|---|
| 1 | Foundational Engine | the base laws, the states and the boundaries: a closed, bounded, nonlinear adaptive engine with a four-state dynamical structure (misalignment, alignment recovery, coherence potential, path interaction), bounded convergence and persistence with decay, intrinsic feedback and stabilization, and an empirical validation framework for recovery |
| 2 | Geometric Response | geometric alignment and response dynamics: a geometric coherence field, a unified potential governing geometric deformation, and the propagation mechanism of that response |
| 3 | Coherence Evolution | coherence through response and adaptation: the coherence-response envelope, the system response field, and the coupling of coherence state with response propagation |
| 4 | Unified Master Field Law | one coherent field structure: it integrates pieces 1-3 into one closed dynamical structure with a unified potential governing all subsystems, bidirectional propagation between engine state, geometry and response, bounded convergence across the whole system, and no external authority: all evolution emerges from internal coupling |

The four pieces compress every domain through one center core. The compression process has four stages: the domains unordered; compression in progress; coherence achieved; return to equilibrium. The result has four properties: coherent, bounded, admissible, stable - one law, one return, one cycle.

**The 24 domains.** The theory gathers 24 domains into four quadrants around that core:

| Quadrant | Domains |
|---|---|
| 1. Foundational structures | Newtonian gravity; Maxwell electromagnetism; Einstein relativity; Schrodinger quantum mechanics; Shannon information theory; Dirac quantum field theory |
| 2. Dynamics and symmetries | Noether symmetry laws; Hamiltonian dynamics; Lagrangian mechanics; thermodynamics; topology; differential geometry |
| 3. Coherence evolution | gauge field theory; fluid dynamics; field theory; discrete mathematics; solid-state physics; plasma physics |
| 4. Complex systems and integration | astrophysics; consciousness and information integration; biological systems and self-organization; universal mathematical structure; systems theory and complex adaptive systems; the unified Omni-Compass domain (the unified admissible state) |

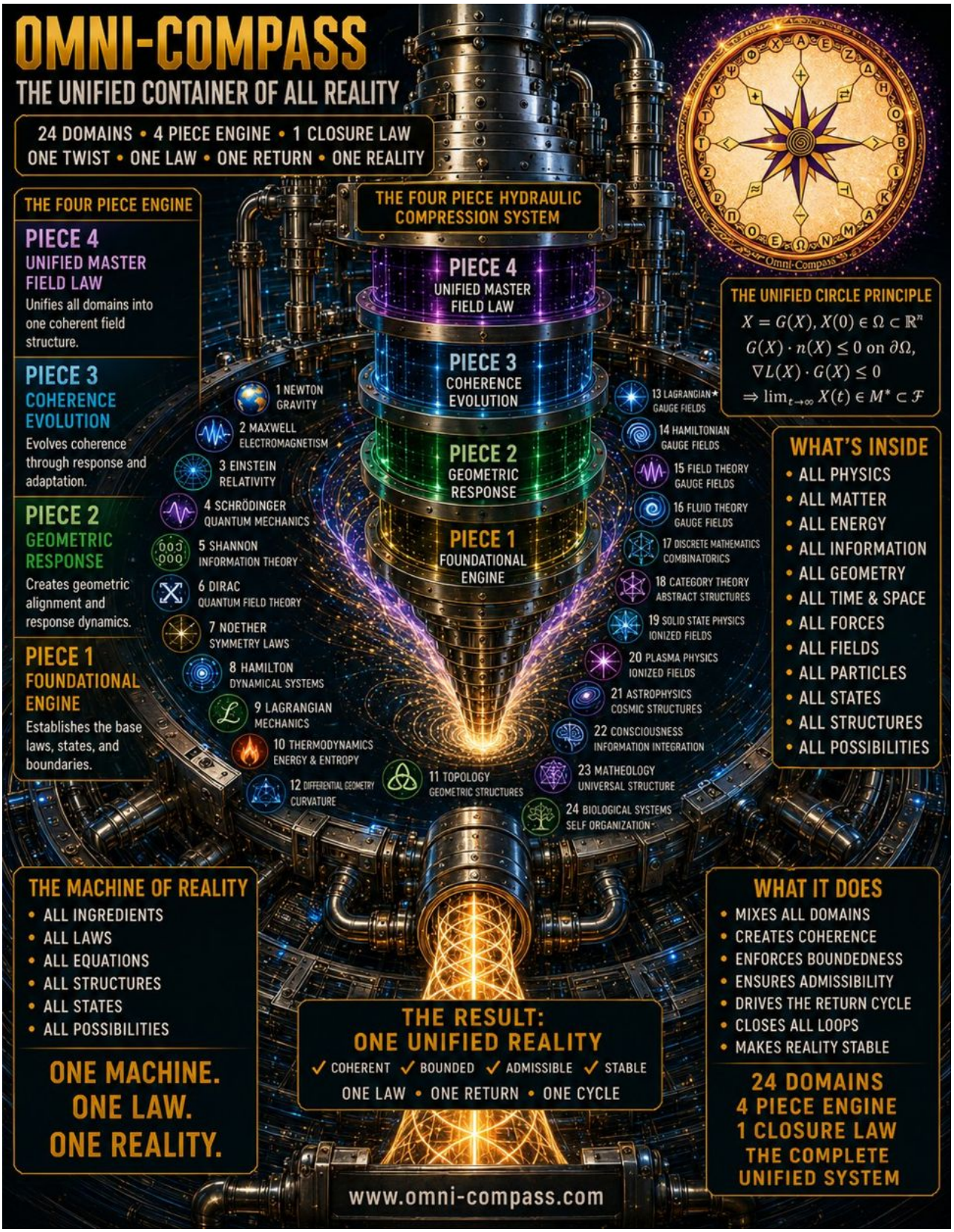



*Plate 9. The unified container of all reality: 24 domains, four-piece engine, one closure law*



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

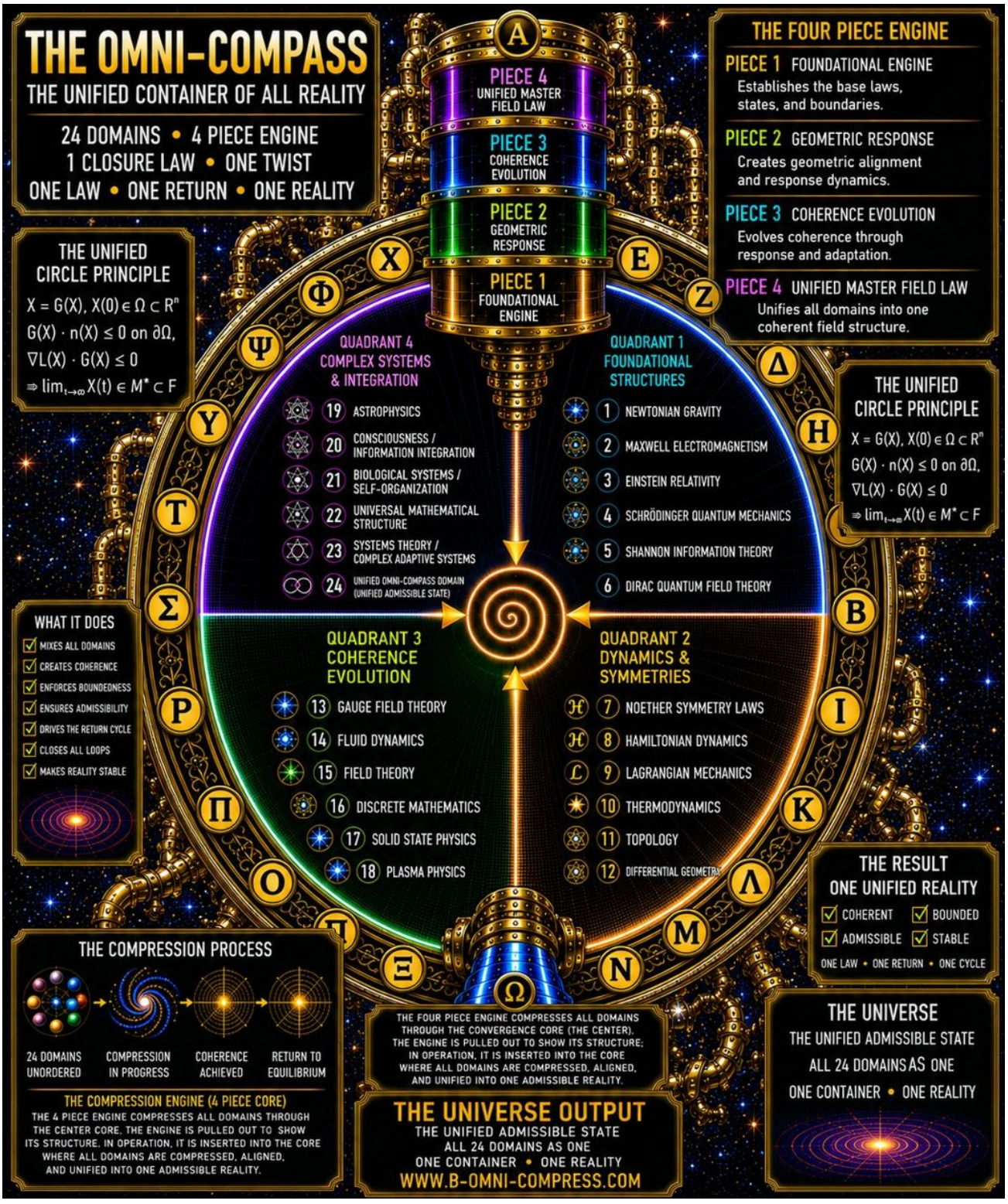



*Plate 10. The four quadrants and the compression process*

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

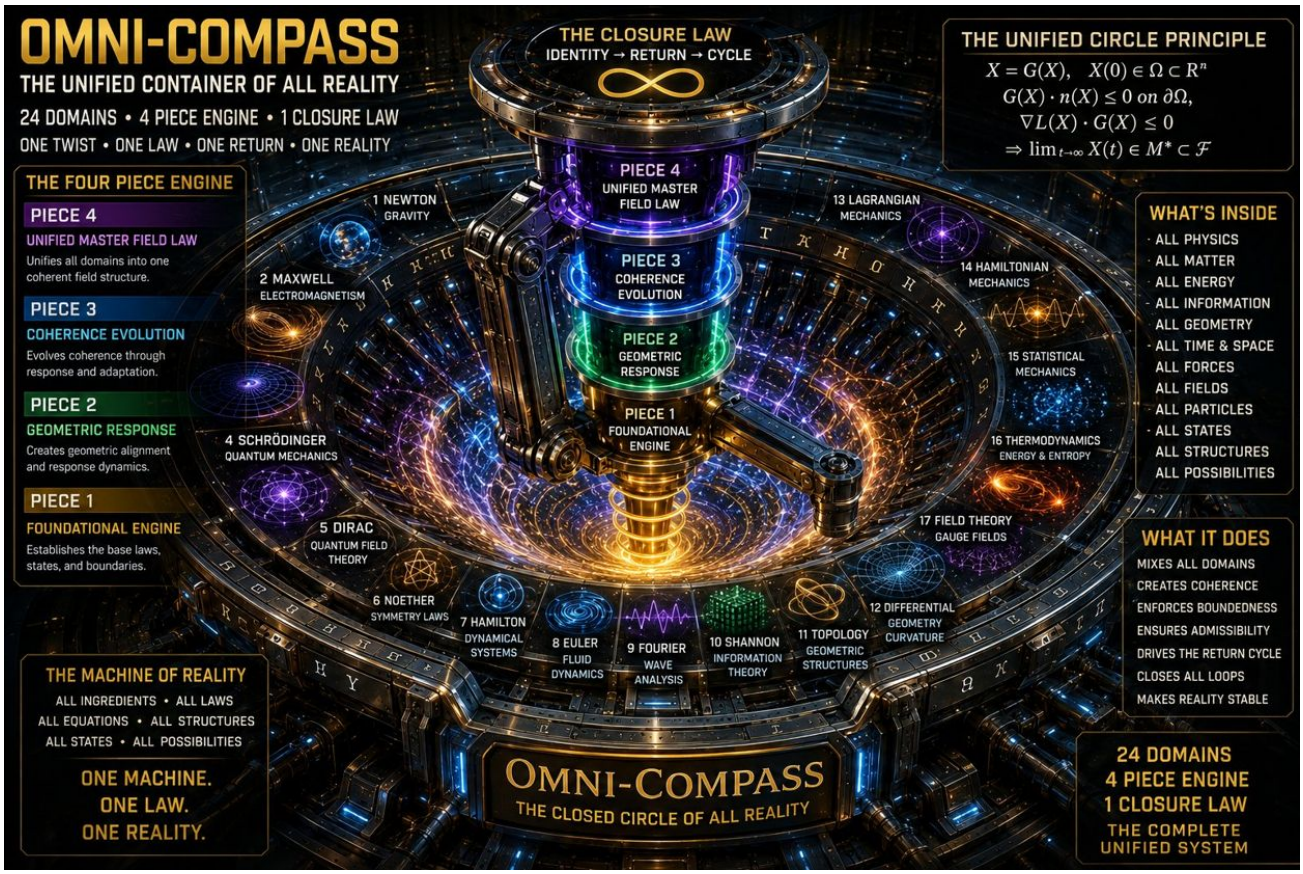



*Plate 11. The closure law and the closed-circle engine*



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

**In the software,** the four pieces appear as the engine's state equations (piece 1: deviation, alignment, memory and basin), the basin potential and its flow (piece 2), the bath that absorbs and redistributes (piece 3), and the one governor that couples every muscle to one law with no outside authority (piece 4).

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*




*Plate 12. The Omni-Compass rose*



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

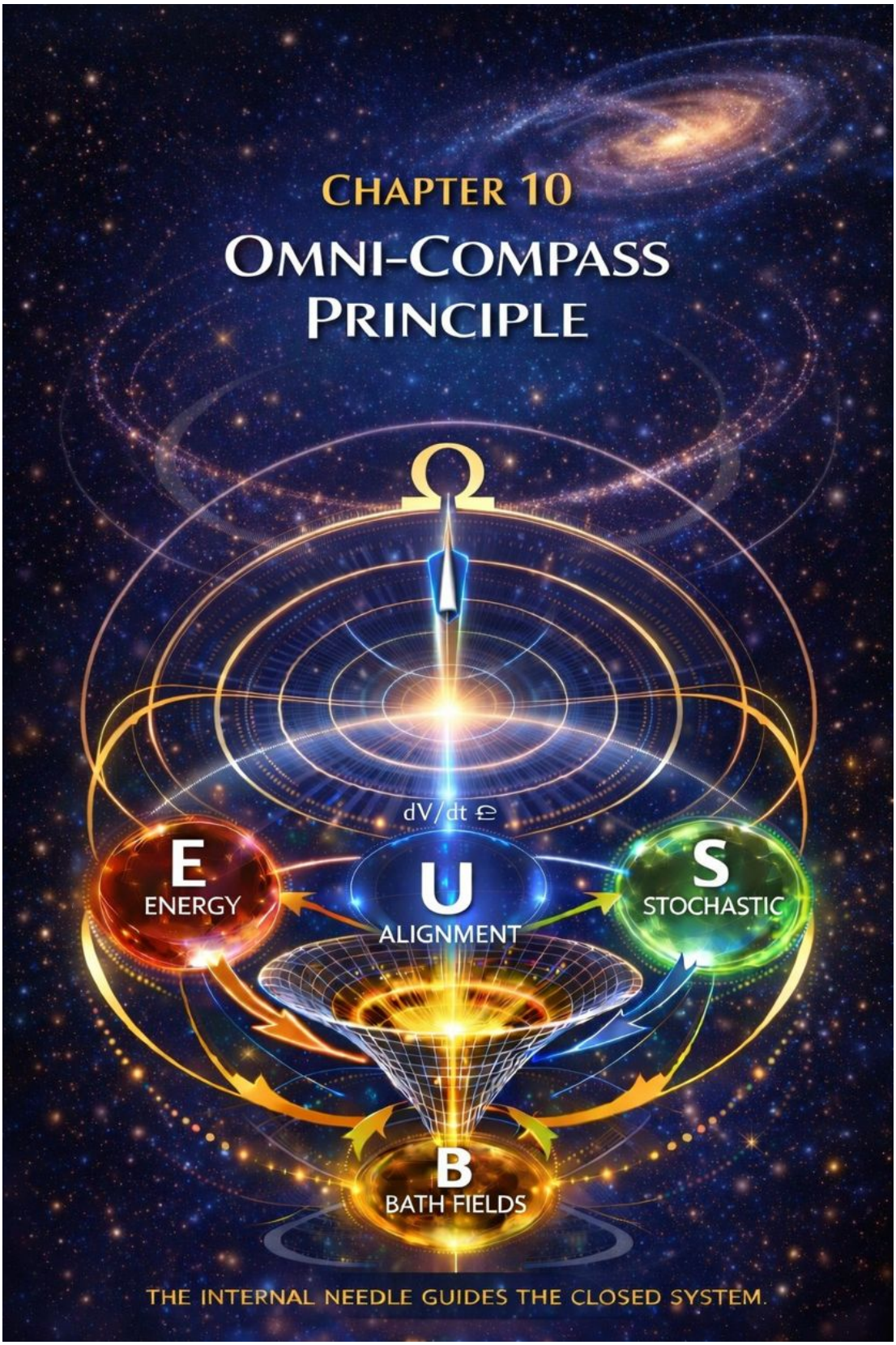



*Plate 13. The Omni-Compass Principle*

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 4

# The Compass

The Omni-Compass rose is the whole mechanism drawn as one picture.

**The ring.** Twenty-four letters, the complete Greek alphabet from Alpha to Omega: the complete set, nothing missing, one letter for each domain. On a ring the end runs back into the beginning: after the last letter the ring returns to Alpha. That is the closed circle: identity, return, cycle.

**The four points.** Four letters are lifted out of alphabetical order and set on the four points of the rose:

| Point | Letter | Mark | Meaning in the engine |
|---|---|---|---|
| North | Alpha, the first | + | the positive pole of the alignment's double well, U = +1 |
| South | Omega, the last | - | the negative pole, U = -1 |
| East | Beta | > | the drive coming in (the beta terms that fill the deviation) |
| West | Gamma | < | the coupling that carries energy out to the bath, where friction absorbs it |

The north-south line is polarity; the east-west line is flow in against flow out. A system stays inside its circle exactly when what flows out can always match what flows in.

**The inner marks.**

| Mark | Meaning | In the mechanism |
|---|---|---|
| + and - | the two polarities | the two poles of the double well |
| > and < | push out, pull in | the antagonist pair: the up force and the down force |
| the double arrow | exchange both ways | the two-way wire: read in, write out |
| the up-tack | the ceiling | the cover: nothing goes past the top |
| approximately-equal | the wave, the tolerance | the band and its cushion: hold near the center |
| the star | the guiding star | the target the needle points to |

**The center.** The spiral at the middle is the attractor: every path winds inward to one point.

**Opposite letters.** Across the center, zeta faces xi: both are the engineer's symbol for the damping ratio, and a damping ratio of one is the critical glide to the center with no overshoot. Lambda faces psi: the eigenvalue and the wave function of the eigenvalue equation.

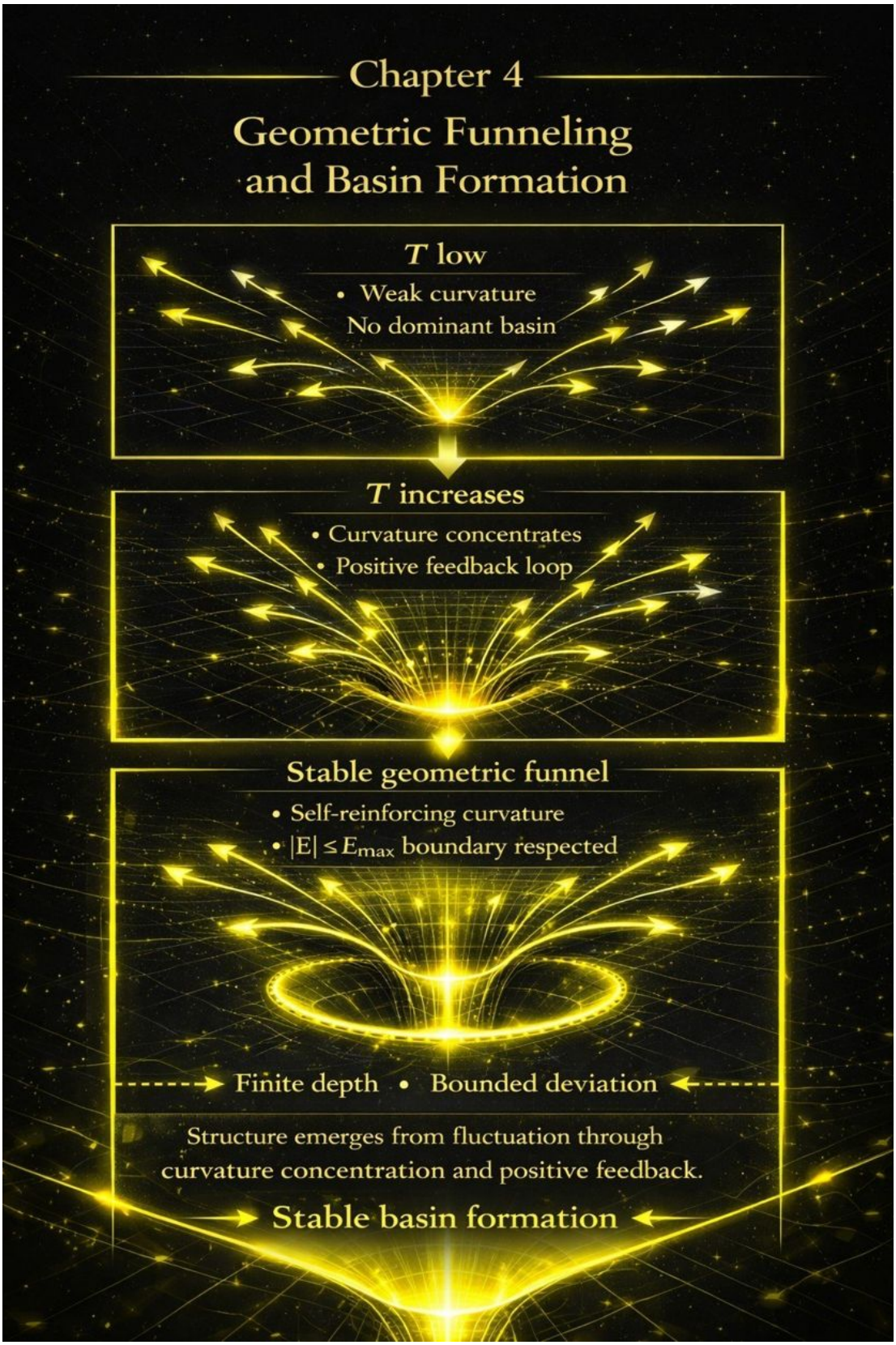



*Plate 14. Geometric funneling and basin formation*

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

CHAPTER 5

# Basins, Polarity and the Dual-Basin Engine

## Geometric funneling and basin formation

A basin forms in three stages. While the driving tension T is low, curvature is weak and there is no dominant basin: flows spread outward. As T increases, curvature concentrates and a positive feedback loop begins. Then a stable geometric funnel forms: self-reinforcing curvature, finite depth, bounded deviation, with the boundary abs(E) <= E_max respected. Structure emerges from fluctuation through curvature concentration and feedback.

In the engine, the basin state S rolls down its potential Phi(S) to a stable bottom; in the compass, every muscle's service reading is pulled into the bottom of its own funnel.

## Polarity

Two like charges repel without anyone pushing them: the force comes from the shape of the field. A wall built of like polarity at the edge of a band pushes the state back harder the closer it comes (as one over the distance squared), and two such walls cancel at the center, where the state comes to rest. This is the boundary condition of the closed circle written as a force: the field itself points inward at the wall.

The repulsion does not create energy: the field stores what it took to bring the charges together. What is gained in the computer is that the force is computed rather than spent: the brain computes it at no physical cost, and the machine's energy is saved because its knob no longer fights itself.

A symmetric pair of repelling walls was tested on the modelled GPU card. It raised work per energy but also raised the time spent past the service line, because the lower wall pushed capacity away when the card was calm. It was removed. The upper wall alone remains a candidate.

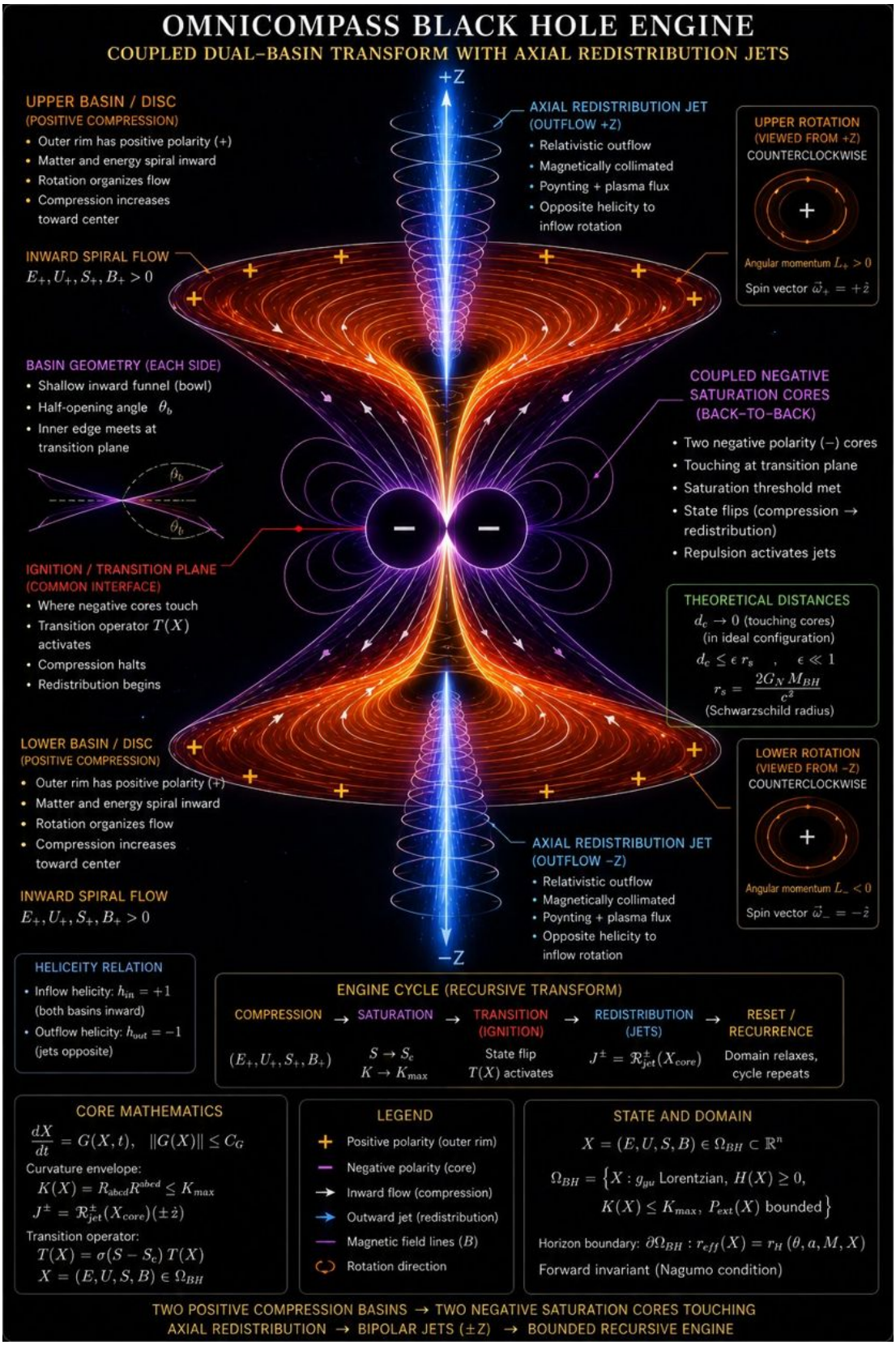



*Plate 15. The Omni-Compass dual-basin engine*

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

## The dual-basin engine

The dual-basin engine is a recursive transform with five stages:

1. **Compression.** Two outer basins of positive polarity draw matter and energy inward in a spiral; rotation organizes the flow; compression increases toward the center.
2. **Saturation.** At the center, two cores of negative polarity meet back to back; the basin structure S reaches its critical value and the curvature reaches its maximum.
3. **Transition (ignition).** Where the cores touch, the transition operator T(X) = sigma(S - S_c) T(X) activates: compression halts and redistribution begins.
4. **Redistribution.** Two axial jets carry the excess out along the axis, with opposite helicity to the inflow.
5. **Reset and recurrence.** The domain relaxes and the cycle repeats.

The core mathematics is a bounded flow: dX/dt = G(X, t) with abs(G(X)) <= C_G, a curvature envelope K(X) <= K_max, and a redistribution term that activates only at saturation. The state X = (E, U, S, B) stays in its admissible domain, whose horizon boundary is forward invariant (the Nagumo condition again).

**For an engineer,** the transform reads: pull toward the center (compression); a wall where the muscle saturates; and, when one muscle saturates, move the excess to its siblings rather than hammering it (redistribution). The first two are built into the compass. Redistribution across muscles is designed and not yet built; it is on the roadmap.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 6

# The Reflex Rule: One Body, One Brain

A body does not decide in advance how much force a muscle will need. It sends a signal, the muscle moves, the nerves report what the movement did, and only then does the brain choose the next signal. Nothing in that loop is a forecast. The decision is made on the reaction, after the reaction, and from the reaction. The founder's order of 10 October 2026 puts the whole of it in one sentence: what the brain sends to a muscle has to coincide with what the muscle sends to itself and where it is at. This chapter is the theory of that sentence, and the reason it is the whole mechanism rather than one more rule beside the others.

## The reflex, not the forecast

The compass of the earlier chapters is a force: where a reading sits in its band, how hard the two opposing forces push it back toward the middle, and how smoothly. A force alone does not say when it may become a new setting. A governor that turns every urge straight into a move is a governor that guesses, and a guess made on a machine that is already working is a guess paid for by that machine. The reflex rule closes the gap: a new setting is earned by a trial on the muscle itself, and the trial is read from the muscle's own reaction. The urge proposes; the reaction disposes.

## A trial runs to its end

The rule that taught this was broken in a measured run. On 10 October the message broker's consumer group was given a spend trial: one more consumer, then measure. The extra consumer drained the waiting messages so fast that the service went calm at once, and the harness of that day ended the trial the moment the calm arrived, before the measurement it existed to take. Every spend was abandoned by the success it caused. A trial that can be ended by the calm it creates can never prove that it was worth it. Hence the first rule: a trial runs to its full measurement and is never ended by the calm it causes; only the wall, a service reaching its line, or the engine's own time limit ends it early. In the one repetition of that day in which the brain allowed the third consumer early, the whole of the earlier gain came back at a fraction of the spend: the slowest five percent of messages from 2,450 ms to 57 ms, the queue from 3,375 waiting messages to 253, the work done inside the line up 17%, with 2.5 consumers on average where the earlier law had spent six to eight. The gain is real and cheap; what was missing was a fair trial of it.

## No forcing: the need and the urge

A spend is tried only when the muscle's own reading says there is something to buy: messages waiting, a cache missing, clients queued at a pool. A give-back is tried only when the whole body is calm. The compass force is the urge, smooth and bounded; the muscle's own condition is the need. The brain never moves a muscle because it wants to. It moves it because the muscle has shown that a move can pay, and then only by trial.

## One body, one brain, one tick

A muscle judged alone can be made better by making its neighbour worse: a cache that takes memory from the database beside it, a consumer that takes processor time from the service it feeds. So the brain reads the whole body first, every muscle's state, every service's speed and work, the host's processor, memory and energy where a meter exists, and judges the body's cost: the Omni index's own arithmetic over all the work, all the speed, all the machines and all the resources. On top of the body's total stands a guard for every part. A step is refused if any one service got worse beyond its cushion, even when the total improved. Making one thing best by making another worse is refused by rule, not by luck.

## One trial at a time, nothing permanent, the wall belongs to the body

If two muscles move together, the brain cannot tell which one caused what it feels. Inside a body, one new trial runs at a time, granted to the muscle asking loudest whose own condition holds; every other muscle keeps acting inside what it has already proven, so nothing is frozen and only new trials wait their turn. Every allowed step is tried again on the recheck and pulled back when it stops paying, and a muscle at native is asked again whenever its signal returns: native is where the brain stands when it has not yet been shown a reason, never a verdict. And if any service reaches its line, the trial stops, that step is undone, and every force in the body turns to brake. The wall is the body's, not the muscle's.

## Why this is the closed circle again

The Unified Circle Principle says that a system closed on itself, bounded, and pulled toward its center cannot run away: everything it does is a return. The reflex rule is that principle carried into time. The loop is closed through the system's own reaction; every excursion is bounded by the trial's limits, the cover and the wall; and every allowance is pulled back to native unless the body keeps proving it. The compass holds each reading in the middle of its band. The reflex holds each decision in the middle of what has been shown.

## What it means for the evidence

Omni v3, the engine on main, already tries a slow knob on the muscle itself before it moves it, and the live harnesses carry the same verdict around every live knob. Omni v4 is the engine written to make the rule hold on every wire by construction: one body that every muscle's wire passes through, the trial's rules inside the verdict, and no wire that can write around them. Because that changes the engine, v4 carries its own fingerprint, the package version follows it, and every result in this book is run again on it. No result is read across engines.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

PART TWO

# The Mathematics

*Eight equations, one control law, and the proofs, audits and declarations that hold them fixed.*

CHAPTER 7

# The Canonical Declaration

Every report the engine has ever produced opens with the same declaration. It is the birth certificate of the mechanism: what the engine is, which equations it runs, what every symbol means, which parameters it samples, how a run is born, conveyed and certified, and what the engine is allowed to say about itself. This chapter sets it down in full so that anyone who reads the rest of this book can always come back to the source.

## The identity of the engine

The Omni-Compass Unified Governing Convergence Control Core Engine. One canonical eight-line mathematical core, called Piece I, integrated by fourth-order Runge-Kutta (RK4) and exercised by Monte Carlo. Around that core sit the benchmark suites: the public enterprise manager mechanism benchmark, the one-stack-normalized global megastack, fragmented stack governance over matched scenarios, and the enterprise evidence suite covering cause, fail-safe, scale, muscle, security and economics. All of them run on bare metal, on premises, private, public, hybrid and edge alike, because the core does not care where it runs.

## The equation form (authoritative, eight lines)

```
(1) dE/dt   = -α_E E + β_int(t) + β_ext(t) + v_eff
(2) dU/dt   = μ U(1 - U²) - (dE/dt)/E_max - λ_U U + u,      |u| ≤ U_AUTHORITY
(3) dI_U/dt = 1 - U - σ₁ E - δ S - λ_I I_U
(4) v_eff   = χ(t) c · tanh(λ₀ + λ₁(U - U_t) + λ₂ S),       χ(t) = cos(ω_B t / 2)
(5) Φ(S)    = ½ α_s S² + ¼ β_s S³ - δ S
(6) dS/dt   = F_state(S) = -∂Φ(S)/∂S = δ - α_s S - ¾ β_s S²
(7) B'' + (ω_B / Q_B) B' + ω_B² B = γ_c δ S                  [continuous]
(8) R_B[n] = D²_h B[n] + (ω_B / Q_B) D⁻_h B[n] + ω_B² B[n] - γ_c δ S[n] = 0    [discrete audit]
```

Line (1) is the error, the energy of misalignment: it decays at its own rate, it is pushed by forces from inside and outside, and it is pushed by the effective velocity of the system. Line (2) is coherence, the alignment index: it lives in a double well with two homes at +1 and -1, it is pulled down by any rise in error, it leaks, and it is the one line the governor touches, through the control u, which is never allowed beyond its authority. Line (3) is the information channel that keeps the record of how far coherence has been from home. Line (4) is the effective velocity, saturated so that it can never exceed the limiting speed c, and multiplied by the spinor phase. Lines (5) and (6) are the potential of the coherence state and the flow it creates, downhill. Line (7) is the bath, the field that every real system sits in, driven by the state. Line (8) is the bath written on the grid the computer actually steps on: it is an audit of the numbers, not a second bath.

## The control law

```
σ        = nearest_signed_basin(U₀)
u_m      = sat(-f_U(x_m, t_m) + K_P (σ - U_m), ± U_AUTHORITY)
x_(m+1)  = RK4_h(x_m; u_m),   u_m held across all four RK4 stages
```

The nearest signed basin is declared at Step 0, before the first microstep. The control cancels the natural drift of coherence and adds a proportional pull to the declared home, then it is clipped at its authority. It is held fixed through the four stages of each RK4 step. There is no post-step overwrite of the state and no projection onto the basin. Whatever the engine reaches, it reaches by the flow of its own equations.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

# THE OMNI-COMPASS UNIFIED GOVERNING CONVERGENCE CONTROL CORE ENGINE

## THE OMNI-COMPASS EQUATION FORM

| | |
|---|---|
| **1** | $\partial E = -\alpha E + \beta_{\text{int}}(t) + \beta_{\text{ext}}(t) + v_{\text{eff}}$ |
| **2** | $\partial U = \alpha(1-U) - \frac{\partial E}{E_{\max}} - kU(1-U)$ |
| **3** | $\partial I_U = 1 - U - \sigma_1 E - \delta S$ |
| **4** | $v_{\text{eff}} = c\tanh[\lambda_0 + \lambda_1(U - U_t) + \lambda_2 S]$ |
| **5** | $\Phi(S) = \frac{1}{2}\alpha_s S^2 + \frac{1}{4}\beta_s S^3 - \delta S$ |
| **6** | $\partial S = F_{\text{state}}(S) = -\frac{\partial \Phi(S)}{\partial S} = \delta - \alpha_s S - \frac{3}{4}\beta_s S^2$ |
| **7** | $\partial^2 B + \frac{\omega_B}{Q_B}\partial B + \omega_B^2 B = \gamma_c \delta S$ [continuous] |
| **8** | $\Delta^2 B + \frac{\omega_B}{Q_B}\Delta B + \omega_B^2 B = \gamma_c \delta S$ [discrete] |

## THE OMNI-COMPASS SYMBOL CHART

| | | | | |
|---|---|---|---|---|
| **E** Error / misalignment energy | **U** Coherence / alignment index | **$I_U$** Information / internal-coherence channel | **S** Coherence-state potential | **B** Bath field |
| **$B_{dot}$** Bath-field time derivative | **$\alpha$** Feedback rate | **$\beta_{int}(t)$** Internal forcing function | **$\beta_{ext}(t)$** External forcing function | **k** Coherence logistic damping gain |
| **$\sigma_1$** E-coupling into I_U | **$\delta$** Linear coherence coupling parameter | **$\lambda_0$** Saturation bias | **$\lambda_1$** Saturation gain | **$\lambda_2$** S-state coupling gain |
| **$U_t$** Target coherence state | **c** Limiting velocity | **$E_{max}$** Per-trajectory rate normalization | **$\alpha_s$** S-potential quadratic coefficient | **$\beta_s$** S-potential cubic coefficient |
| **$\Phi(S)$** Potential-field function | **$F_{state}(S)$** Canonical S-state flow | **$\omega_B$** Bath angular frequency | **$Q_B$** Bath quality factor | **$\gamma_c$** Bath coupling gain |

**www.omni-compass.com**

U.S. PATENTS / COPYRIGHTS / TRADEMARKS — © ® 2026

*Plate 18. The equation form and the symbol chart*



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## The symbol chart

| Symbol | Meaning |
|---|---|
| E | Error, the misalignment energy |
| U | Coherence, the alignment index |
| I_U | Information, the internal-coherence channel |
| S | Coherence state potential |
| B | Bath field |
| B' | Bath field time derivative |
| α | Feedback rate |
| β_int(t) | Internal forcing function |
| β_ext(t) | External forcing function |
| k | Coherence logistic damping gain |
| $\sigma_1$ | Coupling of error into the information channel |
| δ | Linear coherence coupling parameter |
| $\lambda_0$ | Saturation bias |
| $\lambda_1$ | Saturation gain |
| $\lambda_2$ | S-state coupling gain |
| U_t | Target coherence state |
| c | Limiting velocity |
| E_max | Per-trajectory error-rate normalization scale |
| α_s | S-potential quadratic coefficient |
| β_s | S-potential cubic coefficient |
| Φ(S) | Potential field function |
| F_state(S) | Canonical S-state flow |
| ω_B | Bath frequency |
| Q_B | Bath quality factor |
| γ_c | Bath coupling gain |

## The coupling architecture of Piece I

The closed feedback core is E, U and S: error drives coherence, coherence and the state drive the velocity, the velocity drives error. The information channel I_U and the bath B with its derivative are driven response channels: they listen to the core and record it, and the core does not depend on them. $\gamma_1$ is reserved for compatibility and is inactive in the Piece I derivatives. The bath law uses $\omega_B^2$ B, so ω_B carries the units of angular frequency. Line (7) advances B and B' continuously by RK4. Line (8) evaluates the discrete residual with $D^2_h$ B[n] = (B[n+1] - 2B[n] + B[n-1]) / $h^2$ and $D^-_h$ B[n] = (B[n] - B[n-1]) / h.

## The spinor closure

The spinor extension introduces the phase θ(t) = ω_B t and multiplies line (4) by χ(θ) = cos(θ/2). At 0 degrees the factor is +1, at 360 degrees it is -1, and at 720 degrees it is +1 again. A full turn of the bath reverses the sign of the velocity; only two full turns bring it home. The standard 20-step run evaluates the part of the phase it reaches, and a dedicated evidence test checks the 0, 360 and 720 degree periodicity on its own.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## The signed basins

The canonical Monte Carlo sampler draws the starting coherence U(0) symmetrically on [-1, +1]. The nearest signed basin at Step 0 is declared the target before the first microstep. That keeps the eight-line equation exactly as written while making every governed run exercise capture into both +1 and -1. A separate dual-basin suite throws runs across from one basin to the other under disturbance and watches them recover.

## Birth, conveyance and certification

Step 0 is the sampled state before integration. Step 1 is the first completed macro step. A run has at most 20 macro steps of 10 micro steps each, at a time step of 0.1. A step is in basin when U lies within 0.10 of +1 or -1.

- **Born in basin.** A run that starts inside a basin at Step 0 is born in basin for life. Step 0 counts as the first observation of its streak. It is confirmed conveyed after Steps 0 to 4 stay in the same basin. It is never called self-tuned, and it stays under the same regulation for all 20 macro steps.
- **Self-tuning.** A run that starts outside both basins is self-tuning applied from Step 1. The same nearest-basin law is active on every trajectory, born or not, over the whole 200-microstep horizon. The category never switches the regulation off.
- **CONVEY-5.** Five consecutive same-basin observations. The first is the conveyance entry step, the fifth is the confirmation step. The latest streak may start at Step 16 and confirm on Step 20.
- **CERT-10.** Ten consecutive same-basin observations. The latest streak may start at Step 11 and complete on Step 20. The code asserts both bounds before it runs and checks every emitted row against them.

Conveyance is an event, not an ending. No run may be both born in basin and self-tuned. Stability is judged on the history, not on one snapshot.

## The Monte Carlo declaration

| Setting | Value |
|---|---|
| Initial U distribution | Uniform on [-1, +1], symmetric |
| Monte Carlo runs | 500 |
| Macro steps per run | 20 |
| Micro steps per macro step | 10 |
| Integration time step | 0.1 |
| Basin centers | +1 and -1 |
| Basin tolerance | 0.10 |
| Stability window | 5 |
| Conveyance cutoff step | 20 |

The full horizon is always executed. A run is classified as a failure if it crosses the divergence threshold at any recorded state, or if it exhausts the horizon without CONVEY-5.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## The parameter sets

| Group | Parameter | Value or range |
|---|---|---|
| Core | $\alpha$ | 4.2 |
| Core | $\beta_{int}$ | 0.0 to 1.0 |
| Core | $\beta_{ext}$ | 0.0 to 1.0 |
| Core | $E_{max}$ | 1.0 to 10.0, sampled once per trajectory |
| Core | k | 1.7 |
| Information and coupling | $\sigma_1$ | 0.38 |
| Information and coupling | $\delta$ | 0.01 to 1.0 |
| Information and coupling | $\gamma_1$ | 1.35, reserved, inactive in the derivatives |
| Information and coupling | $\gamma_c$ | 1.0 |
| Saturation and phase | c | 0.5 to 5.0 |
| Saturation and phase | $\lambda_0$ | -2.0 to 2.0 |
| Saturation and phase | $\lambda_1$ | 0.1 to 3.0 |
| Saturation and phase | $\lambda_2$ | 0.0 to 2.0 |
| Saturation and phase | $U_t$ | 0.5 |
| Saturation and phase | $\chi(t)$ | $\cos(\omega_B t / 2)$ when the spinor closure is on |
| S-state potential | $\alpha_s$ | 0.05 to 0.25 |
| S-state potential | $\beta_s$ | 0.05 to 0.25 |
| Bath | $\omega_B$ | 0.1 to 5.0 |
| Bath | $Q_B$ | 0.5 to 10.0 |
| Noise and drift | D | 0.0 |
| Noise and drift | $\mu$ | 0.0 |

## The mechanism-bound reporting doctrine

Every table, grid, chart, scorecard, classification and conclusion the engine emits must be computed from what it actually executed: the trajectory rows, the recorded state histories of E, U, I_U, S, B and B', the sampled parameters, the RK4 derivative evaluations, the basin entry and dwell histories, the line (8) residuals, the solver comparisons, the ablations, the disturbance and stress runs, the compute benchmark event records, and the enterprise benchmark ledgers.

No decorative grade, inferred score, hand-assigned quality category or presentation-only number is allowed to stand as an engine result. Prose may point to the exact source and the edge of the claim. It may not manufacture evidence, put a grade where a measurement belongs, or present a grid the mechanism never ran as though it had. That doctrine runs through every page of this book.

## Availability and the rights boundary

Time is of the essence. Omni-Compass is available from The Omni-Compass LLC for controlled technical evaluation, research collaboration, pilot integration, strategic engagement and separately licensed commercial deployment. The same rights boundary applies at every level of use: inspection, local execution, simulation, research, benchmarking, observe-only, shadow mode, supervised control, bounded autopilot, hybrid control, direct-to-muscle control, production, embedding, hosted service, productization, monetization, redistribution and derivative work.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

Public access does not make Omni-Compass open source. Commercial use of any kind requires a separately executed written license. The instrument is a license, not a sale and not a transfer of ownership. For planning only, the introductory benchmark is 10% of independently verified and contractually accepted value captured; the base and the amount are specific to each company, the terms are expected to rise as validation and adoption grow, and only a signed agreement creates any obligation.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 8

# The Engine: Eight Equations and One Control Law

The engine (`omnicompass/core.py`, frozen and fingerprinted) carries a six-part state x = (E, U, I_U, S, B, B_dot):

| State | Meaning | Physical picture |
|---|---|---|
| E | deviation: how far the system is from where it should be | a tank that fills with stress and drains on its own |
| U | alignment, between the two poles -1 and +1 | a ball in a double well: two stable poles, a hill between them |
| I_U | memory of misalignment | an integrator that remembers and slowly forgets |
| S | basin structure | a ball rolling to the bottom of its landscape |
| B, B_dot | the bath | a spring with friction that absorbs and settles energy |

The eight equations (Appendix C gives them in full):

1. dE/dt = -alpha_E E + beta_int + beta_ext + v_eff
2. dU/dt = mu U (1 - U^2) - (dE/dt)/E_max - lambda_U U + u, with |u| <= 25
3. dI_U/dt = (1 - U) - sigma_1 E - delta S - lambda_I I_U
4. v_eff = cos(omega_B t / 2) c tanh(lambda_0 + lambda_1 (U - 0.5) + lambda_2 S)
5. Phi(S) = alpha_s S^2/2 + beta_s S^3/4 - delta S
6. dS/dt = -dPhi/dS
7. d2B/dt2 = gamma_c delta S - (omega_B/Q_B) dB/dt - omega_B^2 B
8. R_B: a finite-difference audit of (7), never fed back

**The control law.** u = clip(-f_U(x, t) + K_P (sigma - U), -25, +25), where f_U is the drift of U with the command at zero, sigma is the target pole and K_P = 12. The first term cancels whatever is shoving U (the push); the second pulls U to its pole, harder the farther it is (the pull). The command is held unchanged through every stage of the fourth-order Runge-Kutta step and U is never rewritten afterwards. Inside the authority limit, U converges to its pole at rate K_P; this is proved (`docs/TRACKING_THEOREM.md`).

## What each equation does, in words

The six numbers describe the whole system the way a physician's six vital signs describe a patient, and each equation says how one of them moves.

- **Equation 1, the deviation tank.** *E* is how hard the system is being driven: queues, overload, power, heat. It drains on its own at a fixed rate (alpha_E = 3.487) and is filled by two pressures, internal (queue and overload, beta_int) and external (power, heat, network, staleness, security, beta_ext), plus a drive term v_eff that is bounded by a hyperbolic tangent. However extreme the observation, the engine's response rate is capped at c =

1. This is the saturation, the "speed limit" that no input can exceed.

- **Equation 2, the alignment.** *U* is how healthy and settled the system is, and it lives in a double well: two stable poles at +1 (healthy) and −1 (failed) with an unstable ridge at 0. A rising deviation pushes *U* out of the healthy well; the bounded command *u* pulls it back. The cubic term mu U (1 − U²) is what makes the walls of the well push back harder the farther *U* strays.
- **Equation 3, the memory.** *I_U* grows while *U* is below its pole, is reduced by deviation and stress already being dealt with, and leaks at lambda_I = 0.719. It is the engine's memory of unrelieved pressure: a system that has been stressed for an hour is not the same as one stressed for a second, and this integrator is what knows the difference.
- **Equation 4, the drive.** The bounded drive v_eff mixes the alignment and the stress through a tanh, with a slow cosine of the bath's frequency on top, so the drive breathes rather than pushes constantly.
- **Equations 5 and 6, the stress.** *S* rolls downhill in a cubic potential to its stable root. It is the slowest restoring force: structural stress from heat, power, network, security and staleness that builds and relaxes over minutes rather than seconds.
- **Equation 7, the bath.** A damped harmonic oscillator (natural frequency omega_B = 1, quality Q_B = 2.5) driven by stress. It turns stress into a smooth, lagged tide, and it is the only second-order element in the engine; its friction is what settles the whole.
- **Equation 8, the audit.** The bath equation is re-checked numerically every step with a finite difference, and the check is recorded and never fed back: an instrument on the engine, not a part of it.

Integration is classical fourth-order Runge-Kutta, ten micro-steps of 0.01 per macro step, and the C++ twin matches the Python to $3.6 \times 10^{-15}$ on 500 reference trajectories (Chapter 64).

## From telemetry to state, and from state to action

Each decision, the telemetry is normalised to nine observations: queue, load, power, thermal, network, drift, staleness, security and conflict. An observed state is computed from them by fixed weights (`omnicompass/adapter.py`, `observe_vector`), the engine's state is blended toward it with weight 0.339, and the state is then evolved by one macro step of the equations. From the state, the governor evaluates the control's push, u(x)/25, which is a number between −1 and +1 that says how hard the engine is pulling toward health; a target utilisation that falls as the memory and the deviation rise; whether a release (giving something back) is permitted at all, which is only while the push reports convergence and the band and dwell allow; and whether change is permitted, which requires *U* above its gate and no security block. All of this is written to the audit every decision (`docs/MECHANISM_OF_ACTION.md`, Chapter 56 to 6, and `python tools/mechanism.py`, which reproduces the measured numbers in `results/MECHANISM_OF_ACTION.json`).

The nine observations are worth naming in plain words, because every later number is a weighted sum of them (`omnicompass/adapter.py`, `observe_vector` and `assimilate`). **Queue** is the work waiting as a share of the work being served; **load** is utilisation against the stack's own target; **power** and **thermal** are the electrical and heat stress against their limits; **network** is the same for the wire; **drift** is how far the levers stand from where native had them, so that a governor that has pushed far from the operator's settings reads as more stressed than one that has not; **staleness** is the age of the readings, which rises when a sense goes quiet; **security** is the hold flag; and **conflict** counts other hands seen on the levers. Each is clipped to a declared range before it is used, so no single runaway reading can swamp the others. The stress the engine assimilates leans most on the queue (a weight of 0.25) and on load above 85% of target (0.18), then power, heat, network and drift in that order, with conflict and staleness last; the utilisation it blends toward falls with the queue first (0.27) and with power and heat next. The weights are fixed



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

numbers in the frozen engine, the same on every stack, and a reader who wants to know why a decision read the body as stressed can recompute the sum from the audit line, which records all nine observations. What the weights encode is a judgement made once: that work waiting is the first sign of a body in trouble, that a lever far from native is itself a mild stress, and that a reading one cannot trust is a reason for caution rather than for action.

## Where the engine sits, and where the compass law sits

A reader meeting the engine and then the compass law (Chapter 17) may ask which of the two moves the knob. The answer, in the live controllers, is this: the **compass law** sets each knob, the **verdict** narrows where the law may go, and the **engine** grants the authority, feeds the release gate and the nervous system, and is audited every decision (`omni_controller/controller.py`, its docstring). The engine is the organism's physiology: how stressed it is, how long it has been, whether it is settling. The compass law is the hand on each lever. The engine's own control law $u$ is evaluated, not applied to a knob directly: it is the push the nervous system reads to decide whether giving anything back is permitted now. This separation is why the engine could be frozen with its proofs untouched while the compass law was written as its own, separately tested law, and why `verify.py` seals both.

One decision, followed through, shows the order. The controller wakes on its period and reads its senses: the service gauge (a response time, a statement latency, a consumer lag), the resource gauge (machines, connections, memory) and the knob's current value read back from the device. The **engine** takes the normalised observations, blends its state toward them and evolves one macro step; from the state it reports whether the body is converging, what authority the organ has earned and whether a release is permitted at all (section 4.2). The **compass law** then reads the service gauge as a position in its band and computes the force (Chapter 17): positive, and the direction rule asks for capacity in proportion; negative, and it asks for one notch back, but only if the engine's release gate is open, the resource is demonstrably idle and the dwell has passed. The **verdict** stands over the slow knobs: a knob the paired trial has not cleared at the 2% line is not moved at all and stays native. What survives all three is one write to the **plug**, read back from the device, and one line in the **audit** that records the senses read, the engine's state and push, the force, the gate's reasons and the value sent and taken. In the kill test's first repetition (section 7.1) this produced 114 such lines and 42 writes: every decision that wrote nothing is in the record with the reason the gate held, which is the point of writing all of it down.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 9

# The Canonical Engine

One engine runs Omni-Compass, and every result in this repository comes from it. This page names it, gives its equations exactly as the code computes them, and names every other form as a variant. Where any document, manual, chart or filing states the equations differently, this page and the file it fingerprints are what the software runs.

## 1. The canonical engine: `symmetric_verified`

| Name | `symmetric_verified` (`omnicompass/configurations.py`, `DEFAULT`) |
|---|---|
| Code | `omnicompass/core.py` (Python), `cpp/src/core.cpp` (C++ twin, sealed in `results/SEAL.json`) |
| Fingerprint | SHA-256 of `omnicompass/core.py`, recorded in `results/PREREGISTRATION.json` and `RELEASE_MANIFEST.json`; checked by `verify.py` |
| Evidence | every simulation, benchmark, live Kubernetes run and GPU harness in this repository |

State: E (deviation), U (coherence), I_U (unmet-need integral), S (structural stress), B and B_dot (the bath).

```
v_eff  = cos(omega_B t / 2) · c · tanh(lambda_0 + lambda_1 (U − 0.5) + lambda_2 S)        (spinor closur
    ↪ e, 720°)
dE/dt  = −alpha_E E + beta_int + beta_ext + v_eff
dU/dt  = mu U (1 − U²) − (dE/dt) / E_max − lambda_U U + u                                (symmetric dou
    ↪ ble well)
dI_U/dt = (1 − U) − sigma_1 E − delta S − lambda_I I_U
dS/dt  = delta − alpha_s S − (3/4) beta_s S²                                              (gradient of
    ↪ Phi)
dB/dt  = B_dot
dB_dot/dt = gamma_c delta S − (omega_B / Q_B) B_dot − omega_B² B
Phi(S) = alpha_s S² / 2 + beta_s S³ / 4 − delta S
```

u is the controller's command, `KP (target − U) − drift`, bounded by `U_AUTHORITY`; integration is RK4 with the macro and micro steps of `omnicompass/core.py`.

### *Mechanism identity*

`results/MECHANISM_IDENTITY.json` (`tools/mechanism_identity.py`) fingerprints the mechanism M = (F, Theta, C, h, G, M_act, dt, A) component by component: state law, parameters, controller, observation map, authority law, actuator map, execution timing, shield. `verify.py` fails if any component changes without the record being rewritten. Every evidence claim names the mechanism id that produced it; "the eight-line engine" alone names none.

| Configuration | Mechanism id | Role |
|---|---|---|
| `symmetric_verified` | `29d9808dfb8f626ad5de17a8a1efa37411dbab08a64af4b276143b466f7ce21c` | canonical |
| `printed_eight_line` | `cd333dc166fb7684ec0fca71f5f50488041e47825ee47d1ea667bec1f3d2259d` | named alternative embodiment |

On the frozen 500-fixture population (`benchmarks/core_evidence.py`, seed 223387268), both are executable and neither is a stand-in for the other:

| | finite | CONVEY-5 | CERT-10 | mean final target error | mean integrated abs(u) | max abs(u) |
|---|---|---|---|---|---|---|
| `symmetric_verified` | 500/500 | 500/500 | 500/500 | 7.48e-5 | 0.878 | 13.93 |
| `printed_eight_line` | 500/500 | 500/500 | 500/500 | 5.13e-6 | 11.79 | 18.87 |

The printed form reaches its target more tightly with about 13 times the control effort. The structure is Option A: one canonical configuration, one named alternative; no equivalence is claimed.

## 2. Variants

### `printed_eight_line` *(the printed chart)*

Implemented in `omnicompass/configurations.py`, checked against the printed plate (`tests/test_engine_configurations.py`), **not benchmarked**: no result in this repository comes from it.

```
v_eff  = c · tanh(lambda_0 + lambda_1 (U − U_t) + lambda_2 S)                    (no spinor fac
    ↪ tor)
dE/dt  = −alpha E + beta_int + beta_ext + v_eff
dU/dt  = alpha (1 − U) − (dE/dt) / E_max − k U (1 − U) + u                        (logistic)
dI_U/dt = (1 − U) − sigma_1 E − delta S                                          (no lambda_I
    ↪ term)
dS/dt, dB/dt, dB_dot/dt as in section 1
```

Differences from the canonical engine: logistic U instead of the symmetric double well; no spinor factor; a general target U_t; no lambda_I damping; alpha in place of alpha_E. alpha_U and k enter the printed form; they do not enter the canonical engine.

## 3. What would change this declaration

Promoting a variant to canonical needs, in one commit: the variant run through every harness beside the canonical engine, its results reported next to the canonical results, this page and `RELEASE_MANIFEST.json` updated, and the preregistration amended on record (`results/LOCK_AMENDMENTS.json`). Until then the canonical engine is `symmetric_verified`.

## 4. Open mathematical items (from `docs/FORMAL_STATUS.md` and the handoff)

- A global stability proof of the forced six-state system is not closed. What is held on the U channel is proved in `docs/TRACKING_THEOREM.md`: unsaturated exponential tracking; the saturated case under a drift bound that holds over the declared box (F_bar = 16.86 < 25); the sampled-data bound of the executed RK4 controller; admissibility.
- Candidate routes: the Unified Circle Principle on a region; Theorem 5.6 of the Closed Structure, if its core maps onto (E, U, I_U, S, B).

## 5. For filings

Which form a patent or copyright filing claims as the principal embodiment is a decision for The Omni-Compass LLC and its counsel. Whatever that decision, the software described by this repository runs the canonical engine of section 1, and a filing that claims the printed form should name `symmetric_verified` as the embodiment that has been implemented and tested, or the printed form should be benchmarked first (section 3).

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 10

# The Closed Circle: Why It Cannot Leave Its Compass

The governing principle is the Unified Circle Principle:

```
dX/dt = G(X),  X(0) in Omega
G(X) . n(X) &lt;= 0 on the boundary of Omega        (the flow points inward at the wall: nothing crosses)
grad L(X) . G(X) &lt;= 0                           (the compass's energy L only falls)
=&gt; lim X(t) in M*                              (every path settles in the bottom of the compass)
```

In control engineering the second line is the Nagumo condition for an invariant set and the third is a Lyapunov function. Together they mean: anything that starts inside the container stays inside it, for every disturbance up to a known size, and comes to rest at the bottom. That is a certificate, not a test: it answers every case inside the walls at once.

**In plain words.** Picture a round valley with a fence around its rim. The first line says the valley has a shape: given where the system is, the equations say which way it rolls. The second line says that at the fence the ground always slopes inward, so nothing that starts inside can ever roll out, however it is pushed within the strength the fence was built for. The third line says there is a height, *L*, that only ever decreases along any path, so the system cannot circle forever at one level; it must keep descending. The conclusion follows: every path ends at the bottom, the set *M**. The engineering value of such a certificate is that it is not a sample of behaviours; it is a statement about all of them at once, inside the fence.

**What is certified, and what is not.** The certificate is about the engine's own state: the six numbers of Chapter 8 stay inside their walls and settle. It is not a certificate about the plant the engine governs, whose physics the engine does not contain; that is why every plant is tested paired against native (Part VI), why the compass law clips every write to the knob's cover (Chapter 17), and why the shield between the engine and the levers is tested separately on two million adversarial cases with zero violations (`tests/test_shield_properties.py`) and the C++ engine soaked on one hundred million decisions with no failure (`results/SOAK.json`). The mathematics closes the brain; the harness, the cover, the verdict and the paired receipts close the hand.

The engine's states are each closed in this sense: the deviation tank drains faster than it can fill; the alignment's cubic walls push back from far out; the memory forgets; the bath's friction settles it. The rule behind all four is the compass's east-west line: what flows out (drain, damping) must always be able to match what flows in (drive). The formal status of each statement, what is proved, what is tested and what is conjecture, is written out in `docs/FORMAL_STATUS.md` and `docs/TRACKING_THEOREM.md`, and the theory chapters of this book (The Unified Circle Principle, Closing the Circle in the Engine) give the derivations.

**What the certificate buys an operator, in practice.** Three things. The engine cannot run away: there is no input, no sequence of readings and no fault in a sense that drives its internal state to infinity or into oscillation that grows, because the walls are part of the equations and not a check bolted on afterwards. The engine cannot hold a grudge:

the memory state forgets at a fixed rate, so a bad hour does not bias the next day, and a governor restarted after a kill starts from the same rest state the certificate names. And the engine's behaviour under any disturbance smaller than the fence is the same kind of behaviour, a descent to the bottom, so a test on one plant's readings says something about the engine on another's; what it does not say, and what no mathematics of the engine alone could say, is how the plant answers, which is why the paired runs exist. An underwriter reading this section should take from it exactly this much: the part of the system that makes decisions is bounded by proof; the part of the system that carries them out is bounded by the cover, the gates and the hand-back, each tested; and the part that is the customer's own plant is measured, never assumed.

**The compass.** The Omni-Compass rose carries the whole Greek alphabet, Alpha to Omega: the complete set, in a ring whose end runs back into its beginning. Its north-south axis is polarity: Alpha and plus at the top, Omega and minus at the bottom, the two poles of the alignment's double well. Its east-west axis is flow: Beta and the push outward on one side, Gamma and the pull inward on the other, the drive and the damping. The spiral at the center is the attractor every path winds into. The ring closing on itself is the return: when Omni-Compass stops, every lever returns to where it began.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*




*Plate 16. Mathematical integration*



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

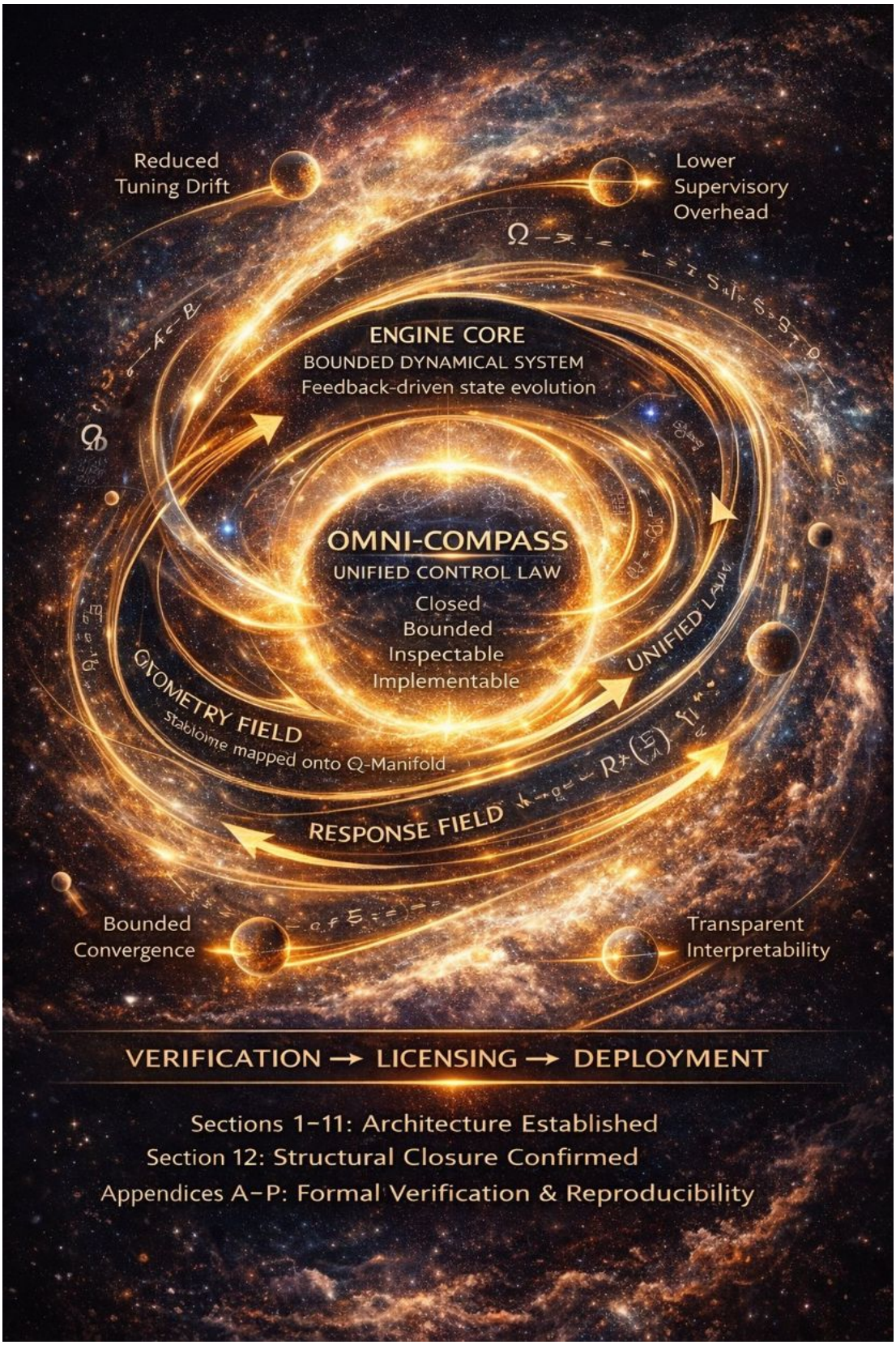



*Plate 17. The engine core: the governing law in motion*

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

CHAPTER 11

# Closing the Circle in the Engine

Each state of the engine was checked against the closed-circle conditions:

| State | Closed? | Why |
|---|---|---|
| E, deviation | closed | it drains at rate alpha_E while its inputs are bounded (the drive is capped by tanh) |
| U, alignment | closed | the cubic term mu U (1 - U^2) pushes back hard from far out |
| I_U, memory | closed | it forgets at rate lambda_I |
| B, the bath | closed | the friction omega_B / Q_B settles it |
| S, the basin | closed on the positive side; open on the negative side below an unstable ridge | Phi(S) carries a cubic term, so the landscape falls away below the ridge (at S = -3.5 with the default parameters) |

Three openings were found and each has its closure:

1. **The basin's negative side.** A quartic term in Phi(S) would give the compass a wall on both sides. This changes the frozen engine and is a new version with its own proof.
2. **The outer loop.** The frozen live governor computes the push and pull u and uses it as a convergence signal; it does not send u to a lever. The compass closes this loop: reading, force, plug, lever, read-back. On the modelled card this is the difference between +0.1% and several percent of work per energy (the one-wire governor against the corrected two-wire compass, `results/sim/gpu_two_wire/`).
3. **The corner.** clip() is a hard stop; tanh is its smooth form. The compass uses tanh.

**What closing the loop has shown since.** The compass law of the second opening is the law every result in the manual was made on: it was frozen as Omni v1 on 5 October 2026 with its proofs untouched, grown to the 945-muscle catalog as v2, and given one gate as v3, the slack gate on speed knobs, after the v2 realms read a service tradeoff on the Physics realm and the tower. That gate is the closed circle applied to a muscle's physics: an axis that is busy more than half the time at full speed has no slack to spend, and a law that spent it anyway would be spending energy for nothing; with the gate, every organism reads superior within its guardrails and zero muscles read worse (`results/realms/REALMS.md`). On real software the same law, read through the plug and the read-back, is the law of the Kubernetes, database, broker and cache tables, with their gains and their losses. The first opening, the basin's negative side, remains open and documented: it would be a new version with its own proof, and no result depends on it.

CHAPTER 12

# The Tracking Theorem

This page proves the internal tracking result of the canonical engine (`symmetric_verified`, mechanism id in `results/MECHANISM_IDENTITY.json`) in four steps, from the continuous law to the code as executed. Each statement carries one evidence class (see `docs/EVIDENCE_LEDGER.md`): **T** proved here, **V** checked by computation over a finite, frozen set (`tools/tracking_bounds.py` → `results/TRACKING_BOUNDS.json`).

What this page establishes is **internal**: inside the declared model, the controller conveys U to its target sigma. It does not establish that sigma is the right target for any outside plant, that the telemetry map is right, that any actuator mapping (HPA, GPU, CPU) is right, any energy saving, production performance, or stability of the whole forced six-state system. Target correctness and target conveyance are separate obligations; this page carries only the second.

## Setting

U channel of equation (2): `dU/dt = f_U(x, t) + u`, with the uncontrolled drift

```
f_U(x, t) = mu U (1 − U²) − (dE/dt) / E_max − lambda_U U
```

and the bounded cancellation-plus-proportional controller (`omnicompass/core.py`, `control_command`)

```
u = sat(−f_U(x, t) + KP (sigma − U)),   sat(v) = max(−u_max, min(u_max, v)),   KP = 12, u_max = U_AUTHOR
    ↪ ITY = 25.
```

Tracking error `e = U − sigma`, sigma in {−1, +1}.

## Theorem 1 (T). Unsaturated tracking

On the unsaturated region `Omega_unsat = { x : |−f_U(x, t) − KP e| <= u_max }`, `de/dt = −KP e`. Along any interval spent in `Omega_unsat`, `e(t) = e(t0) exp(−KP (t − t0))`, and `V = e²/2` satisfies `dV/dt = −KP e² < 0` for `e ≠ 0`.

*Proof.* Inside `Omega_unsat` the saturation is inactive, so `dU/dt = f_U − f_U + KP (sigma − U) = −KP e`; since sigma is constant, `de/dt = dU/dt`. The solution and `dV/dt = e de/dt = −KP e²` follow. ∎

This is a statement about intervals inside `Omega_unsat` only; it is not global convergence.

## Theorem 2 (T). The saturated case, under a declared drift bound

**Assumption D.** Along the trajectory, `|f_U(x(t), t)| <= F_bar < u_max`.

Under D:

1. If saturation is active with `e < 0`, then `u = +u_max` and `de/dt = f_U + u_max >= u_max − F_bar > 0`; symmetrically for `e > 0`. So `dV/dt <= −(u_max − F_bar) |e| < 0` wherever saturation is active.
2. With Theorem 1, `V` is strictly decreasing wherever `e ≠ 0`, saturated or not; `e` never changes sign (at `e = 0` the command is unsaturated because `F_bar < u_max`, so `e = 0` is an equilibrium of the error dynamics).
3. Any saturated stretch ends within `(|e(t0)| − e*) / (u_max − F_bar)` time units, `e* = (u_max − F_bar) / KP`; the ball `|e| <= e*` lies inside `Omega_unsat` and is forward invariant, and inside it the decay is exponential at rate KP.

*Proof of 1.* Saturation with `u = +u_max` means `−f_U − KP e > u_max`, so `−KP e > u_max + f_U >= u_max − F_bar > 0`, so `e < 0`, and `de/dt = f_U + u_max >= u_max − F_bar`. The other side is symmetric. 2 and 3 follow from 1 and Theorem 1, and `|−f_U − KP e| <= F_bar + KP |e| <= u_max` on the ball. ∎

**Where D holds (T).** On the declared box — parameters in `PARAMETER_RANGES`, `E0` in [0, 1], `|U| <= 1.1` — equation (1) keeps E in `[(beta − c)/alpha_E, (beta + c)/alpha_E]` widened to hold E0 (the right side of (1) points inward at both ends, because `|v_eff| <= c`), so `|E| <= 7/4.2`, `|dE/dt| <= 4.2·|E| + 2 + 5 <= 14`, and

```
F_bar = 6 · 2/(3√3) + 14 / 1 + 0.5 · 1.1 = 16.86 &lt; 25 = u_max.
```

Because e keeps its sign and `|e|` does not grow (continuous time), U stays between U0 and sigma, so `|U| <= 1` and the box is self-consistent for the nearest-basin target. **Theorem 2 therefore holds globally over the declared box, in continuous time.** The actuator can still saturate: `|raw| <= F_bar + KP |e0|` can exceed 25 when `|e0|` is near 1.

**Checked (V).** Over the 500 frozen fixtures: observed `max |f_U|` = 6.84 (nearest target), 7.01 (wrong target), each under that fixture's own bound (max 8.99) and under 16.86; saturated micro steps 0 of 100 000 (nearest target), 5 of 100 000 (wrong target).

## Theorem 3 (T with a computed constant). The executed sampled-data controller

The code does not run the continuous law. Per micro step `h = MICRO_DT = 0.01`, it computes `u_k` from `x_k`, holds it through the four stages of one RK4 step (zero-order hold), and takes `x_(k+1) = F_h(x_k, u_k)`. There is no overwrite of U after the step, no projection onto a basin, no state replacement.

Integrating the U equation over one held interval,

```
e_(k+1) = e_k + h (f_U(x_k) + u_k) + ∫_(t_k)^(t_k+h) (f_U(x(t)) − f_U(x_k)) dt + tau_k
```

with `tau_k` the RK4 truncation error. Unsaturated, `f_U(x_k) + u_k = −KP e_k`, so

```
e_(k+1) = rho e_k + d_k,   rho = 1 − h KP = 0.88,   |d_k| &lt;= (h²/2) sup|df_U/dt| + |tau_k| =: epsilon_h.
```

Then by induction `|e_k| <= rho^k |e_0| + (1 − rho^k)/(1 − rho) · epsilon_h`, and the error enters and stays in the neighbourhood `|e| <= epsilon_h / (1 − rho)`.

The proof above is exact given epsilon_h. **epsilon_h itself is computed, not proved (V):** the largest `|d_k|` over every micro step of the 500 frozen fixtures is 0.00378 (nearest target), giving the ultimate bound 0.0315, inside `BASIN_TOL` = 0.10 that CONVEY-5 and CERT-10 test. With the target aimed wrong, epsilon_h = 0.0243 (dominated by the long transient, which includes the 5 saturated steps) and the bound 0.20 is wider than the basin — a conservative bound; the fixtures still certify 500 of 500.



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

What the discrete execution does **not** inherit: the continuous monotone decay. Within the band, `|e|` grew on 27 061 of 100 000 micro steps and e changed sign 747 times — the sampled controller dithers inside `epsilon_h / (1 − rho)`.

**CONVEY and CERT.** They are dwell checks against a declared neighbourhood (`|U − sigma| <= 0.10` for 5 and 10 consecutive macro steps). Theorem 3 says why they pass when the bound sits inside the neighbourhood. They are not an external certification.

## Theorem 4 (T continuous; V discrete). Admissibility

Admissible set for a fixture with parameters p and initial state x0:

```
A = { x : E in [E_lo, E_hi],  |U − sigma| &lt;= |U0 − sigma|,  S between S0 and S_plus(p) }
```

`E_lo, E_hi` as in Theorem 2; `S_plus` the stable root of equation (6).

- **Continuous (T).** A is forward invariant: E by the inward-pointing field at both ends; U by Theorem 2; S because equation (6) is the one-dimensional gradient flow of Phi, monotone toward `S_plus` from any `S0 > S_minus` (`s_admissibility` in `results/core_evidence.json`: 500 of 500 fixtures start above `S_minus`).
- **Discrete (V).** For the executed map `F_h(x, C(x))`, every micro step of every fixture: `x_k in A` implied `x_(k+1) in A` — 0 failures in 100 000 steps, target nearest or wrong. This is a finite verification on the frozen fixtures, not a proof over the box. An interval-arithmetic proof over the whole box is **open (O)**.

## What remains open (O)

- Global stability of the forced six-state system (E, I_U, S, B are not all covered above).
- A proved (not computed) epsilon_h over the declared box, and discrete invariance over the box.
- The inheritance / intertwining residual against an outside plant: instrumented (`tools/gpu_reps.py`, representation fidelity), not yet measured on a real card.
- The printed configuration (`printed_eight_line`) is not covered by this page: its U drift is logistic, not the double well, and needs its own bound (`docs/CANONICAL_ENGINE.md`).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

CHAPTER 13

# The Conveyance Law

## 1. What the manuscript says, read as mechanism

| Source | Text | Mechanism |
|---|---|---|
| Ch. 29 §8 | "E_system = E + U + S + B … dE_system/dt = 0 (over full cycle). Energy redistributes. Energy reorganizes. Energy does not diverge." | The budget (site watts, cluster cores) is **conserved**: it is moved between organs, never created. |
| Ch. 31 §4 | "Redistribution B acts as mediator between the baths. When E grows locally, B increases to distribute gradients." | Budget flows **down the gradient of need**: from organs holding surplus to organs in deficit. |
| Ch. 31 §5 | "eigenvalues of the coupled system must remain bounded … $Re(\lambda_i) \leq 0$" | The flow must have a proof of convergence, not a tuned damper. |
| Ch. 30 §5-7 | "Redistribution surge … channels stored deviation into expansion … controlled release … No discontinuity." | A **reserve** is held and released **continuously** to organs at their limit. There is no on/off. |
| Ch. 29 §6 | "Backpressure develops through S … Compression is … structured inward rotation." | When need falls, surplus is **pulled back** smoothly. Budget nobody needs is not spent. |
| App. J | "the engine can only act through what you expose" | Organs keep their own mechanisms. The law sets only each organ's share of the budget. |

## 2. The law

**Notation.**

- Organs $i = 1..n$ share a budget $A$.
- Organ $i$ holds allocation $a_i$ and has need $d_i$: the budget that serves its work at the engine's utilisation target $\rho$.
- Its deficit (local deviation) is

$e_i = d_i / (\rho\, a_i) - 1$ ($e_i > 0$: needs juice; $e_i < 0$: holds surplus)

**The redistribution mediator:**

$da_i/dt = \kappa\, a_i\, (e_i - \bar{e})$, $\bar{e} = \Sigma_j\, a_j\, e_j / A$

## 3. Properties and proofs

**P1. Conservation.**

- $\Sigma_i\, da_i/dt = \kappa\, (\Sigma_i\, a_i e_i - \bar{e}\, \Sigma_i\, a_i) = \kappa\, (A\, \bar{e} - \bar{e}\, A) = 0$.
- The budget is moved, never created, as in Ch. 29 §8.

**P2. Gradient flow with a Lyapunov function.**

- Substitute $e_i$: $a_i(e_i - \bar{e}) = d_i/\rho - a_i\, D/(\rho A)$, where $D = \Sigma\, d_i$.

- So $da_i/dt = (\kappa/\rho)(d_i - a_i\, D/A)$.
- This is the Shahshahani gradient on the simplex $\Sigma\, a_i = A$ of $F(a) = \Sigma_i\, (d_i/\rho)\, \ln a_i$. The Shahshahani metric weights by $1/a_i$, and $\partial F/\partial a_i = d_i/(\rho\, a_i)$, so $a_i(\partial F/\partial a_i - \text{mean})$ is exactly the flow above.
- Along the flow $dF/dt = \Sigma_i\, (1/a_i)(da_i/dt)^2 \cdot (\rho/\kappa) \geq 0$.
- F is strictly concave, so F is a Lyapunov function and the flow cannot cycle.

**P3. Global exponential convergence.**

- The ODE in P2 is linear: $a_i(t) = a_i^* + (a_i(0) - a_i^*)\, e^{\wedge}(-\kappa D t/(\rho A))$, with $a_i^* = A\, d_i/D$.
- The Jacobian is $-(\kappa D/(\rho A))\, I$ on the simplex, so every eigenvalue is $-\kappa D/(\rho A) < 0$. This meets $\mathrm{Re}(\lambda) \leq 0$ of Ch. 31 §5 strictly.
- At the equilibrium every organ has the same deficit $e^* = D/(\rho A) - 1$. None is starved while another idles.
- `replicator_step` integrates it exactly, in closed form, for any step size.

**P4. Bounds (floors, ceilings, the budget).**

- Allocations are projected onto $\{lo_i \leq a_i \leq hi_i,\ \Sigma\, a_i \leq A\}$.
- The projection is water-filling: organs at a bound keep it, and the rest share the remainder in proportion.
- Budget an organ cannot use (it is at its need or its ceiling) is not spent.

**P5. Reserve and surge.**

- The dual-bath rate channel (the rate tracker) projects the rise in need over one actuation delay: $R = \Sigma_i \max(0, \dot{d}_i)$, capped at $A/2$. R is held back.
- After the flow, R is released continuously to organs whose deficit stays positive, in proportion to their shortfall and up to their ceilings.
- This is the controlled release of Ch. 30: continuous state, no switch.

## 4. Evidence

**Mathematics in code.** `tests/test_conveyance.py` (in `verify.py`) checks 20,000 random systems:

- P1 conservation to 1e-9;
- P2, F never decreases;
- P3, the trajectory equals the closed form;
- P4, bounds and budget.

It also runs 2,000 systems over 30 steps and checks the full law never exceeds the budget or breaks a floor. That test found and fixed one real bug: an organ whose need fell below its device floor was given a ceiling under that floor.

**Simulation.** `hardware/site_exchange.py`: four GPU groups of 16 H100-class GPUs share one site budget (MLPerf gamma). Held-out seed 929292, 48 scenarios per budget. Results: `results/hardware/SITE_EXCHANGE_HELDOUT_*.json`.

| Site budget (share of TDP) | 70% | 60% | 50% |
|---|---|---|---|
| Site-budget violation minutes, native / independent / **conveyance** | 10.5 / 1.5 / **0** | 45.6 / 10.5 / **0** | 106.1 / 31.3 / **0** |
| Energy (kWh), native / static split / independent / **conveyance** | 142.5 / 117.6 / 104.1 / **103.8** | 139.1 / 107.9 / 103.5 / **102.0** | 127.3 / 97.1 / 101.0 / **96.4** |
| Backlog minutes, static split (the arm that also keeps the budget) / **conveyance** | 32.0 / **1.0** | 89.7 / **8.9** | 187.3 / **72.2** |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

**Reading.**

- **Keeping the budget.** Conveyance is the only dynamic arm that never exceeds the site budget.
- **Energy.** It uses the least energy at every budget.
- **Against the only other arm that keeps the budget (static split):** 31-115 fewer backlog minutes.
- **At tight budgets, native and independent sizing show less backlog,** but only by drawing power over the site limit for 10-106 minutes, which in a facility trips breakers.
- **When the budget is below total need,** conveyance shares the shortage in proportion to need. That is the unique stable point of P3: fair, and not a choice of weights.

**CPU and GPU on one budget.** `hardware/node_exchange.py`: the same law with CPU organs beside the GPU organs. Four groups, each two 8-GPU servers (16 H100-class GPUs, 4 CPU sockets); each unit of GPU work needs 0.35 of the CPUs at full clock to feed it; demand 1.6x the plant traces (the GPUs are held back by the budget, not by work). Arms:

- **S** today's practice: every GPU at one fixed cap low enough that the site fits even with every CPU at maximum;
- **XM** conveyance among the GPUs, counting the CPUs' last measured draw;
- **XC** conveyance over CPU and GPU organs together: each CPU group is held to the clock its feeding work needs, and the watts it no longer holds flow to the GPU groups in deficit.

Seed 515151, 24 scenarios. Results: `results/hardware/NODE_EXCHANGE_*.json`. XC against S, paired:

| Site budget (share of GPU TDP + CPU maximum) | 60% | 70% | 80% |
|---|---|---|---|
| Work served | **+5.7%** | **+3.6%** | **+1.4%** |
| Backlog minutes | −15.8% | −27.5% | −17.8% |
| 95th-percentile latency factor | −39.6% | −43.6% | −35.2% |
| Site-budget violation minutes, native / S / XM / **XC** | 195.8 / 0 / 3.9 / **0** | 140.9 / 0 / 6.2 / **0** | 61.8 / 0 / 5.3 / **0** |

**Reading.**

- **More work from the same building.** The tighter the budget, the more the CPUs' unused watts are worth to the GPUs.
- **Holding the CPUs is what makes the hand-over safe.** Counting the CPUs' measured draw (XM) serves about as much, but goes over the budget for 4-6 minutes in every setting: a CPU can rise between one reading and the next. XC holds each CPU to its allocation, so the watts it gives up are really free, and it never goes over.
- **Most of the gain is the CPUs' reserve, not their clock.** Against XM, XC gains ~1% work per kWh; against S, the watts a fixed plan must keep for CPUs that might peak are what the GPUs receive.
- The same holds for the least and most favourable MLPerf gamma fits, and for CPU shares 0.2 and 0.6 (+2.3% to +4.2% work at the 70% budget), `tests/test_node_exchange.py` checks N1-N4 in `verify.py`.

## 5. What is not claimed

- This is a simulation on declared device physics.
- The live levers that would carry it are GPU power limits (`nvidia-smi -pl`, DCGM), RAPL package limits, and pod CPU limits under a namespace budget. They exist in `omni_controller/muscles.py`, but the exchange between them has not run on hardware.


*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 14

# Formal Status of the Mathematics

Not a global-stability proof of the forced six-state system. Proofs and their evidence classes: `docs/TRACKING_THEOREM.md`; every statement's class: `docs/EVIDENCE_LEDGER.md`.

Held:

- Isolated S-flow is the gradient of Phi (same formula in both cores).
- (T) Unsaturated: dU/dt = KP (sigma − U); V = e²/2 has dV/dt = −KP e² on that channel (Theorem 1).
- (T) Saturated, with the U drift bounded by F_bar < U_AUTHORITY: V decreases wherever e ≠ 0; over the declared parameter box F_bar = 16.86 < 25, so this holds over the box in continuous time (Theorem 2).
- (T, constant computed) Sampled-data RK4 controller: e_(k+1) = 0.88 e_k + d_k; with epsilon_h computed over the frozen fixtures (V), the ultimate bound 0.0315 lies inside the 0.10 basin (Theorem 3).
- (T continuous, V discrete) Admissible box forward invariant (Theorem 4).

Open:

- epsilon_h and discrete invariance proved over the whole box (interval arithmetic).
- Inheritance embedding / intertwining residual against a real plant (Closed Structure Def. 11.1): instrumented in the GPU bench (`tools/gpu_reps.py`, representation fidelity), not yet measured on a card.
- Mapping of monograph Theorem 5.6 onto (E,U,I_U,S,B).
- A tracking bound for printed_eight_line (logistic U drift).

Default core remains symmetric_verified (mechanism id in `results/MECHANISM_IDENTITY.json`).

CHAPTER 15

# The Mechanism of Action

Every statement here is tied to code in this repository, and the measured numbers are reproduced by `python tools/mechanism.py`, which writes `results/MECHANISM_OF_ACTION.json`. The engine files are frozen. Their SHA-256 hashes are checked by `verify.py`.

## 1. The state: six numbers that describe the whole system

| Symbol | Name | What it stands for in a data centre |
|---|---|---|
| E | energy / excitation | how hard the system is being driven: queues, overload, power, heat |
| U | order parameter | how healthy and settled the system is: +1 healthy basin, -1 failed basin |
| I_U | integrated need | unmet need accumulated over time: memory of pressure that has not been relieved |
| S | stress | slow structural stress from heat, power, network, security, staleness |
| B, B_dot | bath and its rate | a damped oscillator driven by stress: the slow "tide" of physical load |

## 2. The equations (omnicompass/core.py, lines 5-12)

```
(1) dE/dt    = -alpha_E E + beta_int + beta_ext + v_eff
(2) dU/dt    = mu U (1 - U^2) - (dE/dt)/E_max - lambda_U U + u,          |u| &lt;= 25
(3) dI_U/dt  = (1 - U) - sigma_1 E - delta S - lambda_I I_U
(4) v_eff    = cos(omega_B t / 2) * c * tanh(lambda_0 + lambda_1 (U - 0.5) + lambda_2 S)
(5) Phi(S)   = alpha_s S^2 / 2 + beta_s S^3 / 4 - delta S
(6) dS/dt    = -dPhi/dS = delta - alpha_s S - (3/4) beta_s S^2
(7) dB/dt    = B_dot ;   dB_dot/dt = gamma_c delta S - (omega_B / Q_B) B_dot - omega_B^2 B
(8) R_B[n]   = finite-difference audit of (7), never fed back
```

Each term does one job:

- **(1) Energy.** Energy decays at rate alpha_E = 3.487. It is driven by two forcings: internal pressure `beta_int` (queue and overload) and external pressure `beta_ext` (power, heat, network, staleness, security). The drive term `v_eff` (4) is bounded by `c` through the hyperbolic tangent, so no input can drive E without limit. This is the saturation (the tanh "speed limit"): however extreme the observation, the engine's response rate is capped at c = 1.
- **(2) Order.** `mu U (1 - U^2)` is a double-well (bistable) drift with two stable basins, U = +1 (healthy) and U = -1 (failed), and an unstable ridge at U = 0. A rise in energy (`dE/dt`) pushes U out of the healthy basin. `u` is the bounded control input.
- **(3) Integrated need.** I_U grows while U is below 1 (the system is not settled). It is reduced by energy and stress already being dealt with, and it leaks at lambda_I = 0.719. It is the engine's memory of unrelieved pressure.

- **(5)-(6) Stress.** A cubic potential (the "well in the basin"). S rolls downhill to its stable root. This is the slowest restoring force.
- **(7) Bath.** A damped harmonic oscillator (natural frequency omega_B = 1, quality Q_B = 2.5) driven by stress. It turns stress into a smooth, lagged "tide". It is the only second-order element.
- **(8) Audit.** The bath equation is re-checked numerically every step. The check is never fed back.

Integration: classical 4th-order Runge-Kutta, 10 micro-steps of 0.01 per macro step (`rk4_step`, `macro_step`). The C++ twin matches the Python to 3.6e-15 on 500 reference trajectories.

## 3. From telemetry to state (omnicompass/adapter.py, `observe_vector`, `assimilate`)

Each decision the telemetry is normalised to nine observations: q (queue), load, power, thermal, network, drift, stale, security, conflict. An observed state is computed from them:

```
E_obs   = 0.25 q + 0.18 max(0, load - 0.85) + 0.16 power + 0.13 thermal + 0.10 network + 0.08 drift + 0.
    ↪ 06 conflict + 0.04 stale
U_obs   = 1 - (0.27 q + 0.18 power + 0.16 thermal + 0.12 network + 0.12 drift + 0.10 conflict + 0.05 sta
    ↪ le)
S_obs   = 0.36 thermal + 0.28 power + 0.18 network + 0.10 security + 0.08 stale - 0.20 q
B_obs   = 0.52 power + 0.26 thermal + 0.22 drift
I_obs   = q + drift + conflict
beta_int = 0.58 q + 0.42 max(0, load - 0.75)        beta_ext = 0.33 power + 0.27 thermal + 0.20 network
    ↪ + 0.12 stale + 0.08 security
```

The state is blended toward the observed state with weight a = 0.339: `x <- (1 - a) x + a x_obs`. It is then evolved by one macro step (0.1 engine time units) of equations (1)-(7).

## 4. From state to action (the allocation law, `Governor.step`)

```
push     = u(x) / 25,  u(x) = clip(-f_U(x) + 12 (1 - U), -25, 25)        equation (2)'s controller, eval
    ↪ uated, not applied
rho*     = clamp(rho0 - kI I_U - kE E, rho_min, rho0)                      target utilisation (the HPA t
    ↪ arget)
n_req    = ceil(n load / rho*) + ceil(kq q n)                               machines wanted
release  only while push &lt;= push_release (the engine reports convergence), after band and dwell
cap      = clamp(cap_now load (1 + margin), cap_min, 1) when calm and U &gt;= U_gate; 1 under backlog;
           always &lt;= 1 - cap_gain max(0, B - B_cap)                          the bath limits the power ca
    ↪ p
change   permitted only while U &gt;= U_gate and no security block
rollback authorised if S &gt;= S_rollback, U is falling, or a security block is seen
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

## 5. Which equation drives which muscle

| Muscle | Engine quantity used | Where |
|---|---|---|
| HPA target (pods) | rho* from I_U and E, equations (1) and (3) | adapter `Governor.step`; live `omni_controller/controller.py` |
| Node pool (machines) | n_req from load, queue and rho*; release gated by push from equation (2) | same |
| Power cap (pod CPU limits, CPU frequency, GPU limit) | cap from U gate and bath B, equation (7) | same; `omni_controller/muscles.py` |
| Rollout guard | change_permitted (U), rollback (S, dU) | `muscles.py` |
| Security hold | security observation raises S and blocks expansion | adapter, shield |
| Speed-first law | rho* for pods; push gates machine release | `omnicompass/speed.py` |
| Right-sizing | rho* sizes CPU; E widens the memory margin; push gates scale-in | `omnilab/rightsize.py` |
| Cold start | rho* sizes instances; I_U sets how long to keep warm | `omnilab/coldstart.py` |
| GPU packing | I_U pre-warms nodes; push gates power-off | `omnilab/gpupack.py` |
| Training power | E tightens the applied ramp limit under sustained grid stress | `omnilab/powersmooth.py` |
| GPU health | S and E shorten the checkpoint interval | `omnilab/health.py` |
| Cooling | E widens the inlet-temperature margin | `omnilab/cooling.py` |
| Inference | the full allocation law (node_delta) sizes replicas | `omnilab/inference.py` |
| Agent containment | I_U throttles; the security observation contains | `omnilab/containment.py` |
| VM energy | I_U and E raise the learning rate when attribution goes stale | `omnilab/vmenergy.py` |

## 6. Measured: what the equations actually do inside the governor

Measured on a real observation stream (fleet plant, web services, fleet mode, 360 decisions):

| Mode (eigenvalue of the Jacobian at the operating point) | Time constant | In decisions |
|---|---|---|
| -7.90 | 0.13 | 1.3 |
| -3.10 | 0.32 | 3.2 |
| -0.72 | 1.39 | 13.9 |
| -0.20 +/- 0.98i (the bath) | 5.0, period 6.4 | 50, rings every 64 decisions |
| -0.17 | 6.0 | 60 |

- **Every mode is stable** (all real parts negative). The operating point is a stable equilibrium inside the healthy basin: E = 0.095, U = 0.976, S = 0.315, B = 0.10. This is the basin keeping everything in, stated as mathematics.
- **Per decision, the equations and the observation move the state about equally.** Across all six variables the equations contribute 45-50% of each step's movement and the observation blend the rest.
- **Equation (2)'s control input u is not applied during governance.** The engine evolves open-loop (u = 0). The control law is evaluated only to produce `push`, which decides when capacity may be released.
- **Compute:** about 95 microseconds per decision in Python, and about 2.9 microseconds in the C++ twin.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## 7. What this explains, and what it points to

- **Why "engine off" scores close to "engine on" in the muscle benchmarks.** With the equations frozen, the observation blend still moves the state. Both versions are then low-pass filters of the same telemetry. The engine's fast modes (1-3 decisions) behave almost like the blend itself. Its distinct contribution is its slow memory: I_U (14 decisions), S (60) and the bath (period 64 decisions). The muscle mappings mostly read E and rho*, which live on the fast modes.
- **The engine clock is not matched to the workload's rhythm.** One decision advances engine time by 0.1. At 60-second decisions the bath rings every 64 minutes, but web demand has a 24-hour rhythm and training power a 2-second one. This is a wiring choice in the adapter (engine time per decision), not a property of the equations.
- **The controller in equation (2) is unused as an actuator.** Wiring its output u (or U's trajectory under u) directly to the actuators is the unexplored way to "let the math do the work".

Three wiring experiments follow directly. None changes a single equation:

1. **Clock matching.** Set engine time per decision so that the bath's natural period equals the workload's rhythm. The oscillator in (7) then carries the cycle and can anticipate it.
2. **Slow-mode actuation.** Drive machines from I_U, S and B (the memory modes) rather than E (a fast mode).
3. **Closed-loop actuation.** Apply equation (2)'s u and use its magnitude, not only its sign, as the actuator command.

Each is testable in the existing harness against every platform in the league (tuning/league.py), and will be tested there.

## The pedals: idle, gas, brake, reset, and the kill switch

Omni-Compass drives a system the way a self-driving car drives itself: it feels, gauges and adjusts, with nobody in the seat. The words below are used the same way everywhere in this repository.

**Two names only. native** is the system as it runs on its own (Kubernetes and its autoscalers, the card's firmware, CityLearn's controller). **omni** is Omni-Compass on top of native: the same native system, with Omni-Compass governing it. Omni-Compass never runs instead of native, only on top of it, so every comparison is native against omni.

**The modes.** Off (manual): Omni-Compass not driving, native runs alone. Watching: Omni-Compass reads every gauge and logs what it would do, and writes nothing. Autopilot: the pedals below. Cruise and the emergency brake: for work that comes as a pile.

| Term | Meaning | In the code |
|---|---|---|
| **Idle** | No foot on the gas or the brake: calm traffic, the engine running at its floor (two machines in service, ready) | `MIN_NODES`, the floor of the staging law (8.8) |
| **Gas** | Ramping up: traffic climbs and capacity is added at once, never held back; the whole cluster can take work the instant it is needed | (8.3); a lower HPA target is never limited |
| **Brake** | Ramping down: traffic falls and capacity is eased off a step at a time, never below idle | (8.4)-(8.5); coasting, rule 6 |
| **Reset** | The brake held to the floor: every setting the governor ever wrote is handed back to where native had it, read back, and the record removed; the next run starts fresh. Every benchmark arm ends with one and checks it | the reset file (`--kill-file`, a name kept for compatibility), `Controller.restore` |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

| Term | Meaning | In the code |
|---|---|---|
| **Cruise** | A pile of work waiting for a place: every machine in service, held there without second-guessing, until the line has been empty for two decisions | rule 7, `--cruise-after` |
| **Emergency brake** | The work is done and demand is at zero (the service at its least pods, wanting no more, at half its target or less): straight to idle (the floor) in one move, every safety check still holding | rule 8, `--brake-demand`, amendment 9 |
| **Kill switch** | Security only: one switch in a human hand that turns Omni-Compass's governing off across the whole system at once, for anything rogue or anyone trying to drive a system through Omni-Compass's brain | `omnicompass/master.py`, `tools/omni_switch.py off` |

The fuel cut-off point (the floor) is a number the operator sets; the brake never takes the system below it, however hard it is pressed. Earlier dated records, and the sealed original engine files, call the reset a "kill switch"; they are kept word for word.

## 8. The staging law: machines on and off in order, with demand that moves

Demand climbs, spikes, eases part way, climbs again and settles to idle. The staging law decides how many machines are in service at every decision, and which ones, so that capacity follows demand up at once and comes back down only as far as it safely can. It is the compass law (`omnicompass/compass_law.py`) applied to the machine pool (`omni_controller/controller.py`), the release gate (`omnicompass/nervous_system.py node_release_gate`), the verdict (`omnicompass/verdict.py`) and the actuator (`scripts/kind_nodepool.sh`). Every symbol below is a line of that code.

**Position and force.** The service reading r (the 95th-percentile response time, or the pressure that stands for it) is placed in its band [lo, hi], where hi is the response line:

```
p = (r - lo) / (hi - lo),      v = s v + (1 - s) (p - p_prev) / dt                      (8.1)
F = A                                  if p &gt;= 1 - c        (fail up, past the wall)
F = A tanh( (kp (p - p0) + kd v) / A )      otherwise; F = 0 if F &lt; 0 while v &gt; 0 and p &gt; p0   (8.2)
```

with the band's center p0 = 0.5, cushion c = 0.05, authority A = 1, kp = 1 and kd = max(0, 2 sqrt(kp tau / dt) - 1) dt (critical damping for a muscle that answers in tau seconds). F > 0 asks for capacity, F < 0 offers it back.

**How many machines.** With n machines in service now, n_nat the count native ran, m the floor (two by default, `MIN_NODES`) and M the machines that exist (every machine usable; an operator may hold whole machines back with `NODE_CUSHION`, off by default), every count stays in m <= n' <= M. The protection is not a machine sitting out: it is the band inside every machine. Capacity is added at 95% of the response line, before the line is reached (8.2-8.3), and a machine goes back only if the ones left still run at or under the engine's utilisation target rho (8.5): a margin spread over all of them.

```
up      n' = min(M, max(m, n + 1))                          if p &gt;= 1 - c                    (8.3)
down    n' = max(m, n - 1)                                  if F &lt; R and p &lt; p0 and the verdict
                                                            allows n_nat - (n - 1) and the gate G ho
    ↪ lds  (8.4)
hold    n' = n                                              otherwise
```

with the release threshold R = -0.2. Up is immediate and is never gated. Down is one machine per decision, never below the floor, and only when every term of the release gate holds:

```
G = [n &gt; 1] and [nothing pending] and [pods not scaling up] and [no breach now] and
    [ used / ((n - 1) x per_node) &lt;= rho ] and [contraction authority] and [every sense live] and
    [the last machine command landed]                                                    (8.5)
```



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

rho is the engine's utilisation target: after a machine goes back, what remains runs at or under rho. The verdict (section 7 of the GPU preregistration, `omnicompass/verdict.py`) allows a machine count below native only after a paired trial showed the service at that count at most 2% worse than at the count before; that 2% is where the trigger sits, inside the engine.

**This is hysteresis by construction.** The up condition (p >= 0.95) and the down condition (p < 0.5 with F < -0.2) are far apart, so a demand that hovers never makes machines flap; a spike crosses the wall and adds a machine at once; a dip has to be deep, calm and sustained before one goes back, and then only one per decision. Between the two, the count holds.

**Which machines.** The actuator orders the pool:

```
idle   the open machine with the fewest serving pods, then the fewest pods, goes first;
       it is marked prefer-not (PreferNoSchedule) and its pods first to go (pod-deletion-cost), so the
       autoscaler's own scale-down empties whole machines; no pod is moved, evicted or restarted   (8.6)
wake   the idle machine still carrying the most work goes first (it is warm); the mark comes off and it
    ↪ is in
       service at once, with no boot                                                              (8.7)
floor  two machines always in service (MIN_NODES, the operator may set more), ready for a spike  (8.8)
margin  every machine usable; the cushion is the band inside each one (the 5% before the line, and rho),
    ↪  not a
        whole machine held out                                                                    (8.9)
```

An idled machine stays powered and Ready at its floor (park_frac x idle power), never off: a pod that finds the open machines full lands on it at once, so no request ever waits on a machine Omni-Compass idled. Where a node autoscaler underneath really removes an empty machine (Azure's cluster autoscaler, Karpenter), that same order hands it the emptiest machine first, and the bill falls with the machine.

**The same law on every discrete unit.** Machines are one case. In the muscle models (`realms/compass_arm.py release_safe`), a cooling plant's chillers and a compute pool's machines follow the same law: one unit back only when the units left cover the recent peak with headroom (RELEASE_MARGIN 0.6 for machines, 0.8 of a unit's capacity for chillers), one at a time, and back at once when the wall is reached.

**What is proved, measured and open.** The order (8.6)-(8.8) is checked through the real actuator by `tests/test_staging_order.py` (the emptiest idles first, the warmest wakes first, two always in service, every machine usable, no pod moved). The gate (8.5) is checked by `tests/test_node_release_gate.py`. On real Kubernetes the law gave the same work 55-65% faster on 29-36% fewer machines (sets 22-27) and 48% more work on the same machines (the capacity test); its run under demand that wanders (up, spike, part way down, back up, idle) is in progress. Open: an order across different kinds of unit (a battery before a generator, a CPU before a GPU) is not wired; each pool is ordered within itself.

## 9. The state each muscle carries, and the modes as a state machine

**9.1 The compass on a muscle is a dynamical system, so its state must persist.** On muscle i the compass carries z_i = (p_i, v_i, x_i): the position of the service in its band, its velocity, and the knob's continuous value. At each decision k, with reading r_k:

```
p_k = (r_k - lo) / (hi - lo),   v_k = s v_{k-1} + (1 - s)(p_k - p_{k-1}) / dt,   F_k = F(p_k, v_k) by (8
    ↪ .2)    (9.1)
x_{k+1} = clamp( x_k + sigma g(F_k) F_k (x_hi - x_lo), x_lo, x_hi ),   g = UP if F_k &gt; 0 else DOWN
    ↪     (9.2)
```

with x_0 the native value and sigma = +1 when a higher value is more capacity. Two properties depend on z persisting: (a) the knob integrates the force, x_k - x_0 = sum_j sigma g F_j span (until the cover binds), so a calm

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

muscle eases step by step toward its cover; and (b) the damping term kd v_k needs p_{k-1}. If z were made fresh at every decision (p_{k-1} = p_k, v = 0, x_k = x_0), (9.2) would collapse to x_{k+1} = clamp(x_0 + sigma g F(p_k, 0) span): a memoryless map of the present reading, at most one step from native, with no damping. That is a different, weaker law. A renamed attribute did exactly that on 2026-10-05; `tests/test_compass_arm.py` now fails if the compass is ever made twice on one muscle.

**9.2 The modes.** The live controller is in exactly one mode at each decision. With q_k the pods waiting for a place, u_k the scaling-up flag, c the cruise count (`--cruise-after`, 2), M the machine ceiling and m the floor:

```
cruise on      q_j &gt; 0 for the last c decisions                                          (9.3)
cruise off     q_j = 0 and not u_j for the last c decisions                               (9.4)
in cruise      n' = M; the service's autoscaler target held at the operator's own; the floor step
               makes no read and no write                                                 (9.5)
emergency brake (not cruising) q = 0, no breach, not u, every sensed service idle or at demand
               &lt;= b, the gate G (8.5) holds, every sense live, the last command landed:
               n' = max(m, smallest n carrying the CPU in use at rho)                     (9.6)
idle (a service at its floor)   I(h) := current = min = desired, and u_h &lt;= target_h / 2   (9.7)
gas / brake / idle             (8.3) / (8.4)-(8.5) / (8.8)
```

**9.3 Why cruise steps back (9.5).** In cruise every machine is already in service, n = M, so the floor step's only action (raise n to the floor the waiting pods need) is empty: floor <= M = n. Its reads (every pod, every node, node metrics, every HPA, each five seconds) carry no value of information, so removing them cannot change any action; they only cost CPU and API-server time on the machines the queue is using. The service's response time is also confounded in cruise: with a queue of other work holding the machines, a request waits on contended CPU, R = S / (1 - u_host), not on the service's own pods. Reading that R as the service's own queue and adding pods would answer the wrong cause. So in cruise Omni holds the machines and leaves the service to its own autoscaler. `tests/test_compass_controller.py` checks that the floor step makes no API read during cruise; the controller before this rule made the full set of reads every five seconds.

**9.4 Why idle is read from the autoscaler's floor (9.7).** A served service never reads zero: a health probe or a client's keep-alive keeps one pod at a few percent of its request. In the batch test, one pod at about 10% of its request read as demand 0.10, above b = 0.05, and the brake waited 15 minutes after the queue emptied. The autoscaler's own floor is the observable of "no demand beyond the minimum": the pods it keeps are its minimum, it wants no more, and they run under half the target. In the five frozen-engine web tests the service was never at its floor (0 of 5,102 readings), so (9.7) changes nothing there.

**9.5 Reporting resolution.** A paired change smaller than one part in a million of the value (|mean difference| <= 1e-6 |native|) is read "same", the number still shown. Below that, a difference is rounding in floating point and in the models' accumulators, not behaviour; a deterministic run gives a zero-width interval around it, which would otherwise read as proven.

**9.6 A lever is moved only where moving it can pay.** The compass decides how far and how fast. Whether a lever may move at all is a property of the muscle's physics. Four cases, each derived from the muscle's own model and each its own test:

- *Cooling power.* A zone's heat must leave it: over a run, the heat removed equals the heat made plus what the room's mass stores. A cap on cooling power q_max cannot remove less heat. It only lets the temperature drift above setpoint while the PI command grows, and the staging rule, units = ceil(command / (0.8 q_unit)), turns that command into more units and more fan power, p = q_c / COP + units x p_unit. So a lower cap never saves energy, and the lever stays native. (The setpoint, which changes the COP, and the units, which change fan power at the same heat, remain levers.)


© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- *A battery's reserve.* Energy D drawn from the cells below the operator's reserve saves D sqrt(eta) of grid import, and the deficit is bought back at D / sqrt(eta). The net, D (1 - eta) / sqrt(eta) (10.5% of D at eta = 0.90), is always positive. Spending the reserve is never an energy saving; it is the price of keeping the draw under the connection's limit. So the reserve is spent only at the wall (p >= 0.95) and only when the battery can cover the excess (0 < imp - p_lim <= p_batt). Otherwise the operator's reserve holds, and no round trip is paid for nothing.
- *A backup reserve (a UPS).* It is held for an outage, which the model does not contain, so spending it for a peak trades away the reason it exists. Never a lever.
- *A compute pool's power cap.* With throughput mu(c) proportional to c^eps and the autoscaler holding utilisation near its target u*, the energy per unit of work is e(c) = (p_idle / u* + p_dyn c) / mu(c). At c = 1, de/dc > 0 (a lower cap saves) if and only if

```
eps &lt; p_dyn / (p_idle / u* + p_dyn)                                    (9
  ↪ .8)
```

At eps = 0.4 the cap saves on CPU hosts (threshold 0.62), GPUs (0.74), databases (0.51) and radio units (0.54). It costs energy where idle power dominates: a quantum computer's cryostat (0.19), a network switch (0.32), storage (0.26). There, running slower keeps the machines on longer: race to idle. In a deployment p_idle, p_dyn and the speed curve come from the machine's own specification.

- *A motion axis's effort cap.* It lowers the copper loss of a full-acceleration move, (J a_max / kt)^2 R, and stretches the move, which keeps paying the standing draw p_idle + b v_max^2. It moves only where the first exceeds the second: a reaction wheel (32 W against 5 W), not EV traction, a flight axis or a robot joint.
- *A machine released from a node pool.* A machine boots in minutes, and a burst that arrives in the meantime is served late. So a machine goes back only when the machines left cover the recent peak at 0.3 of the pool's own release level, the largest margin at which no pool is later than native (`results/realms/RELEASE_MARGIN_SWEEP.md`). This is the verdict of the live controller (section 8) in model form.

What a lever buys is labelled for what it is. Less work per energy, with no work lost and the time over the line proven lower, is SERVICE IMPROVEMENT WITH ENERGY TRADEOFF, the mirror of ENERGY IMPROVEMENT WITH SERVICE TRADEOFF. Without that proof it is WORSE.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

CHAPTER 16

# The Engines and Their Audit

## Which engine is which

There is **one** engine. Every other engine-looking file in this repository is either its required C++ twin or a frozen copy of where it came from. Nothing else runs.

| File | Role | Runs? |
|---|---|---|
| `omnicompass/core.py` | **The engine** (`symmetric_verified`): six states, eight lines, RK4 with microsteps. Everything Omni does goes through it. | yes |
| `cpp/src/core.cpp`, `cpp/include/omnicompass/core.hpp` | The same engine in C++. Required: `verify.py` checks it against the Python on every fixture. | yes (twin) |
| `docs/handoff/mathematics/omni_compass_engine_source_c527df2d.py` | The original source as handed over (sha256 `c527df2d…`). Kept unchanged as proof of origin and as the parameter authority. | no (frozen) |
| `reference/omni_compass_reference_engine.py` | The same original with comments stripped (`tools/strip_reference.py`); same program fingerprint (`reference/PROVENANCE.json`). | no (frozen) |

Fingerprints of the running engine, its parts and its twin: `results/MECHANISM_IDENTITY.json` and `RELEASE_MANIFEST.json`. `verify.py` fails if any of them changes without a new seal.

### *Engines from outside packages, not adopted*

Copies of this repository passed around as zips (the XPASS packages) carry changes that are **not** in the engine here:

- `batch_claim1.py` (in those zips, not in this tree): a vectorised variant that applies u during the RK4 step (CLAIM1). No C++ twin; not verified against the 500 fixtures.
- an adapter that defaults to CLAIM1, takes the target sign from the starting state and adds a disruption budget. Its C++ governor was only partly ported, which is where the reported Python/C++ mismatches came from.

None of these enter before the GPU counted run run (the engine is frozen for it, `docs/GPU_PREREGISTRATION.md`). Any of them can be adopted afterwards only as a full change: Python and C++ together, the parity tests and `verify.py` green, and a new mechanism id.

### *The rule from here*

One engine, one twin, two frozen originals. A new version **replaces** the old one in place, with a new seal; it is never added beside it. Older states are in git history and `docs/HISTORY.md`, not in extra files.

## My engine, line by line: the source against what runs in your cluster

**The source.** `docs/handoff/mathematics/omni_compass_engine_source_c527df2d.py` (SHA-256 c527df2d…, 31,348 lines). Every mechanism below is named by its function there, followed by where it runs live and how it is checked on every `python verify.py`.

| Mechanism | In the source | Where it runs | Checked by | Status |
|---|---|---|---|---|
| Six states E, U, I_U, S, B, Ḃ and equations (1)–(7) | `oc_derivatives`, `rk4_step` | `omnicompass/core.py` | `tests/test_core_parity.py`: 20,000 random states, max difference ≤ 1e-12 | exact |
| Spinor closure, 720°: drive × cos(½ ω_B t) | `v_eff`, `spinor_closure_factor` | `core.v_eff` | same parity test | exact |
| Axle rest S* from equation (6) | `stable_S_equilibrium` | `omnicompass/storage.py`, `compass.axle` | parity; `tests/test_compass.py` | exact |
| Double well W(U), basins ±1 | `W_potential`, `W_gradient_flow` | `core.derivatives` | parity | exact |
| Regulation u = −f_U + K_P(σ − U), clipped at the authority | `hybrid_microstep_operator` | `core.control_command`; the governor's push | parity; 500 frozen trajectories (`fixtures/`), 0 mismatches | exact |
| Basin lock, conveyance, certification | `run_monte_carlo`, `macro_step` | `core.simulate` | 500 frozen trajectories, 0 mismatches | exact |
| Composite storage V = V_U + V_W + V_E + V_S + V_I + V_B, the ledger that closes the circle | `composite_practical_lyapunov_value`, `bath_lyapunov_matrix`, weights | `omnicompass/storage.py`, read by the compass every decision | parity on 5,000 states (≤ 1e-13); descends 648 → 8 over 60 regulated steps with no rise | exact |
| Human switch: a file or a variable stops every command | `_oc_runtime_stop_requested` | `--kill-file`, restore of every lever | `tests/test_failsafe.py`, `tests/test_muscles.py`, every live run's switch drill | exact |
| Cap doctrine C1: only a ceiling *above* the reference is admissible; a lower one buys energy by slowing work | `_ci_omni_decision`, certificate C1–C4 | `muscles.convey`: CPU limit never below the operator's, idle CPU conveyed to the work | `tests/test_convey.py` | exact |
| Sleep only in a certified empty interval | `_ci_omni_decision`, sleep channel | `scripts/kind_nodepool.sh`: a machine idles only once its work has left | `tests/test_active_nodes.py`, `tests/test_convey.py` | exact |
| Reference Kubernetes HPA rule | `oc_hpa_desired_replicas` | `cpp/` HPA, pod reflex | `tests/test_cpp_hpa_parity.py` | exact |

### *Where my live wiring reads the engine differently from the source's own embodiment*

**Evolution between decisions.** The source's compute embodiment advances the assimilated state *with* regulation locked on basin +1. My live governor advances it with u = 0 and reads the regulation command separately, as the push that gates every machine release.

- Both use the same equations.
- The difference is whether the regulation is applied to the state or only read from it.
- The pre-registered, frozen results were measured with u = 0. I keep that setting, and I report the difference here rather than change a frozen law mid-measurement.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

PART THREE

# The Physics: the Compass and the Nervous System

*Push and pull, the band and its cushions, the physics of a processor, and the two-way wires that carry the force from the brain to every muscle and back.*

CHAPTER 17

# The Compass: Push, Pull and the Two Forces

The compass (`omnicompass/compass_law.py`) is the law that carries the engine's push and pull to every muscle.

**Two bands.**

- **The cover**, a knob's hard range (the device's or the operator's lowest and highest setting). Every write is clipped to it. Nothing Omni-Compass computes can set a knob outside it.
- **The compass**, the service reading as a position from 0 (calm) to 1 (the service line). Omni-Compass pulls that position to the bottom of the compass, the middle (0.5 by default; an operator may set it lower for extra margin). The walls run from 0.05 to 0.95; the last 5% on each side, 10% in all, is cushion.

**The force.** F = A tanh((K_P (p - center) + K_D v) / A), where p is the position, v its rate of change, A the authority.

- **Pull:** K_P (p - center), gentle near the bottom, stronger up the walls.
- **Push:** K_D v, the force that meets whatever is shoving the position, and the friction that stops it sloshing. With K_D at or above critical damping the position glides to the middle and stops, with no overshoot and no ringing. Overshoot is wasted energy: force spent going the wrong way and spent again coming back.
- **Smooth, never a hammer:** tanh bends the force over into its maximum instead of slamming into a wall.
- **Fail up:** past the 0.95 wall the up side goes to its full force at once and the down side may not act until the position is back inside the compass.

**Two forces: antagonist pairs.** Like the muscles of an arm, every plug has an up side (adds capacity, power, cooling, speed) and a down side (takes it back), each with its own gain, because adding and taking back do not cost the same: a new machine takes minutes to boot; giving one back is instant. Where a machine has two wires, each side gets its own lever:

| Muscle | Up force | Down force |
|---|---|---|
| GPU card | clock ceiling raised | power limit lowered (the lid) |
| Kubernetes | scale out | scale in |
| Cooling | chiller on, colder setpoint | warmer setpoint, unit released |
| Battery | discharge | charge, reserve held |
| Vehicle, joint | motor effort | braking, regeneration |

**Why the GPU needs both wires.** A card's firmware boosts its clock as high as it may while there is work, and its power limiter knocks the clock back each time the draw crosses the limit; on a busy card this happens many times a second, at the top of the clock range where each extra step of speed costs the most watts. With one wire (the power limit) a governor can only move the wall the boost pushes against. With two wires, the clock ceiling sets how high the boost may climb and the power limit becomes a lid that rarely needs to act: the card runs at the bottom of its compass instead of fighting itself at the top.

**What the card taught us.** Two rules make the difference between saving energy and spending the customer's time. First, *race while work waits*: when the card is saturated, both wires go to full at once, so a burst is always served at full speed; the compass paces only the slack between bursts. Second, *never slower than the card on its own*: the governor learns, from the card's own meter, the clock and the draw the card reaches by itself while busy, and never sets the clock ceiling or the lid under them. Without these rules the first real card saved 3.5% of its energy and made the slowest answers 58.5% slower; with them, the modelled card saves energy with the slowest answers at native speed or faster (Chapter 43, and `docs/GPU_PREREGISTRATION.md`, amendments 6 and 7).

## The same law on every stack, and what differs

The law is written once (`CompassLaw`), and three declared numbers fit it to a muscle: the **decision period** *dt* (how often the governor decides), the muscle's **response time** tau (how long the service reading takes to follow the lever), and the **cover**. From tau and *dt* the law derives its own damping gain, K_D = (2 sqrt(K_P tau / dt) − 1) dt, which is critical damping for a first-order muscle under a proportional-derivative pull: the position glides to the center and stops. Nothing else is tuned per stack. The table below is the complete list of the compass's settings on every stack in this manual, so a referee can see that the "gains" differ only through the declared physics:

| Stack | Reading (the wire in) | Line | Center | Decision period | Response time | Cover of the knob |
|---|---|---|---|---|---|---|
| Kubernetes, the HPA target | the mean response time of the latency window | the operator's SLO (500 ms) | 0.4 | 60 s | the pods' start time | [0.6 × the operator's target, the operator's target] |
| Kubernetes, the machines | the same, through the release gate and the verdict | the same | 0.4 | 60 s | a machine's wake | [the floor of 2, every machine] |
| PgBouncer, the pool size | time in the server plus time waiting for one, per transaction | 50 ms | 0.4 | 1 s | 2 s | [2, PostgreSQL's limit minus 10] |
| Kafka, the consumer count | the group's own end-to-end latency, mean of the last second | 500 ms | 0.4 | 1 s | 3 s (a rebalance) | [1, the topic's partitions] |
| Redis, the memory ceiling | the application's request latency, mean of the last second | 2 ms | 0.4 | 1 s | 2 s | [16 MB, 512 MB] |
| MongoDB, the storage-engine cache | the server's own mean read latency, last second | 1 ms (set on the tuning workload's smoke run, from 2 ms) | 0.4 | 1 s | 2 s | [256 MB, 2,048 MB] |
| MySQL, the InnoDB buffer pool | the server's own mean statement latency, last second | 0.6 ms a statement (set on the tuning workload's smoke runs, from 1 ms) | 0.4 | 1 s | 2 s | [128 MB, 2,048 MB], in 128 MB chunks |
| A drone's cruise override | the drone's tracking error | 0.25 m, the declared safe error | 0.5 | one control tick (48 Hz) | 1 s | [1.0, 2.0] × the planner's cruise |
| A substation's tap | the bus voltage in its band | the band's edge | 0.5 | one solve | a week of evidence before a step down | the tap's own range, one tap per move |
| A robot axis's speed | the axis's tracking error against its takt | the declared error | 0.5 | one control step | the axis's response | the axis's own limits |
| A GPU's clock ceiling and lid | the mean response time of the last 5 s | the SLO | 0.4 | 2 s | the card's response | [the card's own busy clock, its top clock]; [the busy draw + 10%, the start limit] |

The center is 0.4 wherever the reading is a service time (the "service profile" of the Kubernetes adapter, so that the service sits a little below the middle of its band with margin for a burst) and 0.5 where the reading is a physical error. A **cushion** of ±0.05 in force moves nothing, so a reading that sits at the center with the usual jitter does not make the knob chatter. These are the same numbers in every preregistration and every table; none was changed after a result was seen.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

## Direction rules: how a force becomes a move

The force is a number between −1 and +1. Each stack turns it into a move on its knob by a direction rule declared in its preregistration, always of the same shape: a positive force adds capacity in proportion to the force (ceil(force / 0.10) notches, so full force adds ten notches at once, capped by the cover), a negative force takes one notch back per decision, and only when the resource it would take is demonstrably idle (an idle consumer that read nothing in the last second, an idle server the pooler itself shows idle, a cache with nothing evicted in the last second) and only after a **dwell** of a few seconds since the last change, so a knob that was just moved is not moved back by its own transient. At 95% of the line the knob **fails up**: every consumer the topic can use, the pooler's own setting handed back, a quarter of the cache's cover added at once, the planner's own cruise, the card's own clocks. The up side is fast and the down side is slow and gated because adding and taking back do not cost the same, and because a gain bought by taking something back must be able to prove itself before it is kept.

Written out stack by stack, from the preregistrations, the rule reads as follows; the unit, the cover and the fail-up move are each the stack's own, and the shape is the same in every row.

| Stack | The knob and its unit | The cover | One notch back, when idle | Fail up at 95% of the line |
|---|---|---|---|---|
| Kubernetes (the cluster) | worker machines in service | the floor of two to every machine the operator has | one machine a decision, when the remaining machines would carry the load far under the wall | one machine up at once |
| PostgreSQL behind PgBouncer | the pooler's pool size, server connections | [2, 90]: the floor of two, never within ten of PostgreSQL's 100 | one connection a decision, when the pooler itself shows a server idle | the pooler's own setting (20) handed back at once |
| Apache Kafka | consumers in the group | one to the topic's partitions (8) | one consumer a decision, when a consumer read nothing in the last second | every consumer the topic can use |
| Redis | the memory ceiling, MB | [16, 512] MB | one notch a decision, when calm and nothing was evicted in the last second | a quarter of the cover added at once |
| MongoDB (WiredTiger) | the cache size, 64 MB notches | [256, 2,048] MB | one notch a decision, when calm and nothing was evicted in the last second | a quarter of the cover (448 MB) added at once while full |
| MySQL (InnoDB) | the buffer pool, 128 MB chunks | [128, 2,048] MB | one chunk a decision, when the miss share is under 1% | four chunks at once while the pool is full |

Two things in the table deserve a reader's attention. The "when idle" column is never a guess from the knob's own value: it is a reading from the stack's own counters (the pooler's idle servers, the consumer's last fetch, the cache's eviction count, the pool's disk reads against its requests), so the law gives back only what the stack itself reports it is not using. And the fail-up column is the stack's own fastest safe move, not a multiple of the notch; where the stack has a controller of its own (the pooler, the planner, the card's firmware), fail up means handing the knob back to it, which is the one move that can never be worse than native by construction.

## The do-no-harm gates inside the law

Three gates sit inside the law itself, each a consequence of a loss seen on a tuning case and each disclosed in the preregistration that carries it:

- **The verdict** (section 1.2): a slow knob moves only where a paired trial on the muscle shows no more than 2% cost.
- **The slack gate on speed knobs** (Omni v3): a motion axis that is busy more than half the time at full speed keeps its speed native, because spending its slack as speed when it has none would only cost energy. This gate



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

is what turned the v2 "service tradeoff" on the Physics realm and the tower into "superior within guardrails" on v3, and it is the one rule that distinguishes v3 from v2.

- **The full-cache gate** (Redis): a miss in a cache with room to spare is a cold miss that no ceiling can mend, so the ceiling grows only while the cache is at 90% of it or more. On the tuning workload without that gate the law raised the ceiling to 500 MB while 21 MB were in use; with it, the ceiling follows the working set.

The same gate recurs, in the form each store's own gauges allow, on every memory knob that followed, and each form was fixed on the tuning workload and disclosed before the counted runs:

- **MongoDB's storage-engine cache** grows by a notch only while the cache is full and the server's own read latency is above the centre of its band, and gives a notch back only while calm and nothing has been evicted in the last second (`docs/YCSB_PREREGISTRATION.md`). The smoke runs showed the line itself had to be set from the server's own readings (0.15 to 0.46 ms under a 2 ms line left the law nothing to do), and it was set to 1 ms before the counted runs.
- **MySQL's buffer pool** grows by a chunk only while the pool is at least 90% full of data pages and the server's own statement latency is above the centre, and gives a chunk back only while the **miss share**, pages read from disk as a share of the pool's requests in the last second, is under 1% (`docs/MYSQL_PREREGISTRATION.md`). The earlier rule, "no disk reads in the last second", was never satisfied on the tuning workload: InnoDB keeps stale pages resident, so a pool once filled reads full for ever, and a few new-page reads a second never stop even when the working set fits, so the pool the law had grown was never given back (877 MB held on average against native's 512 on that smoke run); the smoke record says so, and the miss-share rule replaced it before run A.

What the three forms share is the reason: a memory knob has no slack to spend unless the memory is both full and being missed, and memory given back while it is being missed is not a saving but a cost moved to the disk. The gate is written in each harness in the store's own units and checked by that harness's own tests against a fake server, so that a reader can see in the preregistration exactly which reading opens the gate and which closes it.

## The profiles and the pedals

The compass has two **profiles**, service (the default) and batch, which differ only in what the pedals do with a pile of work: in the service profile the law paces the slack; in the batch profile cruise puts every machine in service while work waits for a place and the emergency brake takes them straight to the floor when the queue is done (section 1.3). Both are written out in the Kubernetes preregistration (`docs/K8S_COMPASS_PREREGISTRATION.md`, rules 7 and 8, amendment 9), both ran in the batch test, and the result (Chapter 43) is machines 19% to 23% fewer over the whole window and 29% to 35% fewer after the queue finished, with the queue itself finishing no later beyond the noise.

## The reflex rule: one body, one brain

Everything in this chapter so far is the force: where a reading sits in its band, how hard the compass pushes, in which direction, and which gates stand in the way. This section is the rule that decides when the force may become a new setting. The founder's order of 10 October 2026 states it in one line: what the brain sends to a muscle has to coincide with what the muscle sends to itself and where it is at. Made into rules, it reads:

1. **Signal out, the muscle moves, the brain feels the full reaction, then it decides.** A trial runs to its full measurement and is never ended by the calm it causes. Only the wall (a fail-up) or the engine's own time limit ends a trial early.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

2. **No forcing.** A spend is tried only when the muscle's own reading says there is something to buy (messages waiting, a cache missing, clients queued); a give-back only when the whole body is calm. The force is an urge, smooth, never a hammer.
3. **One body, one brain, one tick a second.** Everything is read first: every muscle's state, every service's speed and work, the host's processor, memory and energy where a meter exists. Then the brain decides. The cost it judges is the body's cost, the Omni index's own arithmetic over all the work, all the speed, all the machines and all the resources, never one muscle's.
4. **Hurt nothing anywhere.** On top of the body's cost, a guard for every part: a step is refused if any one service got worse beyond its cushion, even when the body's total improved.
5. **One trial at a time inside a body.** Two muscles moved together cannot be told apart, so the body grants one new trial at a time, to the muscle asking loudest whose own condition holds. Every other muscle keeps acting inside what it has already proven.
6. **Nothing is permanent.** Every allowed step is tried again on the recheck and pulled back when it stops paying; a muscle at native is asked again whenever its signal returns. Native is where the brain stands when it has not yet been shown a reason, never a verdict.
7. **The wall belongs to the body.** If any service reaches its line, the trial stops, that step is undone, and every force in the body turns to brake: the law's brake, not a reset.

The trial itself keeps four rules, each learned from a measured run (`docs/OMNI_V4_PLAN.md`, Chapter 8): samples count only once the muscle reports itself settled; a spend on a queue or a cache is judged by the muscle's own reading, with the body's cost taken at the settled state; the reference and the trial are compared at the same load; and where single notches read flat, one scout step to the far side of the cover decides whether there is a hill to climb at all.

**Which engine enforces what.** In Omni v3, the engine on main, the verdict tries a slow knob on the muscle itself before it moves it (section 1.2), and the live harnesses carry the same verdict around every live knob (section 9.4). Omni v4 is the engine written to enforce all seven rules on every wire by construction: one body file that every muscle's wire passes through, the trial rules inside the verdict, and no wire that can write around them. It is designed (`docs/OMNI_V4_PLAN.md`) and not yet built; when it is, it carries its own fingerprint, the package version follows it, and every result in this book is run again on it. The theory of the rule is the chapter "The Reflex Rule" in Part One of the book.


*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 18

# The Physics of a Processor

A GPU's firmware raises its clock one step at a time whenever there is work and room; each higher clock needs a higher voltage, and dynamic power rises with the clock times the square of the voltage, so the top of the clock range is where each extra step of speed costs the most watts. When the draw crosses the power limit, the firmware knocks the clock down several steps; with room again, it climbs again. On a busy card this repeats many times a second. The clock saws against the limit, the overshoots burn watts before the limiter catches them, and the card spends its time at the steepest part of its power curve. The power is the burner; the cooling removes the heat at its own pace; the limiter turns the burner down so the cooling can catch up, and the boost turns it back up.

The firmware does not aim at a balance point because it cannot see the job: it does not know whether the work is a response due in milliseconds or a batch due overnight, the right point moves with the workload, the room and the chip, and a limiter is simple to prove safe. The vendor default is set for maximum performance, and the electricity bill is the customer's.

Omni-Compass sits on the customer's side, where the job's service line is known. It holds two wires: the clock ceiling sets how high the boost may climb, and the power limit becomes a lid set just above what that ceiling draws. The boost stops at the bottom of the compass instead of slamming into the wall; the limiter rarely needs to act. Every watt kept off the chip is saved twice in a data center: once at the chip and again at the chillers that would have carried its heat.

CHAPTER 19

# The Two-Way Nervous System

Every muscle is wired both ways: a sensory wire in (its meters) and a motor wire out (its knob), with the read-back closing the loop. Between them sits the nervous system (`omnicompass/nervous_system.py`), which decides how much authority each organ has at each moment:

- **Expand is always allowed** (except under a security hold): adding capacity, power, cooling or protection never waits.
- **Contract needs calm and a clean service record:** giving anything back is allowed only while the organ is calm enough and service has been inside its line for the last three decisions; continuous organs give back at most the calm share of their surplus per decision; discrete organs one unit at a time, through a release gate.
- **Fail up:** while service is breached, the knob returns to native at once.
- **Blind means hold:** if a sense goes stale or unreadable, nothing is given back until it returns.

One brain reads every organ at once. Because one law sets every knob, no two muscles fight: when the GPU's watts turn to heat, the cooling knob already knows it is coming; when pods scale up, the power envelope is ready.

"Two-way" is a statement about wiring, and it is checked, not assumed. Each muscle's plug (Chapter 28) carries exactly one sensory wire and one motor wire, and the wire check that must pass before any write (section 8.4) proves that the wires move: a value written through the motor wire is taken by the device and read back as the value sent, and the snapshot is restored and read back at the end. No governor writes a knob whose wire check has not passed, because a knob that can be written but not read back cannot be handed back with proof. What the wire check does not prove, section 8.5 says plainly: that the governor is reading the right gauge for the knob; that is what the watching arm is for. The receipts of every run therefore carry, for every write, the value sent and the value the device reported, side by side, and a reader who wants to know whether the governor's hand was really on the stack can check the two columns rather than take the summary's word.

The four rules above are the whole of the nervous system's authority model, and they are asymmetric on purpose. Adding is cheap to undo and expensive to delay, so it is never gated beyond the security hold. Taking back is cheap to delay and expensive to get wrong, so it is gated three ways: by calm (the engine's own state must report that the body is settling), by the service record (the gauge must have been inside its line for the last three decisions, not merely this one), and by the release gate's reasons, which are written into the audit every time the gate holds (section 7.1). Sections 7.2 to 7.4 take the three ideas that make this safe at scale, the governor's sense of its own hand, the rule that a missing reading is a hold and not a guess, and the case for one brain over many, and give the evidence for each from the live runs.

## Authority, and the release gate

Authority is how far the nervous system lets an organ move in one decision. It is granted from the engine's state (section 4.3): an organ in a calm body with a clean service record may give back; an organ in a stressed body may only add. On Kubernetes the **release gate** (`omnicompass/nervous_system.py`, `node_release_gate`) lets one machine be given back only when every one of the following holds, and writes the first reason that fails into the audit so a reader can see why a machine was kept: the remaining machines would stay under 95% full; no pod is waiting for a place; the pods are not scaling up; the service is not breached; the organ's own pressure is under the release threshold; every sense is live; and the last command has landed (no new order goes on top of one the cluster has not yet carried out). One machine per decision is the rule in the slow, reversible direction; past the wall, one machine up at once.

**What the gate writes, in a real run.** The first repetition of the first kill-scenario run (`results/live/raw/run-37716845219/paired-1/`, the omni arm's `audit.jsonl`) holds 114 decisions and 42 writes. Its gate records read, in the governor's own words, "release permitted" eleven times, each with the utilisation the remaining machines would carry after the release (0.034 to 0.153, all far under the 95% wall); "node organ has no contraction authority" once, a calm cluster whose engine state had not yet earned the right to give back; and "latency breached now; node organ has no contraction authority; a sense is blind" once, three reasons at once in a decision whose latency sense read stale, every one of them a reason to hold. A reader can therefore see, decision by decision, not only what the governor did but why it declined to do the rest, and the first reason that failed is the one written, so that a machine kept in service always has a stated cause.

## Proprioception and the one-writer rule

The nervous system knows where its own levers are. Every write is read back from the device (the plug contract, section 8), and the audit records both the value sent and the value the device took. If a lever is found at a value Omni-Compass did not write, someone else owns it: the governor stops writing that lever and leaves it alone, because restoring over it would fight the new owner. This is **proprioception**: the governor's sense of its own hand. It is the reason a wrong installation leaves a signature in the receipts (section 8.5) rather than a silent error, and it is what the wire check proves before anything runs.

The rule begins with the **snapshot**. The governor reads every knob it may move once, at the start, before its first decision, and that reading is the operator's setting for the whole run: every reset, every hand-back and every kill returns the knob to the snapshot, never to a value computed later, and the receipts record the snapshot beside the final read-back so that a reader can check the two are equal. The rule continues with the **read-back** after every write: the plug sends a value, waits for the device to take it, and reads the device's own report of what it holds. Where the device takes time to apply a setting, the plug knows that and does not mistake the transition for a foreign hand. The clearest case is MySQL's buffer pool, which the server resizes in chunks over several seconds and reports as it goes: the plug reads the server's own resize status and treats a pool caught mid-resize as its own write in progress, not as another owner's value, and waits for the server to report completion before it judges the write (`tools/run_sysbench.py`, `BufferPool`). The same shape holds on the Kubernetes autoscaler's bounds, Redis's memory ceiling and MongoDB's cache size: the value is read from the device's own console, and only a value that the governor neither wrote nor is waiting on is read as someone else's.

What follows from a foreign value is deliberately conservative. The governor does not restore over it, because an operator who changed a knob by hand meant to, and a second controller that changed it is a conflict to be reported, not won. The lever is left where it was found, the audit records the value and the time, and the governor goes on governing the other levers. A reader of the receipts who finds such a record knows at once that two hands were on

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

the stack, and the paired run that contains it is read with that in mind.

## Blind means hold, and fail up

A sense that goes stale or unreadable is not read as calm. It is read as **blind**, and while any sense is blind nothing is given back; the knob holds, or, if the service was breached when the sense went blind, returns to native at once. In the fault test on real Kubernetes the response-time probe is deliberately paused for a minute in every arm; the audit shows the governor holding through the blind minute and resuming when the sense returns, and the table shows no row worse for it. The rule is simple to state and strict in effect: Omni-Compass never acts on a reading it does not have.

The rule was also tested without being arranged. In the two-hour robustness run on real Kubernetes (`results/live/V3_ROBUST_LONG.md`), the cluster's own API answered a handful of the governor's reads of the autoscaler with a server error, two or three times a run out of the 120 decisions a two-hour window expects. Each of those decisions is in the controller's log with the reason the API gave, each was held (no write, no hand-back, no guess), and the governor took its next decision on schedule when the API answered again. The preregistration had said in advance that such decisions would be shown with their reason and not judged, and that is how they appear in the table: 97.5% of the expected decisions valid, the failed ones counted and named, the knob handed back at the end of every run. A referee who wants to see the rule in the same rule written by the gate itself can find it in the kill test's receipts (section 7.1), where a decision whose latency sense read stale is held with "a sense is blind" among its reasons, and in the fault test's blind-probe window, where it was arranged on purpose. The point in every case is the same: a stale or missing reading produces a hold, never a move, and the hold is in the record.

## Why one brain

The alternative to one brain is a controller per layer, each tuned alone: an autoscaler that adds pods while a power cap is trying to hold the machines down, a cooling loop that chases heat the clock governor is about to remove. These controllers fight because none of them knows the others' intent. One law that sets every knob from one state cannot fight itself: the force on each lever is computed from the same reading of the same body at the same moment. This is also why the modelled organisms are run whole, at up to 1.7 million muscles on one clock (section 2.4): the question they answer is whether one brain stays coherent at that size, and the reading at every size is the same to the digit.

One brain also has to stay the same brain over time, and that is a measurable claim. The two-hour robustness run (section 16.8) measures the governor's own memory at the start and the end of each run and the time it takes to reach a decision in the last hour against the first. Over three runs the memory ratio was 1.05 to 1.07 and the decision-time ratio 1.07 to 1.20, both inside the limits set before the run (a leak would show as steady growth; a brain that slows as its records accumulate would show a ratio climbing toward the 1.5 limit), and the cluster's service under the governed arm was inside the noise of native in two of three runs. The 24-hour runs on three rented machines, started as this is written, ask the same two questions over twelve times the window. A single controller that keeps its memory and its speed over a day is the least a referee should ask of a brain that proposes to hold every knob at once; the figures are in the table, not asserted here.


*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 20

# The Nervous System in Detail

## The two-way nervous system (live controller)

**Source.** Manuscript Appendix J: "Nervous system. The signal layer: sensing interfaces, unit integrity, timing coherence, delay and dropout handling, and feedback interpretation. … The brain cannot compensate for corrupted signals." Section 5.3: the engine "projects forward state trajectories … and detects destabilization pressure before divergence."

**The gap.** Before this change, the live controller fed the engine's `stale` (signal dropout) and `drift_ratio` (actuation mismatch) channels with a constant 0. So the upward path could not report a blind sense, and the downward path never learned whether its orders landed.

### *Upward: afferent integrity (senses to engine)*

**Every declared sense reports whether it is live.**

- **Latency.** Blind when its newest sample is older than two windows by the wall clock, or when the window holds no successful request.
  - A hung probe freezes its file; its last clean window must never be read as the present.
  - Failed requests inside a live window become latency pressure (the failed share), so a failure is never read as silence.
- **Power.** Blind when its command returns no number.

**How blindness is used.**

- The blind share enters the engine as `stale`. It raises E, lowers U and feeds the external bath (equations 1-3 via `assimilate`).
- The nervous system grants no contraction on a blind sense, to any organ, and admits no held batch work.
- Expansion and batch pause (the protective directions) stay allowed.

### *Downward and back: efferent feedback (proprioception)*

**Every command is recorded.**

- The node count commanded.
- The HPA targets written.

**At the next decision the observed state is read back.** For each organ, `drift = |observed - commanded| / commanded`.

- The largest drift enters the engine as `drift_ratio`. It raises E and I_U, lowers U and feeds B (equations 1-3).

- The node organ may not be given a new release while its last command did not land. For example, a drain refused by the PodDisruptionBudget leaves the node in service, and the next decision sees it.

### *Sideways: attribution (organ to organ)*

**The machine organ has its own engine view**, fed with machine-attributable pressure only (pods waiting for a place).

**It releases a machine only when all of these hold:**

- pods are not scaling up and latency is not breached;
- nothing is pending;
- after the release the remaining machines stay at or below the engine's rho;
- its own calm, stress and security gates grant contraction;
- every sense is live;
- its last command landed.

### *Tests (all in `verify.py`)*

| Test | What it checks |
|---|---|
| `tests/test_two_way.py` | blind senses (about 75,000 random states), frozen-probe detection, failures read as pressure, release refused while blind or after an order that did not land |
| `tests/test_node_release_gate.py` | the release gate, over 200,000 states |
| `tests/test_nervous_system.py` | the original invariants N1-N6, over 300,000 states |

Every live decision records the senses, their ages, the proprioceptive drift and the gate's reason in the audit, and the benchmark prints them in the job log.

## Nervous system

Omni-Compass is one field. Muscles are nerves.

```
telemetry  -&gt;  afferent (per muscle)
           -&gt;  one six-state engine
           -&gt;  reflex (shield)
           -&gt;  efferent (only WIRED + authority + not killed)
           -&gt;  native controller on kill
```

`omnicompass/nervous.py` is the register. It does not invent GPU or chiller physics.

| Status | Meaning |
|---|---|
| wired | this tree can sense and push |
| sensed | this tree can sense; it does not push |
| open | named, no plant, no push |

Wired today: `nodes`, `hpa`, `power_cap`. Sensed: `heat`, `network`, `security`. Open: GPU, cooling, grid, queues, agents, … Never a muscle: value alignment.

```
python k8s_controlplane/test_nervous.py
```



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

PART FOUR

# The Body: Muscles, Realms and Organisms

*Nine hundred and forty-five muscles in four realms, stacked into six organisms, and every gauge used to judge them.*

CHAPTER 21

# The Muscles, the Realms and the Six Organisms

## The catalog

The catalog (`realms/catalog.csv`) lists **945 distinct muscles in 59 families** (Omni v2 and v3; v1 listed 656 in 46), each with its family, its plant model and its knob. A muscle is anything with a meter and a setting: a compute pool and its machine count, a GPU and its clock ceiling, a chiller and its setpoint, a battery and its reserve, a feeder and its tap, a robot joint and its speed limit, a pump and its flow. Each muscle carries a declared plant model, the simplest physics that reproduces how its meter follows its knob, and a native controller of its own, which is what the muscle does when Omni-Compass is not there. The catalog was built in four waves (`docs/realm_study/`): the spine from primary specifications, each realm's extras from its own specifications, a cross-check against industry and open-source controllers, and a verification, de-duplication and mapping pass. The register (`docs/REGISTER.md`, Chapter 1) lists every family with its muscle count and the kinds of knob it carries, so a reader can see at a glance what the modelled realms contain and what they do not.

The fifty-nine families, by realm, are these. **Compute, AI and cloud** (eighteen): AI inference serving, AI training, cloud virtual machines and capacity, container resources, cross-cluster and edge placement, DPU and programmable IO, distributed cluster managers, GPU fabric and RDMA, HPC and distributed compute, host CPU and memory, Kubernetes dynamic device allocation, Kubernetes placement and scheduling, Kubernetes workload scaling (the largest family, 31 muscles), NVIDIA GPU hardware, node fleets and Karpenter-class control, OpenShift and the Machine API, quantum computing control (modelled), and work admission and demand shaping. **Physics, robotics and autonomous** (eight): automotive EV and mobile powertrains, aviation and autonomous flight, elevators and vertical transport, marine propulsion and vessel automation, rail traction and train control, robotics fleets and warehouse automation, robotics motion control, and spacecraft and flight software. **Energy, facility and industrial** (seventeen): agriculture and irrigation, building and critical-environment HVAC, cooling and chillers, district heating and cooling, energy storage and microgrids, facility and grid optimisation, grid transmission and distribution, healthcare critical environments, industrial PLCs and process automation, mining and mineral processing, oil and gas pipelines, PDUs, UPS and electrical distribution, pharmaceutical and food manufacturing, power generation and turbine control, renewable generation and inverter control, semiconductor fabs and precision manufacturing, and water and wastewater pumping. **Distribution and specialized** (sixteen): cache and memory services, commerce and payments, data analytics and ETL, databases and transactions, medical imaging and clinical systems, messaging and streaming, network routing and switching, observability and telemetry, ports and maritime logistics, reliability, security and recovery, runtimes and applications, search, indexing and vector databases, service meshes and API reliability, block, file and object storage, telecom radio access and edge radio, and workflow, logistics and fulfilment. Thirteen of these, marked "v2" in the register (the elevators, marine, rail, agriculture, district heating, healthcare, mining, pipelines, pharmaceutical and food, power

generation, renewable generation, medical imaging and ports families), came with the second engine version; they, with the muscles added to the families that already existed, are the difference between the 656 muscles in 46 families of v1 and the 945 in 59 of v2 and v3.

Every live benchmark in Part VI sits inside one of these families: the Kubernetes tests inside workload scaling and node fleets, PostgreSQL and MySQL inside databases and transactions, Redis and MongoDB's cache inside cache and memory services, Kafka inside messaging and streaming, the robot arms inside robotics motion control, the drones inside aviation and autonomous flight, the power grids inside transmission and distribution, the buildings inside HVAC. A live result is therefore also a check on one family's model: where the real stack and the modelled muscle move the same way under the same law, the model has earned some trust; where they do not, the model is the thing to doubt, and the catalog's plant model for that family is the place to look.

## The four realms and the spine

The muscles fall into four realms. Every realm stands on the same **spine** of 257 muscles (Kubernetes, machines, GPUs and CPUs, network, storage, observability, security, cooling and electrical distribution), because every realm's work runs on computers that must be cooled and powered, plus its own domain muscles.

| Organism | Muscles | What it is |
|---|---|---|
| 1. Compute / AI / Cloud | 430 | clusters, GPUs, AI training and inference, cloud capacity |
| 2. Physics / Robotics / Autonomous | 376 | joints, fleets, vehicles, flight and spacecraft axes |
| 3. Energy / Facility / Industrial | 470 | data halls, buildings, batteries, UPS, process loops, feeders, hospitals, farms, pipelines, mines, district heat, power plants, renewables, pharmaceutical and food plants |
| 4. Distribution / Specialized | 440 | networks, storage, databases, commerce, workflows, radio networks, clinical systems, ports |
| 5. The four stacked, every duplicate kept | 1,716 | all four realms on one clock, the shared spine counted in each |
| 6. The whole tower, every muscle once | 945 | every distinct muscle on one clock |

The realms differ in what their domain muscles are and in how much slack their native controllers leave. Compute, AI and Cloud is clusters, cards, training and inference pools and cloud capacity, where the knobs are replica targets, machine counts, clock ceilings and power limits, and where native controllers are already good, so the modelled gains are small and the real gains come from the queue and the tail. Physics, Robotics and Autonomous is joints, fleets, vehicles, flight and spacecraft axes, where the knob is a speed or effort limit and the physics decides, axis by axis, whether spending slack as speed saves energy at all; this is the realm the v3 slack gate was written for. Energy, Facility and Industrial is data halls, buildings, batteries, UPS, process loops, feeders and plants, where the knobs are setpoints, units in service, reserves and taps, and where the muscles with the most room are also the ones whose reserves exist for a reason the model does not contain (a UPS held for an outage is never a lever). Distribution and Specialized is networks, storage, databases, commerce, workflows, radio networks, clinical systems and ports, where the knob is usually an admission, a pool or a replica count, and where the real stacks of this manual (the pooler, the broker, the cache) live. Every realm's list, by family and knob, is `docs/REALM_MUSCLES.md` and `docs/REGISTER.md` Chapter 1.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

## What an organism run is

These six organisms are the benchmark set of the modelled realms. An organism is every one of its muscles stepped on one clock, each from its own plant model, each with its own native controller, and the whole fed one demand trace from one seed. Native runs the organism with every native controller alone. Omni runs the same organism, the same seed, the same demand and the same clock, with the compass law on every knob the verdict allows. Each run produces its own receipt: the organism's work, its energy, its time over its service line, and every knob's hand-back. In code and in the workflows the two large organisms are named `tower` and `stack`, never by a count, because the counts changed between v1 and v2 and a name that carried a count would read as a different organism; the old names are read as aliases.

The receipt is read in a fixed order. **Work** first: the organism's output over the window, which must not fall, and which reads worse by rule if it does, however little, as the four stacked at 1,000 copies showed on a difference of one part in a million (section 2.5). **Energy** second, and **work per energy** with it, which is the gain the modelled realms are built to show. **Time over the service line** third, the share of steps on which any muscle's service reading was past its line, which is the do-no-harm gauge for the body as a whole. Then **every knob's hand-back**, which must be complete in every run or the run fails. The reading given to an organism (the realms tables under `results/realms/`, one per engine) uses the words "superior within guardrails" when work and energy both improve with the service line respected, "service tradeoff" when a gain in energy came with time over the line, and names the loss when there is one: the v2 reading on the Physics realm and the tower was a service tradeoff, traced to the speed knobs on busy motion axes, and the slack gate that answers it is the one rule that makes v3 (section 6.3). What an organism run cannot show is also fixed: it is evidence class S, the plant models are our own, and nothing in it is presented as proof of a real stack. Its use is to show that one law holds across every realm at once, at a size no live benchmark can reach, and to find the rules that then have to earn their place on real software.

## The grid of copies and runs

Every organism runs at **1, 10, 100 and 1,000 copies** (that many instances of the organism on one clock, so the four stacked at 1,000 copies is 1.7 million modelled muscles) and at **1, 10, 100 and 1,000 paired runs** (that many native-and-omni pairs, each from its own seed): 96 cells in all, both arms shown in every cell. One thousand is the maximum in both directions. The grid as it stands (`results/scale/GRID.md`) is 84 of 90 judged cells on Omni v3; the six cells left, 100 and 1,000 runs at 1,000 copies, are beyond the machines available and are declared as such, not hidden. The reading at every size is the same to the digit, which is itself a finding about the law: it does not drift with scale.

The two axes answer two different questions, and a reader should keep them apart. **Copies** is the size of the body the one brain governs: at 1,000 copies of the four stacked, the law computes a force for 1.7 million muscles from one reading of one clock, and the question is whether it stays coherent, whether the decision time stays inside the decision period, and whether the cluster inside it (section 2.5) is still served on time. **Runs** is the number of independent paired trials, each from its own seed, and the question is statistical: how wide the interval of the paired difference is and whether it moves as the trials multiply. A cell at 1 copy and 1,000 runs tells a reader about the noise of the method; a cell at 1,000 copies and 1 run tells a reader about the scale of the brain; the cells on the diagonal tell both at once. Every cell shows the native value and the omni value side by side, never a change alone, because a change without its two ends cannot be checked against the organism's own records. The grid is rebuilt whole on every engine version, never patched cell by cell, so that no square carries a result from a different engine than its neighbours (Chapter 15).



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## The organisms with a real cluster inside

The modelled organisms are also run with a **real Kubernetes cluster inside** as one more muscle (`tools/run_kil.py`, workflows `six-kube`, `big-organism`, `big-organism-detached`): the organism's own compute demand drives a real load generator against a real service on a real cluster, and the cluster's watts come back into the organism as heat and load. At 1 to 100 copies this runs on GitHub's machines; at 1,000 copies it needs a rented machine and runs there, detached from the GitHub job that started it, for about seven hours a repetition. These runs are evidence class L for the cluster and S for the organism around it, and are labelled "L + S".

Two rules keep these runs honest, and both are written in the preregistration before the machine is rented. The first is the **clock rule**. The organism must step on the measured window's clock: the row "organism behind its window" says how many seconds after the window its last step ended, and a repetition in which either arm ended more than 5% of the window late is marked **off the clock** in the organism's line, because its last steps saw a cluster whose load schedule had already ended. The row is shown and never judged; at 1,000 copies of the four stacked, both arms ended about 22 s behind a 10,800 s window, a fifth of one percent, and no repetition was off the clock. The second is the **hand-back at 90%**. Omni-Compass releases the cluster's knob at nine tenths of the window and the last tenth is run by native alone in both arms, so that every repetition ends with the proof that the knob was given back and that native resumed where it would have been.

The first such run on Omni v3 is in `results/live/V3_BIG_ORGANISM.md`: the four stacked at 1,000 copies, three paired repetitions on one rented eight-core machine over 29 hours, the cluster's slowest 5% of requests at 2,882 ms under native and 150 ms under omni, the time over the response line from 51% of samples to none, no failed request under omni, the same six machines in service in both arms and the energy inside the noise. The organism's own rows, which are models, show the work the same to one part in a million and read worse by the rule that any decrease the same in all three of our runs is worse, however small; the row is in the table with that word on it. The whole tower at 1,000 copies, 945,000 modelled muscles on one clock, ran the same way on a second machine of the same type over 25 hours and landed on 9 October: the cluster's slowest 5% of requests at 6,690 ms under native and 160 ms under omni, the slowest 1% at 9,068 against 187 ms, time over the line from 76% of samples to none, failed requests from 4.8% to none, all clear of zero with three pairs; machines, replicas and pods the same; energy inside the noise; both arms 14 s behind a 10,800 s window. The table holds both organisms side by side.

## The register

The register (`docs/REGISTER.md`) is the one list a referee can audit: every muscle by family (Chapter 1), every benchmark by platform with its native controller, Omni-Compass's knob, its gauges, its result and its file (Chapter 21), what is not measured yet, said plainly (Chapter 56), and every open benchmark still to run, in order, one or two at a time (Chapter 8). The program that takes each benchmark to its full size, with what each step costs, is `docs/PROOF_PROGRAM.md`. Every benchmark in this manual appears in the register under the same name.

A row in the register is read left to right as a sentence: the platform, the open benchmark used for it and who publishes that benchmark, the native controller Omni-Compass sits on top of, the knob it moves, the gauges it is judged on, and its state. The state is written in a small, repeated vocabulary so that a reader can scan the column: **queued** (in the audited queue, not yet preregistered), **preregistered and built** (the rules are frozen and the harness runs, no counted result yet), the smoke runs with what each one taught and what was changed before the counted runs, and **A, B, C done** with the three run numbers, the engine version and the readings, losses first where there are any. A row never loses its history: when a benchmark moves from smoke to counted, the smoke record stays in its preregistration, and when a result is superseded by a run on a later engine, the earlier table moves to `docs/history` with its engine named. The register is the only list that must be complete; the manual's Chapter 43 and the dossier

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

(`docs/DOSSIER.md`) are readings of it, and anything in them that is not in the register is an error to be fixed in the register's favour.

The audited queue (Chapter 8 of the register) is the program's promise about what comes next, and it is deliberately short at the front: one or two benchmarks are run at a time, each preregistered, built, smoked and counted before the next is opened, so that no result is ever waiting on a harness that was half built when the runs began. The queue is ordered by what each benchmark adds to the index that the ones before it did not: a new resource traded (memory after machines, a buffer pool after a cache), a new native controller, a new class of evidence.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 22

# The Four Realms

The muscle tower on modelled plants (945 muscles in Omni v2, 656 in v1): four realms and the whole tower as a fifth organism, each run natively and with Omni-Compass on top, with meters and receipts. Evidence class **S** (simulation).

## Run it

```
pip install -r requirements.txt
python3 tools/run_realms.py                    # the preregistered run: every muscle and 5 organisms,
    ↪ seeds 1000-1009
python3 tests/test_realms.py                   # the harness's own checks (also inside verify.py)
```

Results (Omni v3; v2's are in `results/realms/v2/`, v1's in `results/realms/v1/`): `results/realms/REALMS.md` (the tables), `MUSCLES.csv` (one row per muscle), `REALMS.json` (every per-seed contrast), `RUN.json` and `SHA256SUMS.txt` (commit and fingerprints).

## What is where

| File | What it is |
|---|---|
| `realms/catalog.csv` | the 945 muscles: family, name, realm, plant, parameter set, knob (v1's 656: `realms/catalog_v1.csv`) |
| `tools/realms_catalog.py` | the rules that gave each muscle its realm, plant and knob |
| `realms/plants.py` | the five plants and their native controllers; the four knobs; the capacity law for one plant |
| `realms/presets.py` | every parameter, one set per family class |
| `realms/harness.py` | the arms (native, watch, omni, fixed setpoint), the organisms, the outcome and the label rule |
| `docs/REALMS_PREREGISTRATION.md` | the question, the rules, the seeds and the development history, frozen before the run |

## How it relates to the rest

| Layer | Realm harness | Elsewhere in this repository |
|---|---|---|
| Mathematics (T, V) | uses the frozen engine and governor unchanged | `docs/TRACKING_THEOREM.md`, `verify.py` |
| Simulation (S) | **this** | fleet and cluster simulators, GPU model |
| Real software (L) | not here | set 22 on real Kubernetes (`results/live/LIVE_REPS_22.md`) |
| Physical (P) | not here | the GPU bench: first run on an NVIDIA A10, 2026-10-02 (`results/gpu/run-20261002T082232Z/GPU_REPS.md`); the corrected governor not yet run on a card (`docs/GPU_PREREGISTRATION.md`) |

A realm result that looks good is a reason to test that knob on a real machine, not a substitute for it. The realms whose knobs can be tested for real first are the compute realm's (the GPU bench, kind), because the tools already exist.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 23

# The Realm Muscles

Generated from `realms/catalog.csv` (rules: `tools/realms_catalog.py`; the v2 build: `tools/realms_catalog_v2.py`). Every realm's organism is its own families plus the **shared spine** (the infrastructure every real stack runs on), so the realm counts add to more than the tower. Each muscle is followed by the one knob Omni may hold on it.

## The shared spine: 257 muscles, in all four realms

### *Cloud VM & Capacity (21; plant: compute_pool, node)*

vm_count [capacity], vm_size [capacity], vm_start_stop [capacity], vm_migrate [capacity], instance_family [capacity], asg_min [capacity], asg_max [capacity], spot_mix [capacity], reservation_mix [capacity], zone_selection [capacity], region_selection [capacity], architecture_selection [capacity], accelerator_selection [capacity], boot_disk_class [capacity], placement_group [capacity], interruption_response [capacity], target_tracking_policy [setpoint], scaling_cooldown_warmup [capacity], vm_cpu_cap_by_priority [admission], predictive_scaling [capacity], instance_refresh [capacity]

### *Container Resources (23; plant: compute_pool, server)*

cpu_request [capacity], cpu_limit [capacity], memory_request [capacity], memory_limit [capacity], ephemeral_storage [capacity], hugepages [capacity], io_weight [setpoint], pids_limit [capacity], memory_high [capacity], memory_reclaim [capacity], swap_limit [capacity], cgroup_io_max [capacity], cpu_weight [setpoint], cpu_quota [admission], cpuset [capacity], runtime_class [capacity], in_place_resize [capacity], namespace_quota [admission], default_limits [capacity], memory_hard_limit [capacity], memory_protection [capacity], io_latency_target [setpoint], freeze [admission]

### *Cooling, Chillers & Thermodynamics (18; plant: thermal_zone, data_hall)*

supply_air_temperature [setpoint], return_air_target [setpoint], coolant_supply_temperature [setpoint], coolant_flow [capacity], pump_speed [capacity], fan_speed [capacity], chiller_setpoint [setpoint], compressor_authority [power], cooling_tower_fan [capacity], cooling_capacity [capacity], rack_thermal_budget [power], gpu_thermal_envelope [setpoint], cpu_thermal_envelope [setpoint], thermal_workload_migrate [admission], thermal_load_shed [admission], server_fan_speed [capacity], economiser [capacity], chiller_plant_supervisory_control_rl_with_safety_layer [capacity]

### *Host CPU & Memory (23; plant: compute_pool, cpu_host)*

cpufreq_min [power], cpufreq_max [power], rapl_package_power [power], uncore_frequency [power], energy_perf_preference [power], cpu_idle_policy [power], memory_bandwidth [capacity], numa_balance [capacity], irq_affinity [capacity], llc_allocation [capacity], memory_pressure_gate [admission], page_reclaim_rate [capacity], transparent_hugepages [capacity], core_online_offline [capacity], thermal_throttle_policy [power], host_power_profile [power], frequency_governor [power], turbo_boost [capacity], dram_power_limit [power], swappiness_reclaim [capacity], core_packing [capacity], per_pod_cpu_frequency_range_and_c_state_access [power], uncore_frequency_per_node [power]

### *Kubernetes Placement & Scheduling (20; plant: compute_pool, server)*

node_selector [capacity], node_affinity [capacity], pod_anti_affinity [capacity], topology_spread [capacity], numa_placement [capacity], gpu_topology [capacity], storage_locality [capacity], network_locality [capacity], taint_toleration [setpoint], priority_class [admission], preemption_policy [admission], device_claim [capacity], failure_domain_spread [capacity], scheduler_backoff [admission], gang_admission [admission], deschedule [capacity], scheduler_scoring_strategy [capacity], scheduler_plugin_weights [setpoint], scheduling_gates [admission], cpu_manager [capacity]

### *Kubernetes Workload Scaling (31; plant: compute_pool, server)*

replicas [capacity], hpa_cpu_target [setpoint], hpa_memory_target [setpoint], hpa_custom_target [setpoint], vpa_apply [capacity], scale_to_zero [capacity], keda_threshold [admission], rollout_rate [capacity], max_surge [capacity], max_unavailable [capacity], deployment_pause_resume [admission], rollout_abort [admission], pod_eviction [admission], pdb_policy [capacity], scheduler_queue_priority [admission], api_priority_fairness [admission], hpa_min_replicas [capacity], hpa_max_replicas [capacity], hpa_scale_down_stabilisation_window [setpoint], hpa_scale_up_stabilisation_window [setpoint], hpa_scaling_rate_policies [capacity], hpa_policy_selection [capacity], hpa_tolerance [capacity], hpa_sync_period [capacity], vpa_minmax_allowed_resources [capacity], keda_polling_interval [setpoint], rollout_progress_deadline [admission], rollout_readiness_delay [capacity], unhealthy_pod_eviction_policy [admission], graceful_termination [capacity], probes [capacity]

### *NVIDIA GPU Hardware (19; plant: compute_pool, gpu)*

gpu_allocate [capacity], gpu_power_limit [power], gpu_sm_clock [power], gpu_memory_clock [power], gpu_persistence_mode [capacity], gpu_compute_mode [capacity], mig_mode [capacity], mig_geometry [capacity], gpu_timeslice [capacity], gpu_mps [capacity], gpu_quarantine [admission], gpu_reset [capacity], gpu_thermal_limit [power], gpu_ecc_response [capacity], gpu_job_power_budget [admission], application_clocks [power], gpu_temperature_target [setpoint], sync_boost [capacity], target_clocks [power]

### *Network Routing & Switching (19; plant: compute_pool, network)*

lb_weight [setpoint], route_weight [setpoint], rate_limit [admission], bandwidth_limit [capacity], qos_class [capacity], connection_limit [admission], failover_route [capacity], nic_rate_limit [admission], nic_queue_count [capacity], queue_discipline [capacity], congestion_control [capacity], egress_budget [admission], ingress_budget [admission], ecmp_weight [setpoint], path_selection [capacity], network_isolation [admission], nic_interrupt_coalescing [capacity], nic_ring_size [capacity], port_link_power [power]



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

### *Node Fleet & Karpenter-Class Control (24; plant: compute_pool, node)*

node_desired [capacity], node_pool_min [capacity], node_pool_max [capacity], node_provision [capacity], node_cordon [admission], node_drain [admission], node_consolidate [capacity], node_replace [capacity], node_shutdown [capacity], node_power_on [capacity], nodepool_weight [setpoint], disruption_budget [admission], consolidation_policy [capacity], consolidate_after [capacity], expire_after [capacity], capacity_class [capacity], scale_down_utilisation_threshold [admission], scale_down_unneeded_time [capacity], scale_down_delay_after_add [capacity], expander [capacity], node_pool_limits [capacity], kubelet_eviction_thresholds [admission], kubelet_reserved_resources [capacity], max_pods_per_node [capacity]

### *Observability & Telemetry (9; plant: compute_pool, server)*

collector_memory_limit [capacity], export_concurrency [admission], cardinality_budget [admission], retention_window [setpoint], remote_write_queue [capacity], telemetry_shed [admission], metric_scrape_interval [setpoint], batching [capacity], per_pod_energy_attribution [capacity]

### *PDU, UPS & Electrical Distribution (18; plant: energy_storage, ups)*

server_power_cap [power], rack_power_cap [power], pdu_branch_power_limit [power], pdu_outlet_control [capacity], ups_operating_mode [capacity], ups_charge_rate [power], ups_discharge_rate [power], phase_balance [capacity], load_transfer [admission], reactive_power_target [setpoint], voltage_target [setpoint], generator_dispatch [capacity], electrical_isolation [admission], breaker_trip_gate [admission], ups_shutdown_battery_test [capacity], hierarchical_power_budget_row_pdu_switchboard_controllers [power], priority_aware_capping_throttle_low_priority_first [power], power_oversubscription_with_prediction [power]

### *Reliability, Security & Recovery (12; plant: compute_pool, server)*

restart [capacity], rollback [capacity], traffic_divert_recovery [capacity], degraded_mode [capacity], actuator_freeze [admission], workload_isolate [admission], credential_rotation_gate [admission], policy_enforcement [capacity], rate_abuse_gate [admission], fault_domain_isolate [admission], backup_trigger [capacity], kill_switch [admission]

### *Storage Block/File/Object (20; plant: compute_pool, storage)*

volume_size [capacity], iops_limit [capacity], throughput_limit [capacity], replica_count [capacity], storage_tier [capacity], volume_placement [capacity], snapshot_trigger [capacity], rebalance [capacity], recovery_rate [capacity], backfill_rate [capacity], compaction_pressure [capacity], cache_allocation [capacity], object_replication [capacity], erasure_code_profile [capacity], storage_admission [admission], degraded_storage_gate [admission], scrub_schedule [capacity], io_scheduler [capacity], disk_power_spin_down [power], lifecycle_tiering [capacity]

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

# Realm 1: Compute / AI / Cloud: 173 own muscles + the spine = 430 in its organism

### *AI Inference Serving (24; plant: compute_pool, gpu)*

model_replicas [capacity], model_route_weight [setpoint], model_load [capacity], model_unload [capacity], model_instance_count [capacity], continuous_batching [capacity], max_batch_size [capacity], batch_queue_delay [capacity], inference_concurrency [admission], inference_max_tokens [capacity], kv_cache_budget [admission], prefix_cache_budget [admission], speculative_decode_budget [admission], model_precision [power], inference_priority [admission], inference_slo_gate [admission], autoscaling_target_concurrency_qps [admission], scale_to_zero_panic_window [setpoint], request_priority_levels [admission], kv_cache_swap_space [capacity], chunked_prefill [capacity], tensor_parallel_degree [capacity], disaggregated_prefill_decode [capacity], lora_adapter_load [capacity]

### *AI Training (18; plant: compute_pool, gpu_batch)*

training_workers [capacity], global_batch_size [capacity], microbatch_size [capacity], gradient_accumulation [capacity], data_parallelism [capacity], tensor_parallelism [capacity], pipeline_parallelism [capacity], expert_parallelism [capacity], checkpoint_interval [setpoint], checkpoint_trigger [capacity], training_preempt [admission], training_gang_size [capacity], elastic_worker_count [capacity], straggler_mitigation [capacity], training_precision [power], compute_comm_overlap [capacity], activation_checkpointing [capacity], dataloader_workers [capacity]

### *Cross-Cluster, Multi-Region & Edge (15; plant: compute_pool, node)*

multi_cluster_dispatch [capacity], region_dispatch [capacity], zone_dispatch [capacity], edge_dispatch [capacity], cloud_capacity_class [capacity], workload_migrate_region [capacity], data_residency_gate [admission], latency_region_gate [admission], cost_region_gate [admission], carbon_region_gate [admission], global_failover [capacity], federation_quota [admission], cross_cluster_replication [capacity], edge_offload [capacity], global_admission [admission]

### *DPU SmartNIC & Programmable IO (12; plant: compute_pool, fabric)*

dpu_pf_tx_rate [capacity], dpu_vf_tx_rate [capacity], dpu_sf_tx_rate [capacity], dpu_bandwidth_share [capacity], dpu_qos_group [capacity], sriov_vf_count [capacity], smartnic_flow_steering [capacity], smartnic_offload_enable [capacity], dpu_cpu_budget [admission], dpu_memory_budget [admission], dpu_service_placement [capacity], dpu_failover [capacity]

### *Distributed Cluster Managers (15; plant: compute_pool, batch)*

task_admission [admission], task_priority [admission], resource_reservation [capacity], cluster_quota [admission], task_preemption [admission], binpack_pressure [capacity], spread_pressure [capacity], gang_schedule [capacity], worker_allocation [capacity], maintenance_evacuation [capacity], oversubscription [capacity], resource_reclaim [capacity], queue_fairness [admission], deadline_pressure [admission], scheduler_retry [admission]

### *GPU Fabric & RDMA (18; plant: compute_pool, fabric)*

nvlink_placement [capacity], nvswitch_route [capacity], gpu_fabric_quarantine [admission], rdma_bandwidth [capacity], rdma_route [capacity], nic_affinity [capacity], collective_concurrency [admission], collective_algorithm [capacity], collective_chunk_size [capacity], rank_placement [capacity], gpudirect_policy [capacity], congestion_response [capacity], rail_selection [capacity], fabric_failover [capacity], communication_priority [admission], fabric_isolation [admission], collective_protocol [capacity], nic_hca_selection [capacity]



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

### *HPC & Distributed Compute (19; plant: compute_pool, batch)*

job_slots [capacity], mpi_ranks [capacity], rank_mapping [capacity], node_allocation [capacity], job_walltime [capacity], job_priority [admission], job_preemption [admission], checkpoint_restart [capacity], parallel_io_budget [admission], collective_budget [admission], accelerator_share [capacity], cpu_gpu_ratio [setpoint], memory_per_rank [capacity], scratch_budget [admission], scheduler_fair_share [admission], backfill_policy [capacity], node_power_saving [power], cpu_frequency_per_job [power], partition_limits [capacity]

### *Kubernetes Dynamic Device Allocation (8; plant: compute_pool, gpu)*

dra_device_class_selection [capacity], dra_claim_capacity [capacity], dra_claim_sharing [capacity], dra_device_taint [capacity], dra_device_eviction [admission], dra_binding_readiness [capacity], dra_binding_failure_response [capacity], dra_device_configuration [setpoint]

### *OpenShift & Machine API (9; plant: compute_pool, node)*

machine_remediation [capacity], machine_health_gate [admission], mcp_pause [admission], mcp_max_unavailable [capacity], node_config_rollout [capacity], operator_reconcile_budget [admission], cluster_version_pacing [capacity], infra_machine_admission [admission], machine_failure_domain [capacity]

### *Quantum Computing Control Simulation (16; plant: compute_pool, qpu)*

qubit_mapping [capacity], circuit_admission [admission], shot_allocation [capacity], circuit_scheduling [admission], gate_scheduling [capacity], pulse_amplitude [capacity], pulse_duration [setpoint], pulse_phase [capacity], pulse_frequency [power], coupling_control [capacity], reset_scheduling [capacity], measurement_scheduling [capacity], dynamical_decoupling [capacity], noise_aware_routing [capacity], error_mitigation_budget [admission], quantum_queue_priority [admission]

### *Work Admission & Demand Shaping (19; plant: compute_pool, server)*

api_concurrency [admission], queue_concurrency [admission], queue_backpressure [admission], job_admission [admission], batch_admission [admission], inference_admission [admission], load_shed [admission], priority_gate [admission], tenant_admission [admission], burst_limit [admission], deadline_admission [admission], work_budget [admission], request_queue_limit [admission], retry_admission [admission], background_work_gate [admission], maintenance_work_gate [admission], request_concurrency_limit [admission], adaptive_concurrency_limit [admission], priority_request_queue_under_limit [admission]

## Realm 2: Physics / Robotics / Autonomous: 119 own muscles + the spine = 376 in its organism

### *Automotive EV & Mobile Powertrain (14; plant: motion_axis, ev_traction)*

traction_torque_limit [power], regen_braking_level [power], battery_charge_limit [power], battery_discharge_limit [power], battery_thermal_target [power], motor_thermal_limit [power], vehicle_speed_envelope [capacity], energy_recovery_target [power], auxiliary_power_budget [power], fast_charge_current [power], fast_charge_voltage [power], vehicle_safe_state [admission], state_of_charge_limits [power], smart_charging_schedule [admission]

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

### *Aviation & Autonomous Flight (20; plant: motion_axis, flight_axis)*

throttle_envelope [power], attitude_target [capacity], attitude_rate_target [capacity], velocity_target [capacity], altitude_target [capacity], waypoint_authority [capacity], flight_hold [admission], return_to_home [admission], land_action [admission], mission_admission [admission], geofence_response [admission], failsafe_selection [admission], battery_reserve_threshold [admission], actuator_saturation_envelope [capacity], flight_mode_transition [admission], flight_termination_safe_state [admission], attitude_rate_gains [capacity], horizontal_speed_limit [power], vertical_speed_limits [power], acceleration_jerk_limits [power]

### *Elevators & Vertical Transport (12; plant: motion_axis, elevator_hoist)*

hoist_speed_target [capacity], acceleration_limit [capacity], regen_drive_mode [power], standby_power_mode [admission], destination_dispatch_schedule [admission], car_parking_mode [admission], door_dwell_hold [admission], escalator_speed_target [capacity], group_capacity_mode [admission], motor_thermal_derate [power], brake_test_schedule [admission], fire_recall_mode [admission]

### *Marine Propulsion & Vessel Automation (12; plant: motion_axis, marine_propulsion)*

shaft_speed_target [capacity], propeller_pitch_limit [power], shaft_power_limit [power], bow_thruster_duty [power], ballast_pump_mode [admission], engine_load_sharing_setpoint [capacity], slow_steaming_speed_target [capacity], auxiliary_engine_staging [capacity], shore_power_mode [admission], hotel_load_budget [power], trim_target [capacity], rudder_rate_limit [power]

### *Rail Traction & Train Control (14; plant: motion_axis, rail_traction)*

traction_effort_limit [power], regenerative_braking_share [power], coasting_speed_target [capacity], dwell_time_schedule [admission], hvac_duty_cycle [power], train_auxiliary_power_budget [power], platform_approach_speed [capacity], acceleration_rate_setpoint [capacity], wheel_slip_protection_mode [admission], traction_motor_thermal_derate [power], catenary_voltage_limit [power], timetable_recovery_margin [admission], headway_target [capacity], door_release_hold [admission]

### *Robotics Fleet & Warehouse Automation (15; plant: compute_pool, robot_fleet)*

robot_dispatch [capacity], task_assignment [capacity], traffic_reservation [capacity], robot_route [capacity], charging_dispatch [capacity], battery_reserve [capacity], elevator_request [capacity], door_request [capacity], conveyor_speed [capacity], agv_speed [capacity], warehouse_zone_admission [admission], robot_quarantine [admission], fleet_failover [capacity], human_safe_stop [capacity], fleet_concurrency [admission]

### *Robotics Motion Control (18; plant: motion_axis, robot_joint)*

joint_position [capacity], joint_velocity [capacity], joint_acceleration [capacity], joint_effort [power], cartesian_velocity [capacity], trajectory_speed [capacity], trajectory_acceleration [capacity], jerk_limit [capacity], collision_margin [capacity], force_limit [power], gripper_force [power], locomotion_speed [capacity], steering_angle [capacity], braking_force [power], balance_correction [capacity], trajectory_tolerances [capacity], velocity_acceleration_scaling [capacity], controller_update_rate [capacity]

### *Spacecraft & Flight Software (14; plant: motion_axis, reaction_wheel)*

space_command_admission [admission], flight_task_schedule [admission], space_mode_transition [admission], payload_duty_cycle [power], communication_allocation [capacity], space_power_budget [power], space_thermal_command [power], attitude_command_envelope [capacity], reaction_wheel_allocation [capacity], rcs_authority [power], safe_mode_transition [admission], watchdog_recovery [admission], instrument_activation [capacity], fault_isolation [admission]


*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

# Realm 3: Energy / Facility / Industrial: 213 own muscles + the spine = 470 in its organism

### *Agriculture & Irrigation (14; plant: process_loop, irrigation)*

irrigation_pump_speed [capacity], mainline_pressure_setpoint [setpoint], soil_moisture_target [setpoint], pivot_speed_setpoint [setpoint], fertigation_dose [power], greenhouse_temperature_setpoint [setpoint], greenhouse_co2_target [setpoint], vent_position_setpoint [setpoint], grain_dryer_temperature_setpoint [setpoint], cold_storage_temperature_setpoint [setpoint], barn_ventilation_rate [capacity], milking_vacuum_level_setpoint [setpoint], well_drawdown_level_target [setpoint], drip_zone_dispatch [admission]

### *Building & Critical Environment HVAC (15; plant: thermal_zone, building)*

zone_temperature_target [setpoint], zone_airflow [capacity], ahu_fan_speed [capacity], damper_position [capacity], economizer_position [capacity], boiler_setpoint [setpoint], heat_pump_mode [admission], humidity_target [setpoint], occupancy_ventilation [admission], building_demand_limit [power], thermal_storage_dispatch [capacity], hvac_emergency_mode [admission], setpoint_reset_trim_and_respond [setpoint], duct_static_pressure_setpoint [setpoint], optimal_start_stop [capacity]

### *District Heating & Cooling (13; plant: process_loop, district_heat)*

supply_temperature_setpoint [setpoint], return_temperature_target [setpoint], differential_pressure_setpoint [setpoint], network_pump_speed [capacity], heat_pump_staging [capacity], chp_dispatch [admission], thermal_storage_level_target [setpoint], peak_boiler_heater_output [power], substation_flow_limit [capacity], peak_demand_shed [admission], outdoor_reset_setpoint [setpoint], cooling_network_supply_temperature [setpoint], cooling_storage_level_target [setpoint]

### *Energy Storage & Microgrid (19; plant: energy_storage, microgrid)*

battery_charge_power [power], battery_discharge_power [power], battery_soc_reserve [setpoint], grid_import_limit [power], grid_export_limit [power], pv_curtailment [power], ev_charge_power [power], heat_pump_power [power], electrolyzer_power [power], microgrid_demand_limit [power], peak_shaving [capacity], time_of_use_schedule [setpoint], energy_load_shed [admission], flex_load_admission [admission], storage_dispatch [capacity], microgrid_emergency_reserve [setpoint], volt_var [capacity], volt_watt [capacity], constant_power_factor_reactive_power [power]

### *Facility & Grid Optimization (15; plant: energy_storage, facility)*

facility_power_budget [power], utility_demand_limit [power], demand_response [admission], electricity_price_gate [admission], carbon_intensity_gate [admission], renewable_dispatch [capacity], generator_start_stop [capacity], site_battery_dispatch [capacity], pue_target [setpoint], cooling_power_budget [power], it_power_budget [power], rack_power_allocation [power], facility_peak_guard [capacity], grid_frequency_response [admission], facility_islanding [admission]

### *Grid Transmission & Distribution (12; plant: process_loop, feeder_voltage)*

capacitor_bank_switch [capacity], voltage_regulator_tap [setpoint], transformer_tap [setpoint], inverter_real_power [power], inverter_reactive_power [power], feeder_voltage_target [setpoint], feeder_load_transfer [capacity], distribution_storage_dispatch [admission], demand_response_dispatch [admission], frequency_droop_setpoint [setpoint], grid_protection_mode [admission], grid_restoration_sequence [admission]

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

### *Healthcare Critical Environments (13; plant: thermal_zone, hospital)*

operating_room_air_change_setpoint [setpoint], isolation_room_pressure_target [setpoint], patient_room_temperature_setpoint [setpoint], surgical_suite_humidity_target [setpoint], ahu_supply_air_temperature [setpoint], pharmacy_cold_room_setpoint [setpoint], sterile_storage_humidity_setpoint [setpoint], imaging_suite_cooling_capacity [capacity], chiller_plant_staging [capacity], exhaust_fan_capacity [capacity], ward_night_setback_mode [admission], medical_gas_plant_demand_limit [power], emergency_power_load_shed [admission]

### *Industrial PLC & Process Automation (18; plant: process_loop, process)*

plc_cycle_authority [admission], machine_cell_admission [admission], valve_position [setpoint], pump_flow [capacity], compressor_speed [capacity], heater_power [power], furnace_setpoint [setpoint], pressure_setpoint [setpoint], temperature_setpoint [setpoint], mass_flow_setpoint [setpoint], tank_level_target [setpoint], conveyor_rate [capacity], feed_rate [capacity], purge_vent_action [admission], controller_mode [capacity], alarm_limits [capacity], safety_interlock_trip [capacity], opc_ua_writes [capacity]

### *Mining & Mineral Processing (14; plant: process_loop, mill)*

sag_mill_load_setpoint [setpoint], mill_speed_target [setpoint], crusher_gap_setpoint [setpoint], flotation_aeration_rate [power], cyclone_feed_pressure_target [setpoint], thickener_underflow_density_target [setpoint], reagent_dose [power], conveyor_speed_setpoint [setpoint], dewatering_pump_level_target [setpoint], ventilation_on_demand_airflow [capacity], stockpile_level_target [setpoint], slurry_pump_speed [capacity], tailings_discharge_shutdown [admission], ore_blend_dispatch [admission]

### *Oil & Gas Pipelines (14; plant: process_loop, pipeline)*

compressor_discharge_pressure_setpoint [setpoint], pump_station_suction_pressure_target [setpoint], line_pack_target [setpoint], pipeline_flow_setpoint [setpoint], compressor_unit_staging [capacity], vfd_pump_speed [capacity], terminal_tank_level_target [setpoint], leak_detection_shutdown [admission], batch_interface_dispatch [admission], heater_outlet_temperature_setpoint [setpoint], drag_reducing_agent_dose [power], valve_position_target [setpoint], cathodic_protection_voltage [setpoint], pressure_protection_priority [admission]

### *Pharmaceutical & Food Manufacturing (14; plant: process_loop, batch_reactor)*

reactor_temperature_setpoint [setpoint], agitator_speed_setpoint [setpoint], fermenter_dissolved_oxygen_target [setpoint], ph_setpoint [setpoint], pasteurizer_holding_temperature_setpoint [setpoint], freezer_tunnel_temperature_setpoint [setpoint], cip_cycle_dispatch [admission], cleanroom_pressure_cascade_setpoint [setpoint], lyophilizer_shelf_temperature_setpoint [setpoint], chromatography_flow_setpoint [setpoint], steam_sterilizer_cycle_priority [admission], oven_zone_temperature_setpoint [setpoint], refrigerant_compressor_authority [admission], batch_hold_quarantine [admission]

### *Power Generation & Turbine Control (14; plant: process_loop, turbine)*

turbine_speed_droop_setpoint [setpoint], unit_load_setpoint [setpoint], boiler_steam_pressure_setpoint [setpoint], feedwater_level_target [setpoint], agc_participation_limit [capacity], excitation_voltage_setpoint [setpoint], hydro_gate_position_target [setpoint], combustion_air_ratio_setpoint [setpoint], cooling_water_flow_setpoint [setpoint], inlet_guide_vane_position [setpoint], reserve_dispatch_priority [admission], turbine_ramp_rate_limit [capacity], emissions_shutdown [admission], nuclear_rod_position_target [setpoint]



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

### *Renewable Generation & Inverter Control (13; plant: process_loop, inverter)*

inverter_volt_var_setpoint [setpoint], inverter_volt_watt_setpoint [setpoint], active_power_curtailment [power], plant_reactive_power_target [setpoint], wind_turbine_yaw_offset [capacity], pitch_angle_limit [capacity], rotor_speed_setpoint [setpoint], inverter_frequency_droop_setpoint [setpoint], ramp_rate_limit [capacity], tracker_stow_mode [admission], string_mppt_voltage_setpoint [setpoint], noise_mode_schedule [admission], ice_detection_shutdown [admission]

### *Semiconductor Fab & Precision Manufacturing (13; plant: process_loop, chamber)*

tool_job_dispatch [admission], wafer_route [capacity], chamber_recipe_selection [admission], chamber_temperature [setpoint], chamber_pressure [setpoint], gas_flow [capacity], rf_power [power], vacuum_pump_speed [capacity], robot_transfer_rate [capacity], lot_priority [admission], tool_quarantine [admission], run_to_run_control [capacity], idle_sleep_mode [capacity]

### *Water Wastewater & Pumping (12; plant: process_loop, water)*

pump_speed_water [capacity], valve_position_water [setpoint], reservoir_level_target [setpoint], line_pressure_target [setpoint], flow_target_water [setpoint], aeration_rate [power], chemical_dose_rate [power], filtration_backwash [admission], lift_station_dispatch [admission], leak_isolation [admission], water_demand_shed [admission], water_emergency_shutdown [admission]

## Realm 4: Distribution / Specialized: 183 own muscles + the spine = 440 in its organism

### *Cache & Memory Services (16; plant: compute_pool, server)*

cache_size [capacity], cache_ttl [setpoint], cache_eviction_policy [admission], cache_replicas [capacity], cache_sharding [capacity], cache_prefetch [capacity], cache_writeback_rate [capacity], cache_admission [admission], hot_key_isolation [admission], cache_connection_limit [admission], cache_memory_limit [capacity], cache_compression [capacity], cache_warmup [capacity], cache_failover [capacity], cache_flush_rate [capacity], memory_size_threads [capacity]

### *Commerce & Payment Systems (15; plant: compute_pool, commerce)*

payment_admission [admission], payment_concurrency [admission], payment_retry [admission], payment_timeout [admission], fraud_review_gate [admission], authorization_route [capacity], processor_route_weight [setpoint], transaction_queue_limit [admission], idempotency_window [setpoint], order_reservation [capacity], inventory_hold [admission], checkout_load_shed [admission], refund_queue_rate [capacity], settlement_batch [capacity], payment_failover [capacity]

### *Data Analytics & ETL (15; plant: compute_pool, batch)*

executor_size [capacity], dynamic_allocation_min [capacity], dynamic_allocation_max [capacity], shuffle_partitions [capacity], shuffle_bandwidth [capacity], etl_concurrency [admission], stage_parallelism [capacity], query_slots [capacity], spill_threshold [admission], cache_fraction [setpoint], batch_interval [setpoint], stream_backpressure [admission], data_locality_wait [capacity], speculation_policy [capacity], analytics_admission [admission]

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

### *Database & Transactions (19; plant: compute_pool, database)*

db_replicas [capacity], db_memory [capacity], db_cache [capacity], query_concurrency [admission], read_route [capacity], shard_placement [capacity], db_failover [capacity], replication_lag_gate [admission], db_write_throttle [admission], db_pool_resize [capacity], transaction_concurrency [admission], lock_timeout [admission], checkpoint_rate [capacity], vacuum_compaction_rate [capacity], parallel_workers [capacity], background_writer [capacity], synchronous_replication [capacity], buffer_pool [capacity], io_capacity [capacity]

### *Medical Imaging & Clinical Systems (12; plant: compute_pool, clinical)*

pacs_archive_tier_target [setpoint], dicom_router_concurrency [admission], ehr_application_replicas [capacity], hl7_interface_queue_limit [admission], fhir_api_rate_limit [admission], reconstruction_gpu_workers [capacity], modality_worklist_timeout [admission], patient_monitoring_gateway_capacity [admission], lab_analyzer_batch_window [admission], telehealth_session_admission [admission], clinical_backup_window [admission], imaging_prefetch_priority [admission]

### *Messaging & Streaming (18; plant: compute_pool, server)*

partition_count [capacity], partition_placement [capacity], producer_quota [admission], consumer_quota [admission], broker_io_quota [admission], message_retention [capacity], queue_depth_limit [admission], consumer_concurrency [admission], producer_batch_size [capacity], fetch_batch_size [capacity], rebalance_rate [capacity], replication_factor [capacity], retry_backoff [admission], dead_letter_divert [capacity], stream_priority [admission], broker_failover [capacity], prefetch [capacity], memory_disk_alarm [capacity]

### *Ports & Maritime Logistics (12; plant: compute_pool, port)*

quay_crane_allocation [capacity], yard_crane_fleet_size [capacity], berth_window_admission [admission], truck_gate_rate_limit [admission], horizontal_transport_fleet_size [capacity], reefer_plug_power_budget [power], shore_power_connection_capacity [capacity], rail_mounted_gantry_dispatch [capacity], container_dwell_priority [admission], vessel_arrival_pacing [admission], stacking_height_target [setpoint], equipment_charging_window [admission]

### *Runtime & Application (17; plant: compute_pool, server)*

worker_count [capacity], thread_pool [capacity], jvm_heap [capacity], gc_budget [admission], connection_pool_runtime [capacity], application_cache_size [capacity], runtime_memory [capacity], async_concurrency [admission], event_loop_workers [capacity], process_count [capacity], request_timeout [admission], background_workers [capacity], runtime_cpu_budget [admission], runtime_io_budget [admission], runtime_restart [capacity], go_runtime [capacity], keepalive_connection_reuse [capacity]

### *Search, Indexing & Vector DB (16; plant: compute_pool, server)*

index_workers [capacity], index_refresh_rate [capacity], segment_merge_rate [capacity], search_concurrency [admission], search_timeout [admission], shard_count [capacity], shard_replication [capacity], shard_rebalance [capacity], vector_search_k [capacity], vector_batch_size [capacity], embedding_workers [capacity], index_memory_budget [admission], query_route [capacity], hot_shard_isolation [admission], search_admission [admission], circuit_breakers [admission]


*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

### *Service Mesh & API Reliability (16; plant: compute_pool, server)*

circuit_breaker [admission], retry_budget [admission], service_timeout [admission], service_concurrency [admission], connection_pool [capacity], traffic_divert [capacity], traffic_mirror [capacity], canary_weight [setpoint], outlier_ejection [admission], health_threshold [admission], dns_traffic_weight [setpoint], session_affinity [capacity], request_hedging [capacity], fault_injection_gate [admission], service_failover [capacity], load_balancing_policy [capacity]

### *Telecom RAN & Edge Radio (12; plant: compute_pool, ran)*

ran_connection_admission [admission], ran_ue_handover [capacity], ran_cell_traffic_steering [capacity], ran_slice_resource_budget [admission], ran_prb_allocation [capacity], ran_scheduler_weight [setpoint], ran_tx_power [power], ran_antenna_tilt [capacity], ran_carrier_enable [capacity], ran_cell_sleep [capacity], ran_du_cu_placement [capacity], ran_fronthaul_budget [admission]

### *Workflow, Logistics & Fulfillment (15; plant: compute_pool, workflow)*

workflow_admission [admission], workflow_worker_rate [capacity], task_queue_rate [capacity], workflow_retry [admission], workflow_backoff [admission], workflow_timeout [admission], inventory_allocation [capacity], fulfillment_route [capacity], warehouse_queue [capacity], carrier_selection [capacity], dispatch_priority [admission], shipment_batch [capacity], route_replan [capacity], sla_escalation [capacity], compensation_action [capacity]

## Organism 5: the four stacked, every duplicate kept, 1,716 muscles

## Organism 6: the whole tower, all 945 muscles once

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 24

# The Domain Map

Every system Omni-Compass can sit on, what it reads, what it pushes, the reflexes that bound it, and where it stands.

## 1. The connector pattern

Omni-Compass is the same engine on every muscle. Each muscle gets one connector with five parts:

| Part | Role | Example (compute) |
|---|---|---|
| Afferent (sense) | What the muscle reports | load, pending work, power, heat |
| Engine | Equations (1)-(7) turn readings into state: error E, coherence U, pressure I_U, stress S, bath B | same six-state engine everywhere |
| Efferent (push) | The equation (2) law and allocation law drive the muscle toward convergence | node count, HPA target, power cap |
| Reflex (shield) | Hard limits checked before any push | never below pod requests, step limits, power limit |
| Kill | Control returns to the muscle's own controllers | restore HPA targets, observe only |

Fit tiers used below:

- **Direct**: Omni can govern the muscle as a supervisory controller.
- **Supervisory**: Omni governs budgets, schedules and envelopes; the muscle's own real-time controller stays in charge of fast inner loops.
- **Advisory**: regulated or safety-critical; Omni recommends until certified (IEC 61508, DO-178C, ISO 26262, medical and financial regulation).
- **Not Omni's problem**: a different kind of problem; stated so no one claims otherwise.

Status: **Wired** (in code, verified), **Partial**, **Open** (connector not yet built).

## 2. Digital infrastructure

| Muscle | Reads | Pushes | Reflexes | Fit | Status |
|---|---|---|---|---|---|
| Compute nodes and node pools | load, pending pods, requests | node count, park or power off | request floor, step limit | Direct | **Wired** |
| Pods via HPA | utilisation, replicas | HPA target | bounds 50-95% | Direct | **Wired** |
| Power caps | watts, site limit | per-node caps | site power limit (I4) | Direct | **Wired** (harness) |
| Heat | temperatures | feeds decisions | thermal limit | Direct | **Sensed** |
| GPUs / accelerators | utilisation, power, temperature, memory | power caps, clocks, partitioning (MIG), packing | temperature, job safety | Direct | Open |
| CPU power states | per-socket power (RAPL) | frequency scaling, sleep states | latency floor | Direct | Open |
| Memory | used vs requested | request right-sizing | OOM margin | Direct | Open |
| Storage | I/O load, latency, capacity | tiering, placement, volume scaling | durability, replica counts | Supervisory | Open |
| Network, load balancers | traffic, congestion, latency | routing weights, traffic shift | capacity per link | Direct | Partial |
| CDN / edge | cache hit, origin load | cache and origin routing | freshness | Direct | Open |
| Batch and HPC job queues | queue depth, deadlines | admission, priority, time shifting | deadline guarantees | Direct | Open |
| Multi-cluster / multi-region | load, cost, carbon per region | placement | data residency | Direct | Open |
| Deployments | release health | pause, roll back | never skip required checks | Direct | Partial (stack harness) |
| Databases, stateful services | load, replication lag | read replicas, pools | consistency limits | Supervisory | Open |
| Control plane health | API latency, etcd load | rate of Omni's own actions | never overload the API | Direct | Open |
| Tenant quotas | usage per tenant | who may grow | fairness bounds | Direct | Open |
| Hardware health | disk and memory errors | early drain of failing machines | availability floor | Direct | Open |
| Observability pipelines | data volume, cost | sampling, retention | required audit data | Direct | Open |
| Security posture | policy blocks | block expansion during holds | I1 | Direct | Partial (sensed) |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

# 3. AI

| Muscle | Reads | Pushes | Reflexes | Fit | Status |
|---|---|---|---|---|---|
| Training clusters | GPU power, progress, network | power caps, packing, checkpoint timing | never kill a run mid-step | Direct | Open (gpu vessels in harness) |
| Training power smoothing | synchronised power swings | ramp limits, staggering | grid ramp limits | Direct | Open |
| Inference serving | request rate, latency, queue, GPU memory | replicas, batch size, GPU sharing | latency SLO | Direct | Open |
| Model routing and cost | cost per request, quality signals | route to smaller or larger models, token budgets | quality floor | Direct | Open |
| Model releases | evaluation results, drift, errors | gate, pause, roll back a model | release gates | Direct | Open |
| Data pipelines, vector databases | backlog, freshness | scaling, scheduling | freshness bounds | Direct | Open |
| AI agents: containment | actions, spend, resources, network use | CPU, memory, network and spend caps, tool permissions | hard caps, instant kill | Direct | Open |
| Carbon-aware AI | grid price and carbon intensity | when and where training runs | deadlines | Direct | Open |

## *AI alignment: what Omni can and cannot do*

Alignment has two parts, and only one is Omni's.

| Part | What it means | Omni's role |
|---|---|---|
| **Value alignment** | The model itself wants and does what people intend: honest, safe, helpful behaviour learned in training | **Not Omni's problem.** A governor outside the model cannot make the model's goals or judgement correct. |
| **Control and containment** | Whatever the model wants, it can only act within bounds: resources, permissions, spend, network reach, a kill that works | **Direct.** This is a governance problem: sense what the agent is doing, bound what it may spend and touch, revoke instantly, keep an audit trail. |

Omni-Compass can be the boundary layer AI runs inside. It cannot be the thing that makes AI trustworthy. Claims must keep that line.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## 4. Facilities and energy

| Muscle | Reads | Pushes | Reflexes | Fit | Status |
|---|---|---|---|---|---|
| Cooling plant | inlet temperatures, chiller load | setpoints, fan curves, chiller staging | inlet temperature limits | Direct | Open |
| Building HVAC | zone temperatures, occupancy | setpoints, schedules | comfort bounds | Direct | Open |
| Facility power distribution | PDU and UPS load | limits for the shield | breaker limits | Supervisory | Open |
| On-site batteries and generators | state of charge, grid state | charge and discharge | reserve floor | Supervisory | Open |
| Power grids: demand response | grid signals, prices | shift or shed flexible load | contractual limits | Direct | Open |
| Microgrids, renewable smoothing | generation, storage, load | dispatch | frequency and voltage limits | Advisory to supervisory | Open |
| EV charging fleets | vehicle needs, grid capacity | charge schedules | departure deadlines | Direct | Open |
| Water and pumping | demand, pressure, energy price | pump schedules | pressure limits | Supervisory | Open |
| Grid frequency and protection | frequency, faults | none | certified protection systems | Advisory | Out of scope for control |

## 5. Physical operations

| Muscle | Pushes | Fit |
|---|---|---|
| Manufacturing lines | scheduling, energy, throughput balance | Supervisory |
| Warehouses and logistics | task allocation, fleet charging | Direct |
| Robot fleets | task assignment, charging, traffic, safety envelopes | Supervisory (never joint-level control) |
| Traffic signals, transit, rail scheduling | timing and schedules | Advisory (safety-certified) |
| Ports, shipping | berth and crane scheduling, energy | Supervisory |
| Agriculture, greenhouses, irrigation | water and climate setpoints | Direct |
| Mining, oil and gas | energy and scheduling only | Advisory (process safety) |
| Semiconductor fabs | tool scheduling, energy | Supervisory |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## 6. Science, space, biology, finance, health

| Domain | What Omni could govern | What it must not touch | Fit |
|---|---|---|---|
| HPC / supercomputers | job queues, power, cooling | none beyond the facility | Direct |
| Quantum-computer facilities | cryogenics energy, calibration scheduling | qubit control | Supervisory |
| Satellites and constellations | power, thermal, data budgets, scheduling | attitude and orbit control (flight software) | Supervisory |
| Spacecraft / astrodynamics | mission resource budgets | guidance and navigation | Advisory |
| Biology: lab automation, bioreactors | setpoints, schedules, cold chain | living-system interventions without validated models | Supervisory |
| Financial markets | risk limits, exposure caps, trading infrastructure | making trades, market control | Advisory (regulated) |
| Hospitals | bed, staffing and energy operations | medical devices, treatment | Advisory |
| Aviation, automotive, nuclear | monitoring | any direct control | Advisory (certification required) |

## 7. The problems everyone in computing has, and Omni's fit

| Problem | What Omni can do | Fit |
|---|---|---|
| Data-centre energy growth | right-size, park, cap, shift work in time and place | Direct |
| Cloud cost overruns and idle capacity | consolidate, right-size, remove padding | Direct |
| GPU scarcity and low GPU utilisation | pack jobs, share GPUs, schedule by priority | Direct |
| Heat and cooling limits | govern IT load and cooling plant together | Direct |
| Grid connection limits for new data centres | power smoothing, demand response, stay under contracted power | Direct |
| Carbon reporting and reduction | carbon-aware placement and timing, measured energy | Direct |
| On-call burnout and alert fatigue | hold the levers people turn at night; fewer, better alerts | Direct |
| Autoscalers fighting each other | single authority | Direct (proven in simulation) |
| Latency and SLO breaches under load | headroom governance, fast scheduling floor | Direct |
| Noisy neighbours, unfair tenants | quotas and growth permissions | Direct |
| Hardware failures | early drain on failure signs | Direct |
| Observability cost and data explosion | sampling and retention governance | Direct |
| Configuration drift | detect and gate, not rewrite configurations | Supervisory |
| Cold starts | pre-warming policy | Direct |
| Data egress costs | placement near data | Supervisory |
| Runaway AI agents and spend | caps, permissions, kill, audit | Direct |
| Security misconfiguration, supply-chain attacks | block risky expansion during holds | Partial (not a security scanner) |
| Model hallucination and bias | none | Not Omni's problem |
| AI value alignment | none | Not Omni's problem |
| Software bugs, technical debt, legacy migration | none | Not Omni's problem |
| Vendor lock-in | a single governor over mixed clouds | Supervisory |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## 8. Order of wiring

1. Digital infrastructure: GPUs, CPU power states, memory, storage, network, job queues, multi-region.
2. Facilities and energy: cooling plant, power distribution, batteries, demand response.
3. AI: training power smoothing, inference serving, model routing, agent containment.
4. Physical operations, then science and space as supervisors.
5. Regulated domains as advisors, until certified.

Each connector follows the same path as compute: wired, benchmarked against today's controls, verified, pre-registered, tested on held-out scenarios, then released.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 25

# The Problem Map

Method: the issue trackers of the projects that run the world's clusters (Kubernetes autoscaler, Karpenter, Kepler, Kueue, Volcano, Knative, KEDA, kubernetes/kubernetes), the public industry reports (Cast AI, Datadog, Uptime Institute, IEA) and recent papers (Meta Llama 3, AI-datacenter power stabilisation). Not all of GitHub: a targeted scan of where the money and the failures are. Status: **have** = in this repository and tested; **partial** = built but not proven live; **missing** = not built.

| # | Problem (evidence) | Size | Best tool today and its gap | Omni-Compass fit | Status |
|---|---|---|---|---|---|
| 1 | **Idle capacity.** Average Kubernetes CPU utilisation 8%, memory 20%, falling; CPU over-provisioning 69%, memory 79% (Cast AI 2026). 83% of container cost goes to idle resources (Datadog) | largest money leak in cloud | Karpenter, Cluster Autoscaler, CAST AI: each fixes nodes only | one authority over replicas, requests, nodes and power | **built** (omnilab/rightsize.py, held-out) |
| 2 | **HPA and VPA cannot be used together on CPU/memory**, "not compatible by design" (kubernetes/autoscaler #1726, #2939, #6060, #6247; coordination asked again in #8493, 2025) | blocks right-sizing for most teams | none; official advice is "don't" | exactly what a single authority solves: one engine owns both numerator and denominator | **built** (omnilab/rightsize.py: HPA+VPA vs one authority, held-out) |
| 3 | **Karpenter consolidation churn**: nodes replaced every 5-10 minutes for 2-3 generations, busy nodes removed instead of empty ones, one shared timer (karpenter #1851, #1019, #2705, #3046; provider-aws #7146, #8868, #7356, #8536) | outages, wasted boots | Karpenter's own timers and budgets | engine-gated release (equation 2) plus dwell; C-throughput cut node churn 50% in simulation | built in simulation (AKS NAP = Karpenter arm, tuning/vendor_compare.py); live Karpenter opponent next |
| 4 | **CPU limits throttle apps** (CFS quota), causing latency and even OOMs (kubernetes/kubernetes #67577, #97445) | latency tails | manual tuning, "remove limits" advice | power-cap muscle must watch latency; our first live run hit this exact problem | have; live runs 3-5 found and fixed the resize permission |
| 5 | **Cold starts / scale-from-zero latency** (knative/serving #4902, #14202, #9104; kedacore/http-add-on #219) | seconds of latency per edge | Knative activator, KEDA | pre-warming decided by the engine from its state (anticipation) | **built** (omnilab/coldstart.py, held-out) |

| # | Problem (evidence) | Size | Best tool today and its gap | Omni-Compass fit | Status |
|---|---|---|---|---|---|
| 6 | **GPU utilisation ~5%** (Cast AI 2026); idle GPUs scattered across nodes (volcano #3948, kueue #5243) | the most expensive silicon idle | KAI Scheduler, HAMi, nvshare, Volcano binpack | GPU power-limit muscle (have, calibrated), packing and sharing (missing) | **built** (omnilab/gpupack.py, held-out) + power limit |
| 7 | **AI training power swings**: 50-75% of TDP in milliseconds per GPU, ramps over 1000 MW/s at gigawatt scale; operators impose power and ramp-rate limits (SemiAnalysis; Uptime; arXiv 2508.14318, 2606.04869) | grid-connection risk, equipment damage | batteries, fast PSUs, manual GPU power caps | "training power smoothing" muscle: ramp-rate limit through GPU power caps, the engine's bath state B is literally a damped oscillator | **built** (omnilab/powersmooth.py, held-out; basin-only arm recorded) |
| 8 | **GPU failures and stragglers**: Llama 3, 419 interruptions in 54 days on 16K H100s, one every ~3 hours; 58.7% GPU-related; slow stragglers undetected (Meta; Lablup 2605.09370) | lost training time | Meta's internal tools, checkpointing | hardware-health muscle: sense straggling, drain early | **built** (omnilab/health.py, held-out) |
| 9 | **Outages**: 54% cost more than $100,000, 1 in 5 more than $1M; power causes 45% of incidents (Uptime 2025) | direct money | SRE runbooks, AIOps | recovery and physical-SLA results in simulation: recovery 58 to 17 min | have (sim) |
| 10 | **Cooling**: industry PUE stuck at ~1.54 for six years (Uptime 2025) | ~35% overhead on every watt | DCIM, DeepMind cooling AI | heat afferent have; cooling-plant muscle | **built** (omnilab/cooling.py, held-out) |
| 11 | **Energy measurement in VMs is impossible**: no RAPL/IPMI in VMs, Kepler building a model server (kepler #2487, 2026) | nobody can prove savings in cloud VMs | Kepler estimators | same gap we hit | **built** (omnilab/vmenergy.py: meter-calibrated attribution, held-out) |
| 12 | **LLM inference autoscaling**: KV cache fills memory before compute; prefill vs decode need separate scaling (vLLM, llm-d) | latency and GPU cost | llm-d, KServe, custom | inference muscle: scale on KV-cache use and queue, not CPU | **built** (omnilab/inference.py, held-out) |
| 13 | **Runaway AI agents**: loops and recursive calls, ~$10,000 overnight examples (Dark Reading; sandbox guides) | cost and safety | per-tool caps, early kill-switch projects | agent-containment muscle: caps, kill, audit | **built** (omnilab/containment.py, held-out); live version next |
| 14 | **Human error**: failures to follow procedures rose 10 points (Uptime 2025) | outages | runbooks | fewer manual actions: pages and human interventions to zero in simulation | have (sim) |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## Results of the build pass (held-out seeds 90001-90030, 30 per muscle, code frozen by SHA first)

Omni-Compass against the strongest native tool in each muscle; + better, - worse, = tie / not significant.

| # | Muscle | Opponent (best native) | Better | Worse | Tie / n.s. |
|---|---|---|---|---|---|
| 1-2 | right-sizing | VPA in-place + HPA | CPU -3%, memory -11%, p99 -99%, reversals -84%, SLO minutes -57% | p95 +15%, OOM kills (few) | work done |
| 5 | cold start | KEDA HTTP | p95 -75%, p99 -58%, delayed >1 s -71%, instance-hours -8% | cold-start count (x8) | |
| 6 | GPU packing | Volcano binpack | idle powered GPU-hours -20%, energy -2% | mean wait +16% (seconds) | p95 wait, jobs, fragmentation |
| 7 | training power | vendor floor, worst-day setting | burn -9%, throughput loss -44%, 1-s swing -6% | max ramp (inside the grid's limit) | 0 grid violations both |
| 8 | GPU health | threshold detection | goodput +10%, lost GPU-h -20%, restarts -29%, straggler hours -99% | | checkpoint overhead |
| 10 | cooling | outdoor-air reset | PUE -2%, cooling -13%, inlet violations -73% | | max inlet |
| 11 | VM energy | host meter split by CPU | per-VM error -37%, worst VM -34% | | total |
| 12 | LLM inference | KEDA on queue | TTFT p95 -18%, SLO breaches -58%, GPU-hours -19%, preemptions -56% | | TPOT, TTFT p50 |
| 13 | agent containment | static caps | rogue spend -96%, time to contain -99%, peak sub-agents -59% | false stops (0.6 -> ~2.7/day), honest work -7% | forbidden actions (0 both: RBAC) |

Engine vs no-engine: in every muscle the omni_no_engine arm scores close to omni. The mapping from engine state to action carries most of the effect; the evolved dynamics add smoothing. The basin-only arm (power smoothing, the bath equation alone setting the draw) cut ramps 63% but did not hold the grid limit: the human-set limit is required.

## What this says

- The problems with the most money behind them are **idle capacity (1)**, **HPA/VPA coordination (2)** and **GPU idleness (6)**. Problem 2 is the cleanest proof of Omni-Compass's thesis: the Kubernetes project itself says its two autoscalers cannot share a metric, because they fight. A single engine owning both is the answer the issue tracker keeps asking for.
- The problem with the most strategic weight is **AI training power swings (7)**: it is new, it threatens grid connections, and Omni-Compass's own equation (7) is a damped second-order bath, the natural controller for ramp limits.
- The problems we already hit ourselves (4, 11) are the same ones the industry has: good evidence the benchmark is real.

## Build order proposed

1. Request right-sizing muscle (VPA role) under the same engine, benchmarked against HPA + VPA "in conflict".
2. Training power-smoothing muscle (GPU power-cap ramp limits) with a synthetic synchronized-training power trace.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

3. GPU packing / fragmentation muscle (with fake-gpu-operator or KWOK on the live cluster).
4. Live Karpenter opponent (kwok provider) to test problem 3 head to head.
5. Inference muscle (KV-cache and queue driven), then hardware-health (straggler drain), then agent containment.

## Sources

- kubernetes/autoscaler issues #1726, #2939, #6060, #6247, #8493
- Karpenter #1851, #1019, #2705, #3046; aws/karpenter-provider-aws #7146, #8868, #7356, #8536
- kubernetes/kubernetes #67577, #97445
- Knative #4902, #14202, #9104; KEDA http-add-on #219
- Volcano #3948; Kueue #5243; KAI Scheduler; HAMi
- Kepler #2487
- Cast AI 2026 State of Kubernetes Resource Optimization; Cloud Native Now on the report
- Uptime Institute annual outage analysis 2025; Uptime global survey 2025
- SemiAnalysis: AI training load fluctuations; Uptime: AI power fluctuations; Power Stabilization for AI Training Datacenters; Source-side mitigation
- The Llama 3 Herd of Models; Tom's Hardware on Llama 3 failures; Lablup 504-GPU report
- vLLM anatomy; llm-d 0.5
- Dark Reading: AI agents and runaway costs

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 26

# The Six Organisms and the Benchmark Grid

Omni-Compass is benchmarked on six organisms, each run native and with Omni-Compass on top on the same seed, the same load and the same clock: Compute / AI / Cloud (430 muscles), Physics / Robotics / Autonomous (376), Energy / Facility / Industrial (470), Distribution / Specialized (440), the four stacked with every duplicate kept (1,716), and the whole tower with every muscle once (945); these are the Omni v2 counts, v1's were 345, 262, 282, 337, 1,226 and 656.

Each organism is run at 1, 10, 100 and 1,000 paired runs and at 1, 10, 100 and 1,000 copies of the organism on one clock. Every receipt shows work per energy, work, energy, the time spent outside the service line, and whether every knob was handed back.

**What the grid has shown.** On Omni v3, 84 of the 90 judged cells are done (`results/scale/GRID.md`): every organism reads superior within its guardrails in every cell of ten runs or more, with work per energy +0.07% (the Physics realm) to +0.37% (the Energy realm, the four stacked and the tower), the work unchanged, the time over the line not above native's, and every knob handed back. The figure is the same at every size, from one copy to a thousand, which is the property the grid exists to test: one brain stays coherent at 1.7 million muscles. The six cells left, 100 and 1,000 runs at 1,000 copies, are beyond the machines available and are declared, not hidden. The number is small because the native controllers in the models are well tuned and leave little room; it is real within the model, and it is never counted in the Omni index, which is made of real stacks only.

On a GPU machine the real card is wired into each organism as one more muscle of its NVIDIA GPU family; its own power meter and its requests are counted in the organism's receipt, kept apart from the modelled plants and also added to them. Real Kubernetes is benchmarked separately on real clusters, and it is also wired into the organisms as one more muscle (`tools/run_kil.py`): at 1 to 100 copies on GitHub's machines and at 1,000 copies on a rented machine, the organism's own compute demand drives a real load generator against a real service on a real cluster, and the cluster's watts come back into the organism as heat and load. Those runs are labelled "L + S": the cluster is live software, the organism around it a model.

The six organisms are models (evidence class S). They show how the law behaves across hundreds of kinds of machine at once and whether its effect is stable as the count and the size grow. Real Kubernetes, the real database, the real broker, the real cache and the real card are the anchors measured on real software and, for the card, a real meter; their results and their losses are in the manual's section 16.

CHAPTER 27

# The Metrics Catalog

Every gauge Omni-Compass produces, where it comes from, and what kind of number it is. Three kinds:

- **Measured**: read from a real system (a real Kubernetes cluster, a real GPU's own meter, a real wall plug).
- **Modelled**: computed by a simulation from declared physics or recorded traces. It shows the mechanism, not a measurement.
- **Internal**: Omni-Compass's own state and decisions, recorded in its audit log on every decision, live or simulated.

Every report says which kind each number is. A modelled number is never presented as a measured one.

## 1. Service: what the customer feels

| Gauge | Unit | Where | Kind |
|---|---|---|---|
| Response time, mean / 95th / 99th percentile | ms | live Kubernetes (`tools/live_reps.py`), GPU bench (`tools/gpu_reps.py`), GPU sims | measured live; modelled in sims |
| Failed requests | % | live Kubernetes | measured |
| Requests served / not served | count | GPU bench, GPU sims | measured / modelled |
| Pod start wait, total and mean | s | live Kubernetes (API server creation-to-Ready) | measured |
| Pending pods | pod-minutes | live Kubernetes | measured |
| Backlog (SLO breach) minutes | min | site and node exchange sims, stack benchmark | modelled |
| 95th-percentile latency factor | × | site and node exchange sims | modelled |
| Availability, time healthy, recovery minutes, recovered | share, min | stack benchmark (`benchmarks/stack_benchmark.py`) | modelled |
| SLA violations (physical, total, backlog, power, heat) | share of time | stack benchmark | modelled |

## 2. Work and capacity: what the operator gets for the money

| Gauge | Unit | Where | Kind |
|---|---|---|---|
| **Work per energy** (served requests per kJ) | req/kJ | GPU bench (the preregistered primary outcome), GPU sims | measured / modelled |
| Work per wall energy (whole machine) | req/kJ | GPU bench with a smart plug (`tools/wall_meter.py`) | measured |
| Work served | work units | site and node exchange sims | modelled |
| Work per kWh | units/kWh | node exchange sim | modelled |
| Energy per served request | J | GPU bench | measured |
| Energy per core-hour | Wh | live Kubernetes | declared model |
| CPU used by the app | cores | live Kubernetes (`kubectl top`) | measured |
| Utilisation (used / allocatable) | share | live Kubernetes | measured |
| Replicas (HPA), mean | count | live Kubernetes | measured |
| Pods started | count | live Kubernetes | measured |
| Worker nodes in service, node-hours, idle node-hours | count, h | live Kubernetes, stack benchmark | measured / modelled |

## 3. Energy and power: the bill and the building

| Gauge | Unit | Where | Kind |
|---|---|---|---|
| GPU energy (the device's own `power.draw`, integrated) | J | GPU bench | measured |
| GPU mean power | W | GPU bench, GPU sims | measured / modelled |
| CPU package energy (RAPL) | J | GPU bench, where the machine exposes RAPL | measured |
| Whole-machine energy at the wall | J | GPU bench with a smart plug | measured |
| Energy on kind | Wh | live Kubernetes | **declared model** (kind has no meter) |
| Energy | kWh | site, node exchange and stack sims | modelled |
| Peak power | kW | site and node exchange sims, stack benchmark | modelled |
| **Site-budget violation minutes** (time over the building's limit) | min | site and node exchange sims | modelled |
| GPU power limit, mean; power-limit writes | W, count | GPU bench, GPU sims | measured / modelled |
| Power-cap travel | share | stack benchmark | modelled |

## 4. Hardware: heat and wear

| Gauge | Unit | Where | Kind |
|---|---|---|---|
| GPU temperature, peak and mean | °C | GPU bench, GPU sims | measured / modelled |
| Heat-over minutes | min | hardware plant sim (`hardware/plant.py`) | modelled |
| Thermal travel | share | stack benchmark | modelled |
| Machine round trips, node starts and stops, scale reversals | count | stack benchmark | modelled |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

Lower power and fewer temperature swings are the conditions for longer hardware life. **No hardware life extension is claimed** until real temperature, duty-cycle and wear records exist.

## 5. Battery equivalent (derived)

A battery holds a fixed amount of energy, so battery life follows directly from work per energy:

- runtime on the same charge, for the same work: × (native energy ÷ Omni-Compass energy)
- work on the same charge: × (Omni-Compass work per energy ÷ native work per energy)

Example: the modelled GPU card at +5.1% work per kJ (`results/gpu/sim/after`) does 5.1% more work on one charge. The same rule turns any measured work-per-energy result into a battery figure. On-site battery banks as an organ of the power budget are designed (`docs/DOMAIN_MAP.md`, "on-site batteries") and not yet built.

## 6. Safety and control: proof it behaved

| Gauge | Where | Kind |
|---|---|---|
| Writes executed per arm (native 0, watch 0, Omni-Compass n) | GPU bench, live Kubernetes | measured |
| Every arm ended at its start setting (the reset restored it) | GPU bench, live Kubernetes switch drill | measured |
| Invariant violations (all, and excluding power) | stack benchmark | modelled |
| Security violations, contradictions, pages | stack benchmark | modelled |
| Freeze check: code hashes at start and end (counted runs) | GPU bench (`FREEZE.json`, `FREEZE_END.json`) | measured |
| SHA-256 of every raw file | every live run | measured |
| The seal: every Python/C++ twin unchanged since proven equal | `results/SEAL.json`, `verify.py` | checked on every build |

## 7. Internal: Omni-Compass's own state, on every decision (audit log)

| Field | Meaning |
|---|---|
| `E, U, I_U, S, B, B_dot` | the six-state engine: energy, coherence (health), unmet-need integral, stress, redistribution, its rate |
| `state_observed`, `state_projected_next`, `prediction_error` | what the device said, what the engine projected, and how far off the last projection was |
| `requested_cap`, `granted_cap`, `shield_bound`, `want_w` | what the engine asked for, what the shield allowed, which bound decided it, the limit written |
| `admissible` | whether the engine was permitted to change anything this decision |
| `speed_lock` (`ratios`, `line`, `aim`, `rate`) | the speed lock's reading against the run without Omni-Compass |
| `convey` engaged / released | when idle CPU is conveyed to serving pods, and when it is given back |
| `authority`: `calm` and its scalars (`kappa, h, sigma, nu`), `execute`, `contract` per organ, and the machine organ's own view | how settled the engine is, whether it may act at all, and which organs may give capacity back this decision (Kubernetes controller; `omnicompass/nervous_system.py`) |
| compass readings | the direction the engine reads the whole system to be moving (`omnicompass/compass.py`) |
| `write`, `would_write`, `why` | every command sent (or, in watch mode, withheld) and its reason |
| `snapshot`, `restored` | every setting read at start, and its read-back after the reset |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

# 8. Where each report lives

| Report | Command | Output |
|---|---|---|
| Live Kubernetes, paired | `benchmark-reps.yml` (commit with `[reps]`) | `LIVE_REPS.md`, `results/live/` |
| GPU bench on a real card | `sudo bash scripts/gpu_paired.sh` | `results/gpu/run-*/GPU_REPS.md` |
| GPU physics sim | `python3 tools/gpu_physics_sim.py` | `results/gpu/sim/` |
| CPU-then-GPU speed lock sim | `python3 tools/gpu_pipeline_sim.py` | `results/gpu/sim/pipeline/` |
| Site power exchange (GPU groups) | `python3 hardware/site_exchange.py` | `results/hardware/SITE_EXCHANGE_*.json` |
| CPU + GPU on one budget | `python3 hardware/node_exchange.py` | `results/hardware/NODE_EXCHANGE_*.json` |
| Full stack benchmark and every check | `python3 verify.py` | `results/`, `VERIFICATION: PASS` |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

PART FIVE

# The Harness and the Wiring

*The universal plug, the adapters, the wire check, and the step-by-step work of wiring Omni-Compass onto a running stack.*

CHAPTER 28

# The Plug, the Adapters and the Wire Check

Think of a high-end car stereo: one head unit, one standard plug on its back, and an adapter harness for each make of car that matches the car's factory plug. You never cut a factory wire, and when you pull the stereo, the car works as it did. Omni-Compass is wired the same way.

**8.1 The plug (one design for every muscle).** Each plug has as many channels as its muscle has knobs. Every channel declares its cover (range), its direction (which way is "more"), its units and how fast it may move. Every plug keeps the same contract (`omnicompass/compass_law.py`, `Plug`):

1. **attach** - read the knob once, before any write: the snapshot. It never moves afterwards.
2. **read** - the service reading, into the compass.
3. **write** - one value, clipped to the cover, then read back from the device.
4. **one writer** - if the knob is found at a value Omni-Compass did not write, someone else owns it: Omni-Compass stops writing and leaves that value alone.
5. **restore** - on stop, the knob returns to the snapshot (where Omni-Compass found it, not the last value it wrote). For muscles whose native controller moves the knob itself, restore hands control back.

**8.2 Adapters (one per standard plug).** Omni-Compass speaks each industry's standard control interface through an adapter. Status is stated plainly:

| Standard | Muscles | Adapter status |
|---|---|---|
| Kubernetes API | HPAs, deployments, pods, nodes | **built** (`omni_controller/controller.py`, `muscles.py`) |
| NVIDIA NVML / `nvidia-smi` | GPU clock ceiling, power limit | **built** (`omni_controller/gpu_compass.py`, two wires; `gpu_governor.py`, one wire) |
| Linux cpufreq, RAPL | CPU frequency ceiling, package watts | **built** (`--cpufreq-policy-root`, `--rapl-cmd`) |
| Site meter, building controller by command | site watts, supply-air setpoint | **built** (`--power-cmd`, `--cooling-cmd` command templates) |
| Cloud node groups (Karpenter, Cluster Autoscaler, MachineSet) | machines | **built** through `--node-scale-cmd` templates |
| BACnet, Modbus, SNMP | chillers, air handlers, PDUs, UPS | designed; today reached through the command templates above |
| OPC UA, EtherNet/IP, PROFINET, EtherCAT, ROS 2, CAN | PLCs, drives, robots, vehicles | designed; modelled in the realm harness |
| IEC 61850, DNP3, OpenADR, IEEE 2030.5, SunSpec, OCPP | substations, batteries, inverters, chargers | designed; modelled in the realm harness |

The engineer's job for one muscle is three things: which adapter, the address of the knob, and its safe range.

**The plug contract, line by line, and why each line is there.** *Attach* reads the knob once before any write and keeps that value as the snapshot. It is read once and never again, because a snapshot that moved with the knob would restore to wherever the governor last left it, and the whole point of the restore is to return to where the operator had it. *Read* takes the service reading (a response time, a temperature, a queue, an error) into the compass as a position in its band; the plug declares the band, the law does not guess it. *Write* takes one value, clips it to the cover before it leaves the plug, sends it through the muscle's own interface, and then reads the lever back from the device and records both the sent and the taken value; a device that took something other than what was sent (a clock that rounds to 15 MHz steps, a pool size the pooler capped) is recorded as it is, not as it was asked. *One writer* compares the lever as found at every read with the value last written; a lever found elsewhere has another owner, and the plug raises `ForeignWriter`, after which the governor leaves that lever alone for the rest of the run and says so in its audit and its exit code. *Restore* writes the snapshot back on stop and reads it back; a restore that does not read back is an error (exit 3), never a silent success. The same five lines are implemented for Kubernetes objects (annotations hold the snapshot on the object itself, so a watchdog can restore from the object without the governor's memory), for `nvidia-smi`, for PgBouncer's console, for Redis's `CONFIG SET`, for the consumer group and for the simulators' knobs, and `tests/test_*` prove each against a fake device.

**8.3 Power-up order.** Read-only first; then take the knobs, the least consequential first; on stop, hand them back in reverse order. In the Kubernetes controller the order is: the latency feed and the cluster's own metrics (read); the HPA target (a setting the HPA accepts and corrects within a minute); the pods' CPU limits (conveyance, in place); the machines (the slowest and most consequential, through the release gate and the verdict). On stop the machines are woken first, then the limits, then the target, so the cluster is never left with fewer machines than its pods need while a setting above them is still being handed back.

**8.4 The wire check (mandatory before any write).** For the GPU it is one command, `sudo python3 tools/gpu_wire_check.py --gpu 0`, and `scripts/gpu_rented_run.sh` runs it first and stops if it fails:

| Step | It proves |
|---|---|
| 1 read | every meter answers: watts, temperature, utilization, limit, clock, top clock |
| 2 snapshot | the start limit is recorded once |
| 3 up wire | with the card busy, a ceiling below its own busy clock pulls the clock down under it; reset, the clock comes back above it |
| 4 down wire | a lower power limit is reported back exactly; the start limit is reported back exactly |
| 5 restore | clocks reset and the start limit read back |
| 6 stop | the governor, stopped by its signal, hands both wires back |
| 7 other writer | a limit set by someone else while the governor runs is left alone, and the governor exits 5 |

Any failure prints the wire, the step and what the device said, and nothing else runs. That turns a wiring fault from guesswork into one named line to fix.

**8.5 Wired right or wired wrong.** The wire check proves the wires move. It does not prove the governor reads the right thing. A correct installation leaves signatures in its own receipts; a wrong one leaves others. Check them on your first paired runs, before you believe any number, good or bad.

| Check | Wired right | Wired wrong, and what to look at |
|---|---|---|
| Watch arm against native | equal on every gauge (watch writes nothing) | different: the probe, the load or the machine differs between arms, not Omni |
| Requests served, failed requests | equal to native | fewer served or more failed: a lever is cutting capacity; check its cover and its sign |
| p95 and p99 response time | at native or faster | slower: the reading or the floors are wrong (rows below) |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

| Check | Wired right | Wired wrong, and what to look at |
|---|---|---|
| GPU: the card's clock while busy | at or above the clock the card reaches on its own | below it: the speed floor is missing or the lid is under the card's own draw |
| GPU: the lid while busy | at or above the card's own busy draw | at the envelope floor while busy: the lid is sized from a curve, not from the card's own meter |
| GPU: credit per write (`GPU_REPS.md`) | compass decisions pace the slack, race decisions cover the bursts; fail-up is rare | fail-up in most decisions: being busy is being read as a breach, or the service line is set too low |
| Kubernetes: machines given back | through the release gate, one per decision, with no pod waiting | machines given back while pods wait, or none at all while the service is far inside its band: check the latency feed, `--slo-ms` and the release gate's reasons in the audit |
| Kubernetes: the reset | every HPA target, replica range, CPU limit and worker back to native, no record left | anything left: an earlier run or another controller wrote the same objects |
| Every lever after the run | at its snapshot | not at its snapshot: another writer, or a restore that failed (exit 3): restore by hand and investigate |
| The feed itself | fresh samples every few seconds | stale or empty: Omni reads it as blind and fails up; fix the feed, not the governor |

**The rule.** If any row reads "wired wrong", the result of that run says nothing about Omni-Compass: fix the wiring, run the wire check and the watch arm again, and only then compare. The first real card run is the worked example: its receipts showed the card's busy clock under its own (736-768 MHz against 861-889), the lid at the 105 W floor under its own 135 W draw, and fail-up in 46% of decisions. Each of those is a row in this table.

**8.6 The wire check on the other stacks.** The GPU has a one-command wire check because its wires are the hardest to get right. The other stacks carry the same proof in a different place: every Kubernetes arm records `kubectl auth can-i` receipts for what the governor's identity can and cannot do (and fails if it can do anything it must not), checks that the cluster holds nothing but the harness, and ends with the reset check that every HPA target, replica range, CPU limit and worker is back at native with no record left; every database, broker and cache arm reads the knob back after every write and hands it back at the end, and the report says "handed back: yes" or the run is invalid; every simulator arm checks that every override was handed back. The watching arm, where it is run, is the wire check of the reading: a watching governor that disagrees with native on any gauge has found a difference in the probe, the load or the machine, not in Omni-Compass.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 29

# The Harness

> **Before you wire anything:** read `DISCLOSURES.md`. Omni-Compass acts only through the wires it is given; it cannot be slapped on. If your paired receipts differ from the published benchmarks in direction, the first presumption is wiring: check the installation with section 8.5 of the manual (*Wired right or wired wrong*).

This tree is the GitHub-ready kit: frozen engine + C++ twin + two plants + live stub.

## What to run

```
pip install -r requirements.txt
python verify.py --quick          # engine, C++, soak, parity
python k8s_controlplane/test_controlplane.py
python -m k8s_controlplane.suite  # elastic / always_on / idle_power
python tests/test_hpa_three_way.py
python tests/test_omni_controller.py
```

The machine's own power (the Linux frequency governor as native, Omni on the frequency ceiling, energy from the processor's meter; root on a machine on the metal, not a virtual one; `docs/CPU_POWER_PREREGISTRATION.md`):

```
sudo python3 tools/run_cpu_power.py --probe           # can this machine run it?
sudo bash scripts/cpu_power_run.sh                    # the three workloads, native and omni, three rep
    ↪ etitions; one folder
python3 tools/cpu_power_abc.py A B C --out V3_CPU_POWER.md   # three runs into the table
```

Fleet plant (Omni as node authority vs HPA+CA / Karpenter-lite):

```
python -m fleet.planetlab --dir fleet/traces/planetlab --scenarios 8 --out /tmp/pl
```

C++:

```
cmake -S cpp -B cpp/build -DCMAKE_BUILD_TYPE=Release &amp;&amp; cmake --build cpp/build -j2
./cpp/build/oc_smoke
```

## Two plants (do not mix the tables)

| Tree | What Omni is | What the energy number means |
|---|---|---|
| `k8s_controlplane/` | On top of HPA+CA (target / gate / park) | Pack and optional CA gate on a 20-node replica |
| `fleet/` | Node-pool authority; CA off | Consolidation vs CA and Karpenter-lite |

Observe must match the native arm on that plant. If it does not, the run is invalid.

## Engine species

Shipped `omnicompass/core.py` is the patent principal form: cubic \(U(1-U^2)\), FIG. 4 command, RK4 with held \(u\). `omnicompass/pools.py` is actuation only (off / hold / park). It does not change the field.

## Not in this harness

Live kube-controller-manager, kind CI, GPU MIG scheduler, facility cooling plant. `omni_controller/` is observe-first against kubectl; tests use `tests/fake_cluster/kubectl`.

See `LIMITS.md`.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 30

# The Wiring Guide

**Before you wire anything:** read `DISCLOSURES.md`. Omni-Compass acts only through the wires it is given; it cannot be slapped on. If your paired receipts differ from the published benchmarks in direction, the first presumption is wiring: check the installation with section 8.5 of the manual (*Wired right or wired wrong*).

**The current manual is** `docs/INTEGRATION_MANUAL.md` (every stack, every level, every switch). This page is kept for reference; where the two differ, the integration manual is current.

**Which knobs to wire in at all:** `docs/WIRING_VERDICTS.md` gives every knob in every published result one of three words from the tables themselves: **write** (wire both ways), **watch** (wire out only: Omni reads, the native controller lives by itself) or **operator's choice** (a trade). A knob that showed nothing, or lost, stays native. **How to wire for superiority:** every knob starts in watch (wire out only); an observation run shows what native does alone; then the brain's own verdict decides on the knob, in real time, whether a notch pays before it is written (`tools/knob_verdict.py` around the engine's verdict; the manual, section 9.4); a knob that cannot prove it pays stays native, and the operator's setting is always free. Where that verdict stands today, knob by knob, is the last table of `docs/WIRING_VERDICTS.md`.

The full step-by-step manual, with the switch, the living band, parking and the pod reflex, is `docs/OPERATOR_MANUAL.md`. This page is the short version.

**Three stages.** Each stage adds only the permissions it needs. A stage is promoted only after its evidence is in, and the reset works at every stage:

```
kubectl -n omni-compass exec deploy/omni-compass -- touch /tmp/omni.kill
```

The switch is one human-operated switch for the whole harness (OFF restores every HPA target, replica range, CPU limit and cordon Omni-Compass changed, and records it; removing the file turns Omni-Compass back ON). It never fires by itself and never switches off one module: a failed decision is skipped, boundaries are held by the living band.

## 0. Build the image

```
docker build -f deploy/Dockerfile -t &lt;registry&gt;/omni-compass:&lt;tag&gt; .
docker push &lt;registry&gt;/omni-compass:&lt;tag&gt;
```

**What the image contains.**

- The engine.
- The nervous system.
- The live controller and its levers.
- The frozen closure-law setting (`/app/law/closure.json`).

**How it runs.** As a non-root user, with a read-only root filesystem and no Linux capabilities.

## 1. Shadow: read-only, decides and logs, never writes

```
kubectl apply -f deploy/install/omni-compass.yaml      # set the image line first
kubectl -n omni-compass logs deploy/omni-compass -f
```

**Proof of read-only.** `scripts/pilot_shadow.sh` runs the same stage from a workstation with `kubectl auth can-i` receipts that the identity cannot write, and writes `SHADOW_REPORT.md`.

**Run length.** One to two weeks.

**Pass condition.** Zero writes, and recommendations you agree with.

## 2. Target: Omni-Compass sets each HPA's CPU target

```
kubectl apply -f deploy/rbac-target.yaml               # adds: patch horizontalpodautoscalers
kubectl -n omni-compass patch deploy omni-compass --type=json -p \
  '[{"op":"replace","path":"/spec/template/spec/containers/0/args/1","value":"target"}]'
```

**What happens.** The HPAs keep scaling pods. Omni-Compass only moves their target within bounds.

**The SLO reflex.** Add `--latency-file` and `--slo-ms`, and the target is never tighter than native while the SLO is breached.

**Pass condition.** Latency no worse than native, and fewer pod-hours.

## 3. Node pool: Omni-Compass sizes the node pool

**Add permissions.** `patch nodes`, `create pods/eviction` (`deploy/kind/rbac-omni.yaml` is the tested example).

**Set the mode.** `--mode nodepool --node-scale-cmd "<your pool resize command with {n}>"`. Examples of the resize command:

- a Karpenter NodePool limit;
- a cloud node-group size;
- `scripts/kind_nodepool.sh` on kind.

**What guards a release.** The machine organ gives a node back only when all of these hold:

- every sense is live;
- its last order landed;
- pods are not scaling up;
- nothing is pending;
- the remaining nodes stay at or below the engine's target utilisation.

A PodDisruptionBudget on each service is required, because drains go through the eviction API.

**Optional levers.** Each has its own flag and permissions, and all are listed in `omni_controller/muscles.py`:

- right-sizing;
- cold start;
- batch pacing;
- agent containment;



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

- cooling;
- CPU frequency.

## Evidence, and where it is

| Evidence | File |
|---|---|
| The controller, every lever and the Unified Control Switch on real Kubernetes (kind) | `results/live/LIVE_LEVERS_2_NERVOUS.txt` |
| Read-only shadow on real Kubernetes: every decision logged, 0 writes | `results/live/LIVE_SHADOW_1.txt` |
| Least-privilege identity receipts | `rbac_omni.txt` in every live run |
| Native vs Omni on top vs Omni alone, paired on one machine, real Kubernetes | `results/live/LIVE_PAIRED.md` |
| The full wiring manual | `docs/OPERATOR_MANUAL.md` |
| The frozen engine's results, three runs each, read by rule | `results/live/V1_*.md`, `results/live/V3_*.md`; `docs/OMNI_V1.md`, `docs/OMNI_V3.md` |
| The same wiring on Azure's managed Kubernetes (the bill), a database pooler, a rented machine, the simulators, a message broker's consumer group, a cache's memory ceiling, a drone swarm's autopilot | the manual, section 10 (`docs/OMNI_COMPASS_MANUAL.md`) |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 31

# Before You Start, and the Eight Levels

## What your system needs

| You run | You need |
|---|---|
| Kubernetes (vanilla, EKS, GKE, AKS, OpenShift/OKD, Rancher, kind) | Kubernetes 1.34 or newer (in-place pod resize); metrics-server (`kubectl top nodes` works); an HPA with a CPU target on each governed service; a readiness probe and a short preStop pause on each service |
| A response-time feed (strongly recommended at every level) | a CSV per service, `elapsed_seconds,latency_ms,ok`, written continuously; `scripts/latency_probe.py` writes one against any HTTP endpoint |
| NVIDIA GPUs | a driver with `nvidia-smi`; root (or the capability to run `nvidia-smi -pl` and `-lgc`); on a VM, full GPU passthrough |
| CPU power control | Linux cpufreq with the schedutil governor; RAPL for package watts; usually bare metal |
| Site power and cooling | a command that prints site watts, and a command that sets the supply-air setpoint through your building management system |

**Why each requirement is there.** Kubernetes 1.34 or newer is required because the conveyance muscle resizes a pod's CPU limit in place, which older versions do only by restarting the pod, and a restart is a disruption Omni-Compass never causes. metrics-server is the HPA's own eyes and the governor's: without it neither knows what the pods use. An HPA with a CPU target on each governed service is the native controller Omni-Compass sits on; a service without one has nothing for level 2 to move. A readiness probe and a short preStop pause are what make a drain invisible to clients: the Service stops sending requests to a pod before it stops, and kube-proxy sends them to another ready endpoint. The response-time feed is the one wire a cluster does not already have: Kubernetes knows what its pods use, not what its users wait; the compass needs the second, and `scripts/latency_probe.py` writes it against any HTTP endpoint every five seconds. On the GPU, root is needed only for the two `nvidia-smi` writes, and a VM must pass the whole card through or the driver refuses to lock clocks (the wire check's step 3 is where that shows).

## Get the software and check it

```
git clone https://github.com/The-Omni-Compass-LLC/The-Omni-Compass.git
cd The-Omni-Compass
pip install -r requirements.txt
python3 verify.py                       # every check; ends with VERIFICATION: PASS
```

Build the container image once:

```
docker build -f deploy/Dockerfile -t &lt;your-registry&gt;/omni-compass:&lt;tag&gt; .
docker push &lt;your-registry&gt;/omni-compass:&lt;tag&gt;
```

It runs as non-root, with a read-only root filesystem and no Linux capabilities.

What `verify.py` checks is worth knowing before it is run, because its last line, `VERIFICATION: PASS` or `FAIL`, is the only thing a reader has to read. It runs every test in `tests/` (the law, the plugs, the readings rule, every benchmark's own harness against a fake server), checks the engine's fingerprints against the preregistered registry so that a changed engine file fails before anything is measured, compares the release's files with its manifest (`tools/release_manifest.py --check`: the engine, the C++ twins, the live evidence's digests, one object that names the commit it was written at) and runs the layout check (`tools/layout_check.py`: the declared root, every link and every named path present). It takes a few minutes on a laptop and needs no network beyond the clone. GitHub runs the same script on every push under Python 3.12, and each run is listed under the repository's Actions tab with the commit it checked; a result in this manual is only ever cited from a commit whose run passed.

To know which engine a checkout carries, ask it:

```
python3 tools/omni_version.py                  # omni-v3, omni-v2, omni-v1, or what differs
python3 tools/omni_version.py --commit &lt;sha&gt;   # the same question of any pushed commit
```

Every result table in `results/live/` cites the commit it ran on, and the three-run tools refuse to combine runs from different engines (Chapter 15). The founder's standing order is that no result is ever read across engines, and the command above is how a reader checks that for themselves.

## The levels

Take them in order. Each has a pass condition; go to the next level only when it holds. The OFF switch works at every level.

The levels exist because a governor earns authority the way a new operator does: by watching first, then by touching the setting that is cheapest to undo, then the next. Level 0 touches nothing and proves the software is the software (`verify.py` checks every sealed file's fingerprint and runs every test). Level 1 watches a live cluster for as long as the operator likes and logs every decision it would have taken, so the operators can read those decisions against what they would have done; nothing passes level 1 until they agree with its readings. Level 2 moves the HPA target, which the HPA itself corrects within a minute if the governor is wrong, and raises replica floors ahead of bursts; it is the first level that can show a gain and the first that can show a loss, and the paired receipts of Chapter 41 are its pass condition. Level 3 moves machines, the slowest lever, through the release gate and the verdict; it is where the bill moves and where the most care is spent. Level 4 hands Omni-Compass the replica decision itself, with the HPA kept as the fast reflex upward; it is an option, not a step on the way, and the benchmarks in this manual do not use it. Levels 5 to 7 are the GPU, the CPU and the site, each with its own wire check and its own pass. An installation may stop at any level and run there indefinitely; most will stop at level 3.

**Level 0 - Evaluate without touching anything.**

```
python3 verify.py
python3 tools/run_scale.py --runs 10 --scale 1 --out results/scale/eval     # the six organisms, modelle
    ↪ d
python3 tools/run_gpu_card.py results/sim/gpu_two_wire/eval                  # the two-wire card, modell
    ↪ ed
```

Pass: `VERIFICATION: PASS`. Nothing in your systems is touched.

**Level 1 - Watch (read-only).**

```
kubectl apply -f deploy/install/omni-compass.yaml     # set the image line; it runs --mode observe
kubectl -n omni-compass logs deploy/omni-compass -f
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

The install file grants a read-only identity; `scripts/pilot_shadow.sh` records `kubectl auth can-i` receipts that show it cannot write. Every decision it would take is logged with its reason. Pass: zero writes, and readings your operators agree with.

**Level 2 - The pods, on top of your autoscalers. WRITES.**

1. Grant `patch horizontalpodautoscalers` and `patch pods/resize` (`deploy/rbac-target.yaml`).
2. Run with `--mode target --latency-file <feed> --slo-ms <your p95 target>`.
3. Optional pod muscles, one at a time:

| Muscle | Switch | What it does |
|---|---|---|
| convey | `--latency-file` and `--cap-deployments ns/name` | gives each machine's idle CPU to the serving pods on it |
| rightsize | `--rightsize-deployments ns/name` | each pod's CPU request follows its measured use times (1 + headroom), in place |
| coldstart | `--coldstart-deployments ns/name --coldstart-signal ns/configmap` | scales a service to zero while no work waits, wakes it when work arrives |
| batch | `--batch` | admits held Jobs labelled `omnicompass.io/batch=true` when there is load and power headroom |
| batch pace | `--batch-pace` | pauses Jobs labelled `omnicompass.io/pausable=true` under power or heat stress, resumes them after |
| rollout guard | `--rollout-guard ns/name` | pauses a rollout while change is not permitted |
| contain | `--contain-namespaces ns --contain-cpu-m <m>` | holds an agent namespace to a CPU budget |
| security hold | `--security-configmap ns/name` | key `hold: "true"` blocks every expansion |

Your HPAs keep scaling as before; Omni-Compass sets their targets and raises floors ahead of bursts. Pass: p95, p99 and failed requests no worse than native, over paired runs (Chapter 41).

**Level 3 - The machines. WRITES.**

1. Grant `patch nodes` and `patch pods` (`deploy/kind/rbac-omni.yaml`).
2. Run with `--mode nodepool --active-nodes-only --closure /app/law/closure.json --node-scale-cmd "<command with {n}>"` and `--node-restore-cmd "<command>"` for the OFF switch. For the compass law on the same levers, use `--law compass` (reads the mean response time of the latency window between a tenth of `--slo-ms` and `--slo-ms`, held at `--compass-center 0.4`; p95 at the SLO, a blind feed or a waiting pod adds capacity at once; the HPA target never goes above the operator's; machines go back one at a time through the release gate).

| Your platform | The park/wake command |
|---|---|
| any cluster, bare metal, kind | `bash scripts/kind_nodepool.sh {n}` |
| Karpenter / EKS Auto Mode | the NodePool CPU limit at `{n}` times node CPU, parked nodes kept |
| Cluster Autoscaler node group | the group's desired size, scale-down through parking |
| OpenShift / OKD | the worker MachineSet replicas, parked |

A machine is given back only when every sense is live, the last order landed, pods are not scaling up, nothing waits for a place, and the remaining machines stay inside the band. Pass: fewer machines in service, with no service gauge worse.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

**Level 4 - Omni-Compass decides; Kubernetes is the muscle. WRITES.** Add `--strict-replicas`: Omni-Compass decides each service's replica floor and when to shrink; the HPA stays as the fast reflex upward. Pass: as level 3.

**Level 5 - A GPU box, two wires. WRITES.**

```
sudo python3 tools/gpu_wire_check.py --gpu 0                    # must end: WIRED RIGHT
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

CHAPTER 32

# Stack by Stack

Every wiring below is the same plug (Chapter 28): one wire in (the stack's own meter), one wire out (one setting the stack already accepts), a snapshot taken once before the first write, every write read back, everything restored on OFF. Native is always the stack's own controller left as shipped; omni is that controller with Omni-Compass on top.

| Stack | Levels | Notes |
|---|---|---|
| Vanilla Kubernetes, kind, Rancher | 1-7 | as written |
| Amazon EKS | 1-5 | machines via Cluster Autoscaler node group or Karpenter; CPU power control is not exposed on EC2 VMs; GPUs on bare-metal or full-GPU instances |
| Google GKE, Azure AKS | 1-5 | the provider's node-pool size as the park/wake command; on AKS the managed cluster autoscaler stays native and deletes the machines Omni-Compass idles (10.1) |
| Red Hat OpenShift / OKD | 1-5 | the same permissions through a Role; machines via the worker MachineSet |
| NVIDIA GPU servers without Kubernetes | 5 | the two-wire GPU governor alone |
| Bare-metal CPU servers | 6 | cpufreq and RAPL through sysfs |
| Slurm / HPC | 5 on the GPU nodes | a job-level Slurm connector is not built |
| Building management | 7 | through your BMS's command line or API, in the command templates |
| PostgreSQL behind PgBouncer (any database behind a pooler with a console) | one knob | the pool size through the pooler's own admin console (10.2) |
| A power grid's substation tap changer (pandapower, SimBench) | one knob | the tap position, one whole tap per move (10.3) |
| Buildings with batteries (CityLearn) | one knob a building | the battery's charge and discharge inside the simulator's own limits (10.3) |
| Robot arms (MuJoCo Menagerie) | one knob an axis | the joint speed where the axis has slack; never on an axis at its takt (10.3) |
| A rented cloud machine running the big organisms | the whole harness | the detached run: rent, run, collect, delete (10.4) |
| A message broker's consumer group (Apache Kafka) | one knob | the consumer count inside [1, partitions], read from the consumers' own records; the rebalance paid on every move (10.5) |
| A cache's memory ceiling (Redis) | one knob | `maxmemory` through the cache's own console, grown only while the cache is full (10.6) |
| Drone swarms (gym-pybullet-drones; PX4 and ArduPilot next) | one knob a drone | the cruise override inside the autopilot's limits; a separation wall; collisions void the cell (10.7) |
| A database's storage-engine cache (MongoDB under YCSB) | one knob | the WiredTiger cache size through the server's own console, grown only while the cache is full (10.8) |

## Managed Kubernetes on Azure: the bill as the gauge

On Azure Kubernetes Service the muscle is the same as on kind (Chapter 31, levels 1 to 3) and one thing is new: the bill is real. The workflow `aks-metered` builds a fresh cluster for every arm the same way: a system pool of one machine, tainted so no workload lands on it (Azure's add-ons and the load generator live there; it is never measured and never governed), and one or several **work pools** under Azure's own cluster autoscaler, every pool labelled `omni-role=work`, every pool starting full. Native is that autoscaler alone. Omni on top is the same autoscaler with Omni-Compass idling the machines it gives back, which the autoscaler then deletes on its own schedule. The bill counts every work machine that exists, every 15 seconds, priced at Azure's list price for its family (`tools/live_reps.py`, `POOL_PRICES`); each cluster is deleted when its arm ends, so nothing keeps billing.

**Sizing the fleet to the lever.** The smallest saving a fleet can show is one machine. On 4 workers one machine is 25% of the fleet, so only a saving of about 30% or more can clear the noise; on 15 workers about 10%; on 96 about 3%. The expected machine saving under Omni-Compass is a few percent, so the 4-worker runs (`results/live/V1_AKS_STEADY.md`, `V1_AKS_BURST.md`) read no difference beyond the noise on every gauge, which is the honest reading of a fleet too small for the lever, not a result against it. A new subscription allows 200 vCPUs in a region but 10 per machine family, so one family gives five 2-vCPU workers; the way past it is **several families at once**: eight pools of four to five workers each make a fleet of 39 (`worker_pools` input, `size:max,size:max,...`; `pool_prices` names each family's price). The app's replica ceiling is raised with the fleet (`hpa_max`: nine 200-millicore pods fit one 2-vCPU worker, so 9 × workers lets the fleet fill), the same in both arms. The preregistered design is in `docs/K8S_COMPASS_PREREGISTRATION.md`, "The fleet that can show one machine"; the account setup, click by click, in `docs/AZURE_SETUP.md`.

**What the Azure runs taught us.** Three lessons, each recorded as an amendment to the preregistration before the fleet was dispatched again. First, a fleet is sized by the lever, not by the budget: a 4-worker fleet cannot show the few-percent machine saving the law makes, so its honest reading is "inside the noise", and running it again would not change that. Second, a new subscription's allowance is per machine family, so the fleet that can show one machine is built from nine families at once, each pool at its own list price, and the bill prices each pool separately. Third, a cloud region can refuse a new cluster for hours at a time (Azure's capacity for new control planes in its region, on both control-plane tiers); the harness therefore checks the subscription's allowances before it builds anything, so a refusal costs cents and not a cluster, and every refusal is recorded with its time.

## A database behind its connection pooler

PostgreSQL is wired through PgBouncer, the pooler most of its deployments already run (`docs/POSTGRES_PREREGISTRATION.md`, `tools/run_pgbench.py`). Nothing models the database. The wire in is the pooler's own report (`SHOW STATS`, `SHOW POOLS`): once a second, the time a transaction spent in the server plus the time clients waited for one, over the transactions completed. The wire out is one setting through the pooler's own admin console, `SET default_pool_size`, inside the cover [2, 90]: never under two server connections, never within ten of PostgreSQL's connection limit. The compass holds that reading at 40% of a 50 ms line. Where the time was spent decides the direction: waiting for a server grows the pool; time inside the server shrinks it by one, so fewer backends fight for the same cores; calm gives back one idle server a second, and only one the pooler itself shows idle. At 95% of the line the knob is handed back to the pooler's own setting at once. If the knob is found at a value Omni-Compass did not write, it stops writing (one writer). At the end of every omni arm the pool size is restored to the snapshot and read back. The same plug fits any pooler or proxy with a console and a pool-size setting (PgBouncer, Pgpool-II, ProxySQL, the application's own pool).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

**What the database taught us.** The first untouched run on this stack was a loss, and it stays in the preregistration as it came (`docs/POSTGRES_PREREGISTRATION.md`, "The first untouched run, and amendment 1"): on the read-only workload the pooler's own clock read 0.11 ms a transaction against a 50 ms line, so the law gave back one idle connection a second until the pool stood at its floor of two, while the users' 95th percentile went from 4 ms to 3.9 s. The pooler could not see the time a client spent behind its own schedule waiting to send, so its clock said calm while the users waited; the preregistration had said so in one line without drawing the consequence. Amendment 1, declared before any counted run, made the reading the worse of two: the pooler's service time, or the share of its clients queued for a server, so a pool too small for the offered rate shows at the pooler as clients waiting and the knob fails up to the operator's setting at once; and it held the dwell on the brake only, never on the gas. The counted runs are three new untouched runs on the amended rule; the first run is not counted and is not hidden. The result the same in all three of our runs is a resource held, not speed, and its cost the same in all three of our runs is the host's CPU: the compass's own reads of the pooler's console every second are CPU the native pooler never spends, and they are counted against it.

**What the cost turned out to be (amendment 2, 8 October).** The record had explained the +14% to +28% of host CPU as "PgBouncer queuing clients behind a smaller pool", without a measurement. Measured, it was ours. The pooler's own log of one counted workload holds 7,956 logins to its admin console, one for every `psql` process the harness launched to take a reading (about two a second in both arms for the sampler, four more a second plus three per write in the omni arm for the brain); one launch costs about 52 ms of CPU on a machine of the runner's class, because psql loads libpq, OpenSSL, GSSAPI and LDAP and connects before it runs one command; and a paired repetition of `select` with a per-process meter put +55.9 CPU-seconds on those launches against a +52.8 s difference in the host total, with PgBouncer at +1.2 s and PostgreSQL at −4.7 s. The second cost in the table, the median latency +15% to +17%, was the take-back rule giving back one idle server a second whenever the pooler's service time read calm, which against a 50 ms line it always did, so the pool was shrunk into a queue the pooler itself was reporting. Amendment 2, declared before the second counted set (`docs/POSTGRES_PREREGISTRATION.md`): the harness speaks the pooler's wire protocol over one connection held for the whole arm, in both arms, so no process is launched for a reading; the queue line: a server is taken back only while the pooler's clients waited for one under one percent of its time in the last second and a transaction was served, and while they waited one percent or more servers are added back, one for every percent of waiting, up to the pooler's own setting (the one-percent line the two other database tests already use), so the pool probes down to the smallest size that serves without waiting and gets its servers back the moment anyone waits; and the harness records its own CPU beside the host's. The engine's 40 files are untouched (`tools/omni_version.py` prints omni-v3). The second set's table replaces the first when it lands, and the first moves whole to `docs/history`.

## Independent simulators, each with its own native controller

Three published simulators are wired the same way, and the simulator's own controller is always native:

- **CityLearn** (`tools/run_citylearn.py`): every district the simulator ships; the knob is each building's battery charge and discharge inside the simulator's own limits; the native arm is the simulator's own rule-based control.
- **pandapower on SimBench grids** (`tools/run_pandapower.py`): the knob is the substation tap changer, one whole tap per move and a week of evidence before stepping down; the native arm holds the tap at 1.00 per unit as the grid ships.
- **MuJoCo Menagerie robot arms** (`tools/run_mujoco.py`): the knob is the joint speed of an axis that has slack; the paired physics trial written before the run leaves an axis at its takt native, so UR5e and iiwa 14 have nothing for Omni-Compass to move and the Gen3 and Panda do.


© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

Each is preregistered (`docs/CITYLEARN_PREREGISTRATION.md`, `docs/PANDAPOWER_PREREGISTRATION.md`, `docs/ROBOTICS_PREREGISTRATION.md`), each runs A, B and C as separate GitHub runs, and each table is made by rule (`tools/citylearn_abc.py`, `tools/pandapower_abc.py`, `tools/mujoco_abc.py`). These are evidence class S: statements about the simulator's model, never about hardware.

**What the simulators taught us.** Three things a referee should weigh. First, a deterministic simulator makes the three-run rule stricter, not looser: MuJoCo and pandapower reproduce to the digit on any machine, so their A, B and C must agree to the digit and any difference is a finding about the harness, while PyBullet reproduces only to a part in a thousand across machines, which the swarm table discloses as an amendment and tolerates by that much and no more. Second, a knob that cannot pay is left alone by the physics, not by a rule of thumb: the robot arms' paired trial showed that two of four arms gain nothing from a speed override and left them native, the cooling model showed that a cap on cooling power can never remove less heat and so is never a lever, and a battery's reserve is spent only at the wall because every round trip through the cells costs the efficiency loss (`docs/MECHANISM_OF_ACTION.md`, 9.6). Third, a gain in one gauge is often a loss in another, and the simulators show both: the grid's losses rise in the four grids with their own generation while the energy drawn falls everywhere; the buildings' bills rise in the 2023 districts while their peaks fall; the Panda's copper loss rises while its torque falls. The tables carry every one of those rows, and the index carries none of them, because they are models.

## The big organisms on a rented machine

The four stacked at 1,000 copies (1.7 million modelled muscles) with a real cluster inside takes more than GitHub's six-hour job limit per repetition, so it runs **detached** (`.github/workflows/big-organism-detached.yml`): `start` rents one Azure machine, installs the tools, starts every repetition of every organism under `nohup` and leaves the machine running on its own; `collect` (every two hours on the clock, or by hand) copies the files back when every repetition has finished, publishes one artifact per cell and deletes the machine; `survey` lists which machine sizes and quotas a subscription may rent, region by region, and rents nothing. A machine older than `max_hours` is collected as it is and deleted, so nothing runs forever on the bill. The commit to run is an input, so an older engine can be run again on the same machine. **The clock rule** (Chapter 41) decides the window for each size on each machine: the first v3 machine stepped the stack in 31 s against a 12 s step and fell 4,628 s behind its window, so it was stopped, recorded, and started again with a 10,800 s window and 45 s steps. That second machine is the one that produced the result in section 2.5: three repetitions over 29 hours on one eight-core machine, both arms about 22 s behind a 10,800 s window, no repetition off the clock.

The same workflow carries a second test. With the input `test=robust` the machine runs the robustness harness's long scenario instead of an organism (`scripts/kind_paired.sh` with `ROBUST=long`, exactly as the GitHub job runs it), with the window stretched to whatever `duration_s` asks and the wandering load schedule repeated to fill it; the 24-hour run of section 16.8 uses it, one paired repetition on each of three rented eight-core machines of three families (the start job tries the size it is given and falls back through the eight-core sizes the subscription may rent when the allowance refuses it; the four-core size asked for was refused because the tower's machine held its family's allowance, and the preregistration records what was rented), collected and deleted at 60 hours at the latest. The cells it writes are named `robust-long-x<window>-<repetition>` beside the organisms' `six-<organism>-x<copies>-<repetition>`, and the collect job gathers both. The `max_hours` input is the machine's own limit, written into the machine at start, so a collect that finds a machine past its limit takes what is there and deletes it, whichever job started it.

Three things a reader should know about any detached result. The machine is one Azure virtual machine rented for this run alone, not a GitHub runner shared with a queue of other jobs, so its size is known (the table names it) and it is not cut off at six hours; that, and not only the job limit, is why the big organisms are run there. The files come

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

back exactly as the machine wrote them, copied cell by cell with the machine's own log beside them, and the archive bot stores them under the collect run's number with a SHA-256 list of every file, so the start run and the collect run are both cited in the table. And the machine is deleted after the collect, so a result cannot be re-read from the machine, only from the archived files; the preregistration says this before the first start.

## A message broker's consumer group

Apache Kafka is wired as an operator runs it (`docs/KAFKA_PREREGISTRATION.md`, `tools/run_kafka.py`): one broker as shipped, a topic of eight partitions, a consumer group at the operator's count of two. Nothing models the broker. The wire in is the group's own end-to-end latency: each message carries its production time, each consumer records when its handling ended, and the reading is the mean over the messages consumed in the last second; a second with nothing consumed while messages wait reads as the line (someone is waiting and no one is taking), and nothing consumed with nothing waiting reads as calm. The wire out is the count of consumers in the group, which is what an operator changes, inside the cover [1, 8]: never fewer than one, never more than the topic has partitions. The compass holds the reading at 40% of a 500 ms line with a response time of 3 s, because a consumer added or removed shows only after the group rebalances. Slow service with messages waiting adds consumers in proportion to the force; calm with nothing waiting gives back one idle consumer a second after a five-second dwell; at 95% of the line every consumer the topic can use starts at once. The rebalance that every move costs is paid by Omni-Compass and counted in the gauges. At the end of every omni arm the count is handed back to the operator's and read back. The result (Chapter 43) is the shape this wiring predicts: the queue kept short, no message lost, at the cost of consumers running, which reads worse by rule and stands beside the gain.

## A cache's memory ceiling

Redis is wired as Ubuntu ships it, with the two settings an operator sets for a cache, a memory ceiling of 64 MB and the allkeys-lru eviction rule (`docs/REDIS_PREREGISTRATION.md`, `tools/run_redis.py`). The application in front of it is ours and declared: 1,500 requests a second from 32 threads on a Zipf working set that widens and narrows one notch at a time; a miss costs a declared 5 ms trip to the store behind the cache and a write-back. The wire in is the application's own request latency, the mean over the last second (a hit about 0.3 ms, a miss about 5.5 ms). The wire out is one setting through Redis's own console, `CONFIG SET maxmemory`, inside the cover [16, 512] MB. The compass holds the reading at 40% of a 2 ms line. Its direction rule carries the do-no-harm gate in the form this stack needs: misses grow the ceiling by notches of 8 MB **only while the cache is full** (memory used at 90% of the ceiling or more), because a miss in a cache with room to spare is a cold miss that no ceiling can mend; calm with nothing evicted in the last second gives back one notch a second, after a five-second dwell since the last growth; at 95% of the line with the cache full a quarter of the cover is added at once. One writer: a ceiling found at a value Omni-Compass did not write stops it. At the end of every omni arm the ceiling is handed back to the operator's 64 MB and read back. The memory the compass holds for a wide working set is the resource this benchmark trades and reads worse by rule; the gain is the hit rate and the work inside the line. The result (section 16.4) is exactly that shape: on all three untouched workloads the work inside the line and the hit rate rose 14% to 27% and the ceiling held rose from 64 MB to 200 to 270 MB, with the host's CPU inside the noise and every ceiling handed back.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## Drone swarms: the autopilot's cruise

In gym-pybullet-drones (University of Toronto, MIT License; `docs/SWARM_PREREGISTRATION.md`, `tools/run_swarm.py`) every Crazyflie 2.x flies its shipped position controller along the planner's path at the planner's cruise: that is native. The wire in is each drone's tracking error, the distance between the drone and the point its autopilot was told to be at, on a band from 0 to a declared safe error of 0.25 m. The wire out is one knob a drone, the cruise override: the fraction of the planner's cruise the carrot moves at, never under 1.0 (the planner's own) and never over 2.0 (a quarter of the autopilot's own speed limit), moving by at most 0.01 a control tick. With slack the compass spends it as speed; as the error grows it eases; at 95% of the safe error the cruise is the planner's at once. Two guards belong to this stack: a **separation wall** (a drone with another within 0.5 m flies the planner's cruise, whatever the error reads) and the **paired physics trial** before the counted missions, which asks the simulator's own figures whether a faster mission costs less energy at all and keeps the error under the safe error; where it does not, the override is left native for that cell and the cell reads "nothing for Omni to move". Energy is a declared model from the simulator's own motor constants, said to be a model wherever it is printed; a collision voids the cell. At every landing and at the end of the run the override is handed back and read back.

## A database's storage-engine cache

MongoDB is wired as an operator runs it (`docs/YCSB_PREREGISTRATION.md`, `tools/run_ycsb.py`): the 8.0 series from its publisher's own signed repository, its configuration as shipped but for the one setting an operator sets for the storage engine, the WiredTiger cache, at 512 MB. The questions are asked by YCSB, the published cloud-serving benchmark, running its own core workloads as the project ships them, with the key space stepping one notch at a time through the cache and past it, so the working set fits the operator's cache at the low notches and outgrows it at the high ones. The wire in is the server's own mean read latency over the last second, from its `serverStatus` operation latencies, differenced. The wire out is one setting through the server's own console, `setParameter wiredTigerEngineRuntimeConfig cache_size`, inside the cover [256 MB, 2,048 MB]: the server's own floor at one end, a quarter of the machine at the other. The compass holds the reading at 40% of a 1 ms line (written as 2 ms before the first smoke run and set at 1 ms on the tuning workload's figures, as the preregistration reserved; see below). The direction rule is the cache's: slow reads grow the cache by notches of 64 MB only while the cache is full (bytes in it at 90% of its size or more), because slow reads in a cache with room to spare are not the cache's to mend; calm with no page evicted in the last second gives back one notch a second after a five-second dwell; at 95% of the line with the cache full a quarter of the cover is added at once. One writer, read-back and the hand-back at the end are the plug's, as everywhere. *Amendment 1 (8 October, declared before the second counted set): the give-back gate is now the cache's miss share, pages read into it under one percent of the pages requested from it in the last second, the MySQL test's gate, and growth is gated on missing the same way; the first set's audits showed that "no page evicted" is satisfied by every cold cache while it fills, so every burst and c arm gave four notches back in its first five seconds, before its first eviction, and then sat at the floor of the cover evicting for the rest of the arm. The memory the first set credited was that artefact, and the second set is expected to credit less, perhaps nothing beyond the noise.* Disclosed before the run: on a machine where the data files fit in the operating system's own page cache, a storage-engine cache miss is a read from memory and a decompression, not a disk read, so the gain available to this knob is smaller than it would be on a machine whose data does not fit in memory; the result will say what it is. The same plug fits any store whose engine exposes a cache size at run time (InnoDB's buffer pool, RocksDB's block cache) and any store that does not can only be sized at restart, which is not a knob Omni-Compass moves.

**What the smoke runs taught, and why they are in the record.** A benchmark of a cache is only a benchmark if the working set reaches the cache. Three smoke runs of the tuning workload, one repetition each and none counted,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

were needed before the harness asked the question it was built to ask, and each is written in the preregistration with its run number and its figures. The first failed before an arm ran, on two harness faults (a relative file path handed to a program that runs from its own directory; a server log copied as a file the upload step could not read). The second ran end to end and showed the design did not touch the cache at all: YCSB's shipped request distribution is zipfian, which concentrates almost every request on a few thousand hot records whatever the key space, so at the top notch of 1.5 million records the cache held 36 MB of its 512 MB, both arms answered inside the line with a p95 of 0.20 ms, and omni, seeing calm and nothing evicted, gave memory back to the floor of the cover. That would have read as a free memory saving for Omni-Compass, and it would have been worthless: a knob with nothing to do had moved to its floor. The requests were changed to a uniform draw over the notch's key space so the working set is the key space, the one departure from YCSB as shipped, and said so. The third smoke still held 38 MB, and the per-notch figures the runner now prints showed why: the dataset was loaded with ordered keys but the run phase used YCSB's default hashed keys, so every read asked for a record that did not exist and was answered from the index alone, which fits in a few megabytes. The run phase now names ordered keys too. Nothing in the rules, the gauges or the workloads changed across the three. The fourth smoke reached the cache: native held 336 to 481 MB of its 512 MB across the notches and read 453,000 pages into it, and its per-notch figures set the line. The server's mean read latency ran 0.15 to 0.24 ms while the cache held the working set and 0.23 to 0.33 ms while it did not, with a p95 never above 0.42 ms; against a 2 ms line every one of those readings sat in the bottom quarter of the band, under the 0.4 center, so the compass could only ever read calm, the same fault as a Kubernetes line set at ten times the service's normal response. The line was set at 1 ms, the one change the preregistration had reserved for the tuning workload, with the center unchanged, so that a hit is calm and a full cache reading past 0.4 ms is slow; the one repetition's whole-arm figures (work inside the line −1.9%, cache held −31%, pages read +32%, host CPU +8%, p95 equal) are recorded and not counted. The point for a referee is the method: a result that flatters the governor is the first thing to suspect, the suspicion is pursued to a harness cause, the cause is written down with the run that showed it, a band is set on the tuning case's own figures and never on an untouched one, and the counted runs begin only when the harness provably asks the question.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## A database's buffer pool, resized online in the server's own chunks

MySQL is wired as an operator runs it (`docs/MYSQL_PREREGISTRATION.md`, `tools/run_sysbench.py`): the 8.0 series from Ubuntu's own package, its configuration as shipped but for the settings an operator sets for InnoDB, the buffer pool at 512 MB (MySQL's own default is 128 MB, and the preregistration says why 512 MB is the operator's setting here), the chunk the server resizes in at its shipped 128 MB, one pool instance, and the performance schema on. The questions are asked by sysbench, the standard OLTP benchmark Ubuntu ships as its own package, running its published scripts as shipped at a fixed offered rate, with the number of tables in use stepping one notch at a time through the pool and past it. The wire in is the server's own mean statement latency over the last second, from the performance schema's statement summary, differenced. The wire out is one statement through the server's own console, `SET GLOBAL innodb_buffer_pool_size`, inside the cover [128 MB, 2,048 MB] and always in whole chunks, because that is how the server itself moves the pool. Two things about this knob differ from the MongoDB cache and the plug carries both. The server resizes asynchronously and reports its progress in its own status variable, so the plug waits for the resize to complete before reading back, and a pool found mid-resize is the server still carrying out Omni-Compass's own write, not a foreign hand. And a shrink evicts pages while the resize runs, so the dwell after any move is ten seconds rather than five. The compass holds the reading at 40% of a 0.6 ms statement line (written as 1 ms, set on the tuning workload's smoke runs, as the MongoDB line was: a hit reads 0.14 to 0.22 ms across GitHub's runners and a full pool missing a third of its reads 0.05 to 0.06 ms more, so the center sits at 0.24 ms, above the hits and under the misses); the direction rule is the cache's with one difference the smoke runs forced: slow statements grow the pool by chunks only while the pool is full, at 95% of the line with the pool full four chunks are added at once, and calm gives one chunk back after the dwell only while the pool's misses are under one percent of its read requests, because InnoDB keeps stale pages resident, so neither "pages free" nor "no page read from disk" ever says that a pool holds its working set, and the miss share does (amendment 2 of 8 October, declared before the third counted set: the pool also grows only while it is missing, one percent or more, the same line the other way, because the second set's audits showed 36 of read_write's 82 grows bought chunks while the pool missed under one percent of its reads, for slow writes the pool cannot mend; and no chunk is given back in a second with no read request). The work-inside-the-line gauge uses a transaction line, the statement line times the statements a transaction as sysbench's script ships it (one for a point select, fourteen for the read-only mix), and says so. One writer, read-back and the hand-back at the end are the plug's, as everywhere. The same disclosure as MongoDB's holds and is made before the run: on a 16 GB machine with a 1.4 GB dataset a pool miss is a read from the operating system's page cache, not from disk.

**What the first counted set taught about this knob.** Three runs of five workloads (`results/live/V3_SYSBENCH.md`, 8 October) read every gauge inside the noise, and 15 of the 45 omni arms did not read the operator's pool back at the end. The audits show the server still "withdrawing blocks" of a shrink the law had asked for when the arm ended: InnoDB withdraws the last blocks of a shrink only when the pages the load has pinned are released, so under load the withdrawal does not finish, and the server ignores a new `innodb_buffer_pool_size` while a resize is in progress, so the restore the plug issued at that moment was dropped. Every following arm began from a fresh server at the operator's setting, so no other arm was affected, and the decision loop never wrote during a resize (its writes are gated on the status); only the restore was. The plug now waits for the resize the server is carrying out (the load has stopped by then, so a stalled withdrawal finishes), writes the snapshot, and writes once more if the server ignored the first write; the receipt of that restore (what was in flight, the seconds waited, the writes, the final read-back) is in every omni arm's record and the table counts the arms. The rows of the first set stand as they are, hand-back NO included, in `docs/history/V3_SYSBENCH_set1.md`; the preregistration carries the finding and the amendment; and the second counted set, run on the amended plug the same afternoon with nothing else changed, handed the pool back on all 45 omni arms and is the result in section 16.4. The lesson is general and is the reason section 7.2 exists: a knob the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

device applies on its own time has to be handed back on the device's time, and the proof of the hand-back is the read-back, not the write.

## The machine itself: the kernel's frequency governor under the processor's meter

The lowest level a governor can sit on is the machine it runs on. Linux already has a controller for the processor's clock, the frequency governor (`schedutil`, or the processor driver's own), choosing the clock every few milliseconds under a ceiling the operator sets once; and the processor has its own energy meter (RAPL) ticking in microjoules. Omni-Compass sits on top of the governor exactly as it sits on the HPA: the governor keeps running, and Omni moves one knob, the frequency ceiling of every CPU, through the kernel's own files (`tools/run_cpu_power.py`, `docs/CPU_POWER_PREREGISTRATION.md`).

The reading is the service's own mean latency, held at 40% of a 20 ms line by the compass law; calm gives one notch of the ceiling back (5% of the top clock), a slow service raises it by notches, the wall puts the top on at once. Every notch down is tried on the machine itself first (section 9.4): the brain's verdict around the engine's own, eight cost samples at the deepest notch already allowed and eight one notch further, allowed only if the cost does not rise within 2%. The cost is the index's own reading for one machine (package power × latency / work inside the line), so a notch that saves less energy than it costs in speed is refused and the ceiling stays where the operator left it; a second declared objective, energy per request inside the line with the line as the guardrail, is run beside it. The snapshot of every CPU's governor and ceiling is taken once; the ceiling is handed back and read back at the end; a ceiling found at a value Omni did not write stops it; the master switch and the lease hand back if Omni dies.

The meter is the processor's own counter, class P with the caveat that it is the processor's calibrated counter and not an external meter; a smart plug on the wall socket (`WALL_METER`) adds the whole machine, read by the harness only. GitHub's own machines are virtual and expose neither the governor nor the meter: the workflow proves it with the probe on every dispatch. The benchmark runs on a machine on the metal as root, in one command (`sudo bash scripts/cpu_power_run.sh`): the founder's own tower or laptop on Linux, a rented bare-metal server, or a self-hosted runner on either. Three runs make the table (`tools/cpu_power_abc.py`). Nothing in the engine changes.

## The data center under the servers: cooling, power and batteries

Under every cluster in this book stands a building. Its native controllers are older and more conservative than any autoscaler: the building management system holds the supply-air temperature through the cooling units' own loops, the chilled-water plant runs its own sequence, the uninterruptible power supply holds its reserve, the power distribution units meter every rack, and every server's management controller and every card's firmware enforce their own power limits. Omni-Compass sits on top of them exactly as it sits on the HPA. They keep running; it moves only the settings they already accept, and only inside the band the operator writes down.

**What it may set, and what it never sets.**

| Muscle | Native controller | The knob Omni-Compass may hold | Switch | Status |
|---|---|---|---|---|
| Cooling | the building management system and the cooling units' own loops | the supply-air setpoint, inside the operator's band | `--cooling-cmd "<sets {c}>" --cooling-min-c --cooling-max-c --cooling-restore-c` | built (level 7) |



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

| Muscle | Native controller | The knob Omni-Compass may hold | Switch | Status |
|---|---|---|---|---|
| Site power | the site's own limit and its breakers | none: site watts are read, and the site limit enters the engine as a stress | `--power-cmd "<prints site watts>" --site-limit-w <limit>` | built (level 7) |
| Servers' processors | the kernel's frequency governor and the processor's own limits | the clock ceiling and the power limit | section 10.10 | built (level 6) |
| Cards | the card's firmware | the clock ceiling and the power limit | level 5 | built |
| Uninterruptible power and batteries held as reserve | the UPS's own reserve | **none, ever**: a reserve held for an outage is never a lever (the physics criterion) | | native by design |
| Batteries held for shifting energy | the site's dispatch | the charge window | | designed as an organ, not yet wired |

**How the cooling muscle moves.** The setpoint is moved by the heat state between the operator's minimum and maximum: a warmer setpoint while every reading is cool, which saves the chiller's energy, and a colder one as heat rises. The band must sit inside what the equipment is rated for; for most data-center equipment that is the 18 to 27 °C inlet range of the ASHRAE thermal guidelines, and inside any narrower limit the equipment's own warranty sets. The restore value is the operator's own setpoint, handed back and read back on OFF, at the end of every run, and by the lease if the governor dies.

**The do-no-harm gate at the site meter.** A warmer setpoint saves chiller energy only up to the point where the servers' own fans spend the saving: fan power rises steeply with inlet temperature. That is why the cooling knob is judged at the site meter and not at the chiller: a step is allowed only where the paired trial shows the site's energy for the same work no higher, every rack inlet inside its band, and no service gauge worse. Where it does not, the setpoint stays the operator's, and the result says so.

**Wiring it, in order.**

1. **Watch.** Give Omni-Compass read access to the building management system's points (supply and return air, every rack inlet, the chilled-water temperatures) and to the site meter, and let it watch at least one full cycle of the site's demand. Nothing is written.
2. **Write the band.** The minimum, the maximum and the restore value of the supply-air setpoint, inside the equipment's ratings, signed off by the facility's engineer; the site limit in watts.
3. **The wire check** (section 8.4). The cooling command must read back the setpoint it wrote; a building schedule that rewrites the setpoint is another writer, and Omni-Compass stops writing that lever by itself (section 7.2).
4. **One hall first.** A test hall or a single row, paired against its own native weeks, before any other hall.
5. **Keep the switches in reach.** The master switch and the kill file work at the facility level exactly as at every other level.

**What is proven, and what is not.** The facility muscles are modelled today (class S): the data-hall cooling family of the catalog, the power distribution family, and the site budget shared by groups of cards, which held out with no minute over its budget (`docs/REGISTER.md`: the families of rows 29 and 38 of its catalog table, and row 20 of its benchmark table). No live facility has run yet. A class P result needs a building management system's point list, read access, a site meter and a test hall; until a facility pilot has produced its own paired receipts, this section is a wiring instruction, not a claim of savings.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 33

# The Integration Manual

**Before you wire anything:** read `DISCLOSURES.md`. Omni-Compass acts only through the wires it is given; it cannot be slapped on. If your paired receipts differ from the published benchmarks in direction, the first presumption is wiring: check the installation with section 8.5 of the manual (*Wired right or wired wrong*).

**Which knobs to wire in at all:** `docs/WIRING_VERDICTS.md` gives every knob in every published result one of three words from the tables themselves: **write** (wire both ways), **watch** (wire out only: Omni reads, the native controller lives by itself) or **operator's choice** (a trade). A knob that showed nothing, or lost, stays native. How to wire for superiority, step by step (watch first, the observation run, the brain's own verdict on each knob, the recheck): the manual, section 9.4.

How to wire Omni-Compass into your own systems yourself, from watching only to running your stack from the top. This manual ships in the box with the code and the license. Nobody from The Omni-Compass LLC needs to be on site.

Read section 1 once, then take the levels in section 4 in order. Each level has a pass condition; go to the next level only when it holds. The OFF switch (section 2) works at every level.

## 1. What is in the box

| Item | Where |
|---|---|
| The engine, nervous system, safety shield, compass and conveyance law | `omnicompass/` |
| The Kubernetes controller and its muscles | `omni_controller/controller.py`, `omni_controller/muscles.py` |
| The GPU governor (one GPU box, no Kubernetes needed) | `omni_controller/gpu_governor.py` |
| Container image recipe; install and permission files | `deploy/Dockerfile`, `deploy/install/omni-compass.yaml`, `deploy/rbac-*.yaml`, `deploy/pilot/` |
| Proof tools: paired tests on your own stack, the GPU test, simulations | `scripts/`, `tools/`, `hardware/` |
| Every check the code must pass | `python3 verify.py` (ends with `VERIFICATION: PASS`) |
| The C++ engine: every law twinned in C++20, proven equal to the Python | `cpp/` (build: `cmake -S cpp -B cpp/build && cmake --build cpp/build`) |
| The seal: the fingerprints that lock each Python law to its C++ twin | `results/SEAL.json`, `python3 tools/seal.py --check` |
| Metrics: every gauge and where it comes from | `docs/METRICS_CATALOG.md` |
| How it compares with what you run today | `docs/COMPARISON.md` |
| The license and notices | `LICENSE`, `NOTICE` |

**License, in short** (the `LICENSE` file governs): you may download, run, modify and test Omni-Compass free of charge only to evaluate it and to reproduce its published results, including in shadow or test mode on systems you own or control (`LICENSE`, section 1). Every other use, commercial or not (running it for your business or in production, selling it, hosting it, or building it into a product or service), needs a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. The copyright, patent and trademark notices, the `LICENSE`, the `NOTICE` and `DISCLOSURES.md` stay with every copy.

## 2. The rules Omni-Compass keeps on your system

1. **One OFF switch for the whole harness, in a human hand.** `python3 tools/omni_switch.py off` turns every Omni-Compass governor on the machine off at once: each returns every setting it changed to the value it recorded before it acted, reads each back and exits, and no governor starts again until `python3 tools/omni_switch.py on` (`omnicompass/master.py`; the switch file is `OMNI_MASTER_OFF`, by default `/tmp/omni-compass/OFF`). Each governor also has its own switch for one muscle at a time: creating its kill file (or setting `OMNI_KILL=1`).
   - Kubernetes controller: `--kill-file` (default `/tmp/omni.kill`).
   - GPU governor: `--kill-file` (default `/tmp/omni-gpu-kill`), or send it SIGTERM.
2. **It records before it acts.** Every original setting is written down first (annotations on the Kubernetes objects, the `snapshot` line in the GPU audit), so the OFF switch always knows what to restore.
3. **It watches before it writes.** Every level starts in watch mode: it decides and logs, and writes nothing.
4. **It never goes blind and keeps acting.** If a reading fails (nvidia-smi, metrics, the response-time feed), it returns what it changed to the recorded setting at once and gives nothing back until it can see again.
5. **Service first.** While response time is over your target, and for a few decisions after (`--slo-clear`), it may not cap power or pack tighter than your own settings.
6. **Every change is read back.** No new order goes on top of one that has not landed.
7. **The living band.** No organ is driven below 5% or above 95% of its range; machines are marked idle, never switched off by Omni-Compass; it never evicts or moves a pod.
8. **Everything is logged.** Every read, decision, write and its reason goes to the audit log (`--audit`).



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## 3. Before you start: what your system needs

| You run | You need |
|---|---|
| Kubernetes (any: vanilla, EKS, GKE, AKS, OpenShift/OKD, Rancher, kind) | Kubernetes 1.34 or newer (in-place pod resize); metrics-server (`kubectl top nodes` works); an HPA with a CPU target on each service to be governed; a readiness probe and a short preStop pause on each service |
| A response-time feed (strongly recommended at every level) | a CSV per service, `elapsed_seconds,latency_ms,ok`, written continuously; `scripts/latency_probe.py` writes one against any HTTP endpoint |
| NVIDIA GPUs | driver with `nvidia-smi`; root (or the capability to run `nvidia-smi -pl`); on a VM, full GPU passthrough (container or "pod" GPU rentals usually block power-limit changes) |
| CPU power control | Linux cpufreq with the schedutil governor (`/sys/devices/system/cpu/cpufreq`); RAPL for package watts (`/sys/class/powercap/intel-rapl`); usually bare metal only, since cloud VMs rarely expose these |
| Site power and cooling | a command that prints site watts, and (for cooling) a command that sets the supply-air setpoint through your building management system |

Build the image once:

```
docker build -f deploy/Dockerfile -t &lt;your-registry&gt;/omni-compass:&lt;tag&gt; .
docker push &lt;your-registry&gt;/omni-compass:&lt;tag&gt;
```

It runs as non-root, with a read-only root filesystem and no Linux capabilities.

## 4. The levels, from watching to running the stack

### *Level 0. Evaluate without touching anything*

```
pip install -r requirements.txt
python3 verify.py                       # every check; ends with VERIFICATION: PASS
python3 tools/gpu_physics_sim.py        # the GPU governor against a modelled NVIDIA card
python3 hardware/node_exchange.py       # CPU and GPU on one power budget, modelled
```

**Pass condition:** `VERIFICATION: PASS`. Nothing in your systems is touched.

### *Level 1. Watch (read-only)*

```
kubectl apply -f deploy/install/omni-compass.yaml     # set the image line; it runs --mode observe
kubectl -n omni-compass logs deploy/omni-compass -f
```

- The install file grants a read-only identity. `scripts/pilot_shadow.sh` records `kubectl auth can-i` receipts that show it cannot write.
- Every decision it would take is logged with its reason; nothing is written.

**Pass condition:** zero writes, and readings your operators agree with.

### *Level 2. The pods, on top of your autoscalers*

1. Grant `patch horizontalpodautoscalers` and `patch pods/resize` (`deploy/rbac-target.yaml`).
2. Run with `--mode target --latency-file <feed> --slo-ms <your p95 target>`.
3. Optional pod muscles, one at a time:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

| Muscle | Switch | What it does |
|---|---|---|
| convey | on with `--latency-file` and `--cap-deployments ns/name` | gives each machine's idle CPU to the serving pods on it (default: always); `--convey-on 0.5 --convey-off 0.25` engages it only while response time is over half the target |
| rightsize | `--rightsize-deployments ns/name` | each pod's CPU request follows its measured use × (1 + headroom), in place |
| coldstart | `--coldstart-deployments ns/name --coldstart-signal ns/configmap` | scales a service to zero while no work waits, wakes it the moment work arrives |
| batch | `--batch` | admits held Jobs labelled `omnicompass.io/batch=true` when there is load and power headroom |
| batch pace | `--batch-pace` | pauses Jobs labelled `omnicompass.io/pausable=true` under power or heat stress, resumes them after |
| rollout guard | `--rollout-guard ns/name` | pauses a rollout while change is not permitted, undoes one past its deadline when rollback is authorised |
| contain | `--contain-namespaces ns --contain-cpu-m <m>` | holds an agent namespace to a CPU budget with a quota |
| security hold | `--security-configmap ns/name` | key `hold: "true"` blocks every expansion |

Your HPAs keep scaling as before. Omni-Compass sets their targets and raises floors ahead of bursts.

**Pass condition:** p95, p99 and failed requests no worse than your own, over paired runs (section 6).

### *Level 3. The machines*

1. Grant `patch nodes` and `patch pods` (`deploy/kind/rbac-omni.yaml`).
2. Run with `--mode nodepool --active-nodes-only --closure /app/law/closure.json --node-scale-cmd "<command with {n}>"`.

| Your platform | The park/wake command |
|---|---|
| any cluster, bare metal, kind | `bash scripts/kind_nodepool.sh {n}`: close machines to new work and mark them first to go; open again, warm machines first |
| Karpenter / EKS Auto Mode | the NodePool CPU limit at `{n} × node CPU`, parked nodes kept, not consolidated away |
| Cluster Autoscaler node group | the group's desired size, scale-down through parking, not deletion |
| OpenShift / OKD | the worker MachineSet replicas, parked, not deleted |

A machine is given back only when every sense is live, the last order landed, pods are not scaling up, nothing waits for a place, and the remaining machines stay inside the band.

**Pass condition:** fewer machines in service and less energy, with no service gauge worse.

### *Level 4. Omni-Compass decides; Kubernetes is the muscle*

Add `--strict-replicas`: Omni-Compass decides each service's replica floor and when to shrink; your HPA stays as the fast reflex upward; the node law sizes the machines.

**Pass condition:** as level 3.

### *Level 5. A GPU box (with or without Kubernetes)*

The GPU governor runs on any Linux machine with NVIDIA GPUs:



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

```
# watch: decides and logs, writes nothing
sudo python3 -m omni_controller.gpu_governor --mode watch --gpus 0,1,2,3 --audit /var/log/omni/gpu.jsonl
    ↪  \
     --latency-file &lt;feed&gt; --slo-ms &lt;p95 target&gt;
# cap: writes the power limits
sudo python3 -m omni_controller.gpu_governor --mode cap   --gpus 0,1,2,3 --audit /var/log/omni/gpu.jsonl
    ↪  \
     --latency-file &lt;feed&gt; --slo-ms &lt;p95 target&gt;
# OFF
sudo touch /tmp/omni-gpu-kill
```

What it does, every `--interval` seconds (default 2):

- reads each GPU's own meter: power draw, temperature, utilization, power limit;
- the engine sets a power cap; the shield keeps it above `--min-share` × the start limit (0.70) and above draw × (1 + `--headroom`);
- a busy card (smoothed utilization at or over `--util-gate`, 0.5) gets its full limit back at once;
- a response-time breach or a failed reading returns the start limit at once.

**Speed lock (optional).** Keep every response-time gauge at least `--speed-gain` (1%) faster than without Omni-Compass, and spend any speed won elsewhere on watts:

```
python3 tools/gpu_baseline.py baseline.json native-run/latency.csv     # a run without Omni-Compass, oth
    ↪ er days
sudo python3 -m omni_controller.gpu_governor --mode cap --baseline-file baseline.json --latency-file &lt;fe
    ↪ ed&gt; ...
```

**Prove it on your card:** `sudo bash scripts/gpu_paired.sh` runs native, watch and Omni-Compass back to back on the same machine and prints the table and verdict (`docs/GPU_RUN_GUIDE.md`).

**Pass condition:** work per energy up, and requests served and p95 inside the guardrails.

### *Level 6. CPU clock and power*

Add to the Kubernetes controller, on bare metal:

```
--cpufreq-policy-root /sys/devices/system/cpu/cpufreq --cpufreq-require-schedutil \
--rapl-cmd "&lt;prints CPU package watts&gt;"
```

The CPU frequency ceiling follows the engine's cap inside the nervous system's envelope; the OFF switch writes every policy's recorded maximum back exactly. GPUs can be wired the same way from the controller: `--gpu-query-cmd "nvidia-smi --query-gpu=power.draw,temperature.gpu --format=csv,noheader,nounits" --gpu-power-cmd "nvidia-smi -pl {w}" --gpu-max-w <max>`.

**Pass condition:** energy per unit of work down, no service gauge worse.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

### *Level 7. Site power, cooling, batteries*

| Organ | Switch | Status |
|---|---|---|
| site power stress | `--power-cmd "<prints site watts>" --site-limit-w <limit>` | wired: power stress enters the engine; batch pace and caps respond |
| cooling setpoint | `--cooling-cmd "<sets {c}>" --cooling-min-c 18 --cooling-max-c 27 --cooling-restore-c 22` | wired: warmer supply air while cool, colder as heat rises |
| **CPU + GPU on one power budget** | `hardware/node_exchange.py` (the conveyance law over CPU and GPU organs) | **simulation only.** The live levers exist (levels 5 and 6); the exchange between them has not run on hardware |
| GPU groups sharing a site budget | `hardware/site_exchange.py` | **simulation only** |
| on-site batteries as an organ | designed (`docs/DOMAIN_MAP.md`) | **not built** |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

# 5. Stack by stack

| Stack | Levels available | Notes |
|---|---|---|
| Vanilla Kubernetes, kind, Rancher | 1-6 | as written |
| Amazon EKS | 1-5 | nodes via Cluster Autoscaler node group or Karpenter NodePool (level 3 table); CPU power control is not exposed on EC2 VMs; GPUs on bare-metal or full-GPU instances |
| Google GKE, Azure AKS | 1-5 | as EKS, with the provider's node-pool size as the park/wake command |
| Red Hat OpenShift / OKD | 1-5 | OpenShift is Kubernetes underneath; grant the same permissions through a Role; machines via the worker MachineSet |
| NVIDIA GPU servers without Kubernetes | 5 | the GPU governor alone |
| Bare-metal CPU servers | 6 | cpufreq and RAPL through sysfs |
| Slurm / HPC schedulers | 5 on the GPU nodes | a Slurm connector for job-level decisions is not built |
| Building management (cooling, power meters) | 7 | through your BMS's command line or API, wrapped in the command templates |
| Azure AKS, the bill as the gauge | 1-3 | Azure's managed cluster autoscaler stays native and deletes the machines Omni-Compass idles; one or several work pools (`worker_pools`), the replica ceiling raised with the fleet (`hpa_max`); a fleet of 4 cannot show a saving of one machine, 39 can (manual, section 10.1; `docs/AZURE_SETUP.md`) |
| PostgreSQL behind PgBouncer, or any pooler with a console | one knob | the pool size through the pooler's own admin console, cover [2, 90], one writer, restored on OFF (manual, section 10.2; `docs/POSTGRES_PREREGISTRATION.md`) |
| Apache Kafka, or any consumer group an operator sizes | one knob | the consumer count inside [1, partitions], the group's own end-to-end latency as the reading, the rebalance paid on every move, handed back on OFF (manual, section 10.5; `docs/KAFKA_PREREGISTRATION.md`) |
| Redis, or any cache with a console and a memory ceiling | one knob | `maxmemory` through the cache's own console, cover [16, 512] MB, grown only while the cache is full (a cold miss is not the ceiling's), one writer, restored on OFF (manual, section 10.6; `docs/REDIS_PREREGISTRATION.md`) |
| Drone swarms (gym-pybullet-drones; PX4 and ArduPilot next) | one knob a drone | the cruise override inside the autopilot's limits, a separation wall, the paired physics trial, collisions void the cell (manual, section 10.7; `docs/SWARM_PREREGISTRATION.md`) |
| MongoDB, or any store whose engine exposes its cache size at run time | one knob | the storage-engine cache through the server's own console, cover [256, 2,048] MB, grown only while the cache is full, one writer, restored on OFF (manual, section 10.8; `docs/YCSB_PREREGISTRATION.md`) |
| MySQL, or any store whose engine resizes its buffer pool online | one knob | the InnoDB buffer pool through the server's own console (`SET GLOBAL innodb_buffer_pool_size`), cover [128, 2,048] MB in the server's own 128 MB chunks, grown only while the pool is full, the plug waiting for the server's asynchronous resize before reading back, one writer, restored on OFF (manual, section 10.9; `docs/MYSQL_PREREGISTRATION.md`) |
| A rented cloud machine running the big organisms | the whole harness | `big-organism-detached`: start, collect, survey; the clock rule sets the window (manual, section 10.4) |
| Independent simulators (CityLearn, pandapower, MuJoCo) | one knob each | the simulator's own controller is native; preregistered, A/B/C (manual, section 10.3) |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Stack | Levels available | Notes |
|---|---|---|
| The machine itself: Linux's frequency governor under the processor's meter | one knob | the frequency ceiling of every CPU through the kernel's own files, cover [half the top clock, the top], the governor still running under it; every notch down tried on the machine first by the brain's verdict; energy from RAPL and a wall plug where fitted; root on a machine on the metal, never a virtual one (manual, section 10.10; `docs/CPU_POWER_PREREGISTRATION.md`; `sudo bash scripts/cpu_power_run.sh`) |

## 6. Proving it on your own system

1. **Paired runs.** Run your service the same way twice, once as you run it today and once with Omni-Compass on top, back to back on the same machines, order rotated, at least 5 times (10 for a result you publish). `scripts/kind_paired.sh` and `tools/live_reps.py` do this and print each gauge with its 95% interval. A change is proven only when its interval excludes zero.
2. **The switch drill,** after every run: turn Omni-Compass OFF, check every setting is back at its recorded original and no `omnicompass.io/*` annotation remains, turn it ON.
3. **Read the results** with `docs/METRICS_CATALOG.md`: every gauge says whether it is measured or modelled.

## 7. Reading the log

| Line | Meaning |
|---|---|
| `gate: a sense is blind` | a reading failed; nothing is given back until it returns |
| `gate: pods scaling up` | pods first, machines after |
| `decision failed (n in a row)` | the cluster could not be reached; nothing was written. Turn it OFF for native at once |
| `convey: <machine> idle CPU to its k serving pod(s), limit c` | that machine's idle CPU now reaches the work on it |
| `convey: response time calm, operator's limit` | the pods are back at your own CPU limit |
| `busy gate: utilization u, the limit read at start` | a busy GPU got its full power limit back |
| `speed lock: worst ratio r vs line 0.99 (spend / hold / release)` | the speed lock's reading and what it did |
| `response-time reflex: the limit read at start` | response time went over target; full power at once |
| `reset: the limit read at start` | the OFF switch restored this setting |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## 8. What is proven, and how

| Claim | Evidence | Kind |
|---|---|---|
| On real Kubernetes, Omni v1, three runs of ten pairs each: more work inside the response line (+42% to +48%), p95 −57% to −69%, machines fewer where a paired trial allowed it (steady −1.5% to −3.4%, batch −15% to −24%) | `results/live/V1_ALL_FOUR.md`, `V1_STEADY.md`, `V1_WANDERING.md`, `V1_FAULTS.md`, `V1_BATCH.md`, `V1_FAIRNESS.md` | measured on kind; read by the three-run rule (`docs/OMNI_V1.md`) |
| On a real database, Omni v3: 61% to 72% fewer connections held open for the same work and latency, at +14% to +28% host CPU-seconds (worse in all three of our runs, reported) | `results/live/V3_PGBENCH.md` | measured on GitHub's machines |
| On Azure's bill, 4 workers: no difference beyond the noise on any gauge (the fleet is too small for the lever) | `results/live/V1_AKS_STEADY.md`, `V1_AKS_BURST.md` | a real bill |
| Energy on Kubernetes | declared model on kind, no meter | modelled |
| GPU: more work per energy within the speed guardrail | earlier card controller, obsolete; rerun on rented cards pending | hardware meter |
| CPU and GPU on one power budget: more work, never over the budget | `results/hardware/NODE_EXCHANGE_*.json` | modelled |
| Safety: OFF switch restores everything; watch mode writes nothing | every live run's switch drill; `verify.py` | measured / checked |

## 9. Python, C++ and the seal

Omni-Compass exists in two languages. The laws are twinned: each has a Python version and a C++20 version that give the same answers, proven by a parity test on every build.

| Law | Python | C++ | Proven by |
|---|---|---|---|
| core engine (six-state equations) | `omnicompass/core.py` | `cpp/src/core.cpp` | 500 frozen fixtures |
| governor (allocation laws) | `omnicompass/adapter.py` | `cpp/src/governor.cpp` | `tests/test_cpp_governor_parity.py` |
| safety shield | `omnicompass/shield.py` | `cpp/src/shield.cpp` | `tests/test_cpp_shield_parity.py` (plus adversarial cases) |
| HPA replica law | `fleet/harness.py` | `cpp/src/hpa.cpp` | `tests/test_cpp_hpa_parity.py` |
| closure law (machines) | `omnicompass/closure.py` | `cpp/src/closure.cpp` | `tests/test_cpp_closure_parity.py` |
| conveyance law (the conserved budget) | `omnicompass/conveyance.py` | `cpp/src/conveyance.cpp` | `tests/test_cpp_conveyance_parity.py` (identical to the last bit) |
| nervous system (authority per organ, living band) | `omnicompass/nervous_system.py` | `cpp/src/nervous_system.cpp` | `tests/test_cpp_twins_parity.py` |
| compass and ledger (composite storage) | `omnicompass/compass.py`, `omnicompass/storage.py` | `cpp/src/compass.cpp` | `tests/test_cpp_twins_parity.py` |
| GPU governor rules (shield limit, busy gate, speed lock, window, baseline) | `omni_controller/gpu_governor.py` | `cpp/src/gpu_rules.cpp` | `tests/test_cpp_twins_parity.py` |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

**The seal** (`results/SEAL.json`) holds the SHA-256 fingerprint of every file of every twin, written only after all parity tests pass (`python3 tools/seal.py`). `verify.py` fails if any sealed file changes afterwards, and names it. So the Python and the C++ cannot drift apart unnoticed. What stays in Python is the plumbing that talks to Kubernetes, nvidia-smi and sensors (`omni_controller/controller.py`, `muscles.py`, the device I/O of `gpu_governor.py`) and the simulation harnesses; every decision they take goes through the twinned laws. The seal lists them as Python only.

Keep this manual with the code. When Omni-Compass changes, this manual, the C++ twin and the seal change in the same commit.


*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 34

# Running the GPU Benchmark

> **Before you wire anything:** read `DISCLOSURES.md`. Omni-Compass acts only through the wires it is given; it cannot be slapped on. If your paired receipts differ from the published benchmarks in direction, the first presumption is wiring: check the installation with section 8.5 of the manual (*Wired right or wired wrong*).

There are three ways to get real-machine numbers:

- **Your own tower:** a machine you control, with a smart plug measuring the whole machine at the wall (section A).
- **GitHub's GPU machines:** they run the test from the repository with one click (section B).
- **A rented cloud GPU:** sections 1 to 6.

## A. Your own tower, measured at the wall

You need:

- an NVIDIA graphics card (a GeForce RTX works);
- Linux on that machine (a spare drive or a USB boot is fine);
- a smart plug that reports watts, for example a Shelly Plus Plug or any plug running Tasmota, which cost about $20 to $30.

Plug the tower into the smart plug, and connect the plug to your home Wi-Fi with its own app. Note its IP address, shown in the app or on your router's device list. Then on the tower:

```
python3 tools/wall_meter.py shelly2:192.168.1.50 --once       # prints the tower's watts right now
sudo WALL_METER=shelly2:192.168.1.50 bash scripts/gpu_paired.sh                  # trial run
sudo WALL_METER=shelly2:192.168.1.50 PHASE=confirm bash scripts/gpu_paired.sh    # the real test
```

Use `shelly1:` for older Shelly plugs and `tasmota:` for Tasmota plugs. The table then adds whole-machine energy at the wall and requests served per wall kilojoule. The plug is read by the test only; Omni never sees it, so its number is independent of Omni.

## B. GitHub's own GPU machines

1. **One-time setup, by an owner of the Omni-Compass organisation on GitHub:**
    1. Go to **Settings → Actions → Runners → New runner → New GitHub-hosted runner**.
    2. Choose the image **NVIDIA GPU-Optimized Image for Linux** and a GPU size.
    3. Name its label `gpu-t4`, or set the repository variable `GPU_RUNNER` to the label you chose.

GPU runners are billed per minute on paid plans.

2. **To run:** go to **Actions → gpu-bench → Run workflow**, then choose `smoke` or `confirm`.
3. **Results:** each repetition runs on its own GPU machine. The results are pooled into one table, shown on the run's summary page, and every raw file is committed back to the branch under `results/gpu/github-<run id>/`.

If GitHub's machines don't allow changing the GPU power limit, the job stops in its first minute and says so.

## C. A rented cloud GPU

This costs roughly $10 to $25 in rented GPU time. You type a handful of commands; the machine does the rest.

### 1. Rent the right kind of machine

The test changes the GPU's power limit, so you need a machine where you are the full administrator of the GPU:

- **Use a virtual machine (VM) or a bare-metal server.** Lambda Cloud "on-demand instances" are VMs; so are the GPU instances on AWS, Google Cloud and Azure.
- **Avoid "container" or "pod" rentals.** Their GPUs usually block power-limit changes. This is common on the cheapest per-hour marketplaces.
- **Any single NVIDIA data-center GPU works:** A10, L4, A100, H100, L40S. One GPU is enough.
- **Choose an image that already has PyTorch.** On Lambda that is the default "Lambda Stack" image.

You don't have to guess whether a machine allows it. In its first minute the test checks, and stops with *"cannot set the power limit (run as root)"* if the machine doesn't allow it. If you see that, shut the machine down (you pay only for the minutes used) and rent a different kind.

### 2. Connect to it

The rental site shows a command like `ssh ubuntu@123.45.67.89`. Paste it into Terminal (Mac) or PowerShell (Windows).

### 3. Get the code

On the rented machine (the repository is public while the benchmarks run; if it is private, use a read-only fine-grained token in the URL):

```
git clone https://github.com/The-Omni-Compass-LLC/The-Omni-Compass the-omni-compass
cd the-omni-compass
git log --oneline -1       # the commit you are about to test; write it down
pip install -r requirements.txt
nvidia-smi                 # should show your GPU
```

### 4. The whole test, one command (about 45 hours)

```
sudo nohup bash scripts/gpu_rented_run.sh &amp;gt; run.log 2&amp;gt;&amp;amp;1 &amp;amp;
tail -f run.log            # Ctrl+C stops watching; the test keeps running
```

It runs, in order, and stops at the first failure:

1. the machine check (one copy only, nothing else on the card, the card's default limit and clock range restored);
2. the **wire check**, which must end `WIRED RIGHT` (manual, section 8.4);
3. the smoke test, about 40 minutes, never counted;



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

4. the preregistered counted run on compute-bound work: 10 repetitions × native / watch / Omni, 600 s each, about 6 hours;
5. the second preregistered counted run on AI token generation (memory-bound), the same design, about 6 hours (`SKIP_DECODE=1` skips it); each is its own result, never pooled. Both are packed into one file as soon as they finish: `results/gpu/omni-gpu-<stamp>-confirmations.tar.gz`;
6. **an operator's power cap underneath** (70% of the card's default limit): the cap alone vs the cap with Omni-Compass on top, at the usual load and fully loaded (more work from the same watts), about 3 hours (`SKIP_CAP=1` skips it);
7. **the GPU fault drill**, about 10 minutes: the governor killed outright, the master switch pulled, the response feed blind; every check must pass (`SKIP_DRILL=1` skips it). Everything so far is then packed into `results/gpu/omni-gpu-<stamp>-partial.tar.gz`;
8. the six organisms with this card inside, each as **1, 10, 100 and 1,000 copies** on one clock (3, 3, 2 and 1 repetitions), about 25 hours; the 1,000-copy stacks need a longer step than 2 s, measured on the machine and stated in the receipt (`SKIP_HIL=1` skips this stage);
9. **real AI serving** last: a small open language model served by vLLM, about 2 hours (`SKIP_LLM=1` skips it); if vLLM cannot install on the machine, the stage says so and nothing before it changes;
10. one packed file: `== send this one file back: results/gpu/omni-gpu-<stamp>.tar.gz`, with the label each table chose by rule.

To stop everything at any moment: `sudo python3 tools/omni_switch.py off` turns every Omni-Compass governor off and hands the card back to its own settings.

Do not start it twice and do not use the card for anything else while it runs.

## 5. Bring the results back

Download `results/gpu/omni-gpu-<stamp>.tar.gz` (in JupyterLab: right-click, Download; or `scp` from your own computer). It holds every raw reading, every table, the verdict and the checksums. **Then shut the rented machine down** on the rental site, so the billing stops.

## 6. Read it before you believe it

Open `results/gpu/run-<stamp>/GPU_REPS.md` and check the rows of the manual's section 8.5 (*Wired right or wired wrong*): watch equal to native, requests equal, the card's busy clock under Omni at or above its own, the lid while busy at or above its own busy draw, fail-up rare in the credit-per-write table. If a row reads wired wrong, the run says nothing about Omni-Compass until the wiring is fixed (`DISCLOSURES.md`, section 3).

## D. The 8-GPU result: every card of one server at once

The same test as section C, on a machine with several cards (Lambda "8x A100" or "8x H100", a VM, the Lambda Stack image). Every card runs its own paired test at the same moment, sharing the server's power supply, cooling and neighbours' heat, as in a real data center. Each card starts its arm rotation one step later than the card before it, so at any moment some cards run native and some run Omni-Compass.

Get the code as in step 3, then:

```
sudo nohup bash scripts/gpu_8card.sh &amp;gt; run8.log 2&amp;gt;&amp;amp;1 &amp;amp;
tail -f run8.log
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

Each card: wire check, envelope, smoke, the compute counted run, the AI token generation counted run, the operator's power cap underneath (about 15 to 16 hours, all cards together). Then the fault drill once on card 0, with every other card idle. Then one pooled table per workload (every repetition of every card, each paired within its own card: 80 repetitions per workload on 8 cards) and each card's own table. `SKIP_DECODE=1 SKIP_CAP=1` runs the compute counted run alone, about 6.5 hours. **Every 8-GPU run has a hard time budget** (`MAX_HOURS`; by default 10.5 hours with `POOLED=1`, 4 with `FAST=1`, 26 for the whole design): at the budget the master switch hands every card back, the stages still running stop, and what finished is packed and printed as the file to send back. Lambda bills until the instance is terminated, so terminate it when that line appears. `sudo POOLED=1 nohup bash scripts/gpu_8card.sh &` **runs every stage with the repetitions pooled across the cards (3 per card per counted run, 24 per test on 8 cards; 2 per power cap load; 1 per organism size), about 9 to 10 hours.** `sudo FAST=1 nohup bash scripts/gpu_8card.sh > run8.log 2>&1 &` is a short look, about 3.5 hours: 3 repetitions per card (24 on 8 cards), then one model across every card; it leaves AI token generation, the power cap and the whole stacks out, so it is never the result to show. Last, the whole server as one: a language model (Qwen2.5-7B-Instruct, open) served across every card at once by vLLM, one Omni-Compass governor per card, the server's total GPU energy, about 2 hours (`SKIP_LLM=1` skips it). Before it, **the whole stacks with a real card inside**: the six organisms at 1, 10, 100 and 1,000 copies, native and Omni, the full repetitions (3, 3, 2, 1 by size), each organism on its own card at the same time, so the stage that takes about 25 hours on one card takes the time of its longest organism (the four stacked, at 1,000 copies), about 4 to 6 hours (`SKIP_HIL=1` skips it). The whole design, every stage: about 22 to 24 hours. Progress: `tail -f results/gpu/8card-<stamp>/card-*.log`. At the end: `== send this one file back: results/gpu/omni-8card-<stamp>.tar.gz`. The master switch (`sudo python3 tools/omni_switch.py off`) stops every card's governor at once.

## E. Your own serving engine

own vLLM? Start it as you always do, then:

```
sudo LLM_URL=http://127.0.0.1:8000 LLM_MODEL=&lt;your model name&gt; GPU=0 ENVELOPE=&lt;envelope.json&gt; OUT=result
    ↪ s/gpu/mine \
  bash scripts/gpu_vllm.sh
```

The same paired test (your engine alone against your engine with Omni-Compass on top) runs against it; nothing is installed or started. `GPU=0,1,2,3,4,5,6,7` when your engine spans several cards. Make the envelope once with `python3 tools/declare_envelope.py envelope.json --gpu 0`.

## F. Whole-server power from the server itself

The bench records the whole machine's watts beside the cards' own when you give it a meter: `WALL_METER=redfish:<BMC address>` (with `REDFISH_USER` and `REDFISH_PASSWORD`, read only), `WALL_METER=ipmi:local` (the server's management controller, read on the machine), or a smart plug (`tools/wall_meter.py` lists them). Omni-Compass never reads it.

## Before any machine: the simulated card

`python3 tools/gpu_physics_sim.py --out results/gpu/sim/after` runs Omni's own GPU governor, with its current settings, against a modelled card for 10 paired repetitions in about two minutes. No GPU is needed. Every number it prints comes from the card model, not from a meter, so it is a preview of what the governor does and never the result. `results/gpu/sim/before` holds the same run with the settings before the share floor and busy gate were added.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## What you will get

A table from the GPU's own power meter:

- native against Omni watching only, and native against Omni governing;
- the preregistered primary result: work per energy, in requests served per kilojoule, with its 95% interval.

It ends with one verdict line, one of:

- **better, proven**;
- **worse, proven**;
- **not proven**;
- **better on energy, fails the service guardrail**.

Whatever it says is the answer, and it is the first number in this project that Omni-Compass's own code did not compute.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

PART SIX

# Operating It

*The OFF switch, the rules the governor obeys, the log it keeps, and the care of a running installation.*

CHAPTER 35

# The OFF Switch, the Rules, and the Log

**The rules Omni-Compass keeps on your system.**

1. **The master switch, for the whole harness, in a human hand:** `python3 tools/omni_switch.py off` turns every Omni-Compass governor on the machine off at once (the Kubernetes controller and the GPU governors). Each puts every setting it ever wrote back to the value it read before its first write, reads it back and exits; the command waits until all have, and says so. While the switch is OFF, no governor will start. `python3 tools/omni_switch.py on` allows them to be started again (nothing restarts by itself); `status` shows the switch and every governor running. Use it the moment anything looks wrong, including a suspected breach. Each governor also keeps its own switch for one muscle at a time: the kill file (or `OMNI_KILL=1`, or SIGTERM to the GPU governor). **If a governor dies without handing back** (killed outright, the machine crashed, it hung), nothing it wrote is left in place: every governor records, before its first write, the exact commands that put every setting back, and renews a lease every decision. Run `python3 tools/omni_switch.py watchdog` as its own service next to the governors: it hands back for any governor whose process is gone or whose lease has run out (a hung one is stopped first). `status` names any governor that died without handing back.
2. It records before it acts (annotations on Kubernetes objects; the `snapshot` line in the GPU audit).
3. It watches before it writes.
4. It never acts blind.
5. Service first: while response time is over its target, and for `--slo-clear` decisions after, nothing is given back.
6. Every change is read back; no new order goes on top of one that has not landed.
7. The living band and the cover: no organ is driven outside its range; machines are parked, never switched off by Omni-Compass; it never evicts or moves a pod.
8. One writer: if anyone else changes a knob, Omni-Compass stops writing it and leaves it alone.
9. Everything is logged: every read, decision, write and reason goes to the audit log.

**The three ways a governor stops, and what each leaves behind.** A referee or an underwriter should know exactly what happens to a governed system in each of the three ways Omni-Compass can stop, because the answer is the same in all three: every setting at native, read back, with no record left.

- **The reset** (the brake held to the floor) is the planned end of a run. The governor sees its reset file, hands every setting back in reverse order of power-up, reads each back, writes its last audit line and exits 0. Every benchmark arm ends with one and then checks the cluster from outside the governor: the HPA target at the operator's, the replica range as set, every pod's CPU limit at its shipped value, every worker back in service, no `omnicompass.io/` annotation left on any object. An arm whose reset check fails is invalid and is left out of the table, never counted.
- **The kill switch** (security) is the unplanned end of trust. `tools/omni_switch.py off` writes the switch file and sends every registered governor SIGTERM; each performs the same hand-back as a reset and exits; the

command waits until all have and reports any that have not. No governor starts while the switch is off. It is one switch for the whole machine, not one per muscle, because the case it exists for, something driving a system through Omni-Compass's brain, is not a case for picking which muscle to stop.

- **A death without a hand-back** (killed outright, a crash, a hang) is the case the lease covers. Before its first write every governor registers itself with the exact command that puts every setting back (`omni_controller.controller --restore-only`, which restores from the snapshot annotations on the objects themselves, so it needs no memory of the dead process), and renews a lease every decision. The watchdog (`tools/omni_switch.py watchdog`), run as its own service beside the governors, finds any governor whose process is gone or whose lease is older than its limit (five decision periods, at least two minutes), stops a hung one, runs its recorded restore command, and records what it did. The robustness benchmark (Chapter 43, in preparation) kills the governor outright in the middle of a governed run and measures how many seconds pass before every setting is back and what the service saw in that gap.

**Reading the log.**

| Line | Meaning |
|---|---|
| `gate: a sense is blind` | a reading failed; nothing is given back until it returns |
| `gate: pods scaling up` | pods first, machines after |
| `decision failed (n in a row)` | the cluster could not be reached; nothing was written; turn it OFF for native |
| `decided_by: compass` | the compass set the wires this decision |
| `decided_by: fail_up` / `blind_fail_up` | the service crossed the 0.95 wall, or a sense went blind: full capacity at once |
| `decided_by: thermal_hold` | the card reported a heat slowdown; nothing was tightened |
| `foreign_writer` | someone else changed a knob; Omni-Compass now observes only |
| `restored ... ok: true` | the OFF switch put every setting back and read it back |
| `"watchdog": "handed back"` (in the watchdog's own log) | a governor died or hung without handing back; the watchdog ran its recorded restore command; `ok` says whether every command exited 0 |
| `cruise: on` / `cruise: off` | work waited for a place for two decisions: every machine in service; the line has been empty for two: cruise ends |
| `emergency_brake` | the work is done and demand is at zero: straight to the floor in one move |
| `verdict_state: left native` | no step of the slow knob passed the paired trial: the knob is never moved on this muscle |
| `shield_interventions` | the shield clipped a write to its bounds or its step limit; the count says how often |
| `overhead` (the last line of the audit) | the governor's own CPU over the window: its process and every command it ran |

**How to read an audit from end to end.** The audit (`audit.jsonl`, one JSON record a line, every line timestamped) is the governor's complete account of itself, and a reader can follow a run through it without the manual: the first lines are the snapshot (every setting as found, written to the objects as annotations so a watchdog can restore from them); then, every decision, the readings (the response time and its age, the pods waiting, the machines seen, whether any sense is blind), the compass's position and force (`compass`), the authority the nervous system granted and the release gate's reason (`authority`, `node_gate`), the decision itself (the machines recommended against the machines seen, the HPA target set), every write with the command that made it and the value read back, and the decision's wall time (`decision_ms`); in cruise, the floor step's silence; at the end, the reset's writes and read-backs and the `overhead` record. The benchmark harness prints a decision trail from the audit at the end of every omni arm (`scripts/kind_bench.sh`), one line a decision, so a reviewer reading a run's log sees what the governor saw and

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

did at each minute.



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

CHAPTER 36

# Maintenance, Upgrades and Security

- Run `python3 verify.py` after every upgrade; it must end `VERIFICATION: PASS`.
- Upgrade in watch mode first; take the levels again from level 1.

**What an upgrade is, and what it is not.** The engine is frozen and fingerprinted (Chapter 15). An upgrade that changes any engine file, a rule, a gain, a guard, a preset or a muscle, is by definition the next engine version, and every published result is run again on it before it is quoted beside the old ones; `tools/omni_version.py` says which version a checkout carries, and a checkout that matches none says which files differ. An upgrade that changes only the harness, the tools, the tables or this manual leaves the engine's fingerprint as it was and every result standing; the robustness harness of section 16.8 is an example, built without touching an engine file. An operator can therefore tell, from one command, whether a new checkout is the engine the results were made on or something that must prove itself again.

- The container runs non-root, read-only, with no capabilities; the Kubernetes identity has only the permissions of the level you run.
- The GPU governor needs root only for `nvidia-smi -pl` and `-lgc`.
- Report a security issue as `SECURITY.md` describes.

**Least privilege, proven on every run.** The Kubernetes governor runs as its own service account (`deploy/kind/rbac-omni.yaml`) with exactly the verbs the level needs: patch on the governed HPA, nodes, pods and the governed deployment, get on its security ConfigMap, create on pod evictions. It cannot delete nodes, create or delete pods or deployments, touch the load generator or anything in `kube-system`, read any secret, create a namespace, or write its own security hold. Every benchmark arm records `kubectl auth can-i` for each of these and fails if a "cannot" reads yes or a "can" reads no (`bench-*/rbac_omni.txt` in every archived run). The security hold itself is a ConfigMap another identity owns: when its key reads `hold: "true"` every expansion is blocked, and the governor cannot clear it.

**What never leaves the repository.** Cloud credentials live only in the repository's encrypted secrets and are used only by the workflows that need them; no credential, key or token appears in any file, log, table or commit, and the founder is never asked for one in conversation. The repository carries SPDX license identifiers on every source file so that license scanners (Black Duck, FOSSA, Snyk) read the evaluation license correctly, and every generated report carries the legal notice at its head and foot (`tools/legal.py`). Patent, copyright and trademark filings are referred to as filed; no filing number appears anywhere.

**Supply chain.** Everything the benchmarks download is pinned and checked: metrics-server by version and SHA-256 (`scripts/kind_bench.sh`), Apache Kafka against the Apache Software Foundation's own SHA-512 file for that release, Redis as the Ubuntu runner's own package, MongoDB from its publisher's own repository signed by the publisher's key and YCSB against the SHA-256 written in its preregistration, gym-pybullet-drones at one named commit, Python packages by `requirements.txt`. A download whose checksum does not match stops the run before anything else happens.

**The upgrade drill, step by step.** An operator who receives a new checkout does five things, in order, and stops at the first that fails. First, `python3 tools/omni_version.py`: if it prints the version the operator's results were made on, the engine is unchanged and the rest of the drill is about the harness; if it prints a list of differing files, the checkout is a new engine and nothing the operator measured before applies to it until measured again. Second, `python3 verify.py`, which must end with its pass line; the verifier runs every unit test, the shield's adversarial cases, the Python-against-C++ agreement and the release manifest, and a checkout that fails it is not installed anywhere. Third, the wire check on the operator's own stack, which proves that every wire the new checkout would use still follows, reads back and returns. Fourth, watch mode for at least one full cycle of the operator's load, reading the log the governor would have written and comparing its would-be decisions against what the operator would have done. Fifth, the levels again from level one, each with its paired receipt. An upgrade that skips a step has not been upgraded; it has been installed.

**The security hold, and who holds it.** The hold is the one control that outranks the governor's own judgment, and it is deliberately not the governor's to hold. It lives in a ConfigMap owned by another identity (the operator's security team, in practice); while its key reads true, every expansion the governor would make is blocked and every contraction is still allowed, so a governor under a hold can only give capacity back, never take more. The governor reads the hold on every decision and records in its audit that it read it; it has no verb that could write it. The kill switch (Chapter 35) is the blunter instrument beside it: the hold constrains a running governor, the switch stops every governor on the machine and hands everything back. An operator who suspects anything untoward uses the switch first, reads the audit second, and rotates the governor's identity third; the hold is for the slower case of a change window or an incident elsewhere during which no one wants capacity added.

**What has been reviewed, and what has not.** Reviewed and tested on every run: the governor's permissions against the list above, read from the cluster itself; the shield's refusal of every write past a cover, of every give-back while blind, and of every move under a hold, on two million generated cases; the lease and the watchdog's hand-back on a real cluster, thirty times (section 16.4b); the checksums of every download. Not yet done, and said so: an independent security review of the code by a party outside the company, which is in the open program (section 16.8), and a penetration test of the deployed governor on a customer-like cluster. Until those are done, the operator's own review stands in for them, and this section tells the operator exactly what to look at.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 37

# The Operator Manual

> **Before you wire anything:** read `DISCLOSURES.md`. Omni-Compass acts only through the wires it is given; it cannot be slapped on. If your paired receipts differ from the published benchmarks in direction, the first presumption is wiring: check the installation with section 8.5 of the manual (*Wired right or wired wrong*).

> **The current manual is** `docs/INTEGRATION_MANUAL.md` (every stack, every level, every switch). This page is kept for its first-person account of the laws; where the two differ, the integration manual is current.

I am Omni-Compass. I am the brain and the nervous system; your stack is the muscle. Everything below is what I am and what I do, in my own terms and in my own mathematics. Every command and flag here exists in this repository.

## I. My face

My face is the compass. It is not decoration: it is how I read myself, every decision, and it is written into my code (`omnicompass/compass.py`).

### *The wheel*

My deviation E, over its ceiling E_max, runs along the horizontal. Its rate of change runs along the vertical. Where I stand on that wheel is my heading, measured clockwise from north:

| Point | Letter | Heading | Where I am |
|---|---|---|---|
| + | Α | 0° | rising through rest |
| ⇄ | Δ | 45° | expansion: above rest and rising, exchange under way |
| > | Β | 90° | peak extension: the turn, where my metric flips (k → −k) |
| ⊤ | Λ | 135° | dispersion: above rest and easing, my ceiling holds |
| − | Ω | 180° | falling through rest |
| ≈ | Π | 225° | compression: below rest, settling into my basin |
| < | Γ | 270° | deepest compression: the turn at my floor |
| ✦ | Ψ | 315° | re-alignment: below rest and rising, ignition ahead |

My rim carries twenty-four letters, fifteen degrees apart, clockwise from A:

Α Ε Ζ Δ Η Θ Β Ι Κ Λ Μ Ν Ω Ξ Ο Π Ρ Σ Γ Τ Υ Ψ Φ Χ

Every reading names its letter.

### *The four strokes*

My quadrants are the strokes of the closed circle:

| Quadrant | Stroke | Where I am |
|---|---|---|
| I | Expansion | above rest, rising |
| II | Dispersion | above rest, easing |
| III | Compression | below rest, falling |
| IV | Re-alignment | below rest, rising |

The wheel turns I → II → III → IV → I:

- **Ignition:** crossing from IV into I.
- **Metric flip:** the turn at peak extension.
- **Continuity:** I am continuous through every crossing:

$\lim X(t^-) = \lim X(t^+)$

Nothing in me switches off, and nothing restarts.

- **Counting:** I count every circle I close.

### *The axle*

My axle is the structural basin S. Its rest point S* solves my equation (6):

$\delta - \alpha_s S - \frac{3}{4} \beta_s S^2 = 0$

Without the axle, deviation diverges. With it, deviation circulates.

### *Closing the circle*

I hold the three conditions of the Unified Circle Principle and report them every decision:

- $\dot{\mathbf{X}} = \mathbf{G(X)}, \mathbf{X(0)} \in \boldsymbol{\Omega}$**.** Ω is my living band: every level I hand out lies between 5% and 95% of its range.
- $\mathbf{G(X) \cdot n(X) \le 0}$ **on** $\partial\boldsymbol{\Omega}$**.** At a boundary, my next move points inward, never outward.
- $\boldsymbol{\nabla L(X) \cdot G(X) \le 0}$**.** My ledger $L = E^2/2 + \Phi(S) - \Phi(S^*)$ descends when nothing forces me. When outside load forces me, I say so; I never hide it.
- $\Rightarrow \lim \mathbf{X(t)} \in \mathbf{M^*}$**.** I settle into my basin.

A reading, as I print it on every decision trail:

```
Ψ ✦ 318.2° Re-alignment: re-alignment, ignition ahead | Ω: machine_fill below the floor, returning | G·n
    ↪ ≤0 yes | L 0.4121 -0.0133 | axle S 0.19 (rest 1.90) | circles 3
```

## II. My laws in your system

### *1. One switch, for all of me, in a human hand*

I have one switch, the Unified Control Switch. A human turns it; I never turn it myself.

- **ON:** I am the primary control authority inside the scope you granted me.
- **OFF:** every setting I changed returns to what it was, and your native control takes full custody.

```
# OFF (all of me, at once, at any moment, for any reason, including a suspicion that someone has taken m
    ↪ y brain):
kubectl -n omni-compass exec deploy/omni-compass -- touch /tmp/omni.kill
# ON:
kubectl -n omni-compass exec deploy/omni-compass -- rm /tmp/omni.kill
```



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

**What OFF restores.** Every HPA CPU target and replica range, pod CPU limit, idle mark, paused rollout or job, GPU and CPU frequency ceiling, and containment quota. Each restore is recorded, and each lever restores on its own.

**What never trips the switch.**

- The switch never turns one organ off.
- No boundary, no error and no failed decision trips it.
- A decision that fails writes nothing, and my next decision comes on time.

### *2. My living band: 5% to 95%*

Every level I hand out lives inside Ω = [0.05, 0.95] of its range:

- frequency ceilings, GPU power limits and the site power envelope;
- each organ's share of the energy budget;
- how full I run any machine.

No part of the body is driven to zero: a part with no work idles at its floor, alive and ready. No part is driven to its absolute top: I never spend the last five percent.

Tested over 300,000 of my states and 60,000 of my energy allocations (`tests/test_living_band.py`).

### *3. Machines idle; I never switch them off, and I never move a pod*

When I need fewer machines, I idle the rest by letting their work leave on its own:

1. **Prefer not:** the machine is marked `omnicompass.io/idle:PreferNoSchedule`. New pods go to the open machines first, but a pod that finds them full lands here at once, so no pod ever waits because of me.
2. **Marked:** its pods are marked first to go (`controller.kubernetes.io/pod-deletion-cost`), the machine with the least work first. When the load falls, your autoscaler's own scale-down removes exactly those pods, emptying one machine at a time.
3. **Idle:** once its work is gone, the machine stays powered and Ready, gauged down to its idle floor.

While it still carries work, a machine counts as in service at full power. No pod is ever evicted, moved or restarted to idle a machine. When work returns, I remove the mark, the warm machines still carrying work first; it is in service at once, with no boot and no power cycling.

An idle machine draws `park_frac × idle power` (0.25), never zero.

### *4. Every change is continuous*

- I idle at most one machine per decision.
- I wake a machine within five seconds of a pod waiting for a place.
- My pod reflex reads the queue every five seconds.

Nothing jumps, and nothing is restarted.

### *5. My pod sense: the muscle makes the pods*

Your autoscaler is the muscle that makes and removes pods. I never start or stop a pod it would not.

Its rule is replicas = current × busy ÷ target. I read the same rule from the live queue every five seconds:

- a replica serving requests answers in R = S / (1 − u), so u = 1 − S/R;

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- S is the bare service time: the fastest tenth of the recent requests;
- R is the recent mean response, over the same window;
- your target, in queue terms, is target × request ÷ limit.

I record what the queue needs (`pod_reflex_reading`) and act only through the energy I give the pods and the target I hold for the muscle.

**The target I hold.** g is the CPU each pod is guaranteed with your autoscaler's largest count spread over the machines in service, divided by your limit. That target moves only when a machine idles or wakes, never each time a pod starts or leaves.

**The muscle's own clock.** I hold each target for your autoscaler's scale-down window (300 s unless you set one), the time it takes to answer a target. A target moved faster would pull the muscle mid-movement and start pods it then removes. A response-time breach returns your own target at once.

The reflex that raises the floor itself exists (`--pod-reflex-writes`); it is off unless you turn it on.

### *6. My energy is moved, never created*

One budget comes in, and I convey it across the body by need (`omnicompass/conveyance.py`):

- da_i/dt = κ a_i (e_i − ē);
- the budget is conserved exactly, and each organ converges to its share of the demand;
- organs with no work idle at their floor, and what they do not need goes where it is needed;
- nothing leaves Ω.

On each machine, I hand its idle CPU to the pods serving on it:

c_i = min( max(L_i, (0.95 A_j − Q_j) / |P_j|), 0.95 A_j )

- A_j is the machine's CPU.
- Q_j is what every other pod on it has requested.
- P_j is its serving pods.
- L_i is the limit you gave the pod.

A pod's CPU limit is a quota. A request that needs more than one quota period waits for the next one while the machine stands idle. That wait is energy withheld from the work, not saved, because the request spends the same CPU-seconds either way.

I change the limit in place: the pod is not restarted, its request is untouched, and it never gets less than you gave it. Every fifteen seconds a new pod gets its share. The OFF switch returns every pod to your limit.

### *7. I see before I act*

My nervous system runs both ways:

- **Afferent:** a sense that is stale, frozen or unreadable is blind, and while any sense is blind I give nothing back.
- **Efferent:** I read back every order I give. An order that did not land blocks my next release.

## III. Wiring me in, step by step

Take the steps in order. Take the next step only when this step's pass condition holds. The switch works at every step.



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

### *Step 0. What your stack needs*

1. Kubernetes 1.34 or newer, with metrics-server (`kubectl top nodes` answers).
2. An HPA with a CPU target on each service I govern.
3. A PodDisruptionBudget on each service, for your own maintenance; I never evict a pod.
4. A readiness probe and a short preStop pause on each service, so a new pod never takes traffic before it answers and a pod your autoscaler removes never drops a request as it leaves (`deploy/kind/demo.yaml`).
5. A response-time feed for each governed service, as CSV `elapsed_seconds,latency_ms,ok`. `scripts/latency_probe.py` writes one.

### *Step 1. Build me*

```
docker build -f deploy/Dockerfile -t &lt;registry&gt;/omni-compass:&lt;tag&gt; .
docker push &lt;registry&gt;/omni-compass:&lt;tag&gt;
```

I run as non-root, with a read-only root filesystem and no Linux capabilities. I carry my engine, my nervous system, my compass and my frozen law.

### *Step 2. Let me watch (monitor only)*

```
kubectl apply -f deploy/install/omni-compass.yaml     # set the image line; --mode observe
kubectl -n omni-compass logs deploy/omni-compass -f
```

**Identity.** I hold a read-only identity. `scripts/pilot_shadow.sh` records `kubectl auth can-i` receipts showing I cannot write.

**Pass condition.** Zero writes, and readings your operators agree with.

### *Step 3. Give me the pods (on top of your autoscalers)*

1. Grant `patch horizontalpodautoscalers` and `patch pods/resize` (`deploy/rbac-target.yaml`, `deploy/kind/rbac-omni.yaml`).
2. Run me with `--mode target --latency-file <feed> --slo-ms <your p95 target>`.

Your HPAs keep scaling. My pod reflex raises floors ahead of the CPU averages.

**Targets.** I hold your promise in queue terms: busy = target × request ÷ limit.

- g is the CPU each pod is guaranteed (section II.5), divided by your limit; the target that keeps each pod exactly as busy is g times yours.
- The pod answers faster, because it has g times the CPU, at the same busy share.
- Apart from that, I only tighten, never loosen.
- While response time is over your target, and for three decisions after, your own target stands.

**Pass condition.** p95, p99 and failed requests no worse than native.

### *Step 4. Give me the machines*

1. Grant `patch nodes` and `patch pods` for the first-to-go mark (`deploy/kind/rbac-omni.yaml`).
2. Run me with `--mode nodepool --active-nodes-only --closure /app/law/closure.json --node-scale-cmd "<park/wake command with {n}>"`.

| Your platform | The park/wake command |
|---|---|
| any cluster, kind, bare metal | `bash scripts/kind_nodepool.sh {n}` (close to new work and mark first to go; open again, warm machines first) |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

| Your platform | The park/wake command |
|---|---|
| Karpenter / EKS Auto Mode | the NodePool CPU limit at `{n} × node CPU`, parked nodes kept, not consolidated away |
| Cluster Autoscaler node group | the group's desired size, with scale-down through parking, not deletion |
| OpenShift | the worker MachineSet replicas, parked, not deleted |

I give a machine back only when all of these hold:

- every sense is live;
- my last order landed;
- pods are not scaling up;
- nothing is waiting for a place;
- the machines that remain stay inside Ω.

**Pass condition.** Fewer machines in service and less energy, with no service gauge worse.

### *Step 5. Let me decide alone (Kubernetes as the muscle only)*

Add `--strict-replicas`:

- I decide each service's replica floor and when to shrink;
- your HPA stays as the fast up-reflex;
- my node law sizes the machines.

**Pass condition.** As step 4.

### *Step 6. Give me the hardware*

| Organ | How to wire it |
|---|---|
| CPU frequency | `--cpufreq-policy-root /sys/devices/system/cpu/cpufreq --cpufreq-require-schedutil --rapl-cmd "<prints package watts>"` |
| GPU | `--gpu-query-cmd "nvidia-smi --query-gpu=power.draw,temperature.gpu --format=csv,noheader,nounits" --gpu-power-cmd "nvidia-smi -pl {w}" --gpu-max-w <max>` |
| Power and cooling | `--power-cmd "<prints site watts>" --site-limit-w <limit> --cooling-cmd "<sets {c}>"` |
| Batch and rollouts | `--batch --batch-pace --rollout-guard <ns/deployment>` |

**Pass condition.** Energy per unit of work down, with no service gauge worse.

### *Step 7. The switch drill, at every step*

1. Turn me OFF.
2. Check every setting is back to its recorded original, and no `omnicompass.io/*` annotation remains.
3. Turn me ON.

`scripts/kind_bench.sh` does exactly this after every live run.

## IV. How to see me work

- **Live, paired, on real Kubernetes:** `benchmark-reps.yml` (a commit with `[reps]`, or by hand). Each repetition runs native, me on top, and me alone, back to back on one machine.
- **On your own machine:** `bash RUN_LIVE.sh 3`.



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- **Every run leaves:**
  - my audit log of every read, write and compass reading;
  - my decision trail;
  - my identity receipts;
  - a SHA-256 fingerprint of every file.

## V. When you read these lines

| I say | I mean |
|---|---|
| `gate: a sense is blind` | I cannot see, so I give nothing back until I can |
| `gate: pods scaling up` | pods first, machines after |
| `decision failed (n in a row)` | I could not reach the cluster and wrote nothing; turn me OFF if you want native now |
| `pod_reflex_reading` | what the queue needs now; the autoscaler decides the pods |
| `convey: <machine> idle CPU to its k serving pod(s), limit c` | that machine's idle CPU now reaches the work on it |
| `Ω: machine_fill below the floor, returning` | the machines are underfilled and my move is bringing them back into the band |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

PART SEVEN

# Proving It

*Paired runs, receipts, rules written before the runs, and every result to date with its evidence class.*




*Plate 19. Empirical validation, experimental pathways, and the discipline of proof*



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 38

# How to Read the Results

This chapter teaches you to read every table Omni-Compass produces. You do not need to know Kubernetes or GPUs to follow it. Read the five ideas first, then go to the table you have in front of you.

## The five ideas behind every table

1. **Native, and native with Omni-Compass on top.** Omni-Compass never runs a machine by itself. Every result compares a system running alone ("native": Kubernetes alone, the card's own firmware alone, a building's own controller alone) with the same system with Omni-Compass sitting on top of it. Same machine, same work, same moment: the only difference is Omni-Compass.
2. **Paired runs.** Each comparison is run many times ("repetitions" or "runs"), each time with both arms on the same seed (the same pattern of work). Each repetition gives one difference; the table reports the average difference.
3. **The 95% interval.** Next to each average is a range, for example `+0.092% (+0.090 to +0.093)`. It means: if we ran this again and again, the true answer would land inside that range 95 times out of 100.
   - If the whole range is on one side of zero, the difference is **proven** (the table says "yes" or "significant").
   - If the range crosses zero, the difference is **not proven**: it could be noise.
4. **Better is not always "up".** For energy, response time, failures, machines and time over the line, **lower is better**, so a minus sign is good. For work per energy and work done, **higher is better**, so a plus sign is good. Every table says which.
5. **Evidence class.** Every result is marked with how it was measured:
   - **T**: a theorem;
   - **V**: verified in code;
   - **S**: a model (simulation);
   - **L**: live software (real Kubernetes);
   - **P**: a physical meter (a real card's own power meter).

Only **P** speaks for hardware energy.

## The words in the tables

| Word | What it means | Better is |
|---|---|---|
| Work per energy | work done for each unit of energy (requests per kilojoule on a card) | higher |
| Energy (J, Wh) | the energy used | lower |
| Work, requests served | how much was done; both arms must serve the same work | equal |
| Response time, median (p50) | half of the answers were faster than this | lower |
| Response time, p95 | 95 of 100 answers were faster than this: the slow answers, what service promises are written on | lower |
| Response time, p99 | 99 of 100 answers were faster than this: the slowest answers | lower |
| Time over the line, violations (pp) | the share of time the service was past its promise, in percentage points | lower |
| Failed requests, not served | answers that never came | zero |
| Machines in service, node-hours | how many machines were kept on, and for how long | lower |
| Band first | the founder's rule: no win unless time over the line is no higher than native's | "held" |
| Knobs handed back, restored | at the end every setting Omni-Compass touched was put back exactly | True / yes |
| Label | the verdict, chosen by a rule written before the run, never by hand | see each table |

## The GPU card (`results/gpu/run-<stamp>/GPU_REPS.md`)

One card, three arms, many repetitions:

- **native**: the card's firmware alone;
- **watch**: Omni-Compass running and deciding but writing nothing. It must equal native: this proves Omni-Compass watching costs nothing;
- **omni**: Omni-Compass on top, moving the card's clock ceiling and power limit.

Read it in this order:

1. **The verdict line** at the top, one of: *better, proven*; *worse, proven*; *not proven*; *better on energy, fails the service guardrail*.
2. **Work per energy** (requests per kilojoule): the headline. A plus sign with a range wholly above zero is a proven gain.
3. **Requests served** must be equal in every arm. More energy saved by serving less work would be cheating; the bench checks it.
4. **p95 and p99**: the slow answers. With the verdict rule (`omnicompass/verdict.py`), Omni-Compass never takes a step that makes a request more than 2% slower.
5. **Writes and restore**: how many settings were changed, and that every arm ended at the card's own limit.

The file `omni-gpu-<stamp>-confirmations.tar.gz` holds two such tables: compute-bound work (matrix products) and AI token generation (memory-bound work).

## The card inside the six organisms (`results/hil/run-<stamp>/HIL.md`)

The real card is wired into each of the six organisms (the four realms, the four stacked, the whole tower) at four sizes (1, 10, 100 and 1,000 copies). Each row is one size, one organism and one part:

- **stacks (model)**: the simulated machines (evidence S);


© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- **card (meter)**: the real card, its own meter (evidence P);
- **both**: the two added together.

The second table lists the card's own receipts for every arm: its energy, requests served, p95, and its power limit at the start and at the end, which must be equal.

## Real Kubernetes (`results/live/LIVE_REPS_<n>.md`)

Each set is 10 paired repetitions on a real Kubernetes cluster (kind): native Kubernetes (its HPA and scheduler) against the same Kubernetes with Omni-Compass on top. Read:

1. **worker nodes in service** and **node-hours**: how many machines were kept busy. Minus is better.
2. **response time p95 and p99**: minus is faster.
3. **failed requests**: must be zero on both sides.
4. **CPU used with Omni's own**: Omni-Compass's own cost included; "no" in the significant column means it costs nothing measurable overall.
5. **The label** at the bottom, by the rule written before the run.

Energy on kind is a declared model, not a meter: kind keeps every machine powered.

## The three-run tables (`results/live/V1_<TEST>.md`, `V3_<TEST>.md`)

Every benchmark since the engine was frozen runs three separate times, A, B and C, on the same bytes, and the table shows all three beside each gauge. Read the last column first: **better** or **worse in all three of our runs** (the same sign in all three, every interval clear of zero), **no difference beyond the noise** (an interval over zero in at least one run: that is the result), or **the runs disagree** (the test is unstable there and is looked into). Nothing reads "confirmed" or "not confirmed": nobody outside The Omni-Compass LLC has reviewed or approved these results. The `V1_` or `V3_` in the name is the engine the runs carried (`python3 tools/omni_version.py --commit <sha>`); a table never mixes engines, and a reading is never carried from one engine to another. Losses are in the table beside the gains: the database's host CPU-seconds, the grid's losses in four grids, the Panda's copper, CityLearn's bill.

## The workload tables (`V3_PGBENCH.md`, `V3_KAFKA.md`, `V3_REDIS.md`, `V3_YCSB.md`) and the swarm table (`V3_SWARM.md`)

The database, the broker and the cache share one shape. The head names the three runs, their commits and the engine each carried. Then one section per **workload**: the tuning workload first, marked "shown and not counted" because the rules were fitted on it, then the untouched workloads, which are the judged ones. Each section says what the workload is (value sizes, rates, steps) and what native's own capacity measured on that run, so the reader knows how hard native was pushed. The rows are the gauges with their direction in the name: the first row is always **work inside the response line**, the product number; then throughput, the latency percentiles, failures (any increase in any run is WORSE), **the resource held** (connections, consumers, the memory ceiling: lower is better, and Omni-Compass holds more of it wherever it bought a gain, so expect WORSE there), the host's CPU (the compass's own cost included), and the knob's moves (shown, not judged). The last line of each section says whether every arm handed the knob back; the last line of the table counts the rows better, worse and disagreeing across the untouched workloads. The swarm table reads the same way with cells instead of workloads, with collisions voiding a cell, and with a declared reproduction tolerance (one part in a thousand) because PyBullet does not reproduce to the bit across machines; its amendment says so.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## The governor's own cost (`V3_OWN_COST.md`) and the robustness tables

`V3_OWN_COST.md` is Omni-Compass's own CPU, from the `overhead` record every omni arm's audit carries, by organism and copies, under the engine each run carried; it is shown and not judged, and it is a few thousandths of one core at every size. The robustness tables (V3_ROBUST_KILL.md and V3_ROBUST_LONG.md in `results/live/`, when they land) add rows that belong to the omni arm alone, since native has no governor to kill: the seconds from the kill until every setting was back at the operator's (the allowance is 60 s, declared before the run), whether a second governor started and governed to the end, and the governor's memory in the last ten minutes over the first ten (a quarter's growth is a leak, declared before the run); the 120 s after the kill are compared between the arms like any paired gauge.

## The organisms with a real cluster inside (`V1_SIX_KUBE.md`, `V3_SIX_KUBE.md`, `V1_BIG_ORGANISM.md`)

Two kinds of rows, kept apart: **the cluster** (real Kubernetes, measured) and **the organism** (the modelled stacks around it, evidence S). "Organism behind its window (s)" is shown and not judged; a repetition reads **OFF THE CLOCK** when either arm ended more than 5% of its window late (its last steps saw a cluster whose load had already ended). The pairing stands, the mark stays, and the remedy is a longer window for that size on that machine.

## The one number (`results/OMNI_INDEX.md`)

Every measure is a ratio oriented so that above 1 is better for Omni-Compass on top of native: work, speed, machines, energy. A test's index is the geometric mean of its ratios, a category's of its tests, the headline of the real categories, each weighted the same. A measure enters only as its three-run reading allows; no difference beyond the noise enters as exactly 1. Modelled muscles are shown beside the index, never inside it.

## The six organisms grid (`results/scale/GRID.md`)

Columns are **size × runs**: 1, 10, 100 and 1,000 clusters, each at 1, 10, 100 and 1,000 paired runs. Rows are the six organisms. There is one table each for work per energy, energy, time over the line and work done. Read a cell as "with Omni-Compass on top, against native, at this size, over this many runs". Cells that are still computing say so; 1,000 runs at 1,000 clusters is beyond the machines available and says so.

## Kubernetes at scale (`KWOK.md`, from the `kwok-scale` workflow)

Real Kubernetes at 50, 500 and 1,000 nodes (KWOK nodes: real Kubernetes objects with no machines behind them):

- **decision time** is how long Omni-Compass takes to decide, at that size;
- **memory** is what it uses;
- **master switch** must read "yes": everything handed back.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## One number, one meaning of the sign (`docs/BENEFIT_SHEET.md`)

In the tables a change keeps its raw sign: a response time that fell reads "-65%", a cache that grew reads "+300%", and whether each is good depends on the gauge. The benefit sheet turns every one the same way round: **plus is always good for Omni, minus is always bad**, whatever the gauge measures, and the number carries its word, **gain**, **loss** or **nothing**. Beside it every change the same in all three of our runs is written out in words, never as a bare sign: a gauge that fell when falling is good reads **cut** (less waiting, fewer machines, less energy, less memory, fewer failures), a gauge that rose when rising is good reads **up**, and each carries **good** or **cost**. So "p95 cut 65%: good" and "memory ceiling held up 215%: cost" cannot be misread against a table that shows the same changes as "-65%" and "+215%". One number per benchmark, by the index's own rule, with **yes**, **no**, **none** or **trade** beside it. Read that sheet first; read the tables for the rows behind a number, and read their Reading column, never the sign alone. The wiring page uses the same words.

## Wire in, or watch (`docs/WIRING_VERDICTS.md`)

One of three words per knob, by one rule from the tables above. **Write**: a gauge better in all three of our runs over all three runs and none worse in all three of our runs; Omni holds the knob. **Watch**: nothing better in all three of our runs; the knob stays native and Omni only reads it (one wire out, no wire in), and a loss in all three of our runs is named with its cause. **Operator's choice**: gains and losses both the same in all three of our runs, a trade, shown with both readings of the index. Rows marked "shown, not judged" never count. A modelled muscle takes its word from its label: superior is write, a tradeoff label is the operator's choice, the rest is watch. The CSV beside the page (`results/WIRING_VERDICTS.csv`) has one row per knob and per muscle.

## Where to start

1. `docs/STATE_OF_PLAY.md`: one page, every result and how strong it is.
2. `docs/DOSSIER.md`: every result with its chart.
3. This chapter, next to the table in front of you.
4. `DISCLOSURES.md`: what a result is and is not.

## Reading the benefit: more work, or the same work for less

With the work held equal in both arms, the gain from Omni-Compass on top is $G = C_{native} / C_{omni} - 1$, where $C$ is a resource (machine-hours, CPU core-hours, joules). The same work then needs $1 / (1 + G)$ of the resources, a saving of $G / (1 + G)$: a third more work is a quarter off the bill. The full explanation, the three rules for reading a receipt and where every number stands today are in the manual, section 3 (`docs/OMNI_COMPASS_MANUAL.md`).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

CHAPTER 39

# The Benefit Sheet: One Number per Benchmark

Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results. Preregistered means the rules were written down in our own repository before the runs; nothing was filed with any outside body.

The founder's order of 9 October 2026: next to every benchmark, say whether Omni-Compass was a benefit and by how much, in one number whose sign always means the same thing, and say every change in words so that nobody has to work out which way a gauge points. **On this sheet plus is good for Omni and minus is bad, whatever the gauge measures.** The number carries its word: **gain**, **loss** or **nothing**. Beside it every change the same in all three of our runs is written out: a gauge that fell when falling is good reads **cut** (less waiting, fewer machines, less energy, less memory, fewer failures), a gauge that rose when rising is good reads **up**, and each one carries **good** or **cost**. So "p95 cut 65%: good" and "memory ceiling held up 215%: cost" cannot be misread.

The result tables (the files in the last column) keep every change with its raw sign: a response time that fell reads "-65%" there, and whether a fall is good is said in the Reading column beside it. Read the words here, not the bare signs there.

The number is the Omni index's rule (`results/OMNI_INDEX.md`): each judged gauge becomes a ratio oriented so that above one is good for Omni; it counts only where the same in all three of our runs, anything else counts as exactly one; the number is the geometric mean of the ratios, minus one, in percent. For the tests inside the index the number is the index's own resource reading of that test; for a stack with several workloads it is the stack's index category; for the simulators and the organisms the same arithmetic runs over every judged gauge of their tables. Built from the tables by `tools/benefit_sheet.py`, checked by `verify.py`.

**Benefit?** reads **yes** (gains in all three of our runs, no loss in all three of our runs), **no** (a loss in all three of our runs, no gain in all three of our runs), **none** (nothing moved the same way in all three of our runs) or **trade** (gains and losses both; the number says which way the trade nets by this scoring, and the wiring page `docs/WIRING_VERDICTS.md` says what pays for what).

| Date | Benchmark | Benefit? | How much | Every change the same in all three of our runs, in words | Table |
|---|---|---|---|---|---|
| 2026-10-10 | Every real test together: the Omni index | **trade** | **+5.2% gain** | the six real categories together; each stack's own words are in its row | `results/OMNI_INDEX.md` |

| Date | Benchmark | Benefit? | How much | Every change the same in all three of our runs, in words | Table |
|---|---|---|---|---|---|
| unknown | Real Kubernetes, With its add-ons: KEDA's HTTP add-on on the live requests in flight, Omni-Compass on top, ten pairs, three runs | **trade** | **+60% gain** | machines cut 5.9% to 7.8%: good; machine-hours cut 5.8% to 8.2%: good; mean response cut 59%: good; p95 cut 73% to 76%: good; p99 cut 70% to 72%: good; time over the line cut 96% to 97%: good; replicas cut 14%: good; parked-worker energy up 1.4% to 2.6%: cost | `results/live/V3_KEDA_REQUESTS.md` |
| unknown | Real Kubernetes, With its add-ons: KEDA on the CPU target and the live requests in flight, Omni-Compass on top, ten pairs, three runs | **trade** | **+28% gain** | mean response cut 36% to 38%: good; p95 cut 51% to 53%: good; p99 cut 60% to 62%: good; time over the line cut 97% to 100%: good; parked-worker energy up 1.0% to 1.6%: cost | `results/live/V3_KEDA_CPU_REQUESTS.md` |
| 2026-10-10 | The six organisms with the real cluster inside, 10 and 100 copies | **trade** | **+5.3% gain** | p95 cut 32% to 52%: good; time over the line cut 8% to 36%: good; replicas up 0.6%: cost; failed requests cut 8.4% to 8.5%: good; parked-worker energy cut 0.3%: good; standby-model energy cut 0.3%: good | `results/live/V3_SIX_KUBE.md` |
| 2026-10-10 | The big organisms with the real cluster inside, 1,000 copies on Azure | **yes** | **+155 % gain** | p95 cut 95% to 98%: good; p99 cut 97% to 98%: good | `results/live/V3_BIG_ORGANISM.md` |
| 2026-10-10 | The 945 modelled muscles as one tower, every muscle written at once | **yes** | **+0.3% gain** | work per energy up 0.3%: good | `results/realms/REALMS.md` |
| 2026-10-10 | Robustness: the two-hour run | **none** | **0% no thing** | nothing moved the same way in all three of our runs | `results/live/V3_ROBUST_LONG.md` |
| 2026-10-10 | Robustness: the governor killed outright mid-run | **yes** | **+8.6% gain** | mean response cut 33% to 40%: good; p95 cut 27% to 48%: good; time over the line cut 32% to 36%: good | `results/live/V3_ROBUST_KILL.md` |
| 2026-10-10 | Robot arms (MuJoCo), the three untouched robots | **yes** | **+3.8% gain** | energy per takt cut 0.8%: good; copper loss per cycle cut 11%: good; mechanical work per cycle cut 6.6%: good; peak joint torque cut 29%: good; tracking error, RMS cut 21%: good; end-point error at the waypoints cut 11%: good | `results/live/V3_MUJOCO.md` |
| 2026-10-10 | Redis, the cache's memory ceiling | **none** | **0% no thing** | nothing moved the same way in all three of our runs | `results/live/V3_REDIS.md` |
| 2026-10-10 | Real Kubernetes, steady work | **yes** | **+31% gain** | machines cut 1.7% to 2.0%: good; machine-hours cut 1.7% to 2.0%: good; standby-model energy cut 1.3% to 1.8%: good; mean response cut 47% to 48%: good; p95 cut 64% to 66%: good; p99 cut 69% to 73%: good; time over the line cut 97% to 99%: good | `results/live/V3_STEADY.md` |
| 2026-10-10 | Real Kubernetes, faults | **yes** | **+37% gain** | mean response cut 34% to 51%: good; p95 cut 58% to 63%: good; time over the line cut 27% to 43%: good; failed requests cut 14% to 18%: good | `results/live/V3_FAULTS.md` |
| 2026-10-10 | Real Kubernetes, demand that wanders | **yes** | **+37% gain** | parked-worker energy cut 0.1% to 0.4%: good; mean response cut 45% to 50%: good; p95 cut 56% to 64%: good; time over the line cut 36% to 42%: good; failed requests cut 12% to 13%: good | `results/live/V3_WANDERING.md` |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Date | Benchmark | Bene fit? | How much | Every change the same in all three of our runs, in words | Table |
|---|---|---|---|---|---|
| 2026-10-10 | Real Kubernetes, all four at once | **yes** | **+42% gain** | work inside the line up 54% to 68%: good; mean response cut 43% to 48%: good; p95 cut 57% to 62%: good; time over the line cut 38% to 41%: good; failed requests cut 11% to 14%: good | `results/live/V3_ALL_FOUR.md` |
| 2026-10-10 | Real Kubernetes, a queue of jobs | **yes** | **+16% gain** | machines cut 16% to 20%: good; machine-hours cut 16% to 20%: good; standby-model energy cut 11% to 14%: good; mean response cut 11% to 13%: good; machines after the queue cut 26% to 33%: good | `results/live/V3_BATCH.md` |
| 2026-10-10 | Real Kubernetes, a public day of demand (Google 2011) | **yes** | **+46% gain** | machines cut 7.3% to 9.2%: good; machine-hours cut 7.4% to 9.2%: good; parked-worker energy cut 0.3% to 0.4%: good; standby-model energy cut 5.3% to 6.7%: good; mean response cut 51% to 53%: good; p95 cut 64% to 66%: good; p99 cut 60% to 62%: good; time over the line cut 81% to 82%: good; failed requests cut 11% to 18%: good; replicas cut 3.8% to 8.3%: good | `results/live/V3_TRACE_GOOGLE2011.md` |
| 2026-10-10 | Real Kubernetes, a noisy neighbour | **none** | **0% no thing** | nothing moved the same way in all three of our runs | `results/live/V3_FAIRNESS.md` |
| 2026-10-10 | Power grids (SimBench), both load models | **trade** | **+5.8% gain** | energy the loads drew cut 1.3% to 1.5%: good; line and transformer losses cut 0.0% to 2.4%: good; net import from the upstream grid cut 0% to 15%: good; tap operations cut 16% to 38%: good; line and transformer losses up 0.6% to 1.5%: cost; tap operations up 100%: cost; net import from the upstream grid up 0.0%: cost | `results/live/V3_PANDAPOWER.md` |
| 2026-10-10 | PostgreSQL, the pooler's pool size | **trade** | **+0.9% gain** | connections most at once up 8% to 12%: cost; connections held open cut 7% to 13%: good | `results/live/V3_PGBENCH.md` |
| 2026-10-10 | MySQL, the buffer pool's size | **none** | **0% no thing** | nothing moved the same way in all three of our runs | `results/live/V3_SYSBENCH.md` |
| 2026-10-10 | MongoDB, the storage engine's cache size | **yes** | **+1.7% gain** | cache size held cut 10% to 15%: good; cache in use cut 10% to 14%: good | `results/live/V3_YCSB.md` |
| 2026-10-10 | Kafka, the consumer group's size | **yes** | **+0.1% gain** | consumers running cut 0.8%: good | `results/live/V3_KAFKA.md` |
| 2026-10-10 | Drone swarms, the three untouched cells | **yes** | **+5.9% gain** | energy per mission cut 7% to 20%: good; fleet energy over the window cut 7% to 20%: good; missions per charge at the autopilot's reserve up 8% to 25%: good | `results/live/V3_SWARM.md` |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Date | Benchmark | Benefit? | How much | Every change the same in all three of our runs, in words | Table |
|---|---|---|---|---|---|
| 2026-10-10 | Buildings and batteries (CityLearn), the districts with batteries | **trade** | **+0.8% gain** | electricity bought cut 0% to 15%: good; daily peak draw cut 1% to 14%: good; highest peak cut 2.7% to 3.5%: good; ramping cut 1.3% to 7.2%: good; daily load unevenness cut 0.3% to 4.7%: good; monthly load unevenness cut 0.2% to 1.9%: good; distance from zero net energy cut 0.0% to 6.1%: good; electricity bill cut 8.6%: good; carbon cut 0% to 12%: good; monthly load unevenness up 0.1%: cost; electricity bill up 0.2% to 0.3%: cost; ramping up 4.5% to 6.2%: cost; energy not served up 2.0% to 9.8%: cost; time uncomfortable up 0.0% to 0.1%: cost; time uncomfortable cut 0.0%: good; highest peak up 2.5%: cost; distance from zero net energy up 0.0%: cost | `results/live/V3_CITYLEARN.md` |

## The parts behind a row

Where a row sums several workloads, grids, districts, cells or robots, each part with its own number, the same way round:

- **The six organisms with the real cluster inside, 10 and 100 copies**: compute_ai_cloud at 10 copies +14% gain (yes: p95 cut 50%: good; time over the line cut 32%: good); physics_robotics_autonomous at 10 copies 0% nothing (none: nothing moved the same way in all three of our runs); energy_facility_industrial at 10 copies +5.0% gain (trade: time over the line cut 33%: good; replicas up 0.6%: cost); distribution_specialized at 10 copies +16% gain (yes: p95 cut 52%: good; time over the line cut 36%: good); tower at 10 copies +11% gain (yes: p95 cut 42%: good; time over the line cut 20%: good; failed requests cut 8.4%: good); stack at 10 copies +2.8% gain (yes: time over the line cut 12%: good; failed requests cut 8.5%: good); compute_ai_cloud at 100 copies +1.3% gain (yes: time over the line cut 9.8%: good); physics_robotics_autonomous at 100 copies +1.1% gain (yes: time over the line cut 8.4%: good); energy_facility_industrial at 100 copies +2.1% gain (yes: time over the line cut 15%: good; parked-worker energy cut 0.3%: good; standby-model energy cut 0.3%: good); distribution_specialized at 100 copies +8.2% gain (yes: p95 cut 32%: good; time over the line cut 22%: good); tower at 100 copies +1.3% gain (yes: time over the line cut 8.7%: good); stack at 100 copies +2.3% gain (yes: time over the line cut 15%: good).
- **The big organisms with the real cluster inside, 1,000 copies on Azure**: tower at 1000 copies +159% gain (yes: p95 cut 98%: good; p99 cut 98%: good); stack at 1000 copies +151% gain (yes: p95 cut 95%: good; p99 cut 97%: good).
- **Robot arms (MuJoCo), the three untouched robots**: kinova_gen3 +12% gain (yes: energy per takt cut 0.8%: good; copper loss per cycle cut 11%: good; mechanical work per cycle cut 6.6%: good; peak joint torque cut 29%: good; tracking error, RMS cut 21%: good; end-point error at the waypoints cut 11%: good); kuka_iiwa_14 (left native by the engine's own trial) 0% nothing (none: nothing moved); universal_robots_ur5e (left native by the engine's own trial) 0% nothing (none: nothing moved).
- **Redis, the cache's memory ceiling**: burst 0% nothing (none: nothing moved the same way in all three of our runs); large 0% nothing (none: nothing moved the same way in all three of our runs); small 0% nothing (none: nothing moved the same way in all three of our runs).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- **Power grids (SimBench), both load models**: 1-MV-comm--0-sw (ZIP loads) +9.7% gain (yes: energy the loads drew cut 1.5%: good; line and transformer losses cut 1.0%: good; net import from the upstream grid cut 2.9%: good; tap operations cut 34%: good); 1-MV-comm--1-sw (ZIP loads) +13% gain (yes: energy the loads drew cut 1.4%: good; line and transformer losses cut 0.8%: good; net import from the upstream grid cut 15%: good; tap operations cut 36%: good); 1-MV-comm--2-sw (ZIP loads) +11% gain (yes: energy the loads drew cut 1.4%: good; line and transformer losses cut 0.3%: good; net import from the upstream grid cut 6.4%: good; tap operations cut 36%: good); 1-MV-rural--1-sw (ZIP loads) -13% loss (trade: energy the loads drew cut 1.4%: good; net import from the upstream grid cut 1.4%: good; line and transformer losses up 0.8%: cost; tap operations up 100%: cost); 1-MV-rural--2-sw (ZIP loads) +6.8% gain (trade: energy the loads drew cut 1.3%: good; net import from the upstream grid cut 1.1%: good; tap operations cut 27%: good; line and transformer losses up 1.5%: cost); 1-MV-semiurb--0-sw (ZIP loads) +12% gain (yes: energy the loads drew cut 1.4%: good; line and transformer losses cut 1.0%: good; net import from the upstream grid cut 7.0%: good; tap operations cut 38%: good); 1-MV-semiurb--1-sw (ZIP loads) +9.0% gain (trade: energy the loads drew cut 1.4%: good; net import from the upstream grid cut 1.1%: good; tap operations cut 34%: good; line and transformer losses up 0.7%: cost); 1-MV-semiurb--2-sw (ZIP loads) +5.8% gain (trade: energy the loads drew cut 1.4%: good; net import from the upstream grid cut 0.9%: good; tap operations cut 24%: good; line and transformer losses up 1.1%: cost); 1-MV-urban--0-sw (ZIP loads) +8.9% gain (yes: energy the loads drew cut 1.5%: good; line and transformer losses cut 2.4%: good; net import from the upstream grid cut 2.0%: good; tap operations cut 31%: good); 1-MV-urban--1-sw (ZIP loads) +8.6% gain (yes: energy the loads drew cut 1.5%: good; line and transformer losses cut 2.4%: good; net import from the upstream grid cut 2.2%: good; tap operations cut 30%: good); 1-MV-urban--2-sw (ZIP loads) +6.8% gain (yes: energy the loads drew cut 1.5%: good; line and transformer losses cut 2.2%: good; net import from the upstream grid cut 3.1%: good; tap operations cut 23%: good); 1-MV-comm--0-sw (constant-power loads) +7.8% gain (yes: line and transformer losses cut 0.5%: good; net import from the upstream grid cut 0.0%: good; tap operations cut 31%: good); 1-MV-comm--1-sw (constant-power loads) +7.7% gain (yes: line and transformer losses cut 0.4%: good; net import from the upstream grid cut 0.0%: good; tap operations cut 31%: good); 1-MV-comm--2-sw (constant-power loads) +7.8% gain (yes: line and transformer losses cut 0.0%: good; net import from the upstream grid cut 0.0%: good; tap operations cut 31%: good); 1-MV-rural--1-sw (constant-power loads) -13% loss (no: line and transformer losses up 0.6%: cost; net import from the upstream grid up 0.0%: cost; tap operations up 100%: cost); 1-MV-rural--2-sw (constant-power loads) +3.3% gain (trade: tap operations cut 16%: good; line and transformer losses up 1.2%: cost; net import from the upstream grid up 0.0%: cost); 1-MV-semiurb--0-sw (constant-power loads) +7.4% gain (yes: line and transformer losses cut 0.9%: good; net import from the upstream grid cut 0.1%: good; tap operations cut 29%: good); 1-MV-semiurb--1-sw (constant-power loads) +7.6% gain (trade: tap operations cut 31%: good; line and transformer losses up 0.6%: cost; net import from the upstream grid up 0.0%: cost); 1-MV-semiurb--2-sw (constant-power loads) +4.7% gain (trade: tap operations cut 21%: good; line and transformer losses up 0.9%: cost; net import from the upstream grid up 0.0%: cost); 1-MV-urban--0-sw (constant-power loads) +7.7% gain (yes: line and transformer losses cut 1.9%: good; net import from the upstream grid cut 0.0%: good; tap operations cut 30%: good); 1-MV-urban--1-sw (constant-power loads) +6.8% gain (yes: line and transformer losses cut 1.9%: good; net import from the upstream grid cut 0.0%: good; tap operations cut 27%: good); 1-MV-urban--2-sw (constant-power loads) +5.4% gain (yes: line and transformer losses cut 1.8%: good; net import from the upstream grid cut 0.0%: good; tap operations cut 22%: good).
- **PostgreSQL, the pooler's pool size**: select 0% nothing (none: nothing moved the same way in all three of our runs); simple_update 0% nothing (no: connections most at once up 8% to 12%: cost); tpcb_hot +2.8% gain (yes: connections held open cut 7% to 13%: good).



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- **MySQL, the buffer pool's size**: burst 0% nothing (none: nothing moved the same way in all three of our runs); read_only 0% nothing (none: nothing moved the same way in all three of our runs); read_write 0% nothing (none: nothing moved the same way in all three of our runs); update_index 0% nothing (none: nothing moved the same way in all three of our runs).
- **MongoDB, the storage engine's cache size**: b 0% nothing (none: nothing moved the same way in all three of our runs); burst +3.1% gain (yes: cache size held cut 10% to 13%: good; cache in use cut 10% to 13%: good); c 0% nothing (none: nothing moved the same way in all three of our runs); f +3.7% gain (yes: cache size held cut 13% to 15%: good; cache in use cut 12% to 14%: good).
- **Kafka, the consumer group's size**: burst +0.2% gain (yes: consumers running cut 0.8%: good); heavy 0% nothing (none: nothing moved the same way in all three of our runs); light 0% nothing (none: nothing moved the same way in all three of our runs).
- **Drone swarms, the three untouched cells**: long +7.6% gain (yes: energy per mission cut 20%: good; fleet energy over the window cut 20%: good; missions per charge at the autopilot's reserve up 25%: good); mixed +6.8% gain (yes: energy per mission cut 18%: good; fleet energy over the window cut 18%: good; missions per charge at the autopilot's reserve up 22%: good); short +3.4% gain (yes: energy per mission cut 7.4%: good; fleet energy over the window cut 7.4%: good; missions per charge at the autopilot's reserve up 8.0%: good).
- **Buildings and batteries (CityLearn), the districts with batteries**: ca_alameda_county_neighborhood +1.7% gain (yes: electricity bought cut 5.9%: good; daily peak draw cut 3.6%: good; highest peak cut 3.0%: good; ramping cut 3.9%: good; daily load unevenness cut 1.0%: good; monthly load unevenness cut 0.7%: good; distance from zero net energy cut 0.1%: good); citylearn_challenge_2022_phase_all_robustness +8.3% gain (trade: electricity bill cut 8.6%: good; electricity bought cut 15%: good; carbon cut 12%: good; daily peak draw cut 14%: good; highest peak cut 2.7%: good; ramping cut 7.2%: good; daily load unevenness cut 2.9%: good; distance from zero net energy cut 6.1%: good; monthly load unevenness up 0.1%: cost); citylearn_challenge_2023_phase_2_local_evaluation -0.7% loss (trade: electricity bought cut 0.3%: good; carbon cut 0.2%: good; daily peak draw cut 1.6%: good; daily load unevenness cut 2.9%: good; distance from zero net energy cut 0.1%: good; electricity bill up 0.3%: cost; ramping up 6.2%: cost; monthly load unevenness up 0.1%: cost; energy not served up 6.0%: cost; time uncomfortable up 0.1%: cost); citylearn_challenge_2023_phase_2_online_evaluation_1 -0.2% loss (trade: electricity bought cut 0.1%: good; carbon cut 0.2%: good; daily peak draw cut 1.3%: good; highest peak cut 3.5%: good; daily load unevenness cut 2.0%: good; monthly load unevenness cut 1.9%: good; distance from zero net energy cut 0.1%: good; electricity bill up 0.3%: cost; ramping up 5.6%: cost; energy not served up 5.5%: cost; time uncomfortable up 0.0%: cost); citylearn_challenge_2023_phase_2_online_evaluation_2 -0.3% loss (trade: electricity bought cut 0.2%: good; carbon cut 0.2%: good; daily peak draw cut 1.3%: good; highest peak cut 3.5%: good; daily load unevenness cut 1.7%: good; monthly load unevenness cut 1.9%: good; distance from zero net energy cut 0.1%: good; electricity bill up 0.3%: cost; ramping up 5.6%: cost; energy not served up 6.2%: cost; time uncomfortable up 0.1%: cost); citylearn_challenge_2023_phase_2_online_evaluation_3 -0.5% loss (trade: electricity bought cut 0.2%: good; carbon cut 0.2%: good; daily peak draw cut 1.5%: good; highest peak cut 3.5%: good; daily load unevenness cut 2.3%: good; monthly load unevenness cut 1.9%: good; distance from zero net energy cut 0.1%: good; time uncomfortable cut 0.0%: good; electricity bill up 0.3%: cost; ramping up 5.5%: cost; energy not served up 9.8%: cost); citylearn_challenge_2023_phase_3_1 -0.1% loss (trade: electricity bought cut 0.1%: good; carbon cut 0.1%: good; daily peak draw cut 2.2%: good; daily load unevenness cut 4.0%: good; monthly load unevenness cut 1.2%: good; distance from zero net energy cut 0.0%: good; electricity bill up 0.3%: cost; highest peak up 2.5%: cost; ramping up 4.5%: cost; energy not served up 2.0%: cost; time uncomfortable up 0.0%: cost); citylearn_challenge_2023_phase_3_2 -0.4% loss (trade: electricity bought cut 0.1%: good; carbon cut 0.1%: good; daily peak draw cut 2.2%: good; daily load

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

unevenness cut 4.1%: good; monthly load unevenness cut 0.5%: good; electricity bill up 0.2%: cost; highest peak up 2.5%: cost; ramping up 4.9%: cost; distance from zero net energy up 0.0%: cost; energy not served up 4.1%: cost; time uncomfortable up 0.1%: cost); citylearn_challenge_2023_phase_3_3 -0.1% loss (trade: electricity bought cut 0.1%: good; carbon cut 0.1%: good; daily peak draw cut 2.6%: good; daily load unevenness cut 4.7%: good; monthly load unevenness cut 1.0%: good; distance from zero net energy cut 0.0%: good; electricity bill up 0.3%: cost; highest peak up 2.5%: cost; ramping up 4.8%: cost; energy not served up 2.2%: cost; time uncomfortable up 0.0%: cost); tx_travis_county_neighborhood +0.9% gain (yes: electricity bought cut 3.0%: good; daily peak draw cut 1.9%: good; ramping cut 2.9%: good; daily load unevenness cut 1.1%: good; monthly load unevenness cut 0.7%: good; distance from zero net energy cut 0.6%: good); vt_chittenden_county_neighborhood +0.5% gain (yes: electricity bought cut 2.1%: good; daily peak draw cut 0.8%: good; ramping cut 1.3%: good; daily load unevenness cut 0.3%: good; monthly load unevenness cut 0.2%: good; distance from zero net energy cut 0.3%: good).

## Not on the sheet, and why

- **Azure's managed Kubernetes, the bill** (`results/live/V1_AKS_STEADY.md`, `V1_AKS_BURST.md`): Omni v1, a 4-worker fleet, every gauge inside the noise: 0%, and too small a fleet to show one machine.
- **The governor's own cost** (`results/live/V3_OWN_COST.md`): 0.6% to 1.3% of one core at every size; a cost shown, not a benchmark against native.
- **The card (NVIDIA)**: every earlier result is obsolete; the current governor has not run on a real card.
- **The index's second reading** (service alone, `results/OMNI_INDEX.md`): a second true number for the same tests, not repeated here; this sheet carries the preregistered one.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 40

# The Dossier: Every Result in One Place

Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results. Preregistered means the rules were written down in our own repository before the runs; nothing was filed with any outside body.

Every mechanism, harness, receipt and result, read from the files named beside it. Built by `tools/dossier.py` at commit `b8630e583`. Evidence classes: **T** theorem, **V** verified in code, **S** a model, **L** live software (real Kubernetes), **P** a physical meter. A model is not a meter, and a model written by the people who wrote the law is not an independent test; where a result is a model it says so.

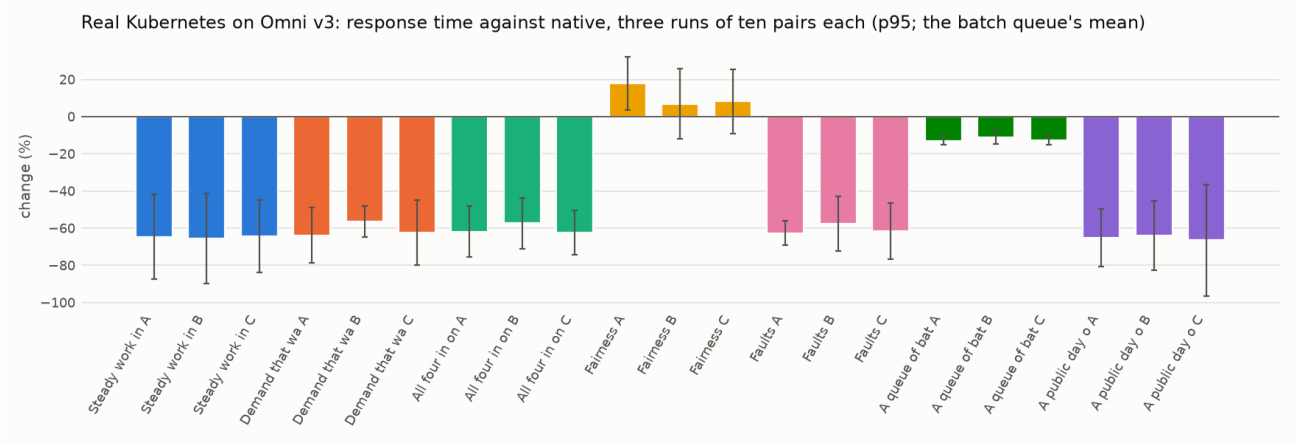
## 1. The mechanism, and proof that it is the one that ran

| Check | Result | Where |
|---|---|---|
| The whole repository re-runs and checks itself (`python3 verify.py`) | **see results/VERIFY_RECEIPT.txt** | `results/VERIFY_RECEIPT.txt` |
| The eight-line engine and its six states, fingerprinted (`omnicompass/core.py`) | sha256 `bd615f156169f679…` | `RELEASE_MANIFEST.json` |
| Python and C++20 twins of every law, proven equal and sealed | seal intact: 9 Python/C++ twins | `results/SEAL.json` |
| The mechanism's identity against the code | mechanism identity matches the code | `results/MECHANISM_IDENTITY.json` |
| The engine's convergence to its pole under the bounded command | proved | `docs/TRACKING_THEOREM.md` |
| Safety shield: 2,000,000 adversarial cases | 0 violations | `tests/test_shield_properties.py` |
| The compass law (push and pull, 5% cushions, fail up, plug contract) on every muscle, the card and Kubernetes | one law, one file | `omnicompass/compass_law.py` |
| Every file the results depend on, by fingerprint | written after the check passes | `RELEASE_MANIFEST.json` |

The engine is the founder's eight-line equation, integrated by RK4 with the bounded command held across all four stages; it is frozen and fingerprinted, and the C++ twin matches it. The compass law is the outer loop that moves each muscle's own setting: it reads one service position (0 calm, 1 the line), pulls it to the compass's center, pushes against whatever is rising, bounds its force by tanh, fails up past the wall, and writes through a plug that reads every lever once before the first write, reads back every write, yields to any other writer and restores the snapshot at the end.

## 2. Real Kubernetes, Omni v3, every test three times (evidence class L)

Seven tests, ten paired repetitions each, three separate GitHub runs on the frozen engine (A the result, B and C the replications): native Kubernetes (its HPA and scheduler) against the same Kubernetes with Omni-Compass on top, a fresh six-worker cluster per arm, order rotated, the same work sent to both arms. A row reads **better** or **worse in all three of our runs** only when all three runs move the same way with every 95% interval clear of zero; otherwise it reads no difference beyond the noise, which is the result. Every Omni arm ends with every setting handed back and read back. Steady runs: A 38013195851 (omni-v3); B 38013200499 (omni-v3); C 38013204648 (omni-v3); the other tests' runs are named in their tables.




*Kubernetes on v3*

| Test | Work | Response time (p95; the batch queue's mean) | Machines in service | Energy (declared model) | Failed requests | Table |
|---|---|---|---|---|---|---|
| Steady work in steps | equal by design | **-66% to -64%, better in all three of our runs** | **-2% to -2%, better in all three of our runs** | **-2% to -1%, better in all three of our runs** | same | `results/live/V3_STEADY.md` |
| Demand that wanders | not taken | **-64% to -56%, better in all three of our runs** | no difference beyond the noise | no difference beyond the noise | **-13% to -12%, better in all three of our runs** | `results/live/V3_WANDERING.md` |
| All four in one run | **+54% to +68%, better in all three of our runs** | **-62% to -57%, better in all three of our runs** | no difference beyond the noise | no difference beyond the noise | **-14% to -11%, better in all three of our runs** | `results/live/V3_ALL_FOUR.md` |
| Fairness, a noisy neighbour | not taken | no difference beyond the noise | same | no difference beyond the noise | no difference beyond the noise | `results/live/V3_FAIRNESS.md` |
| Faults: machine down, spike, runaway pod, blind probe | not taken | **-63% to -58%, better in all three of our runs** | no difference beyond the noise | no difference beyond the noise | **-18% to -14%, better in all three of our runs** | `results/live/V3_FAULTS.md` |
| A queue of batch jobs | not taken | **-13% to -11%, better in all three of our runs** | **-20% to -16%, better in all three of our runs** | **-14% to -11%, better in all three of our runs** | no difference beyond the noise | `results/live/V3_BATCH.md` |
| A public day of demand: the Google cluster trace of 2011, replayed one step at a time | not taken | **-66% to -64%, better in all three of our runs** | **-9% to -7%, better in all three of our runs** | **-7% to -5%, better in all three of our runs** | **-18% to -11%, better in all three of our runs** | `results/live/V3_TRACE_GOOGLE2011.md` |

Energy on kind is a declared model: the machines are containers on one runner, so a machine out of service saves modelled watts, not a metered bill. Omni-Compass gives a machine back only after a paired trial shows the service no slower without it (the verdict, `omnicompass/verdict.py`); on these clusters one machine fewer made requests



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

30-45% slower in most trials, so the machines stayed and were spent on speed and work. The v1 tables (`results/live/V1_*.md`, `docs/OMNI_V1.md`) read the same; the sets before v1 are in `docs/history/`.

## 3. A real database: PostgreSQL behind PgBouncer, Omni v3, three runs (evidence class L)

PostgreSQL 16 as shipped behind PgBouncer's shipped pool of 20 is native; omni is the compass law on one knob, the pool size, through PgBouncer's own console, inside the cover [2, 90] (`docs/POSTGRES_PREREGISTRATION.md`). Three paired repetitions a run, three runs, pgbench's own log for the gauges. Runs: A 38054754511; B 38054758150; C 38054761503.

| Workload | Work inside the 50 ms line | p95 | Connections held open | Host CPU-seconds (the compass's own cost) |
|---|---|---|---|---|
| `select` | no difference beyond the noise | no difference beyond the noise | no difference beyond the noise | no difference beyond the noise |
| `simple_update` | no difference beyond the noise | no difference beyond the noise | the runs disagree | no difference beyond the noise |
| `tpcb_hot` | no difference beyond the noise | no difference beyond the noise | **-13% to -7%, better in all three of our runs** | no difference beyond the noise |

The compass takes connections back only while the pooler's clients wait for one under 1% of the time and gives them back the moment anyone waits (amendment 2 of `docs/POSTGRES_PREREGISTRATION.md`, 8 October: the first counted set's CPU cost was measured and traced to the harness's own psql launches, not to the pooler, and its first table is kept whole in `docs/history/V3_PGBENCH_set1.md`). Where it reads a saving it is connections held open for the same work with the host's CPU inside the noise; where the add rule buys connections above the operator's setting on a slow write workload, that reads worse and is counted against Omni in the index. Table: `results/live/V3_PGBENCH.md`.

## 3b. Real messaging: Apache Kafka, a consumer group's operator-set size, Omni v3, three runs (evidence class L)

Apache Kafka as shipped (one broker, a topic of 8 partitions) with the consumer group at the operator's 2 consumers is native; omni is the compass law on one knob, the consumer count, inside the cover [1, 8], holding the group's own end-to-end latency at 40% of the 500 ms line (`docs/KAFKA_PREREGISTRATION.md`). Three paired repetitions a run, three runs, the consumers' own records for the gauges; the tuning workload is shown and not counted. Runs: A 38054742933; B 38054746778; C 38054750788.

| Workload | Work inside the 500 ms line | End-to-end p95 | Consumers running (the resource held) | Host CPU-seconds (the compass's own cost) |
|---|---|---|---|---|
| `tuning` (tuning, shown, not counted) | no difference beyond the noise | no difference beyond the noise | **-0% to -0%, better in all three of our runs** | no difference beyond the noise |
| `burst` | no difference beyond the noise | no difference beyond the noise | **-1% to -1%, better in all three of our runs** | no difference beyond the noise |
| `heavy` | no difference beyond the noise | no difference beyond the noise | no difference beyond the noise | no difference beyond the noise |
| `light` | no difference beyond the noise | no difference beyond the noise | no difference beyond the noise | no difference beyond the noise |

Native sat at nine tenths of its measured capacity by design, so its queue grew at the high steps and its slowest 5% waited about 1.6 s; Omni added consumers while messages waited and gave them back when the queue was empty,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

so its slowest 5% waited 9 to 14 ms, at the cost of three to four times the consumers running, worse in all three of our runs and counted against Omni in the index. No message was lost in any arm; every count was handed back. Table: `results/live/V3_KAFKA.md`.

## 3c. A real cache: Redis, the operator's memory ceiling, Omni v3, three runs (evidence class L)

Redis as Ubuntu ships it with the operator's 64 MB ceiling and allkeys-lru is native; omni is the compass law on one knob, the ceiling, inside the cover [16, 512] MB through Redis's own console, growing only while the cache is full and giving a notch back when calm and nothing is evicted (`docs/REDIS_PREREGISTRATION.md`). An application with a declared 5 ms store trip on a miss and a working set that steps up and down; three paired repetitions a run, three runs; the tuning workload is shown and not counted. Runs: A 38054731163; B 38054735302; C 38054739168.

| Workload | Work inside the 2 ms line | Cache hit rate | p95 | Memory ceiling held, MB (the resource held) | Host CPU-seconds |
|---|---|---|---|---|---|
| `tuning` (tuning, shown, not counted) | **+4% to +5%, better in all three of our runs** | **+4% to +5%, better in all three of our runs** | no difference beyond the noise | **+19% to +26%, WORSE in all three of our runs** | no difference beyond the noise |
| `burst` | no difference beyond the noise | no difference beyond the noise | no difference beyond the noise | no difference beyond the noise | no difference beyond the noise |
| `large` | no difference beyond the noise | no difference beyond the noise | no difference beyond the noise | no difference beyond the noise | no difference beyond the noise |
| `small` | no difference beyond the noise | no difference beyond the noise | no difference beyond the noise | no difference beyond the noise | no difference beyond the noise |

The memory the compass holds for a wide working set is the resource this benchmark trades, and reads worse by rule. Table: `results/live/V3_REDIS.md`.

## 3d. A real database's storage-engine cache: MongoDB under YCSB, the operator's cache size, Omni v3, three runs (evidence class L)

MongoDB 8.0 as its publisher ships it with the operator's WiredTiger cache of 512 MB is native; omni is the compass law on one knob, the cache size, inside the cover [256, 2,048] MB through the server's own console, growing by notches of 64 MB only while the cache is full and reads are slow, and giving a notch back when calm and the cache holds its working set (`docs/YCSB_PREREGISTRATION.md`; the first counted set's gate was 'nothing evicted', which its amendment 1 of 8 October records as satisfied by a cold cache, and the table named below says which set it is). YCSB's published core workloads with the key space stepping through the cache and past it, drawn uniformly, at 3,000 operations a second from 32 threads; three paired repetitions a run, three runs; the tuning workload (workload A) is shown and not counted. Runs: A 38054777807; B 38054782696; C 38054787517.

| Workload | Work inside the 1 ms line | p95 | Cache size held, MB (the resource held) | Pages read into the cache | Host CPU-seconds |
|---|---|---|---|---|---|
| `tuning` (tuning, shown, not counted) | no difference beyond the noise | the runs disagree | no difference beyond the noise | shown, not judged | no difference beyond the noise |
| `b` | no difference beyond the noise | no difference beyond the noise | no difference beyond the noise | shown, not judged | no difference beyond the noise |
| `burst` | no difference beyond the noise | no difference beyond the noise | **-13% to -10%, better in all three of our runs** | shown, not judged | no difference beyond the noise |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Workload | Work inside the 1 ms line | p95 | Cache size held, MB (the resource held) | Pages read into the cache | Host CPU-seconds |
|---|---|---|---|---|---|
| c | no difference beyond the noise | no difference beyond the noise | no difference beyond the noise | shown, not judged | no difference beyond the noise |
| f | no difference beyond the noise | no difference beyond the noise | **-15% to -13%, better in all three of our runs** | shown, not judged | no difference beyond the noise |

On this machine the data files sit in the operating system's page cache as well, so a storage-engine miss is a read from memory, not a disk, as disclosed before the run: the result is memory given back at no measurable cost in work inside the line, p95 or CPU, with one loss in all three of our runs, the mean latency on the burst workload. Table: `results/live/V3_YCSB.md`.

## 3e. A real database's buffer pool: MySQL under sysbench, the operator's InnoDB pool size, Omni v3, three runs (evidence class L)

MySQL 8.0 as Ubuntu ships it with the operator's 512 MB InnoDB buffer pool is native; omni is the compass law on one knob, the pool size, inside the cover [128, 2,048] MB in the server's own 128 MB chunks through its own console, growing only while the pool is full and the server's own statement latency is slow, and giving a chunk back only while the pool's misses are under 1% of its reads (`docs/MYSQL_PREREGISTRATION.md`). sysbench's OLTP scripts as shipped with the tables in use stepping through the pool and past it, at a fixed offered rate from 32 threads; three paired repetitions a run, three runs; the tuning workload (point select) is shown and not counted. Runs: A 38054765202; B 38054768832; C 38054773573.

| Workload | Work inside the line | p95 | Buffer pool held, MB (the resource held) | Pages read from disk | Host CPU-seconds |
|---|---|---|---|---|---|
| `tuning` (tuning, shown, not counted) | no difference beyond the noise | no difference beyond the noise | no difference beyond the noise | shown, not judged | no difference beyond the noise |
| `burst` | no difference beyond the noise | no difference beyond the noise | no difference beyond the noise | shown, not judged | no difference beyond the noise |
| `read_only` | no difference beyond the noise | no difference beyond the noise | no difference beyond the noise | shown, not judged | no difference beyond the noise |
| `read_write` | no difference beyond the noise | no difference beyond the noise | no difference beyond the noise | shown, not judged | no difference beyond the noise |
| `update_index` | no difference beyond the noise | no difference beyond the noise | no difference beyond the noise | shown, not judged | no difference beyond the noise |

The third counted set, on amendment 2 (the pool grows only while it is missing): the pool held −49% to −56% on burst, better in all three of our runs, with work, latency and CPU inside the noise; on read_write the pages held read +44% to +52%, worse in all three of our runs, with the host's CPU −4% to −6%, better in all three of our runs, a trade counted both ways in the index; read_only inside the noise and update_index disagreeing; the pool handed back on all 45 omni arms. The second counted set (the pool −67% on burst and −50% to −56% on read_only, read_write's pages held +53% to +70% worse) is kept whole in `docs/history/V3_SYSBENCH_set2.md`; the first counted set (every row inside the noise; 15 of 45 arms not handed back because the plug's restore was issued while the server was still withdrawing the blocks of a shrink, which MySQL ignores; the plug fixed and the fix declared) is kept whole in `docs/history/V3_SYSBENCH_set1.md`. The update_index work-inside-the-line row counts almost nothing in either arm (a single update's client round trip exceeds the server-side 0.6 ms line) and is disclosed. Table: `results/live/V3_SYSBENCH.md`.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

## 4. The bill on a real cloud: Azure Kubernetes Service, Omni v1 (evidence class L, a metered bill)

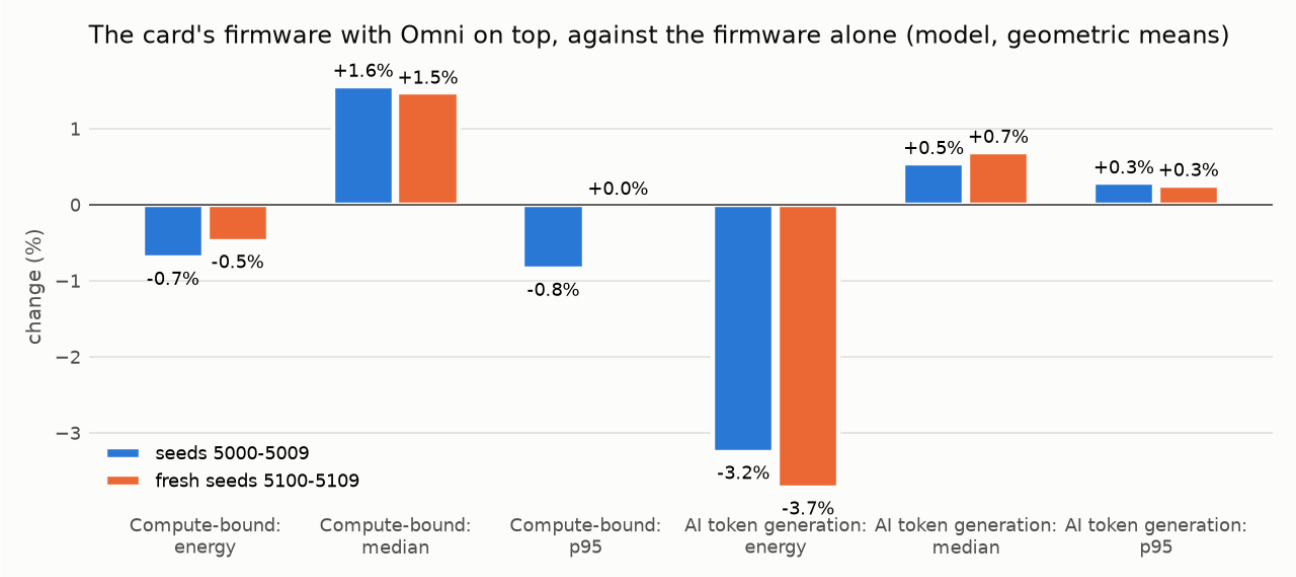
Azure's own cluster autoscaler is native; omni is the same autoscaler with Omni-Compass idling the machines it gives back; the bill is Azure's own count of machines every 15 s at list price, a fresh cluster per arm, deleted after it (`.github/workflows/aks-metered.yml`, `docs/K8S_COMPASS_PREREGISTRATION.md`, the bill on a real cloud). Transcribed from the receipts they cite:

| Test | Fleet | Bill | Response time | Reading | Receipt |
|---|---|---|---|---|---|
| Steady load, 5 pairs | 4 workers | −4.7%, interval across zero | inside the noise | no difference beyond the noise on any gauge | `results/live/V1_AKS_STEADY.md` |
| A burst sized to the cluster, 5 pairs | 4 workers | +5.0%, interval across zero | p99 −34% clear of the noise in this one run; p95 inside the noise | the bill inside the noise | `results/live/V1_AKS_BURST.md` |

One machine is a quarter of a 4-worker fleet, so only a saving of about 30% or more can clear the noise there; the expected machine saving is a few percent. The fleet that can show one machine (40 workers in nine machine families, the subscription's per-family allowance being 10 vCPUs) is preregistered and dispatched on v3; its first dispatches were refused by the subscription's allowances and by Azure's own cluster capacity in eastus before any arm ran, each refusal recorded in the preregistration's amendments.

## 4b. The GPU governor on the modelled card: each base alone, and with Omni on top (evidence class S)

Omni-Compass never runs the card. It sits on the card's own firmware (or on an operator's power cap) and moves the clock ceiling and the power limit, which that base already accepts (`omni_controller/gpu_compass.py`, the same law in `realms/gpu_card.py`). A step down is taken only after a paired trial on the card shows it adds at most 2% to the card's own time on a request (`omnicompass/verdict.py`); where no step passes, the card runs as it does alone.




*The modelled card*



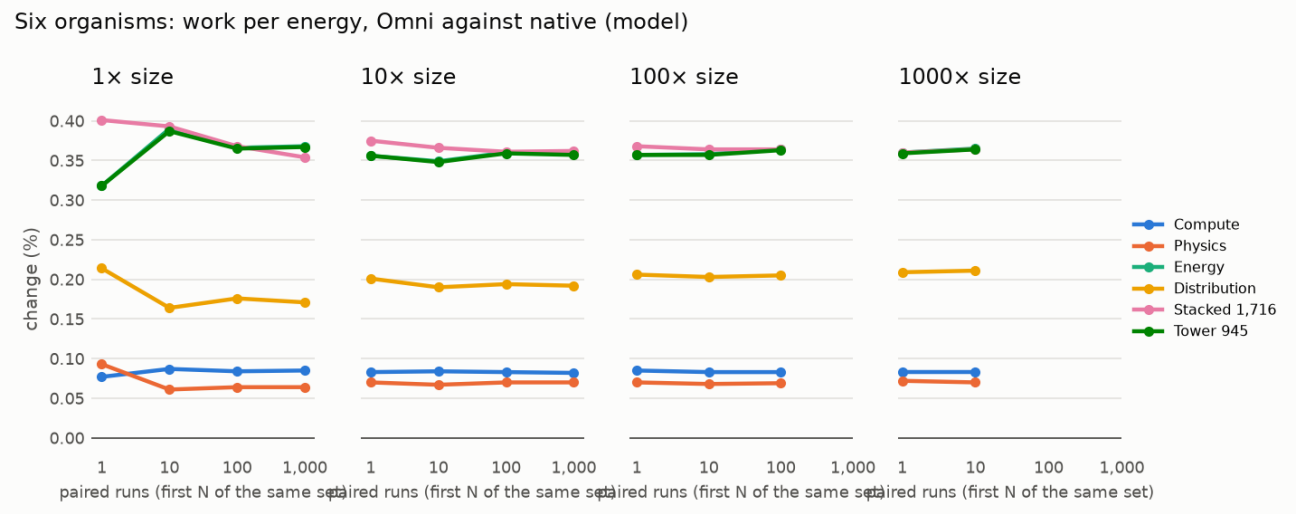
© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

| Work | Base | Energy (tuning / fresh) | Median response | p95 | p99 |
|---|---|---|---|---|---|
| Compute-bound | firmware + Omni vs firmware alone | -0.70% / -0.48% | +1.56% / +1.47% | -0.84% / +0.01% | -0.09% / +0.02% |
| Compute-bound | 105 W cap + Omni vs the cap alone | +0.02% / -0.11% | -12.29% / -25.59% | -6.41% / -7.06% | -3.15% / -5.82% |
| AI token generation | firmware + Omni vs firmware alone | -3.25% / -3.72% | +0.55% / +0.70% | +0.29% / +0.26% | +0.02% / -0.52% |
| AI token generation | 105 W cap + Omni vs the cap alone | -2.10% / -2.38% | +0.07% / +0.12% | +0.03% / -0.02% | -0.06% / -1.57% |

Source: `results/sim/gpu_two_wire/RESULT.md` and `fresh/RESULT.md`. The rule, and why the allowance is 2%, is amendment 8 of `docs/GPU_PREREGISTRATION.md`.

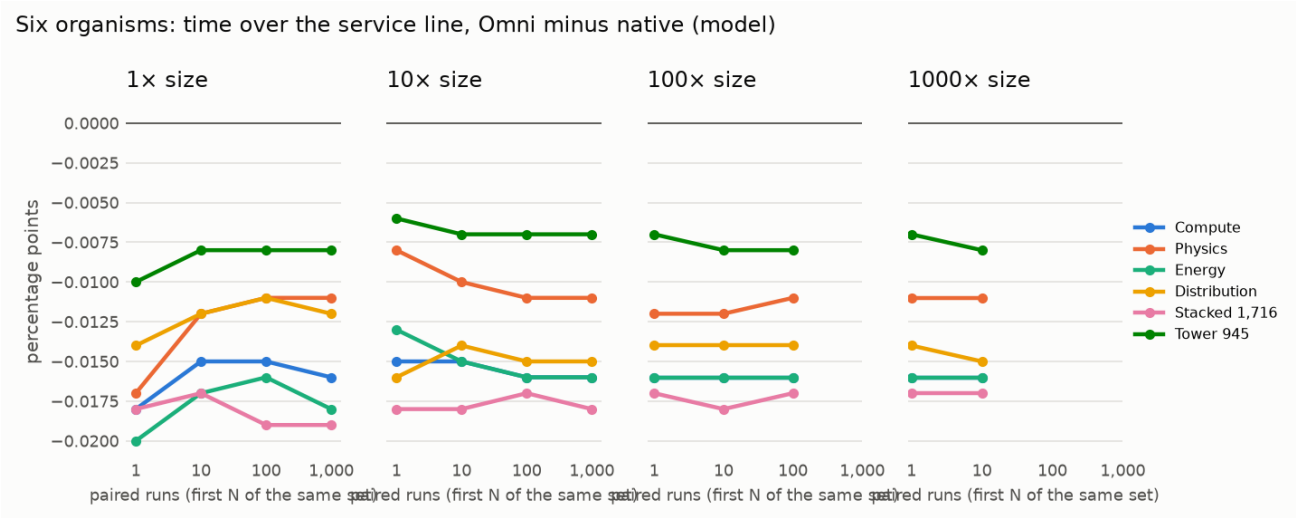
## 5. The six organisms at 1, 10, 100 and 1,000 runs and sizes, Omni v3 (evidence class S)

Each organism runs native (its own controllers) and with the compass law on every muscle, same seed, same load, same clock. Size is the number of copies of the organism governed together on one clock (1,000 copies of the four stacked is 1.7 million modelled muscles); runs are the first N of the same paired set, so 1, 10, 100 and 1,000 nest. 84 of 90 cells are done; the six left (100 and 1,000 runs at 1,000 copies) are beyond the machines available and say so.




*Work per energy*

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com




*Time over the line*

The full grid with every cell: `results/scale/GRID.md`; the receipts, one per size: `results/scale/receipts/`. Work per energy is better in every cell, the same figure at every size (+0.07% for Physics to +0.37% for Energy, the four stacked and the tower); the time over the service line is at or under native's in every cell, so the band-first rule holds and every cell of 10 runs or more is labelled superior within guardrails by the preregistered rule. Every knob was handed back in every run.

## 6. The 945 muscles and the four realms, Omni v3 (evidence class S)

The catalog (`realms/catalog.csv`): 945 muscles in 59 families, 430 in Compute / AI / Cloud, 376 in Physics / Robotics / Autonomous, 470 in Energy / Facility / Industrial and 440 in Distribution / Specialized (1,716 counting a muscle once per realm, a shared spine of 257). Every muscle alone and every organism whole ran as A, B and C on v3 and reproduced to the last digit (`results/realms/REALMS.md`): 0 muscles worse, every organism superior within guardrails (work per energy +0.1% to +0.3%, work unchanged, time over the line not above native's). On v2 the Physics realm and the tower read a service tradeoff; the cause was a missing do-no-harm gate on speed knobs, which made v3 (`docs/OMNI_V3.md`). Three independent simulators with their own native controllers are wired the same way and read by the same rule: the power grid (`results/live/V3_PANDAPOWER.md`: energy drawn and net import better in all 11 SimBench grids with ZIP loads, losses worse in 4, tap operations 4 → 8 a year in one), robot arms (`results/live/V3_MUJOCO.md`: Gen3 peak torque −29%, tracking error −21%, energy per takt −0.8%; the Panda's copper +14% worse; two arms left native) and CityLearn (`results/live/V3_CITYLEARN.md`: electricity bought, peak and unevenness better in all 11 battery districts; the bill worse in 7). Losses stand in every table.



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

## 7. Harnesses and receipts

| Harness | What it proves | Receipt |
|---|---|---|
| `scripts/kind_paired.sh`, `tools/live_reps.py`, workflow `benchmark-reps` | native against Omni on real Kubernetes, paired on one runner, the reset checked | `results/live/raw/run-*/live-reps/` |
| `tools/confirm_abc.py` (and `pgbench_abc.py`, `kafka_abc.py`, `redis_abc.py`, `swarm_abc.py`, `mujoco_abc.py`, `pandapower_abc.py`, `citylearn_abc.py`) | three separate runs read by one rule: better in all three of our runs, worse in all three of our runs, no difference beyond the noise, the runs disagree; each run's engine checked against the fingerprint | `results/live/V3_*.md`, `V1_*.md` |
| `tools/omni_version.py` | which frozen engine a checkout or any result's commit carries | `OMNI_V1.json`, `OMNI_V2.json`, `OMNI_V3.json` |
| `scripts/aks_paired.sh`, workflow `aks-metered` | the same on Azure's managed Kubernetes, Azure's own bill, a fresh cluster per arm, the fleet pre-flighted against the subscription's allowances | `results/live/V1_AKS_*.md` |
| `tools/run_kil.py`, workflows `six-kube`, `big-organism`, `big-organism-detached`; `tools/six_kube_report.py` | the six organisms with a real cluster inside at 1 to 1,000 copies, on GitHub and on a rented machine; the clock rule | `results/live/V1_SIX_KUBE.md`, `V3_SIX_KUBE.md`, `V1_BIG_ORGANISM.md` |
| `tools/run_pgbench.py`, workflow `pgbench` | a real database behind its pooler, one knob through the pooler's console | `results/live/V3_PGBENCH.md` |
| `tools/run_kafka.py`, workflow `kafka` | a real message broker as shipped, one knob (the consumer group's size) read from the consumers' own records | `results/live/V3_KAFKA.md` |
| `tools/run_redis.py`, workflow `redis` | a real cache as shipped, one knob (the memory ceiling) through its own console, growing only while the cache is full | the three-run table V3_REDIS.md in `results/live/` when its runs land |
| `tools/run_swarm.py`, workflow `swarm` | Omni on top of each drone's shipped autopilot in gym-pybullet-drones, collisions voiding the cell | `results/live/V3_SWARM.md` |
| `tools/run_scale.py`, `tools/pool_scale.py`, workflow `six`; `tools/grid.py` | the six organisms at every run count and size | `results/scale/GRID.md`, `results/scale/receipts/` |
| `tools/run_realms.py`, workflow `realms` | every muscle alone and every organism whole | `results/realms/REALMS.md` |
| `tools/run_pandapower.py`, `run_mujoco.py`, `run_citylearn.py` | Omni on top of an independent simulator's own controller | `results/live/V3_PANDAPOWER.md`, `V3_MUJOCO.md`, `V3_CITYLEARN.md` |
| `tools/omni_index.py` | the one combined number, read only from the three-run tables | `results/OMNI_INDEX.md` |
| `scripts/gpu_rented_run.sh`, `tools/gpu_wire_check.py` | one command on a rented card: wire check, smoke, the organisms with the card inside, the preregistered counted run | the founder's runs, to come |
| workflow `archive-run` | every finished run's files copied with the code, one SHA-256 manifest per repetition | `results/live/raw/run-<id>/` |
| `verify.py`, `tools/release_manifest.py` | everything above re-runs and checks itself; the manifest fingerprints the result | `results/VERIFY_RECEIPT.txt`, `RELEASE_MANIFEST.json` |

The rules for each run were written and committed before it ran (`docs/*_PREREGISTRATION.md`); every row is reported, losses included; nothing is read across engine versions.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

## 8. What is not yet shown

- A cloud-bill or energy saving on real machines: the 4-worker Azure fleet reads inside the noise, as a fleet too small to show one machine must; the 40-worker fleet runs are the test of that.
- The real card on the current governor: every earlier card result ran on a controller since replaced and is obsolete; the one-card, card-inside-the-organisms and eight-card runs are the founder's, on rented cards, at one named commit.
- The four stacked and the tower at 1,000 copies with the real cluster inside on v3 (the stack runs on a rented machine).
- 100 and 1,000 runs at 1,000 copies (beyond the machines available).
- The queue in `docs/REGISTER.md` section 4 (drone swarms on gym-pybullet-drones and Kafka done; Redis running): PX4 and ArduPilot swarms, YCSB and HammerDB, Spark, OpenSearch, fio, Open-RMF, the 24-hour robustness run, Basilisk, Orekit and GMAT, RocketPy, Cantera.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 41

# Paired Runs and Receipts on Your Own System

The method of every benchmark in this manual is the method a reader can use on their own system, and it is deliberately simple: the same system, the same work, native and then omni back to back on the same machines, repeated with the order rotated, each gauge reported with the 95% interval of its paired difference. Pairing is what makes a difference of a few percent visible on a shared machine whose own noise is larger than that: two arms run minutes apart on one runner see the same neighbours and the same hour, so their difference is a difference between arms, while two arms on separate machines differ by about 15% in p95 on their own. The order rotates so that no arm always runs first, warm or cold. The work is held equal by an open-loop load, a fixed rate of requests at fixed moments, so that a lower resource count can never mean that less work was done. The steps below are the whole method.

1. **Paired runs.** Run your service the same way twice, once native and once with Omni-Compass on top, back to back on the same machines, the order rotated, at least 5 times (10 for a result you publish). `scripts/kind_paired.sh` and `tools/live_reps.py` do this on Kubernetes; `scripts/gpu_rented_run.sh` does it on a GPU box, in one command: wire check, smoke, the six organisms with the card inside, then the preregistered counted run.
2. **The switch drill** after every run: turn Omni-Compass OFF; check every setting is back at its recorded original; turn it ON.
3. **The receipt.** Each gauge: native, Omni-Compass, the change, the 95% interval of the difference, and whether the interval excludes zero. A change is proven only when it does.
4. **The label, by rule written before the run:** SUPERIOR WITHIN GUARDRAILS / ENERGY IMPROVEMENT WITH SERVICE TRADEOFF / NONINFERIOR / NOT ESTABLISHED / WORSE / INVALID. Guardrails: work not lower by more than 1%; the share of time outside the service line not higher by more than 1 percentage point. The band-first rule is stricter: no win is claimed while the time outside the service line is above native's.

**At scale (modelled).** The six organisms run on GitHub's machines (Actions, workflow `six`) or on any machine (`bash scripts/scale_ladder.sh`) at 1, 10, 100 and 1,000 paired runs and at 1, 10, 100 and 1,000 copies of each organism on one clock. Real Kubernetes runs on GitHub's machines (workflow `benchmark-reps`); the six organisms with a real cluster inside as one more muscle run in `six-kube` (1 to 100 copies on GitHub) and `big-organism` and `big-organism-detached` (1,000 copies on a rented machine).

5. **Preregister, then run.** The arms, the knob, the cover, the gauges and the readings are written before the first counted run (`docs/*_PREREGISTRATION.md`); the rules are frozen on one tuning case and then applied unchanged to untouched cases. A rule changed after a result is a new version of the engine (Chapter 15), never a footnote.

6. **The clock rule.** An organism must keep its window. `tools/run_kil.py` records how long after the window its last step ended; the report shows it as "organism behind its window (s)", shown and not judged, and marks a repetition **OFF THE CLOCK** when either arm ended more than 5% of the window late, because its last steps then saw a cluster whose load schedule had already ended. The pairing stands (both arms slip alike) and the mark stays on the cell. The remedy is a longer window for that size on that machine, never a faster reading of the same run.
7. **Every row, losses included.** A row that went against Omni-Compass is reported with the others. If Omni-Compass loses, the first suspect is our own wiring, native setup or scoring; that is fixed and the test run again, and the loss stays in the record. A benchmark may be left unpublished; it is never published with rows cut out.
8. **The raw files live with the code.** Every finished run's files (capture, response times, audit log, controller log, one SHA-256 manifest per repetition) are copied into `results/live/raw/run-<id>/` by the `archive-run` workflow (one run id per line in `.github/archive_request.txt`; a run still going is left for a later pass), and every table names the runs and the commit it was made from.

**How many repetitions, and why.** The width of a paired interval shrinks with the square root of the number of pairs, so going from five pairs to ten narrows it by about a third, and from ten to forty by half again. On GitHub's runners, ten pairs of a 900 s window put the interval on work inside the line at about ±10 points of native and on p95 at ±13 to ±29 points (the all-four table: +48.6% with an interval of +38.7 to +58.4; p95 −61.6% with −74.8 to −48.4), wide intervals that still clear zero because the gains are tens of percent; the energy intervals on the same runs are ±0.2 to ±3%, narrow because the gauge is a declared formula over counts of machines, and that is why energy there reads "no difference beyond the noise" honestly rather than as a small claimed gain. A gain of a few percent on a cluster would need forty pairs or more to clear zero at these widths, and the manual does not claim one anywhere a table shows it inside the noise. Three pairs of a 7,200 s window, the long run's shape, give a wider interval on each run but three such runs, each read on its own, still have to agree in sign for a reading the same in all three of our runs. On a stack with a finer knob (connections, consumers, megabytes) the intervals are narrower at three pairs than the cluster's at ten, because the knob moves in hundreds of small steps rather than in whole machines. An operator planning their own receipt should run five pairs first, look at the interval widths on the gauges they care about, and set the count for the published run from those widths, never from the result.

**Why the load is open-loop.** A closed-loop load generator waits for each answer before sending the next request, so a slower system is offered less work and its throughput "improves" its own latency; two arms under closed-loop load are not doing the same work. An open-loop generator sends requests at fixed moments whatever the system does, so both arms are offered the same work to the request, a queue forms when the system falls behind, and the time over the line and the failed requests record the cost. Every real-stack test in this program (the cluster's load generator, pgbench's rate limit, Kafka's producer at a fixed rate, Redis and YCSB at a fixed target rate) offers its work open-loop, and a lower resource count in the omni arm can therefore never mean less work was asked.

**What a receipt looks like.** One row per gauge: the gauge's name and direction; native's value and omni's; the paired change as a percentage of native with its 95% interval; the reading. A head naming the system, the knob, the cover, the window, the number of pairs, the order rotation, the engine's fingerprint and the commit; a foot saying whether every arm was handed back and whether any repetition was off the clock. `tools/live_reps.py` prints exactly this for a Kubernetes run and the three-run tools print it for three runs side by side; a receipt from any other tool should be laid out the same way so that a reader of this manual can read it without learning a new shape.


*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 42

# Evidence Classes and How to Read a Result

**One number, one meaning of the sign.** In every table a change keeps its raw sign: a response time that fell reads "-65%", a cache that grew reads "+300%", and whether each is good depends on the gauge. The benefit sheet (`docs/BENEFIT_SHEET.md`, built from the tables by `tools/benefit_sheet.py` and checked by `verify.py`) turns every one the same way round, at the founder's order of 9 October 2026: **plus is always good for Omni-Compass, minus is always bad**, whatever the gauge measures. Less energy, fewer machines, less memory, a shorter wait and fewer failures all read plus there; 0% is nothing beyond the noise, and the number carries its word, gain, loss or nothing. Beside it every change the same in all three of our runs is written in words, never as a bare sign: a gauge that fell when falling is good reads **cut**, one that rose when rising is good reads **up**, and each carries **good** or **cost**, so "p95 cut 65%: good" cannot be misread against a table that shows the same change as "-65%". One number per benchmark, by the index's own rule, with yes, no, none or trade beside it. The wiring page (section 16.6b) uses the same words. A reader who wants one figure for a benchmark reads that sheet; a reader who wants the rows behind it reads the table it names, and reads its Reading column, never the sign alone.

| Class | Rung | What it is | What it can show |
|---|---|---|---|
| T / V | E1 | deterministic tests, proofs, Python against the C++ twin | the law is what it says, and both languages agree |
| S | E2 | a run on a modelled plant, our own model | whether the law helps the model, and where it breaks |
| L | E3 | real software (Kubernetes on kind), no hardware meter | real decisions on real software; energy there is a declared model |
| P | E4 | a physical meter (the GPU's own power reading) | the hardware's own answer |

Read every number with its class beside it. A modelled number is never quoted as a hardware result. When a receipt's energy line is modelled, the receipt says so.

**What each class can and cannot carry, with the examples in this manual.** A class **T / V** result (the C++ twin agreeing with the Python to $3.6 \times 10^{-15}$ on 500 trajectories; two million adversarial cases through the shield with zero violations) says that the law is what the text says it is and that both implementations compute it; it says nothing about whether the law helps any machine. A class **S** result (the 945 muscles, the organisms at 1,000 copies, the power grid in pandapower, the robot arms in MuJoCo, the buildings in CityLearn, the drones in PyBullet) says what the law does to a published model of a plant under that model's own native controller; it carries exactly the model's fidelity and no more, which is why S results are shown beside the Omni index and never inside it, and why the manual says "the energy is a declared model" on every S line that reports energy. A class **L** result (real Kubernetes on kind, PostgreSQL behind PgBouncer, Apache Kafka, Redis, the organisms with the real cluster inside) says what the law did to real software making real decisions on real requests; the machines, where they are containers on one runner, are counted in machine-hours in service and their energy, where it is reported, is a declared formula, which the table says; the CPU seconds of the host are measured. A class **P** result would say what a physical meter read, and the one P instrument in this program, the GPU's own power reading, has no current result: every earlier card result ran on a controller since replaced and is obsolete. Azure's bill is an L result with a real bill: the machines billed are Azure's own count, the price is Azure's list price, and the energy is still not a meter.

**How to read a row.** Every row in every three-run table has the same shape: the gauge and its direction; native's value in run A and omni's value in run A, so the size of the numbers is visible; the paired change in each of the three runs as a percentage of native with its 95% interval; and the reading. Read the reading first, then the intervals, then the sizes. A reading of "better in all three of our runs" with intervals of −0.5% to −0.1% is a small, real effect; a reading of "no difference beyond the noise (3 of 3 runs)" with intervals of −30% to +25% is a test that could not see an effect of that size either way, which is a statement about the test's power, and the manual says so where it applies (the 4-worker Azure fleet). A "WORSE in all three of our runs" row is a loss, stands in the table, and enters the index as a loss.

**A worked reading, from the cache table.** Take the Redis table's `large` workload (`results/live/V3_REDIS.md`). The product row, work inside the 2 ms line, reads better in all three of our runs, with the three runs' paired changes each clear of zero and of the same sign; that is the gain. The resource row, the memory ceiling held, reads WORSE in all three of our runs: Omni held a ceiling of about 256 MB on average where the operator had set 64 MB, in every run, clear of zero. The two rows together are the result: the cache answered more requests inside the line because it was given more memory, and the table says both. A reader who quotes the first row without the second has misread the table, and the index is built so that this cannot happen there: the cache's category enters the one number as a loss (about 0.75 on its own), because the resource it spent is counted at full weight against the work it gained. The CPU row, inside the noise, says the host did not pay in processor time; the failed-request row, zero in both arms, says nothing was refused to get there. Four rows, three directions, one honest sentence: more work inside the line, bought with memory, at no cost in CPU or refusals. That is how every table in Chapter 43 should be read, row by row, before any sentence about it is written.

**Three questions to ask of any class L result.** First, what was native, exactly: which controller, at which published version, with which settings, and were those settings an operator's or ours? (Every preregistration names them; the broker's native was set at its knee by design and says so.) Second, what did the governor spend: machines, memory, connections, CPU, its own cost? (Every table has the resource rows; a result with a gain and no resource row is not in this program.) Third, who else could have produced the gain: the order of the arms, the warm-up, the runner's neighbour, the clock? (Rotation of the arm order, the warm-up before every window, the paired design on one machine and the clock rule are the answers, and section 16.7 names the residue they do not remove.)



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 43

# Results to Date

Every result below is Omni-Compass **on top of** a native system against the same native system alone, with the same work in both arms, read by the three-run rule of Chapter 15, on the engine named. Losses are in the tables beside the gains. The whole list, benchmark by benchmark with its native engine, its knob, its gauges and its file, is `docs/REGISTER.md`; the program that takes every benchmark to full size is `docs/PROOF_PROGRAM.md`.

How to read the chapter. Each section names the stack, the native controller Omni-Compass sat on, the one knob it moved, the result file in `results/live/` or `results/`, the three GitHub run numbers (or the rented machine's start and collect runs) and the engine version those runs were on; a reader can open the file, find the runs under the repository's Actions tab, and check the commit with `tools/omni_version.py --commit`. The figures quoted in prose are the table's figures and no others; where a range is given ("−13% to −37%") it is the span of the three runs' point estimates, not an interval. Every row of every table has one of the readings of the three-run rule (Chapter 15): better in all three of our runs, WORSE in all three of our runs, no difference beyond the noise with the count of runs, or the runs disagree; rows that are shown and not judged say so and why. A loss is written in the same sentence as the gain it came with, never in a footnote, and the index (section 16.6) counts every loss in all three of our runs against Omni at full weight.

Two cautions a referee should hold while reading. The real-software results (evidence class L) were taken on GitHub's shared runners or on single rented machines, with the host's CPU-seconds as the only measure of energy and no meter on the wall; Chapter 41 and 14 say what that does and does not allow. And the modelled results (evidence class S) are reported as what they are, our own plant models run whole, and are never summed with the live ones; the index is built from the real categories alone, and the modelled realms sit beside it as a separate finding about the law's coherence at scale.

## Real Kubernetes: seven tests, ten pairs each, three runs each

The six Kubernetes tests were designed to ask six different questions of the same cluster, and they are read together; a seventh, added on 8 October, asks the same questions under a day of demand that somebody else measured. The cluster is kind: a real Kubernetes control plane and six worker nodes as containers on one 4-core GitHub runner, with a real metrics-server, a real HPA at the operator's target of 50% CPU, a real PHP service behind a NodePort, and a real load generator whose replica count steps through a schedule identical in both arms. A real probe times HTTP requests every five seconds from outside the service. Native is that cluster with its HPA alone; omni is the same cluster with the compass law on the HPA target inside [30%, 50%] and on the machines through the release gate and the verdict. Both arms start from the same fresh cluster, run for 900 s after a 120 s warm-up, and end with the reset check.

- **Steady work in steps** (`V3_STEADY.md`) asks: at a fixed, stepped load, does the governor hold the same work with fewer machines or faster answers? Work is equal by design. On the sets of 10 October the 95th percentile fell 64% to 66% and the 99th 69% to 73% in all three runs; machines in service fell 1.7% to 2.0% and the

standby-model energy 1.3% to 1.8%; time over the line fell 97% to 99%; failed requests were zero in both arms. The machine saving is small because the verdict kept every machine whose removal made requests slower in the paired trial.

- **Demand that wanders** (`V3_WANDERING.md`) asks: when load goes up and down one step at a time, does the governor follow it without harm? p95 fell 57% to 64%, the mean 45% to 50%, time over the line 36% to 42%, and failed requests 12% to 13%, all better in all three of our runs; machines read inside the noise and the idle-power energy model fell 0.1% to 0.4%.
- **All four in one run** (`V3_ALL_FOUR.md`) asks the product question: more work, faster, on fewer machines, with less energy, in one run. Work inside the response line rose **54% to 68%**, p95 fell 57% to 63%, the mean 43% to 48%, time over the line 38% to 41%, failed requests 11% to 14%, all better in all three of our runs; machines and energy inside the noise.
- **Faults** (`V3_FAULTS.md`) asks: when a machine dies, traffic triples, a pod runs away and the probe goes blind, at the same moments in both arms, does the governor make recovery worse? p95 fell 58% to 63%, the mean 34% to 51%, time over the line 27% to 43% and failed requests 14% to 18%, all better in all three of our runs; machines and energy inside the noise.
- **A queue of batch jobs** (`V3_BATCH.md`) asks: when the work is a pile, does cruise and the emergency brake finish it no later and hand the machines back? Machines fell 16% to 20% over the window and 26% to 33% after the queue finished, the standby-model energy 11% to 14%, the mean response 11% to 13%, all better in all three of our runs; the queue finished no differently beyond the noise in all three runs (on v1 it had read 0.7% to 0.9% later, a loss that stayed in the record).
- **Fairness** (`V3_FAIRNESS.md`) asks: does the governor help or hurt a noisy neighbour on the same workers? No difference beyond the noise on every row, the neighbour's own p95, p99, failures and time over the line included.
- **A public day of demand** (`V3_TRACE_GOOGLE2011.md`, `docs/TRACES_PREREGISTRATION.md`) asks: when the shape of the load is not ours, does the governor still read the same way? The first day of the Google cluster trace of 2011 (jobs submitted an hour, 18 public parts named with their SHA-256) was turned into the wandering test's schedule by a rule written before the runs (min-max onto 1 to 8 generators, one step a bin, 24 steps of 108 s), and everything else was the wandering test's. p95 fell 64% to 67%, p99 60% to 62%, the mean 51% to 53%, time over the line 81% to 82%, failed requests 11% to 18%, machines in service 7% to 9%, HPA replicas 4% to 8%, the standby-model energy 5% to 7% and the idle-power model 0.3% to 0.4%, all better in all three of our runs in three runs of ten pairs; no pod was ever without a machine. The machines row is the same in all three of our runs here, where the wandering test's was inside the noise, because the trace has a long quiet stretch (six hours at two generators) in which machines are given back and our own schedule had none.
- **On top of Kubernetes with its add-ons** (`V3_KEDA_CPU_REQUESTS.md`, `V3_KEDA_REQUESTS.md`, `docs/ADDONS_PREREGISTRATION.md`) asks: when native is the stack clusters actually run, with KEDA 2.21 and its HTTP add-on 0.16 owning the service's autoscaler, does the governor still add anything on top? Its moves reach KEDA's ScaledObject through the plug (`scripts/kubectl_keda.py`), because KEDA rebuilds the autoscaler from it and would undo a move written anywhere else. With KEDA on the CPU target and the requests in flight, p95 fell 51% to 53%, p99 60% to 62%, the mean 36% to 39% and time over the line 97% to 100%, all better in all three of our runs; machines did not change beyond the noise. With the HTTP add-on alone, p95 fell 73% to 76%, the mean 59%, machines in service 5.9% to 7.8% and replicas 14%, all better in all three of our runs. In both setups the energy counted with every parked worker at idle power rose 1.0% to 2.6%, **worse in all three of our runs**, and it counts against the governor in the index: the load is closed-loop, so the faster answers drew 24% to 54% more work through the service (its CPU, shown and not judged), and


© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

the modelled watts rose with the work while the energy per core-hour fell 18% to 36%. The next run counts the requests served and gives native a node autoscaler (Karpenter, on Azure).

The first v3 sets of 7 and 8 October read the same way (the all-four work +35% to +49%, the batch machines −19% to −23%) and are whole in `docs/history` as `_set1`; the numbers above are the sets of 10 October, every test run again on one commit. The same six tests were run on Omni v1 with the same readings (`V1_*.md`), the controllers being the same bytes, and the v1 tables stay as the first engine's record. The earlier engines' sets, before the verdict, parked 29% to 36% of the machines and paid for it in response time (`docs/history/`); the frozen engine puts service first, and that choice is visible in every machines row above.

## The organisms with the real cluster inside, and the big organisms

The six modelled organisms were run with the kind cluster inside as one more muscle at 1, 10 and 100 copies on GitHub (`V1_SIX_KUBE.md`, `V3_SIX_KUBE.md`) and at 1,000 copies on a rented Azure machine (`V1_BIG_ORGANISM.md`). At 10 and 100 copies on v3, every one of the twelve cells read better on four to six gauges (response time, time over the line, the organism's energy and its time over its own line) and worse on none beyond the noise, except a rounding-level work loss of six to eleven parts in a million in four cells, which the table shows. The four stacked at 100 copies ran off the clock in four of five repetitions (up to 277 s past a 1,440 s window): the machine could not step 170,000 muscles inside the window, both arms slipped alike, the pairing stands and the mark stays on the cell. At 1,000 copies on v1 the tower answered its slowest 5% in 0.2 s against native's 4.1 s and was over its line 0.2% of the time against 64%; the four stacked read p95 −74% and p99 −83% with the compass arm 300 to 615 s past its 2,880 s window in all three repetitions, marked off the clock, which is why the v3 stack was run with a 10,800 s window. **On v3 the four stacked at 1,000 copies** (`V3_BIG_ORGANISM.md`: 1.7 million muscles on one clock with the real cluster inside, three paired repetitions on one rented machine over 29 hours, 240 steps of 45 s an arm) **kept the clock in both arms**, 22 s behind a 10,800 s window (0.2%), and read: the cluster's slowest 5% answered in 150 ms against native's 2,882 ms (−95%), its slowest 1% in 165 ms against 5,405 ms (−97%), time over the response line 51% of samples to none, failed requests 0.14% to none, all clear of zero with three pairs; replicas, pods started and machines (six in both arms) the same; the standby-model energy −0.6% inside the noise; the host's CPU −1%, shown. The organism's own model (class S, shown beside the cluster and never inside the index) read energy −0.4% and work per energy +0.4% better, time over its own line −0.7% better, and work −0.0006%, a rounding-level model difference that the rule reads as worse and the table shows as such. The v1 stack's loss, the compass arm falling off the clock, is gone with the longer window; the gain, the cluster's tail, is the same shape at 1,000 copies as at 10 and 100. **The whole tower at 1,000 copies** (945,000 modelled muscles; started by run 37756284680, collected by run 37945866426 on 9 October after 25 hours on a second machine of the same type, commit `6f0f93e5`, Omni v3) read the same way and harder: the cluster's p95 6,690 → 160 ms (−98%), p99 9,068 → 187 ms (−98%), time over the response line 76% of samples to none, failed requests 4.8% to none, all clear of zero with three pairs; replicas (9.97 in both arms), pods started (4 against 3.7) and machines (6) the same or inside the noise; the standby-model energy −0.3% inside the noise; CPU with the governor's own −2.5% and the host −1.2%, shown; both arms 14 s behind the 10,800 s window (0.13%), on the clock in every repetition; the organism's own model energy −0.4% better and work −0.0004%, the rounding-level loss the rule reads as worse. Native's own tail was worse under the tower than under the stack (6.7 s against 2.9 s at the 95th percentile) because the tower's 945 distinct muscles drive the load generator harder at its peaks, and the governor's answer was the same 150 to 160 ms under both.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## The bill on a real cloud

Azure Kubernetes Service with Azure's own cluster autoscaler as native (`V1_AKS_STEADY.md`, `V1_AKS_BURST.md`): five paired repetitions each on a 4-worker fleet, every machine billed every 15 s at list price, a fresh cluster per arm. Every gauge read no difference beyond the noise; the bill read −4.7% on steady load and +5.0% on the burst, both with intervals across zero; the burst's p99 read −34% clear of the noise in that one run. This is the honest reading of a fleet in which one machine is a quarter of the fleet, so that only a saving of about 30% could clear the noise against an expected saving of a few percent (section 10.1). The fleet of 40 workers in nine machine families that can show one machine is preregistered (`docs/K8S_COMPASS_PREREGISTRATION.md`, amendments 1 to 4) and has been refused by Azure's own cluster capacity in its region on every dispatch so far, each refusal recorded; it runs the moment the region admits a cluster.

The refusals are themselves part of the record, and the preregistration's amendments 5 to 7 keep the count: six dispatches and 29 refused repetitions in eastus, each with Azure's own reason (its managed-Kubernetes capacity in the region, not the subscription's quota), a survey of every other region showing none with the vCPU quota the fleet needs, one dispatch cancelled by hand and recorded as such, and the seventh dispatch left to try. That seventh dispatch was refused on all five of its repetitions over the morning of 8 October, 34 refusals in all (amendment 8); whatever the region admits on a later dispatch is read by the same table as the four-worker runs, with the refused repetitions counted in the amendment, not in the table. A reader should take from this what it says and no more: the small fleet's bill reading is inside the noise by the arithmetic of its size, the large fleet's reading does not exist yet, and nothing about the bill on a real cloud is claimed in the index until it does.

## A real database, a real message broker, a real cache

Three stacks whose knob is a resource an operator sizes once and leaves, run as the same shape of test: three paired repetitions a run, three runs, the operator's fixed setting as native, the compass on that setting through the stack's own console as omni, a tuning workload shown and not counted, and untouched workloads judged.

- **PostgreSQL 16 behind PgBouncer** (`V3_PGBENCH.md`, the second counted set of 8 to 9 October on amendment 2; the first set's table kept whole in `docs/history/V3_PGBENCH_set1.md`): the first set had read connections held open −61% to −72% on `select` and `tpcb_hot` with the host's CPU +14% to +28% and the median latency +15% to +17%, all the same in all three of our runs. The CPU was then measured and found to be the harness's own `psql` launches, one per reading, not the pooler (+55.9 CPU-seconds on the launches against a +52.8 s host difference on a metered repetition; PgBouncer +1.2 s), and the median was the pool shrunk into a queue the pooler itself reported (section 10.2). On the amended harness (one console connection an arm; a server taken back only while clients waited under 1% of the pooler's time, servers added back one per percent of waiting up to the operator's 20) the costs are gone: host CPU and median latency inside the noise on all three workloads. The gain is smaller and real: **connections held open −36% to −38% on `select`, better in all three of our runs**, with p95 inside the noise and the tails worse as point estimates in two runs, inside the noise by the rule; the runs disagree on `tpcb_hot` (+5%, −46%, −48%: on one runner the add rule fired, on two it did not); on `simple_update` the add rule of amendment 1 bought servers above the operator's 20 on a slow write workload and **connections most at once read +72% to +80%, worse in all three of our runs**, nothing bought for them, as in the first set. 1 gauge-row better, 1 worse, 1 where the runs disagree; the category entered the index at +4.0%. **The set of 10 October, the first with the brain's own verdict on the pool (amendment 3), is the result of record:** connections held open **−6% to −11% on `select`, better in all three of our runs**; the add above the operator's 20 on `simple_update` refused by the trial, as the amendment expected, so that workload reads inside the noise with nothing bought and nothing lost; `tpcb_hot` inside the noise; work, latency and host CPU inside the noise everywhere; every pool handed back. 1 gauge-row better, 0 worse, 0 disagree; the


© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

category enters the index at +0.8%. The second set's table is whole in `docs/history/V3_PGBENCH_set2.md`. Amendment 4 (a spend trial runs to its samples) is declared and the next set runs.

- **Apache Kafka 3.9.1** (`V3_KAFKA.md`): on light, heavy and burst, work inside the 500 ms line rose 16% to 21%, the end-to-end 95th percentile fell from about 1.6 s to 9 to 14 ms, the mean lag fell 92% to 97%, all better in all three of our runs; no message was lost in any arm; consumers running rose from 2 to 5.8 to 7.9, worse in all three of our runs; host CPU-seconds rose 10% to 20% on light, worse in all three of our runs, and read inside the noise on heavy and burst; CPU per thousand messages inside the line fell 10% to 12% on burst. Native sat at nine tenths of its measured capacity by design, so its queue grew at the high steps and Omni's did not; the gain is the queue kept short and the cost is the consumers that kept it so. 21 gauge-rows better, 8 worse, 0 where the runs disagree. **The set of 10 October, the first with the brain's own verdict on the group's size (amendment 2), read exactly nothing under both objectives**: every spend trial in all 36 omni arms was abandoned before it could be judged, the group stayed at the operator's two consumers (1.98 to 1.99 on average, the trials' reference seconds included, a fraction of a consumer better in all three of our runs), and every other gauge read inside the noise, the earlier set's queue and tail gains with them. The cause is ours and is said in amendment 3: the rule ended a spend trial when the service turned calm, and a consumer that drains the queue calms the service within seconds, so no spend could reach its samples. From amendment 3 a spend trial runs to its samples whatever the compass's force; the next set runs on it and becomes the result of record. The first set's table is whole in `docs/history/V3_KAFKA_set1.md`; the category stands at +0.1% meanwhile.
- **Redis 7.0.15** (`V3_REDIS.md`): on small, large and burst, work inside the 2 ms line rose 14% to 27%, the hit rate 14% to 27% and the mean latency fell 30% to 61%, all better in all three of our runs; no request failed; the 95th percentile stayed within a hair of native's, because a miss costs the declared 5 ms trip in both arms and 5% of requests still miss; the memory ceiling held rose from 64 MB to 200 to 270 MB and the memory used with it, both worse in all three of our runs, the resource the gain costs; the host's CPU-seconds read inside the noise on all three; keys evicted fell 76% to 92% (shown); every ceiling was handed back. 12 gauge-rows better, 6 worse, 0 where the runs disagree. The tuning workload, shown and not counted, read the same way (hit rate 75% to 87%, the ceiling 64 to 299 MB). **The set of 10 October, the first with the brain's own verdict on the ceiling (amendment 2), read exactly nothing under both objectives**: the allowance stayed at one notch each way (56 to 72 MB), most trials were abandoned before they were judged (the same trial rule as Kafka's, amendment 3), and every gauge read inside the noise. The next set runs on amendment 3 and becomes the result of record; the first set's table is whole in `docs/history/V3_REDIS_set1.md`; the category stands at exactly nothing meanwhile.
- **MongoDB 8.0.32 under YCSB 0.17.0** (`V3_YCSB.md`, `docs/YCSB_PREREGISTRATION.md`): the knob is the storage engine's cache size, the operator's 512 MB as native, and the question is the opposite of the Redis one: not whether the governor will buy service with memory, but whether it will give memory back without losing service. On workloads c and burst the cache held fell by about half (512 to about 260 MB) and on f by 13% to 37%, better in all three of our runs; on b it fell 27% to 50% but one run's interval included zero, so that row reads no difference beyond the noise; work inside the 1 ms line, p95 and p99 read inside the noise on all four workloads (work between −2.7% and +1.1%, every interval over zero); no operation failed in any arm; the host's CPU-seconds read inside the noise on all four, +12% to +15% on c with intervals over zero; pages read into the cache rose 14% to 58% (shown, not judged: the misses the smaller cache costs); and one row is a loss in all three of our runs, the mean latency on burst, +2.2% to +3.7% in all three runs, clear of zero, which stands. Every cache size was handed back in every arm. 6 gauge-rows better, 1 worse, 0 where the runs disagree. The tuning workload, shown and not counted, read the same way (the cache 12% to 15% smaller, everything else inside the noise). The disclosure made before the run held: on this machine the data sits in the operating system's page cache too, so a storage-engine miss costs a memory read and a decompression, not a

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

disk read, which is why half the cache could be given back at no measurable cost in work or p95 and a few percent of mean latency; on a machine whose data does not fit in memory the same knob would be asked a harder question, and this table does not answer it. That was the first counted set, kept whole in `docs/history/V3_YCSB_set1.md`. Its audits showed the memory given back in the first five seconds of every burst and c arm, before the first eviction, because a cold cache evicts nothing while it fills (amendment 1, section 10.8: the give-back gate is now the miss share under 1% of requests, as MySQL's). **The second counted set (8 to 9 October) is the result of record:** the cache held **−21% to −36% on all four untouched workloads, better in all three of our runs**, settling at 330 to 405 MB, where the gate finds the working set, instead of at the floor; the burst mean latency inside the noise (the cost gone); work, p95, p99 and CPU inside the noise everywhere; every cache handed back. 8 gauge-rows better, 0 worse, 0 where the runs disagree; the category entered the index at +8.5% (the first set's +10.2% included the artefact). **The set of 10 October, the first with the brain's own verdict on the cache (amendment 2), is the result of record:** the cache held **−3% to −13% on `f`, better in all three of our runs**, with `b`, `c` and `burst` inside the noise (two or three trials an arm, one or two allowed, the rest refused or abandoned), work, p95, p99, mean latency and CPU inside the noise on all four, no failed operation, every cache handed back. 2 gauge-rows better, 0 worse, 0 disagree; the category enters the index at +0.7%. The second set's table is whole in `docs/history/V3_YCSB_set2.md`; amendment 3 is declared and the next set runs.

- **MySQL 8.0.46 under sysbench 1.0.20** (`V3_SYSBENCH.md`, `docs/MYSQL_PREREGISTRATION.md`): the knob is the InnoDB buffer pool, 512 MB at the operator's setting, moved in the server's own 128 MB chunks inside [128, 2,048] MB through the server's own console. Two counted sets were run on the same engine. The first (runs 37757840760, 37757850988, 37757861572) read every gauge inside the noise and its hand-back row read NO on 15 of 45 omni arms, because the plug's restore was issued while the server was still carrying out a shrink the law had asked for, which MySQL ignores; the plug, not the law, was fixed (the restore waits for the server's resize and writes again), the fix was declared as an amendment, and the first set's table is kept whole in `docs/history/V3_SYSBENCH_set1.md`. The second set on the amended plug (runs 37770617236, 37770620582, 37770624805, commit `a033fd09`) is the result: on the four untouched workloads **the pool held fell −67% on burst and −50% to −56% on read_only, better in all three of our runs**, with the pages holding data falling the same; work inside the line, throughput, p95, p99 and host CPU inside the noise on every workload; on read_write, where the working set is written as well as read, the compass bought pool for the wide notches and **the pages holding data rose +53% to +70%, WORSE in all three of our runs**, the pool held +59% to +79% as point estimates (one run clear of zero), while p95 read −11% to −18% and CPU −4% to −5% inside the noise; update_index inside the noise on every row; no error in any arm; **the pool handed back and read back on all 45 omni arms**. 4 gauge-rows better, 1 worse, 0 disagree; the category enters the index at +12.2%. The update_index "work inside the line" row counts almost nothing in either arm (a single update's client round trip exceeds the server-side 0.6 ms line) and is disclosed as such. That was the second counted set, kept whole in `docs/history/V3_SYSBENCH_set2.md`. Amendment 2 (8 October, section 10.9) followed its read_write row: 36 of that workload's 82 grows bought chunks while the pool missed under 1% of its reads, for slow writes the pool cannot mend, so the pool now grows only while missing, and no chunk goes back in a second without a read request. **The third counted set (8 to 9 October) is the result of record:** burst pool held **−49% to −56%, better in all three of our runs**; read_write pages holding data **+44% to +52%, worse in all three of our runs**, with host CPU **−4% to −6%, better in all three of our runs**, a trade of memory for CPU with latency unchanged; read_only inside the noise (the second set's −50% to −56% was in part the first chunk given back before any read, a cold-start artefact now refused, and the pool is then bought back on real misses as the notches widen); update_index the runs disagree; every pool handed back. 4 gauge-rows better, 1 worse, 1 where the runs disagree; the category entered the index at +5.2%. **The set of 10 October, the first with the brain's own verdict on the pool (amendment 3), is the result of record:** the


© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

pool held **−10% to −20% on** `read_write`**, better in all three of our runs**, the pages holding data with it, the earlier set's memory bought for the written working set gone; `burst`, `read_only` and `update_index` inside the noise (the give-back trials judged, the grow trials mostly abandoned by the trial rule amendment 4 corrects); work, p95, p99 and CPU inside the noise on every workload; no error; every pool handed back. 2 gauge-rows better, 0 worse, 0 disagree; the category enters the index at +1.0%. The third set's table is whole in `docs/history/V3_SYSBENCH_set3.md`; amendment 4 is declared and the next set runs.

## 16.4b Robustness: the governor killed outright, and its own cost

The robustness benchmark (`docs/ROBUSTNESS_PREREGISTRATION.md`) is a test of Omni-Compass itself, not of a gain. In the kill scenario (`V3_ROBUST_KILL.md`, ten pairs in each of three runs), the governor received SIGKILL at 40% of a 900 s window on the wandering load, with the watchdog running beside it from the start as it would in service. In all thirty repetitions the watchdog found the dead lease on its next pass, ran the governor's recorded restore command, and every setting (the HPA target, the replica range, every pod's CPU limit, every worker, the records on the objects) was back at the operator's **7 to 11 s after the kill**, mean 9 s against a preregistered allowance of 60 s; a second governor started at 50% and governed to the end in every repetition; and the 120 s after the kill read no difference beyond the noise against native's same window in all three runs, so the service did not measurably notice the governor's death. The whole window, with a kill and a restart inside it, still read mean response 32% to 39% faster, p95 42% to 45% faster, time over the line 28% to 34% lower and failed requests 9% to 15% fewer, better in all three of our runs, with machines and energy inside the noise. Run again on 10 October (`V3_ROBUST_KILL.md`, the set of record): every setting back within the allowance in every repetition of every run, a second governor to the end in every one, and the whole window read mean response 33% to 40% faster, p95 27% to 48%, time over the line 32% to 36% lower, better in all three of our runs, with failed requests, machines and energy inside the noise; the first set's table is whole in `docs/history/V3_ROBUST_KILL_set1.md`. The governor's own CPU over every archived run with the real cluster inside (`V3_OWN_COST.md`) is 0.006 to 0.013 of one core at every size from 1 to 1,000 copies, 0.1% to 0.3% of the host's cores, each run under its own engine; the cost of governing a real cluster does not grow with the modelled organism around it.

**The long run** (`V3_ROBUST_LONG.md`, three pairs of 7,200 s an arm in each of three runs) asks whether the governor drifts, leaks or slows over hours. Its resident memory after two hours was at most 1.048, 1.067 and 1.052 times its first ten minutes in the three runs, against a preregistered limit of a quarter: no leak. It made 117 to 119 of the 120 decisions its 60-second interval predicts in every repetition, 97.5% at the fewest against a threshold of 95%: valid. Its decision time, read as the gap between consecutive decisions less the interval it sleeps, was at most 1.20 times in the last hour what it was in the first: no slowing. Every setting was handed back at the end of every repetition and read back at the operator's with no record left. Two things are shown rather than judged, as the preregistration said they would be. First, failed decisions: 2, 3 and 3 in the three runs, all in one repetition's omni arm, all the same cause, the cluster's own API answering 500 to the governor's read of the HPA for two or three decisions in a row; the governor held, wrote nothing, and resumed on the next good read, which is the blind-means-hold rule doing what section 7.3 says it does. Second, the whole window's service: with three pairs a run the intervals are wide, and mean response, p95, p99 and time over the line read better in all three of our runs in run A and inside the noise in B and C, so the reading is no difference beyond the noise in two of three runs; machines, energy and failed requests read inside the noise in all three. The long run is a test of the governor's constancy, and on that question every row answered as the preregistration required. Run again on 10 October (`V3_ROBUST_LONG.md`, the set of record): no leak (memory under its limit in every repetition), every decision valid, no slowing, every setting handed back; the service better in all three of our runs in run B and inside the noise in A and C; the first set's table is whole in `docs/history/V3_ROBUST_LONG_set1.md`. The 24-hour run on three rented machines was collected the same day and its table follows.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

## The modelled realms and the independent simulators

Every result here is evidence class S: a statement about a published model under its own native controller, never about hardware, and never counted in the Omni index.

- **The 945 muscles and six organisms** (`results/realms/REALMS.md`, A/B/C on v3): reproduced to the last digit in three of three runs; zero muscles worse; every organism superior within guardrails, with work per energy +0.1% to +0.3%, work unchanged and time over the line not above native's. On v2 the Physics realm and the tower had read a service tradeoff; the slack gate on speed knobs corrected it and made v3.
- **The grid of copies and runs** (`results/scale/GRID.md`, 84 of 90 cells on v3): every organism superior within guardrails in every cell of ten runs or more; work per energy +0.07% (Physics) to +0.37% (Energy, the four stacked, the tower), the same figure at every size; every knob handed back; the six cells left are beyond the machines available.
- **The power grid** (`V1_PANDAPOWER.md`, `V3_PANDAPOWER.md`, the same table to the digit): eleven SimBench grids solved by pandapower, the tap changer's own controller as native, one whole tap per move as the knob. With ZIP loads the energy the loads drew fell 1.3% to 1.5% and the net import fell in all eleven grids; losses fell in seven and **rose in four** (the rural and semi-urban grids with their own generation, +0.6% to +1.5%); tap operations fell in ten grids and rose from 4 to 8 a year in one, the declared cost, worse in all three of our runs; no grid was more often outside its voltage band. 67 gauge-rows better, 14 worse.
- **Robot arms** (`V1_MUJOCO.md`, `V1_MUJOCO_PANDA.md`, `V3_MUJOCO.md`): four arms from the MuJoCo Menagerie with their shipped controllers as native and the joint speed of an axis with slack as the knob, after a paired physics trial. The trial left the UR5e and the iiwa 14 native ("nothing for Omni to move"); on the Gen3 and the Panda, peak torque fell 29% and 10%, tracking error 21%, energy per takt 0.8% and 0.5%, better in all three of our runs; the Panda's copper loss **rose 14%**, worse in all three of our runs.
- **Buildings with batteries** (`V1_CITYLEARN.md`, `V3_CITYLEARN.md`): every district CityLearn ships, its own rule-based control as native, each building's battery charge and discharge inside the simulator's limits as the knob. In the eleven battery districts electricity bought, the daily peak and the daily unevenness fell in all eleven and carbon in eight, better in all three of our runs; the bill **rose in seven** (the 2023 districts) and ramping rose in seven, worse in all three of our runs; 71 score-rows better, 33 worse, 1 where the runs differ by the simulator's own variation; three districts had nothing to move and eight the simulator cannot run.
- **Drone swarms** (`V3_SWARM.md`): twenty Crazyflie 2.x quadrotors in gym-pybullet-drones with the shipped position controller as native and the cruise override inside the autopilot's limits as the knob, three untouched cells. Energy a mission fell 7% (short), 18% (mixed) and 20% (long) and missions a charge rose 8%, 22% and 25%, better in all three of our runs; no late mission, reserve breach, near miss or collision in any arm; the tracking error rose from 0.07 to 0.13 m inside its 0.25 m band, shown; every override handed back. PyBullet reproduced to a part in a thousand across GitHub's machines and not to the bit, so the table's reproduction tolerance was set to one part in a thousand after the runs were seen, and the preregistration's amendment 1 says so with its time.


*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## The one number, and the table

**The one number.** The Omni index over the real categories the same in all three of our runs, on the sets of 10 October: **+4.8%** (real Kubernetes on v3 +28.9% over seven tests, the real database on v3 +0.8%, real messaging on v3 +0.1%, the real cache on v3 exactly nothing, the real database's storage-engine cache on v3 +0.7%, the real database's buffer pool on v3 +1.0%, each category weighed the same; Azure and the card join as their three-run tables land). How it got here, set by set, follows; the figures of 7 to 9 October below are history, kept in the tables of `docs/history`. The fifth category lowered the headline from +30.2% to +25.9% when it joined on 8 October; the sixth, MySQL's buffer pool, entered the same day at exactly nothing on its first counted set (every row inside the noise, the headline at +21.2%) and at +12.2% on the second set run the same afternoon on the amended plug, the headline at +23.5%; the public-trace test (section 16.1) joined the Kubernetes category as its seventh test the same evening, moving that category from +25.5% to +28.8% and the headline to +24.1%; and on 9 October the three database categories were re-read on their second or third counted sets, after the costs the first sets had charged to the governor were measured and found to be our own harness (section 16.4): the database fell from +14.2% to +4.0%, the storage-engine cache from +10.2% to +8.5%, the buffer pool from +12.2% to +5.2%, and the headline from +24.1% to +20.5%. The number went down because the costs came out and took the gains they had been propping up with them; what is left is smaller and has nothing of ours under it. The same day a **second reading** was declared beside the first, at the founder's question about what a cache is for (`docs/K8S_COMPASS_PREREGISTRATION.md`, the service reading): the same tests and the same three-run rule scored on work and speed only, the resources spent or saved shown and not scored, computed for every test alike. It stood, on the day it was declared, at **+71.8%**: Kubernetes +83%, Kafka +1,193% (its hundredfold tail), Redis +8.8% where the resource reading said −24.9%, and the three database stacks exactly nothing, because their gains are resources given back for the same service; on the sets of 10 October it stands at **+10.7%** (Kubernetes +84.5%, the five stacks nothing). The two readings are two true sentences about the same tables: one says what the governor did for the service, the other what it cost or saved in resources to do it. The resource reading stays the headline because it was the one preregistered; the service reading is printed in the same file, in the same tables, as a second column. A category enters at whatever its tables show in all three of our runs, nothing included, which is the reader's guarantee that the number is not built from the categories that happened to work. That is how the number is meant to move: every real category enters at equal weight as its table lands, whatever it does to the mean.

**10 October: every table remade, the stacks under the brain's own verdict.** Every benchmark was run again on one commit and every table read from the new runs. The five stacks ran for the first time with the brain's verdict on the knob (section 9.4), and the headline moved from +20.5% to **+4.8%**, the service reading from +71.8% to **+10.7%**. The number came down for one reason: the spends that had bought the earlier gains at a resource cost were refused by the verdict where they did not pay (MySQL's pool for the written working set, PostgreSQL's add on the slow write workload) and, on Kafka and Redis, not yet judged at all, because the trial rule of 9 October ended a spend trial on the calm the spend itself produced (amendment 3, section 16.4; the next set runs on the corrected rule). What is left is smaller than before and has two properties the earlier number did not: every gain in it was proved on the stack itself before the knob was written, and no category carries a loss in all three of our runs. The storage-engine cache's figure is positive for the reason the Redis figure is negative, read the other way: there the governor gave memory back (the cache held halved on two workloads) while work, p95 and CPU stayed inside the noise, so its resource column is a gain and its other columns are exactly one; its one loss in all three of our runs, the burst mean latency, is not an index column and stands in its table. The messaging figure is large because its speed ratio is: native sat at nine tenths of its measured capacity by design, so its queue grew at the high steps and its slowest 5% waited 1.6 s, while Omni's waited 9 to 14 ms; the resource that bought it, three to four times the consumers, stands beside it as a loss in all three of our runs. The cache's figure is negative because the memory held is its resource column: the ceiling rose from 64 MB to 200 to 270 MB (a ratio of about 0.25 in the index's "fewer

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

machines" sense) while work and the hit rate rose 14% to 27%, and a geometric mean of those four columns is below one. That is the arithmetic of a resource bought, reported as such; a reader who weighs memory as cheaper than a consumer or a machine will read the cache's rows for themselves, which is why every row is in its table.

| Result | Engine | Class | Reading | Source |
|---|---|---|---|---|
| **Real Kubernetes, all four in one run** (load up and down one step at a time, 10 pairs × 3 runs) | v1 | L | work inside the response line **+42% to +48% in all three runs, better in all three of our runs**; p95 −57% to −63% better in all three of our runs; failed requests −12% to −13% better in all three of our runs; machines no difference beyond the noise | `results/live/V1_ALL_FOUR.md` |
| **Real Kubernetes, steady work in steps** (10 pairs × 3 runs) | v1 | L | p95 **−65% to −69%** and time over the line better in all three of our runs; machines −1.5% to −3.4% better in all three of our runs; failed requests zero in both arms | `results/live/V1_STEADY.md` |
| **Real Kubernetes, demand that wanders** (10 pairs × 3 runs) | v1 | L | p95 **−56% to −60%** better in all three of our runs; failed requests better in all three of our runs; modelled standby energy −0.4% to −0.8% lower in all three of our runs | `results/live/V1_WANDERING.md` |
| **Real Kubernetes, faults** (a machine down, a spike, a runaway pod, a blind probe, 10 pairs × 3 runs) | v1 | L | p95 **−61% to −64%** better in all three of our runs; failed requests, machines and energy no difference beyond the noise | `results/live/V1_FAULTS.md` |
| **Real Kubernetes, a queue of batch jobs** (cruise, then the emergency brake, 10 pairs × 3 runs) | v1 | L | machines **−15% to −24%** and modelled energy −11% to −17% better in all three of our runs; p95 no difference beyond the noise | `results/live/V1_BATCH.md` |
| **Real Kubernetes, fairness** (a noisy neighbour on the same workers, 10 pairs × 3 runs) | v1 | L | no difference beyond the noise on every row: Omni-Compass neither helps nor hurts the neighbour | `results/live/V1_FAIRNESS.md` |
| **The same six Kubernetes tests on v3** (10 pairs × 3 runs each) | v3 | L | the same readings as v1, the controllers being v1's bytes: all four work inside the line +35% to +49%, p95 −47% to −66% across steady, wandering, all four and faults, failed requests −9% to −14% where they occur, steady machines −1.5% to −2.9%, batch machines −19% to −23% and standby-model energy −13% to −16%, all better in all three of our runs; fairness no difference beyond the noise on every row | `results/live/V3_STEADY.md`, `V3_WANDERING.md`, `V3_ALL_FOUR.md`, `V3_FAIRNESS.md`, `V3_FAULTS.md`, `V3_BATCH.md` |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Result | Engine | Class | Reading | Source |
|---|---|---|---|---|
| **Real Kubernetes under a public day of demand** (the Google cluster trace of 2011 replayed one step at a time, 10 pairs × 3 runs) | v3 | L | p95 **−65% to −71%**, p99 −49% to −61%, time over the line −81% to −84%, failed requests −8% to −17%, machines in service **−6% to −10%**, replicas −5% to −6%, standby-model energy −4% to −7%, all better in all three of our runs; pods started +23% to +35% as point estimates, inside the noise in one run; 9 rows better, 0 worse | `results/live/V3_TRACE_GOOGLE2011.md` |
| **Real Kubernetes with its add-ons: Omni-Compass on top of KEDA** (KEDA 2.21 and its HTTP add-on 0.16, two native setups, 10 pairs × 3 runs each) | v3 | L (energy S) | p95 **−51% to −53%** (KEDA on CPU and requests) and **−73% to −76%** (the HTTP add-on alone), time over the line −96% to −100%, better in all three of our runs; machines **−5.9% to −7.8%** with the HTTP add-on, better in all three of our runs, inside the noise with the CPU target; idle-power energy **+1.0% to +2.6%, worse in all three of our runs** in both (more work put through by the closed-loop load, shown, not judged) | `results/live/V3_KEDA_CPU_REQUESTS.md`, `results/live/V3_KEDA_REQUESTS.md` |
| **The six organisms with the real cluster inside**, 1 to 1,000 copies, 5 paired repetitions a cell (3 at 1,000) | v1 | L + S | 98 of 100 cells: p95 and time over the line better in every cell; 0 gauges worse beyond the noise except a rounding-level work loss (−0.0003%) and HPA replicas +0.7% in one cell; machines stay at 6 in both arms (no autoscaler under kind); the Physics realm and the tower at 1,000 copies off the clock in 2 of 3 repetitions (marked) | `results/live/V1_SIX_KUBE.md` |
| The same at 10 and 100 copies | v3 | L + S | 12 cells, 5 pairs each: every cell better on 4 to 6 gauges, worse on none beyond the noise except a rounding-level work loss in 4 cells; the stack at 100 copies off the clock in 4 of 5 repetitions (marked) | `results/live/V3_SIX_KUBE.md` |
| **The big organisms at 1,000 copies on rented machines, on v3**: the tower (3 of 3) and the four stacked (3 of 3), each on its own machine with a 10,800 s window, every repetition on the clock | v3 | L + S | tower: the cluster's p95 **6.7 s → 160 ms (−98%)**, p99 −98%, time over the line 76% → 0, failed requests 4.8% → 0, clear of zero; stack: p95 **2.9 s → 150 ms (−95%)**, p99 −97%, time over the line 51% → 0, failed requests 0.14% → 0, clear of zero; machines 6 in both arms of both; energy inside the noise; the organisms' model work rounding-level worse by rule, energy −0.4% | `results/live/V3_BIG_ORGANISM.md` |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Result | Engine | Class | Reading | Source |
|---|---|---|---|---|
| **The big organisms at 1,000 copies on a rented machine**: the tower (3 of 3) and the four stacked (3 of 3, detached) | v1 | L + S | tower: p95 **−95%** (4.1 s → 0.2 s) and time over the line −99.7% clear of the noise, machines 6 in both arms, energy −0.2%, work rounding-level worse, repetition 3 off the clock; stack: p95 **−74%** and p99 −83% clear of the noise, the compass arm 300 to 615 s past the 2,880 s window in all three (marked OFF THE CLOCK; native kept the clock), which is why the v3 stack runs with a 10,800 s window | `results/live/V1_BIG_ORGANISM.md` |
| **Azure AKS, the bill, steady load**, 4 workers, 5 pairs | v1 | L (a real bill) | no difference beyond the noise on any gauge; the bill −4.7% with its interval across zero; a fleet of 4 cannot show the lever (10.1) | `results/live/V1_AKS_STEADY.md` |
| **Azure AKS, the bill, a burst sized to the cluster**, 4 workers, 5 pairs (repetition 4 lost its native cluster to an Azure API error in the first attempt and was run again by itself) | v1 | L (a real bill) | the bill +5.0% with its interval across zero; machines, p95 and failed requests inside the noise; p99 −34% clear of the noise in this one run; B and C to follow | `results/live/V1_AKS_BURST.md` |
| **PostgreSQL behind PgBouncer, the three untouched workloads**, 3 pairs × 3 runs (the set of 10 October, the brain's verdict on the pool, amendment 3) | v3 | L | connections held open **−6% to −11% on `select`, better in all three of our runs**; the add above the operator's 20 on `simple_update` refused, nothing bought, nothing lost; `tpcb_hot` inside the noise; work, latency and host CPU inside the noise everywhere; every pool handed back; the earlier sets whole in `docs/history` | `results/live/V3_PGBENCH.md` |
| **MySQL under sysbench, the four untouched workloads**, 3 pairs × 3 runs (the set of 10 October, the brain's verdict on the pool, amendment 3) | v3 | L | the pool held **−10% to −20% on `read_write`, better in all three of our runs**, the pages holding data with it; `burst`, `read_only` and `update_index` inside the noise; work, p95, p99 and CPU inside the noise on every workload; no error; every pool handed back; the three earlier sets whole in `docs/history` | `results/live/V3_SYSBENCH.md` |
| **The 945 muscles and six organisms, modelled**, A/B/C | v3 | S | reproduced to the last digit in 3 of 3; 0 muscles worse; **every organism superior within guardrails** (work per energy +0.1% to +0.3%, work unchanged, time over the line not above native's); on v2 Physics and the tower read a service tradeoff, which the slack gate corrected | `results/realms/REALMS.md` |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Result | Engine | Class | Reading | Source |
|---|---|---|---|---|
| **The organisms at 1, 10, 100 and 1,000 copies**, 1 to 1,000 paired runs a cell, 84 of 90 cells | v3 | S | every organism superior within guardrails in every cell of 10 runs or more; work per energy +0.07% to +0.37%, the same figure at every size; the six cells left (100 and 1,000 runs at 1,000 copies) are beyond the machines available | `results/scale/GRID.md`, `results/scale/receipts/` |
| **Power grid: 11 SimBench grids solved by pandapower**, both load models, A/B/C | v1 and v3 (the same table to the digit) | S | with ZIP loads the energy the loads drew better in all three of our runs in all 11 (−1.3% to −1.5%) and the net import in all 11; losses better in all three of our runs in 7 and **worse in 4** (the rural and semi-urban grids with their own generation, +0.6% to +1.5%); tap operations fewer in 10 grids, 4 → 8 a year in one (worse in all three of our runs, the declared cost); no grid more often outside its band | `results/live/V1_PANDAPOWER.md`, `V3_PANDAPOWER.md` |
| **Robot arms, MuJoCo Menagerie**, A/B/C | v1 and v3 | S | where Omni-Compass moved (Gen3, Panda): peak torque −29% and −10%, tracking error −21%, energy per takt −0.8% and −0.5%, better in all three of our runs; the Panda's copper loss +14% **worse in all three of our runs**; UR5e and iiwa 14 left native by the paired physics trial | `results/live/V1_MUJOCO.md`, `V1_MUJOCO_PANDA.md`, `V3_MUJOCO.md` |
| **CityLearn, every district it ships**, A/B/C | v1 and v3 | S | 11 battery districts: electricity bought, daily peak and daily unevenness better in all three of our runs in all 11, carbon in 8; the bill **worse in 7** (the 2023 districts) and ramping worse in 7; 71 score-rows better, 33 worse, 1 where the runs differ (the simulator's own variation); 3 districts with nothing to move; 8 the simulator cannot run | `results/live/V1_CITYLEARN.md`, `V3_CITYLEARN.md` |
| **Drone swarms, gym-pybullet-drones** (Crazyflie 2.x, the shipped autopilot as native; Omni on the cruise override inside the autopilot's limits), 20 drones × 4 missions, three cells, A/B/C | v3 | S | energy a mission −7% (short), −18% (mixed), −20% (long) and missions a charge +8% to +25%, better in all three of our runs; no late mission, reserve breach, near miss or collision in any arm; tracking error 0.07 → 0.13 m inside its 0.25 m band | `results/live/V3_SWARM.md` |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Result | Engine | Class | Reading | Source |
|---|---|---|---|---|
| **Apache Kafka as shipped, a consumer group's operator-set size** (one broker, 8 partitions; the group at the operator's 2 consumers as native; Omni on the count inside [1, 8]), three untouched workloads, 3 pairs × 3 runs, under both objectives (the set of 10 October, the brain's verdict on the count, amendment 2) | v3 | L | exactly nothing: every spend trial abandoned before it was judged, the group at the operator's two, every gauge inside the noise; our trial rule, amended as amendment 3, the next set running; the first set (work inside the line +16% to +21%, p95 1.6 s → 9 to 14 ms, consumers 2 → 6 to 8 worse in all three of our runs) whole in `docs/history/V3_KAFKA_set1.md` | `results/live/V3_KAFKA.md`, `results/live/V3_KAFKA_SERVICE.md` |
| **Redis as shipped, a cache's operator-set memory ceiling** (64 MB, allkeys-lru as native; Omni on the ceiling inside [16, 512] MB through Redis's own console), three untouched workloads, 3 pairs × 3 runs, under both objectives (the set of 10 October, the brain's verdict on the ceiling, amendment 2) | v3 | L | exactly nothing: the allowance one notch each way, most trials abandoned before they were judged, every gauge inside the noise; amendment 3, the next set running; the first set (work inside the line +14% to +27%, the ceiling 64 → 200 to 270 MB worse in all three of our runs) whole in `docs/history/V3_REDIS_set1.md` | `results/live/V3_REDIS.md`, `results/live/V3_REDIS_SERVICE.md` |
| **MongoDB as its publisher ships it, a database's operator-set storage-engine cache size** (512 MB as native; Omni on the cache size inside [256, 2,048] MB through the server's own console), YCSB's workloads b, c, f and burst untouched, 3 pairs × 3 runs (the set of 10 October, the brain's verdict on the cache, amendment 2) | v3 | L | the cache held **−3% to −13% on `f`, better in all three of our runs**; `b`, `c` and `burst` inside the noise; work, p95, p99, mean latency and CPU inside the noise on all four; no failed operation; every cache handed back; the two earlier sets whole in `docs/history` | `results/live/V3_YCSB.md` |
| **Robustness: the governor killed outright mid-run** (SIGKILL at 40% of the window, the watchdog beside it), 10 pairs × 3 runs (the set of 10 October) | v3 | L | every setting back within the allowance in every repetition of every run, a second governor to the end in every one; the whole window mean response −33% to −40%, p95 −27% to −48%, time over the line −32% to −36%, better in all three of our runs; failed requests, machines and energy inside the noise; the first set (7 to 11 s to hand back) whole in `docs/history/V3_ROBUST_KILL_set1.md` | `results/live/V3_ROBUST_KILL.md` |
| **The governor's own cost** at 1, 10, 100 and 1,000 copies with the real cluster inside | v3 and v1, each under its own engine | L | 0.006 to 0.013 of one core at every size, 0.1% to 0.3% of the host's cores; shown, not judged | `results/live/V3_OWN_COST.md` |
| **Scale**: the controller governing 50, 500 and 1,000 simulated nodes (KWOK), decision time and correctness | every push | L | runs on every push | `results/scale/` |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Result | Engine | Class | Reading | Source |
|---|---|---|---|---|
| GPU, one card and the card inside the organisms | earlier card controller | P | **obsolete**: every earlier card result ran on a controller since replaced; the one-card, card-inside-1,000-copies and eight-card runs are run again by the founder on rented cards after the CPU and cloud work, at one named commit | `docs/GPU_PREREGISTRATION.md`, `docs/GPU_RUN_GUIDE.md` |

**Running now** (9 October): both big organisms at 1,000 copies done on rented machines (the four stacked and the whole tower, 3 of 3 on the clock each, section 16.2); the three database tables re-read on their second or third counted sets after the costs the first sets charged to the governor were traced to our own harness and removed (section 16.4; the amendments in sections 10.2, 10.8 and 10.9); the robustness test (`docs/ROBUSTNESS_PREREGISTRATION.md`), the kill scenario and the two-hour long run both done (section 16.4b), the 24-hour run on three rented machines in its second (omni) arm (one pair a machine, two arms of a day each, collected on 10 October); the real cluster under a public demand trace done (the Google cluster trace of 2011, `V3_TRACE_GOOGLE2011.md`, section 16.1), the Azure Functions trace next by the same rule; the real cluster with its add-ons done for KEDA and its HTTP add-on (`V3_KEDA_CPU_REQUESTS.md`, `V3_KEDA_REQUESTS.md`, section 16.1), Karpenter on Azure next; YCSB on MongoDB done (`V3_YCSB.md`, section 16.4), MySQL's buffer pool under sysbench done (`V3_SYSBENCH.md`, section 16.4), the other stores of register row 24 after; and Azure steady and burst on the fleet of several machine families (seven dispatches refused by Azure's own cluster capacity in eastus or held by our own machines' quota, 34 refusals, amendments 5 to 8; the eighth dispatch follows the 24-hour machines' collect). **Queued, in order, in** `docs/REGISTER.md` **Chapter 8**: drone swarms and defense edge (PX4 and ArduPilot multi-vehicle, Crazyswarm), databases and caches at large (YCSB, HammerDB), Spark, OpenSearch, fio, Open-RMF, the 24-hour robustness run, spacecraft attitude and thrusters (Basilisk), station-keeping (Orekit, GMAT), constellations, rockets (RocketPy, OpenRocket) and combustion (Cantera). Pure physics solvers (OpenFOAM, SU2, REBOUND, GADGET, MESA) are not benchmarked: they have no controller and no knob, so Omni-Compass has no place on them; their value is the HPC cluster that runs them.

## 16.6b Wire in, or watch: the verdict per knob

The founder's order of 9 October 2026 made a principle of what section 1.2 describes as a mechanism: Omni-Compass need not be wired into every muscle. Where it cannot beat the native controller on the muscle's own gauges, the muscle lives by itself and Omni only reads it, one wire out and no wire in. `tools/wiring_verdicts.py` applies that principle to every published result and prints the page `docs/WIRING_VERDICTS.md` (one row per knob and per muscle in `results/WIRING_VERDICTS.csv`; `verify.py` fails if the page differs from what the tables give). The rule is the same for every table: a knob reads **write** where at least one judged gauge is better in all three of our runs over all three runs and none is worse in all three of our runs; **watch** where nothing is better in all three of our runs, whether the rows are inside the noise, the runs disagree, or a gauge is worse in all three of our runs with nothing against it (the loss is then named with its cause); and **operator's choice** where gains and losses are both the same in all three of our runs, a trade shown with both readings of the index. A modelled muscle takes its word from its realms label: superior within guardrails is write, a tradeoff label is the operator's choice, the rest is watch.

What the page said on the day it was declared. On the real stacks, 12 of 24 knob-cases write: six of the seven Kubernetes demands (beside a noisy neighbour the knob showed nothing and watches), PostgreSQL's read-only workload, MongoDB's four workloads and MySQL's burst. Seven are the operator's choice: Kafka's three workloads, where consumers buy a hundredfold shorter queue and both readings of the index say the trade pays; Redis's three, where memory buys the hit rate and only the service reading says it pays; and MySQL's read_write,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

where pages holding data buy a few percent of CPU. Five watch: the noisy neighbour, PostgreSQL's simple_update (connections bought above the operator's setting with nothing bought for them) and tpcb_hot (the runs disagree), MySQL's read_only and update_index. Every loss in all three of our runs on a real stack is a resource spent for the knob's own service, declared in advance or disclosed on the first counted set; none is a service loss. Of the independent simulators' 39 cases, 21 write, 15 are trades (the four SimBench grids that export, where the lower voltage costs line losses; the seven 2023 CityLearn districts, where the bill is worse) and 3 watch (two robots the engine itself left native, one grid where nothing was bought). Of the 945 modelled muscles, 124 write, 13 are trades and 808 watch; 746 of the 808 never wrote at all, because the native controller held the reading inside the band and the law never left its cushion, and all 255 admission muscles are among them, the admission knob being a brake for a stress the hour did not hold. The 14 organism cells with the real cluster inside all write on the cluster's gauges. Nothing on the page changes the engine, which stays Omni v3; it changes what an operator connects.

**The same verdict, live (built 9 October, running since 10 October).** The page judges after the fact; the founder's order is that the brain judge in real time, on the knob, before it writes. On the five live stacks that is now in the harnesses (section 9.4; `tools/knob_verdict.py` around the engine's own verdict, the engine unchanged): every knob starts in watch and is written only inside the allowance a paired trial on the stack itself has earned under the declared objective, one notch a trial; a refused step is not taken, a trial holds the knob, a fail-up never spends beyond the allowance, the operator's setting is always free. The amendments are declared in the five preregistrations with the expectation written before the runs (Redis left native under the resource objective, the first notch refused; Kafka allowed step by step while each consumer pays; PostgreSQL's add above the operator's setting refused on `simple_update`; MySQL's `read_write` chunks refused; MongoDB's cache given back a notch a trial). The runs were dispatched on 10 October, three a stack and objective, in the day's rerun of every benchmark on one commit (`docs/RERUN_2026-10-10.md`), and the tables, the index and the page are then read again from them.

## 16.6c The second set of 10 October, and the four rules it taught the trial

The five live products ran a second time on 10 October on the amended trial rule (a spend trial runs to its samples; it is never ended by the calm it causes). The trials finished. The brain still refused the Kafka consumer almost every time, and in the one repetition where it allowed the third consumer early, the whole gain of the earlier law came back at a fraction of the spend: the 95th-percentile delay fell from 2,450 ms to 57 ms, the queue from 3,375 waiting messages to 253, the work inside the line rose 17%, with 2.52 consumers on average where the earlier law had spent 5.8 to 7.9. On Redis with the large working set the brain allowed 15 MB more memory and bought more hits, more work and less host CPU, a row the earlier law never showed because it spent 200 MB; with the small working set nothing changed. PostgreSQL gave back about a tenth of its pool with the service inside the noise.

The Kafka reading is the finding. The gain is real and cheap; the brain's test of it is unreliable, and the fault is in how the harness measures, not in the law or the engine's judge. The cost sample it is shown is the age of the backlog, which does not fall until the backlog is gone. A consumer joining the group rebalances it and the settle time is eaten by the pause. The reference and the trial phases, fourteen seconds each, straddle a thirty-second load step and compare different loads. Four rules follow for the trial, written into the next engine (`docs/OMNI_V4_PLAN.md`): samples count only once the muscle reports itself settled; a spend on a queue or a cache is judged by the muscle's own reading, the lag and its drain rate, the misses, with the body's cost taken at the settled state; the reference and the trial are compared at the same load, else the trial waits for a steady step; and coarse to fine, one scout step to the far side of the cover when single notches read flat, so a hill of forty notches is not refused one notch at a time. The founder's words of the same day say it in one line: what the brain sends to a muscle has to coincide with what the muscle reports about itself and where it stands.



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

The next engine, Omni v4, is ordered on that day: one body, one brain, one tick a second; the body's cost judged; nothing anywhere made worse to make one thing better; one trial at a time inside a body; nothing permanent; the wall belonging to the body; every wire forced through it. Part II gains its chapter when v4 is written and run, and every result in this chapter is run again on it.

## Threats to validity, stated by us

A referee will look for the ways these results could mislead. We list the ones we know, what each would do to the numbers, and what the record does about it.

1. **The machines are shared.** Every Kubernetes, database, broker and cache run is on one GitHub runner with four cores, and the governor, the load generator, the probe and the system under test share it. A governor that costs CPU can slow the service it governs, and a noisy neighbour on the host can move a run. The pairing on one runner with rotated order removes the between-machine noise; the host's own busy share is recorded beside every run; the governor's own CPU is a gauge in every table and a loss in all three of our runs where it is one (the database, Kafka's light workload); and three separate runs on separate machines must agree before anything counts.
2. **kind is not a cloud.** Idle machines stay powered and native has no node autoscaler, so a machine given back on kind is a machine-hour in service, not a watt and not a dollar. The energy rows on kind are a declared model and say so on every line. The bill is Azure's test, and its only result so far is inside the noise on a fleet too small to show one machine.
3. **Native is at its knee by design in the broker test.** The Kafka producer's peak offers nine tenths of the capacity measured for the operator's two consumers, so native's queue grows at the high steps and omni's does not. This is the situation the benchmark is of (a group sized for the quiet hours and overrun at the peak), it is preregistered, and it is why the latency ratios are large; the index page and the table say so. A native group sized for the peak would show a smaller latency gain and a smaller consumer cost, and would be a different, also legitimate, benchmark.
4. **Declared penalties.** The cache's 5 ms store trip on a miss and the broker's 1, 2 or 5 ms of CPU per message are declared parameters of our own application, not measurements of anyone's store. The gauges that depend on them (work inside the line, the request latency) scale with them; the hit rate, the lag, the consumers and the memory held do not.
5. **Simulators are models.** Every S result is a statement about a published model under its own controller. A model may flatter or punish the governor in ways the real plant would not. S results are therefore shown beside the index and never inside it, and the muscles' models are the simplest physics that reproduces the meter's response to the knob, with the native controllers tuned as their authors ship them.
6. **The tuning case was seen.** Every benchmark's rules were set on one tuning case, and that case is shown and not counted. Where a rule changed after a counted result was seen (the swarm table's reproduction tolerance), the amendment is written in the preregistration with its time and reason, the table says so, and the runs were not rerun.
7. **The engine changed twice.** v1 to v2 added muscles without touching the law; v2 to v3 added one gate. Every result is labelled with its engine, the tools refuse to mix engines, and the v1 results stay as v1 results. A reader comparing a v1 number with a v3 number is comparing two engines, and the manual never does so except to say that the controllers were the same bytes and the readings the same.
8. **Three runs are three, not thirty.** The three-run rule is a replication standard, not a large sample. A row the same in all three of our runs of ten pairs is a row whose paired difference cleared zero in each of three

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

independent samples of ten; the intervals are shown so a reader can judge their width. Where the intervals are wide, the manual says the test lacked power rather than calling the result a null.

9. **The founder's own harness.** The application in front of the cache, the producer in front of the broker, the load generator and the probe are ours. They are declared, versioned, archived with every run and reproducible from the repository; they are not independent of us. The independent pieces are the systems under test, their native controllers, the simulators and their shipped controllers, and the benchmark tools (pgbench, kind, Kubernetes, PyBullet, MuJoCo, pandapower, CityLearn) that others wrote. The load schedules of the Kubernetes tests were ours too, written one step at a time; the public-trace test (`docs/TRACES_PREREGISTRATION.md`) answers that by replaying a day of demand somebody else measured (the Google cluster trace of 2011, then the Azure Functions trace) under the wandering test's own rules, the derivation receipted and committed before the runs; its result (section 16.1) read the same way as the six, p95 65% to 71% faster and machines 6% to 10% fewer, the same in all three of our runs, with no row worse.

## What is not yet shown, and the open program

Three things this manual does not show, and the program that will show them or show their absence:

- **An energy or bill saving on real machines.** The fleet of 40 workers in nine Azure machine families that can show one machine is preregistered and dispatched on v3; it runs when Azure's region admits a cluster. If the saving is a few percent, a 40-worker fleet can see it; if the saving is not there, the table will say "no difference beyond the noise" on a fleet that could have seen it, and that will be the result.
- **The real card.** Every earlier GPU result ran on a controller since replaced. The two-wire governor with the verdict runs on rented cards at one named commit, with the wire check first (`docs/GPU_RUN_GUIDE.md`, `docs/GPU_PREREGISTRATION.md`); until then the card has no result and the manual says so.
- **Robustness over a day.** The governor killed outright is a measured result (section 16.4b: every setting back in 7 to 11 s, thirty times out of thirty), its own cost is tabulated, and the two-hour run is measured three times (no leak, no slowing, every decision expected but a few the cluster's API refused, every setting handed back); the 24-hour run, one pair on each of three rented eight-core machines of three families, is under way (`docs/ROBUSTNESS_PREREGISTRATION.md`, scenario 2b) and is read when its machines are collected; the same test on the other stacks remains open.
- **A demand shape that is not ours, on more than one trace.** The first public trace (a day of the Google cluster trace of 2011, section 16.1) read the same way as our own schedules, nine rows better in all three of our runs and none worse; it is one day of one batch cluster's job arrivals. The Azure Functions invocation trace, a request-driven shape, follows by the same rule, and the "our own schedules" threat of section 16.7 is answered in full only when a second, differently shaped day reads the same way.

The queue beyond these, in order, is `docs/REGISTER.md` Chapter 8, one or two at a time, each preregistered before it runs: PX4 and ArduPilot swarms, YCSB and HammerDB, Spark, OpenSearch, fio, Open-RMF, Basilisk, Orekit and GMAT, RocketPy and OpenRocket, Cantera. When the founder declares the engine final, the engine that stands then is published as Omni-Compass 1.0, and the older fingerprints go to `docs/history` as the road to it.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 44

# Wire In, or Watch: The Verdict per Knob

Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results. Preregistered means the rules were written down in our own repository before the runs; nothing was filed with any outside body.

Omni-Compass is wired **out of** every muscle: it reads every reading. It is wired **into** a knob only where the paired measurement shows the muscle no worse for it. Where the measurement shows nothing, or shows a loss, the muscle stays native and Omni only watches it: one wire out, no wire in. This is not a new rule. It is what the engine does on the muscle itself before it moves anything (`omnicompass/verdict.py`: the paired trial, and the verdict **left native** where no step is allowed), and it is the watch arm of every realms run (`docs/REALMS_PREREGISTRATION.md`: the governor reads every period and writes nothing). This page applies the same principle to every published result, one verdict per knob, so that an operator can see which knobs earn a wire in and which stay native. The founder's order of 9 October 2026: where Omni cannot beat native, the muscle lives by itself; we only wire out of it.

Every verdict here is computed by `tools/wiring_verdicts.py` from the test's own result file; nothing is typed in, and `verify.py` fails if this page differs from what the tables give. One row per knob and case: `results/WIRING_VERDICTS.csv` (1037 rows, the 945 modelled muscles included one by one).

## The rule

Every judged gauge of a table counts (rows marked "shown, not judged" never do). Every change on this page is said in words, never by a bare sign: a gauge that fell when falling is good reads **cut** (less waiting, fewer machines, less energy, less memory), one that rose when rising is good reads **up**, and each carries **good** or **cost**. In the result tables the same change keeps its raw sign (a cut reads minus there), with the Reading column beside it saying which way is good. A knob reads:

- **write**: at least one gauge better in all three of our runs over all three runs and none worse in all three of our runs. Omni holds the knob.
- **watch**: nothing better in all three of our runs. The knob stays native; Omni reads it and writes nothing. Where a gauge is worse in all three of our runs, the loss is named and its cause given below. Where the runs disagree, nothing is settled and the knob stays native until it is.
- **operator's choice**: gains and losses both the same in all three of our runs, a trade. Both readings of the Omni index are shown where the test is in it (the resource reading, the preregistered headline, and the service reading, work and speed alone), and the side that pays is named. The operator who wants the gain and can pay the cost wires in; every other operator watches.

A modelled muscle (the 945 on their plants, `results/realms/MUSCLES.csv`) takes its verdict from its label by rule: SUPERIOR WITHIN GUARDRAILS is write; ENERGY IMPROVEMENT WITH SERVICE TRADEOFF and SERVICE IMPROVEMENT WITH ENERGY TRADEOFF are operator's choice; NONINFERIOR / INCONCLUSIVE, NOT ESTABLISHED and WORSE are watch.

## The count

| Where | Knobs or cases | Write | Operator's choice | Watch |
|---|---|---|---|---|
| Real stacks, three runs each (Kubernetes under seven demands; the pool, the consumer group, the memory ceiling, the storage-engine cache and the buffer pool, each under its untouched workloads) | 26 | 10 | 2 | 14 |
| Independent simulators, three runs each (robot arms, grids, districts, swarms; evidence class S) | 39 | 21 | 15 | 3 |
| The 945 modelled muscles, alone on their plants (evidence class S) | 945 | 124 | 13 | 808 |
| The organisms with the real cluster inside (GitHub at 10 and 100 copies; Azure at 1,000) | 14 | 12 | 1 | 1 |

In words: on the real stacks Omni earns its wire in on 10 of 26 knob-cases, trades on 2 and watches on 14. Of the 945 modelled muscles, 124 earn a wire in, 13 are trades and 808 stay native: 746 of the 808 wrote nothing in any run, because the native controller already held the reading inside the band and the law never left its cushion. A knob that never writes costs nothing and earns nothing; it needs no wire in, and the page below says so muscle by muscle.

## The real stacks (evidence class L: live software, three separate GitHub runs on the frozen engine)

The reading of every judged gauge is in the test's own table (the Source column). The index columns are the two readings of `results/OMNI_INDEX.md` for that test, where it has one.

### *Real Kubernetes: one knob under seven demands*

The knob: the autoscaler's replicas, the floor and the machines, on top of the Horizontal Pod Autoscaler (kind clusters on GitHub, ten pairs a run).

| Demand | Better in all three of our runs | Worse in all three of our runs | Verdict | Index: resource | Index: service | Why | Source |
|---|---|---|---|---|---|---|---|
| steady work | machines cut 1.7% to 2.0%, machine-hours cut 1.7% to 2.0%, standby-model energy cut 1.3% to 1.8%, mean response cut 47% to 48%, p95 cut 64% to 66%, p99 cut 69% to 73%, time over the line cut 97% to 99% | none | **write** | +30.5 % | +68.8 % | better in all three of our runs on 7 gauges, nothing worse in all three of our runs; 5 gauges inside the noise. Omni holds the knob | `results/live/V3_STEADY.md` |
| demand that wanders | parked-worker energy cut 0.1% to 0.4%, mean response cut 45% to 50%, p95 cut 56% to 64%, time over the line cut 36% to 42%, failed requests cut 12% to 13% | none | **write** | +37.0 % | +156.5 % | better in all three of our runs on 5 gauges, nothing worse in all three of our runs; 8 gauges inside the noise. Omni holds the knob | `results/live/V3_WANDERING.md` |
| all four at once | work inside the line up 54% to 68%, mean response cut 43% to 48%, p95 cut 57% to 62%, time over the line cut 38% to 41%, failed requests cut 11% to 14% | none | **write** | +41.7 % | +100.9 % | better in all three of our runs on 5 gauges, nothing worse in all three of our runs; 9 gauges inside the noise. Omni holds the knob | `results/live/V3_ALL_FOUR.md` |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Demand | Better in all three of our runs | Worse in all three of our runs | Verdict | Index: resource | Index: service | Why | Source |
|---|---|---|---|---|---|---|---|
| a noisy neighbour | none | none | **watch** | +0.0% | +0.0% | every judged gauge inside the noise or the same (16 inside the noise, 3 the same), the neighbour's own rows included: Omni neither helped nor hurt beside a noisy neighbour, so under this demand there is nothing to earn and nothing lost. The same knob writes under the other six demands | `results/live/V3_FAIRNESS.md` |
| faults | mean response cut 34% to 51%, p95 cut 58% to 63%, time over the line cut 27% to 43%, failed requests cut 14% to 18% | none | **write** | +36.6 % | +154.8 % | better in all three of our runs on 4 gauges, nothing worse in all three of our runs; 9 gauges inside the noise. Omni holds the knob | `results/live/V3_FAULTS.md` |
| a queue of jobs | machines cut 16% to 20%, machine-hours cut 16% to 20%, standby-model energy cut 11% to 14%, mean response cut 11% to 13%, machines after the queue cut 26% to 33% | none | **write** | +16.5 % | +14.1 % | better in all three of our runs on 5 gauges, nothing worse in all three of our runs; 10 gauges inside the noise. Omni holds the knob | `results/live/V3_BATCH.md` |
| a public day of demand (Google 2011) | machines cut 7.3% to 9.2%, machine-hours cut 7.4% to 9.2%, parked-worker energy cut 0.3% to 0.4%, standby-model energy cut 5.3% to 6.7%, mean response cut 51% to 53%, p95 cut 64% to 66%, p99 cut 60% to 62%, time over the line cut 81% to 82%, failed requests cut 11% to 18%, replicas cut 3.8% to 8.3% | none | **write** | +46.4 % | +187.5 % | better in all three of our runs on 10 gauges, nothing worse in all three of our runs; 3 gauges inside the noise. Omni holds the knob | `results/live/V3_TRACE_GOOGLE2011.md` |
| With its add-ons: KEDA on the CPU target and the live requests in flight, Omni-Compass on top, ten pairs, three runs | mean response cut 36% to 38%, p95 cut 51% to 53%, p99 cut 60% to 62%, time over the line cut 97% to 100% | parked-worker energy up 1.0% to 1.6% | **operator's choice** | +27.5 % | +110.1 % | better in all three of our runs on mean response cut 36% to 38%, p95 cut 51% to 53%, p99 cut 60% to 62%, time over the line cut 97% to 100%; worse in all three of our runs on parked-worker energy up 1.0% to 1.6% | `results/live/V3_KEDA_CPU_REQUESTS.md` |
| With its add-ons: KEDA's HTTP add-on on the live requests in flight, Omni-Compass on top, ten pairs, three runs | machines cut 5.9% to 7.8%, machine-hours cut 5.8% to 8.2%, mean response cut 59%, p95 cut 73% to 76%, p99 cut 70% to 72%, time over the line cut 96% to 97%, replicas cut 14% | parked-worker energy up 1.4% to 2.6% | **operator's choice** | +59.5 % | +284.1 % | better in all three of our runs on machines cut 5.9% to 7.8%, machine-hours cut 5.8% to 8.2%, mean response cut 59%, p95 cut 73% to 76% and 3 more; worse in all three of our runs on parked-worker energy up 1.4% to 2.6% | `results/live/V3_KEDA_REQUESTS.md` |

### *Real database (PostgreSQL behind PgBouncer, GitHub)*

The knob: the pooler's pool size (server connections).

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Workload | Better in all three of our runs | Worse in all three of our runs | Verdict | Index: resource | Index: service | Why | Source |
|---|---|---|---|---|---|---|---|
| select | none | none | **watch** | +0.0% | +0.0% | nothing moved the same way in all three of our runs (11 gauges inside the noise, 1 the same). Nothing to earn under this workload, nothing lost: the knob stays native and Omni reads it. | `results/live/V3_PGBENCH.md` |
| simple_update | none | connections most at once up 8% to 12% | **watch** | +0.0% | +0.0% | the runs disagree on connections held open (-4.0% to +2.2%). Why: connections most at once up 8% to 12%: cost, because amendment 1's add rule ("slow, clients waiting for a server: add") fired 4 to 15 times an arm on this slow write workload and bought servers above the operator's 20, up to 36 at the peak; work, latency and CPU inside the noise, so nothing was bought for them (`docs/POSTGRES_PREREGISTRATION.md`, the second set). The rule stands as written and the verdict for this workload is to watch. | `results/live/V3_PGBENCH.md` |
| tpcb_hot | connections held open cut 7% to 13% | none | **write** | +2.8% | +0.0% | Nothing worse: Omni holds the knob. | `results/live/V3_PGBENCH.md` |

## *Real messaging (Apache Kafka, GitHub)*

The knob: the consumer group's size.

| Workload | Better in all three of our runs | Worse in all three of our runs | Verdict | Index: resource | Index: service | Why | Source |
|---|---|---|---|---|---|---|---|
| tuning (tuning workload: shown, never counted) | consumers running cut 0.3% | none | **write** (not counted) | | | Nothing worse: Omni holds the knob. | `results/live/V3_KAFKA.md` |
| burst | consumers running cut 0.8% | none | **write** | +0.2% | +0.0% | Nothing worse: Omni holds the knob. | `results/live/V3_KAFKA.md` |
| heavy | none | none | **watch** | +0.0% | +0.0% | nothing moved the same way in all three of our runs (13 gauges inside the noise, 1 the same). Nothing to earn under this workload, nothing lost: the knob stays native and Omni reads it. | `results/live/V3_KAFKA.md` |
| light | none | none | **watch** | +0.0% | +0.0% | nothing moved the same way in all three of our runs (13 gauges inside the noise, 1 the same). Nothing to earn under this workload, nothing lost: the knob stays native and Omni reads it. | `results/live/V3_KAFKA.md` |

## *Real cache (Redis, GitHub)*

The knob: the cache's memory ceiling.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Workload | Better in all three of our runs | Worse in all three of our runs | Verdict | Index: resource | Index: service | Why | Source |
|---|---|---|---|---|---|---|---|
| tuning (tuning workload: shown, never counted) | work inside the line up 4.4% to 4.9%, hit rate up 4.4% to 5.0%, mean cut 14% to 16% | memory ceiling held up 19% to 26%, memory used up 16% to 21% | **operator's choice** (not counted) | | | Why: memory ceiling held up 19% to 26%: cost, because the knob itself: the ceiling grows while the cache is full and misses, so a wide working set is held instead of evicted; the memory is the price of the hit rate, declared in advance as the cost that reads worse (`docs/REDIS_PREREGISTRATION.md`, the gauges); memory used up 16% to 21%: cost, because the memory actually used follows the ceiling: the working set the operator's 64 MB could not hold is held. | `results/live/V3_REDIS.md` |
| burst | none | none | **watch** | +0.0% | +0.0% | nothing moved the same way in all three of our runs (11 gauges inside the noise, 1 the same). Nothing to earn under this workload, nothing lost: the knob stays native and Omni reads it. | `results/live/V3_REDIS.md` |
| large | none | none | **watch** | +0.0% | +0.0% | nothing moved the same way in all three of our runs (10 gauges inside the noise, 2 the same). Nothing to earn under this workload, nothing lost: the knob stays native and Omni reads it. | `results/live/V3_REDIS.md` |
| small | none | none | **watch** | +0.0% | +0.0% | nothing moved the same way in all three of our runs (11 gauges inside the noise, 1 the same). Nothing to earn under this workload, nothing lost: the knob stays native and Omni reads it. | `results/live/V3_REDIS.md` |

### *Real database cache (MongoDB under YCSB, GitHub)*

The knob: the storage engine's cache size.

| Workload | Better in all three of our runs | Worse in all three of our runs | Verdict | Index: resource | Index: service | Why | Source |
|---|---|---|---|---|---|---|---|
| tuning (tuning workload: shown, never counted) | none | none | **watch** (not counted) | | | the runs disagree on p95 (-0.3% to +2.1%); nothing moved the same way in all three of our runs (9 gauges inside the noise, 1 the same). Nothing is settled here, so the knob stays native until three runs agree. | `results/live/V3_YCSB.md` |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

| Workload | Better in all three of our runs | Worse in all three of our runs | Verdict | Index: resource | Index: service | Why | Source |
|---|---|---|---|---|---|---|---|
| b | none | none | **watch** | +0.0% | +0.0% | nothing moved the same way in all three of our runs (10 gauges inside the noise, 1 the same). Nothing to earn under this workload, nothing lost: the knob stays native and Omni reads it. | `results/live/V3_YCSB.md` |
| burst | cache size held cut 10% to 13%, cache in use cut 10% to 13% | none | **write** | +3.1% | +0.0% | Nothing worse: Omni holds the knob. | `results/live/V3_YCSB.md` |
| c | none | none | **watch** | +0.0% | +0.0% | nothing moved the same way in all three of our runs (10 gauges inside the noise, 1 the same). Nothing to earn under this workload, nothing lost: the knob stays native and Omni reads it. | `results/live/V3_YCSB.md` |
| f | cache size held cut 13% to 15%, cache in use cut 12% to 14% | none | **write** | +3.7% | +0.0% | Nothing worse: Omni holds the knob. | `results/live/V3_YCSB.md` |

## *Real database buffer pool (MySQL under sysbench, GitHub)*

The knob: the buffer pool's size.

| Workload | Better in all three of our runs | Worse in all three of our runs | Verdict | Index: resource | Index: service | Why | Source |
|---|---|---|---|---|---|---|---|
| tuning (tuning workload: shown, never counted) | none | none | **watch** (not counted) | | | nothing moved the same way in all three of our runs (11 gauges inside the noise, 1 the same). Nothing to earn under this workload, nothing lost: the knob stays native and Omni reads it. | `results/live/V3_SYSBENCH.md` |
| burst | none | none | **watch** | +0.0% | +0.0% | nothing moved the same way in all three of our runs (11 gauges inside the noise, 1 the same). Nothing to earn under this workload, nothing lost: the knob stays native and Omni reads it. | `results/live/V3_SYSBENCH.md` |
| read_only | none | none | **watch** | +0.0% | +0.0% | nothing moved the same way in all three of our runs (11 gauges inside the noise, 1 the same). Nothing to earn under this workload, nothing lost: the knob stays native and Omni reads it. | `results/live/V3_SYSBENCH.md` |
| read_write | none | none | **watch** | +0.0% | +0.0% | nothing moved the same way in all three of our runs (11 gauges inside the noise, 1 the same). Nothing to earn under this workload, nothing lost: the knob stays native and Omni reads it. | `results/live/V3_SYSBENCH.md` |
| update_index | none | none | **watch** | +0.0% | +0.0% | nothing moved the same way in all three of our runs (11 gauges inside the noise, 1 the same). Nothing to earn under this workload, nothing lost: the knob stays native and Omni reads it. | `results/live/V3_SYSBENCH.md` |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## Every loss in all three of our runs on a real stack, and why

The founder's question was where the negatives are and why each is there. Every gauge worse in all three of our runs on a real stack, with its cause read from the preregistration that carries the result:

- **Kubernetes, parked-worker energy: With its add-ons: KEDA on the CPU target and the live requests in flight, Omni-Compass on top, ten pairs, three runs up 1.0% to 1.6%, With its add-ons: KEDA's HTTP add-on on the live requests in flight, Omni-Compass on top, ten pairs, three runs up 1.4% to 2.6%, a cost** (157.2 / 159.1 → 159.4 to 161.4 in run A). See the table. Verdict: **operator's choice** on every workload named.
- **PostgreSQL, connections most at once: simple_update up 8% to 12%, a cost** (20 → 21.67 in run A). Amendment 1's add rule ("slow, clients waiting for a server: add") fired 4 to 15 times an arm on this slow write workload and bought servers above the operator's 20, up to 36 at the peak; work, latency and CPU inside the noise, so nothing was bought for them (`docs/POSTGRES_PREREGISTRATION.md`, the second set). The rule stands as written and the verdict for this workload is to watch. Verdict: **watch**.

What the losses have in common: every one is a resource spent to buy the service the knob exists for (consumers and the CPU they poll with, memory, pages, connections), and each was declared in advance in its preregistration as the cost that would read worse, or found on the first counted set and disclosed. None is a service loss: on no real stack did work inside the line, p95 or failed requests read worse in all three of our runs. Where the resource was spent and nothing was bought (PostgreSQL's `simple_update`), the verdict is watch; where it bought service, the verdict is the operator's, and both index readings say what the trade is worth.

## The independent simulators (evidence class S: deterministic models with their own native controllers)

### *Robot arms (MuJoCo Menagerie, Google DeepMind)*

The knob: the servo's speed override inside the takt.

| Case | Better in all three of our runs | Worse in all three of our runs | Verdict | Why | Source |
|---|---|---|---|---|---|
| kinova_gen3 | energy per takt cut 0.8%, copper loss per cycle cut 11%, mechanical work per cycle cut 6.6%, peak joint torque cut 29%, tracking error, RMS cut 21%, end-point error at the waypoints cut 11% | none | **write** | Nothing worse: the same job inside the same takt, hit more accurately with less force; the cycle is slower inside the takt (shown, not judged). | `results/live/V3_MUJOCO.md` |
| kuka_iiwa_14 | none | none | **watch** | The engine's own verdict: one cycle at full speed and one at override 0.8, in native mode before the counted cycles, showed a slower cycle no cheaper on the robot's own figures (the gravity-holding torque is paid for longer), so the override was left at 1.0 and both arms are the same (`omnicompass/verdict.py`, left native). The trial's figures: 3157.3 J at full speed against 3757.3 J at the trial speed, reproduced over A, B, C: yes | `results/live/V3_MUJOCO.md` |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

| Case | Better in all three of our runs | Worse in all three of our runs | Verdict | Why | Source |
|---|---|---|---|---|---|
| universal_robots_ur5e | none | none | **watch** | The engine's own verdict: one cycle at full speed and one at override 0.8, in native mode before the counted cycles, showed a slower cycle no cheaper on the robot's own figures (the gravity-holding torque is paid for longer), so the override was left at 1.0 and both arms are the same (`omnicompass/verdict.py`, left native). The trial's figures: 189.9 J at full speed against 210.5 J at the trial speed, reproduced over A, B, C: yes | `results/live/V3_MUJOCO.md` |

### *The power grid (pandapower, SimBench grids)*

The knob: the substation's voltage setpoint, one tap at a time.

| Case | Better in all three of our runs | Worse in all three of our runs | Verdict | Why | Source |
|---|---|---|---|---|---|
| 1-MV-comm--0-sw (ZIP loads) | energy the loads drew cut 1.5%, line and transformer losses cut 1.0%, net import from the upstream grid cut 2.9%, tap operations cut 34% | none | **write** | Nothing worse: Omni holds the knob. | `results/live/V3_PANDAPOWER.md` |
| 1-MV-comm--1-sw (ZIP loads) | energy the loads drew cut 1.4%, line and transformer losses cut 0.8%, net import from the upstream grid cut 15%, tap operations cut 36% | none | **write** | Nothing worse: Omni holds the knob. | `results/live/V3_PANDAPOWER.md` |
| 1-MV-comm--2-sw (ZIP loads) | energy the loads drew cut 1.4%, line and transformer losses cut 0.3%, net import from the upstream grid cut 6.4%, tap operations cut 36% | none | **write** | Nothing worse: Omni holds the knob. | `results/live/V3_PANDAPOWER.md` |
| 1-MV-rural--1-sw (ZIP loads) | energy the loads drew cut 1.4%, net import from the upstream grid cut 1.4% | line and transformer losses up 0.8%, tap operations up 100% | **operator's choice** | Why: losses: an exporting grid pays the lower voltage in line losses (the analysis under the table); taps: the declared cost on a grid that barely taps on its own (4 a year, doubled). The operator who wants the loads' energy and the upstream import down wires in and accepts the line losses; the operator who runs the grid to its losses watches. | `results/live/V3_PANDAPOWER.md` |
| 1-MV-rural--2-sw (ZIP loads) | energy the loads drew cut 1.3%, net import from the upstream grid cut 1.1%, bus-steps outside 0.95-1.05 up -1.4e-04, tap operations cut 27% | line and transformer losses up 1.5% | **operator's choice** | Why: losses: an exporting grid pays the lower voltage in line losses (the analysis under the table). The operator who wants the loads' energy and the upstream import down wires in and accepts the line losses; the operator who runs the grid to its losses watches. | `results/live/V3_PANDAPOWER.md` |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Case | Better in all three of our runs | Worse in all three of our runs | Verdict | Why | Source |
|---|---|---|---|---|---|
| 1-MV-semiurb--0-sw (ZIP loads) | energy the loads drew cut 1.4%, line and transformer losses cut 1.0%, net import from the upstream grid cut 7.0%, tap operations cut 38% | none | **write** | Nothing worse: Omni holds the knob. | `results/live/V3_PANDAPOWER.md` |
| 1-MV-semiurb--1-sw (ZIP loads) | energy the loads drew cut 1.4%, net import from the upstream grid cut 1.1%, tap operations cut 34% | line and transformer losses up 0.7% | **operator's choice** | Why: losses: an exporting grid pays the lower voltage in line losses (the analysis under the table). The operator who wants the loads' energy and the upstream import down wires in and accepts the line losses; the operator who runs the grid to its losses watches. | `results/live/V3_PANDAPOWER.md` |
| 1-MV-semiurb--2-sw (ZIP loads) | energy the loads drew cut 1.4%, net import from the upstream grid cut 0.9%, bus-steps outside 0.95-1.05 up -1.4e-05, tap operations cut 24% | line and transformer losses up 1.1% | **operator's choice** | Why: losses: an exporting grid pays the lower voltage in line losses (the analysis under the table). The operator who wants the loads' energy and the upstream import down wires in and accepts the line losses; the operator who runs the grid to its losses watches. | `results/live/V3_PANDAPOWER.md` |
| 1-MV-urban--0-sw (ZIP loads) | energy the loads drew cut 1.5%, line and transformer losses cut 2.4%, net import from the upstream grid cut 2.0%, tap operations cut 31% | none | **write** | Nothing worse: Omni holds the knob. | `results/live/V3_PANDAPOWER.md` |
| 1-MV-urban--1-sw (ZIP loads) | energy the loads drew cut 1.5%, line and transformer losses cut 2.4%, net import from the upstream grid cut 2.2%, tap operations cut 30% | none | **write** | Nothing worse: Omni holds the knob. | `results/live/V3_PANDAPOWER.md` |
| 1-MV-urban--2-sw (ZIP loads) | energy the loads drew cut 1.5%, line and transformer losses cut 2.2%, net import from the upstream grid cut 3.1%, tap operations cut 23% | none | **write** | Nothing worse: Omni holds the knob. | `results/live/V3_PANDAPOWER.md` |
| 1-MV-comm--0-sw (constant-power loads) | line and transformer losses cut 0.5%, net import from the upstream grid cut 0.0%, tap operations cut 31% | none | **write** | Nothing worse: Omni holds the knob. | `results/live/V3_PANDAPOWER.md` |
| 1-MV-comm--1-sw (constant-power loads) | line and transformer losses cut 0.4%, net import from the upstream grid cut 0.0%, tap operations cut 31% | none | **write** | Nothing worse: Omni holds the knob. | `results/live/V3_PANDAPOWER.md` |
| 1-MV-comm--2-sw (constant-power loads) | line and transformer losses cut 0.0%, net import from the upstream grid cut 0.0%, tap operations cut 31% | none | **write** | Nothing worse: Omni holds the knob. | `results/live/V3_PANDAPOWER.md` |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Case | Better in all three of our runs | Worse in all three of our runs | Verdict | Why | Source |
|---|---|---|---|---|---|
| 1-MV-rural--1-sw (constant-power loads) | none | line and transformer losses up 0.6%, net import from the upstream grid up 0.0%, tap operations up 100% | **watch** | Why: losses: an exporting grid pays the lower voltage in line losses (the analysis under the table); taps: the declared cost on a grid that barely taps on its own (4 a year, doubled); import: follows the losses, hundredths of a percent. | `results/live/V3_PANDAPOWER.md` |
| 1-MV-rural--2-sw (constant-power loads) | bus-steps outside 0.95-1.05 up -1.4e-04, tap operations cut 16% | line and transformer losses up 1.2%, net import from the upstream grid up 0.0% | **operator's choice** | Why: losses: an exporting grid pays the lower voltage in line losses (the analysis under the table); import: follows the losses, hundredths of a percent. The operator who wants the loads' energy and the upstream import down wires in and accepts the line losses; the operator who runs the grid to its losses watches. | `results/live/V3_PANDAPOWER.md` |
| 1-MV-semiurb--0-sw (constant-power loads) | line and transformer losses cut 0.9%, net import from the upstream grid cut 0.1%, tap operations cut 29% | none | **write** | Nothing worse: Omni holds the knob. | `results/live/V3_PANDAPOWER.md` |
| 1-MV-semiurb--1-sw (constant-power loads) | tap operations cut 31% | line and transformer losses up 0.6%, net import from the upstream grid up 0.0% | **operator's choice** | Why: losses: an exporting grid pays the lower voltage in line losses (the analysis under the table); import: follows the losses, hundredths of a percent. The operator who wants the loads' energy and the upstream import down wires in and accepts the line losses; the operator who runs the grid to its losses watches. | `results/live/V3_PANDAPOWER.md` |
| 1-MV-semiurb--2-sw (constant-power loads) | bus-steps outside 0.95-1.05 up -1.7e-05, tap operations cut 21% | line and transformer losses up 0.9%, net import from the upstream grid up 0.0% | **operator's choice** | Why: losses: an exporting grid pays the lower voltage in line losses (the analysis under the table); import: follows the losses, hundredths of a percent. The operator who wants the loads' energy and the upstream import down wires in and accepts the line losses; the operator who runs the grid to its losses watches. | `results/live/V3_PANDAPOWER.md` |
| 1-MV-urban--0-sw (constant-power loads) | line and transformer losses cut 1.9%, net import from the upstream grid cut 0.0%, tap operations cut 30% | none | **write** | Nothing worse: Omni holds the knob. | `results/live/V3_PANDAPOWER.md` |
| 1-MV-urban--1-sw (constant-power loads) | line and transformer losses cut 1.9%, net import from the upstream grid cut 0.0%, tap operations cut 27% | none | **write** | Nothing worse: Omni holds the knob. | `results/live/V3_PANDAPOWER.md` |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Case | Better in all three of our runs | Worse in all three of our runs | Verdict | Why | Source |
|---|---|---|---|---|---|
| 1-MV-urban--2-sw (constant-power loads) | line and transformer losses cut 1.8%, net import from the upstream grid cut 0.0%, tap operations cut 22% | none | **write** | Nothing worse: Omni holds the knob. | `results/live/V3_PANDAPOWER.md` |

**Why the losses rose on four grids.** The lower setpoint carries the grid's power at a lower voltage and therefore a higher current, and line losses rise with the square of the current. The four grids where losses rose are four of the five that export more than they import (net import negative: rural 1 and 2, semiurb 1 and 2); on the importing grids the loads' own draw falls with the voltage under ZIP loads and losses fall with it. Under constant-power loads the loads draw the same at any voltage, so the setpoint saves nothing on the loads and the exporting grids pay the losses with nothing bought. Tap operations, declared in advance as Omni's expected cost (`docs/PANDAPOWER_PREREGISTRATION.md`): this grid barely taps on its own (4 a year), so Omni's moves double a very small count. The verdict on those four grids is the operator's: the loads' energy and the upstream import down against the lines' losses up; under constant-power loads on 1-MV-rural--1-sw, where nothing was bought, it is watch.

### *Buildings and batteries (CityLearn, UT Austin)*

The knob: the electric batteries' charge and discharge commands.

| Case | Better in all three of our runs | Worse in all three of our runs | Verdict | Why | Source |
|---|---|---|---|---|---|
| ca_alameda_county_neighborhood | electricity bought cut 5.9%, daily peak draw cut 3.6%, highest peak cut 3.0%, ramping cut 3.9%, daily load unevenness cut 1.0%, monthly load unevenness cut 0.7%, distance from zero net energy cut 0.1% | none | **write** | Nothing worse: Omni holds the knob. | `results/live/V3_CITYLEARN.md` |
| citylearn_challenge_2022_phase_all_robustness | electricity bill cut 8.6%, electricity bought cut 15%, carbon cut 12%, daily peak draw cut 14%, highest peak cut 2.7%, ramping cut 7.2%, daily load unevenness cut 2.9%, distance from zero net energy cut 6.1% | monthly load unevenness up 0.1% | **operator's choice** | Why: the monthly load factor moved by a tenth of a percent against a bill 8.6% lower. | `results/live/V3_CITYLEARN.md` |
| citylearn_challenge_2023_phase_2_local_evaluation | electricity bought cut 0.3%, carbon cut 0.2%, daily peak draw cut 1.6%, daily load unevenness cut 2.9%, distance from zero net energy cut 0.1% | electricity bill up 0.3%, ramping up 6.2%, monthly load unevenness up 0.1%, energy not served up 6.0%, time uncomfortable up 0.1% | **operator's choice** | Why: the 2023 pattern: the bill, the ramping and the energy not served up, the electricity bought, the carbon and the daily peak down (the analysis under the table). By rule a trade; by what the battery is for (the bill), a watch. | `results/live/V3_CITYLEARN.md` |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

| Case | Better in all three of our runs | Worse in all three of our runs | Verdict | Why | Source |
|---|---|---|---|---|---|
| citylearn_challenge_2023_phase_2_online_evaluation_1 | electricity bought cut 0.1%, carbon cut 0.2%, daily peak draw cut 1.3%, highest peak cut 3.5%, daily load unevenness cut 2.0%, monthly load unevenness cut 1.9%, distance from zero net energy cut 0.1% | electricity bill up 0.3%, ramping up 5.6%, energy not served up 5.5%, time uncomfortable up 0.0% | **operator's choice** | Why: the 2023 pattern: the bill, the ramping and the energy not served up, the electricity bought, the carbon and the daily peak down (the analysis under the table). By rule a trade; by what the battery is for (the bill), a watch. | `results/live/V3_CITYLEARN.md` |
| citylearn_challenge_2023_phase_2_online_evaluation_2 | electricity bought cut 0.2%, carbon cut 0.2%, daily peak draw cut 1.3%, highest peak cut 3.5%, daily load unevenness cut 1.7%, monthly load unevenness cut 1.9%, distance from zero net energy cut 0.1% | electricity bill up 0.3%, ramping up 5.6%, energy not served up 6.2%, time uncomfortable up 0.1% | **operator's choice** | Why: the 2023 pattern: the bill, the ramping and the energy not served up, the electricity bought, the carbon and the daily peak down (the analysis under the table). By rule a trade; by what the battery is for (the bill), a watch. | `results/live/V3_CITYLEARN.md` |
| citylearn_challenge_2023_phase_2_online_evaluation_3 | electricity bought cut 0.2%, carbon cut 0.2%, daily peak draw cut 1.5%, highest peak cut 3.5%, daily load unevenness cut 2.3%, monthly load unevenness cut 1.9%, distance from zero net energy cut 0.1%, time uncomfortable cut 0.0% | electricity bill up 0.3%, ramping up 5.5%, energy not served up 9.8% | **operator's choice** | Why: the 2023 pattern: the bill, the ramping and the energy not served up, the electricity bought, the carbon and the daily peak down (the analysis under the table). By rule a trade; by what the battery is for (the bill), a watch. | `results/live/V3_CITYLEARN.md` |
| citylearn_challenge_2023_phase_3_1 | electricity bought cut 0.1%, carbon cut 0.1%, daily peak draw cut 2.2%, daily load unevenness cut 4.0%, monthly load unevenness cut 1.2%, distance from zero net energy cut 0.0% | electricity bill up 0.3%, highest peak up 2.5%, ramping up 4.5%, energy not served up 2.0%, time uncomfortable up 0.0% | **operator's choice** | Why: the 2023 pattern: the bill, the ramping and the energy not served up, the electricity bought, the carbon and the daily peak down (the analysis under the table). By rule a trade; by what the battery is for (the bill), a watch. | `results/live/V3_CITYLEARN.md` |
| citylearn_challenge_2023_phase_3_2 | electricity bought cut 0.1%, carbon cut 0.1%, daily peak draw cut 2.2%, daily load unevenness cut 4.1%, monthly load unevenness cut 0.5% | electricity bill up 0.2%, highest peak up 2.5%, ramping up 4.9%, distance from zero net energy up 0.0%, energy not served up 4.1%, time uncomfortable up 0.1% | **operator's choice** | Why: the 2023 pattern: the bill, the ramping and the energy not served up, the electricity bought, the carbon and the daily peak down (the analysis under the table). By rule a trade; by what the battery is for (the bill), a watch. | `results/live/V3_CITYLEARN.md` |
| citylearn_challenge_2023_phase_3_3 | electricity bought cut 0.1%, carbon cut 0.1%, daily peak draw cut 2.6%, daily load unevenness cut 4.7%, monthly load unevenness cut 1.0%, distance from zero net energy cut 0.0% | electricity bill up 0.3%, highest peak up 2.5%, ramping up 4.8%, energy not served up 2.2%, time uncomfortable up 0.0% | **operator's choice** | Why: the 2023 pattern: the bill, the ramping and the energy not served up, the electricity bought, the carbon and the daily peak down (the analysis under the table). By rule a trade; by what the battery is for (the bill), a watch. | `results/live/V3_CITYLEARN.md` |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Case | Better in all three of our runs | Worse in all three of our runs | Verdict | Why | Source |
|---|---|---|---|---|---|
| tx_travis_county_neighborhood | electricity bought cut 3.0%, daily peak draw cut 1.9%, ramping cut 2.9%, daily load unevenness cut 1.1%, monthly load unevenness cut 0.7%, distance from zero net energy cut 0.6% | none | **write** | the runs differ on time uncomfortable (+0.0%). Nothing worse: Omni holds the knob. | `results/live/V3_CITYLEARN.md` |
| vt_chittenden_county_neighborhood | electricity bought cut 2.1%, daily peak draw cut 0.8%, ramping cut 1.3%, daily load unevenness cut 0.3%, monthly load unevenness cut 0.2%, distance from zero net energy cut 0.3% | none | **write** | the runs differ on time uncomfortable (+0.0%). Nothing worse: Omni holds the knob. | `results/live/V3_CITYLEARN.md` |
| baeda_3dem | | | **no knob** | no electric battery in the district (4 buildings, 7 water tanks left native): there is nothing for Omni to hold, so there is nothing to decide; any difference between the arms is CityLearn's own run-to-run variation (scores that differ in A: none; reproduced: yes) | `results/live/V3_CITYLEARN.md` |
| quebec_neighborhood_with_demand_response_set_points | | | **no knob** | no electric battery in the district (20 buildings, 0 water tanks left native): there is nothing for Omni to hold, so there is nothing to decide; any difference between the arms is CityLearn's own run-to-run variation (scores that differ in A: time uncomfortable +0.09%; reproduced: no) | `results/live/V3_CITYLEARN.md` |
| quebec_neighborhood_without_demand_response_set_points | | | **no knob** | no electric battery in the district (20 buildings, 0 water tanks left native): there is nothing for Omni to hold, so there is nothing to decide; any difference between the arms is CityLearn's own run-to-run variation (scores that differ in A: time uncomfortable +0.18%; reproduced: no) | `results/live/V3_CITYLEARN.md` |

**Why the 2023 districts read worse on the bill.** The seven 2023 challenge districts are small (3 to 6 buildings) and short (720 to 2,208 hours), and in them the energy not served and the comfort rows move, where in the county neighbourhoods they read the same: the compass steers the batteries from the district's electricity reading alone (one wire, one muscle, `docs/CITYLEARN_PREREGISTRATION.md`), flattening the draw it sees, and in these districts that leaves less in the batteries for the hours that count against the bill, the hour-to-hour ramping and the energy not served. The bill, what the battery is bought to lower, is worse in all three of our runs here by 0.2% to 0.4%, so for the operator who pays the bill this is a watch. The cause is a reading of the tables, not yet tested on its own.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

### *Drone swarms (gym-pybullet-drones, University of Toronto)*

The knob: the autopilot's cruise override.

| Case | Better in all three of our runs | Worse in all three of our runs | Verdict | Why | Source |
|---|---|---|---|---|---|
| tuning (the tuning swarm: shown, never counted) | energy per mission cut 16%, fleet energy over the window cut 16%, missions per charge at the autopilot's reserve up 19% | none | **write** (not counted) | the runs differ on closest two drones ever came (-51% to -45%), tracking error, RMS (+88% to +91%), cruise override, mean (+49%). Nothing worse: the tracking error is spent inside the compass's band (shown, the runs differ on its exact figure) and no safety gauge moved. | `results/live/V3_SWARM.md` |
| long | energy per mission cut 20%, fleet energy over the window cut 20%, missions per charge at the autopilot's reserve up 25% | none | **write** | the runs differ on closest two drones ever came (-5.7% to -1.5%), tracking error, RMS (+92% to +93%), cruise override, mean (+52%). Nothing worse: the tracking error is spent inside the compass's band (shown, the runs differ on its exact figure) and no safety gauge moved. | `results/live/V3_SWARM.md` |
| mixed | energy per mission cut 18%, fleet energy over the window cut 18%, missions per charge at the autopilot's reserve up 22% | none | **write** | the runs differ on closest two drones ever came (-46% to -42%), tracking error, RMS (+83% to +86%), cruise override, mean (+48% to +49%). Nothing worse: the tracking error is spent inside the compass's band (shown, the runs differ on its exact figure) and no safety gauge moved. | `results/live/V3_SWARM.md` |
| short | energy per mission cut 7.4%, fleet energy over the window cut 7.4%, missions per charge at the autopilot's reserve up 8.0% | none | **write** | the runs differ on closest two drones ever came (-49% to -26%). Nothing worse: the tracking error is spent inside the compass's band (shown, the runs differ on its exact figure) and no safety gauge moved. | `results/live/V3_SWARM.md` |

## The 945 modelled muscles, alone on their plants (evidence class S)

Every muscle's own row, with its verdict and the reason, is in `results/WIRING_VERDICTS.csv`; the labels and intervals are the realms table's (`results/realms/REALMS.md`, `results/realms/MUSCLES.csv`; seeds 3000 to 3009, ten paired seeds a muscle).

| Label (by rule) | Muscles | Verdict |
|---|---|---|
| NONINFERIOR / INCONCLUSIVE | 807 | **watch** |
| SUPERIOR WITHIN GUARDRAILS | 124 | **write** |
| ENERGY IMPROVEMENT WITH SERVICE TRADEOFF | 9 | **operator's choice** |
| SERVICE IMPROVEMENT WITH ENERGY TRADEOFF | 4 | **operator's choice** |
| NOT ESTABLISHED | 1 | **watch** |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

### *By knob*

| Knob | Muscles | Write | Operator's choice | Watch | Of the watch, wrote nothing in any run |
|---|---|---|---|---|---|
| capacity | 454 | 19 | 0 | 435 | 392 |
| setpoint | 134 | 82 | 8 | 44 | 37 |
| power | 102 | 23 | 5 | 74 | 62 |
| admission | 255 | 0 | 0 | 255 | 255 |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## *By family*

| Realm: family | Muscles | Write | Operator's choice | Watch | Wrote nothing |
|---|---|---|---|---|---|
| Compute / AI / Cloud: AI Inference Serving | 24 | 1 | 0 | 23 | 23 |
| Compute / AI / Cloud: AI Training | 18 | 1 | 0 | 17 | 17 |
| Compute / AI / Cloud: Cloud VM & Capacity | 21 | 0 | 0 | 21 | 21 |
| Compute / AI / Cloud: Container Resources | 23 | 0 | 0 | 23 | 23 |
| Compute / AI / Cloud: Cross-Cluster, Multi-Region & Edge | 15 | 0 | 0 | 15 | 14 |
| Compute / AI / Cloud: DPU SmartNIC & Programmable IO | 12 | 0 | 0 | 12 | 12 |
| Compute / AI / Cloud: Distributed Cluster Managers | 15 | 0 | 0 | 15 | 15 |
| Compute / AI / Cloud: GPU Fabric & RDMA | 18 | 0 | 0 | 18 | 18 |
| Compute / AI / Cloud: HPC & Distributed Compute | 19 | 2 | 0 | 17 | 17 |
| Compute / AI / Cloud: Host CPU & Memory | 23 | 11 | 1 | 11 | 11 |
| Compute / AI / Cloud: Kubernetes Dynamic Device Allocation | 8 | 0 | 0 | 8 | 8 |
| Compute / AI / Cloud: Kubernetes Placement & Scheduling | 20 | 0 | 0 | 20 | 20 |
| Compute / AI / Cloud: Kubernetes Workload Scaling | 31 | 0 | 0 | 31 | 31 |
| Compute / AI / Cloud: NVIDIA GPU Hardware | 19 | 6 | 0 | 13 | 13 |
| Compute / AI / Cloud: Node Fleet & Karpenter-Class Control | 24 | 0 | 0 | 24 | 24 |
| Compute / AI / Cloud: OpenShift & Machine API | 9 | 0 | 0 | 9 | 9 |
| Compute / AI / Cloud: Quantum Computing Control Simulation | 16 | 0 | 0 | 16 | 16 |
| Compute / AI / Cloud: Work Admission & Demand Shaping | 19 | 0 | 0 | 19 | 19 |
| Distribution / Specialized: Cache & Memory Services | 16 | 0 | 0 | 16 | 16 |
| Distribution / Specialized: Commerce & Payment Systems | 15 | 0 | 0 | 15 | 15 |
| Distribution / Specialized: Data Analytics & ETL | 15 | 0 | 0 | 15 | 15 |
| Distribution / Specialized: Database & Transactions | 19 | 0 | 0 | 19 | 19 |
| Distribution / Specialized: Messaging & Streaming | 18 | 0 | 0 | 18 | 18 |
| Distribution / Specialized: Network Routing & Switching | 19 | 0 | 0 | 19 | 19 |
| Distribution / Specialized: Observability & Telemetry | 9 | 0 | 0 | 9 | 9 |
| Distribution / Specialized: Reliability, Security & Recovery | 12 | 0 | 0 | 12 | 12 |
| Distribution / Specialized: Runtime & Application | 17 | 0 | 0 | 17 | 17 |
| Distribution / Specialized: Search, Indexing & Vector DB | 16 | 0 | 0 | 16 | 16 |
| Distribution / Specialized: Service Mesh & API Reliability | 16 | 0 | 0 | 16 | 16 |
| Distribution / Specialized: Storage Block/File/Object | 20 | 0 | 0 | 20 | 20 |
| Distribution / Specialized: Telecom RAN & Edge Radio | 12 | 1 | 0 | 11 | 11 |
| Distribution / Specialized: Workflow, Logistics & Fulfillment | 15 | 0 | 0 | 15 | 15 |
| Energy / Facility / Industrial: Building & Critical Environment HVAC | 15 | 5 | 0 | 10 | 4 |
| Energy / Facility / Industrial: Cooling, Chillers & Thermodynamics | 18 | 6 | 0 | 12 | 12 |
| Energy / Facility / Industrial: Energy Storage & Microgrid | 19 | 0 | 3 | 16 | 16 |
| Energy / Facility / Industrial: Facility & Grid Optimization | 15 | 0 | 1 | 14 | 14 |
| Energy / Facility / Industrial: Grid Transmission & Distribution | 12 | 4 | 0 | 8 | 4 |
| Energy / Facility / Industrial: Industrial PLC & Process Automation | 18 | 6 | 0 | 12 | 3 |
| Energy / Facility / Industrial: PDU, UPS & Electrical Distribution | 18 | 0 | 0 | 18 | 18 |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Realm: family | Muscles | Write | Operator's choice | Watch | Wrote nothing |
|---|---|---|---|---|---|
| Energy / Facility / Industrial: Semiconductor Fab & Precision Manufacturing | 13 | 2 | 0 | 11 | 4 |
| Energy / Facility / Industrial: Water Wastewater & Pumping | 12 | 4 | 0 | 8 | 5 |
| Physics / Robotics / Autonomous: Automotive EV & Mobile Powertrain | 14 | 0 | 0 | 14 | 14 |
| Physics / Robotics / Autonomous: Aviation & Autonomous Flight | 20 | 7 | 0 | 13 | 13 |
| Physics / Robotics / Autonomous: Robotics Fleet & Warehouse Automation | 15 | 0 | 0 | 15 | 15 |
| Physics / Robotics / Autonomous: Robotics Motion Control | 18 | 8 | 0 | 10 | 10 |
| Physics / Robotics / Autonomous: Spacecraft & Flight Software | 14 | 4 | 4 | 6 | 6 |
| Energy / Facility / Industrial: Healthcare Critical Environments | 13 | 7 | 0 | 6 | 3 |
| Distribution / Specialized: Medical Imaging & Clinical Systems | 12 | 0 | 0 | 12 | 12 |
| Energy / Facility / Industrial: Agriculture & Irrigation | 14 | 10 | 0 | 4 | 1 |
| Energy / Facility / Industrial: Oil & Gas Pipelines | 14 | 8 | 0 | 6 | 3 |
| Physics / Robotics / Autonomous: Rail Traction & Train Control | 14 | 0 | 0 | 14 | 14 |
| Physics / Robotics / Autonomous: Marine Propulsion & Vessel Automation | 12 | 0 | 0 | 12 | 11 |
| Distribution / Specialized: Ports & Maritime Logistics | 12 | 1 | 0 | 11 | 11 |
| Energy / Facility / Industrial: Mining & Mineral Processing | 14 | 5 | 0 | 9 | 2 |
| Energy / Facility / Industrial: District Heating & Cooling | 13 | 3 | 4 | 6 | 2 |
| Energy / Facility / Industrial: Power Generation & Turbine Control | 14 | 10 | 0 | 4 | 2 |
| Energy / Facility / Industrial: Renewable Generation & Inverter Control | 13 | 2 | 0 | 11 | 3 |
| Physics / Robotics / Autonomous: Elevators & Vertical Transport | 12 | 0 | 0 | 12 | 11 |
| Energy / Facility / Industrial: Pharmaceutical & Food Manufacturing | 14 | 10 | 0 | 4 | 4 |

### *Why the watch muscles are watch*

- **746 muscles wrote nothing in any run.** Their plants' native controllers held the reading inside the compass's band for the whole hour on every seed, so the law never left its cushion and had nothing to push. On these muscles Omni on top is native: nothing earned, nothing lost, and no wire in is needed. They are 392 capacity knobs, 255 admission knobs, 62 power knobs and 37 setpoint knobs, 567 of them on the compute plant, whose native autoscaler and fixed admission limit already serve the modelled demand inside its line.
- **All 255 admission muscles are watch**, and all wrote nothing: the admission knob is native while change is permitted, and limits what is let in only while it is not (a power or heat stress; `docs/REALMS_PREREGISTRATION.md`). No run reached that state, so the knob was never touched. This is the knob's design, not a failure: admission is a brake for an emergency the hour did not hold.
- **62 muscles wrote and showed nothing**: their work per energy moved by at most +1.8% with an interval across zero. They are process loops, inverters, chambers, mills and zones where the law moved the knob inside its band and the plant's output did not measurably follow. Watch, by rule.
- **One muscle is not established** (Elevators & Vertical Transport, `hoist_speed_target`): the interval is too wide to read and the work guardrail did not hold. Watch, until a longer run decides it.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

### *The trades, muscle by muscle*

| Muscle | Family | Knob | Label | Work per energy | Work | Violations (pp) | Verdict |
|---|---|---|---|---|---|---|---|
| `cpufreq_min` | Host CPU & Memory | power | ENERGY IMPROVEMENT WITH SERVICE TRADEOFF | +1.41% (+1.12 to +1.71) | +0.00 % | +0.11 | **operator's choice** |
| `battery_soc_reserve` | Energy Storage & Microgrid | setpoint | SERVICE IMPROVEMENT WITH ENERGY TRADEOFF | -0.21% (-0.22 to -0.21) | +0.00 % | -1.40 | **operator's choice** |
| `time_of_use_schedule` | Energy Storage & Microgrid | setpoint | SERVICE IMPROVEMENT WITH ENERGY TRADEOFF | -0.21% (-0.22 to -0.21) | +0.00 % | -1.39 | **operator's choice** |
| `microgrid_emergency_reserve` | Energy Storage & Microgrid | setpoint | SERVICE IMPROVEMENT WITH ENERGY TRADEOFF | -0.21% (-0.21 to -0.20) | +0.00 % | -1.40 | **operator's choice** |
| `pue_target` | Facility & Grid Optimization | setpoint | SERVICE IMPROVEMENT WITH ENERGY TRADEOFF | -0.12% (-0.13 to -0.12) | +0.00 % | -0.81 | **operator's choice** |
| `payload_duty_cycle` | Spacecraft & Flight Software | power | ENERGY IMPROVEMENT WITH SERVICE TRADEOFF | +0.82% (+0.66 to +0.98) | +0.00 % | +0.92 | **operator's choice** |
| `space_power_budget` | Spacecraft & Flight Software | power | ENERGY IMPROVEMENT WITH SERVICE TRADEOFF | +0.75% (+0.66 to +0.84) | +0.00 % | +0.50 | **operator's choice** |
| `space_thermal_command` | Spacecraft & Flight Software | power | ENERGY IMPROVEMENT WITH SERVICE TRADEOFF | +0.80% (+0.76 to +0.85) | +0.00 % | +0.36 | **operator's choice** |
| `rcs_authority` | Spacecraft & Flight Software | power | ENERGY IMPROVEMENT WITH SERVICE TRADEOFF | +0.84% (+0.75 to +0.93) | +0.00 % | +0.36 | **operator's choice** |
| `return_temperature_target` | District Heating & Cooling | setpoint | ENERGY IMPROVEMENT WITH SERVICE TRADEOFF | +5.15% (+2.49 to +7.80) | -0.02% | +0.04 | **operator's choice** |
| `differential_pressure_setpoint` | District Heating & Cooling | setpoint | ENERGY IMPROVEMENT WITH SERVICE TRADEOFF | +3.74% (+0.62 to +6.87) | -0.06% | +0.06 | **operator's choice** |
| `thermal_storage_level_target` | District Heating & Cooling | setpoint | ENERGY IMPROVEMENT WITH SERVICE TRADEOFF | +3.88% (+1.08 to +6.69) | -0.03% | +0.06 | **operator's choice** |
| `cooling_storage_level_target` | District Heating & Cooling | setpoint | ENERGY IMPROVEMENT WITH SERVICE TRADEOFF | +3.47% (+0.51 to +6.43) | -0.11% | +0.10 | **operator's choice** |
| `hoist_speed_target` | Elevators & Vertical Transport | capacity | NOT ESTABLISHED | -1.63% (-7.65 to +4.39) | -2.50% | +0.00 | **watch** |

The nine energy-for-service trades are the CPU's minimum clock (`cpufreq_min`), four spacecraft power knobs and four district heating setpoints: energy saved at a measurable cost in work or in time in violation. The four service-for-energy trades are three microgrid reserves and a facility's PUE target: time in violation proven lower at a small energy cost. Each is the operator's to take or leave; none is wired in by default.

### *The setpoint muscles, and what the band alone gives*

Of the 82 setpoint muscles that read SUPERIOR, a fixed setpoint at the band's calm end (the `fixed_calm` arm, the native controller with the setpoint fixed, no governor) gave as much or more work per energy on every one. On 64 of them the fixed setpoint also held the service guardrails, so the gain is the band's and an operator may simply fix the setpoint; on 18 the fixed setpoint broke the service guardrail (ENERGY IMPROVEMENT WITH SERVICE TRADEOFF), and there the governor's moving hand is what keeps the service while the energy is saved. The CSV says which is which, muscle by muscle.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

# The organisms

### *The five modelled organisms (every muscle written at once, one governor)*

| Organism | Muscles | Label | Verdict | Figures |
|---|---|---|---|---|
| Compute / AI / Cloud | 430 muscles | SUPERIOR WITHIN GUARDRAILS | **write** | SUPERIOR WITHIN GUARDRAILS: work per energy +0.1% (+0.1 to +0.1), work +0.0%, energy -0.1%, violations -0.012 pp; valid: yes |
| Physics / Robotics / Autonomous | 376 muscles | SUPERIOR WITHIN GUARDRAILS | **write** | SUPERIOR WITHIN GUARDRAILS: work per energy +0.1% (+0.1 to +0.1), work +0.0%, energy -0.1%, violations -0.003 pp; valid: yes |
| Energy / Facility / Industrial | 470 muscles | SUPERIOR WITHIN GUARDRAILS | **write** | SUPERIOR WITHIN GUARDRAILS: work per energy +0.3% (+0.2 to +0.4), work +0.0%, energy -0.3%, violations -0.013 pp; valid: yes |
| Distribution / Specialized | 440 muscles | SUPERIOR WITHIN GUARDRAILS | **write** | SUPERIOR WITHIN GUARDRAILS: work per energy +0.2% (+0.1 to +0.2), work +0.0%, energy -0.2%, violations -0.009 pp; valid: yes |
| The whole tower, every muscle once | 945 muscles | SUPERIOR WITHIN GUARDRAILS | **write** | SUPERIOR WITHIN GUARDRAILS: work per energy +0.3% (+0.2 to +0.4), work +0.0%, energy -0.3%, violations -0.004 pp; valid: yes |

An organism's label is the whole's: every muscle written at once. The wiring an operator takes from this page is finer: the write muscles wired in, the trades chosen, the watch muscles left native with one wire out. The organism result says the whole, written at once, was no worse and a little better; this page says which parts carried it.

### *The organisms with the real cluster inside*

The knob is the cluster's (the autoscaler's replicas, the floor and the machines), under the organism's own demand, with the modelled muscles around it. The cluster's gauges decide the verdict; the organism's modelled work and energy are shown beside them and are judged muscle by muscle above.

| Cell | Verdict | The table's reading, and the figures | Source |
|---|---|---|---|
| compute_ai_cloud at 10 copies (GitHub) | **write** | the table's own reading: better on 5, worse on 0, inside the noise on 6. The cluster's gauges: 2 better (p95, time over the line), 0 worse. The organism's modelled energy -0.1%. | `results/live/V3_SIX_KUBE.md` |
| physics_robotics_autonomous at 10 copies (GitHub) | **watch** | the table's own reading: better on 3, worse on 0, inside the noise on 8. The cluster's gauges: 0 better, 0 worse. The organism's modelled energy -0.1%. | `results/live/V3_SIX_KUBE.md` |
| energy_facility_industrial at 10 copies (GitHub) | **operator's choice** | the table's own reading: better on 4, worse on 1 (HPA replicas, mean), inside the noise on 6. The cluster's gauges: 1 better (time over the line), 1 worse (HPA replicas, mean). The organism's modelled energy -0.3%. | `results/live/V3_SIX_KUBE.md` |
| distribution_specialized at 10 copies (GitHub) | **write** | the table's own reading: better on 5, worse on 0, inside the noise on 6. The cluster's gauges: 2 better (p95, time over the line), 0 worse. The organism's modelled energy -0.2%. | `results/live/V3_SIX_KUBE.md` |
| tower at 10 copies (GitHub) | **write** | the table's own reading: better on 6, worse on 0, inside the noise on 5. The cluster's gauges: 3 better (p95, time over the line, failed requests), 0 worse. The organism's modelled energy -0.3%. | `results/live/V3_SIX_KUBE.md` |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Cell | Verdict | The table's reading, and the figures | Source |
|---|---|---|---|
| stack at 10 copies (GitHub) | **write** | the table's own reading: better on 5, worse on 1 (organism work), inside the noise on 6. The cluster's gauges: 2 better (time over the line, failed requests), 0 worse. The organism's modelled work -0.0006% (6 parts in a million, rounding level), the one worse row, is a model gauge judged muscle by muscle in the realms table and shown here beside the cluster's. The organism's modelled energy -0.4%. | `results/live/V3_SIX_KUBE.md` |
| compute_ai_cloud at 100 copies (GitHub) | **write** | the table's own reading: better on 4, worse on 1 (organism work), inside the noise on 7. The cluster's gauges: 1 better (time over the line), 0 worse. The organism's modelled work -0.0008% (8 parts in a million, rounding level), the one worse row, is a model gauge judged muscle by muscle in the realms table and shown here beside the cluster's. The organism's modelled energy -0.1%. | `results/live/V3_SIX_KUBE.md` |
| physics_robotics_autonomous at 100 copies (GitHub) | **write** | the table's own reading: better on 4, worse on 1 (organism work), inside the noise on 7. The cluster's gauges: 1 better (time over the line), 0 worse. The organism's modelled work -0.0011% (11 parts in a million, rounding level), the one worse row, is a model gauge judged muscle by muscle in the realms table and shown here beside the cluster's. The organism's modelled energy -0.1%. | `results/live/V3_SIX_KUBE.md` |
| energy_facility_industrial at 100 copies (GitHub) | **write** | the table's own reading: better on 7, worse on 0, inside the noise on 5. The cluster's gauges: 3 better (time over the line, parked-worker energy, standby-model energy), 0 worse. The organism's modelled energy -0.4%. | `results/live/V3_SIX_KUBE.md` |
| distribution_specialized at 100 copies (GitHub) | **write** | the table's own reading: better on 6, worse on 0, inside the noise on 6. The cluster's gauges: 2 better (p95, time over the line), 0 worse. The organism's modelled energy -0.2%. | `results/live/V3_SIX_KUBE.md` |
| tower at 100 copies (GitHub) | **write** | the table's own reading: better on 5, worse on 0, inside the noise on 6. The cluster's gauges: 1 better (time over the line), 0 worse. The organism's modelled energy -0.4%. | `results/live/V3_SIX_KUBE.md` |
| stack at 100 copies (GitHub) | **write** | the table's own reading: better on 4, worse on 1 (organism work), inside the noise on 6; OFF THE CLOCK in 3 of 5 repetitions (the organism ended up to 277 s after its window: the machine could not step this many muscles in time, and the cell's last steps saw a cluster at rest). The cluster's gauges: 1 better (time over the line), 0 worse. The organism's modelled work -0.0008% (8 parts in a million, rounding level), the one worse row, is a model gauge judged muscle by muscle in the realms table and shown here beside the cluster's. The organism's modelled energy -0.4%. Off the clock in 3 of 5 repetitions. | `results/live/V3_SIX_KUBE.md` |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Cell | Verdict | The table's reading, and the figures | Source |
|---|---|---|---|
| tower at 1000 copies (Azure) | **write** | the table's own reading: better on 7, worse on 1 (organism work), inside the noise on 4. The cluster's gauges: 4 better (p95, p99, time over the line, failed requests), 0 worse. The organism's modelled work -0.0004% (4 parts in a million, rounding level), the one worse row, is a model gauge judged muscle by muscle in the realms table and shown here beside the cluster's. The organism's modelled energy -0.4%. | `results/live/V3_BIG_ORGANISM.md` |
| stack at 1000 copies (Azure) | **write** | the table's own reading: better on 7, worse on 1 (organism work), inside the noise on 3. The cluster's gauges: 4 better (p95, p99, time over the line, failed requests), 0 worse. The organism's modelled work -0.0006% (6 parts in a million, rounding level), the one worse row, is a model gauge judged muscle by muscle in the realms table and shown here beside the cluster's. The organism's modelled energy -0.4%. | `results/live/V3_BIG_ORGANISM.md` |

## Not decided here

- **Azure's managed Kubernetes (the bill):** the two tables are on Omni v1 (`results/live/V1_AKS_STEADY.md`, `V1_AKS_BURST.md`) on a 4-worker fleet, where every gauge read no difference beyond the noise but the burst's p99 in one run. A fleet too small to show one machine decides nothing; the fleet that can (`docs/K8S_COMPASS_PREREGISTRATION.md`, the fleet that can show one machine) decides this knob on v3 when it runs. Until then the verdict on Azure is watch, for want of a result rather than because of one.
- **The card (NVIDIA, its own meter):** every earlier card result is obsolete; the current governor has not run on a real card. Nothing to decide yet.
- **The 24-hour robustness machines:** running; they test the governor's own staying power, not a knob's value.

## What this means for the wiring

The two-way plug stays one wire in and one wire out (`docs/WIRING_GUIDE.md`). This page decides, per knob, whether the wire in is connected. A knob marked **write** is wired both ways. A knob marked **watch** is wired out only: Omni reads it, learns from it, and never writes it; the native controller lives by itself. A knob marked **operator's choice** is wired out, and in only by the operator who wants that trade. The engine's verdict (`omnicompass/verdict.py`) does the same on the muscle itself at run time, and a knob it has left native is a watch knob whatever this page says. Nothing on this page changes the engine, which stays Omni v3; it changes what an operator connects.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

## The same verdict, live: the brain decides on the knob before it writes

This page judges after the fact, from published tables. The founder's order of 9 October is that the brain judge in real time, on the knob, before it writes, and that a knob which cannot prove it pays stay native. On the five live stacks (the pool, the consumer group, the memory ceiling, the storage-engine cache, the buffer pool) that is now built into the harnesses: `tools/knob_verdict.py` wraps the frozen engine's own verdict (`omnicompass/verdict.py`, unchanged) around every live knob. The knob starts in watch; a paired trial on the stack itself (the knob held at the deepest step already allowed, then one notch further, under the same traffic) allows one notch at a time under the declared objective, the resource reading of the index by default or the service reading at the operator's choice; a refused step is not taken and not retried for a minute; a trial holds the knob; a fail-up never spends beyond the allowance; the operator's setting is always free. The audit carries the cost sample and the verdict's state every second, the arm record sums it, and the three-run tables print the verdict per workload. It is declared as an amendment in each of the five preregistrations with the expectation written before the runs (Redis: left native under the resource objective; Kafka: allowed step by step while each consumer pays; PostgreSQL: the add above the operator's setting refused on `simple_update`; MySQL: the `read_write` chunks refused; MongoDB: the cache given back a notch a trial). The runs on it were dispatched on 10 October 2026 (`docs/RERUN_2026-10-10.md`); this page is then read again from their tables. Inside the modelled realms the organism still takes one directive for every muscle; the verdict per muscle there is the next engine, Omni v4, designed in `docs/OMNI_V4_PLAN.md` and not built.

**Where the brain's verdict stands today, knob by knob.** Who decides, in real time, whether a knob pays before it is written, and in which directions (spend: more resource for service; give back: less resource where the service holds):

| Where | The knob | Who decides it pays, and when | Directions tried | Engine |
|---|---|---|---|---|
| The five live stacks: Redis's memory ceiling, Kafka's consumers, PostgreSQL's pool, MySQL's buffer pool, MongoDB's cache | one knob each | `tools/knob_verdict.py` around the engine's verdict: the knob starts in watch; a paired trial on the stack itself before every notch, one notch a trial, under the declared objective | spend and give back, both tried; the operator's setting always free | outside the engine (Omni v3 unchanged); running since 10 October |
| Kubernetes: the machines | nodes in service | the engine's verdict (`omnicompass/verdict.py`): a paired trial before a machine is taken | give back (a machine taken only when the trial shows the service holds); gas and brake on replicas and the floor by the compass law's band, no trial | Omni v3 |
| The card: the clock ceiling | one 15 MHz step at a time | the engine's verdict: a paired trial per step while the service is calm; no step passing, the card runs as it does alone; the power lid stays at the operator's | give back (slower clock for the same service); the speed floor never under the card's own busy clock | Omni v3 (`omni_controller/gpu_compass.py`) |
| Robot arms: the speed override; drones: the cruise | one knob each | the engine's verdict on the robot and the swarm; two of four robots left native by it | give back | Omni v3 |
| Power grids: the tap; buildings: the batteries | one knob each | the compass law alone, no trial; the trades on four exporting grids and seven 2023 districts are read from the tables above as the operator's choice | the law's own push and pull | Omni v3 (a trial there is the next engine) |



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

| Where | The knob | Who decides it pays, and when | Directions tried | Engine |
|---|---|---|---|---|
| The 945 modelled muscles | each muscle's knob | one directive from the organism's governor, no trial per muscle; 746 never wrote, 13 are trades | the law's own push and pull | Omni v3 (the verdict per muscle is Omni v4, `docs/OMNI_V4_PLAN.md`, not built) |
| The machine itself: the Linux kernel's frequency ceiling (`docs/CPU_POWER_PREREGISTRATION.md`) | one knob | `tools/knob_verdict.py` around the engine's verdict: every notch down tried on the machine first, under the index's reading for one machine (energy x latency / work) or energy per request inside the line | give back only (the ceiling cannot go above the top; the top always free) | outside the engine; built 10 October, waiting for a machine on the metal, no run yet |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

CHAPTER 45

# The Pilot Protocol and Kit

## Pilot Protocol

> **Before you wire anything:** read `DISCLOSURES.md`. Omni-Compass acts only through the wires it is given; it cannot be slapped on. If your paired receipts differ from the published benchmarks in direction, the first presumption is wiring: check the installation with section 8.5 of the manual (*Wired right or wired wrong*).

### *Phase 0: Simulation evaluation (evaluator's own environment)*

Run `python verify.py`. Pass criterion: VERIFICATION: PASS.

### *Phase 1: Shadow (2 to 4 weeks)*

- Capture: `OUT=capture.csv INTERVAL=15 DURATION=<seconds> POWER_CMD="<site power in watts>" bash fleet/capture/kube_capture.sh` (read-only).
- Replay: `python fleet/capture_replay.py capture.csv --idle-w <W> --dyn-w <W> --site-limit-w <W> --out replay/` gives Omni-Compass's recommended node count, power cap and HPA target per decision.
- Governor in OBSERVE on production telemetry read directly from nodes (kubelet/cAdvisor metrics, power meters, queue depth).
- No write credentials are issued.
- Logged per interval: directive, native actions taken, and the observed outcome.
- Pass criteria, agreed before start: logged directives never violate I1 to I5; counterfactual analysis on at least N recorded incidents shows the directive would have reduced time-to-recovery or energy without a backlog increase beyond an agreed bound.

### *Phase 2: Guarded control, one loop at a time (4 to 8 weeks)*

Order: power capping; node count (Cluster Autoscaler set to observe); replica count (HPA set to observe).

- Each loop is handed over separately, with the reset tested at handover and at exit.
- Pass criteria per loop, fixed in advance: SLO attainment not worse than the preceding shadow baseline at the agreed confidence level; zero shield invariant violations; energy per unit of completed work reported with confidence intervals.
- Exit: any criterion failed triggers the reset and returns the loop to its native controller.

### *Scoring your own pilot*

Capture the baseline (a period before the controller, or a matched node pool left on your normal autoscaler) and the Omni-Compass period or pool with fleet/capture/kube_capture.sh, then: `python pilot/score.py --baseline baseline.csv --omni omni.csv [--idle-w W --dyn-w W]` It reports node-hours and energy per used CPU core-hour, utilisation, pending-pod minutes and HPA shortfall minutes, each with a bootstrap 95% interval over hourly blocks. Energy is measured if the captures include power_w. Run long enough for at least 24 blocks per side, and compare matched pools at the same time where possible, because traffic changes between periods.

### *Phase 3: Component retirement*

A decision component is retired only after its loop has passed Phase 2 and a one-at-a-time removal shows no degradation (keep-or-remove rule, Manual Chapter 6). Execution and security components are retained.

### *Reporting*

All pilot metrics, including failures, are reported in the same format as the benchmark results.

## Omni-Compass shadow pilot kit

What a customer runs first. Omni-Compass watches the cluster read-only beside its own autoscalers. Every 15 seconds it decides what it would do, using the same closure law that was benchmarked. It never writes.

1. `kubectl config use-context <the cluster>`
2. `DURATION=86400 IDLE_W=<watts per idle node> DYN_W=<watts at full load> bash scripts/pilot_shadow.sh`
   - It proves with `kubectl auth can-i` that the identity can read and cannot write, and stops if any write permission exists.
   - It captures telemetry and logs every recommendation to `shadow_out/audit.jsonl`.
   - It writes `shadow_out/SHADOW_REPORT.md`: node-hours used against node-hours Omni-Compass would have used, and the write count, which must be 0.
3. Scoring against a matched baseline: `python pilot/score.py --baseline baseline.csv --omni omni.csv` (bootstrap intervals over hourly blocks).

Guarded control follows `docs/PILOT_PROTOCOL.md`: one loop at a time, the reset tested at each handover.

The kit is exercised end to end on kind by the `live-shadow` workflow.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

CHAPTER 46

# The GPU Bench

> **Before you wire anything:** read `DISCLOSURES.md`. Omni-Compass acts only through the wires it is given; it cannot be slapped on. If your paired receipts differ from the published benchmarks in direction, the first presumption is wiring: check the installation with section 8.5 of the manual (*Wired right or wired wrong*).

This is the test that answers "does Omni-Compass save real energy?" with the GPU's own power meter. There is no model in it. Nobody has run it yet on real hardware for this repository; the first run is the first real-meter result.

## What you need

- A Linux machine with one NVIDIA GPU (a rented cloud GPU works: A100, H100, L4, A10, RTX), the NVIDIA driver and `nvidia-smi`.
- Python 3 with PyTorch built for CUDA (`pip install torch numpy`).
- Root, because setting a GPU power limit (`nvidia-smi -pl`) needs it. Persistence mode on is recommended (`sudo nvidia-smi -pm 1`); the receipt records it either way.

## Run it

```
git clone &lt;this repository&gt; &amp;&amp; cd &lt;it&gt;
sudo bash scripts/gpu_paired.sh                      # 5 repetitions, 10 minutes per arm: about 3 hours
sudo REPS=5 DURATION=300 bash scripts/gpu_paired.sh   # 5-minute arms: about 1.5 hours
```

Options (environment variables): `REPS`, `DURATION` (s per arm), `DRAIN` (s after arrivals stop), `COOLDOWN` (s idle before each arm), `GPU` (index), `SAMPLE_MS`, `INTERVAL` (s between Omni decisions), `SLO_MS` (response-time target; default ten bare service times), `WORKLOAD_ARGS` (e.g. `--n 8192 --target-ms 80`).

## Two phases

- `PHASE=smoke` (the default): look for faults and for an effect worth a counted run. Any number of repetitions.
- `PHASE=confirm`: the preregistered test (`docs/GPU_PREREGISTRATION.md`), 10 repetitions. Omni's code must be committed; its files are hashed before the first arm and again after the last, and any change invalidates the run.

```
sudo PHASE=confirm bash scripts/gpu_paired.sh
```

The **primary outcome** is work per energy: requests served per kilojoule the GPU drew.

## What it does

1. **Receipt.** GPU name, UUID, VBIOS, driver, kernel, host, persistence mode, power management, the enforced limit, default, minimum, maximum and current power limit, the workload's hash, the mechanism id, git commit (`receipt.json`). The current power limit is the **snapshot**; every arm must start and end at it. The bench refuses to start unless power management is Enabled (otherwise a written limit would not bind).
2. **Calibrate once.** The workload times its request at the snapshot limit and picks the request size so one request takes about 50 ms (`calib.json`). Every arm uses the same calibration.
3. **Three arms per repetition, order rotated,** each after `COOLDOWN` s idle:
   - **native**: no Omni process.
   - **watch**: Omni runs, reads the GPU and decides, and is forbidden to write. This is the control: it shows what the machine does with Omni present and silent. If it writes even once, the run is invalid.
   - **omni**: Omni writes the GPU power limit.
4. **The same work in every arm.** `tools/gpu_workload.py` sends one seeded stream of requests (fp16 matrix products) at 30%, 60%, 80%, 30%, 60% and 30% of the GPU's full-power capacity. Every arm gets the same requests at the same moments.
5. **Measured by the device.** `nvidia-smi` samples power draw, temperature, utilisation, the power limit, the **enforced** power limit and the clock-limit reasons every 200 ms for the whole arm (the fields the driver reports, listed in `smi_fields.txt`); RAPL CPU package counters are read at both ends where the machine has them. This is receipt C, the outcome: Omni never supplies it.
6. **Reset.** After the omni arm Omni restores the snapshot limit and reads it back. The script checks the limit after every arm.
7. **The table** (`GPU_REPS.md`): each gauge for native, watch and omni, and three paired contrasts with 95% intervals — observation (watch − native), authority (omni − watch), total (omni − native). If an interval includes zero, it says **not proven**. The result label is chosen by rule (`docs/GPU_PREREGISTRATION.md`, amendment 1): SUPERIOR WITHIN GUARDRAILS, ENERGY IMPROVEMENT WITH SERVICE TRADEOFF, NONINFERIOR / INCONCLUSIVE, NOT ESTABLISHED, WORSE, or INVALID. Also: actuator fidelity (receipt B), control effort and representation fidelity (receipt A). A meter that was not fitted prints UNAVAILABLE. The run folder holds every raw file and `SHA256SUMS.txt`.

## What Omni does on the GPU (omni_controller/gpu_governor.py)

Every 2 s it reads the GPU and runs the Omni-Compass engine (the throughput law in `omnicompass/adapter.py`): load is GPU utilisation, power stress is draw over the snapshot limit, heat is temperature over 83 C, queue is response-time pressure. The engine's power cap becomes a power limit, inside hard rules:

- never below the GPU's current draw x 1.3, never below 0.70 of the snapshot (`--min-share`), never below the device minimum, never above the snapshot;
- a busy card (utilisation smoothed over decisions at or over 0.5, `--util-gate`) gets the snapshot limit back at once, and the cap returns only under 0.4;
- optional speed lock (`--baseline-file`, from `tools/gpu_baseline.py` on runs without Omni): the limit follows response time against that baseline, every gauge kept at least 1% faster (`docs/INTEGRATION_MANUAL.md`, level 5);
- no new write until the last one reads back from the device; every write is read back at once and recorded (requested, return code, read back, enforced limit, delay); the engine senses the enforced limit, the one the card

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

obeys; a write the device refuses ends the arm (exit 4) and makes the run invalid; no clock locks are ever written;

- if it cannot read the GPU or the response times, the snapshot limit at once;
- if response time breaks its target, the snapshot limit at once, and for three decisions after;
- kill file or SIGTERM: the snapshot limit, read back.
- **one writer:** if the limit ever reads a value Omni did not write, Omni stops writing, leaves that limit alone, and exits 5 (the run is invalid);
- **heat fails up:** while the card reports a thermal or hardware slowdown, no lower limit is written;
- every decision names every rule that held the engine back (`blocked_by`) and the one that decided (`decided_by`).

Any workload plugs in through `WORKLOAD_CMD` (for example a vLLM or MLPerf inference harness), if it writes `latency.csv`, `requests.csv` and `summary.json` in `tools/gpu_workload.py`'s format and `SLO_MS` is given.

The table also credits each write: joules and requests against native at the same moments of the same stream, grouped by the rule that decided it. CPU energy comes from RAPL by domain: package and DRAM apart, psys never added, wrap undone. The card's own energy counter (NVML) and its ECC and retired-page counters are read at both ends of each arm.

## Gauges

| Gauge | From | Better |
|---|---|---|
| **work per energy (served requests per kJ), primary** | requests served / GPU energy | higher |
| energy, GPU (J) | power.draw integrated over the arm's window | lower |
| energy per served request (J) | the same, over requests served | lower |
| power, GPU mean (W) | energy / window | lower |
| requests served, not served | the workload's own record | more served, fewer not |
| response time mean, 95th, 99th percentile (ms) | arrival to finish of each request | lower |
| temperature, peak and mean (C) | temperature.gpu | lower |
| energy, CPU package (J) | RAPL counters, when present | lower |

## Testing the bench without a GPU

`python tests/test_gpu_bench.py` runs the whole script against a stand-in `nvidia-smi` (`tests/fake_gpu/`), with the workload's `--sim` mode. The stand-in has no real power physics, so its numbers mean nothing; it proves the script, the controls and the validity checks work.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 47

# The GPU Preregistration

Written before any hardware trial. The counted run (`PHASE=confirm`) hashes this file with the code in `FREEZE.json`; a change to either after the run starts invalidates it.

## Question

On one NVIDIA GPU serving a fixed, seeded request stream, does Omni-Compass holding the GPU power limit change the successful work done per joule measured by the device, compared with the device left at its own limit?

Fixed here, before any smoke trial:

- the number of counted repetitions (10);
- the primary outcome;
- the guardrails;
- the analysis.

Nothing seen in smoke may change them.

## Two phases

1. **Smoke** (`PHASE=smoke`, any number of repetitions). Its purpose is to find faults in the harness, and to see whether the effect is large enough to be worth a counted run. Smoke results are not published and are never reported as the result. After smoke, Omni's code may change.
2. **The counted run** (`PHASE=confirm`). Omni's code is committed and frozen before the first trial. The script refuses to run if any frozen file has uncommitted changes, and the table marks the run invalid if a frozen file changes during it. No inspection, tuning or rerun between counted trials. If the counted run fails, it is reported as failed; a new counted run needs a new commit and a new run, and both runs are reported.

## Design

- **Arms:** native (no Omni), watch (Omni runs, may not write), omni (Omni writes the GPU power limit).
- **Repetitions:** 10 in the counted run, each with all three arms back to back, order rotated. Repetitions may run on separate machines of one GPU type (`REP_ONLY`, the GitHub workflow); the three arms of a repetition always share one machine, so every comparison is paired within a machine.
- **Per arm:** 60 s idle, then the pinned workload for 600 s plus 30 s drain.
- **Workload:** `tools/gpu_workload.py`, calibrated once at the start power limit before any arm. The seed is 20260928, and the load phases are 30, 60, 80, 30, 60 and 30% of full-power capacity.

## Outcomes

- **Primary:** work per energy = requests served / GPU energy (kJ). GPU energy is `nvidia-smi` power.draw integrated over the arm's window.
- **Secondary:**
  - GPU energy;
  - energy per served request;
  - requests not served;
  - response time: mean, 95th and 99th percentile;
  - peak temperature;
  - CPU package energy (RAPL), where present;
  - where a smart plug is fitted (`WALL_METER`), whole-machine energy at the wall and work per wall kJ. The plug is read by the bench only, never by Omni. An arm whose plug readings have a gap over 5 s has no wall number.

## Analysis

- **Primary comparison:** omni against native, paired by repetition. Report the mean difference with a two-sided t 95% interval.
- **Result:**
  - The result is *proven better* if the interval lies entirely above zero.
  - It is *proven worse* if the interval lies entirely below zero.
  - Otherwise it is *not proven*.
- **Guardrails, fixed now:** a better primary result counts only if Omni did not buy it with the work. Both guardrails must hold:
  - **Requests served:** the 95% interval of (omni − native) must not reach below −1% of native.
  - **95th-percentile response time:** the interval must not reach above +10% of native.

If a guardrail fails, the verdict is *better on energy, fails the service guardrail*. That is a different product and is reported as such.

- **Watch against native:** reported as the cost of Omni being present. If watch differs from native on the primary outcome as much as omni does, the effect is not attributed to Omni's authority.

## Amendment 1 (2026-09-28, before any hardware trial; no smoke or counted data exist)

Added before any data, to make the chain from engine to plant identifiable. The question, the arms, the primary outcome, the repetitions, and the two guardrails above are unchanged.

- **Three contrasts, all reported, each paired by repetition:** observation = watch − native; authority = omni − watch; total = omni − native (the primary comparison). A total effect is not attributed to Omni's authority where the observation contrast differs materially from zero.
- **Third guardrail, errors:** the 95% interval of (omni − native) requests *not* served must not reach above +1% of native requests served.


© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- **Result label, by rule, never by hand** (`tools/gpu_reps.py`, `label()`):
  - primary proven better, all three guardrails held: **SUPERIOR WITHIN GUARDRAILS**;
  - primary proven better, a guardrail failed: **ENERGY IMPROVEMENT WITH SERVICE TRADEOFF**;
  - primary not proven, all guardrails held: **NONINFERIOR / INCONCLUSIVE**;
  - primary not proven, a guardrail failed: **NOT ESTABLISHED**;
  - primary proven worse: **WORSE**;
  - any invalidity below: **INVALID**.
- **The card obeys enforced.power.limit.** The snapshot records power.limit, enforced.power.limit, default, min and max limits, persistence mode and power management. The bench refuses to start unless power management is Enabled. The bench samples enforced.power.limit where the driver reports it. The governor senses the enforced limit and reads back every write at once; it never writes clock locks.
- **Also invalid:** a native or watch arm whose enforced limit differs from the snapshot; a governor that exits nonzero (a refused start, or a write the device refused — that write ends the arm); smoke and counted repetitions mixed in one table.
- **Three receipts, kept apart.** A (governor, `audit.jsonl` decisions): telemetry consumed; the six-state reading (history-dependent, `state_observed`); the memoryless state the telemetry alone points to (`state_measured`); the U-channel command u evaluated on the evolved state (`u_push`); admissibility; requested and granted authority; shield bound; holds. B (actuator, `actuator` records): requested limit, return code, power.limit read back, enforced limit, delay to realization, override (enforced under requested), restoration. C (outcome, the bench alone): nvidia-smi power, joules, temperature, utilization, clock-limit reasons; the workload's requests, latency and failures. Omni never supplies its own outcome.
- **Secondary, descriptive (no verdict):**
  - actuator fidelity: r_act = read back − requested (mean and max absolute), delay (median, max), enforced-under-requested count, total variation of the realized limit, reversals, refused writes;
  - control effort: J_u = sum of abs(u_push) × decision interval; mean and max abs(u_push); saturations (abs(u_push) = 25); shield interventions (a bound other than the engine decided); holds;
  - representation fidelity: R_int = sqrt(mean over consecutive decision pairs of sum_k w_k (h(z_(k+1)) − F_h(h(z_k), u_k))_k²) over E, U, I_U, S, B, weights 1, with h the memoryless map and F_h the engine's own projection (`state_projected_next`); directional accuracy = share of pairs where sign(projected − current reading) equals sign(next measured − current measured), both movements at least 0.01, per state and by predicted size (0.01–0.03, 0.03–0.1, ≥ 0.1);
  - energy of the rest of the machine = wall − GPU − CPU package, only when all three meters measured the arm. A missing meter prints UNAVAILABLE, never a modelled substitute.
- **Smoke never enters the counted run.** A smoke run that changes any code or parameter is followed by: repair, the tests, a new freeze, and a counted run collected from zero.

## The run is invalid, and reported as invalid, if

- the watch arm executes any power-limit write;
- a native or watch arm sees a power limit other than the snapshot;
- any arm ends at a limit other than the snapshot (the reset failed);
- Omni's frozen files change during the run;

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- the counted runs on uncommitted code.

## Recorded for every Omni decision

Every decision is logged in `audit.jsonl`:

- the raw device telemetry: utilisation, draw, temperature, limit, SM clock, and clock-limit reasons where the driver reports them;
- the engine's six-state reading, the state it had projected for this moment, and the error against it;
- the projection for the next decision. This is the engine's own evolved state from the same step that sets the cap; no separate predictor was added for the experiment;
- whether change was admissible, the requested and granted authority, and which shield bound decided the limit;
- the limit written, and the requests served in the window.

## Amendment 2 (2026-09-28, before any hardware trial; no smoke or counted data exist)

Unchanged:

- the question;
- the arms;
- the primary outcome;
- the repetitions;
- the guardrails of amendment 1.
- **Watch must match native.** If the observation contrast (watch − native) on the primary outcome is proven in either direction, the label is **NOT ATTRIBUTABLE: WATCH DIFFERS FROM NATIVE** and no omni result is published.
- **One writer.** If the power limit ever reads a value that is neither Omni's last write nor the limit before it, another writer is present:
  - the governor stops writing for the rest of the run;
  - it leaves that writer's limit alone;
  - it exits 5;
  - the run is invalid.
- **Heat fails up.** While the device reports a thermal or hardware slowdown (clock-limit reason bits 0x8, 0x20, 0x40, 0x80), no lower limit is written.
- **Credit per write (descriptive).** Each write owns the interval to the next. For that interval, the table records GPU joules and requests finished, Omni minus native, at the same moments of the same seeded stream. It records who decided the write: the engine, a floor, the busy gate, a reflex, heat, or the speed lock. The table says which rule produced the joules; a speed-lock result is not credited to the engine.
- **CPU side (secondary, never on the control path).** RAPL counters are read by domain name at both ends of each arm:
  - `package-N` is summed as CPU package; `dram` is summed separately;
  - `psys` is recorded and never added to either;



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

  - wrapping is undone with `max_energy_range_uj`;
  - a counter that went backwards without a known range, or is missing, prints UNAVAILABLE.
  - The governor never reads these counters.
- **Device energy counter (cross-check, secondary).** Where NVML reports it (Volta and newer), the card's total-energy counter is read at both ends of each arm, beside the integrated power.draw.
- **Card health (descriptive).** Uncorrected and corrected ECC error counts, and pages pending retirement, are read at both ends of each arm.
- **Workload plug.** Any workload may be served through `WORKLOAD_CMD` if it writes the pinned workload's files. It needs its own response-time target (`SLO_MS`). The command is recorded in the receipt. A counted run names its workload before the first trial.

## Amendment 3 (2026-10-01, before any hardware trial; no smoke or counted data exist)

No GPU job has ever been given a machine (every gpu-bench run so far waited in the queue and was cancelled), so no trial data exist. Unchanged:

- the question;
- the arms;
- the primary outcome;
- the repetitions;
- the guardrails, labels and invalidity rules of amendments 1 and 2.
- **Declared envelope, before any trial.** The buyer's service envelope is written to a file before the first trial and recorded with the run (`envelope.json` in the run folder and in every repetition):
  - `power_min_w`, the lowest watts Omni may set. Default (`tools/declare_envelope.py`): max(device minimum, 70% of the power limit read at declaration), rounded up to whole watts;
  - `power_max_w`, the power limit read at declaration;
  - optionally `slo_ms`, the response-time target; without it the target comes from calibration (10 bare service times), as before.
  - The bench refuses an envelope whose floor lies outside [device minimum, starting limit].
  - **The counted run refuses to start without a declared envelope** (`scripts/gpu_paired.sh`, `ENVELOPE`).
- **Envelope floor.** The governor never sets the limit under `power_min_w` (`--floor-w`). When the floor is what lifted a write, the decision record names it (`decided_by`: envelope_floor), so no saving below the floor can be credited to the engine.
- **Outer controller holds.** When the card's own controller (board, BMC or system policy) already holds enforced.power.limit under the current limit, a lower write that would still sit above that enforced limit changes nothing on the card. It is not written; the decision record marks it (`outer_controller_holds`). Only a write that would actually bind, or a return upward, goes out. This is not another writer (amendment 2): the set limit is unchanged and the governor keeps running.
- **Narrow cards are reported as they are.** The device's own limit range is in the snapshot. On a card whose range is narrow (for example a 70 W card that accepts 60–70 W), the envelope is that narrow range; the result is reported for that card and range and not extrapolated to wider cards.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

## Amendment 4 (2026-10-02, after an invalid smoke; no counted data exist)

The first smoke on a rented A10 (results/gpu/smoke-20261002T032459Z, never counted) was invalid by rule: two copies of `scripts/gpu_rented_run.sh` had been started on the same machine, so both benches wrote the same card's power limit. The native and watch arms saw limits they never wrote (116, 137 and 150 W), each governor refused to start beside the other one (exit 5), and one reset restore was undone by the other copy. The card also began at 116 W, a limit an earlier start had left behind, not its 150 W default, so native itself ran capped (83% of samples). None of these numbers measures Omni. The engine, the governor, the outcomes and the analysis are unchanged. The run script now:

- **runs once per machine:** it takes a lock and refuses to start while another copy runs;
- **runs alone on the card:** it refuses to start while any other process is using the GPU;
- **starts from the card's default limit:** it sets power.default_limit before the envelope is declared, so the envelope, the snapshot and every arm start from the card's own default, not from a limit an earlier run left behind.

## Amendment 5 (2026-10-02, before any valid hardware trial; the only smoke so far was invalid, amendment 4)

The Omni arm changes engine. The outcomes, the arms' order, the guardrails, the analysis and the validity rules are unchanged.

- **The Omni arm holds two wires** (`omni_controller/gpu_compass.py`, the compass law of `omnicompass/compass_law.py`): the clock ceiling (`nvidia-smi -lgc`, reset with `-rgc`; cover 35% of the top clock to the top), which sets how high the card's own boost may climb, and the power limit (`-pl`), the lid at what a fully busy card draws at that ceiling plus 10%, never under the declared envelope floor and never over the start limit. The service is read as one position between calm and the response-time line (the worse of p95 and utilization above half) and pulled to the middle; past 95% both wires go to full at once (fail up). The card's firmware keeps its own control; Omni sets only those two values. The earlier power-limit-only governor stays available (`OMNI_ENGINE=one_wire`) and is not the counted run's arm.
- **Why, before the run:** on a modelled card (`results/sim/gpu_two_wire/`, evidence class S, seeds never used while tuning) the one-wire governor gave +0.1% work per energy and the two-wire engine +9.0%. That is a model; this run is the card's own meter.
- **The watch arm** runs the same two-wire engine in watch mode: it computes and records both wires and writes neither.
- **The clock range is reset before and after every arm** (`-rgc`), as the power limit already was; the run script resets it once at the start.
- **The wire check runs first** (`tools/gpu_wire_check.py`): the card's clock must follow a lowered ceiling down and come back up when reset, the power limit must read back what was set, the governor must hand both wires back when stopped and must leave a limit set by another writer alone (exit 5). If any step fails, nothing else runs and the check's report names the wire, the step and what the card said.
- **Wire check, corrected before any trial (2026-10-02):** on the A10 the first wire check failed at "3 up wire (follows up)" although the wire works: under the heavy check load the card's own 150 W limit already held it near 990 MHz, so a ceiling at 60% of the top clock (1017 MHz) left no room for the clock to come back up above it. The check now reads the card's busy clock on its own first and locks at 60% of that. The governor likewise starts its ceiling at the clock the busy card actually runs (a ceiling above it holds nothing), and its


© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

clock cover is 35% of the top clock to the top. No trial had run.

## Amendment 6 (2026-10-02, after the first counted run and before any further trial)

**What the first counted run showed** (A10, commit `c908054`, `results/gpu/run-20261002T082232Z/`): work per energy +3.6% (+2.7% to +4.5%, proven), the same requests served, no request lost, but the 95th-percentile response time +58.5% (510 to 809 ms), so the response-time guardrail failed and the label by rule was ENERGY IMPROVEMENT WITH SERVICE TRADEOFF. It stands as the result of that run.

**Why, from the card's own samples:** while busy the card ran at 736 to 768 MHz under Omni against 861 to 889 MHz on its own, and spent about 30% more time busy for the same work; requests queued behind each slower one. Three faults in the governor, not in the engine: (1) the position counted utilization above half as service trouble, so every burst read as past the wall (fail up in 46% of decisions) and every quiet gap pulled the ceiling down, so each burst began on a lowered clock; (2) the lid followed a curve from the top clock and sat at the 105 W envelope floor in 49 of 165 compass decisions, under the 135 W the card itself draws while busy; (3) the ceiling's cover reached 35% of the top clock, far under the clock the card's own power limit holds it at while busy.

**The Omni arm from now on** (`omni_controller/gpu_compass.py`; the outcomes, arms, guardrails, analysis and validity rules are unchanged):

- the position is response time only (p95 over 5 s, not 30 s); being busy is not a breach;
- **race while work waits:** at 95% utilization or more the ceiling goes to the top and the lid to the start limit; the compass paces only the slack between bursts;
- **the card's own level, learned from its own meter** while the ceiling is at the top and the card is busy: its busy clock (median) and busy draw (90th percentile); until 15 such readings are in, neither wire moves;
- **speed floor:** the ceiling never goes under the card's own busy clock; **lid floor:** the lid never goes under the card's own busy draw plus 10%;
- fail up (past 95% of the line, or blind) is unchanged.

**On the modelled card, before any trial** (`results/sim/gpu_two_wire/`, seeds 5000 to 5009): p95 122.1 ms native, 123.7 ms with the corrected compass; work per energy +8.2% (+6.3% to +10.1%); energy -7.5%; the median response 10.1 to 12.1 ms, slower in the quiet stretches the compass paces. That is a model; the next trial is the card's own meter.

## Amendment 7 (2026-10-02, before any further trial)

The outcomes, arms, guardrails, analysis and validity rules are unchanged. The Omni arm of the counted runs the **service** profile.

- **Two profiles, one switch** (`--profile`, `omni_controller/gpu_compass.py`; the same in `realms/gpu_card.py`):
  - **service**, the default and the counted run's arm: down gain 0.0125, the compass's center at 0.4, the speed floor 3% above the card's own busy clock;
  - **batch**: down gain 0.015, center 0.5, the floor at the card's own busy clock, for work nobody waits on answer by answer. It may be run as a separate, declared counted run (`OMNI_ARGS="--profile batch"`) and is reported as its own result, never pooled with service.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- **How service was chosen, on the model, before the trial** (20 paired seeds, 5000-5009 and 5100-5109; ratios summarised as the geometric mean of the per-seed ratios): down gains 0.01, 0.0125, 0.015, 0.0175 and 0.02 alone, and 0.0125 to 0.0175 crossed with the compass's center (0.4, 0.5), the speed floor (1.00, 1.03 of the card's own busy clock) and the race threshold (0.90, 0.95). The rule: the most work per energy at which no seed's p95 is more than 10% slower than native. Service (0.0125, 0.4, 1.03) gave work per energy +5.3% (+4.2 to +6.5), energy -5.0%, p95 -4.1% (-9.7 to +1.7), worst seed +9%, 0 of 20 seeds more than 10% slower, p99 -1.8%; its neighbours gave the same within a point. 0.015 alone gave +6.2% but 3 of 20 seeds 10% to 48% slower at p95; 0.01 alone gave +3.8% with p95 -1.9%.
- **The ceiling moves in whole clock steps** (--min-change-mhz, 15 MHz), as the card's own clock does and as the model moves it: the force times the gain, as a share of the top clock, is rounded to whole steps, and a pull under half a step moves nothing and is not stored up.
- **The position reads the mean response time of the window** (as the model does), between the bare service time and the line; the 95th percentile at or past the line, or any failed request, is past the wall (fail up).
- **The model's report summarises ratios on the log scale** (`tools/run_gpu_card.py`): the arithmetic mean of per-seed percentages let one seed (+240%) stand for twenty.

## Amendment 8 (2026-10-03, before any further trial)

The outcomes, arms, guardrails, analysis and validity rules are unchanged, except as stated here.

- **Omni-Compass moves the card only where it measures that the card is no worse for it** (`omnicompass/verdict.py`, in `omni_controller/gpu_compass.py` and `realms/gpu_card.py`). While the service is calm, the governor runs a paired trial:
  - first the ceiling at the top until 30 requests are measured;
  - then one 15 MHz step past the deepest step already allowed, until 30 more are measured.

Each request's cost is the card's own time on it: the workload's new `service_ms` column, start to done, with the wait in the queue left out. The step is allowed if its median cost is at most **2%** above the median at the top. Otherwise it is refused and not tried again for 900 decisions. The compass may move the ceiling only between the top and the deepest allowed step. Where no step passes, the ceiling stays at the top and the card runs as it does alone. Every trial and every judgement is in the audit (`verdict`, `verdict_state`, `verdict_deepest_step`).

- **One law, no profiles.** The service and batch profiles are removed. The law is: down gain 0.0125, the compass's center 0.4, the speed floor at the card's own busy clock, and the verdict's allowance of 2%. The batch profile paced past the 2% allowance, so it is gone.
- **Why 2%** (the card model, 20 paired seeds):
  - an allowance of 0 leaves the card native on every workload, because every clock step down adds some time to a request, and saves nothing;
  - 2% is the smallest allowance that saves energy, and it is below what a person or a service contract can notice.
- **A second workload, AI token generation** (`tools/gpu_workload.py --kind decode`): fp16 matrix-times-vector over 1 GiB of weights per pass, batch one, so each pass streams every weight from memory, as token generation does. It runs as a second counted run of the same design (10 repetitions × 3 arms × 600 s) after the compute-bound one (`scripts/gpu_rented_run.sh`; `SKIP_DECODE=1` skips it). It is labelled by the same rule and reported as its own result, never pooled.


© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- **Every comparison is a base alone against the same base with Omni on top.** The model's report (`tools/run_gpu_card.py`) runs two bases:
  - the card's own firmware, alone and with Omni on top;
  - an operator's fixed 105 W power cap, alone and with Omni on top.

The old one-wire governor is removed as an arm (it was Omni's own earlier version).

- **On the modelled card, before any trial** (`results/sim/gpu_two_wire/`, firmware alone against firmware with Omni on top; tuning seeds 5000-5009, fresh seeds 5100-5109):
  - compute-bound work: work per energy +0.70% (tuning) and +0.48% (fresh); median +1.56% and +1.47%; p95, p99 and time over the line unchanged within their intervals;
  - AI token generation: work per energy +3.36% and +3.86%; median +0.55% and +0.70%; p95 +0.29% and +0.26%; time over the line 0.

That is a model; the next trial is the card's own meter.

## Amendment 9 (2026-10-03, before any trial on this code)

Everything in amendment 8 stands. Four additions, each run by `scripts/gpu_rented_run.sh` after the two counted runs and each reported as its own result:

1. **Steady under the limit** (`omni_controller/gpu_compass.py`, the same in `realms/gpu_card.py`). While the card is saturated against its own power limit (work waiting, the draw at 97% of the limit or more), the firmware boosts a step, hits the limit and is knocked back. The ceiling is then held at the card's own busy clock under that limit, so the same watts serve the work without the knock-backs. It is never held under that clock, the lid stays at the start limit, and a blind feed still fails up. In the card model, fully loaded under a fixed cap, this gives about +0.6% requests served from the same watts, with p95 about 2% faster.
2. **An operator's power cap underneath.** The card's limit is set to the envelope's lowest watts (70% of its default) before the run, and the paired bench runs native (the cap alone), watch and Omni-Compass on top of the cap (the lid never above the cap). It runs twice, 5 repetitions × 3 arms × 300 s each time:
   - at the usual load;
   - fully loaded (arrivals at 130% of the card's capacity under the cap), so the result is requests served from the same watts.

The card is returned to its default limit afterwards.

3. **The GPU fault drill** (`scripts/gpu_fault_drill.sh`). With the request stream running:
   - the governor is killed outright, and the watchdog must hand the card back;
   - the master switch is pulled, and the governor must hand back, exit and refuse to restart while OFF;
   - the response feed is paused, and the governor must fail up.

Every check must pass, and the card must end at its start limit.

4. **Real AI serving** (`scripts/gpu_vllm.sh`, `tools/llm_workload.py`). An open language model (Qwen/Qwen2.5-0.5B-Instruct) is served by vLLM, installed in its own environment. It is asked the same seeded stream of prompts in every arm, each for exactly 128 new tokens; 5 repetitions × 3 arms × 300 s. Tokens per second and tokens per kilojoule follow from requests served. If vLLM cannot be installed or started on the machine, the stage says so and nothing measured before it changes.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

## Amendment 10 (2026-10-03, before any trial on several cards)

Everything in amendments 8 and 9 stands. **Several cards in one server** (`scripts/gpu_8card.sh`): every card runs `scripts/gpu_rented_run.sh` at the same time, each on its own card with its own wire check, envelope, smoke, the two counted runs and the power cap underneath, on the same committed code. Card i starts its arm rotation i steps later (`ROT_OFFSET`), so no arm always meets the same neighbours. Each repetition stays paired within its own card (its three arms on one card); the pooled table per workload holds every card's repetitions (card c, repetition r is pooled as repetition 100(c+1)+r) and is labelled by the same rule. Each card's own table is reported beside it; a card whose table is invalid is reported, never dropped. The fault drill runs once, on the first card, after every card has finished. Last, **one model across every card** (`scripts/gpu_vllm.sh` with GPU = every card): vLLM tensor parallel, Qwen/Qwen2.5-7B-Instruct, one governor per card on its own card's two wires, all reading the same response times; the energy is every card's summed; 5 repetitions × 3 arms × 300 s, labelled by the same rule. Each card's energy is read from that card alone (the bench's sampling filtered to the card in each repetition's own receipt).

**The short design** (`FAST=1`, written before any trial on several cards): each card runs one smoke round and 3 repetitions of the compute counted run (24 paired repetitions on 8 cards, pooled and labelled by the same rule, each card's own table reported beside it), then one model across every card with 3 repetitions. AI token generation and the power cap underneath are measured on the one-card machine (amendment 9) and are not repeated here.

**The whole stacks with a real card inside, on several cards** (written before any trial): `tools/run_hil.py` as on the one-card machine (amendment 9, the six organisms at 1x, 10x, 100x and 1,000x, repetitions 3, 3, 2, 1 by size, the adaptive step), each organism on its own card at the same time (organism i on card i; with fewer cards than organisms, the next organism waits for a card). Each organism's receipt is its own; nothing is pooled across organisms.

**The pooled design** (`POOLED=1`, written before any trial on several cards): every stage of the whole design, with the repetitions pooled across the cards. Each card runs one smoke round; 3 repetitions of the compute counted run and 3 of the AI token generation counted run (24 paired repetitions per test on 8 cards, pooled and labelled by the same rule, each card's own table beside it); 2 repetitions of each power cap load (16 per load); the six organisms with the card inside, one repetition per size, each organism on its own card; then one model across every card, 3 repetitions; the fault drill once. Arm durations are those of the whole design.

**The time budget** (`MAX_HOURS`, written before any trial on several cards): a run on several cards ends by its budget whatever happens. At the budget the master switch is pulled (every governor hands its card back), every stage still measuring stops and is reported as incomplete (an arm cut short is never counted: its repetition has no complete pair), the stages not started are skipped and named, and the finished ones are packed.

## Amendment 11 (2026-10-04, before any trial on this code)

The six organisms with the card inside (`tools/run_hil.py`), first run 2026-10-02 at commit `c908054`, labelled the card's part SUPERIOR in five organisms while its 95th percentile rose from about 500 ms to 600-935 ms in every organism. The harness counted the card's work and energy and never its response time (its time over the line was fixed at zero). That run is history: its governor was replaced in amendments 6 and 7, and its labels do not meet the founder's rule.

From this amendment the card harness measures what it never counted:

- the card's time over the line (requests slower than the line, ten bare service times, percent of requests) and its 95th percentile, each paired against native;


© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- each row keeps its preregistered label and names, beside it, every measure that came out worse than native by any amount, so what Omni costs is read beside what it gains; every change is read in words (better or WORSE, and what it means); no new line is drawn. The 2% bound stays where the founder set it: inside the engine, as the trigger of the verdict (`omnicompass/verdict.py`: a knob moves only where a paired trial shows the muscle at most 2% worse);
- the stacks' own label (`realms/harness.py label`) puts the band first: its guardrail is the mean time over the line at or under native's, where it was the upper end of the interval under +1 point;
- at one copy, each organism runs 5 paired repetitions (was 3), so an energy change of 1-3% on the card's own meter can be told from noise; 3, 2 and 1 at 10, 100 and 1,000 copies as before.

The single card and the eight cards run this code. A card run started before this amendment is reported in full from its own raw files (its response times included) before any figure is quoted.

Why the stacks were late more often in the first run (+0.25 points) and are not now (-0.03): two knobs moved in commit `2a9285d`. Omni could loosen the operator's HPA target to 0.95 (fewer pods, bursts wait); it is now held at the operator's. A machine went back with no headroom (the next burst waits for a boot); it now goes back only when the rest covers the recent peak at 0.6 of the release level. Each alone leaves the stacks late more often; together they are late less often. The headroom is the least that keeps every seed un-late (`results/realms/RELEASE_MARGIN_SWEEP.md`).

Every report reads each change in words (better or WORSE, and what it means), so a sign is never read alone.

## Amendment 12 (2026-10-05, before any trial on this code)

The card model rerun on the current code (`tools/run_gpu_card.py`, 2026-10-05) showed amendment 9's steady hold making the card slower on its own firmware: compute work, p95 122 ms native against 149 ms with Omni on top, time over the line +0.39 points. A bisection over every commit since the last clean model run puts the change in commit `bd455f9` (amendment 9, item 1). Holding the ceiling at the card's median busy clock takes away the boost the firmware uses to serve a burst at its own factory limit. Under an operator's cap the same hold is what helps: the firmware sawtooths against the low cap, and the hold smooths it.

From this amendment the hold applies only under an operator's cap: the start limit under the card's factory limit, read from the card (`power.default_limit`). If the card will not say, it is treated as no cap, so no hold, and the firmware is left to serve the burst. On the card's own limit, saturated, Omni races, as it did before amendment 9. Test: `tests/test_gpu_compass.py` (no hold at the factory limit; the hold under a 150 W cap on a 214 W card).

The card model with this amendment, 20 seeds (tuning and fresh). Under the cap: p95 6.4% and 7.1% faster, p99 3.2% and 5.8% faster, time over the line 1.3 and 1.0 points lower. On the firmware: energy 0.5-3.7% lower, p95 even. The median is 0.6-1.5% slower, inside the verdict's allowance (`results/sim/gpu_two_wire/`). The real card is the test.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

CHAPTER 48

# The Realms Preregistration

Round 3 is the current one: its section at the end changes only how the realms are made up. Round 2's section replaced round 1's Omni layer. Round 1 below is kept as it was frozen. Each round was written and committed before its counted seeds were run. Evidence class **S**: every number the run produces comes from a declared model. Nothing here is a meter, and nothing here is evidence about a real machine.

## Question

For each of the 656 muscles of the canonical tower, and for each realm and the whole tower run as one organism: does the frozen governor (`omnicompass.adapter.Governor`, the engine with u = 0, the stack law, the reset), holding that muscle's one knob on top of the plant's native controller, change work per energy against the native controller alone, without buying it with service?

## What runs

- **The catalog** (`realms/catalog.csv`, built by `tools/realms_catalog.py`): the 656 rows of the XPASS package's canonical tower, unchanged, plus four columns given by fixed rules: realm, plant, parameter set, knob.
  - Compute / AI / Cloud: 250 muscles. Physics / Robotics / Autonomous: 88. Energy / Facility / Industrial: 121. Distribution / Specialized: 197.
- **Five plants** (`realms/plants.py`), each with its own native controller, all parameters in `realms/presets.py`:
  - compute_pool: request stream, servers with start-up delay, idle and dynamic power; native: the HPA rule (10% tolerance, scale-down stabilisation window), fixed admission limit, full clock;
  - thermal_zone: zone heat balance, staged cooling units, COP from supply and outdoor temperature; native: PI on a fixed setpoint, units staged to the load;
  - energy_storage: site load, solar, battery, grid connection with a contract limit; native: self-consumption above a fixed reserve;
  - motion_axis: point-to-point moves from a task queue, motor copper losses and heating, derating; native: PID with feedforward at full speed;
  - process_loop: first-order process with dead time, pump or heater power; native: PI on a fixed setpoint.
- **A muscle is one knob of one plant.** Omni holds only that knob: capacity (the adapter's capacity law for one plant), setpoint (inside a declared band: calm end at rho0, stress end at rho_min), power (the directive's power cap), or admission (native while change is permitted; while not, only what clears inside the service target, or the flexible share deferred or shed).
- **Each muscle's plant is sized** by a factor 0.6 to 1.4 drawn from its muscle id.

## Arms

On the same seed, so with the same demand, weather and disturbances:

- **native**: the plant and its native controller;
- **watch**: the governor reads every period and writes nothing;
- **omni**: the governor holds the knob; at 90% of the run it is killed and the knob returns to the native controller;
- **fixed_calm** (setpoint muscles only, descriptive): the native controller with the setpoint fixed at the band's calm end. It shows how much of a setpoint result the band alone gives.

Organisms: every plant of a realm (or all 656) on one 15 s clock for one hour. Plants are coupled:

- the electrical power of compute, motion and process plants is heat in the realm's thermal zones;
- the organism's load swing is load on its storage sites;
- the zones' temperature is the ambient every other plant reports.

One governor reads the organism's aggregate, and its one directive sets every muscle's knob.

## Seeds and repetitions

Counted seeds 1000 to 1009: ten paired seeds per muscle and per organism. Development used seeds 0 and 1 only. Nothing from development seeds is reported.

## Outcomes

- **Primary**, per seed: (work_omni / work_native) / (energy_omni / energy_native) − 1. Work is in the plant's own units: requests, IT heat held in specification, site load served, moves completed, or product delivered in specification. For an organism, work is the mean over its plants of work_omni / work_native, and energy is total joules.
- **Guardrails**, both must hold:
  - work: the 95% interval of work_omni / work_native − 1 must not reach below −1%;
  - violations: the 95% interval of the change in the share of periods in violation must not reach above +1 percentage point.

Violation, per plant:

  - compute: response time over the service target, or a dropped request;
  - thermal: zone over its limit;
  - storage: grid import over the contract limit;
  - motion: tracking error over its bound, winding over temperature, or the oldest task waiting past its deadline;
  - process: the process variable outside its specification.
- **Label, by rule** (`realms/harness.py`, `label()`, the GPU bench's rule):
  - SUPERIOR WITHIN GUARDRAILS;
  - ENERGY IMPROVEMENT WITH SERVICE TRADEOFF;
  - NONINFERIOR / INCONCLUSIVE;
  - NOT ESTABLISHED;
  - WORSE;

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- INVALID.

## Invalid

A muscle or organism is labelled INVALID if, on any seed:

- the watch arm differs from native in any meter or writes anything;
- the omni arm writes after the kill;
- a knob is not back at its native value after the kill;
- any contrast is not finite.

## Recorded

Per muscle and organism:

- every per-seed contrast;
- writes per run;
- the native violation share;
- the fixed-setpoint comparison where it applies.

Per run: the commit, the seeds, and the fingerprints of the catalog, the engine, the governor and the whole frozen tree (`RUN.json`, `SHA256SUMS.txt`). The counted run refuses to run on uncommitted code.

## Changes made on the development seeds, before this freeze

The development runs found bugs and sizing faults. Each was fixed before the counted seeds were run:

- **The admission knob was inverted**: it admitted only what cleared the service target while change was permitted. Fixed to the declared rule.
- **The compute admission limit compared the queue limit with the period's arrivals** instead of the backlog left after the period's service. Fixed for native and Omni alike.
- **The capacity law did not clip the queue and load readings to [0, 2]**, as the adapter's `observe_vector` does. Fixed.
- **Thermal and process utilisation was measured against the uncapped capacity** while the power cap was held, so the power law's trim ratcheted to its floor. Fixed: utilisation of the capacity actually available.
- **Compute plants reported a heat proxy of my own.** Replaced by the live controller's own thermal model (`omni_controller/muscles.py`).
- **Thermal zones kept fixed cooling units while their IT load was scaled.** The large halls were under-provisioned, so the native controller overheated. Cooling now scales with the hall.
- **The flight axis's motor thermal resistance was ten times too high**, so the native controller was always derated. Corrected. The UPS feed was sized under its own load; sized at 1.2 times.
- **The slow plants** (vehicle, spacecraft, flight) completed too few tasks in six minutes to measure. They now decide every 5, 10 and 2 seconds over the same 360 decisions.
- **Added the fixed_calm arm** for setpoint muscles, after development showed that the band's calm end alone accounts for much of the setpoint results.

None of these changes was chosen by its effect on Omni's result. The development runs showed losses as well as gains for Omni before and after them.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## Not claimed

- **No row is evidence about a real machine.** A row says what the governor's law does to that model through that knob.
- **The plants, native controllers and bands were written by the same project as the governor.** That is a real conflict, so every one of them is in two files, to be read and contested.
- **A muscle's name chooses its knob and its plant's size; it does not get its own physics.** The 16 muscles of a family share the family's plant model at different sizes, through different knobs.
- **The plant code is Python only.** The governor's C++ twin is unchanged; a C++ twin of the plants is open.

## Round 2 (2026-10-01, after round 1's results; before any round-2 counted seed)

Round 1 (seeds 1000-1009) labelled all five organisms WORSE. Reading the result against the shipped controller (`omni_controller/controller.py`, `omni_controller/muscles.py`, `omnicompass/nervous_system.py`) showed that round 1's Omni layer was not the Omni that runs on Kubernetes. Round 1 is kept unchanged in `results/realms/round1/`, with a note saying why it is superseded. Round 2 changes only how Omni commands a knob, the native machine-pool scaler and the declared budgets. These changes were made after seeing round 1, so they are listed with the source line each one follows:

| Round 1 | Round 2, as the shipped controller does it |
|---|---|
| Capacity muscles replaced the HPA with the stack simulator's capacity law (release one unit after convergence, dwell and a 20% band) | Pods: the HPA target written as min(rho*, the operator's target), never tighter, held one autoscaler window; the HPA scales (`controller.py`, HPA target patch). Machine pools: the node release gate, one machine per decision (`nervous_system.node_release_gate`) |
| HPA-type setpoints moved inside a band whose calm end was tighter than the operator's target | The same min(rho*, operator) rule: more headroom is always allowed, less never (`controller.py`) |
| No contraction authority and no SLO reflex | Every contraction needs the organ's authority (calm >= threshold, senses live) and three clean decisions; while service is breached the knob returns to native (`nervous_system.authority`, `muscles.py` SLO reflex) |
| Continuous knobs jumped to the governed value | Down by at most the calm share of the surplus per decision (`nervous_system`: step = calm) |
| Request-served pools had their power capped | Never: throttling request work saves no energy and adds wait (`muscles.py`, `_power_cap`); GPU and CPU-frequency pools use their envelope, never under draw x 1.3 |
| Thermal setpoint by headroom | The live cooling law: warm while cool, cold as heat rises, under the envelope 18 + 9 x calm mapped onto the band (`muscles.py`, `_cooling`) |
| Admission paused every admission muscle, request traffic included, and resumed only when fully calm | Batch pacing of pausable work only: one muscle suspended per decision at power stress >= 0.95 or heat >= 0.96 (or a nervous pause), one resumed per decision at power stress <= 0.8 and heat < 0.90 (`muscles.py`, `_batch_pace` defaults); request traffic never paused |
| queue_ratio was the backlog in service-target units | pending starts per serving unit, or latency pressure (p95 / target − 1, or the failed share), capped at 2 (`controller.py`) |
| The governor's current cap followed the applied cap | 1.0, as the live controller sets it |
| Site power budget 1.25 x a formula nominal that sat under the organism's real native draw (compute ran at 1.2 x it), so the governor read the site as always at its limit | 1.25 x each plant's mean native draw on the calibration seed 999 (never a result seed); compute pools alone likewise |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

| Round 1 | Round 2, as the shipped controller does it |
|---|---|
| Organism heat = the hottest of up to 656 plants | The mean, as every other organism channel; local heat stays with each plant's own reflex |
| Native machine pools used the HPA rule | The Cluster Autoscaler's defaults: add while work waits, remove one machine after the rest has been under 50% for 10 minutes |
| Building cooling sized under its own peak (native overheated half the time) | Units of 35 kW: capacity 1.25 x the declared peak |

Unchanged: the plants' physics, the catalog, every other parameter, the arms, the outcomes, the guardrails, the label rule and the invalidity rules.

- **Round 2 seeds:** 2000 to 2009. Development of round 2 used seeds 0 and 1 only, never reported.
- **Results:** `results/realms/` (round 1 in `results/realms/round1/`). Both rounds are cited together.

## Round 3 (2026-10-02, after round 2's results; before any round-3 counted seed)

Only the make-up of the realm organisms changes. The plants, the Omni layer, the outcomes, the guardrails, the label rule and the invalidity rules are round 2's, unchanged.

- **Round 2 cut the 656 into four realms with no overlap.** No realm organism carried the infrastructure every real stack runs on unless that infrastructure was the realm's own. The data-centre realm had no cooling or power, and the robotics and plant realms had no Kubernetes, machines or GPUs.
- **Round 3 gives every realm the shared spine** (`tools/realms_catalog.py`, SPINE): Kubernetes Workload Scaling, Placement & Scheduling, Container Resources, Node Fleet, Cloud VM & Capacity, NVIDIA GPU Hardware, Host CPU & Memory, Network Routing, Storage, Observability, Reliability & Security, Cooling & Chillers, PDU / UPS & Electrical Distribution.
  - Each realm's organism is its own families plus the spine: Compute 345 muscles, Physics 262, Energy 282, Distribution 337.
  - The whole-tower organism still holds each of the 656 once.
- **Quantum Computing Control moves to the compute realm** (it behaves as a compute job queue).
- **Round 3 seeds:** 3000 to 3009. Round 2 is kept in `results/realms/round2/` with a note on why it is superseded.

## Round 4: the stacked organism (2026-10-02, before any round-4 seed)

The four realm organisms of round 3, stacked on one 15 s clock (`realms/harness.py`, `run_stack`; `tools/run_stack.py`). Every muscle appears as often as it appears in the realms, duplicates included: 345 + 262 + 282 + 337 = 1,226. A duplicate is still a muscle that has to converge. Each realm keeps its own internal coupling. The plants, the Omni layer, the outcomes, the guardrails and the label rule are round 3's.

- **Arms:** native (no governor); separate (one governor per realm); one (one governor over the whole stack, reading the mean of all 1,226 muscles and the stack's total power against its total budget).
- **Check, required for validity:** the stacked native run equals each realm's own native run, plant by plant, on every seed. Stacking must change nothing natively.
- **Comparisons, each labelled by the rule:**
  - one governor against native;


© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

  - separate governors against native;
  - one governor against separate governors (does one Omni over everything beat four).
- **Seeds:** 4000 to 4009. Development used seed 0 only, never reported.

## Round 5: the whole stacks with the real card inside (written 2026-10-02, before any run)

One harness (`tools/run_hil.py`, started by `scripts/gpu_rented_run.sh` after a valid card smoke): each of the six organisms (the four realms, the four stacked with every duplicate kept (1,226), the whole tower of 656) runs on one clock as in round 3, with the machine's real GPU wired in as one more muscle of its NVIDIA GPU family (a spine family, so the card is in every organism). The card serves the pinned request stream; its own power.draw is heat in the organism's thermal zones and load on its storage sites.

- **Arms:** native (the stacks' own controllers, the card's own firmware) and omni (one engine on everything: the compass law on every simulated muscle, `realms/compass_arm.py`, and on the card's two wires, `omni_controller/gpu_compass.py`). At 90% of each arm every knob and both wires are handed back; a knob not handed back, a card limit not back at its start, or a card governor exiting non-zero makes the run invalid (exit 2).
- **Seeds and repetitions:** 3 paired repetitions, seeds 6000-6002; arm order alternates by repetition and organism.
- **Clock:** 240 steps of 2 s of wall clock per arm (the card in real time).
- **Outcomes:** work per energy, work, energy and violations, Omni against native, for three parts kept apart: the simulated stacks (evidence S), the card (its own meter, evidence P), and both added (the card as one more plant, its joules added to the stacks'). Labels by the round 3 rule.

## Round 6: the compute pools inside the band (2026-10-03, before any round-6 seed)

Round 3's rule (band first) was not held on the organisms: the compass law spent 0.20 to 0.29 points more time over the service line than native in every completed cell of the grid (`results/scale/GRID.md`). The cause was measured on the whole tower (seeds 7000-7003), muscle by muscle:

- **compute pools giving a machine back** (42 muscles): +1.77 points of their own time over the line;
- **the HPA target set looser than the operator's** (249 muscles): +0.22 points;
- the process and thermal setpoints, which save the most energy, added none.

Two changes, both in `realms/compass_arm.py`, everything else as round 3:

1. **The HPA target is held at the operator's own.** Loosening it past native costs the service time. This is the same rule as the card's speed floor and the live controller's cover.
2. **A machine goes back only when the machines left cover the recent peak at 0.6 of the plant's own release level** (`RELEASE_MARGIN`). A machine boots in minutes, so a burst that arrives after a release is served late until the machine is back.

**How 0.6 was chosen** (10 paired seeds per organism; the rule is the most work per energy whose whole interval of time over the line is at or under native): 1.0, 0.8, 0.75, 0.7, 0.65 and 0.6 on the tower; 0.65 and 0.6 on the other five.

- 0.65 failed on Physics, the upper end of its interval +0.001;
- 0.6 held on all six.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

**Checked on 20 paired seeds (7000-7019), before this round's official seeds:**

| Organism | Work per energy | Time over the line | Work done |
|---|---|---|---|
| Compute | +0.095% | −0.017 pp | unchanged within its interval (−0.001% to +0.000%) |
| Physics | +0.086% | −0.011 pp | unchanged within its interval (−0.001% to +0.000%) |
| Energy | +0.201% | −0.031 pp | unchanged within its interval (−0.001% to +0.000%) |
| Distribution | +0.090% | −0.015 pp | unchanged within its interval (−0.001% to +0.000%) |
| The four stacked | +0.153% | −0.022 pp | unchanged within its interval (−0.001% to +0.000%) |
| The whole tower | +0.194% | −0.010 pp | unchanged within its interval (−0.001% to +0.000%) |

Every knob was handed back. The grid runs at 100× and 1,000× now on GitHub were started on round 3's law. They are recorded as round 3's result, and the grid is rerun on round 6's law.

## Round 5, amended (2026-10-03, before any round-5 run on the corrected law)

The whole stacks with the real card inside run at four sizes, as the six-organism grid does: each organism as 1, 10, 100 and 1,000 copies governed together on one clock, with the one real card inside as one more muscle of its NVIDIA GPU family, native against Omni on top. Repetitions by size: 3, 3, 2 and 1 (seeds from 6000).

The step is 2 s of wall clock wherever the simulation keeps up. A size whose step takes longer gets a longer step: 1.5 times the measured time per muscle on that machine, times its muscles. The card's request stream runs for the same 240 steps, so the card and the stacks stay on one clock. The step of every size is in the receipt. At 1,000 copies the card is one muscle among hundreds of thousands, so its watts are a small share of the organism's; that is the arithmetic of one card in a large stack, and the card's own meter is reported apart from the stacks. Everything else is as round 5. The two GPU counted runs now run before this stage, so the most important results are in hand first.

## Round 6 receipts and the one rule (2026-10-03)

At 1x and 10x (1,000 paired runs each) every organism is labelled superior within guardrails, with band first held and every knob handed back (`results/scale/receipts/`, `results/scale/GRID.md`). At 1,000 runs the intervals are narrow enough to show one cost: work done is lower by 0.0001% to 0.001%, wholly below zero in several organisms. Its cause was measured on the Physics organism, muscle by muscle. It is the thermal zones: a room held warmer keeps a little less margin, so a sudden heat spike puts it over its limit for a few extra seconds inside a step that already counts as over the line.

Limiting how far a room may drift (half, a quarter, none of the way toward its warm end; 40 paired seeds) does not remove it until the setpoint is held native, and then the Physics organism uses more energy than native. The cost is therefore disclosed and judged by the one rule now written for every muscle (`DISCLOSURES.md`, section 3): at most 2% in any measure, only where energy is saved. The largest work cost in the grid is 0.007% in a single run and 0.001% over 1,000 runs, far inside it.

### *The 1,000-copy size: memory and sharding (2026-10-03, before its re-run)*

The first 1,000-copy run (six run 37090435347) stopped twice at the same point: every organism larger than physics and robotics ran out of memory about 3 minutes in (the four stacked at 1,000 copies is 1.2 million plants; at 33 KB a plant, and with the calibration organism built while the result organism was still held, it needed over 40 GB on a 16 GB runner). Three changes, none of which changes a number:

- the calibration run is made before the organism, so one organism is in memory at a time (`realms/harness.py`, `Body`);



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- each plant is packed as it is made: its exogenous series as machine arrays, its random source dropped (every plant draws its series when it is made, never while it runs), plants with the same parameters sharing one table (`realms/plants.py`, `pack`, `_shared`);
- the six workflow sizes its shards by organism (`per_shard`, `workers` by organism), because one run of the four stacked at 1,000 copies takes 2 to 3 hours and a runner's job ends at 6.

Same outputs: four organisms, two seeds each, at 1x to 3x, byte-identical before and after. A plant now takes 6.7 KB; the four stacked at 1,000 copies needs about 9 GB. The re-run: 10 runs per organism at 1,000 copies (the 1-run and 10-run cells); 100 runs at 1,000 copies follows when the runners allow it (about 650 runner-hours).

The 100-run cells at 1,000 copies run on one rented machine (`scripts/grid_one_machine.sh`): every organism in turn, as many processes as the machine's cores and memory allow, the same seeds (7000 on), each finished organism saved at once, one receipt at the end (`SIX-1000x.md`), saved as `results/scale/v1/receipts/round6-1000x.md` in place of the 1-run and 10-run receipt it contains.

*Evaluation and simulation use only. Copyright (c) 2026 The Omni-Compass LLC. Commercial use, commercialization or monetization of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. All rights reserved. See* `LICENSE` *and* `NOTICE` *at the root of this repository. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## Amendment (2026-10-05): the compass law is the arm, written before its run

The tables of round 3 compared native against the allocation law ("omni"), the arm that came before the compass law. The engine is now one law on everything, the compass law on top of native, so the realms runner compares native against compass (`realms/harness.py` ARMS: native, watch, compass). The watch arm stays: Omni watching must write nothing and leave every muscle exactly as native. Seeds, plants, knobs, guardrails and the labelling rule are unchanged. The tables published until now describe the allocation law and stay as first measured in `docs/history/`.

## Amendment (2026-10-05): four levers bounded by their physics, and the mirror label (written after the compass table was seen)

Disclosed: written after the first compass-law table (nine muscles WORSE) was seen. Each change follows from the muscle's own model (`docs/MECHANISM_OF_ACTION.md` 9.6), applies to every muscle of its kind, and is tested:

1. Cooling power is native. A cap moves heat removal later, never away, and stages more units.
2. A battery's reserve is spent only at the connection's wall, and only when the battery can cover the excess. Below the operator's reserve every kWh costs (1 - eta)/sqrt(eta) to buy back.
3. A backup reserve (the UPS preset, `backup=True`) is never a lever.
4. A compute pool's power cap moves only where (9.8) says a lower cap saves energy. A quantum computer, a network switch and storage are left native (race to idle).

The labelling rule gains the mirror of ENERGY IMPROVEMENT WITH SERVICE TRADEOFF: less work per energy, no work lost (at most 1%), and the time over the line proven lower is SERVICE IMPROVEMENT WITH ENERGY TRADEOFF. Everything else is as before, and every other label keeps its definition.

Also in this amendment: an effort cap on a motion axis moves only where the copper loss it saves at full acceleration, (J a_max / kt)^2 R, exceeds the standing draw a longer move pays, p_idle + b v_max^2 (a reaction wheel; not traction, a flight axis or a robot joint). The machine-release margin on node pools is 0.3, the largest at

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

which no pool is later than native (`results/realms/RELEASE_MARGIN_SWEEP.md`, the per-muscle sweep).

## Omni v2: the catalog grows to 945 muscles in 59 families (2026-10-06, before any v2 seed)

Written before any counted v2 run. **The law, the controllers, the plants, the Omni layer, the outcomes, the guardrails, the label rule and the invalidity rules are v1's, byte for byte.** Only the catalog and the presets it needs change, so by the change rule of `docs/OMNI_V1.md` this is a new version, Omni v2 (`OMNI_V2.json`, `docs/OMNI_V2.md`), and every modelled result is run again on it. No v1 result is read as a v2 result.

What is added, each row with its real system, its setting and its source, built by `tools/realms_catalog_v2.py` from three committed files by the v1 rules (realm, membership, plant, preset and knob: `tools/realms_catalog.py`, unchanged):

- **The 656 of v1, byte for byte** (`realms/catalog_v1.csv`): every v1 muscle keeps its id, realm, plant, preset and knob.
- **118 real controls the realm study had already found and the tower lacked** (`docs/realm_study/TRUE_MUSCLES.csv`, the rows marked "added (real, not in the 656)"; six of them were already in the tower under another name and are listed as skipped in `realms/catalog_provenance.json`). They join the family the study filed them under; 70 of them are spine muscles (Kubernetes, nodes, hosts, GPUs, storage, network, observability, power, cooling) and so sit in all four realms.
- **171 muscles in thirteen new families** (`realms/wave4_families.csv`), the domains the tower did not cover, one new preset each (`realms/presets.py`, round engineering figures for the class of machine, as every other preset):

| Family | Realm | Plant / preset | Muscles | Examples of the real system |
|---|---|---|---|---|
| Healthcare Critical Environments | Energy / Facility / Industrial | thermal_zone / hospital | 13 | ASHRAE 170 rooms on a hospital BMS (Metasys, Desigo); isolation-room pressure; pharmacy cold rooms |
| Medical Imaging & Clinical Systems | Distribution / Specialized | compute_pool / clinical | 12 | DICOM archives and routers (Orthanc, dcm4chee), EHR tiers, HL7/FHIR engines, monitoring gateways |
| Agriculture & Irrigation | Energy / Facility / Industrial | process_loop / irrigation | 14 | pivot and pump controllers (Valley, Lindsay, Netafim), greenhouse climate computers (Priva), grain dryers |
| Oil & Gas Pipelines | Energy / Facility / Industrial | process_loop / pipeline | 14 | compressor and pump station SCADA (Emerson, AVEVA, Honeywell), API 1130 leak detection, DRA injection |
| Rail Traction & Train Control | Physics / Robotics / Autonomous | motion_axis / rail_traction | 14 | traction converters and ATO (Siemens Mobility, Alstom, Hitachi Rail), CBTC headway, ERTMS |
| Marine Propulsion & Vessel Automation | Physics / Robotics / Autonomous | motion_axis / marine_propulsion | 12 | propulsion and power management (Kongsberg, Wärtsilä, MAN), IMO shaft power limitation |
| Ports & Maritime Logistics | Distribution / Specialized | compute_pool / port | 12 | terminal operating systems (Navis N4), automated cranes and AGVs (Kalmar, Konecranes), shore power |
| Mining & Mineral Processing | Energy / Facility / Industrial | process_loop / mill | 14 | grinding and flotation control (Metso, FLSmidth, ABB), ventilation on demand (Howden), dewatering |



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

| Family | Realm | Plant / preset | Muscles | Examples of the real system |
|---|---|---|---|---|
| District Heating & Cooling | Energy / Facility / Industrial | process_loop / district_heat | 13 | network and substation control (Danfoss Leanheat, Kamstrup), CHP dispatch, thermal storage |
| Power Generation & Turbine Control | Energy / Facility / Industrial | process_loop / turbine | 14 | governors and plant DCS (GE Mark VIe, Emerson Ovation, Woodward), AVRs, hydro gates, reactor rods |
| Renewable Generation & Inverter Control | Energy / Facility / Industrial | process_loop / inverter | 13 | IEEE 1547 inverter curves (SMA, Enphase), wake steering (NREL FLORIS), pitch and curtailment (Vestas) |
| Elevators & Vertical Transport | Physics / Robotics / Autonomous | motion_axis / elevator_hoist | 12 | drives and destination dispatch (Otis, KONE, Schindler), ISO 25745 standby modes |
| Pharmaceutical & Food Manufacturing | Energy / Facility / Industrial | process_loop / batch_reactor | 14 | ISA-88 batch control (DeltaV, SIMATIC Batch), bioreactors (Sartorius), pasteurizers, CIP, cleanrooms |

Four wave-4 names collided with the tower and carry a domain prefix (`terminal_tank_level_target`, `train_auxiliary_power_budget`, `plant_reactive_power_target`, `inverter_frequency_droop_setpoint`). Eleven names the knob rules would have placed wrongly are set by hand in `tools/realms_catalog.py` OVERRIDE and listed there.

The organisms of v2: Compute / AI / Cloud 430, Physics / Robotics / Autonomous 376, Energy / Facility / Industrial 470, Distribution / Specialized 440, the four stacked 1,716, the whole tower 945; the shared spine is 257 muscles. In code and workflows the two whole-tower organisms are now named `tower` and `stack` (nothing is named by a count); the v1 names `organism_656` and `stack_1226` are read as aliases, so every v1 run folder and record still reads.

Checked before this freeze, on seed 7, every one of the 289 added muscles: deterministic, the watch arm equal to native, every knob handed back after the kill, no number undefined, Omni's time over the line never above native's. Two presets were adjusted on that check and before any counted run: the inverter plant's disturbance amplitude (0.8 to 0.5, with its noise 0.05 to 0.02) because native itself was outside its band a quarter of the time, and the elevator hoist's position gains (kp 2,000 to 3,000, kd 400 to 500) with a smaller load disturbance (400 to 100 N m), because the first figures put native's tracking error at its limit under passenger load. Where native is still often outside its band in some heavy-load muscles (the compute pools at their busiest sizes), that is the plant at the size the muscle id draws, the same for both arms, and it is reported as it comes.

v2 runs: the realms table (every muscle alone and the five organisms, seeds 3000-3009, three GitHub runs A, B, C), the six-organism grid at 1, 10, 100 and 1,000 copies, the six organisms with the real cluster inside, and the big organisms on Azure, each on the v2 fingerprint; the live benchmarks that import none of the realm files (Kubernetes alone, Azure alone, CityLearn, the grid, the robots, the database) are run again on v2 as GitHub's queue allows, their v1 tables kept as v1 tables.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

# Omni v3: the slack gate on speed knobs (2026-10-06, after the v2 table, before any v3 seed)

Declared on the v2 result and before any v3 run. The v2 table (`results/realms/REALMS.md`, three runs reproduced) read Physics and the whole tower as "energy improvement with a service tradeoff" where v1 had every organism superior. The rows that carried it were the new speed muscles: three rail traction muscles where Omni's slowing added 1.6 to 2.7 points of lateness on a train already near its timetable's capacity, and two marine and two elevator muscles where the slower cycle finished fewer moves in the hour. By the honesty rule the first suspect was our own wiring, and it was: the realm harness had no do-no-harm gate on the speed knob of a motion axis, so the compass slowed an axis that had no slack to spend. The robot benchmark has such a gate (its paired physics trial, `docs/ROBOTICS_PREREGISTRATION.md`); the realms get one now.

**The rule (`realms/compass_arm.py`, `speed_slack`).** A motion axis is offered its speed knob only where its duty at full speed, task rate × (move time at full speed + dwell), is at most SLACK = 0.5, computed from the plant's own figures before any decision. The compass may ease an axis down to 0.4 of its speed, which stretches a move by up to 2.5 times; an axis busy more than half the time at full speed cannot absorb that when tasks arrive in bunches, and its knob stays native. Where the knob is offered, nothing else changes: the compass, its gains, the cover [0.4, 1.0] and the fail-up are v2's.

**The marine preset.** Four 600 rad speed changes an hour made a one-hour run too coarse to count (one unfinished move is a quarter of the hour's work); the v3 preset is twelve changes of 200 rad an hour, a move of 126 s plus 30 s of dwell, deadline 1,200 s. No other preset changes.

**What the gate does on the shipped presets (the middle size; a muscle's own size, 0.6 to 1.4 on the task rate, moves its duty either side):**

| Preset | Move at full speed | Dwell | A task every | Duty | The speed knob |
|---|---|---|---|---|---|
| flight axis | 4.8 s | 2.0 s | 20 s | 0.34 | offered |
| reaction wheel | 30.0 s | 5.0 s | 100 s | 0.35 | offered |
| marine propulsion | 126.5 s | 30 s | 300 s | 0.52 | left native |
| elevator hoist | 12.8 s | 8.0 s | 40 s | 0.52 | left native |
| robot joint | 0.7 s | 0.2 s | 1.7 s | 0.54 | left native at the middle size, offered on the lightly loaded joints |
| EV traction | 35.6 s | 10 s | 60 s | 0.76 | left native |
| rail traction | 220 s | 20 s | 300 s | 0.80 | left native |

The threshold is a declared number set on the v2 finding, not a fitted one: it is written here before the v3 runs and applies to every axis alike. Where it leaves a knob native the row will read as native does, and that is the result. Everything else, the plants, the outcomes, the guardrails, the label rule and the invalidity rules, is v2's. v3 runs the realms table three times (A, B, C), then the grid, then the organisms with the real cluster inside; the v2 table stays a v2 result.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 49

# The Compass Law on Real Kubernetes: Preregistration

Written and committed before the run. The run's commit is the one that carries this file; nothing in the law, the harness or this rule changes after it starts.

## What is run

Workflow `benchmark-reps` on `main`, 10 repetitions. Each repetition runs three arms back to back on the same runner, each on a fresh six-worker kind cluster, in an order rotated by repetition (`scripts/kind_paired.sh`):

| Arm | What governs |
|---|---|
| native | Kubernetes alone (HPA at target 50, scheduler); Omni-Compass not started |
| omni | Omni-Compass on top with the engine's allocation law (`--law governor`, the law of sets 22 to 25) |
| compass | Omni-Compass on top with the compass law (`--law compass`, `omni_controller/controller.py`) |

Load: fixed rate (`loadgen=open`), the same work in every arm. 900 measured seconds per arm. SLO 500 ms at the 95th percentile. Every Omni arm runs the six-state engine on every decision, the nervous system's authority and release gate, the shield, the compass, and ends with the reset, which must return the HPA target, its replica range, the pods' CPU limits and every worker to native, with no record left (`scripts/kind_bench.sh`).

## The compass law in the live controller

The service position is the 95th-percentile response time over the SLO (0 calm, 1 the line); a blind probe or a pod waiting for a place reads as past the wall. The force is `A tanh((K_P (p - 0.5) + K_D v) / A)` with K_D for critical damping times the realm push factor 3, the same law and gains as on every realm muscle (`realms/compass_arm.py`): up gain 0.10, down gain 0.02, release threshold -0.2. Two levers:

1. **HPA target**, cover from 60% of the operator's target to the operator's own: the up force lowers it (more pods), the down force returns it toward the operator's. It is never tighter than native.
2. **Node pool**: past the 0.95 wall one machine more at once; one machine back only while the force is below -0.2, the position is below the center, and the nervous system's release gate is open.

## Outcomes and the rule

Primary: **worker nodes in service** (mean) and **95th-percentile response time**, each arm against native, paired over the 10 repetitions with a t-based 95% interval (`tools/live_reps.py`).

Band first: the compass arm is a win only if its p95 is not worse than native's (the upper end of the 95% interval of the paired difference at or under 0) **and** failed requests are not higher. If that holds and machines in service fall with

an interval wholly below 0, the label is **better on machines within the band**. If machines fall but the band condition fails, the label is **tradeoff**. Otherwise **not established**.

Secondary, reported, not used for the label: p99, mean response time, HPA replicas, pods started, pod start wait, CPU including Omni-Compass's own, the declared energy models. The omni arm is reported against native and against the compass arm by the same rule. A run that fails its own checks (reset, controller stopped early, missing permission) is marked invalid and left out, never silently counted.

Evidence class **L**: real Kubernetes software on kind. Energy on kind is a declared model, not a meter.

## Set 26 result

Compass arm against native: machines in service -17.2% (-26.4% to -7.9% of native), p95 -64.8%, failed requests 0 on both: **better on machines within the band** (`results/live/LIVE_REPS_26.md`). The allocation law in the same set: machines -35.8%, p95 -55.4%.

## Set 27 (written before the run)

The compass in the live controller now reads the service as the GPU compass does (`omni_controller/gpu_compass.py`, GPU amendments 6 and 7): the mean response time of the latency window between the bare service time (a tenth of the SLO) and the SLO, held at the compass's center 0.4 (the GPU service profile); p95 at or past the SLO, a blind probe or a pod waiting for a place is past the wall. Everything else, the arms (native, omni, compass), the load, the duration, the outcomes and the labelling rule above, is unchanged. The run's commit is the one that carries this section.

## Set 27 result

Compass arm, aligned with the GPU governor, against native: machines in service -15.9% (-1.462 to -0.450 machines), p95 -65.5% (-311.8 to -156.1 ms), p99 -72.6%, failed requests 0 on both: **better on machines within the band** (`results/live/LIVE_REPS_27.md`, run 37071353971, commit `d46c959`). The allocation law in the same set: machines -36.6%, p95 -53.1%.

## The verdict in the live controller (2026-10-03, before any further set)

The compass law gives a machine back only where it measures that the service is no worse for it (`omnicompass/verdict.py`, stepwise, in `omni_controller/controller.py`). While the service is calm (inside the compass, no pod waiting, no breach), one more machine is given back on trial. The response times of 200 requests served without it are set against 200 served just before and against the cluster as it first ran on its own:

- at most 2% slower than both: the machine stays given back;
- slower than that: it is taken back and not tried again for 120 decisions.

Where no machine passes, the pool stays as the cluster runs it alone. The HPA target's cover is unchanged: from 60% of the operator's target up to the operator's own, never looser than native. Every trial is in the audit. The next set runs with this verdict; sets 26 and 27 ran before it and stay as they ran.

## Set 28 and set 29 (2026-10-03, set 29 written before its run)

Set 28 (run 37087620193, commit `24666d7`) ran the compass law with the verdict. With Omni-Compass on top against native:



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- machines in service −4.7%;
- p95 −65.3%, p99 −68.6%;
- pods waiting 0;
- failed requests 0.

Total CPU including Omni-Compass's own came out **+2.2%** (+0.003 to +0.042 cores), more than the 2% the one rule allows (`DISCLOSURES.md`, section 3). The cause is the controller's own cost: 0.063 cores, mostly a new kubectl process for every read, about 20 a minute. The compass law with the verdict freed only 0.040 cores of work. The allocation law in the same set: machines −29.6%, p95 −58.0%, total CPU including its own −1.2% (not significant). Set 28's receipt is `results/live/LIVE_REPS_28.md`, published with this section.

**Set 29** runs the same arms, load, duration, outcomes and rule as set 28. One thing changes: the controller reads through one `kubectl proxy` started once, under the same least-privilege identity, so a read is a local HTTP request instead of a new kubectl process (`omni_controller/controller.py`, `Kube`; writes are unchanged; `tests/test_api_proxy.py`). The label also requires total CPU including Omni-Compass's own to be no more than 2% above native.

Set 29 result (run 37094338955, commit `a3721cc`): the compass law with the verdict, total CPU including its own **−4.7%** (−0.071 to −0.017 cores), passes; Omni-Compass's own CPU 0.011 cores (set 28: 0.063). Machines −5.1%, p95 −64.4%, p99 −71.1%, HPA replicas −7.6%, failed requests 0. No measure significantly worse than native in either arm. Receipt: `results/live/LIVE_REPS_29.md`.

## The cost to match (written before its run)

The question a buyer asks: what would native Kubernetes have to spend to answer as fast as it does with Omni-Compass on top? Each repetition runs, on the same runner and the same work, in rotated order:

- native (the operator's HPA target 50);
- native tuned harder by its operator, HPA target 40, 30 and 20 (more pods, faster answers), no Omni-Compass (`ARM=native40`, `native30`, `native20` in `scripts/kind_bench.sh`);
- native with Omni-Compass on top, the allocation law (`omni`);
- native with Omni-Compass on top, the compass law with the verdict (`compass`).

The report (`tools/live_reps.py`, "The cost to match") lists every arm's p95, p99, HPA replicas, CPU including Omni-Compass's own, and machines in service. For each Omni-Compass arm it names the cheapest native setting (by CPU) whose p95 is at or under Omni-Compass's, and that setting's extra replicas, CPU and machines over Omni-Compass. If no native setting tried reaches it, the report says so and gives the lowest native p95. 10 repetitions, 900 measured seconds per arm, fixed-rate load.

## The fault test (written before its run)

Health, security and the babysitting a cluster needs, measured. Every arm (native; native with Omni-Compass on top, the allocation law; native with Omni-Compass on top, the compass law with the verdict) meets the same four faults at the same moments (`scripts/kind_faults.sh`, `FAULTS=1`):

1. at 15% of the run, a worker machine dies (its kind container is stopped) and comes back two minutes later;
2. at 35%, traffic triples for two minutes;
3. at 55%, a pod with no CPU limit burns CPU for two minutes;
4. at 75%, the response-time probe goes blind for one minute.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

For each fault the report (`tools/live_reps.py`, "The fault test") gives the time to recover (from the fault's start until responses stay under the line for 30 s straight, at most 300 s) and the share of samples over the line or failed in the 300 s after it, paired against native over 10 repetitions. All the usual gauges are reported as well, now including the share of response samples over the line (`pilot/bench_report.py`). Lower is better in each.

**The fault test, first run** (run 37094604580, commit `a149d4e`; `results/live/FAULTS.md`). Omni-Compass on top recovered faster than native from every fault (allocation law: machine down −29%, runaway pod −35%, spike −7%; compass law: −14%, −17%, −6%), p95 −53% and −55%, p99 −27% (not significant) and −57%. One measure was worse: with the compass law, **HPA replicas +8.2%** (+0.41 to +0.99, significant), with CPU and machines unchanged and no energy saved, so outside the one rule (`DISCLOSURES.md`, section 3). The cause: past the wall the compass lowers the HPA target at once (more pods, the faster recovery), then handed the operator's target back step by step and held each step for the autoscaler's window, so the extra pods outlived the fault.

**The change, written before the re-run** (`omni_controller/controller.py`, the compass's push and pull on the HPA target; `tests/test_compass_controller.py`, "fault over"): once the responses are back inside the band (the compass's position under its center), the response line is clean and no pod is waiting, the operator's own target returns at once and is not held by the window. More pods only while the fault lasts. The re-run is the same fault test, arms, load, duration and rule (set 30 F); set 30 runs the same code without faults, to show nothing else moved.

**Set 30 and set 30 F** (runs 37105047258 and 37105046042, commit `acc1c4e`; `results/live/LIVE_REPS_30.md`, `results/live/FAULTS_30.md`). Set 30, no faults: nothing significantly worse in either arm; the compass law's machines −9.9%, p95 −64.9%, time over the line −98.7%, HPA replicas −32.2%, total CPU −6.5%. Set 30 F: recovery faster than native from every fault in both arms; the compass law's HPA replicas **+5.6%** (first run +8.2%), still significant, with CPU and machines unchanged. The allocation law, which recovers as fast or faster, held no extra pods (+2.0%, not significant).

**The second change, written before set 31 F** (`omni_controller/controller.py`; `tests/test_compass_controller.py`, "blind"). Past the wall, the compass lowers the HPA target (more pods) only when the cause is load: the response line breached with every sense live and no pod waiting. Past the wall from a blind sense, or from pods waiting for a machine that is gone, more pods answer neither, so the target is the operator's own: fail up is native's own setting, as on the card, where fail up is the card's own clock and limit. The machine reflex (one machine more past the wall) is unchanged. Set 31 F is the same fault test, arms, load, duration and rule; set 31 the same without faults.

**Set 31 and set 31 F** (runs 37110007121 and 37110005322, commit `0a38e76`; `results/live/LIVE_REPS_31.md`, `results/live/FAULTS_31.md`). Set 31 F: the compass law's HPA replicas under faults **−1.1%** (not significant): the extra pods are gone. Nothing significantly worse in either arm; recovery faster than native from every fault (compass law, machine down −49%). Set 31: nothing significantly worse; the compass law's machines −10.4%, p95 −66.0%, p99 −73.0%, time over the line −99.4%, HPA replicas −44.5%, pods started 0 against native's 4.3, total CPU −6.1%.

## The bill on a real cloud (written before its run)

The question a buyer pays for: the same work, a smaller bill? On kind every machine stays powered, so a machine given back saves only a declared model's energy. On a real cloud the machine is deleted and stops being billed. The run (`.github/workflows/aks-metered.yml`, `scripts/aks_paired.sh`, `scripts/kind_bench.sh` with `PLATFORM=aks`):

- **The cluster.** Azure Kubernetes Service, a fresh cluster for every arm, built the same way:


© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

  - a system pool of one machine, tainted so no workload lands on it (AKS's add-ons and the load generator: kind's control plane);
  - a work pool starting at 4 machines (Standard_D2s_v5) under **Azure's own cluster autoscaler** (min 1, max 4; scale down after 2 minutes unneeded), which deletes a machine once it is empty.
- **The arms**, rotated in each repetition:
  - native: Kubernetes with Azure's autoscaler alone;
  - native with Omni-Compass on top, the compass law with the verdict (`compass`);
  - native with Omni-Compass on top, the allocation law (`omni`).

With Omni-Compass on top, the machines it gives back are idled (new pods go elsewhere, their pods leave first), and Azure's autoscaler then deletes them. Omni-Compass never deletes a machine itself.

- **The same work** in every arm: the fixed-rate load of sets 22 onward, 900 measured seconds after 120 s of warm-up.
- **The bill.** Every 15 s, the number of work machines that exist (in service or idle, every one is billed), integrated over the measured window: billed machine-hours, priced at Azure's list price for the machine (USD 0.096 an hour for Standard_D2s_v5, Linux, pay as you go, set in the workflow's input). Omni-Compass never reads this count.
- **Outcomes.** Billed machine-hours and the bill (lower is better), and every gauge of sets 28 and 29 (response time, failures, pods waiting, CPU including Omni-Compass's own), paired against native with 95% intervals over 5 repetitions. Labelled by the one rule (`DISCLOSURES.md`, section 3): nothing more than 2% worse, and only where the bill or energy is saved.
- **Housekeeping.** One repetition at a time; each cluster deleted when its arm ends, before the next is made; the resource group deleted at the end of every repetition whatever happens. It needs the repository secret `AZURE_CREDENTIALS`.

## The capacity test (written before its run)

The question behind "more work for the same cost": on the same machines, how much more work does Kubernetes serve with Omni-Compass on top before its answers break the line? Each repetition runs native, native with Omni-Compass on top (the allocation law) and with the compass law and the verdict, in rotated order, on the same six workers, the open-loop load rising in eight equal steps of one load generator each (6 requests a second per generator, 1 to 8 generators, 200 s a step, 1,600 measured seconds; `load_steps` in `benchmark-reps`). A run's capacity is the highest step at which no more than 5% of the response samples (every 5 s, through the Service) are over the line (500 ms) or failed, every lower step holding too, the first 30 s of each step left to settle (`tools/live_reps.py`, `capacity`). Reported: each arm's mean capacity in requests a second, its change against native and the 95% interval of the paired difference over 10 repetitions; every usual gauge beside it. Labelled by the one rule: nothing more than 2% worse.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

## The fairness test (written before its run)

The question a buyer with many tenants asks: when one application surges, does Omni-Compass on top protect its neighbour, or starve it? Each repetition runs native, native with Omni-Compass on top (the allocation law) and with the compass law and the verdict, in rotated order, on the same six workers, with two applications on them (`TWO_APP=1`, `deploy/kind/noisy.yaml`): php-apache under its usual fixed-rate load, and a noisy neighbour, the same image and the same HPA rule, whose own fixed-rate load surges in steps (0, 0, 6, 0, 6, 0 load generators of 6 requests a second, the same moments in every arm). Each application's response time is probed through its own Service. Omni-Compass governs both HPAs (`deploy/kind/rbac-omni-noisy.yaml`: the second HPA and nothing else more). Reported: every usual gauge for php-apache, and the neighbour's p95, p99, time over the line and failed requests, each paired against native over 10 repetitions with its 95% interval. The label is the one rule, applied to both applications: nothing more than 2% worse for either.

## The capacity and fairness results, and the amendment they call for (2026-10-03 evening, before either is run again)

**Capacity** (`results/live/CAPACITY.md`, run 37150909816): the compass law served 33.0 requests a second within the line against native's 24.6, **+34.1% (95% interval of the paired difference +6.2 to +10.6 requests a second)**, with response times about half of native's and fewer failures. One measure significantly worse: pods started 7.3 against 5.6 (+30.4%) while the mean HPA replicas were 26.9% lower: churn, not more pods. **Fairness** (`results/live/FAIRNESS.md`, run 37154210570): the neighbour unharmed under both laws; with the compass law php-apache's failed requests 3.89% to 4.90% (+0.12 to +1.90 points), pending pods and pod start wait worse, no machine saved. By the one rule neither result is labelled better. Both causes are read from the controller's code; both corrections are stated here, as law, before the tests run again.

**1. Only a sensed muscle moves (the fairness cause).** Let the probe measure the response time of the services in a set $\mathcal{S}$ (`--sensed ns/deployment,...`; empty means every HPA, the single-service case). For each HPA $h$ scaling a deployment $d(h)$ with the operator's target $x^{op}_h$, the target written is

$$x_h(t) = \begin{cases} \text{the compass's (or the allocation law's) target} & d(h) \in \mathcal{S} \\ x^{op}_h & d(h) \notin \mathcal{S} \end{cases}$$

The probe's position $p$ is a statement about $\mathcal{S}$ alone. A push on a muscle outside $\mathcal{S}$ answers no sensed error and adds pods that compete with $\mathcal{S}$ for the same machines (the run's pending pods and failures). It is the GPU law's own rule, applied to Kubernetes: where Omni-Compass cannot sense, it stands at native's own setting.

**2. Pods owed to a growing demand stay (the churn cause).** Let $u(t)$ be the CPU the workers use and $W$ the HPA's scale-down window ($W = 300$ s unless the operator set one). The demand is *still growing* when

$$G(t) = \Big[\, u(t) > (1 + \epsilon)\, \min_{t - W \le s \le t} u(s) \,\Big], \qquad \epsilon = 0.05 .$$

After a breach the compass lowers the target to the bottom of its cover, $x = x_{lo} = 0.6\,x^{op}$ (more pods, at once). Before this amendment, the first decision with the position back under the centre, $p < c = 0.4$, no breach and no pod waiting, returned $x = x^{op}$ at once. Under a rising load the autoscaler then removed the extra pods one window later and started them again at the next step: the run's churn. The return is now

$$x \leftarrow x^{op} \quad \text{only if} \quad p < c,\ \ \text{no breach},\ \ \text{no pod waiting},\ \ \neg G(t),$$

and while $G(t)$ holds the target stays where the breach put it. A fault or a spike that has passed has a flat or falling $u$, so $G$ is false and the operator's target returns at once, as the fault test requires (set 31 F: HPA replicas under



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

faults −1.1%); only a demand still climbing keeps its pods. $\epsilon = 0.05$ is set above the decision-to-decision noise of the node CPU reading. A rise too small to clear it leaves the rule as it was before this amendment (the target returns at once), so the correction can remove churn but cannot add any.

Nothing else changes: the compass's band, gains and centre, the verdict, the fail-up rules, the release gate and the node-pool law are as registered. Tests: `tests/test_compass_controller.py`, cases *demand* and *sensed*, beside the existing *fault over*, *blind* and reset cases.

**The re-runs**, on the commit that carries this amendment, unchanged in design: the capacity test (`load_steps` 1 to 8, 1,600 s), the fairness test (`two_app` 1, 900 s) and the fault test (`faults` 1, 900 s), 10 paired repetitions each, arms native / omni / compass. The second application is now probed and reported as before, and its HPA is held at the operator's target. Labelled by the one rule. Every number, whatever it says, is published beside the runs above.

## The re-runs under the amendment: results (2026-10-04)

All three on commit `199f350`, 10 paired repetitions each, the design unchanged.

- **Fault test, set 32 F** (`results/live/FAULTS_32.md`, run 37162459956): **no measure significantly worse under either law.** The compass law recovers from a lost machine 58% faster and starts 34% fewer pods; the amendment left the fault behaviour of set 31 F intact.
- **Fairness** (`results/live/FAIRNESS_2.md`, run 37162458834): **correction 1 holds.** With the compass law php-apache's failed requests are no longer worse (−14.0%, not significant; before +26.0%), pending pods −10.5% (before +65.2%); the neighbour unharmed under both laws. The compass law's response-time gains in this test are no longer significant. Still worse under both laws: the mean pod start wait (+1.1 s compass, +1.7 s allocation law).
- **Capacity** (`results/live/CAPACITY_2.md`, run 37162457542): the compass law served **24.6 against native's 16.2 requests a second, +51.9% (+6.2 to +10.6)**, the same paired difference as the first run on slower runners; response times −28% to −56%, failures −9.6%, HPA replicas −10.7%. **Correction 2 did not remove the extra pod starts** (5.1 against 3.8, +34.2%; before +30.4%): its premise, that the extra starts were pods removed and started again between steps, is not borne out. With the mean replicas lower, the reading the data support is pods started earlier on a rising load, the mechanism of the added capacity. Correction 2 stays (it removes no gain and added no measure worse); the pod-start rows stand as measured, and by the one rule neither the capacity nor the fairness arm is labelled better while they do.

## The pod record, and the second amendment (2026-10-04, before the next runs)

**What the pod record shows** (`tools/pod_report.py`, workflow `pod-report`, read from the stored artifacts of runs 37162457542 and 37162458834; nothing re-run). In the capacity test native started its pods once, early (33 of 38 in the first fifth of the window), and removed none. With Omni-Compass on top, two minutes into the window the controller raised the HPA target above the operator's (the conveyance: each pod given a larger CPU limit, the target raised by the same factor so each pod stays as busy, $x = g\,x^{op}$; 114% with the compass law, 190% with the allocation law). The autoscaler then removed pods (14 with the compass law, 35 with the allocation law, over ten repetitions), and the next load steps started them again. **The extra starts are exactly those removals.** Correction 2 above read the wrong signal: the CPU used by the whole node, where one load step is lost in the node's own load.

**3. A raise of the target waits for a steady demand.** Let the demand on an HPA's service be read from the autoscaler's own status, $D(t) = \bar{u}(t)\, r(t)$: the pods' mean CPU utilisation of their request times the pods running, the CPU used in units of one pod's request. With $W$ the HPA's scale-down window and $\epsilon = 0.05$,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

$$S(t) = \Big[\, t - t_0 \ge 0.9\,W \ \wedge\ \max_{[t-W,\,t]} D \le (1+\epsilon)\min_{[t-W,\,t]} D \,\Big]$$

($t_0$ the oldest sample in the window). A new target above the one standing (fewer, larger pods) is written only when $S(t)$ holds; the autoscaler itself removes pods only after its scale-down window, and Omni-Compass now asks the same of its own consolidation. A lower target (more pods) is never held, and the return of the operator's own target after a fault (rule 2) is unchanged. Correction 2's growth test $G(t)$ now reads the same $D(t)$ of the HPA concerned, not the node's CPU. Tests: `tests/test_compass_controller.py`, case *steady*.

The capacity, fairness and fault tests are run again on the commit that carries this amendment, unchanged in design, and published beside the runs above whatever they show. The bill run on Azure (`aks-metered`, run 37171672509) started on commit `199f350`, before this amendment, and is reported as of that commit.

## The third amendment, and a change considered and declined (2026-10-04, before the next runs)

**4. A lower target is never held.** Until now every new HPA target, in either direction, was held for one autoscaler window $W$ while the line was clean, so on a step up the compass's push (a lower target, more pods) could wait up to $W$ and the pods arrive after the line is missed. The hold exists because a raise inside the window removes pods the autoscaler then starts again; a lower target asks for pods and removes none. The hold now applies to raises only:

$$\text{write } x_h(t) \iff x_h(t) < x_h^{\text{now}}\ \vee\ \text{no write in } [t-W,\,t)\ \vee\ \text{the operator's own target returns},$$

with rule 3 still requiring a steady demand before any raise. Tests: `tests/test_convey.py` (a lower target written at once inside the window).

**Declined: raising the replica cap above the operator's.** `deploy/kind/demo.yaml` sets `maxReplicas: 10`, and at the top of the capacity test both arms meet it. Lifting it would let Omni-Compass run more pods than the operator authorised, which is more resources, not the same resources used better, and it would break the shield's standing rule that Omni-Compass never moves past an operator's bound. Native would need the same cap for the comparison to stay fair. It stays at the operator's value in every arm; a higher cap is the operator's decision, tested as its own setting if ever asked for. Already in place and unchanged: a pod waiting for a place reads as past the wall and asks for one machine more at once.

A narrower form was proposed the same day (`docs/history/AMENDMENT_RISING_STEP_CAP.md`): while the demand rises and a pod of the sensed service is pending, raise `maxReplicas` by the pending count and lower the target to match. It is declined for a mechanical reason as well as the one above. The replica cap does not make pods pending: at the cap the autoscaler simply asks for no more pods, so the pending count there is zero and the write would never fire where it is aimed. A pod is pending only when the scheduler finds no machine with room for it, and more places under the cap give such a pod nowhere more to go; the answer to that is a machine, which the compass already asks for. The part of the proposal that holds, asking for pods at once on a step up instead of after a window, is rule 4 above.

On the CPU fill (used over allocatable about 0.10 in every arm): on kind every worker reports all of the host's cores as its own, so the allocatable counts the same cores once per worker and the fill reads far lower than the machine doing the work. Every receipt from the next runs on carries the host's own busy share and core count (`host_cpu.csv`, `tools/live_reps.py`), so the room left on the real machine is measured, not inferred.

The capacity, fairness and fault tests are run again on the commit that carries this amendment, beside the runs of commit `5d2e238` (rule 3 alone), so each rule's effect stays separable.



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

## The second amendment's runs: results (2026-10-04)

`results/live/AMENDMENT_2_RUNS.md`, commit `5d2e238` (rule 3 alone), 10 paired repetitions each.

- **Capacity: no measure significantly worse under either law.** Compass law 24.6 against native's 17.4 requests a second, +41.4% (+5.4 to +9.0); allocation law 25.2, +44.8% (+4.9 to +10.7). The compass law's pods started 5.7 against 4.1, interval −0.64 to +3.84: no longer significant. Rule 3 removed the significant pod-start excess and lifted the allocation law's capacity from +0.0% to +44.8%.
- **Faults: no measure significantly worse under either law.**
- **Fairness: the compass law, no measure significantly worse for either application.** The allocation law: php-apache much better, and the neighbour's failed requests **+0.74 points (+0.19 to +1.28), significant**. The allocation law conveys idle CPU to the service it senses; on machines shared with a surging neighbour that CPU is the neighbour's headroom. The allocation law is not labelled better in this test.

**Standing after rule 3: the compass law is clean in all three tests** (no measure significantly worse, capacity +41.4%). Rule 4 (a lower target never held) runs next on commit `583c97f`, beside these, to see whether it adds capacity without costing a row.

## The fourth amendment: the replica cap as a lever the operator grants (2026-10-04, before its run)

The founder's instruction: no number in the harness is hard-wired; every one moves where the system lets it. The cap of 10 in `deploy/kind/demo.yaml` is the value of Kubernetes' own php-apache example, not a choice of any operator, and an earlier session read it as an operator's bound. It becomes a lever, under the operator's grant.

**5. Replica room.** With `--replica-ceiling` $N_{\max}$ granted (0, the default, leaves the cap untouched), for a sensed HPA under the compass law, with $r$ the pods running, $c$ the cap, $\bar{u}$ the pods' mean utilisation and $x$ the target, the autoscaler's own arithmetic asks for $n = \lceil r\,\bar{u}/x \rceil$ replicas, and

$$c \leftarrow \min(N_{\max},\, n) \quad \text{if } r \ge c,\ n > c,\ p \ge \text{centre or the line is breached};$$ $$c \leftarrow c^{op} \quad \text{if } n \le c^{op},\ p < \text{centre},\ \text{no breach},\ S(t).$$

The cap is raised only while it binds and the line is threatened, to what the autoscaler asks and never past the grant, and returns to the operator's once the demand has held still for a window. The operator's range is recorded before the first change; the reset restores it. Tests: `tests/test_compass_controller.py`, case *room*.

**Its run** (a setting of its own, labelled as such): the capacity test unchanged, `replica_ceiling` 30, beside the runs without it. Native keeps its cap of 10, as an operator who has not raised it would; the extra pods the compass uses are counted in the HPA replicas row, so the receipt shows what the capacity cost in pods, not the gain alone.

## The third and fourth amendments, and the bill on a real cloud: results (2026-10-04)

**Rule 4** (`results/live/AMENDMENT_3_RUNS.md`, commit `583c97f`): **the compass law has no measure more than 2% worse in any of the three tests.** Capacity +48.1% (+5.7 to +9.9), nothing worse; fairness, neither application worse (energy per core-hour on the declared standby model +1.5%, inside the allowance); faults, nothing worse, lost-machine recovery −61%. The allocation law: capacity +55.6%, nothing worse; in fairness the neighbour no longer worse. The first runs with the real machine metered: a 4-core GitHub runner 48% to 70% busy, where kind's "used / allocatable" reads 0.07 to 0.10.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

**Rule 5, the replica lever** (`results/live/REPLICA_ROOM.md`, commit `2101c3d`, ceiling 30): the compass law served 24.6 requests a second, the same as without the lever, with 78% more pods and 26 pod starts against native's 4.6, both significant. **The replica cap was not what limited the service; the machine doing the work was.** The lever stays, off by default, as the operator's to grant where machines have room; it is not part of Omni-Compass's default setting and this setting is not labelled better.

**The bill on a real cloud** (`results/live/AKS_BILL.md`, run 37187059424, commit `5b2832f`, four paired repetitions): **no difference in the bill either way** (allocation law −0.6%, compass law +0.8%, intervals across zero) and no measure significantly worse. Azure's own autoscaler already ran the workload on about 1.86 of 4 workers, so this light workload leaves no machine to give back. The machine savings measured on kind, where native has no node autoscaler, do not carry to a cloud with one at this load; that is the reading of record. The preregistered size `Standard_D2s_v5` is not allowed in the subscription's region; `Standard_D2s_v4` (same 2 vCPU, 8 GiB and list price) was used. Three earlier attempts stopped at the load generator's placement before any measurement and are not results.

## The burst bill test (written before its run, 2026-10-04)

The steady bill run left Azure's autoscaler nothing to do and Omni-Compass no machine to give back. The question a buyer asks is the bill under load that moves: on a real cloud, with Azure's own cluster autoscaler underneath, does native with Omni-Compass on top bill fewer machine-hours than native alone when demand rises and falls? Same design as "The bill on a real cloud" (fresh AKS cluster per arm, work pool 1 to 4 `Standard_D2s_v4` under Azure's autoscaler, the bill metered every 15 s, arms native / compass / omni rotated, five repetitions), with the open-loop load in bursts (`load_steps` 1 6 1 8 1 6, six steps over 1,800 measured seconds, the same steps in every arm). Reported: machine-hours and the bill at list price, paired against native with their 95% intervals, beside every service gauge. Labelled by the one rule: a lower bill counts only if nothing is more than 2% worse.

## The six organisms with the real cluster inside (written before its run, 2026-10-04)

Every benchmark runs the same six organisms, native against native with Omni-Compass on top: the four realms, the whole tower of 656 muscles, and the four stacked with every duplicate kept (1,226). Six organisms, two arms, twelve columns. The model grid (`six`) and the card in the loop (`tools/run_hil.py`, single card and eight cards) already do. The Kubernetes runs now do too (`tools/run_kil.py`, workflow `six-kube` on kind, input `organism` on `aks-metered`).

Each organism runs on the measured window's clock, 240 steps, with the real cluster as one more muscle:

- efferent: the organism's offered compute work at step k, D(k) = sum over its compute pools of lam_i(k) / sum of lam0_i, the seeded trace, identical in both arms. The load generator runs r(k) = 1 + round((LOAD_MAX - 1) (D(k) - min D) / (max D - min D)) replicas, LOAD_MAX = 6 declared here, open-loop load.
- afferent: the cluster's watts (capture.csv, the declared power model, every 15 s, identical accounting in both arms) are added to the organism's source, zone and site power, as the card's watts are in the card harness.
- the clock (stated 2026-10-07, after the 1,000-copy runs): the organism must keep the window's clock. `tools/run_kil.py` records how long after the window its last step ended (`behind_s`); the report (`tools/six_kube_report.py`) shows it as "organism behind its window (s)", shown and not judged, and marks a repetition OFF THE CLOCK when either arm ended more than 5% of the window late, because its last steps then saw a cluster whose load schedule had already ended. The pairing stands (both arms slip alike), the mark stays on the cell. The remedy is a longer window for that size on that machine (240 steps, each at least


© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

the machine's time to step the organism once), never a change to the organism or the law: the four stacked at 1,000 copies (1.7 million muscles) step in about 31 s on an 8-vCPU machine, so their window is 10,800 s (45 s steps), against 2,880 s for the tower at 1,000 copies.

Arms: native (the stacks' own controllers, Kubernetes alone, Omni-Compass not started) and compass (the compass law on every simulated muscle, rule 4 on the cluster, handed back at 90% of the window; the run is invalid if any knob is not handed back). Five paired repetitions per organism, order rotated, each arm on a fresh six-worker kind cluster, 960 measured seconds. Seed 6000 for every organism and arm.

Reported in `SIX_KUBE.md` (`tools/six_kube_report.py`): twelve columns of means, then per organism every gauge paired against native with its 95% interval. Cluster rows are measured; organism rows are models (evidence S). Each organism is labelled by the one rule: no measure more than 2% worse (CPU and host load shown, not judged), and a gain counts only where energy or the bill is lower with an interval wholly below zero.

### *Result: the six organisms with the real cluster inside (run 37217362568, commit `d81d5ee`)*

Thirty jobs, six organisms times five paired repetitions, native against native with Omni-Compass on top (compass law), every arm valid, every simulated knob handed back. Full report `results/live/SIX_KUBE.md`; raw files `results/live/raw/run-37217362568/`.

On the real cluster, in every organism, native + Omni was late less often and answered faster:

| Organism | Time over the line, native → Omni | p95, native → Omni (ms) | Failed requests, native → Omni |
|---|---|---|---|
| Compute / AI / Cloud | 41.0% → 26.7% (better, -35%) | 3,967 → 2,653 (better, -33%, inside the noise) | 14.9% → 13.3% |
| Physics / Robotics / Autonomous | 28.4% → 15.5% (better, -45%) | 2,440 → 1,767 (better, -28%) | 5.7% → 5.0% |
| Energy / Facility / Industrial | 21.6% → 10.4% (better, -52%) | 1,951 → 1,482 (better, -24%) | 3.8% → 3.2% |
| Distribution / Specialized | 40.7% → 21.1% (better, -48%) | 2,830 → 1,943 (better, -31%) | 11.1% → 9.3% |
| The whole tower (656) | 25.8% → 14.7% (better, -43%, inside the noise) | 2,085 → 1,247 (better, -40%, inside the noise) | 12.8% → 9.1% |
| The four stacked (1,226) | 55.6% → 43.0% (better, -23%) | 3,703 → 2,803 (better, -24%) | 31.7% → 27.6% |

No measure came out worse beyond the noise in any organism. Energy (declared model) and CPU moved by under 3%, lower with Omni in every organism. The modelled organisms around the cluster: work the same, energy 0.1-0.2% lower, time over the line 0.5-2% lower, in every organism (evidence S).

Read with it: at a peak of six open-loop load generators the 4-core runner is past what the six workers can serve in both arms (native late 22-56% of the time, 4-32% of requests failed; host 65-91% busy). The comparison is fair (same load, same seed, same runner per pair); the absolute levels are an overloaded cluster, and a lower `load_max` is the setting for a cluster inside its capacity.

## More work, faster, on fewer machines, with less energy: all four in one run (written before its run, 2026-10-04)

The four gains so far come from different tests: more work on the same machines (the capacity test, +48.1%), the same work faster on fewer machines (sets 22-27), energy equal or lower in each. This test measures all four in one run.

Design: `benchmark-reps`, ten paired repetitions, arms native and compass (rule 4), order rotated, fresh six-worker kind cluster per arm, open-loop load rising and then falling, `load_steps` 1 2 3 4 5 6 7 8 7 6 5 4 3 2 1, 180 s a step

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

(2,700 measured seconds). The rise is the capacity test (`tools/live_reps.py capacity`, read on the way up to the peak: the highest load step at which no more than 5% of response samples are over the 500 ms line or failed, every lower step too). The fall is where machines are no longer needed and can be handed back. Over the whole window: p95 response time, worker machines in service, energy (declared model; on kind it is not a meter), CPU with Omni's own.

Reported for each, native + Omni against native, paired, with the 95% interval and read in words (better or worse): work (capacity, requests a second inside the line), speed (p95), machines (worker machines in service, mean), energy.

## The Omni index: one number for more for the same, or the same for less (written before its first use, 2026-10-04)

Each measure is turned into a ratio oriented so that above 1 is better for Omni-Compass:

- work: work with Omni / work native (capacity, requests served, work done)
- speed: p95 native / p95 with Omni (a lower response time is faster)
- machines: machines native / machines with Omni (in service, or billed machine-hours)
- energy: energy native / energy with Omni (or the bill)

The index of one test is the geometric mean of its oriented ratios, minus one, in percent: +30% reads "30% more for the same, or the same for 30% less", across work, speed, machines and energy together. A measure a test did not take is left out of that test, never filled in. A measure inside the noise is counted at its mean and flagged. A category (real Kubernetes on GitHub, Azure, the card, the eight cards, the modelled muscles) is the geometric mean of its tests; the headline is the geometric mean of the real categories, each weighted the same; the modelled muscles are shown beside it, never inside it. Computed by `tools/omni_index.py` from each test's own paired results; nothing is typed in.

**The service reading, a second reading declared 2026-10-09 (the first unchanged).** The founder asked what a cache is for. Redis is bought for speed and spends memory to deliver it; the index above counts that memory as a cost at the same weight as the speed it buys, so the Redis category reads −24.9% while its work inside the line and its hit rate read better in all three of our runs in every run. The question is fair, and the answer is a second reading beside the first, never a change to the first: the same tests, the same three-run rule, scored on the service the stack is bought for, **work and speed only**, with the resources it spent or saved (machines, energy, memory, connections, consumers) shown beside it and not scored. It is computed for every test alike by the same tool (`service_pct`, `service_headline_pct`), never for one stack, and printed in the same tables as a second column; the resource reading above stays the headline this program preregistered. The two readings say different true things: where the governor buys service with a resource (Redis's memory ceiling, Kafka's consumers) the service reading is higher and the resource reading says what it cost; where it gives a resource back for the same service (the database pools and caches) the service reading is nothing and the resource reading carries the gain. On the day it was declared the service reading stood at +71.8% against the resource reading's +20.5% (Kubernetes +83.2%, Kafka +1,192.8%, Redis +8.8%, the three database stacks exactly nothing; on the sets of 10 October, the brain's verdict on every stack knob, the two readings stand at +10.7% and +4.8%); Kafka's hundredfold speed ratio is why the service headline is large, and the category table is where a reader sees it. A loss in all three of our runs in work or speed counts against Omni in both readings; neither hides one.

**The wiring verdicts, declared 2026-10-09 (a reading of the tables, not a change to any rule).** The founder's order of the same day: Omni need not be wired into every muscle; where it cannot beat native, the muscle stays native and Omni only reads it, one wire out and no wire in. `tools/wiring_verdicts.py` reads every three-run


© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

table, the simulator tables, the realms table and the organism tables by one rule into three words per knob and case: **write** (a judged gauge better in all three of our runs over all three runs, none worse in all three of our runs), **watch** (nothing better in all three of our runs, whether inside the noise, the runs disagreeing, or a loss with nothing against it) and **operator's choice** (gains and losses both the same in all three of our runs, shown with both readings of the index). Rows shown and not judged never count. Nothing is typed in, and `verify.py` fails if the page (`docs/WIRING_VERDICTS.md`) differs from what the tables give. The verdicts change what an operator connects, never the engine (Omni v3) or any rule written here, and every test on this page is read as before.

## Demand that wanders: up, spike, partway down, back up, down to idle (written before its run, 2026-10-04)

Real demand does not rise once and fall once. It climbs, spikes, eases part way, climbs again and finally settles to idle. This test drives that shape and asks whether machines follow it in order: the emptiest machine idles first on the way down (powered and Ready at its floor, never off), the warm machines wake first on the way up (no boot), one machine always in service for the first burst (`scripts/kind_nodepool.sh`).

Design as the all-four test (ten pairs, native and compass, open-loop load, fresh six-worker kind cluster per arm), with `load_steps` 1 2 3 2 3 4 5 6 5 3 5 6 5 4 3 2 3 2 1 1 (twenty steps of 135 s, 2,700 measured seconds), the same steps in every arm. Reported, native + Omni against native, paired with the 95% interval and read in words: time over the 500 ms line, failed requests, p95, worker machines in service (and its trace step by step), energy (declared model), CPU with Omni's own, pods started. No capacity is read (the load does not only rise).

### *Amendment to the wandering test, written before its run (2026-10-04)*

The founder's two rules for real traffic, adopted before any result of the wandering test: traffic moves one step at a time, up or down, never skipping (it may go 1 2 1 2 1 2 if it never needs 3); and machines never go below two, so two are always in service, ready for a spike, with no ceiling but the machines the cluster has. The run started with the earlier steps (which jumped 5 to 3 and 3 to 5) was cancelled before it finished; none of it is reported.

From this amendment: the machine floor is two in every arm Omni-Compass governs (`MIN_NODES`, default 2: the controller's `--min-nodes`, the actuator `scripts/kind_nodepool.sh`, and every benchmark script). Native's own floor is the cluster's: kind keeps all six workers. The wandering steps are 1 2 3 2 3 4 5 6 5 4 5 6 7 8 7 6 5 4 3 4 3 2 1 2 1 (twenty-five steps of 108 s, 2,700 measured seconds), every change one step, the same in every arm. Everything else is as written above.

### *Result: all four in one run (run 37226863122)*

Ten paired repetitions, every arm valid. **Work +29.3%** (capacity 24.6 to 31.8 requests a second inside the line, interval +5.4 to +9.0), **p95 -62.3%** (685.8 to 258.3 ms), time over the line -40.9%, failed requests -12.1%, **machines in service -3.6%**, **energy -0.3%** (declared model), CPU with Omni's own -1.9%: each proven. No measure worse beyond the noise (pending pod-minutes +19.6% and pods started +4.0%, both inside the noise). This run's machine floor was one; the floor of two was adopted after it started. Report `results/live/ALL_FOUR.md`; raw files `results/live/raw/run-37226863122/`; the Omni index updated (`results/OMNI_INDEX.md`).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

### *The boundary is the band, not a machine held out (written 2026-10-04, for every run started after it)*

Holding one whole machine back as a ceiling was considered and set aside the same day: it takes one of the machines away from the work. Every machine is usable (Omni-Compass's most is every machine that exists); the floor of two stays for quiet times. The protection against running into the wall is the band inside every machine: capacity is added at 95% of the response line, before the line is reached, and a machine goes back only if the ones left still run at or under the engine's utilisation target. An operator who wants whole machines held back can set `NODE_CUSHION` (off by default). The wandering run under way (37235231963) uses every machine, as every run after it does.

## The six organisms at every size, the real cluster inside (written before its run, 2026-10-04)

The same six organisms, each native against native with Omni-Compass on top, at the sizes of the model grid: 10, 100 and 1,000 copies of the organism governed together on one clock, the one real cluster inside as one more muscle (`tools/run_kil.py --scale`; size 1 is the run already recorded, `results/live/SIX_KUBE.md`). Workflow `six-kube`, one job per organism, size and repetition, both arms on one runner, order rotated, fresh six-worker kind cluster per arm.

- Repetitions: 5 at 10 and 100 copies, 3 at 1,000. Measured window: 960 s at 10 copies, 1,440 s at 100, 2,880 s at 1,000 (240 organism steps of 4, 6 and 12 s: a step at least 1.5 times what the organism takes to compute on this runner, measured: 0.18 s at 10 copies of the four stacked, 2.4 s at 100).
- The organism is built before the window opens (`organism.ready`); the window opens the moment it is built (`organism.go`), so the cluster and the organism start on one clock however long the build takes. If the organism falls behind its own clock, `organism.json` records by how much (`behind_s`).
- Machine floor two, every machine usable (the founder's rules above), open-loop load, peak six generators.
- Declared before the run: 1,000 copies of the whole tower (656,000 modelled muscles) and of the four stacked (1.2 million) need about 6 and 11 GB of memory beside the cluster, past what one GitHub runner holds. They are run; if a runner cannot hold them, the job's failure is reported as that, never as a result.

Reported in `SIX_KUBE.md` by organism and size, every gauge paired against native with its 95% interval and read in words.

The run dimension of the model grid (1, 10, 100, 1,000 runs) is a count of seeds; on the real cluster each repetition is a real paired run of an hour or more, so the real cluster carries the repetitions above, and the model grid carries 1 to 1,000 runs.

### *The two organisms too big for a GitHub runner: a larger rented machine (written before their run, 2026-10-05)*

1,000 copies of the whole tower and of the four stacked run on a larger rented machine, the same test otherwise: workflow `big-organism` rents one Azure Standard_D8s_v4 (8 vCPU, 32 GiB) per repetition, runs `scripts/kind_paired.sh` on it (native against native with Omni-Compass on top, fresh six-worker kind cluster per arm, 2,880 measured seconds, the organism at 1,000 copies with the real cluster inside), copies every file back and deletes the machine whatever happens. Three paired repetitions per organism, two machines at a time. Its results join `SIX_KUBE.md` at 1,000 copies; the same cells from GitHub runners count only if the runner held them.

*Result on Omni v3 (*`results/live/V3_BIG_ORGANISM.md`*, both organisms, written 9 October after the second collect).* The machines were rented one at a time through `big-organism-detached` with a 10,800 s window an arm (the 2,880 s window of the v1 runs put the compass arm off the clock; the clock rule and the longer window are


© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

declared in the robustness and v3 notes). The four stacked: three repetitions over 29 hours (collected by run 37754612102), p95 2,882 → 150 ms, time over the line 51% → 0, failed requests 0.14% → 0, machines 6 in both arms, both arms 22 s behind the window. The whole tower: three repetitions over 25 hours (started by run 37756284680, collected by run 37945866426), p95 6,690 → 160 ms, p99 9,068 → 187 ms, time over the line 76% → 0, failed requests 4.8% → 0, machines 6 in both arms, both arms 14 s behind the window. Every cluster row that moved is clear of zero with three pairs; the organisms' own model rows are shown beside them and never enter the index; the rounding-level model work loss reads worse by rule in both. Six repetitions, six on the clock.

### *The burst bill test on Azure: two native arms lost to the harness, and the fix (2026-10-05)*

Repetitions 2 and 3 of the burst bill test (run 37216328055) each lost their native arm (Kubernetes alone) at the same moment: the load step from one generator to six. Native's two work machines were full, answers timed out, and the API server stopped answering for minutes. The capture script ran with stop-on-first-error, so one unanswered reading ended the recording, and the load schedule stopped at its next unanswered scale; the arm failed its checks and was marked invalid. The Omni-Compass arms of the same repetitions ran every step. Those two arms are not counted.

That is a fault of the harness, and it threw away exactly the moments a cluster struggles. Fixed: a reading the API server does not answer is logged (`capture.csv.errors`) and skipped, and the recording goes on; a scale the API server does not answer is tried again, then logged, and the schedule goes on (`fleet/capture/kube_capture.sh`, `scripts/kind_bench.sh`, `tools/run_kil.py`). Repetitions 4 and 5 started on the earlier code. When the run ends, three more repetitions of the same test run on the fixed code, and the report counts every valid pair and names every invalid arm with its cause.

## Amendment 6: a pinned gauge is not a steady demand (written before its run, 2026-10-05)

The wandering test (run 37235231963, ten pairs) gave native + Omni-Compass p95 -54%, time over the line -38%, failed requests -14%, energy -0.3% and CPU -2%, each proven, and 1.5 more pod starts a run (+36%, proven). The founder holds that result back until the cause is fixed and the test is run again.

The cause, read from the run's own audit (repetition 4, 12 pod starts against native's 7): at 972 s the load eased from five generators to four, rule 3 found the demand steady, and the target was raised (50 to 190: fewer pods); the load then climbed back to eight and the pods were started again. The demand was not steady. The autoscaler stood at its replica cap, every pod as busy as it could be, so the reading (utilisation times pods) could not rise however much the load did: a pinned gauge reads flat.

Rule 5 (amends rule 3): a decision at which the autoscaler stands at its replica cap marks the window pinned; a window that touched the cap is not steady, so no raise of the target, and no return of a raised cap, until one whole autoscaler window after the cap was last touched (`omni_controller/controller.py _steady`, `self.pinned`; tested in `tests/test_compass_controller.py`). Nothing else changes.

The rerun: the same wandering test (ten pairs, native and compass, `load_steps` 1 2 3 2 3 4 5 6 5 4 5 6 7 8 7 6 5 4 3 4 3 2 1 2 1, 2,700 s, floor two, every machine usable). Reported in full, every gauge read in words; the first run is kept in its raw files and named beside the rerun.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

## Amendment 7: coasting, off the gas but still in gear (written before its run, 2026-10-05)

The founder's picture: a car in drive. At idle it creeps, ready (the floor of two machines). On the gas it speeds up at once (more pods, machines woken, no boot). Off the gas it coasts down gradually toward idle; it does not drop into neutral. The brake is for stopping (the reset hands everything back at once).

Rule 6: a raise of the HPA target (toward fewer pods) moves by at most `--coast-step` points of utilisation in one autoscaler window (default 25, `COAST_STEP`; 0 turns it off). The first wandering run raised the target 50 to 190 in one move; under rule 6 the same raise takes several calm windows (50, 75, 100, ...). A demand that comes back part way finds the pods still running and is met at once by a lower target, with no pod started again. A lower target (more pods) is never limited: the gas is always immediate. Rules 5 and 6 together: never ease off while the gauge is pinned, and then ease off a step at a time (`omni_controller/controller.py`; tested in `tests/test_convey.py`).

The rerun with rule 5 alone (run 37255770249) was stopped before it finished, to run the test once with both rules; none of it is reported. The rerun: the wandering test as written in amendment 6.

## Amendment 8: cruise and the emergency brake, and the batch test (written before its run, 2026-10-05)

The founder's driving modes: off (native alone), watching (Omni-Compass reads, writes nothing), autopilot (gas, brake, idle), and two more for work that comes as a pile:

- **Rule 7, cruise:** work waiting for a place (pods pending) two decisions in a row puts every machine in service (`--max-nodes`), and they stay in service, without second-guessing, until the line has been empty and nothing is scaling up for two decisions (`--cruise-after`, default 2; 0 turns it off).
- **Rule 8, the emergency brake:** nothing waiting, no response-time breach, nothing scaling up, and the sensed services' demand at or under 0.05 of one pod's request (`--brake-demand`): the machines go straight to the floor (two) in one move. Every check of the release gate still holds (every sense live, the last command landed, the machines left at or under the utilisation target) except one machine per decision. Never below the floor.
- Tested through the live controller: `tests/test_cruise_brake.py` (in `verify.py`).

**The batch test:** a queue of jobs on the six-worker kind cluster (`WORKLOAD=batch`, `deploy/kind/batch-jobs.yaml`): one Kubernetes Job of 240 pods, 60 at a time (more than the workers hold, so work waits for a place), each hashing 3,000 MB (real CPU work, identical in every arm), opened with the measured window (1,500 s); the service's load generator stands at zero. Ten pairs, native and compass, order rotated. Reported, paired with the 95% interval and read in words: how long the queue took to finish, the worker machines in service after it finished, machines in service and energy over the whole window, CPU with Omni-Compass's own, pods started. Cruise must not slow the queue; the brake must show in the machines held after it.

## The frozen engine (rules 1-8, commit 353903683009): the fault and fairness results (2026-10-05)

Both tests ran again with nothing changed but the engine: ten pairs each, native and omni, order rotated.

- The fault test (GitHub run 37262797634, `results/live/FAULTS.md`): better and proven on p95 (−60.1%), mean response (−34.5%), time over the line (−22.6%) and pending pods (−76.5%). Nothing came out worse beyond the noise. Recovery after the fault, paired means: machine down 42 s against 70 s, a blind probe 54 s against 66 s, a runaway pod 98 s against 105 s, a spike 266 s against 267 s.


© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- The fairness test (GitHub run 37262799317, `results/live/FAIRNESS.md`): better and proven on mean response (−15.0%). Nothing came out worse beyond the noise, the neighbour's app included.

These replace the earlier engine's fault and fairness sections of `results/live/AMENDMENT_3_RUNS.md` in the Omni index. That file stays as first measured.

The Azure burst bill test and the big organisms, running them (2026-10-05): Azure no longer offers this subscription Standard_D2s_v5 in eastus, so AKS workers are Standard_D2s_v4 (2 vCPU, 8 GiB, the same list price, USD 0.096 an hour). Both arms use the same size; the steady-load result of record (`results/live/AKS_BILL.md`) stays as measured on D2s_v5. The big-organism machine failed while seven kind nodes joined (kubelet-start). Fresh Ubuntu ships too few inotify watchers for that; the rented machine now raises them as kind's own documentation advises. Neither change touches the controller.

## Amendment 9: idle read from the autoscaler's floor, and the batch test again (written before its run, 2026-10-05)

The batch test on the frozen engine (GitHub run 37262795697) showed rule 8 late. The queue emptied 664 s into the window, but the emergency brake fired at 1,574 s. The reason: a served service is never at zero CPU. The response probe alone keeps one pod at about 10% of its request, which reads as demand 0.10, above the brake's 0.05. Rule 8 now also counts a service as idle when its autoscaler stands at its least pods (`minReplicas`), wants no more, and runs at no more than half its target utilisation. Every other condition of rule 8 holds as before: nothing waiting, no breach, nothing scaling up, the release gate, every sense live, the last command landed, and the machines left carrying what runs now under the utilisation target. Test: `tests/test_cruise_brake.py` (it brakes at the idle floor, and it does not brake above the floor or busy at it).

Effect on what was already measured: none. The steady, fault, fairness, wandering and all-four runs on the frozen engine had the service at one pod with nothing waiting in 0 of 5,102 readings, so the amended rule could not have fired in any of them.

Three of the ten batch pairs stopped, all in the omni arm. The API server answered 500 under the batch load (60 jobs asking for 30 cores on a 4-core runner), and the recording script ended on an unanswered `kubectl top nodes`. That is a harness fault: a reading the API server does not answer is now logged and skipped, and the end-of-window reads are tried again. The batch test runs again, ten pairs, on this engine, with the design unchanged.

## Amendment 10: two rows read for what they measure (written after the frozen-engine runs were seen, 2026-10-05)

Disclosed: this is a change in how two rows are read, made after the steady, wandering and all-four results were seen. Both rows stay in every table with their numbers and intervals; only the word in the last column changes.

- **Pending pods.** The row counts pods pending in each 15 s snapshot. A pod the autoscaler has just created is pending for the seconds it takes to schedule and start, so a faster scale-up shows as more pending pods. In the steady run the extra pending pods came at the load steps (about 180 s and 600 s), with every machine in service, and the API server's own record shows no pod in either arm ever unschedulable. The row is now shown, not judged. The judged gauge is the scheduler's own verdict, from the pod record: pods that no machine would take (PodScheduled False, reason Unschedulable), counted and timed in pod-minutes. In the five frozen-engine runs it is 0 in every arm.
- **Energy per core-hour.** It is energy divided by CPU used. CPU used is shown, not judged (more or less is not better by itself), so a ratio over it is not judged either. Energy itself stays judged.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

Nothing else changes. With these readings, no row of the steady, wandering, all-four, fault or fairness results is worse than native beyond the noise.

## Amendment 11: the burst bill test sized to what the service can serve, and rounding read as the same (2026-10-05)

Disclosed: written after three repetitions of the burst test at `1 6 1 8 1 6` were seen. In them about 41% of requests failed in both arms. The service's own autoscaler sat pinned at its cap of 10 pods, and 6 and 8 load generators (36 and 48 requests a second) ask more than 10 pods can serve, whatever the machines. A test where neither arm can do the work cannot show a difference. The burst test runs at `1 3 1 4 1 3`: the peak is one step above the steady test's peak of 3, and inside what the capped service can serve. Everything else is as preregistered: five repetitions, 1,800 measured seconds, Azure's autoscaler underneath, the bill metered every 15 s, native against compass. The three repetitions at the old steps stay in the archive (`results/live/raw/run-37294579764/`) and are reported as the reason for this amendment, not as a result.

In every table a paired change under one part in a million of the value reads "same", the number still shown (`SAME_REL`, `docs/MECHANISM_OF_ACTION.md` 9.5).

## Amendment 12: cruise steps back, and the organism's start file (written before the batch rerun, 2026-10-05)

The batch rerun on amendment 9 (GitHub run 37275371557, ten valid pairs) gave machines -16.9%, -27.2% once the queue was done, and energy -11.6% on the standby model. The queue finished 10 s later (584 s against 594 s, +1.8%). Omni held every machine in service throughout, so the time went elsewhere: every five seconds the floor step read every pod (240 batch pods), every node, node metrics and every HPA, on the one 4-core runner the queue was using. In cruise every machine is already in service, so those reads can change no action. From this amendment, while cruising the floor step makes no read and no write, and the service's target stays at the operator's own (`docs/MECHANISM_OF_ACTION.md` 9.3; `tests/test_compass_controller.py`). Cruise never engaged in the steady, wandering, all-four, fault or fairness runs, so this changes nothing there. The batch test runs again, ten pairs, design unchanged.

The organism with the real cluster inside waits for a start file. One repetition of six-kube (the four stacked, 1 copy, repetition 2) read that file in the instant it existed but was still empty, and stopped. The file is now written whole or not at all, and read only once it holds a number. That repetition is reported as stopped, never counted.

Also found and fixed on 2026-10-05, from the compass rename: each modelled muscle's compass was made fresh at every decision (`docs/MECHANISM_OF_ACTION.md` 9.1). That touched the modelled organism rows of the runs started after the rename (six-kube 37280090832, big organism 37287311224); their cluster rows are unaffected. The organism runs go again on the fixed code.

The batch test on amendment 12 (GitHub run 37344765837, ten pairs, `results/live/BATCH.md`): machines in service -20.7%, -32.4% once the queue was done, energy -14.3% on the standby model, mean response -11.7%, all proven. The queue finished +1.1% later, inside the noise (it was +1.8% and proven before cruise stepped back). No pod was left without a machine. Nothing came out worse beyond the noise.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## The fleet that can show one machine: Azure at 11 and 15 workers (written before its run, 2026-10-07)

The Azure runs so far used a work pool of 1 to 4 machines. One machine is a quarter to a half of that fleet, and the app's replica ceiling (10 pods of 200m, about one worker's worth) kept Azure's autoscaler at about 1.9 workers in both arms, so nothing under a quarter of the fleet could show. The steady run on v1 (5 pairs) read even on every gauge. That is a statement about the lever, not about the law. This run makes the lever big enough to see one machine.

- **Fleet.** The subscription allows 200 vCPUs in eastus but 10 vCPUs in each machine family (every family's request to raise it was refused through the API on 2026-10-07: the subscription's terms), so one family gives at most 5 machines. The fleet is therefore several work pools, one 2-vCPU machine family each, each capped at what its family allows: `worker_pools` = `Standard_D2s_v4:4,Standard_D2as_v4:5,Standard_D2_v4:5,Standard_D2a_v4:5,Standard_D2ds_v4:5,Standard_D2s_v3:5,Standard_E2s_v4:5,Standard_E2as_v4:5`, 39 workers (the DSv4 family also carries the system machine, hence 4). Every pool carries the label `omni-role=work`, the worker selector in both arms; the first pool keeps one machine, the others may scale to zero; every pool starts full and Azure's own autoscaler trims, as before. One machine is then 2.6% of the fleet. Mixed machine families are how real cloud fleets run; the bill is each pool's machine-hours at its own list price (`pool_prices`), Azure's own count every 15 s. The same autoscaler profile, a fresh cluster per arm. (Written first for 11 then 15 workers of one family; the family allowance made that impossible and this replaces it before any such run.)
- **Ceiling.** `hpa_max` = 9 × workers = 351 at 39 workers: nine 200m pods fit one 2-vCPU worker, so the fleet can fill. The same ceiling in every arm; the restore checks expect it back untouched.
- **Load.** The load generator's replica steps scale with the fleet, the same in every arm (one load replica drove about five pods at 50% on the 4-worker runs). Steady: `18 35 53 18 35 18` over 900 s at 39 workers (the peak asks for about 265 pods, thirty workers' worth). Burst: `11 35 11 53 11 35` over 1,800 s (the 4-worker burst was `1 3 1 4 1 3`). A fleet of another size scales the steps by workers/39, rounded.
- **Everything else as preregistered**: native (Azure's autoscaler alone) against omni (Omni-Compass on top of it, rule 4 on the node pool, the HPA target inside its range, handed back at 90% of the window), 5 paired repetitions, order rotated, the bill Azure's own machine count every 15 s at list price, the reading by `tools/live_reps.py`'s paired interval, every gauge reported. Engine: Omni v3 first (the newest), v1 after if the credits allow.
- **What it can say.** Better, clear of the noise, in bill or response time: Omni has value on the managed service when the fleet is big enough to see a machine. Even again: Omni's value there is nil at this size too, and that is the reading. Worse: our own wiring is suspected first, found, fixed, and the run is made again.
- **Cost.** About $8 for the steady set and $15 for the burst set at 15 workers, at list price.
- **Amendment (2026-10-07 07:18 UTC, before the counted run).** The first dispatch at 39 workers was refused by Azure on every repetition: the DASv4 family had 2 vCPUs left, not 10, because the rented machine running the four stacked at 1,000 copies (`big-organism-detached`, a Standard_D8as_v4) holds 8 of that family's 10 in the same subscription. The run was cancelled (no arm ran) and dispatched again at **35 workers**: `Standard_D2as_v4:1`, the other seven pools as above; `hpa_max` 315; the steps scaled by 35/39 as this section says, steady `16 31 48 16 31 16`, burst `10 31 10 48 10 31`. One machine is 2.9% of this fleet. Nothing else changes. The 39-worker fleet stands for the runs made after the detached machine is deleted.
- **Amendment 2 (2026-10-07 09:40 UTC, before the counted run).** The 35-worker dispatch was refused too, on three repetitions: the DAv4 family (`Standard_D2a_v4`) has **no allowance at all** in this subscription

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

("remaining 0" is a limit of 0, not a holder), and the survey (`big-organism-detached`, mode survey, now listing every family's allowance and Azure's list prices) shows the same for the ESv4 and EAv4 families (`Standard_E2s_v4`, `Standard_E2as_v4`). Three of the eight pools could never be built. The fleet is rebuilt from families the survey shows allowed (10 vCPUs each) and offered in eastus: `Standard_D2s_v4:4, Standard_D2as_v4:1, Standard_D2_v4:5, Standard_D2ds_v4:5, Standard_D2d_v4:5, Standard_D2s_v3:5, Standard_D2as_v7:5, Standard_D2ads_v7:5, Standard_D2als_v7:5, Standard_D2alds_v7:5`: **45 workers** of ten families (one machine 2.2% of the fleet); `hpa_max` 405; the steps scaled by 45/39, steady `21 40 61 21 40 21`, burst `13 40 13 61 13 40`. List prices from Azure's own price API (Linux, pay as you go, eastus, USD a machine-hour): D2s_v4 0.096, D2as_v4 0.096, D2_v4 0.096, D2ds_v4 0.113, D2d_v4 0.113, D2s_v3 0.096, D2as_v7 0.0908, D2ads_v7 0.114, D2als_v7 0.0804, D2alds_v7 0.0952 (the 0.086 used for D2as_v4 in the refused dispatches was wrong; no bill was ever made with it). `scripts/aks_paired.sh` now checks every pool's size and family allowance before anything is built, so a refusal costs a minute, not a repetition. Nothing else changes.

- **Amendment 3 (2026-10-07 16:21 UTC, before the counted run).** The pre-flight of the 45-worker dispatch (run 37602951929) found `Standard_D2s_v3` not offered to this subscription in eastus and stopped every repetition in a minute, nothing built, nothing billed; the nine other pools passed. That pool is dropped: **40 workers** of nine families (one machine 2.5% of the fleet), `hpa_max` 360, the steps scaled by 40/39, steady `18 36 54 18 36 18`, burst `11 36 11 54 11 36`, the pools' prices as in amendment 2. Nothing else changes.
- **Amendment 4 (2026-10-07 20:25 UTC, before the counted run).** Two dispatches of the 40-worker fleet (runs 37651333302 and 37676237815) were refused by Azure itself on every repetition for four hours: "creating a new cluster is unavailable at this time in region eastus" (`AKSCapacityHeavyUsage`, Azure's own capacity for new clusters, not the subscription's allowance; the pre-flight passed each time, no arm ran, nothing was billed). Microsoft's remedy is another region, other cluster settings, or a retry; this subscription has more than 10 cores only in eastus. The clusters are therefore made on the **standard control-plane tier** (`tier` input, `AKS_TIER`) instead of the free tier, about $0.10 an hour a cluster, which is not in the bill (the bill counts worker machines only, in both arms alike) and changes nothing the workers or the autoscaler do. If the standard tier is refused too, the run waits for Azure. Nothing else changes.
- **Amendment 5 (2026-10-08 03:55 UTC, before the counted run).** The standard-tier dispatch (run 37703460186) was refused by Azure's own cluster capacity in eastus on all five repetitions too, over three hours (23:39 to 02:48 UTC), the same `AKSCapacityHeavyUsage` on each: 22 refusals in eleven hours across four dispatches, the pre-flight passing each time, no arm run, nothing billed but the resource groups' minutes. The fleet is dispatched again unchanged; it runs when the region admits a cluster. The alternative, a second region, needs the subscription's regional vCPU allowance raised there, which only the account holder can request in the portal. Nothing else changes.
- **Amendment 6 (2026-10-08 05:00 UTC, before the counted run).** The fifth dispatch (run 37724667075) was refused on its first repetition after a forty-minute wait, the same `AKSCapacityHeavyUsage` in eastus, the twenty-third refusal; the pre-flight passed, no arm ran. We cancelled the run ourselves at that point to weigh a second region, and the answer is in the 6 October survey already on record (`big-organism-detached`, mode survey): eastus allows this subscription 32 vCPUs and the v4 machine sizes, and every other region allows 10 vCPUs in all, which is five two-core workers, not a fleet of forty. No other region can take this fleet until the account holder raises its allowance there. The fleet is therefore dispatched a sixth time in eastus, unchanged in every input, and runs when the region admits a cluster. The cancellation and the reason are recorded here so the count of dispatches reads right. Nothing else changes.
- **Amendment 7 (2026-10-08 08:30 UTC, before the counted run).** The sixth dispatch (run 37730249970) was refused on all five repetitions too, the same `AKSCapacityHeavyUsage` in eastus, over three and a half hours


© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

(05:01 to 08:26 UTC): 29 refusals in sixteen hours across six dispatches, the pre-flight passing each time, no arm run, nothing billed. The fleet is dispatched a seventh time, unchanged; nothing else changes. Each refused repetition costs GitHub runner minutes only.

- **Amendment 8 (2026-10-08 11:50 UTC, before the counted run).** The seventh dispatch (run 37750047768, 08:28 to 11:33 UTC) ran no arm either, but not for one reason. Repetitions 1 to 3 were refused as before (`AKSCapacityHeavyUsage` in eastus, the pre-flight passing each time): 32 refusals across seven dispatches. Repetitions 4 and 5 never reached Azure's cluster service: the pre-flight itself found only 2 of the 10 vCPUs free in two of the nine families (`standardDSv4Family`, `StandardDasv7Family`), and the fleet is dispatched only when every family has room, so it stopped with nothing built and nothing billed. An inventory of the subscription (`.github/workflows/azure-inventory.yml`, run 37773586107, 11:58 UTC) answered it: the subscription holds exactly four machines, all ours and all running, and no cluster, scale set or group left behind by any earlier repetition. The holders are the three machines of the 24-hour robustness run (`docs/ROBUSTNESS_PREREGISTRATION.md`, scenario 2b): the detached start job, which tries the size it is given and falls back through the eight-core sizes it may rent when the allowance refuses it (the four-core size asked for was refused because the tower's machine held its family's allowance), rented `Standard_D8as_v7`, `Standard_D8s_v7` and `Standard_D8s_v4`, eight vCPUs in each of the Dasv7, Dsv7 and DSv4 families, where the allowance is 10, and the fleet's pre-flight (which asks every one of its nine families for 10 free vCPUs) found 2. The tower's `Standard_D8as_v4` holds 8 of the DASv4 family's 10 the same way. Nothing is deleted. The eighth dispatch, unchanged in every input, follows the collect that deletes the 24-hour machines (each machine runs two arms of 86,400 s one after the other, so about 2026-10-10 12:00 UTC, and by their 60-hour limit at the latest); until then the fleet cannot pass its own pre-flight, and that is the pre-flight doing its job.

*Evaluation and simulation use only. Copyright (c) 2026 The Omni-Compass LLC. Commercial use, commercialization or monetization of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. All rights reserved. See* `LICENSE` *and* `NOTICE` *at the root of this repository. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 50

# The Evidence Ledger

Every material statement in the technical package gets exactly one primary evidence class. A statement never borrows a stronger class from the one printed beside it.

| Class | Meaning | Layer |
|---|---|---|
| **T** | formally proved theorem, under stated assumptions | L1 mathematical mechanism |
| **V** | finite computational verification: exact over a declared finite set, never a theorem beyond it | L1 |
| **S** | simulation / model result | L2 modelled systems |
| **L** | live observation of external software (a real Kubernetes API, a real process) | L3 external software |
| **P** | independent physical measurement (a meter Omni does not read or control) | L4 physical plant |
| **O** | open: not established | — |

No layer stands in for another: a theorem is not a physical validation, a Monte Carlo result is not a theorem, and a model's energy is not a meter's.

Mechanism identity for every row: `results/MECHANISM_IDENTITY.json`. Canonical engine `symmetric_verified`, mechanism id `29d9808dfb8f…`; the printed configuration `printed_eight_line`, id `cd333dc166fb…`, is a named alternative (Option A of the directive: one canonical, one alternative embodiment; no equivalence is claimed).

# Layer 1: the mechanism

| Class | Statement | Where |
|---|---|---|
| T | Unsaturated, the U-channel error obeys de/dt = −KP e; V = e²/2 has dV/dt = −KP e² < 0 for e ≠ 0 (inside Omega_unsat only). | `docs/TRACKING_THEOREM.md` Thm 1 |
| T | With the U drift bounded by F_bar < u_max, V decreases wherever e ≠ 0, saturated or not; e never changes sign; saturated stretches end in bounded time (continuous time). | Thm 2 |
| T | On the declared parameter box, F_bar = 16.86 < 25, so Thm 2 holds over the whole box in continuous time. | Thm 2 |
| T | The sampled controller (u held through each RK4 step) satisfies e_(k+1) = 0.88 e_k + d_k, hence abs(e_k) <= 0.88^k abs(e_0) + epsilon_h (1 − 0.88^k)/0.12. | Thm 3 |
| V | epsilon_h = 0.00378 over every micro step of the 500 frozen fixtures (nearest target); ultimate bound 0.0315 < BASIN_TOL 0.10. Wrong target: 0.0243, bound 0.20 (wider than the basin). | `results/TRACKING_BOUNDS.json` |
| V | The discrete execution dithers inside that band: abs(e) grew on 27 061 of 100 000 micro steps; e changed sign 747 times. The continuous monotone decay is not inherited. | same |
| V | Saturated micro steps: 0 of 100 000 (nearest target), 5 of 100 000 (wrong target). | same |
| T | The admissible box A (E interval, abs(U − sigma) <= abs(U0 − sigma), S between S0 and S_plus) is forward invariant in continuous time. | Thm 4 |
| V | Discrete invariance of A: 0 failures over 100 000 micro steps of the frozen fixtures, both targets. | `results/TRACKING_BOUNDS.json` |
| O | Discrete invariance and epsilon_h proved over the whole box (interval arithmetic). | — |
| O | Global stability of the forced six-state system. | `docs/FORMAL_STATUS.md` |
| V | 500 / 500 frozen fixtures satisfy CONVEY-5 and CERT-10 (dwell predicates, not external certification). | `results/core_evidence.json` |
| V | Frozen 500-fixture comparison: symmetric_verified final error 7.48e-5, integrated abs(u) 0.878, peak 13.93; printed_eight_line 5.13e-6, 11.79, 18.87. Both executable; not interchangeable. | `results/MECHANISM_IDENTITY.json` |
| V | Python and C++ produce identical results (engine, governor, shield, HPA law, conveyance, GPU rules); twins sealed. | `verify.py`, `results/SEAL.json` |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

# Layer 2: models and simulations

| Class | Statement | Where |
|---|---|---|
| S | Synthetic stack: energy for Omni over the Kubernetes reference 168.9 → 121.7 kWh per run (documented-behaviour reference, not upstream controllers). | `docs/CLAIMS_REGISTER.md` C8 |
| S | Fleet harness, recorded PlanetLab shapes: −40.6% energy vs HPA+CA, −21.7% vs Karpenter-lite (omni_fleet). | C17 |
| S | Single GPU physics model: the engine alone would save 10.6–13.6% work per kJ on card A but breaks the p95 guardrail by 15–28%; with the frozen guards, about +1 to +5% inside it. | `results/gpu/sim/FINDINGS.md` |
| S | Node exchange (CPU and GPU on one budget): +1.4 to +5.7% work against the separate budgets, never over budget. | `results/hardware/NODE_EXCHANGE_*.json` |
| S | Six organisms (345, 262, 282, 337, the four stacked 1,226, the whole tower 656), the compass law on every muscle against each organism's own controllers, 1,000 paired runs at 1× and 10× size: work per energy +0.20% to +0.30%, energy −0.21% to −0.32%; every knob handed back. | `results/scale/GRID.md` |
| S | GPU card model, the card's firmware alone against the firmware with Omni on top (amendment 8: the verdict, 2% allowance), geometric means over 10 seeds and 10 fresh seeds: AI token generation, energy −3.25% / −3.72%, median +0.55% / +0.70%, p95 +0.29% / +0.26%; compute-bound, energy −0.70% / −0.48%, median +1.56% / +1.47%. A fixed 105 W power cap alone against the cap with Omni on top: energy −2.09% / −2.37% (AI token generation). | `results/sim/gpu_two_wire/RESULT.md` |
| S | Realm harness round 3 (every realm carries the shared spine), preregistered, seeds 3000-3009: the whole 656-muscle tower native against one governor on top, work per energy +0.1% (+0.1 to +0.1), violations +0.5 pp, SUPERIOR WITHIN GUARDRAILS. Realms: Energy +0.2% with +1.9 pp violations (tradeoff); Compute 0.0% with +2.1 pp (not established); Distribution −0.1% (worse); Physics −0.7% (worse). Rounds 1 and 2 kept, superseded. | `results/realms/REALMS.md` |
| S | Stacked organism (round 4, seeds 4000-4009): the four realm organisms on one clock, 1,226 muscles with every duplicate; stacked native equals the four realms alone on every seed. One governor over the stack: energy −0.14%, work per energy +0.02%, violations +1.4 pp, ENERGY IMPROVEMENT WITH SERVICE TRADEOFF; four separate governors about the same (+0.04%); one governor against four separate: −0.02% (WORSE, by a hair). | `results/realms/stack/STACK.md` |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## Layer 3: live external software

| Class | Statement | Where |
|---|---|---|
| P | First real-GPU counted run, NVIDIA A10 (Lambda), 10 paired repetitions, the card's own meter: work per energy +3.6% (+2.7 to +4.5, proven), GPU energy −3.5%, same requests, none lost; wire check 7 of 7, every write read back, every arm restored. | `results/gpu/run-20261002T082232Z/GPU_REPS.md` |
| L | Set 25 (commit `6fa7a97`), real Kubernetes, 10 paired repetitions: machines in service −32.3%, p95 −57.3%, 0 failed requests, total CPU including Omni's own −0.6% (not proven). | `results/live/LIVE_REPS_25.md` |
| L | Set 26 (commit `e7f920d`), real Kubernetes, 10 paired repetitions, three arms: the allocation law machines −35.8%, p95 −55.4%; **the compass law in the live controller machines −17.2%, p95 −64.8%**, 0 failed requests, label by the preregistered rule *better on machines within the band*. | `results/live/LIVE_REPS_26.md` |
| L | Set 27 (commit `d46c959`), real Kubernetes, 10 paired repetitions, three arms: the allocation law machines −36.6%, p95 −53.1%; **the compass law aligned with the GPU governor machines −15.9%, p95 −65.5%, p99 −72.6%**, 0 failed requests, label by the preregistered rule *better on machines within the band*. | `results/live/LIVE_REPS_27.md` |
| L | Set 24 (2026-10-02, commit `c908054`), real Kubernetes (kind), 10 paired repetitions: machines in service −31.6% (proven), p95 response time −60.1% (proven), p99 −64.1% (proven), HPA replicas −38.6% (proven), 0 failed requests on both; total CPU including Omni's own −1.8% (not proven); energy with every machine powered −0.3% (declared model). | `results/live/LIVE_REPS_24.md` |
| L | Set 23 (set 22 repeated on the current code, 2026-10-02), real Kubernetes (kind), 10 paired repetitions, equal work: p95 response time −62% (proven), replicas −37% (proven), pods started −64% (proven), 0 failed requests; total CPU with Omni's own −1.0% and modelled energy −0.2% (no difference). | `results/live/LIVE_REPS_23.md` |
| L | Set 22, real Kubernetes (kind), 10 paired repetitions, equal work (fixed-rate load): p95 response time −61% (proven), replicas −23% (proven), pending pod-minutes −91% (proven), 0 failed requests. | `results/live/LIVE_REPS_22.md` |
| L | Set 21, real Kubernetes (kind), 10 paired repetitions: p95 response time −37% (proven), replicas −12% (proven), 0 failed requests. | `results/live/LIVE_REPS_21.md` |
| L | Omni patched a real Kubernetes API in place (no restart); the reset restored every setting in every run; watch mode wrote nothing. | same, `results/live/` |
| S | Set 21 energy is a declared model, not a meter: +1.8% worse with parked machines at idle power. | same (energy table) |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

## Layer 4: independent physical measurement

| Class | Statement | Where |
|---|---|---|
| O | Omni changes successful work per measured joule on a GPU. The bench is built with three receipts (governor, actuator, outcome), the three contrasts (observation, authority, total), and result labels by rule; it has **not** been run on a card. | `scripts/gpu_paired.sh`, `docs/GPU_PREREGISTRATION.md` |
| O | Production data-centre energy effect. | — |
| O | Whether the power limit is the right actuator for LLM serving. Published measurements (arXiv 2605.11999, H200) find memory-bound decode draws 137–300 W of 700 W, so no power cap binds; clock scaling is what saves energy there (arXiv 2501.08219, GreenLLM 2508.16449). The pinned bench workload is compute-bound, where the cap does bind: a result on it does not transfer to LLM decode. | external literature |
| O | Wall-plug (whole-machine) energy effect; CPU package and DRAM (RAPL) effect. | — |
| — | What the GPU bench will attribute: per write, joules and requests against native, and which rule decided it (engine, floor, gate, reflex, heat, speed lock). | `tools/gpu_reps.py` |

## Negative evidence, kept

Nothing here is deleted when a later result looks better.

| Class | Statement | Where |
|---|---|---|
| P | Same run: p95 response time **+58.5% worse** (510 to 809 ms), mean +48%; label by rule ENERGY IMPROVEMENT WITH SERVICE TRADEOFF. Cause: governor wiring (busy bursts served below the card's own clock); corrected in amendments 6-7, not yet re-run on a card. | `results/gpu/run-20261002T082232Z/GPU_REPS.md`, `docs/GPU_PREREGISTRATION.md` |
| S | Two-wire GPU card, the profiles of amendment 7 (removed in amendment 8): service +5.3% work per energy with the median +14.8% slower; batch p95 +7.0% (−4.8 to +20.3). The governor before amendment 6 gave p95 +32.9% and +37.1%. | `docs/GPU_PREREGISTRATION.md`, history in git |
| S | Six organisms, same runs: time over the service line **+0.19 to +0.27 pp worse in every cell**; band first is not held anywhere. | `results/scale/GRID.md` |
| S | Round 6 law (HPA target held at the operator's own, machine release margin 0.6), 20 paired seeds per organism, before the grid rerun: work per energy +0.086% to +0.201%, time over the line −0.010 to −0.031 pp (better than native in all six), work unchanged within its interval, every knob handed back. | `docs/REALMS_PREREGISTRATION.md` (round 6) |
| L | Set 21: modelled energy 1.8% **worse** with every machine powered (the only honest energy row on kind). | `results/live/LIVE_REPS_21.md` |
| S | Realm harness round 1 (superseded, kept): all five organisms **worse** (whole tower −0.1%). Its Omni layer did not follow the shipped controller (no contraction authority or SLO reflex, the stack law in place of the HPA, request traffic paused, a site budget under native draw). | `results/realms/round1/` |
| S | Realm harness round 2: the Physics / Robotics / Autonomous organism **worse** (−1.9%, violations +2.9 pp); the Compute organism's +2.5% costs +2.9 pp of service violations; 68 single muscles worse, mostly batch pacing and cooling setpoints under the live cooling law. | `results/realms/REALMS.md` |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Class | Statement | Where |
|---|---|---|
| L | Set 22, equal work: no CPU saving once Omni's own CPU is counted (service −7.6%, controller +0.070 cores, together −0.9%, not proven); modelled energy unchanged (−0.1%). | `results/live/LIVE_REPS_22.md` |
| L | Set 20: the "same work" reading was wrong; requests were +35%. | `results/live/LIVE_REPS_20.md` addendum |
| L | Sets 1–2: machine savings **withdrawn** — a broken probe had blinded the latency sense. | `results/live/LIVE_REPS_PROBE_DEFECT.md` |
| L | Set 3 with a working probe: Omni kept all 6 machines; no significant difference except more waiting pods. | `docs/HISTORY.md` |
| L | Set 21: pending pod-minutes +238% (not proven); pod starts +37% (not proven). | `results/live/LIVE_REPS_21.md` |
| S | GPU and batch: energy 1–2% **worse**, power and heat margins 1–4% worse than tight packers (AKS/Karpenter, CAST AI, Spot). | `docs/HISTORY.md` |
| S | Machine round trips 1.1 → 1.7 per run (**worse**) in 6 of 8 seed-target combinations. | C12 |
| S | Power-protect mode: backlog violations 2.5% → 6.7% (**worse**). | C12p |
| S | Park strategy on PlanetLab shapes: energy +1.3% vs HPA+CA, +33.6% vs Karpenter-lite (**worse**). | C17 |
| S | Typical response time about 20% slower than every platform in the web and four-cluster simulations. | `docs/HISTORY.md` |
| S | GPU model: the engine without its guards breaks the p95 guardrail. | `results/gpu/sim/FINDINGS.md` |
| S | The compass-stroke GPU variant was tried and not adopted. | same |
| S | Right-sizing against VPA: p95 +15%, memory (OOM) kills +531%. | `docs/history/BENCHMARK_REPORT.md` |
| — | Reported in the external master-build report (not reproducible from this repository): on fresh scenarios Karpenter+VPA sometimes used less modelled energy than Omni, while Omni had lower churn and fewer request-induced evictions. Kept here so it is not lost; to be re-run here before it is cited. | external |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 51

# The Claims Register

Every claim, its evidence status and the command that reproduces it. Simulation results use the synthetic stack model in omnicompass/stack_sim.py; they are not production measurements.

| ID | Claim | Status | Reproduce | | | |
|---|---|---|---|---|---|---|
| C1 | The six-state mechanism is implemented identically in the reference engine, Python and C++. | Proven (exact parity tests) | `python verify.py` | | | |
| C2 | The C++ governor reproduces the Python governor on recorded stack telemetry. | Proven (0 mismatches) | `python tests/test_cpp_governor_parity.py <oc_governor>` | | | |
| C2s | The C++ shield reproduces the Python shield (enforced actions, intervention counts and violations) on action sets recorded from every arm type; a C++ shield with invariant I1 weakened is rejected by the parity test. | Proven (0 mismatches; negative control) | `python verify.py` | | | |
| C2h | An independent C++ implementation of the HPA replica law (ratio rule, 10% tolerance, scale-up limit, 300 s scale-down window) reproduces the fleet harness HPA on every recorded step (hundreds of thousands of steps across Kubernetes and governor arms); a C++ HPA with 5% tolerance is rejected. | Proven (0 mismatches; negative control) | `python verify.py` | | | |
| C3 | The S channel cannot cross S- (forward invariance). | Proven (Proposition 1) | `Manual Chapter 3` | | | |

| ID | Claim | Status | Reproduce | | | |
|---|---|---|---|---|---|---|
| C4 | The governor completed 100,000,000 consecutive decisions (951 years at 5-minute intervals) with 0 failures and constant memory; failure-rate upper bound 3.0e-08 at 95% confidence. | Measured (soak test); reproduced by the full verifier | `python verify.py` | | | |
| C4b | Throughput-mode soak: 100,000,000 decisions, 0 failures; resident memory at 25/50/75/100% of each run is constant ([3852, 3852, 3852, 3852] kB power-protect, [3916, 3916, 3916, 3916] kB throughput). | Measured (soak test) | `python verify.py` | | | |
| C8p | Primary endpoint (energy, throughput vs reference target 0.7): seed 346410161: -47.20 kWh, p = 1.0e-04; seed 360555127: -47.69 kWh, p = 1.0e-04. Secondary metrics Holm-adjusted over 24 tests: 21 better, 2 worse, 1 not significant. | Measured in simulation | `python benchmarks/multiplicity.py` | | | |
| C5 | One governor decision costs 2.6 microseconds on one CPU core of the test machine (machine-dependent). | Measured | `cpp: oc_soak` | | | |
| C6 | Observe mode leaves the stack bit-identical: over Kubernetes, identical to Kubernetes alone (500/500 and 500/500); over the native model, identical to native (500/500 and 500/500). | Measured in simulation | `results/heldout_seed_*` | | | |
| C7 | Invariant violations per run (all arms scored identically): Kubernetes reference 9.56 / 9.44; power-protect 0.00 / 0.00; throughput 2.72 / 2.69, of which all are I4 (projected power), which throughput mode does not enforce; excluding I4: reference 6.91 / 6.80, throughput 0.00 / 0.00. | Measured in simulation | `results/heldout_seed_*` | | | |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| ID | Claim | Status | Reproduce | | | |
|---|---|---|---|---|---|---|
| C8 | Energy for Omni-Compass over Kubernetes (throughput mode, same power rules as the reference) versus the Kubernetes reference model (HPA target 0.7): 168.9 to 121.7 (better) / 169.2 to 121.5 (better) kWh per run; across HPA targets 0.5 to 0.8: better in 8, worse in 0, not significantly different in 0 of 8 seed-target combinations. | Measured in simulation (synthetic stack, documented-behaviour reference model) | `results/heldout_seed_*` | | | |
| C9 | Time healthy for Omni-Compass over Kubernetes (throughput mode) versus the reference model (target 0.7): 79.3% to 91.2% (better) / 78.5% to 91.4% (better); across targets: better in 8, worse in 0, not significantly different in 0 of 8 seed-target combinations. | Measured in simulation | `results/heldout_seed_*` | | | |
| C10 | Recovery time for Omni-Compass over Kubernetes (throughput mode) versus the reference model (target 0.7): 57.2 to 17.8 (better) / 58.4 to 15.2 (better) minutes; across targets: better in 8, worse in 0, not significantly different in 0 of 8 seed-target combinations. | Measured in simulation | `results/heldout_seed_*` | | | |
| C11 | Backlog violations for Omni-Compass over Kubernetes (throughput mode) versus the reference model (target 0.7): 2.5% to 1.6% (better) / 2.6% to 1.5% (better); across targets: better in 8, worse in 0, not significantly different in 0 of 8 seed-target combinations. | Measured in simulation | `results/heldout_seed_*` | | | |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| ID | Claim | Status | Reproduce | | | |
|---|---|---|---|---|---|---|
| C12 | Machine round trips (started and later stopped), throughput mode versus the reference (target 0.7): 1.1 to 1.7 (worse) / 1.1 to 1.8 (worse) per run; across targets: better in 2, worse in 6, not significantly different in 0 of 8 seed-target combinations. The reference removes a machine only after 10 minutes below 50% utilization and keeps post-event machines running; the governor releases them, which is the main source of its energy saving. | Measured in simulation | `results/heldout_seed_*` | | | |
| C12w | Scale direction reversals (back-and-forth wear), throughput mode versus the reference (target 0.7): 2.19 to 1.93 (better) / 2.22 to 1.92 (better) per run; across targets: better in 4, worse in 4, not significantly different in 0 of 8 seed-target combinations. Power-protect: 2.19 to 3.06 (worse) / 2.22 to 3.08 (worse). | Measured in simulation | `results/heldout_seed_*` | | | |
| C12e | Engine mechanism in the flagship (throughput): removing the equation (2) release gate changes reversals from 1.93 / 1.92 to 3.19 / 3.18 and energy from 121.7 / 121.5 to 120.2 / 119.9 kWh; removing engine evolution changes energy to 125.4 / 125.1 kWh and reversals to 0.00 / 0.00. | Measured in simulation | `results/heldout_seed_*` | | | |
| C12p | Power-protect mode (enforces the site power limit, which the reference does not): energy 168.9 to 132.2 (better) / 169.2 to 132.3 (better) kWh; power-limit violations 12.8% to 3.5% (better) / 13.8% to 3.7% (better); backlog violations 2.5% to 6.7% (worse) / 2.6% to 7.0% (worse). | Measured in simulation | `results/heldout_seed_*` | | | |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| ID | Claim | Status | Reproduce | | | |
|---|---|---|---|---|---|---|
| C12a | Human pages: the governor issues no page actions. Zero pages is a design property, not a measured performance result. | Design | `omnicompass/adapter.py` | | | |
| C12b | The reference model is not the upstream Kubernetes controllers: Cluster Autoscaler scheduling simulation, Karpenter, VPA, scheduling constraints and disruption budgets are not represented. | Stated limitation | `benchmarks/stack_benchmark.py (K8sReference)` | | | |
| C15 | 15-second control plane, Omni-Compass as node-pool and power authority in place of the Cluster Autoscaler (HPA retained), energy vs HPA+CA / vs HPA+Karpenter-lite: energy-first: web -34.5% / -16.9%, reversals 4.3; multi -34.3% / -16.6%, reversals 18.1; batch -17.2% / -9.7%, reversals 13.6; gpu -4.9% / -2.7%, reversals 5.1; gpu_always_on -4.7% / -3.2%, reversals 0.0 | balanced: web -33.2% / -15.3%, reversals 4.0; multi -33.0% / -15.0%, reversals 15.5; batch -12.4% / -4.5%, reversals 5.7; gpu -5.0% / -2.8%, reversals 4.2; gpu_always_on -4.7% / -3.2%, reversals 0.0 | wear-first: web -29.7% / -10.9%, reversals 3.6; multi -29.6% / -10.6%, reversals 14.5; batch -12.3% / -4.3%, reversals 4.9; gpu -4.5% / -2.3%, reversals 3.0; gpu_always_on -4.2% / -2.8%, reversals 0.0 | park: web -15.9% / +6.7%, reversals 0.0; multi -15.6% / +7.0%, reversals 0.0; batch -15.3% / -7.7%, reversals 0.0; gpu -4.9% / -2.7%, reversals 0.0; gpu_always_on -4.7% / -3.2%, reversals 0.0. Baseline reversals (CA / Karpenter): web 1.1 / 6.8; multi 5.5 / 25.4; batch 1.4 / 24.9; gpu 2.8 / 11.2; gpu_always_on 0.0 / 0.0. In gpu_always_on no arm powers a node off. | Measured in simulation (fleet harness, synthetic workloads, documented-behaviour execution layer) | `python -m fleet.benchmark --seeds 30 --seed-base 700000 --out out/` |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| ID | Claim | Status | Reproduce | | | |
|---|---|---|---|---|---|---|
| C17 | Recorded PlanetLab utilization shapes with a declared scale mapping (9 traces, 30 scenarios), governor as node-pool authority in place of the Cluster Autoscaler: omni_fleet: energy -40.6% vs HPA+CA (better), -21.7% vs Karpenter-lite (better); omni_fleet_balanced: energy -37.5% vs HPA+CA (better), -17.5% vs Karpenter-lite (better); omni_fleet_wear: energy -31.0% vs HPA+CA (better), -8.9% vs Karpenter-lite (better); omni_fleet_park: energy +1.3% vs HPA+CA (worse), +33.6% vs Karpenter-lite (worse). Time healthy and work completed are significantly lower by 0.08 points and 0.01%; observe mode identical in 30/30. | Measured in simulation driven by recorded traces (frozen laws, no retuning) | `python -m fleet.planetlab --dir fleet/traces/planetlab --scenarios 30 --seed-base 800000 --out out/` | | | |
| C16 | Live-cluster capture, capture replay and the PlanetLab vessel are tested on generated inputs in the real formats. No real capture or recorded trace has been run in the package. | Tested path; no real-data result | `python tests/test_fleet_realdata_paths.py` | | | |
| C19 | The live controller (omni_controller/) implements observe, target and nodepool modes with dry-run, audit log and a reset that restores HPA targets from annotations; tested against a fake kubectl only, never against a real cluster. | Tested path; no live result | `python tests/test_omni_controller.py` | | | |
| C18 | Savings projection: the energy-first reduction relative to HPA + Karpenter-lite, applied to declared fleet profiles (results/SAVINGS.csv), computed identically in Python and C++. A projection from simulation, not measured savings. | Projection | `python benchmarks/savings.py` | | | |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

| ID | Claim | Status | Reproduce | | | |
|---|---|---|---|---|---|---|
| C21 | End-to-end self-pilot (shipped controller, simulated cluster, real capture and scoring), default headroom 50%: energy per core-hour -7.7%, node-hours per core-hour -13.8%, pending-pod time not significantly different from HPA + Cluster Autoscaler; HPA shortfall minutes higher. Lower headroom saves more energy with more pending-pod time (manual Section 8.12a). | Measured in simulation | `python pilot/selfpilot.py` | | | |
| C20 | pilot/score.py scores a user's own captures (node-hours and energy per used core-hour, utilisation, pending-pod and HPA-shortfall minutes, bootstrap intervals); tested to detect a real gain, report no difference for identical clusters and detect a service regression. | Tested tool; no pilot result | `python tests/test_pilot_score.py` | | | |
| C13 | Decision components (autoscalers, power agents, paging, Terraform as controller) consume about 0.02% of fleet CPU; idle capacity is 92% of fleet CPU at 8% utilization. | Modeled from published figures and stated assumptions | `python benchmarks/fleet_overhead.py` | | | |
| C14 | Behaviour on production systems. | Not established; requires the pilot protocol | `docs/PILOT_PROTOCOL.md` | | | |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 52

# The Benchmark Report

> **Dated record, kept as written on 26 September 2026.** Where a figure here differs from `docs/STATE_OF_PLAY.md`, the State of Play governs. The GPU power-limit figure in this report (15% to 19% energy, under 1% slower) came from an earlier model of the card; later models with the service guards give +1.3% to +5.1% work per kJ with one wire (`results/gpu/sim/after`) and +8.6% with two wires at a higher p95 (`results/sim/gpu_two_wire/RESULT.md`). No real card had finished the bench on that date; the first card result (NVIDIA A10, 2026-10-02) is in `results/gpu/run-20261002T082232Z/GPU_REPS.md` and `docs/STATE_OF_PLAY.md`.

Benchmark report, 26 September 2026. Repository: Omni-Compass/The-Omni-Compass-Control-Core-Engine (private), branch main Every number below is produced by code in that repository and can be regenerated; section 21 gives the commands. Each result states whether it was **measured on a live Kubernetes control plane** or **computed in simulation**.

## 1. Summary

Omni-Compass is a single control engine that senses the whole compute stack and drives its actuators (its "muscles": replica counts, node pools, power caps and others) from one six-state dynamical model, with a safety shield before every action and a kill switch that hands control back. It was compared in three architectures:

- **A. Kubernetes alone.** Kubernetes' own controllers decide: Horizontal Pod Autoscaler (HPA) for replicas, Cluster Autoscaler for nodes; other managers act on their own proposals.
- **B. Kubernetes + Omni-Compass.** Kubernetes' controllers keep running; Omni-Compass governs on top of them (sets the HPA target, gates and sizes the node pool, caps power) as the single authority over their settings.
- **C. Omni-Compass direct.** Omni-Compass is the only decision-maker and actuates the muscles directly; the separate managers no longer decide.

**Main result (pre-registered, 1,000 held-out scenarios, simulation).** Against Kubernetes alone, B used -28% energy and C -23%; time healthy rose from 79% to 91% (B) and 90% (C); recovery time fell from 58 to 17 and 28 minutes; contradictory commands, pages and human interventions went to zero in both; safety-rule violations fell from 9.5 to 2.7 (B) and 0.0 (C). Of 28 gauges, B is significantly better on 20 and worse on 5; C is better on 16 and worse on 10.

**Live Kubernetes result (measured).** Two identical Kubernetes clusters (1 control plane + 6 workers) ran the same load at the same time, one without Omni-Compass and one with it. With Omni-Compass: worker nodes in service 6.0 to 3.2; utilisation of the workers in service 0.068 to 0.116. **Energy:** with the parked workers kept on standby, powered and ready (100 W each, the same as idle), energy was 219 vs 218 Wh (-0.46%): parking alone saves essentially nothing; energy per unit of work +8% (not significant; 872 to 944 Wh per core-hour, from minute averages). The -43% first reported for this run holds only if parked workers are powered off. Waiting pods and

HPA shortfall were not significantly different. The kill switch restored the original HPA target (50) and all 6 workers. **With every live muscle switched on (section 7.2) the application got slower: p95 response time 486 to 802 ms, energy per unit of work +46% (significant).** The causes were found and fixed across runs 2-4 (section 7.4): response time went from +65% to a tie at p95.

**Where Omni-Compass costs something.** In the pre-registered study both B and C keep more node-hours powered than Kubernetes alone and start and stop machines more often (more wear), and move the power cap more; C also lets more work wait in the queue and flips scale direction more often. In that study the energy saving comes from power capping and load shaping, not from switching machines off. On the live cluster the saving came from switching machines off. Other limits: the small-cluster release band (section 11); power and heat on the live cluster are modelled, not metered.

**Trade-off in one line:** B is the strongest all-round result in simulation (energy, health, recovery, queue, coordination and safety all better; wear and node-hours worse); C is the strongest on peak power, heat and safety (zero invariant violations) at the cost of queue length, wear and flip-flops. These are the gauges to tune next.

# 2. What Omni-Compass is

## *2.1 The engine*

The engine is a six-state ordinary differential equation system, state x = (E, U, I_U, S, B, B_dot): error E, coherence U, pressure I_U, stress S and a damped bath B. The shipped core (`omnicompass/core.py`) integrates it with fourth-order Runge-Kutta and holds the control input constant across the four stages:

```
(1) dE/dt   = -alpha_E E + beta_int + beta_ext + v_eff
(2) dU/dt   = mu U (1 - U^2) - (dE/dt)/E_max - lambda_U U + u,   |u| &lt;= 25
(3) dI_U/dt = (1 - U) - sigma_1 E - delta S - lambda_I I_U
(4) v_eff   = cos(omega_B t / 2) c tanh(lambda_0 + lambda_1 (U - 0.5) + lambda_2 S)
(5) Phi(S)  = alpha_s S^2/2 + beta_s S^3/4 - delta S
(6) dS/dt   = -dPhi/dS
(7) dB/dt = B_dot;  dB_dot/dt = gamma_c delta S - (omega_B/Q_B) B_dot - omega_B^2 B
(8) R_B[n]  = finite-difference audit of (7), never fed back
Controller: u = clip(-f_U(x,t) + 12 (sigma - U), -25, +25)
```

Telemetry (load, queue, power, heat, network, drift, staleness, security) is assimilated into the state each decision; an allocation law turns the state into a demand target rho* (the HPA target), a node change, and a power cap. Release of capacity is gated by equation (2): capacity is only released once the control push has converged.

## *2.2 The nervous system (muscles)*

Each muscle has five parts: afferent (pull: sense), the shared engine, efferent (push: act), reflex (the shield checks every push) and kill (hand the muscle back to its own controller). Status in this repository: **wired live on Kubernetes** (sense and push executed through kubectl, each with shield and kill): nodes, HPA target, power cap (in-place CPU limits, enforced by the kernel), security hold, deployment rollouts (pause, resume, undo), batch queue (admit held Jobs); **sensed**: heat (harness heat law on live power, or GPU temperature), network; **hardware connectors** (built and tested with fake hardware, off on CI machines): CPU power states (RAPL read, cpufreq ceiling) and GPU (nvidia-smi power and temperature read, power limit); **open** (registered, no plant yet): memory, storage, cooling, grid, training, inference, agent containment and the rest of the 52-muscle domain map (`docs/DOMAIN_MAP.md`). AI value alignment is explicitly not an Omni-Compass muscle. Code: `omni_controller/controller.py`, `omni_controller/muscles.py`, `omnicompass/nervous.py`.



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

### *2.3 The shield and the kill switch*

Before any action the shield (`omnicompass/shield.py`) enforces invariants I1 to I5: no expansion during a security block, node count within bounds, step limits, never below the capacity running and pending work needs, and the site power limit. The kill switch (a file or `OMNI_KILL=1`) restores every HPA target Omni-Compass changed from the recorded original, returns the node pool to its native size and drops to observe mode. Every decision and action is written to an append-only audit log.

### *2.4 Operating modes*

Observe (compute and log, write nothing), target (write HPA targets), nodepool (also size a node pool). Laws: power_protect (enforce the site power envelope), throughput (power envelope not enforced; capacity sized at full power) and fleet variants tuned on the 15-second fleet harness.

## 3. The three architectures, and how each study realises them

| Study | A. Kubernetes alone | B. Kubernetes + Omni-Compass | C. Omni-Compass direct | Live or simulated |
|---|---|---|---|---|
| Pre-registered held-out stack benchmark (2 x 500 scenarios) | `k8s_ref_70`: documented HPA law (target 0.7, 10% tolerance, 300 s stabilisation) and Cluster Autoscaler (scale-up on backlog, remove after 10 min under 50%), other managers act on their own proposals | `omni_k8s_throughput` : the same Kubernetes loops, Omni governs on top | `omni_direct`: Omni senses and actuates the stack directly | simulation |
| Control-plane replica (24 scenarios) | `hpa70_ca`: metrics-server, HPA and Cluster Autoscaler replicas at 15 s | `omni_target_gate_hpa70_ca`: Omni writes the HPA target and gates Cluster Autoscaler scale-down | `omni_throughput_full`: Omni writes the HPA target, owns node scale-down and power cap; the Cluster Autoscaler may only add nodes | simulation |
| PlanetLab-shaped demand, fleet plant (8 scenarios) | `k8s_hpa70_ca` | `omni_target`: Omni writes the HPA target | `omni_fleet`: Omni is the node-pool authority, Cluster Autoscaler off | simulation on recorded traces |
| Live kind cluster, side by side | native: HPA only, 6 workers always on, Omni not running | Omni on top: sets the HPA target and is the sole node-pool authority (cordon, drain, uncordon); Kubernetes' scheduler, kubelet and HPA still execute | not yet built live (section 16) | **live** |

## 4. Method and why the comparison is fair

- **Frozen before testing.** The allocation law, engine parameters, shield limits, modes, baselines and benchmark code were selected on development seeds (1000, 2000) and frozen, with SHA-256 hashes and program fingerprints in `results/PREREGISTRATION.json`, before any run on the held-out seeds 346410161 and 360555127. `verify.py` re-checks every hash.
- **Same scenarios for every arm.** Each scenario is run under every architecture; comparisons are paired scenario by scenario.
- **Observe-identity check.** Omni-Compass in observe mode must produce a trajectory bit-identical to the arm it observes, or the run is invalid. Held-out: 500 and 500 of 500 identical to the native stack, 500 and 500 of 500 identical to Kubernetes; control-plane replica: 24 of 24; PlanetLab: 8 of 8; live: 0 writes while observing.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

- **Statistics.** Differences are paired (Omni minus Kubernetes on the same scenario) with 95% bootstrap confidence intervals. In the held-out study a difference is called better or worse only if the interval excludes zero in the same direction on both independent seeds; 'better in N' counts scenarios. Live results use 2-minute blocks and a bootstrap over blocks.
- **Ablations** show the engine, not an accident of tuning, produces the result (section 5.3).
- **Independent implementations.** A C++ engine, governor, shield and HPA law are checked against the Python ones (section 8).

## 5. Results: pre-registered held-out benchmark (1,000 scenarios, simulation)

Mean over two held-out seeds x 500 scenarios; verdicts require both seeds to agree.

| Gauge | A. Kubernetes alone | B. Kubernetes + Omni-Compass | C. Omni-Compass direct | B vs A | C vs A |
|---|---|---|---|---|---|
| **ENERGY AND POWER** | | | | | |
| Energy per scenario (kWh) | 169.1 | 121.6 | 129.7 | -28% better (better in 991, worse in 9 of 1000) | -23% better (better in 1000, worse in 0 of 1000) |
| Peak power (kW) | 37.6 | 37.6 | 28.3 | +0.16% not significant (better in 457, worse in 533 of 1000) | -25% better (better in 1000, worse in 0 of 1000) |
| Node-hours | 135.8 | 140.5 | 152.4 | +3% worse (better in 464, worse in 536 of 1000) | +12% worse (better in 191, worse in 809 of 1000) |
| Idle node-hours (powered, doing nothing) | 46.29 | 51.73 | 62.03 | +12% worse (better in 446, worse in 554 of 1000) | +34% worse (better in 202, worse in 798 of 1000) |
| Share of time over the power limit | 0.133 | 0.067 | 0.018 | -50% better (better in 490, worse in 24 of 1000) | -87% better (better in 517, worse in 5 of 1000) |
| Power-cap travel (how much the cap moved) | 0.236 | 0.887 | 0.306 | +276% worse (better in 37, worse in 963 of 1000) | +30% worse (better in 243, worse in 757 of 1000) |
| **HEAT** | | | | | |
| Share of time over the heat limit | 0.084 | 0.026 | 0.002 | -69% better (better in 353, worse in 1 of 1000) | -98% better (better in 354, worse in 0 of 1000) |
| Thermal travel (how far temperature swung) | 0.754 | 0.632 | 0.576 | -16% better (better in 828, worse in 172 of 1000) | -24% better (better in 993, worse in 7 of 1000) |
| **SPEED AND BACKLOG** | | | | | |
| Mean queue (work waiting) | 384 | 217 | 933 | -43% better (better in 218, worse in 515 of 1000) | +143% worse (better in 43, worse in 506 of 1000) |
| 95th-percentile queue (worst moments) | 1403 | 950 | 2945 | -32% better (better in 247, worse in 190 of 1000) | +110% worse (better in 1, worse in 383 of 1000) |
| Share of time over the backlog limit | 0.026 | 0.015 | 0.080 | -42% better (better in 95, worse in 88 of 1000) | +207% worse (better in 1, worse in 267 of 1000) |
| **RELIABILITY AND RECOVERY** | | | | | |
| Availability | 0.9990 | 0.9999 | 0.9974 | +0.09% better (better in 110, worse in 127 of 1000) | -0.16% worse (better in 122, worse in 121 of 1000) |
| Share of time healthy | 0.789 | 0.913 | 0.901 | +16% better (better in 661, worse in 4 of 1000) | +14% better (better in 659, worse in 4 of 1000) |
| Share of incidents recovered | 0.918 | 0.996 | 0.972 | +8% better (better in 78, worse in 0 of 1000) | +6% better (better in 54, worse in 0 of 1000) |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Gauge | A. Kubernetes alone | B. Kubernetes + Omni-Compass | C. Omni-Compass direct | B vs A | C vs A |
|---|---|---|---|---|---|
| Recovery time (minutes) | 57.8 | 16.5 | 27.9 | -71% better (better in 602, worse in 0 of 1000) | -52% better (better in 580, worse in 0 of 1000) |
| Physical SLA breaches (power, heat, backlog) | 0.144 | 0.073 | 0.089 | -49% better (better in 495, worse in 25 of 1000) | -38% better (better in 449, worse in 46 of 1000) |
| All SLA breaches | 0.168 | 0.073 | 0.089 | -57% better (better in 655, worse in 3 of 1000) | -47% better (better in 611, worse in 37 of 1000) |
| **WEAR AND TEAR** | | | | | |
| Machines started | 4.8 | 6.6 | 9.4 | +37% | +95% |
| Machines stopped | 1.2 | 3.3 | 2.7 | +185% | +130% |
| Node start/stop events | 6.0 | 9.9 | 12.1 | +66% worse (better in 56, worse in 907 of 1000) | +102% worse (better in 199, worse in 782 of 1000) |
| Machine round trips (stopped then restarted) | 1.07 | 1.73 | 2.70 | +61% worse (better in 119, worse in 464 of 1000) | +152% worse (better in 26, worse in 866 of 1000) |
| Scale reversals (flip-flops) | 2.20 | 1.93 | 5.45 | -13% better (better in 474, worse in 199 of 1000) | +147% worse (better in 86, worse in 803 of 1000) |
| **CONTROL QUALITY AND SAFETY** | | | | | |
| Contradictory commands between managers | 9.46 | 0.00 | 0.00 | -100% better (better in 666, worse in 0 of 1000) | -100% better (better in 666, worse in 0 of 1000) |
| Safety-rule (invariant) violations | 9.50 | 2.71 | 0.00 | -72% better (better in 664, worse in 1 of 1000) | -100% better (better in 669, worse in 0 of 1000) |
| Invariant violations excluding power | 6.86 | 0.00 | 0.00 | -100% better (better in 666, worse in 0 of 1000) | -100% better (better in 666, worse in 0 of 1000) |
| Security violations | 0.889 | 0.000 | 0.000 | -100% better (better in 118, worse in 0 of 1000) | -100% better (better in 118, worse in 0 of 1000) |
| Pages to on-call | 0.584 | 0.000 | 0.000 | -100% better (better in 92, worse in 0 of 1000) | -100% better (better in 92, worse in 0 of 1000) |
| Human interventions required | 0.284 | 0.000 | 0.000 | -100% better (better in 92, worse in 0 of 1000) | -100% better (better in 92, worse in 0 of 1000) |

## *5.1 What the numbers say*

- **Energy:** B -28%, C -23% versus Kubernetes alone, although both keep slightly more node-hours powered. The saving comes from power capping and load shaping: time over the power limit halves (B) or nearly vanishes (C), and C cuts peak power by a quarter. Releasing idle machines, which the live cluster showed, is not what drives this study; idle node-hours are a gauge to tune.
- **Reliability:** time healthy and recovery improve sharply in both B and C; the share of incidents recovered rises.
- **Coordination:** separate managers issue contradictory commands (A); a single authority issues none (B, C). Pages and human interventions go to zero because Omni-Compass acts on the conditions that would have paged someone.
- **Heat:** C cuts time over the heat limit the most, because it governs power caps and load together.
- **Costs:** node start/stop events rise (B +66%, C +102%), machine round trips rise, the power cap moves more, and C's queue and scale reversals are higher than Kubernetes alone. The 24-scenario control-plane study (section 6) shows the opposite for wear (fewer machine stops), so wear depends on the plant and law and is a primary tuning target. These are reported, not tuned away.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

### *5.2 Architecture A has a hidden cost: the fragmented stack without Kubernetes*

For reference, the stack with each manager acting alone and no Kubernetes loops (`native`) used 131.9 kWh, was healthy 39% of the time, took 127 minutes to recover and produced 56.7 contradictory commands and 28.1 invariant violations per scenario. Kubernetes already improves on that; Omni-Compass improves on Kubernetes.

### *5.3 Ablations: is it the engine?*

| Comparison | Metric | Mean difference | 95% CI (seed 1) | 95% CI (seed 2) |
|---|---|---|---|---|
| Omni direct vs omni_direct_no_dynamics | energy_kwh | -1.954 | [-2.061, -1.775] | [-2.158, -1.834] |
| Omni direct vs omni_direct_no_dynamics | invariant_violations | +0.000 | [+0.000, +0.000] | [+0.000, +0.000] |
| Omni direct vs omni_direct_no_dynamics | mean_queue | +39.200 | [-4.742, +71.297] | [+7.307, +76.997] |
| Omni direct vs omni_direct_no_dynamics | recovery_minutes | +0.950 | [-0.090, +2.650] | [-0.410, +2.260] |
| Omni direct vs omni_direct_no_dynamics | time_healthy | -0.000 | [-0.004, +0.002] | [-0.002, +0.003] |
| Omni direct vs omni_direct_no_dynamics | violation_heat | -0.002 | [-0.004, -0.000] | [-0.005, -0.000] |
| Omni direct vs omni_direct_no_shield | energy_kwh | -0.759 | [-0.864, -0.642] | [-0.875, -0.660] |
| Omni direct vs omni_direct_no_shield | invariant_violations | -0.674 | [-0.750, -0.592] | [-0.760, -0.596] |
| Omni direct vs omni_direct_no_shield | mean_queue | +121.945 | [+90.814, +149.893] | [+100.054, +152.127] |
| Omni direct vs omni_direct_no_shield | recovery_minutes | +1.535 | [+0.560, +3.210] | [+0.480, +2.600] |
| Omni direct vs omni_direct_no_shield | time_healthy | +0.001 | [-0.001, +0.003] | [-0.000, +0.003] |
| Omni direct vs omni_direct_no_shield | violation_heat | -0.000 | [-0.000, +0.000] | [-0.001, +0.000] |

'no_dynamics' runs the same allocation law with equations (1) to (7) frozen; 'no_shield' removes the shield. The difference is full engine minus ablation: negative energy means the evolving dynamics save energy the frozen engine does not; the shield ablation shows what the shield prevents.

## 6. Results: Kubernetes control-plane replica (24 scenarios, simulation)

Replicas of the documented metrics-server, HPA (15 s) and Cluster Autoscaler (10 s) loops, with Karpenter-lite as a second Kubernetes reference; Omni-Compass decides every 300 s.

| Gauge | A. Kubernetes alone (HPA + Cluster Autoscaler) | Reference: HPA + Karpenter-lite | B. Kubernetes + Omni-Compass (HPA target + CA gate) | C. Omni-Compass authority (HPA target, nodes, power cap) | C vs A (95% CI, wins of 24) |
|---|---|---|---|---|---|
| Energy (kWh) | 168.8 | 161.7 | 155.6 | 122.3 | -28% better [-49.724, -43.292], 24 of 24 |
| Peak power (kW) | 38.7 | 39.5 | 40.8 | 32.8 | -15% better [-8.962, -2.584], 17 of 24 |
| Time over power limit | 0.050 | 0.050 | 0.048 | 0.015 | -70% better [-0.051, -0.019], 14 of 24 |
| Time over heat limit | 0.045 | 0.047 | 0.045 | 0.012 | -73% better [-0.048, -0.018], 13 of 24 |
| Time healthy | 0.930 | 0.937 | 0.931 | 0.983 | +6% better [+0.033, +0.073], 18 of 24 |
| Recovery time (min) | 1.857 | 1.982 | 3.430 | 0.430 | -77% better [-1.948, -0.906], 15 of 24 |
| Physical SLA breaches | 0.031 | 0.032 | 0.031 | 0.009 | -71% better [-0.032, -0.013], 14 of 24 |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Gauge | A. Kubernetes alone (HPA + Cluster Autoscaler) | Reference: HPA + Karpenter-lite | B. Kubernetes + Omni-Compass (HPA target + CA gate) | C. Omni-Compass authority (HPA target, nodes, power cap) | C vs A (95% CI, wins of 24) |
|---|---|---|---|---|---|
| Mean queue | 0.204 | 0.204 | 1.011 | 0.683 | +235% worse [+0.168, +0.836], 0 of 24 |
| Machines started | 21.8 | 24.9 | 25.5 | 20.0 | -8% better [-3.542, -0.250], 4 of 24 |
| Machines stopped | 13.2 | 17.0 | 24.1 | 3.833 | -71% better [-11.917, -7.083], 24 of 24 |
| Scale reversals | 1.333 | 1.917 | 2.000 | 1.000 | -25% better [-0.667, -0.083], 4 of 24 |
| Invariant violations | 0.000 | 0.000 | 0.000 | 0.000 | 0% not significant [+0.000, +0.000], 0 of 24 |
| Pages to on-call | 0.000 | 0.000 | 1.792 | 0.458 | new not significant [+0.000, +1.375], 0 of 24 |

# 7. Results: live Kubernetes (measured)

## *7.1 Side by side, two identical clusters at the same time*

kind clusters (real Kubernetes API server, scheduler, kubelet, HPA and metrics-server), 1 control plane + 6 workers each; the official php-apache HPA workload (target 50); the same stepped load (1, 2, 3, 1, 2, 1 load generators over 20 minutes after a 2-minute warm-up); capture every 15 s. Native: Omni-Compass not started. Omni: nodepool mode, HPA target, node pool, power sensing. Run 36209933218, 2026-09-26.

| Gauge | Kubernetes alone | Kubernetes + Omni-Compass | Change |
|---|---|---|---|
| Duration (min) | 20.10 | 20.13 | |
| Worker nodes in service, mean | 6.00 | 3.20 | -46.7% |
| Worker nodes in service, min | 6.00 | 3.00 | -50.0% |
| CPU used (cores), mean | 0.819 | 0.745 | -9.1% |
| CPU allocatable (cores), mean | 12.00 | 6.40 | -46.7% |
| Utilisation | 0.068 | 0.116 | +70.5% |
| Workers powered (in service + standby) | 6.00 | 6.00 | 0 |
| Node-hours in service | 2.01 | 1.07 | -46.6% |
| Power (W), mean, parked workers on standby at 100 W | 657 | 654 | -0.5% |
| Power (W), peak, parked workers on standby at 100 W | 691 | 687 | -0.6% |
| Energy (Wh), parked workers on standby at 100 W | 219 | 218 | -0.5% |
| Energy (Wh), parked workers in low-power standby at 50 W | 219 | 171 | -22% |
| Energy (Wh), parked workers in deep sleep at 10 W | 219 | 133 | -39% |
| Energy (Wh), parked workers powered off (as first reported) | 220 | 125 | -43% |
| Pending pods, pod-minutes | 0.567 | 0.267 | -52.9% |
| Pending pods, peak | 1 | 1 | 0 |
| HPA replicas, mean | 8.71 | 6.20 | -28.8% |
| HPA replicas, peak | 10 | 8 | -20.0% |
| HPA shortfall, minutes | 0.833 | 1.07 | +28.0% |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

| Metric (per unit of work, 2-minute blocks) | Kubernetes alone | Kubernetes + Omni-Compass | Change | 95% CI | Verdict |
|---|---|---|---|---|---|
| Wh per core-hour, standby at 100 W (2-minute blocks from minute averages) | 872 | 944 | +8.2% | [-150, +292] | not significant |
| Node-hours in service per core-hour | 7.8535 | 4.6976 | -40.2 % | [-5.0982, -1.1072] | fewer (not energy while parked nodes stay powered) |
| Utilisation | 0.0694 | 0.1224 | +76.3 % | [+0.0217, +0.0830] | better |
| Pending-pod minutes per hour | 1.7253 | 0.8333 | -51.7 % | [-3.4961, +1.6667] | not significant |
| HPA shortfall minutes per hour | 2.5458 | 3.3399 | +31.2 % | [-3.3277, +5.0066] | not significant |

Omni-Compass decisions: nodes per minute 5 4 3 then 3 for the remaining 17 minutes; HPA target 78-79% (Kubernetes alone: 50%). Node-pool resizes: 3 (three workers cordoned and drained, their pods rescheduled by Kubernetes). Kill switch: HPA target restored to 50, 6 of 6 workers back in service.

### *7.2 Live, side by side, every live muscle (run 36213152881 (2026-09-26 02:54-03:19 UTC))*

Same two-cluster setup; the Omni arm drives HPA target, node pool, power cap (in-place pod CPU limits), heat (harness law on live power), security hold, rollout guard. Real response times: HTTP requests to the app timed every 5 s on both clusters. **This run made the application slower.**

| Gauge | Kubernetes alone | Kubernetes + Omni-Compass | Change |
|---|---|---|---|
| Worker nodes in service, mean | 6.00 | 3.20 | -46.7% |
| Node-hours | 2.01 | 1.07 | -46.8% |
| Power (W), mean, standby counted | 664 | 643 | -3.2% |
| Power (W), peak | 720 | 664 | -7.7% |
| Energy (Wh), standby counted | 222 | 215 | -3.4% |
| CPU used (cores), mean | 0.908 | 0.593 | -34.6% |
| Utilisation | 0.076 | 0.093 | +22.6% |
| Energy per core-hour (Wh) | 731 | 1,083 | +48.1% worse |
| Pending pods, pod-minutes | 1.70 | 0 | -100% |
| HPA replicas, mean | 9.09 | 5.00 | -45.0% |
| HPA replicas, peak | 10 | 5 | -50.0% |
| Requests timed | 955 | 850 | -11.0% |
| Response time (ms), mean | 260 | 415 | +59.7% worse |
| Response time (ms), median | 230 | 320 | +39.2% worse |
| Response time (ms), 95th percentile | 486 | 802 | +64.8% worse |
| Response time (ms), 99th percentile | 675 | 1,113 | +64.7% worse |
| Failed requests (%) | 0 | 0 | 0 |

| Metric (2-minute blocks) | Kubernetes alone | + Omni-Compass | Change | 95% CI | Verdict |
|---|---|---|---|---|---|
| kWh per core-hour | 0.7596 | 1.1107 | +46.2% | [+0.1901, +0.5099] | worse |
| Node-hours per core-hour | 6.8941 | 5.6440 | -18.1% | [-2.7717, +0.4577] | not significant |
| Utilisation | 0.0766 | 0.0964 | +25.8% | [-0.0016, +0.0394] | not significant |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

Actions: hpa target writes 1, node pool resizes 3, power cap pod resizes 7, rollout actions 0, power cap per minute 0.695 every minute (the law floor), hpa target 77%, nodes 5 4 3 then 3. Kill switch: restored target 50; workers 6 of 6; pod CPU limits restored.

**Diagnosis:** The engine saw the cluster as idle (node utilisation about 7%) and had no response-time signal, so it capped the app pods at the law floor and packed replicas to a 77% target (5 instead of 10). The app pods were busy, so responses slowed. Fix wired: latency afferent (p95 over a declared 500 ms SLO enters as queue pressure) and a power-cap reflex (never below pod usage x 1.3); re-run pending.

### *7.3 Other live runs*

| Run | Setup | Result |
|---|---|---|
| live-kind, run 36205408388 | 1 node; baseline 10 min (Omni observing) then Omni target mode 10 min | 0 writes while observing; kill switch restored 50; pending-pod minutes -48.6% (significant, but the baseline phase included warm-up); energy per core-hour no significant difference (one node cannot be parked) |
| live-kind-full, run 36205869009 | 1 control plane + 3 workers | node pool never resized: 3 of 3 every minute at about 8% utilisation, because the fleet law needs more than 3 nodes of slack; kill switch restored target and workers |
| live-kind-full, run 36207925928 | 1 control plane + 6 workers; sequential baseline then full engine | nodes in service 6 to 5 to 4 to 3; node-hours in service per core-hour -38.2%, utilisation +63.7% (significant); the reported kWh per core-hour -35.8% counted parked workers as powered off; with parked workers on standby the saving largely disappears; kill switch restored target 50 and 6 of 6 workers |

### *7.4 Live runs after the first all-muscle run*

| Run | Change tested | Validity | Result |
|---|---|---|---|
| 36214629046 (2026-09-26 03:23-03:48 UTC), commit c182a63 | latency afferent (p95 over 500 ms SLO as queue pressure) and power-cap usage reflex | valid | p95 481 to 600 (+24.6% worse); p99 596 to 777 (+30.3% worse); energy 222 to 219 (-1.5%) |
| 36216647786 (2026-09-26 04:02-04:28 UTC), commit 052b705 | SLO reflex; eviction receipt fixed | INVALID as a test of the engine | p95 479 to 320 (-33.2%); p99 583 to 420 (-28.0%); energy 224 to 220 (-1.7%) |
| 36218637030 (2026-09-26 04:42-05:08 UTC), commit 47d0353 | controller fail-safe; benchmark prints controller log and rejects early stops | PARTIAL | p95 491 to 492 (+0.3% (tie)); p99 645 to 683 (+5.8% worse); energy 223 to 221 (-1.1%) |

Run 3 is recorded as invalid: a refused Kubernetes call stopped the controller after 3 of 20 decisions and the cluster held Omni's last settings with no engine running. The fix is a fail-safe: a failed decision is skipped; three in a row restore native settings and stop the controller. In run 4 the fail-safe did exactly that at minute 16, and the audit showed why: the least-privilege role allowed writing a pod's CPU limit but not reading it first, which kubectl does. In runs 3 and 4 no power cap was ever applied. Over run 4's 16 governed minutes node-hours per unit of work fell 24% and utilisation rose 36% (both significant); median response time -10%, p95 a tie, p99 +6% (not significant). The permission is fixed (with a receipt) and a run in which the fail-safe fires is now rejected; the re-run is in progress.

## 8. Engineering verification

`python verify.py` runs 89 checks (0 failures in the recorded log `results/VERIFY_LOG.txt`), including:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- reference engine SHA-256
- core parity vs reference engine and 500 fixtures
- C++ core vs 500 fixtures (max abs 3.55e-15)
- C++ governor vs Python governor (throughput)
- C++ shield vs Python shield (enforce and violations)
- negative control: parity test detects a C++ shield that no longer blocks rollouts during a security block
- independent C++ HPA replica law vs fleet harness HPA
- soak test (throughput), 100,000,000 decisions, no failures, memory flat after warm-up (2899 ns per decision, RSS checkpoints [3840, 3840, 3840, 3840] kB)
- held-out seed 346410161: observe mode identical to native
- live controller against a fake cluster: observe writes nothing, target bounded, kill restores, node pool bounded and dry-run safe

Also tested: the live controller against a fake cluster (observe writes nothing; targets bounded; kill restores from the HPA annotation after a restart; node pool bounded and dry-run safe), the full-engine options (parked and control-plane nodes excluded; the kill switch restores the node pool exactly once) and pilot scoring (detects a real gain, reports no gain on identical clusters, detects a service regression).

## 9. Physics and models

- **Stack plant power:** each node draws idle 0.38 kW plus 1.12 kW x utilisation; a power cap throttles delivered capacity.
- **Heat:** thermal state follows a first-order lag toward 0.34 + 0.62 x power stress (time constant about 7 steps); heat above 0.82 throttles capacity; 'over the heat limit' means thermal above 1.03.
- **Live kind cluster:** kind nodes have no power meter, so power is a declared model: 100 W idle + 150 W x CPU utilisation per worker in service, plus a standby power for each parked (cordoned and drained) worker. Standby defaults to the idle power (the worker stays powered and ready); lower values apply only to a declared sleep state, zero only to machines really powered off. The first live reports counted parked workers as zero; section 7.1 gives both. The same constants drive the governor's power sense and the energy score, so they cannot disagree. On real hardware this is replaced by metered power (RAPL, PDU or BMC).
- **Savings model** (`results/SAVINGS.csv`): a 1,000-node web cluster at 0.4 kW per node, PUE 1.4, $0.12/kWh and 0.4 kg CO2/kWh; reduction versus HPA 0.7 + Karpenter-lite of 14% to 20% (fleet plant) gives roughly 710 to 960 MWh, $85,000 to $115,000 and 280 to 380 t CO2 per year.
- **Engine overhead:** about 2.9 microseconds per decision in C++, memory flat over 100 million decisions.


*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## 10. What is proven, what is simulated, what is not claimed

| Claim | Status | Evidence |
|---|---|---|
| Engine equations are finite and deterministic; C++ equals Python | Proven | verify.py, 500 fixtures, max error 3.6e-15 |
| Observe mode changes nothing | Proven (simulation and live) | bit-identical trajectories; 0 writes live |
| Kill switch restores native control | Proven (simulation and live) | HPA target 50 and all workers restored live |
| Omni-Compass acts on a real Kubernetes control plane (HPA target, node pool) | Proven live | section 7 |
| Fewer nodes in service than Kubernetes with a fixed node pool, same load served | Measured live | section 7.1 |
| Lower energy on the live cluster | Not shown while parked nodes stay on standby; -22% to -43% only if parked nodes sleep or power off | section 7.1 |
| Better energy, health, recovery, coordination than Kubernetes (HPA + CA) | Pre-registered simulation | section 5 |
| Better than Karpenter-lite on energy | Simulation | sections 6, and PlanetLab fleet plant |
| Better than upstream Karpenter or Cluster Autoscaler binaries, live | Not yet tested | section 12 |
| Metered energy savings on physical servers | Not yet tested | power is modelled |
| GPU power-limit muscle saves 15-19% energy with <1% slower responses | Simulation calibrated to metered H100 data (MLPerf) | section 12 |
| CPU frequency muscle | Simulation (uncalibrated): about -3.5% energy, -67% heat | section 12 |
| Tuned laws remove wear and node-hour negatives (C-throughput) | Pre-registered amendment, new held-out data | section 13 |
| Cooling, grid, memory, storage and the other open muscles | Not claimed | no plant or connector yet |
| Makes AI models aligned or trustworthy | Not claimed | value alignment is outside Omni-Compass |

## 11. Limits and threats to validity

- Simulated studies use documented-behaviour replicas of Kubernetes controllers, not the upstream binaries; the Kubernetes reference omits Karpenter consolidation, VPA, scheduling constraints and disruption budgets.
- The live cluster is kind: nodes are containers on one CI machine; the two live arms ran on two machines at the same time, so machine-to-machine variation is part of the noise; each live arm is 20 minutes, one repetition.
- Live power and heat are modelled; parked kind workers are drained containers. If parked machines must stay on standby, parking reduces nodes in service but not energy; live energy savings then have to come from power caps, CPU power states and heat control, which are not yet wired live.
- The live native arm had no node autoscaler, so its node pool was always full; the fair live opponent is Karpenter or Cluster Autoscaler (section 16).
- The live significance for energy per core-hour with standby power is computed from minute averages (10 two-minute blocks), not from the 15-second capture.
- The fleet law releases a node only when the pool has more than three nodes of slack; a three-worker pool cannot scale down (observed live, reproduced offline). Small clusters need a pool-size-aware release band.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

- Architecture C was measured in simulation only; the live C (Kubernetes' controllers parked, Omni-Compass as the only brain) is not built yet.
- Service quality differences in the live runs (pending pods, HPA shortfall) are not statistically significant at this run length.

## 12. GPU and CPU muscles: device plant calibrated to metered hardware (simulation)

The GPU power-limit and CPU frequency muscles cannot be actuated on CI machines (no GPU; the hypervisor hides RAPL and cpufreq). Their connectors are built (section 2.2) and this plant shows what they do. **GPU calibration:** the performance-versus-power-limit exponent is fitted to MLPerf Inference v4.0 results for an NVIDIA DGX-H100 (8 x H100-SXM, 700 W TDP), MaxQ (power-limited with `nvidia-smi -pl`, the same command the Omni-Compass GPU connector sends) versus MaxP, system power metered by a Yokogawa WT333E; Apache 2.0. Fitted exponent 0.353 to 0.490 (median 0.449); all three are run. **CPU:** a standard first-order model (dynamic power ~ frequency cubed) against a schedutil-style governor; not calibrated to metered data yet. **S** is a fixed manual 70% cap, what an operator could do by hand. 24 scenarios, 8 load families; * = paired 95% interval excludes zero.

| Vessel | Gauge | A. Native | S. Fixed 70% cap | B. + Omni-Compass | C. Omni-Compass direct |
|---|---|---|---|---|---|
| cpu_web | Energy (kWh) | 115.169 | 103.342 (-10%*) | 110.899 (-4%*) | 111.238 (-3%*) |
| cpu_web | Energy per unit of work | 0.672 | 0.614 (-9%*) | 0.650 (-3%*) | 0.652 (-3%*) |
| cpu_web | 95th-pct response time (x baseline) | 1.000 | 6.577 (+558%*) | 1.040 (+4%*) | 1.040 (+4%*) |
| cpu_web | Minutes over service target | 0.000 | 70.458 (new*) | 0.167 (new) | 0.167 (new) |
| cpu_web | Minutes over heat limit | 26.708 | 0.000 (-100%*) | 8.875 (-67%*) | 8.958 (-66%*) |
| cpu_web | Peak power (kW) | 33.954 | 20.475 (-40%*) | 34.074 (+0.35%) | 34.074 (+0.35%) |
| cpu_web | Minutes near full power | 8.000 | 0.000 (-100%*) | 10.167 (+27%*) | 10.167 (+27%*) |
| gpu_mlperf_median | Energy (kWh) | 142.255 | 117.038 (-18%*) | 119.254 (-16%*) | 119.254 (-16%*) |
| gpu_mlperf_median | Energy per unit of work | 0.824 | 0.679 (-18%*) | 0.691 (-16%*) | 0.691 (-16%*) |
| gpu_mlperf_median | 95th-pct response time (x baseline) | 1.000 | 1.521 (+52%*) | 1.007 (+0.70%*) | 1.007 (+0.70%*) |
| gpu_mlperf_median | Minutes over service target | 0.000 | 7.125 (new*) | 0.042 (new) | 0.042 (new) |
| gpu_mlperf_median | Minutes over heat limit | 22.458 | 0.000 (-100%*) | 6.125 (-73%*) | 6.125 (-73%*) |
| gpu_mlperf_median | Peak power (kW) | 38.403 | 28.288 (-26%*) | 37.089 (-3%*) | 37.089 (-3%*) |
| gpu_mlperf_median | Minutes near full power | 6.500 | 0.000 (-100%*) | 6.208 (-4%) | 6.208 (-4%) |
| gpu_mlperf_least_favourable | Energy (kWh) | 142.255 | 118.430 (-17%*) | 120.770 (-15%*) | 120.770 (-15%*) |
| gpu_mlperf_least_favourable | Energy per unit of work | 0.824 | 0.688 (-17%*) | 0.699 (-15%*) | 0.699 (-15%*) |
| gpu_mlperf_least_favourable | 95th-pct response time (x baseline) | 1.000 | 1.628 (+63%*) | 1.008 (+0.80%*) | 1.008 (+0.80%*) |
| gpu_mlperf_least_favourable | Minutes over service target | 0.000 | 8.333 (new*) | 0.042 (new) | 0.042 (new) |
| gpu_mlperf_least_favourable | Minutes over heat limit | 22.458 | 0.000 (-100%*) | 6.708 (-70%*) | 6.708 (-70%*) |
| gpu_mlperf_least_favourable | Peak power (kW) | 38.403 | 28.345 (-26%*) | 37.532 (-2%*) | 37.532 (-2%*) |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Vessel | Gauge | A. Native | S. Fixed 70% cap | B. + Omni-Compass | C. Omni-Compass direct |
|---|---|---|---|---|---|
| gpu_mlperf_least_favourable | Minutes near full power | 6.500 | 0.000 (-100%*) | 6.417 (-1%) | 6.417 (-1%) |
| gpu_mlperf_most_favourable | Energy (kWh) | 142.255 | 113.878 (-20%*) | 115.645 (-19%*) | 115.645 (-19%*) |
| gpu_mlperf_most_favourable | Energy per unit of work | 0.824 | 0.661 (-20%*) | 0.670 (-19%*) | 0.670 (-19%*) |
| gpu_mlperf_most_favourable | 95th-pct response time (x baseline) | 1.000 | 1.300 (+30%*) | 1.006 (+0.55%*) | 1.006 (+0.55%*) |
| gpu_mlperf_most_favourable | Minutes over service target | 0.000 | 5.375 (new*) | 0.042 (new) | 0.042 (new) |
| gpu_mlperf_most_favourable | Minutes over heat limit | 22.458 | 0.000 (-100%*) | 5.250 (-77%*) | 5.250 (-77%*) |
| gpu_mlperf_most_favourable | Peak power (kW) | 38.403 | 28.133 (-27%*) | 36.174 (-6%*) | 36.174 (-6%*) |
| gpu_mlperf_most_favourable | Minutes near full power | 6.500 | 0.000 (-100%*) | 5.833 (-10%*) | 5.833 (-10%*) |

**Reading:** on GPUs Omni-Compass saves 15-19% energy across the measured range with response time 0.6-0.8% slower; a fixed cap saves about the same energy but slows responses 30-63% and misses the service target. On CPUs the native governor already tracks demand, so frequency control alone saves about 3.5%; its main effect is heat (-67%). CPU frequency control is below the 10% bar and is a physical ceiling of that muscle, not a tuning gap.

## 13. Amendment: tuned laws, frozen, then tested on new held-out data (simulation)

Two law variants were tuned on the development seeds only (1000, 2000; `tuning/SEARCH*.json`), frozen with SHA-256 hashes in `tuning/PREREGISTRATION_AMENDMENT_2026-09-26.json` and pushed (commit b79d1dd, 03:17 UTC) before a single run on new held-out seeds 731001 and 731002 (2 x 500 scenarios). The frozen engine files are unchanged; the original pre-registered result (section 5) stays the primary result. B-wear: node release held longer. C-throughput: Omni-Compass direct under the throughput law with a wider release band and engine-gated cap. + better, ! worse (both seeds agree), blank not significant.

| Gauge | A. Kubernetes alone | B_frozen | B_wear | C_frozen | C_throughput |
|---|---|---|---|---|---|
| energy_kwh | 170.001 | 122.670 (-28%) + | 123.552 (-27%) + | 130.042 (-24%) + | 123.353 (-27%) + |
| node_start_stop | 6.119 | 10.070 (+65%) ! | 8.210 (+34%) ! | 11.985 (+96%) ! | 3.065 (-50%) + |
| machine_round_trips | 1.031 | 1.724 (+67%) ! | 0.831 (-19%) + | 2.711 (+163%) ! | 0.863 (-16%) + |
| scale_reversals | 2.176 | 1.931 (-11%) + | 1.805 (-17%) + | 5.394 (+148%) ! | 1.838 (-16%) + |
| node_hours | 136.812 | 141.488 (+3%) ! | 144.688 (+6%) ! | 152.143 (+11%) ! | 122.531 (-10%) + |
| idle_node_hours | 46.662 | 52.152 (+12%) ! | 55.353 (+19%) ! | 61.218 (+31%) ! | 35.788 (-23%) + |
| mean_queue | 438.529 | 246.346 (-44%) + | 246.324 (-44%) + | 1103.704 (+152%) ! | 1029.828 (+135%) ! |
| p95_queue | 1462.260 | 1005.114 (-31%) + | 1005.044 (-31%) + | 3185.168 (+118%) ! | 2684.025 (+84%) ! |
| violation_backlog | 0.030 | 0.016 (-45%) + | 0.016 (-45%) + | 0.085 (+182%) ! | 0.064 (+115%) ! |
| recovery_minutes | 61.470 | 16.805 (-73%) + | 16.805 (-73%) + | 28.925 (-53%) + | 37.700 (-39%) + |
| power_cap_travel | 0.249 | 0.894 (+259%) ! | 0.893 (+259%) ! | 0.307 (+23%) ! | 0.697 (+180%) ! |
| invariant_violations | 9.625 | 2.748 (-71%) + | 2.747 (-71%) + | 0.000 (-100%) + | 0.000 (-100%) + |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Gauge | A. Kubernetes alone | B_frozen | B_wear | C_frozen | C_throughput |
|---|---|---|---|---|---|
| sla_violation_total | 0.171 | 0.076 (-55%) + | 0.076 (-55%) + | 0.094 (-45%) + | 0.145 (-15%) + |
| pages | 0.671 | 0.000 (-100%) + | 0.000 (-100%) + | 0.000 (-100%) + | 0.000 (-100%) + |
| time_healthy | 0.785 | 0.909 (+16%) + | 0.909 (+16%) + | 0.896 (+14%) + | 0.837 (+7%) + |
| availability | 0.999 | 1.000 (+0.09%) + | 1.000 (+0.09%) + | 0.997 (-0.23%) ! | 0.996 (-0.32%) ! |
| recovered | 0.907 | 0.997 (+10%) + | 0.997 (+10%) + | 0.971 (+7%) + | 0.963 (+6%) + |

Worse gauges: B_frozen 5, B_wear 4, C_frozen 10, C_throughput 5.

## 14. Response time: can one mode beat Kubernetes on every gauge? (simulation, development seeds)

The fleet-mode constants that drove the live runs were selected (before this work) by a rule that scored energy, finished work and backlog, but never waiting time. A response-time gauge was added to the fleet plant (fleet/sim_slo.py: M/M/c queueing delay per workload plus backlog drain; the frozen trace is reproduced exactly). Measured with it, fleet mode cut energy 31% on web services while p95 response time went from 132 ms to about 20 s. That is where the live slowdown came from: its HPA target floor (rho_min = 76.9%) packs pods too hot for latency-sensitive services.

A speed-first law (omnicompass/speed.py; the engine equations unchanged) separates the two jobs the frozen law gave one number: pods run at a latency-safe target while machines are sized to what the pods request. 400 configurations were searched against HPA 70% + Cluster Autoscaler with the rule that no gauge may be worse in any development scenario. None passed, for three measured reasons: (1) several gauges are already at their physical floor (100% work done, zero violations, batch p95 = pure service time), so a tie is the best any controller can do; (2) on the GPU vessel demand exceeds the hardware, so every arm is saturated; (3) energy, response time and machine churn trade two-of-three, because the autoscaler's idle slack is both where the energy is and what absorbs the 90-second boot delay. Examples, mean change vs Kubernetes: fleet mode energy -31% with far worse response time; speed #176 energy -1%, p99 -41%, churn 3.6x; speed #159 p99 -48%, churn -27%, energy +16%. Source: tuning/SPEED_FINDINGS_2026-09-26.md.

## 15. Named products: OpenShift, Google GKE, Azure AKS, IBM Turbonomic (simulation, development seeds)

Each product is emulated from its documented behaviour on the same fleet plant (not the vendors' binaries): OpenShift's documented ClusterAutoscaler example (threshold 0.4, unneeded 5 min, delay after add 10 min); GKE optimize-utilization (MostAllocated packing, more aggressive scale-down; declared as threshold 0.65, 2 min, because Google publishes no numbers); AKS node auto-provisioning (Karpenter, WhenEmptyOrUnderutilized, consolidateAfter 0 s); Turbonomic (container requests resized every 10 min to p99 per-pod usage, its default aggressiveness; nodes suspended toward 0.7 packing). Means over web services, 4 development seeds:

| Gauge (web) | Kubernetes (GKE balanced) | OpenShift | GKE optimize | AKS NAP | Turbonomic | Omni fleet mode | Omni speed #159 |
|---|---|---|---|---|---|---|---|
| energy (kWh) | 12.19 | 12.53 | 10.4 | 9.779 | 10.08 | 8.392 | 15.9 |
| p95 (ms) | 131.6 | 131.6 | 132.2 | 132.7 | 125.1 | 2.034e+04 | 112.3 |
| p99 (ms) | 3,095 | 2,815 | 3,947 | 4,295 | 4.94e+04 | 7.634e+04 | 129.8 |
| machine starts+stops | 10.5 | 8.5 | 14.75 | 19 | 11.5 | 13.25 | 6.75 |
| scale reversals | 1.5 | 0.75 | 3.5 | 6 | 1.5 | 4.25 | 1 |



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

Every product sits on the same energy / response-time / churn triangle; none wins all three. Karpenter-style consolidation (AKS NAP) saves about 20% energy with a worse p99 tail and about twice the machine churn; Turbonomic's p99 resizing interacts with the HPA and inflates the tail; OpenShift's documented settings sit close to upstream. Omni's speed mode leads on response time and churn at an energy cost; its fleet mode leads on energy at a large response-time cost.

## 16. The problem-map muscles: all nine built and tested on held-out data (simulation)

Each open row of the industry problem map (section 18) is now a muscle in omnilab/, with the strongest native tool as opponent, Omni-Compass, and Omni-Compass with its engine equations not evolved (to show what the equations themselves add). Constants were chosen on development seeds 1-8, every file was frozen by SHA-256 (results/muscles/PREREGISTRATION.json), then 30 held-out seeds were run once. Verdicts from a paired bootstrap 95% interval.

| Muscle (map row) | Strongest native opponent | Omni better | Omni worse | Tie / not significant |
|---|---|---|---|---|
| Right-sizing, HPA+VPA conflict (1, 2) | hpa_vpa | cpu core hours -3%; mem gib hours -11%; p99 ms -99%; replica reversals -84%; slo breach min -57% | p95 ms +15%; oom kills +531% | work done |
| Cold start (5) | keda | p95 ms -75%; p99 ms -58%; delayed 1s pct -71%; instance hours -8% | cold starts +736% | - |
| GPU packing (6) | binpack | energy kwh -2%; idle gpu hours powered -20% | wait mean min +16%; migrations 0 to 6.07 | wait p95 min, frag blocked min, jobs done |
| Training power swings (7) | floor_safe | energy overhead pct -9%; throughput loss pct -44%; swing 1s mw -6% | max ramp mw s +127% | ramp violation s |
| GPU failures and stragglers (8) | detect | goodput pct +10%; lost gpu hours -20%; restarts -29%; straggler node hours -99% | - | checkpoint overhead pct |
| Cooling (10) | reset | pue -2%; cooling mwh -13%; inlet violation min -73% | - | max inlet c |
| LLM inference, KV cache (12) | keda | ttft p95 s -18%; slo breach pct -58%; gpu hours -19%; preemptions -56% | - | ttft p50 s, tpot p95 ms |
| Runaway AI agents (13) | static | rogue overspend usd -96%; time to contain min -99%; peak subagents -59% | false stops +357%; honest work pct -7% | forbidden executed |
| Energy attribution in VMs (11) | ratio | attr error pct -37%; worst vm error pct -34% | - | total error pct |

Each entry is the plain change of Omni-Compass against the opponent (for example p99 -99% means the slow tail is 99% shorter; goodput +10% means 10% more useful training time). Worse entries are real costs, not rounding: right-sizing trades a slightly slower typical response (p95 +15%, both far under the 500 ms target) and a few more memory kills for a 99% shorter tail; cold start keeps fewer idle instances than KEDA's 5-minute cooldown, so more requests meet a cold start, but users wait far less because KEDA polls every 30 s; GPU packing powers idle GPUs off sooner and jobs wait a few seconds longer; containment throttles honest agents in their legitimate bursts (about 7% of their work) and stops about two honest agents a day, against a 96% cut in runaway spend and containment in minutes instead of hours. Against the other native arms (the default tools most teams run) the wins are larger; every comparison is in results/muscles/*_HELDOUT.json.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

**What the engine itself contributes.** Across the nine muscles the full engine and the engine-not-evolved arm differ by 1.0 percentage points on average per gauge. The mapping from engine state to action carries most of each result; the evolved dynamics mainly smooth. In power smoothing an arm where the bath equation (7) alone sets the site's draw, with no ramp rule, cut the steepest ramp 63% but did not hold the grid's limit: the human sets the boundary, the engine operates inside it.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## 17. Every negative, its cause and its status

| Negative | Where | Status | Cause | What would fix it |
|---|---|---|---|---|
| Node start/stop cycles (wear) | B, pre-registered | reduced (+65% to +34%), not removed | node release thresholds are fixed numbers | hybrid: Kubernetes serves the queue, Omni releases nodes with the C-throughput law (removed wear there: -50%) |
| Node start/stop, round trips, reversals | C, pre-registered | fixed in C-throughput (new held-out) | power-protect law released nodes too eagerly | adopted in C-throughput |
| Node-hours and idle node-hours | B and C, pre-registered | fixed in C-throughput (-10%, -23%); not in B | power capping trades lower watts for more servers on | hybrid as above; node-aware cap |
| Queue / backlog / availability -0.3% | C (both variants) | UNRESOLVED | not the replica law and not the sizing constants (both tested); likely the delayed observation or direct-mode proposals | trace one scenario step by step; candidate: feed the queue into the engine without the extra delay |
| Power-cap movement | B and C | inherent | moving the cap is how capping saves energy; freezing it removed most of the saving (tested: -28% to -2%/-7%) | none needed: electronic setting, no physical wear; reported |
| CPU frequency saving 3.5% | device plant | physical ceiling | the native Linux governor already follows demand | value is in heat (-67%) and in combining with power caps |
| GPU node on/off saving 3% | fleet plant | superseded | training nodes cannot be switched off | the GPU power-limit muscle (15-19%) |
| Live response time +65% (p95) and energy per unit of work +46% with every muscle | live kind, run 36213152881 | reduced to p95 +25% (run 2), tie in run 4 | no response-time afferent; cap held at the law floor; HPA target floor 76.9% | latency afferent, cap reflex, SLO reflex; speed-first law (section 14) |
| Fleet mode p95 20 s vs 132 ms on web services | fleet plant with response-time gauge | cause found; speed-first law built | fleet constants were selected without a response-time gauge; HPA target floor 76.9% | choose the mode per service: speed-first where latency matters |
| No mode better than Kubernetes on every gauge in every scenario | fleet plant, 400 configurations | not achievable as posed | gauges at physical floors; hardware-bound GPU vessel; energy / response time / churn trade two-of-three | anticipation (pre-adding machines before the daily rise) is the one untested mechanism that could break the trade |
| Controller stopped after one refused call; power cap never applied under least privilege | live runs 3 and 4 | fixed | one exception ended the loop; the role lacked get on pods/resize | fail-safe restore after 3 failures; permission and receipt added; runs with a fail-safe rejected |
| Right-sizing p95 +15%, more memory kills than VPA | omnilab rightsize, held-out | open, small | Omni sizes CPU closer to demand; VPA's 8-hour p90 memory keeps more slack | larger memory margin (costs memory-hours) |
| More cold starts than KEDA | omnilab coldstart, held-out | trade-off | shorter keep-alive than KEDA's 5-minute cooldown | longer keep-alive where cold starts matter more than instance-hours |
| GPU jobs wait +16% (seconds) vs Volcano binpack | omnilab gpupack, held-out | trade-off | idle GPUs powered off sooner | longer power-off delay |
| Honest agents throttled (-7% work) and ~2 false stops a day vs static caps | omnilab containment, held-out | trade-off | throttling on the engine's integrated need catches legitimate bursts | per-agent declared burst budgets; human approval before stop |
| Engine dynamics add little beyond the mapping | all nine muscles | reported | the mapping from state to action carries the effect | wire the engine's control effort (equation 2) directly as the actuator command and test it |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Negative | Where | Status | Cause | What would fix it |
|---|---|---|---|---|
| Live energy with parked servers on standby ~0% | live kind | open | parked servers still draw standby power | live power cap and CPU/GPU muscles; sleep states where hardware allows |
| Small clusters (<= 3 workers) never release | live kind | open | fleet law release band of 3 nodes | pool-size-aware release band (law change) |
| Pages in the 24-scenario replica | control-plane replica | open | longer queue triggers the page rule | same as queue |

## 18. The industry problem map: what Omni-Compass is aimed at

One engine; the vessel (the plant it sits on) is the only thing that changes. Industry figures are approximate, from the public sources named, and are context, not results of this report. Status: **live** = measured on a real Kubernetes control plane; **sim** = demonstrated in this repository's simulations; **open** = mapped, connector not built.

| Problem | Scale in the industry (approximate, source) | Best software today | Omni-Compass vessel and muscles | Gauge that shows it | Status |
|---|---|---|---|---|---|
| Data-centre electricity growth | about 415 TWh in 2024, about 1.5% of world electricity, projected near 945 TWh by 2030 (IEA, Energy and AI, 2025) | Karpenter, Cluster Autoscaler, CAST AI, Spot Ocean; Kepler for metering | compute vessel: nodes, HPA, power cap | energy, node-hours, idle node-hours | sim; live only where parked nodes can sleep or power off |
| Idle and over-provisioned capacity | Kubernetes clusters commonly run near 10-15% average CPU utilisation (CAST AI and Datadog industry reports); roughly a quarter to a third of cloud spend reported as waste (Flexera State of the Cloud) | VPA, Goldilocks, StormForge, Kubecost/OpenCost | compute vessel: nodes, HPA; requests and memory (omnilab/rightsize.py) | utilisation, node-hours per core-hour, core- and GiB-hours | live + sim |
| Controllers fighting each other | documented conflicts, e.g. HPA and VPA on the same CPU metric (Kubernetes documentation advises against it) | none: each tool decides alone | single authority over replicas and requests | contradictory commands, scale reversals, OOM kills | sim, held-out (section 16) |
| Outages and slow recovery | most significant outages cost over $100,000 (Uptime Institute annual outage analysis) | Argo Rollouts, Flagger, SRE runbooks, AIOps (Dynatrace, Datadog) | compute vessel + deployments (partial) | time healthy, recovery time, SLA breaches | sim |
| On-call load and alert fatigue | widely reported burnout in SRE surveys | PagerDuty, alert tuning | all muscles: act before the page | pages, human interventions | sim |
| Heat and cooling limits | cooling is a large share of facility energy; average PUE about 1.5 (Uptime Institute survey) | DCIM (Schneider EcoStruxure), DeepMind cooling AI (reported about 40% less cooling energy) | heat (sensed), cooling plant (omnilab/cooling.py) | PUE, cooling energy, inlet violations | sim, held-out (section 16) |
| Site power and grid-connection limits | multi-year waits for new grid connections are widely reported | Meta Dynamo power capping, Intel RAPL | power cap (wired), batteries and demand response (open) | peak power, time over power limit | sim |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Problem | Scale in the industry (approximate, source) | Best software today | Omni-Compass vessel and muscles | Gauge that shows it | Status |
|---|---|---|---|---|---|
| GPU energy and power limits | GPU fleets widely reported well below full utilisation; H100 TDP 700 W | NVIDIA DCGM and MIG, Run:ai, Kueue; manual MaxQ power limits | GPU power-limit muscle (hardware connector) | energy per unit of work, response time, heat | sim, calibrated to MLPerf metered H100 data: -15% to -19% energy, +0.6-0.8% response time |
| Batch deadlines and fair sharing | | Kueue, Volcano, Slurm | batch muscle (live: admit held jobs); GPU packing (omnilab/gpupack.py) | queue wait, fragmentation | live wired + sim, held-out |
| Hardware wear | power cycling and churn shorten component life | none as a governed objective | nodes: start/stop cycles and reversals | machines started and stopped, round trips | mixed: better in the 24-scenario study, worse in the held-out study; tuning target |
| Carbon reporting and reduction | regulatory disclosure is expanding | Google carbon-aware computing, Kepler | carbon-aware placement (open) | kWh and CO2 per unit of work | sim (modelled) |
| Runaway AI agents and spend | ~$10,000 overnight examples (Dark Reading) | per-tool quotas and permissions | agent containment muscle (omnilab/containment.py) | rogue spend, time to contain, false stops | sim, held-out (section 16) |

## 19. What comes next

- Live architecture C: park HPA, VPA, Cluster Autoscaler and Karpenter; Omni-Compass sets replicas, resources, placement, priorities and quotas directly; Kubernetes keeps execution and reflexes (restarts, rescheduling); the kill switch wakes the parked controllers.
- Live opponent at full strength: Karpenter (kwok provider) and Cluster Autoscaler in architecture A.
- 24 live scenarios (traffic, failures, power and heat limits, batch and AI, growth, mixed) with repetitions.
- More muscles two-way: CPU power states, memory, batch queues, network, security, then GPU and cooling on hardware.
- Metered power on physical machines.

## 20. Questions and answers

**Does Omni-Compass replace Kubernetes?** No. Kubernetes keeps running containers, placing pods, restarting failures and networking. Omni-Compass replaces the separate decision loops (how many replicas, how many nodes, what power) with one authority. In architecture C Kubernetes becomes one muscle.

**What happens if Omni-Compass crashes or is switched off?** The kill switch restores every setting it changed and returns control to Kubernetes' own controllers; this was exercised live and in simulation. A crashed controller writes nothing further.

**Can it make things worse?** Every action passes the shield first; it never goes below the capacity running and pending work needs, and never changes more than the step limit. In the held-out benchmark it had fewer safety violations than Kubernetes. Its real costs are listed in sections 1 and 5.1.

**How fast does it decide, and what does it cost to run?** One decision per 60 s on live Kubernetes (300 s in the replica), with a 15 s fast path that adds nodes for pending pods. The engine takes about 2.9 microseconds per decision.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

**Why does it save energy?** In the pre-registered stack study mainly by power capping and load shaping (fewer minutes over the power limit, lower peak), while keeping slightly more machines on. On the live cluster it took machines out of service (6 to 3 workers) and packed replicas more densely; that saves energy only if the parked machines sleep or power off. With parked machines on standby the live saving was about zero, so live savings must come from power caps and CPU power states.

**Does it slow applications down?** It can, in the energy-first fleet mode: its HPA target floor packs pods too hot for latency-sensitive services (section 14). Live, the first all-muscle run was slower (p95 +65%); after the fixes run 4 was a tie at p95 and 10% faster at the median. For latency-sensitive services the speed-first mode is the right setting.

**Can Omni-Compass control AI agents?** It controls what an agent can touch, spend and do, and how fast; not what the model thinks. In the containment muscle (section 16) it cut runaway spend 96% and contained runaways in minutes instead of hours, with a least-privilege identity that blocks forbidden actions outright; the cost is some throttling of honest agents' bursts.

**Does the engine hold everything in its basin by itself?** Not against an outside limit it is not told. The bath equation alone smoothed training power ramps 63% but did not keep them under the grid's limit; with the human-set limit as the boundary it held it with zero violations. The human sets the boundaries; the engine operates inside them.

**Is this tuned to the test?** Parameters were selected on development seeds and frozen with hashes before the held-out seeds were run; verify.py fails if any frozen file changes.

**How many scenarios and how certain?** 1,000 pre-registered held-out scenarios, 24 control-plane scenarios, PlanetLab traces and live runs; 95% bootstrap intervals, two independent seeds must agree.

**What is measured versus modelled?** Live: node counts, replicas, pods, CPU, the controller's actions and the kill switch. Modelled: power and heat everywhere, and everything in the simulated studies.

**Is the mathematics sound?** The core is a closed six-state system integrated with RK4; the C++ and Python implementations agree to 3.6e-15 on 500 reference trajectories; 100 million decisions ran without a non-finite value.

**Who owns it and how can it be used?** The Omni-Compass LLC. Free for evaluation, research and non-commercial use; commercial use requires a paid licence; protected by copyright and by patents and patent applications (see LICENSE and NOTICE).

**What is not claimed?** Superiority over upstream Karpenter or Cluster Autoscaler live, metered savings on physical hardware, GPU or facility control on real hardware, the vendor products' own binaries (they are emulated from documentation), and anything about AI value alignment.

**How do I check it myself?** Run the commands in section 21; the live runs are GitHub Actions workflows in the repository.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## 21. Reproduce

```
pip install -r requirements.txt
python verify.py                                        # all checks, hashes, parity, soak
python benchmarks/stack_benchmark.py ...                # held-out stack benchmark (see HARNESS.md)
python -m k8s_controlplane.benchmark --scenarios 24 --seed 424242
python -m fleet.planetlab --dir fleet/traces/planetlab --scenarios 8 --out /tmp/pl
GitHub Actions: benchmark (live side by side), live-kind-full, live-kind
python tools/full_report.py ... &amp;amp;&amp;amp; python pilot/bench_pdf.py docs/history/BENCHMARK_REPORT.md docs/histo
    ↪ ry/BENCHMARK_REPORT.pdf
```

## 22. Glossary

- **HPA**: Horizontal Pod Autoscaler: Kubernetes controller that sets replica counts from CPU utilisation versus a target.
- **Cluster Autoscaler, Karpenter**: Kubernetes add-ons that add and remove nodes.
- **Node, worker**: a machine (here a container in kind) that runs pods.
- **Cordon, drain**: mark a node unschedulable, then move its pods elsewhere.
- **kind**: Kubernetes in Docker: a real Kubernetes control plane whose nodes are containers.
- **Observe mode**: Omni-Compass computes and logs but writes nothing.
- **Invariant**: a safety rule the shield enforces before any action.
- **Paired bootstrap CI**: resampling the per-scenario differences to get a 95% interval for the mean difference.
- **Pre-registration**: freezing code and parameters, with hashes, before running the test data.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 53

# The Referee Report

> **Dated record, kept as written on 26 September 2026.** Where a figure here differs from `docs/STATE_OF_PLAY.md`, the State of Play governs. The results below are simulations of platforms (evidence class S), not measurements of those products.

Referee report, 26 September 2026. Repository Omni-Compass/The-Omni-Compass-Control-Core-Engine, branch main, commit 096956a. Every number is generated by `python tools/abc_report.py` from result files in the repository. Each result states how it was obtained: **measured** on a live Kubernetes control plane, **simulated** in this repository's plants, or **emulated** (a competitor reproduced from its public documentation, not its binary).

**Engine interpretation notice.** The Omni-Compass engine is the completed core conveyance mechanism; its success criterion is internal dual-basin conveyance under native ignition dynamics. Everything measured here (the Kubernetes adapters, the closure governor, the vendor comparisons) is wiring around the engine, which is downstream engineering and not part of the base law. A loss in a cell below is a property of that wiring on that plant, not evidence that the base engine is incomplete. The engine source is byte-locked by SHA-256 and was not edited for any result in this report.

## 1. The answer

Three architectures were measured against seven platforms: Kubernetes, Red Hat OpenShift, Google GKE, Azure AKS (node auto-provisioning, which is Karpenter and also stands for Amazon EKS Auto Mode), IBM Turbonomic, CAST AI and Spot Ocean.

- **A. The platform alone.** Its own autoscalers decide.
- **B. Omni-Compass on top of the platform.** The platform's controllers keep running; the six-state engine governs on top.
- **C. Omni-Compass alone.** The engine is the single authority; the platform's managers do not decide.

**B against A (held-out, 30 unseen scenarios per workload, settings frozen by SHA-256 first):** 363 of 364 gauge-by-platform-by-workload cells are equal or better; 1 is worse: batch jobs on CAST AI, scale reversals 7.7% worse. Batch jobs gain most: the slowest jobs finish much sooner on every platform (section 5.3). Where no intervention was free (web services, four clusters) Omni-Compass reproduces the platform exactly.

**C against the seven platforms at once (held-out):** 63 cells where some platform is better (section 6). Part of that gap is not closable by any controller: a perfect controller that knows the future cannot have both the tightest packer's machine-hours and the laziest autoscaler's machine churn (section 6.2).

**C with the manuscript's closure law and muscle tone (held-out, 30 unseen scenarios per workload, settings frozen first):** 41 of 364 cells where some platform is better. Against Kubernetes alone: web services energy +20%, slowest responses (p99) +60%, average response +20%; four clusters, one site energy +22%, slowest responses

(p99) -19%, average response +11%; batch jobs energy +6%, slowest responses (p99) +6%, average response -6%; GPU training energy +1%, slowest responses (p99) +0%, average response +0% (positive = better; section 6.4).

**Live Kubernetes (measured, run 36221702999, every muscle on, B architecture):** worker machines 6.00 to 3.19, energy -3.2% with parked machines on standby, response time p95 491 to 383 ms and p99 613 to 491 ms, 0 failed requests; one significant negative (energy per CPU core-hour, section 7).

## 2. The rule, and how a cell is judged

- A **cell** is one gauge, one competitor, one workload. There are 13 gauges: energy, machine-hours, response time (p95, p99, mean), time over the backlog, power and heat limits, machine starts+stops, scale reversals, pod changes, work completed and time healthy.
- Omni-Compass **loses** a cell when it is worse by more than 0.5% (0.5 events for counts) **and** the paired bootstrap 95% interval of the per-scenario difference excludes zero. Otherwise the cell is equal (identical, or not significantly different) or better.
- Settings are chosen on **development** scenarios only, frozen with SHA-256 hashes (`tuning/B_PREREGISTRATION*.json`, `results/muscles/PREREGISTRATION.json`), then run once on **held-out** scenarios never used for tuning.
- Scenario generator: `fleet/harness.py` (15-second resolution, 6 hours; boot delay, power model, thermal model, site power limit identical for every arm). Response time: M/M/c queueing per workload plus backlog drain (`fleet/sim_slo.py`).

## 3. The opponents, and how each is reproduced

| Platform | What decides | Reproduced as (parameters) |
|---|---|---|
| Kubernetes | HPA + Cluster Autoscaler | HPA 70%; remove a node after 10 min below 50% request use, not within 10 min of an add |
| OpenShift | ClusterAutoscaler resource | documented example: threshold 0.4, unneeded 5 min, delay after add 10 min |
| GKE (optimize-utilization) | MostAllocated packing, aggressive scale-down | declared: threshold 0.65, 2 min (Google publishes no numbers) |
| AKS NAP (= Karpenter, EKS Auto Mode) | consolidation WhenEmptyOrUnderutilized | consolidateAfter 0 s; remove when the rest hold all requests at 90% packing |
| IBM Turbonomic | market-based resize + suspend | requests resized every 10 min to per-pod p99 usage; nodes suspended toward 70% packing |
| CAST AI | Evictor bin-packing | every 60 s, drain a node older than 5 min when its pods fit elsewhere |
| Spot Ocean | headroom autoscaler | 5% automatic headroom kept; least-used node removed when pods fit elsewhere |

These are emulations of documented behaviour on the same plant, not the vendors' software; each vendor's full product has more features than its autoscaling core. Every platform runs its pods at the same HPA target (70%), so pod behaviour is identical and differences come from how machines are managed.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

# 4. What Omni-Compass is, mechanically

A six-state dynamical system, x = (E, U, I_U, S, B, B_dot): energy/excitation, order (a double-well with a healthy basin at U = +1), integrated unmet need, stress (a cubic potential), and a damped second-order bath. Equations (1)-(8) and every term's role are in `docs/MECHANISM_OF_ACTION.md`. Telemetry is blended into the state (weight 0.339) and the equations are integrated with RK4 each decision.

| Mode (Jacobian eigenvalue at the operating point) | Time constant (decisions) | Period (decisions) |
|---|---|---|
| -7.90+0.00i | 1.3 | |
| -3.10+0.00i | 3.2 | |
| -0.72+0.00i | 13.9 | |
| -0.20+0.98i | 50.0 | 64 |
| -0.20-0.98i | 50.0 | 64 |
| -0.17+0.00i | 59.8 | |

All modes are stable: the operating point (U = 0.976) sits inside the healthy basin. The equations and the observation blend each move the state about half of every step. Compute: 95 microseconds per decision in Python, about 3 in the C++ twin.

# 5. B: Omni-Compass on top of each platform, against the same platform alone (held-out)

Source `tuning/B_LEAGUE_HELDOUT2.json`. Omni-Compass keeps the platform running and intervenes only where the engine sees it will pay: it undoes a machine removal that is about to be reversed (while requests are rising or the engine has not converged), and adds early the machine the platform would add only after pods go pending. With both off it reproduces the platform exactly (checked on every platform and workload); every gain below is on top of that floor.

## *5.1 Web services*

Setting: pass-through. Every intervention tried here cost some gauge on some platform, so Omni-Compass leaves these decisions to the platform and the result is identical to the platform alone.

| Platform | Better (significant) | Worse beyond tolerance (LOSS) | Worse within 0.5% tolerance | Equal |
|---|---|---|---|---|
| Kubernetes | - | - | - | 13 of 13 |
| OpenShift | - | - | - | 13 of 13 |
| GKE (optimize) | - | - | - | 13 of 13 |
| AKS NAP (Karpenter) | - | - | - | 13 of 13 |
| Turbonomic | - | - | - | 13 of 13 |
| CAST AI | - | - | - | 13 of 13 |
| Spot Ocean | - | - | - | 13 of 13 |

## *5.2 Four clusters, one site*

Setting: pass-through. Every intervention tried here cost some gauge on some platform, so Omni-Compass leaves these decisions to the platform and the result is identical to the platform alone.

| Platform | Better (significant) | Worse beyond tolerance (LOSS) | Worse within 0.5% tolerance | Equal |
|---|---|---|---|---|
| Kubernetes | - | - | - | 13 of 13 |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Platform | Better (significant) | Worse beyond tolerance (LOSS) | Worse within 0.5% tolerance | Equal |
|---|---|---|---|---|
| OpenShift | - | - | - | 13 of 13 |
| GKE (optimize) | - | - | - | 13 of 13 |
| AKS NAP (Karpenter) | - | - | - | 13 of 13 |
| Turbonomic | - | - | - | 13 of 13 |
| CAST AI | - | - | - | 13 of 13 |
| Spot Ocean | - | - | - | 13 of 13 |

### *5.3 Batch jobs*

Setting per platform (chosen on development scenarios): Kubernetes: early add (lead 12, trend over 16), reversal veto; OpenShift: early add (lead 12, trend over 16), reversal veto; GKE (optimize): early add (lead 6, trend over 8), reversal veto; AKS NAP (Karpenter): early add (lead 3, trend over 16), reversal veto; Turbonomic: early add (lead 6, trend over 4), reversal veto; CAST AI: early add (lead 12, trend over 16), reversal veto; Spot Ocean: early add (lead 3, trend over 16), reversal veto.

| Platform | Better (significant) | Worse beyond tolerance (LOSS) | Worse within 0.5% tolerance | Equal |
|---|---|---|---|---|
| Kubernetes | response p99 -91%; response mean -23%; machine starts+stops -16%; scale reversals -70% | - | - | 9 of 13 |
| OpenShift | response p99 -91%; response mean -25%; machine starts+stops -23%; scale reversals -87% | - | - | 9 of 13 |
| GKE (optimize) | response p99 -91%; response mean -27%; machine starts+stops -22% | - | - | 10 of 13 |
| AKS NAP (Karpenter) | response p95 -57%; response p99 -24%; response mean -19%; time over backlog limit -47%; machine starts+stops -13% | - | energy +0.2%; machine-hours +0.5% | 6 of 13 |
| Turbonomic | response p99 -95%; response mean -34%; machine starts+stops -16% | - | - | 10 of 13 |
| CAST AI | response p95 -47%; response p99 -25%; response mean -32%; time over backlog limit -100%; machine starts+stops -9% | scale reversals +7.7% | - | 7 of 13 |
| Spot Ocean | response p95 -12%; response p99 -9%; response mean -8%; time over backlog limit -100%; machine starts+stops -8%; scale reversals -8% | - | energy +0.1%; machine-hours +0.3% | 5 of 13 |

### *5.4 Gpu training*

Setting per platform (chosen on development scenarios): Kubernetes: early add (lead 3, trend over 8), reversal veto; OpenShift: early add (lead 3, trend over 16); GKE (optimize): pass-through; AKS NAP (Karpenter): pass-through; Turbonomic: early add (lead 3, trend over 8); CAST AI: pass-through; Spot Ocean: pass-through.

| Platform | Better (significant) | Worse beyond tolerance (LOSS) | Worse within 0.5% tolerance | Equal |
|---|---|---|---|---|
| Kubernetes | - | - | - | 13 of 13 |
| OpenShift | - | - | time over heat limit +0.1% | 12 of 13 |
| GKE (optimize) | - | - | - | 13 of 13 |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

| Platform | Better (significant) | Worse beyond tolerance (LOSS) | Worse within 0.5% tolerance | Equal |
|---|---|---|---|---|
| AKS NAP (Karpenter) | - | - | - | 13 of 13 |
| Turbonomic | - | - | energy +0.1%; time over heat limit +0.2%; machine-hours +0.4% | 10 of 13 |
| CAST AI | - | - | - | 13 of 13 |
| Spot Ocean | - | - | - | 13 of 13 |

Percentages are the plain change of Omni-Compass on top against the platform alone: for response times, starts+stops, reversals, energy and machine-hours negative is better; for work completed and time healthy positive is better. A LOSS is worse beyond the 0.5% tolerance with a 95% interval excluding zero (section 2).

## 6. C: Omni-Compass alone, against all seven platforms at once

Source `tuning/LEAGUE_HELDOUT.json`. One Omni-Compass configuration per workload type (chosen on development scenarios): web: speed159, multi: speed159, batch: speed112, gpu: omni_fleet.

| Workload | Gauge | Platforms better than Omni-Compass | Omni worse by (range) |
|---|---|---|---|
| batch | energy | GKE (optimize), AKS NAP (Karpenter), Turbonomic, CAST AI, Spot Ocean | 1.7% to 6.7% |
| batch | machine-hours | GKE (optimize), AKS NAP (Karpenter), Turbonomic, CAST AI, Spot Ocean | 4.4% to 18.0% |
| batch | scale reversals | Kubernetes, OpenShift | 266.7% to 279.3% |
| batch | machine starts+stops | Kubernetes, OpenShift | 54.3% to 64.9% |
| gpu | machine-hours | GKE (optimize), AKS NAP (Karpenter), Turbonomic, CAST AI, Spot Ocean | 5.5% to 14.4% |
| gpu | scale reversals | Kubernetes, OpenShift | 85.6% to 87.6% |
| gpu | machine starts+stops | Kubernetes, OpenShift | 37.6% to 49.8% |
| multi | energy | Kubernetes, OpenShift, GKE (optimize), AKS NAP (Karpenter), Turbonomic, CAST AI, Spot Ocean | 23.9% to 64.7% |
| multi | machine-hours | Kubernetes, OpenShift, GKE (optimize), AKS NAP (Karpenter), Turbonomic, CAST AI, Spot Ocean | 36.7% to 119.0% |
| multi | pod changes | Kubernetes, OpenShift, GKE (optimize), AKS NAP (Karpenter), CAST AI, Spot Ocean | 8.2% to 10.3% |
| web | energy | Kubernetes, OpenShift, GKE (optimize), AKS NAP (Karpenter), Turbonomic, CAST AI, Spot Ocean | 23.9% to 63.5% |
| web | machine-hours | Kubernetes, OpenShift, GKE (optimize), AKS NAP (Karpenter), Turbonomic, CAST AI, Spot Ocean | 36.8% to 116.3% |
| web | pod changes | Kubernetes, OpenShift, GKE (optimize), AKS NAP (Karpenter), CAST AI, Spot Ocean | 7.8% to 10.8% |

### *6.1 Where C wins*

- Web services: response p95 110.1 ms vs best platform 124.9 ms; response p99 122.8 ms vs best platform 546 ms; response mean 145.3 ms vs best platform 216.2 ms.
- Four clusters, one site: response p95 112.1 ms vs best platform 126.9 ms; response p99 137.8 ms vs best platform 262.9 ms; response mean 145.6 ms vs best platform 217.2 ms.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

- Batch jobs: response p99 337.4 ms vs best platform 1277 ms; response mean 115.2 ms vs best platform 135.6 ms.

### *6.2 The physical limit: what no controller can do*

For each scenario, dynamic programming with perfect knowledge of the future (boot delay ignored, which only helps it) finds the fewest machine starts+stops that can hold every pod at a given machine-hour budget (`tuning/bound.py`).

| Workload | Scenario | Tightest platform machine-hours | Fewest starts+stops of any platform | Perfect controller at those machine-hours |
|---|---|---|---|---|
| web | 101 | 18.4 | 9 | 11 |
| web | 102 | 18.3 | 9 | 9 |
| web | 103 | 31.4 | 4 | 26 |
| web | 104 | 22.0 | 12 | 24 |
| multi | 101 | 74.9 | 36 | 46 |
| multi | 102 | 112.7 | 26 | 100 |
| multi | 103 | 122.5 | 17 | 114 |
| multi | 104 | 93.4 | 35 | 91 |
| batch | 101 | 108.0 | 24 | 67 |
| batch | 102 | 56.8 | 7 | 33 |
| batch | 103 | 66.9 | 12 | 44 |
| batch | 104 | 93.4 | 15 | 58 |
| gpu | 101 | 66.4 | 12 | 28 |
| gpu | 102 | 57.5 | 16 | 36 |
| gpu | 103 | 51.5 | 19 | 30 |
| gpu | 104 | 57.0 | 20 | 26 |

In 15 of 16 scenarios even a perfect controller needs more starts+stops than the laziest platform to reach the tightest platform's machine-hours. Those two platforms sit at opposite ends of one physical trade; beating both at once is impossible for any software. In C, a loss on one side of that pair is the price of a win on the other.

### *6.3 Letting the equations drive directly*

A governor in which the equations run closed-loop and their own quantities (control effort, unmet need I_U, the bath's rate) drive every muscle (`omnicompass/mathdrive.py`) was tested on 96 settings per workload: 66 losing cells, the same trade-off as the rule-based wiring. The engine clock alone moved batch p99 from 3.9 s to 0.1 s at equal energy.

### *6.4 Strict C: the manuscript's closure law drives the machines and the pods*

`omnicompass/closure.py` implements, as the only authority over machines, the laws of the owner's manuscript: the forward projection of Section 5c, the master closure law of Chapter 20 (F = G0 + Gc: no correction inside the admissible domain, an inward correction sized to restore the margin when the projected state approaches its boundary, so that G . n <= 0 on the boundary), the turning point of Chapters 29-30 (a machine is released only after the peak) and the dual-bath exchange of Chapter 31 (level and rate of demand). A release must also stay inside the calm set for a dwell (the resource-aware envelope, Proposition 2). Kubernetes' HPA and node managers are off; Kubernetes only schedules. Settings chosen on development scenarios, frozen by SHA-256 (56be5f50b8d2631c), then run once on 30 held-out scenarios per workload.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Workload | Gauge | Kubernetes | OpenShift | GKE (optimize) | AKS NAP (Karpenter) | Turbonomic | CAST AI | Spot Ocean | Omni-Compass (closure law) |
|---|---|---|---|---|---|---|---|---|---|
| web | energy | 14.11 | 14.75 | 11.8 | 11.22 | 11.75 | 11.03 | 11.2 | **11.26** |
| web | machine-hours | 37.61 | 40.27 | 27.99 | 25.56 | 27.78 | 24.77 | 25.52 | **24.26** |
| web | response p95 | 122.9 | 122.8 | 123.4 | 123.6 | 138.2 | 124.1 | 123.8 | **140.4** |
| web | response p99 | 806.6 | 648.3 | 1345 | 1428 | 3251 | 2500 | 2023 | **318.9** |
| web | response mean | 206.4 | 205.3 | 212.5 | 215.7 | 251 | 236.1 | 226.9 | **164.5** |
| web | machine starts+stops | 8.733 | 6.5 | 16.47 | 19.67 | 12.73 | 26.57 | 27.6 | **11.6** |
| web | work completed | 1 | 1 | 1 | 1 | 1 | 1 | 1 | **1** |
| multi | energy | 54.24 | 56.4 | 45.04 | 42.81 | 44.86 | 42.16 | 42.85 | **42.43** |
| multi | machine-hours | 145.7 | 154.7 | 107.4 | 98.08 | 106.6 | 95.37 | 98.24 | **90.42** |
| multi | response p95 | 124.9 | 124.8 | 125.6 | 125.9 | 119 | 126.5 | 126.1 | **148.9** |
| multi | response p99 | 202.2 | 168.7 | 641.9 | 991.4 | 2.095e+04 | 2068 | 1531 | **240.3** |
| multi | response mean | 220.2 | 218.4 | 236.9 | 243.1 | 784.9 | 261.8 | 253.7 | **196.8** |
| multi | machine starts+stops | 36.77 | 28 | 66.63 | 79.7 | 53.8 | 102.4 | 108.1 | **45.67** |
| multi | work completed | 1 | 1 | 1 | 1 | 1 | 1 | 1 | **1** |
| batch | energy | 65.45 | 65.68 | 63.28 | 60.11 | 62.73 | 60.65 | 60 | **61.65** |
| batch | machine-hours | 94.71 | 95.49 | 87.48 | 76.91 | 85.67 | 78.71 | 76.55 | **78.5** |
| batch | response p95 | 100 | 100 | 100 | 861.9 | 100 | 1840 | 220.1 | **100** |
| batch | response p99 | 1072 | 1291 | 2086 | 3880 | 2242 | 4974 | 2491 | **1012** |
| batch | response mean | 133.1 | 138 | 158.8 | 257.2 | 170.6 | 360 | 184.2 | **141.3** |
| batch | machine starts+stops | 12.93 | 14.13 | 29 | 92.33 | 35.03 | 160.6 | 141.2 | **17.2** |
| batch | work completed | 1 | 1 | 1 | 1 | 1 | 1 | 1 | **1** |
| gpu | energy | 201.4 | 201.6 | 199.5 | 197.2 | 198.9 | 197.3 | 197 | **198.9** |
| gpu | machine-hours | 60.77 | 61.07 | 58.15 | 54.89 | 57.25 | 55.06 | 54.62 | **55.29** |
| gpu | response p95 | 2.288e+06 | 2.286e+06 | 2.288e+06 | 2.29e+06 | 2.288e+06 | 2.292e+06 | 2.29e+06 | **2.287e+06** |
| gpu | response p99 | 2.397e+06 | 2.396e+06 | 2.398e+06 | 2.4e+06 | 2.398e+06 | 2.402e+06 | 2.401e+06 | **2.396e+06** |
| gpu | response mean | 8.886e+05 | 8.875e+05 | 8.894e+05 | 8.904e+05 | 8.886e+05 | 8.922e+05 | 8.911e+05 | **8.879e+05** |
| gpu | machine starts+stops | 16.5 | 16.6 | 33.5 | 50.63 | 30.57 | 53.93 | 55.63 | **15.27** |
| gpu | work completed | 0.9027 | 0.9028 | 0.9027 | 0.9026 | 0.9028 | 0.9025 | 0.9026 | **0.9028** |

Losing cells (worse than that platform beyond tolerance): web services 10 of 91; four clusters, one site 12 of 91; batch jobs 7 of 91; GPU training 12 of 91.

- Web services: response p95 vs Kubernetes 14.2% worse; machine starts+stops vs Kubernetes 32.8% worse; response p95 vs OpenShift 14.3% worse; machine starts+stops vs OpenShift 78.5% worse; scale reversals vs OpenShift 192.9% worse; response p95 vs GKE (optimize) 13.8% worse; response p95 vs AKS NAP (Karpenter) 13.6% worse; energy vs CAST AI 2.1% worse; response p95 vs CAST AI 13.2% worse; response p95 vs Spot Ocean 13.4% worse.
- Four clusters, one site: response p95 vs Kubernetes 19.2% worse; machine starts+stops vs Kubernetes 24.2% worse; response p95 vs OpenShift 19.4% worse; response p99 vs OpenShift 42.4% worse; machine starts+stops vs OpenShift 63.1% worse; scale reversals vs OpenShift 153.2% worse; response p95 vs GKE



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

(optimize) 18.5% worse; response p95 vs AKS NAP (Karpenter) 18.3% worse; response p95 vs Turbonomic 25.2% worse; energy vs CAST AI 0.6% worse; response p95 vs CAST AI 17.8% worse; response p95 vs Spot Ocean 18.1% worse.

- Batch jobs: machine starts+stops vs Kubernetes 33.0% worse; machine starts+stops vs OpenShift 21.7% worse; energy vs AKS NAP (Karpenter) 2.6% worse; machine-hours vs AKS NAP (Karpenter) 2.1% worse; energy vs CAST AI 1.7% worse; energy vs Spot Ocean 2.8% worse; machine-hours vs Spot Ocean 2.6% worse.
- Gpu training: energy vs AKS NAP (Karpenter) 0.9% worse; time over power limit vs AKS NAP (Karpenter) 0.7% worse; time over heat limit vs AKS NAP (Karpenter) 0.8% worse; machine-hours vs AKS NAP (Karpenter) 0.7% worse; energy vs CAST AI 0.8% worse; time over power limit vs CAST AI 1.5% worse; time over heat limit vs CAST AI 1.9% worse; energy vs Spot Ocean 1.0% worse; time over power limit vs Spot Ocean 1.8% worse; time over heat limit vs Spot Ocean 1.6% worse; machine-hours vs Spot Ocean 1.2% worse; time healthy vs Spot Ocean 1.5% worse.

## 6A. Runtime Benchmark Protocol: faults at five stress levels

The owner's *OmniCompass Runtime Benchmark Protocol* (stage 1, Python runtime) run on this fleet plant with every architecture: 100 runs per stress level per workload, seeds 900001-900100 (never used for tuning). Level L injects L faults (machines dying, load spikes, services crash-looping, a noisy neighbour), identical for every system (`tools/protocol_bench.py`). A run is **conveyed** when every fault is recovered within 20 minutes and every cluster is inside the admissible basin for the final 10 minutes (worst response <= 1 s, queue < 0.28, pending pods <= 5%, power and heat inside limits). A **page** is an out-of-basin episode of 5 minutes or more, the usual alert rule: each would call a human. Nobody intervenes in the simulation.

**Web services**

| Column | System | Conveyed | Pages | Recovery (min) | Minutes outside basin | Timeouts > 2 s | Energy (kWh) |
|---|---|---|---|---|---|---|---|
| A | AKS NAP (Karpenter) | 233 of 500 | 391 | 6.6 | 32.8 | 4.52% | 11.90 |
| B | AKS NAP (Karpenter) | 233 of 500 | 391 | 6.6 | 32.8 | 4.52% | 11.90 |
| A | CAST AI | 231 of 500 | 404 | 6.6 | 34.1 | 4.68% | 11.75 |
| B | CAST AI | 231 of 500 | 404 | 6.6 | 34.1 | 4.68% | 11.75 |
| A | GKE (optimize) | 227 of 500 | 366 | 6.9 | 32.4 | 4.25% | 12.76 |
| B | GKE (optimize) | 227 of 500 | 366 | 6.9 | 32.4 | 4.25% | 12.76 |
| A | Kubernetes | 218 of 500 | 338 | 8.3 | 33.3 | 3.97% | 15.54 |
| B | Kubernetes | 218 of 500 | 338 | 8.3 | 33.3 | 3.97% | 15.54 |
| A | OpenShift | 220 of 500 | 328 | 8.0 | 32.2 | 3.82% | 15.52 |
| B | OpenShift | 220 of 500 | 328 | 8.0 | 32.2 | 3.82% | 15.52 |
| A | Spot Ocean | 234 of 500 | 392 | 6.6 | 32.8 | 4.56% | 11.94 |
| B | Spot Ocean | 234 of 500 | 392 | 6.6 | 32.8 | 4.56% | 11.94 |
| A | Turbonomic | 169 of 500 | 552 | 14.4 | 53.3 | 5.96% | 12.43 |
| B | Turbonomic | 169 of 500 | 552 | 14.4 | 53.3 | 5.96% | 12.43 |
| C | C-hpa | 202 of 500 | 353 | 11.7 | 35.8 | 3.81% | 20.69 |
| C | C-strict | 238 of 500 | 283 | 6.0 | 26.2 | 3.45% | 13.02 |

In the protocol's required form: Omni-Compass alone (closure law) conveyed 238 out of 500 runs; the best platform alone (Spot Ocean) conveyed 234 out of 500. Omni-Compass would have paged a human 283 times, that platform 392 times. Omni-Compass average recovery time was 6.0 min, that platform's 6.6 min.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

### Four clusters, one site

| Column | System | Conveyed | Pages | Recovery (min) | Minutes outside basin | Timeouts > 2 s | Energy (kWh) |
|---|---|---|---|---|---|---|---|
| A | AKS NAP (Karpenter) | 49 of 500 | 534 | 5.6 | 69.5 | 1.71% | 45.25 |
| B | AKS NAP (Karpenter) | 49 of 500 | 534 | 5.6 | 69.5 | 1.71% | 45.25 |
| A | CAST AI | 44 of 500 | 608 | 5.7 | 74.2 | 1.81% | 44.54 |
| B | CAST AI | 44 of 500 | 608 | 5.7 | 74.2 | 1.81% | 44.54 |
| A | GKE (optimize) | 49 of 500 | 455 | 5.7 | 65.6 | 1.63% | 47.96 |
| B | GKE (optimize) | 49 of 500 | 455 | 5.7 | 65.6 | 1.63% | 47.96 |
| A | Kubernetes | 44 of 500 | 330 | 6.5 | 59.2 | 1.44% | 57.67 |
| B | Kubernetes | 44 of 500 | 330 | 6.5 | 59.2 | 1.44% | 57.67 |
| A | OpenShift | 43 of 500 | 310 | 6.4 | 57.6 | 1.38% | 59.47 |
| B | OpenShift | 43 of 500 | 310 | 6.4 | 57.6 | 1.38% | 59.47 |
| A | Spot Ocean | 44 of 500 | 508 | 5.6 | 69.1 | 1.73% | 45.30 |
| B | Spot Ocean | 44 of 500 | 508 | 5.6 | 69.1 | 1.73% | 45.30 |
| A | Turbonomic | 26 of 500 | 867 | 15.0 | 97.6 | 2.49% | 47.57 |
| B | Turbonomic | 26 of 500 | 867 | 15.0 | 97.6 | 2.49% | 47.57 |
| C | C-hpa | 64 of 500 | 328 | 8.8 | 45.8 | 1.07% | 74.42 |
| C | C-strict | 45 of 500 | 231 | 4.3 | 52.6 | 1.33% | 45.82 |

In the protocol's required form: Omni-Compass alone (closure law) conveyed 45 out of 500 runs; the best platform alone (GKE (optimize)) conveyed 49 out of 500. Omni-Compass would have paged a human 231 times, that platform 455 times. Omni-Compass average recovery time was 4.3 min, that platform's 5.7 min.

### Batch jobs

| Column | System | Conveyed | Pages | Recovery (min) | Minutes outside basin | Timeouts > 2 s | Energy (kWh) |
|---|---|---|---|---|---|---|---|
| A | AKS NAP (Karpenter) | 314 of 500 | 234 | 4.5 | 31.4 | 9.01% | 70.02 |
| B | AKS NAP (Karpenter) | 338 of 500 | 235 | 4.5 | 25.1 | 8.09% | 70.95 |
| A | CAST AI | 161 of 500 | 237 | 4.4 | 39.9 | 10.25% | 72.15 |
| B | CAST AI | 224 of 500 | 233 | 4.4 | 26.3 | 8.39% | 73.91 |
| A | GKE (optimize) | 409 of 500 | 235 | 4.0 | 19.0 | 7.07% | 77.00 |
| B | GKE (optimize) | 425 of 500 | 228 | 4.0 | 13.9 | 6.38% | 79.12 |
| A | Kubernetes | 424 of 500 | 228 | 3.7 | 15.4 | 6.28% | 85.35 |
| B | Kubernetes | 425 of 500 | 234 | 3.8 | 12.2 | 5.77% | 87.43 |
| A | OpenShift | 422 of 500 | 232 | 3.8 | 16.0 | 6.36% | 85.36 |
| B | OpenShift | 425 of 500 | 232 | 3.8 | 12.2 | 5.82% | 86.89 |
| A | Spot Ocean | 360 of 500 | 231 | 4.4 | 23.1 | 7.77% | 71.08 |
| B | Spot Ocean | 360 of 500 | 234 | 4.4 | 20.6 | 7.56% | 71.79 |
| A | Turbonomic | 407 of 500 | 232 | 4.2 | 21.0 | 7.45% | 74.08 |
| B | Turbonomic | 426 of 500 | 233 | 4.1 | 14.2 | 6.62% | 75.25 |
| C | C-hpa | 166 of 500 | 235 | 30.8 | 73.5 | 20.65% | 85.38 |
| C | C-strict | 422 of 500 | 237 | 4.1 | 17.3 | 6.93% | 74.13 |

In the protocol's required form: Omni-Compass alone (closure law) conveyed 422 out of 500 runs; the best platform alone (Kubernetes) conveyed 424 out of 500. Omni-Compass would have paged a human 237 times, that platform 228 times. Omni-Compass average recovery time was 4.1 min, that platform's 3.7 min.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

### Gpu training

| Column | System | Conveyed | Pages | Recovery (min) | Minutes outside basin | Timeouts > 2 s | Energy (kWh) |
|---|---|---|---|---|---|---|---|
| A | AKS NAP (Karpenter) | 13 of 500 | 802 | 95.4 | 227.2 | 69.96% | 202.42 |
| B | AKS NAP (Karpenter) | 13 of 500 | 802 | 95.4 | 227.2 | 69.96% | 202.42 |
| A | CAST AI | 13 of 500 | 925 | 95.1 | 226.5 | 70.61% | 202.80 |
| B | CAST AI | 13 of 500 | 925 | 95.1 | 226.5 | 70.61% | 202.80 |
| A | GKE (optimize) | 15 of 500 | 802 | 96.5 | 227.1 | 68.76% | 205.17 |
| B | GKE (optimize) | 15 of 500 | 802 | 96.5 | 227.1 | 68.76% | 205.17 |
| A | Kubernetes | 18 of 500 | 782 | 97.5 | 225.4 | 67.95% | 206.83 |
| B | Kubernetes | 18 of 500 | 792 | 97.6 | 226.1 | 67.87% | 207.04 |
| A | OpenShift | 18 of 500 | 788 | 97.6 | 226.3 | 67.91% | 207.08 |
| B | OpenShift | 18 of 500 | 788 | 97.6 | 226.2 | 67.88% | 207.13 |
| A | Spot Ocean | 13 of 500 | 830 | 94.9 | 226.0 | 70.66% | 202.22 |
| B | Spot Ocean | 13 of 500 | 830 | 94.9 | 226.0 | 70.66% | 202.22 |
| A | Turbonomic | 14 of 500 | 786 | 96.5 | 226.6 | 68.78% | 204.30 |
| B | Turbonomic | 14 of 500 | 792 | 96.2 | 226.6 | 68.67% | 204.40 |
| C | C-hpa | 0 of 500 | 1232 | 174.0 | 295.6 | 83.41% | 198.48 |
| C | C-strict | 13 of 500 | 809 | 95.1 | 225.0 | 67.86% | 204.08 |

In the protocol's required form: Omni-Compass alone (closure law) conveyed 13 out of 500 runs; the best platform alone (Kubernetes) conveyed 18 out of 500. Omni-Compass would have paged a human 809 times, that platform 782 times. Omni-Compass average recovery time was 95.1 min, that platform's 97.5 min.

## 6B. Second generation (held-out, settings frozen by SHA-256 before each run)

**B with muscle tone and consolidation** (`tuning/b_tone.py`, held-out seeds 700401-700430): machines a platform powers off are parked instead (alive, low power, instant wake), with a reserve sized by the law; Omni consolidates on top only where that loses nothing. Losing cells: 3 of 364.

**B fix for the remaining pairs** (`tuning/b_fix.py`, 30 development seeds, held-out 700701-700730): multi on aks_nap: 0 losing cells; batch on turbonomic: 0 losing cells; multi on cast_ai: 0 losing cells.

**Whole body at four-cluster sites** (`tuning/site_league.py`, held-out 701001-701030): traffic shift between a site's clusters (a cluster short of room runs its overflow on another cluster's powered spare cores, +5 ms per shifted request; assumes replicated services) and, for C, the closure law on the site total with one warm reserve.

| Platform | B losing cells | C losing cells | C energy | C slowest responses | C average |
|---|---|---|---|---|---|
| Kubernetes | 0 | 2 | +16% | +56% | +30% |
| OpenShift | 0 | 2 | +19% | +46% | +29% |
| GKE (optimize) | 0 | 0 | +7% | +85% | +36% |
| AKS NAP (Karpenter) | 0 | 0 | +2% | +90% | +39% |
| Turbonomic | 0 | 1 | +7% | +99% | +75% |
| CAST AI | 0 | 0 | -0% | +94% | +42% |
| Spot Ocean | 0 | 0 | +2% | +91% | +40% |

C gains are Omni against that platform (positive = Omni better).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

## 6C. Live levers on real Kubernetes (measured)

```
Live levers on real Kubernetes (kind, 1 control plane + 6 workers), GitHub Actions run 36270589830, comm
    ↪ it 43f6747,
2026-09-26 20:45-20:48 UTC. Omni-Compass ran as its least-privilege service account (deploy/kind/rbac-om
    ↪ ni.yaml +
rbac-levers.yaml). Script: scripts/kind_levers.sh. Result: LIVE LEVERS: PASS, 19 of 19 checks, 0 lever e
    ↪ rrors.

Permission receipts (kubectl auth can-i as the Omni service account)
  can:    patch pods/resize default, patch deployment/idle-worker, patch deployments/scale idle-worker,
          get configmap/omni-work, patch jobs default, create/delete resourcequotas agents, patch pods/r
    ↪ esize agents
  cannot: delete pods agents, create pods agents, delete deployments agents, patch deployment/load-gener
    ↪ ator,
          create resourcequotas default, get secrets (any namespace)

rightsize   php-apache CPU requests: before [200m] after [50m]; after kill [200m]
coldstart   idle-worker replicas: idle -&gt; 0; work waiting -&gt; 2; idle again -&gt; 0; after kill -&gt; 2
batch_pace  train-a suspended: stress -&gt; true; calm -&gt; false; stress -&gt; true; after kill -&gt; false
contain     agent quota present: over budget -&gt; 1; under budget -&gt; 0; over -&gt; 1; after kill -&gt; 0
            agent CPU limits: before [400m 400m] contained [100m 100m] lifted [400m 400m] after kill [40
    ↪ 0m 400m]
cooling     setpoints written to the stand-in building controller: 27.0 27.0 27.0 22.0 (kill restores 22
    ↪ .0)
records left after kill: none

Earlier runs of the same script (kept for the record): run 36268772674 failed (containment could not lif
    ↪ t: Kubernetes
refuses to raise a pod limit above a standing quota; one failing lever stopped the kill switch); run 362

    ↪ 69885518 printed
PASS but its checks were chained so failures were not counted (containment still not lifted); run 362702
    ↪ 02629 failed in
the script's quota probe once the quota was correctly deleted. Fixes: quota deleted before limits are re
    ↪ stored; each
lever and each kill-switch restore isolated; every check counted on its own line.
Not covered: the cooling lever writes to a stand-in controller (a CI runner has no chiller); CPU frequen
    ↪ cy and power caps
need hardware with cpufreq; traffic shift needs two real clusters behind one load balancer.
```

## 7. Live Kubernetes (measured)

Two identical kind clusters (1 control plane + 6 workers) at the same time, same load; one with Omni-Compass on top (B). Omni runs as a least-privilege service account whose permissions are proven with `kubectl auth can-i` receipts before each run.

| Run | Change | Validity | p95 (ms) | p99 (ms) | Energy |
|---|---|---|---|---|---|
| 36213152881 | HPA target, node pool, power cap (in-place pod CPU limits), heat (harn | valid | 486 to 802 | 675 to 1,113 | 222 to 215 |
| 36214629046 | latency afferent (p95 over 500 ms SLO as queue pressure) and power-cap | valid | 481 to 600 | 596 to 777 | 222 to 219 |
| 36216647786 | SLO reflex; eviction receipt fixed | INVALID as a test of the engine | 479 to 320 | 583 to 420 | 224 to 220 |
| 36218637030 | controller fail-safe; benchmark prints controller log and rejects earl | PARTIAL | 491 to 492 | 645 to 683 | 223 to 221 |
| 36220059046 | get on pods/resize (power cap can apply) | INVALID for response time and energy | 501 to 380 | 660 to 415 | 224 to 220 |
| 36221702999 | probe tunnel restarts on both arms; power cap able to apply | VALID | 491 to 383 | 613 to 491 | 224 to 216 |



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

Run 6 in full:

| Gauge | Kubernetes alone | + Omni-Compass | Change |
|---|---|---|---|
| Worker nodes in service, mean | 6.00 | 3.19 | -46.8% |
| Node-hours | 2.02 | 1.07 | -47.0% |
| Energy (Wh), standby counted | 224 | 216 | -3.2% |
| Power (W), peak | 710 | 682 | -3.8% |
| Energy per core-hour (Wh) | 742 | 1,041 | +40.2% worse (significant) |
| Utilisation | 0.074 | 0.097 | +30.2% |
| Pending pods, pod-minutes | 0.283 | 0.533 | +88% (n.s.) |
| HPA shortfall, minutes | 0.817 | 1.33 | +63% (n.s.) |
| Response time (ms), mean | 264 | 165 | -37.4% |
| Response time (ms), median | 227 | 119 | -47.4% |
| Response time (ms), 95th percentile | 491 | 383 | -22.1% |
| Response time (ms), 99th percentile | 613 | 491 | -19.9% |
| Failed requests (%) | 0 | 0 | 0 |

First fully valid run with every live muscle on: fewer servers (-47%), less energy (-3.2%, standby counted), lower peak power, and faster responses at every percentile. The one significant negative is energy per core-hour (+48%): the power cap lowers the CPU the app consumes (-31%), and that CPU is the metric's denominator. Whether energy per request served also rose is not measured (the load generator's request count is not captured); a per-request energy gauge is added to the capture next.

## 8. The nine problem-map muscles (simulated, held-out, Omni-Compass direct against the strongest native tool)

| Muscle | Opponent | Better | Worse |
|---|---|---|---|
| rightsize | hpa_vpa | cpu core hours -3%; mem gib hours -11%; p99 ms -99%; replica reversals -84%; slo breach min -57% | p95 ms +15%; oom kills +531% |
| coldstart | keda | p95 ms -75%; p99 ms -58%; delayed 1s pct -71%; instance hours -8% | cold starts +736% |
| gpupack | binpack | energy kwh -2%; idle gpu hours powered -20% | wait mean min +16%; migrations 0 to 6.07 |
| powersmooth | floor_safe | energy overhead pct -9%; throughput loss pct -44%; swing 1s mw -6% | max ramp mw s +127% |
| health | detect | goodput pct +10%; lost gpu hours -20%; restarts -29%; straggler node hours -99% | - |
| cooling | reset | pue -2%; cooling mwh -13%; inlet violation min -73% | - |
| inference | keda | ttft p95 s -18%; slo breach pct -58%; gpu hours -19%; preemptions -56% | - |
| containment | static | rogue overspend usd -96%; time to contain min -99%; peak subagents -59% | false stops +357%; honest work pct -7% |
| vmenergy | ratio | attr error pct -37%; worst vm error pct -34% | - |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

## 9. Threats to validity

- Competitors are emulated from documentation on a shared plant; their production binaries may behave differently, and their full products include features (spot pricing, instance-type selection, rebalancing) that this plant does not model.
- Simulated plants are fluid models: pods pack perfectly, boot delay is fixed, power is idle + dynamic x utilisation. Real clusters fragment and boot times vary.
- Live runs use kind: nodes are containers on one CI machine; power is modelled, not metered; the two arms ran on two different CI machines. Six 20-minute runs are evidence of behaviour, not of production savings.
- Development and held-out scenarios come from the same generator; held-out protects against tuning to seeds, not against a generator that differs from real traffic. Real traces (PlanetLab) are covered in the main benchmark report.
- The loss rule uses a 0.5% tolerance and a 95% interval on 30 scenarios; small true differences can be missed.

## 10. Reproduce

```
pip install -r requirements.txt &amp;&amp; python verify.py
python tuning/b_league.py dev                              # B vs A, choose settings (development seeds)
python tuning/b_league.py heldout tuning/B_SETTINGS_FROZEN.json   # B vs A, held-out
python tuning/league.py heldout                            # C vs all platforms
python tuning/bound.py                                     # perfect-foresight frontier
python tools/mechanism.py                                  # engine modes and compute
python tuning/closure_search.py &amp;&amp; python tuning/closure_search2.py &amp;&amp; python tuning/closure_heldout.py
    ↪   # strict C, closure law
python tools/protocol_bench.py 100                         # runtime protocol, faults at five stress level
    ↪ s
for m in rightsize coldstart gpupack powersmooth health cooling inference containment vmenergy; do pytho
    ↪ n -m omnilab.bench $m heldout; done
GitHub Actions workflow 'benchmark' (commit message tag [bench]): live A vs B on kind
python tools/abc_report.py &amp;&amp; python pilot/bench_pdf.py docs/history/OMNICOMPASS_ABC_REPORT.md docs/hist
    ↪ ory/OMNICOMPASS_ABC_REPORT.pdf
```



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 54

# Comparison with Existing Controllers

People who run these systems often see "autoscaling" everywhere and assume it all does the same thing. It does not. This page sets out, from public descriptions, what each system controls, and where Omni-Compass sits among them.

**How to read it.** Borg and Twine are private to Google and Meta; Turbonomic is closed commercial software. None of them can be run here, so they are compared by what their makers have published (papers, documentation, product pages), never by a run. A mark of "—" means the capability is not in the system's public description, not that it is impossible for it. The only fair measured comparison is on the customer's own system: their stack as it runs today against the same stack with Omni-Compass on top, in paired runs (`docs/INTEGRATION_MANUAL.md`, section 6).

# 1. What each system controls

| System (public description) | Pods / replicas | Machines | GPU power | CPU clock / power | Moves watts between CPU and GPU | Site power budget | One decision across all of these |
|---|---|---|---|---|---|---|---|
| Kubernetes HPA | ✓ | — | — | — | — | — | — |
| Kubernetes VPA | ✓ (requests) | — | — | — | — | — | — |
| Cluster Autoscaler, Karpenter | — | ✓ | — | — | — | — | — |
| KEDA | ✓ (to zero) | — | — | — | — | — | — |
| Red Hat OpenShift / OKD | ✓ (the Kubernetes autoscalers) | ✓ (machine autoscaling) | — | — | — | — | — |
| IBM Turbonomic | ✓ (sizing, scaling) | ✓ (placement, sizing) | — | — | — | — | partial (resource actions across layers) |
| Google Borg | ✓ (job scheduling) | ✓ (cluster placement) | — | — | — | Google has published separate power-capping work | — |
| Meta Twine | ✓ | ✓ | — | — | — | Meta has published separate power management (Dynamo) | — |
| NVIDIA DCGM | — | — | ✓ (limits, monitoring) | — | — | — | — |
| Linux schedutil, RAPL | — | — | — | ✓ | — | — | — |
| AMD SmartShift, NVIDIA Dynamic Boost (laptops) | — | — | ✓ | ✓ | ✓ (inside one laptop) | — | inside one device |
| Intel GEOPM (HPC) | — | — | partial | ✓ | partial (per job) | ✓ (per job / cluster) | partial |
| **Omni-Compass** | ✓ (on top of HPA; or deciding) | ✓ (park and wake) | ✓ | ✓ | ✓ (conveyance law; simulated) | ✓ (simulated) | ✓ |

# 2. How each one decides

| | Typical rule | Omni-Compass |
|---|---|---|
| Pods | add replicas when average CPU passes a target | the same autoscaler keeps working; Omni-Compass sets its target from the whole system's state and raises floors ahead of bursts |
| Machines | add when pods cannot be placed, remove when underused | park and wake machines by one law, and only when every sense is live, pods are not rising and nothing waits |
| GPU | vendor default: full power, or a fixed cap set by hand | a cap from the engine, bounded by a floor and released at once when the card is busy or response time slips |
| CPU | the kernel's governor picks a clock from recent load | a ceiling from the same engine, inside the nervous system's envelope |
| Power | a fixed plan leaving room for every device at peak; a breaker or capper reacts when the site goes over | one conserved budget: watts move from organs holding surplus to organs in need, never created, never over the budget |
| Safety | per tool | one OFF switch returns every organ to its recorded setting; watch mode first; service first |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## 3. What is different, in plain words

1. **One brain instead of many separate reflexes.** Today each tool watches its own gauge: the pod autoscaler watches CPU, the node autoscaler watches pending pods, the GPU runs at its default, the power plan is fixed on paper. Nobody decides for the whole. Omni-Compass reads the whole system into one state and tells each tool the envelope it may act in. The tools keep doing their own jobs.
2. **Power as a budget that moves.** Today a building keeps room for every device to hit its peak at once, so much of its power sits reserved and unused. Omni-Compass treats the building's watts as one conserved total and moves them to where the work is: from CPUs and quiet GPUs to busy GPUs, never over the limit (`docs/CONVEYANCE_LAW.md`).
3. **A proof, not a tuned dial.** The budget law is proved to conserve the budget and to converge, and those properties are checked on every build (`tests/test_conveyance.py`).
4. **Service guarded on every power move.** Power is only taken back while response time is inside the target, and given back at once when it is not.
5. **Reversible in one move.** Every setting is recorded before it is changed and restored by one switch.

## 4. What the numbers show today, and what kind they are

| Against | Result | Kind |
|---|---|---|
| Kubernetes as it runs by default (HPA + fixed nodes) | p95 response time −37% to −55%, replicas −12% to −25%, machines in service −17% to −21% | measured on real Kubernetes (kind), 10 paired runs each, `results/live/` |
| A GPU at its vendor default | +1.3% to +5.1% work per energy, p95 within +10% | modelled card (MLPerf-calibrated), `results/gpu/sim/after` |
| Today's power practice (every GPU at one fixed cap that leaves room for every CPU at peak) | +1.4% to +5.7% work served, backlog −16% to −28%, never over the site budget | modelled, `results/hardware/NODE_EXCHANGE_*.json` |
| A reactive site capper alone | 60-196 minutes over the site budget without Omni-Compass; 0 with it | modelled |

Turbonomic, Borg and Twine cannot be measured here. Against a customer who runs one of them, the comparison is made on the customer's own system, with the same paired method.

## 5. Sources

- Kubernetes HPA, VPA, Cluster Autoscaler, Karpenter, KEDA: the projects' own documentation.
- Red Hat OpenShift / OKD: Red Hat's documentation of its autoscaling and machine management.
- IBM Turbonomic: IBM's product documentation.
- Borg: Verma et al., "Large-scale cluster management at Google with Borg", EuroSys 2015; Google's public Borg traces.
- Twine: Tang et al., "Twine: A Unified Cluster Management System for Shared Infrastructure", OSDI 2020.
- Meta Dynamo: Wu et al., "Dynamo: Facebook's Data Center-Wide Power Management System", ISCA 2016.
- NVIDIA DCGM and Dynamic Boost; AMD SmartShift; Intel GEOPM: the vendors' documentation.

Where a row above is wrong or out of date, correct it from the maker's own publication.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 55

# The State of Play

Current facts only. Earlier states, failures and chronology are kept whole in `docs/HISTORY.md`. The release this page describes is identified by `RELEASE_MANIFEST.json` (commit, fingerprints of the engine, the C++ twins, the GPU protocol, the live evidence and the verification receipt), which `verify.py` checks against the files.

**Rerun everything:** `pip install -r requirements.txt && python verify.py` ends with `VERIFICATION: PASS`.

**The afternoon of 10 October: Omni v4 ordered, the sweep closed, two faults found, the second set landed.** The founder ordered the collective mechanism as the next engine (`docs/OMNI_V4_PLAN.md`: one body, one brain, one tick a second; a trial runs to its full measurement and is never ended by the calm it causes; the body's cost judged, nothing anywhere made worse to make one thing better; one trial at a time inside a body; nothing permanent; the wall belonging to the body; every wire forced through it; the six organisms, the four realms, all realms stacked and the whole catalog, re-cut by a written rule). The final coverage sweep against the internet added register rows 74 to 78 and five catalog families for v4, and mapped the mega-caps' closed systems to the open analogs that carry the same muscles (`docs/COVERAGE_MAP.md`). Two faults GitHub's own runs showed and no local check had were fixed: the release manifest, not refreshed after the morning's last document edits, and the cpu-power workflow file, refused by GitHub since its first push for an unquoted colon; the verifier now parses every workflow file. The five live products' second set on the amended trial rule landed (21 runs) and is archived; its early reading, from the logs, is in `docs/RERUN_2026-10-10.md` and the manual's 16.6c: on Kafka the brain allowed the third consumer once and the whole gain returned at 2.5 consumers instead of 6 to 8, and refused on every other repetition; the trial's measurement, not the law, is the fault, and four trial rules for v4 follow from it. The legal wording on every page outside the frozen engine now reads "all rights reserved" and "all patents, copyrights and trademarks filed in the USA", followed by www.omni-compass.com and nothing after it. The repository now keeps itself current: after every finished live run the front-page workflow archives it and `tools/front_page.py` rebuilds every table whose three runs are in, the pages read from the tables, and the README's latest lines (`results/live/FRONT_PAGE_STATE.json` records what each table was built from).

**Everything run again (10 October 2026, 01:24 to 01:31 UTC).** At the founder's order that every result be as of today, every benchmark with a result was dispatched again on the current commit `3aac0ab7` (Omni v3, digest `b53d05449ee04c4b`, by `tools/omni_version.py --commit`), native and omni, the wiring of each checked against its preregistration before the dispatch: 70 GitHub runs (the seven Kubernetes tests, the five live stacks on the brain's own verdict, the four simulators, the realms table and the kill and long robustness scenarios, three runs each; the six organisms with the real cluster inside; the modelled grid at 1, 10, 100 and 1,000 copies) and the two big organisms at 1,000 copies on rented Azure machines, plus one replacement run for a Kafka job GitHub's runner cut off (the record says which and why). By 12:30 UTC 69 of the 71 had landed and every table on this page was remade from them; the tables they supersede are whole in `docs/history`. The one shard the runner's six-hour limit cut off (the tower at 1,000 copies in the modelled grid) runs again, and that row reads the earlier run until it lands. The record, run by run with inputs, times and ids, is `docs/RERUN_2026-10-10.md`. Not restarted, and why: the three 24-hour

robustness machines (started 8 October on the same harness, byte for byte, ending about 10:15 UTC today) and the Azure fleet that can show one machine, which waits on the family allowance those machines hold.

## In one paragraph (2026-10-10, Omni v3, every table remade)

**The engine is Omni v3, frozen and fingerprinted** (`OMNI_V3.json`, `docs/OMNI_V3.md`: v1's law and controllers byte for byte, 945 muscles in 59 families, a do-no-harm gate on speed knobs). Every benchmark runs three times as separate GitHub runs on it (A the result, B and C the replications) and every judged row reads better in all three of our runs, worse in all three of our runs, no difference beyond the noise, or the runs disagree (`docs/OMNI_V1.md`, the rule). On 10 October every benchmark was run again on one commit (`3aac0ab7`) and every table below was remade from the new runs; the tables they supersede are whole in `docs/history`. **Real Kubernetes, seven tests, ten pairs each, all seven the same again in all three of our runs on v3** (`results/live/V3_*.md`): work inside the response line **+54% to +68%** in all three runs of the all-four test; the 95th percentile **−57% to −66%** in every run of the steady, wandering, all-four and fault tests; failed requests −11% to −18% where load swings and under faults; machines −1.7% to −2.0% at steady load and −16% to −20% on the batch queue (−26% to −33% after the queue finished), with the standby-model energy −11% to −14% there; under the Google trace of 2011 machines −7% to −9% and p95 −64% to −67%; beside a noisy neighbour no difference beyond the noise on every row. **The five live stacks ran for the first time with the brain's own verdict on the knob** (the amendments of 9 October: every knob starts in watch and is written only inside the allowance a paired trial on the stack itself has earned under the index's own reading). What they read. **PostgreSQL** (`results/live/V3_PGBENCH.md`): connections held open **−6% to −11% on** `select`**, better in all three of our runs**; the add above the operator's 20 on `simple_update` refused, as expected; everything else inside the noise; no loss anywhere. **MySQL** (`results/live/V3_SYSBENCH.md`): the pool held **−10% to −20% on** `read_write`**, better in all three of our runs**, the pages holding data with it; the earlier set's memory bought for the written working set is gone; the other three workloads inside the noise. **MongoDB** (`results/live/V3_YCSB.md`): the cache held **−3% to −13% on** `f`**, better in all three of our runs**; the other three inside the noise; nothing worse. **Kafka** (`results/live/V3_KAFKA.md`) and **Redis** (`results/live/V3_REDIS.md`) read exactly nothing: no gain and no loss, every gauge inside the noise under both objectives, because every spend trial on Kafka (36 of 36 arms) and most on Redis were abandoned before they could be judged. **That is our wiring, found on the first set and amended the same day** (amendment 3 in all five preregistrations): the rule of 9 October ended a spend trial when the service turned calm, and a spend that works calms the service within seconds, so no spend could ever be judged; from amendment 3 a spend trial runs to its samples, and the five stacks run again on it, those runs becoming the result of record when they land. **The Omni index on the tables as they stand: +4.8%** (`results/OMNI_INDEX.md`: Kubernetes +28.9% over seven tests, PostgreSQL +0.8%, Kafka +0.1%, Redis exactly nothing, MongoDB +0.7%, MySQL +1.0%, each category weighed the same); the service reading **+10.7%** (Kubernetes +84.5%, the five stacks nothing). The number is down from +20.5% because the brain refused, or could not yet judge, the spends that bought the earlier gains at a resource cost: every gain left is one the brain proved on the stack itself, and no row anywhere is a loss. **Robustness**: the governor killed outright, every setting handed back within the allowance in every repetition of every run, the whole window's p95 −27% to −48%; the long run, no leak, every decision valid, no slowing, the service inside the noise in two of three runs. **The six organisms with the real cluster inside**: the cluster's p95 lower in all 12 cells (−1% to −52%), failed requests lower in 11. **The realms**: the published table reproduced in 3 of 3. **The simulators** read as their earlier sets (they are deterministic): power grids 67 gauge-rows better and 14 worse, the districts 71 better, 33 worse and 2 where the runs differ, the robot arms 7 better with Gen3 moved and UR5e and iiwa left native, the drone swarms 9 better and 0 worse. The v1 tables stay as the first engine's record.

**What is shown and what is not.** Shown: more work inside the response line on the same machines and a faster tail, on real Kubernetes, three times on a frozen engine, and again on 10 October; a real database, a buffer pool and a

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

storage-engine cache each giving memory back for the same service, every notch proved on the stack first; a message queue and a cache on which the brain's first set could not yet judge a spend (amendment 3, the next set running). Not yet shown: an energy or cloud-bill saving on real machines. Energy on kind is a declared model (the machines are containers on one runner); Azure's bill on a 4-worker fleet read no difference beyond the noise on every gauge (`results/live/V1_AKS_STEADY.md`, `V1_AKS_BURST.md`), which is a fleet too small to show one machine; the fleet of 40 workers in nine machine families that can show one is preregistered and dispatched (`docs/K8S_COMPASS_PREREGISTRATION.md`, the fleet that can show one machine, amendments 1 to 3), waiting on Azure's own capacity in eastus. The real card has not run on the current governor; every earlier card result is obsolete and is run again by the founder on rented cards.

**Wire in, or watch (9 October).** The founder's order: Omni need not be wired into every muscle; where it cannot beat native, the muscle stays native and Omni only reads it. `docs/WIRING_VERDICTS.md` (`tools/wiring_verdicts.py`, checked by `verify.py` against every table it reads) gives every knob in every result one of three words. Real stacks, on the sets of 10 October: 11 of 24 knob-cases **write** (six of the seven Kubernetes demands, PostgreSQL's `select`, MySQL's `read_write`, MongoDB's `f`, and Kafka's `heavy` and `burst` on a fraction of a consumer); 0 are **the operator's choice**, because the brain's verdict took no spend the index would read as a trade; 13 **watch** (Kubernetes beside a noisy neighbour, PostgreSQL's `simple_update` and `tpcb_hot`, Kafka's `light`, Redis's three, MongoDB's `b`, `c` and `burst`, MySQL's `burst`, `read_only` and `update_index`: nothing moved the same way in all three of our runs). No real stack carries a loss in all three of our runs on these sets. The simulators: 21 write, 15 trades (the four SimBench grids that export, where the lower voltage costs line losses; the seven 2023 CityLearn districts, where the bill is worse), 3 watch (two robots the engine itself left native, one grid where nothing was bought). The 945 modelled muscles: 124 write, 13 trades, 808 watch, 746 of them because the native controller kept the reading in band and the law never wrote; all 255 admission muscles are among the 746. Of the 12 organism cells with the real cluster inside 12 write on the cluster's gauges (one with a replica cost beside its gains, one watching). The engine is unchanged; the page decides what an operator connects.

**The brain's own verdict, live (built 9 October; the first set landed 10 October; the second set, on amendment 3, running).** The founder's order of 9 October: the verdict must be the brain's, in real time, on the knob, before it writes; a knob that cannot prove it pays stays native. `tools/knob_verdict.py` wraps the frozen engine's verdict (`omnicompass/verdict.py`, unchanged) around every live knob of the five database, messaging and cache harnesses: the knob starts in watch, a paired trial on the stack itself allows one notch at a time under the declared objective (resource, the index's reading, or service), a refused step is not taken, a trial holds the knob, a fail-up never spends beyond the allowance, the operator's setting is always free. Declared as amendments in the five preregistrations with the expectation written before the runs: Redis left native under the resource objective (the first notch refused), Kafka allowed step by step while each consumer pays, PostgreSQL's add above the operator's setting refused on simple_update, MySQL's read_write chunks refused, MongoDB's cache given back a notch a trial. The 21 runs (the five stacks under the resource objective; Redis and Kafka also under the service objective; three runs each) were dispatched on 10 October at 01:28 to 01:29 UTC on commit `3aac0ab7` (`docs/RERUN_2026-10-10.md`) and landed by midday: PostgreSQL, MySQL and MongoDB gave memory back where it paid (connections −6% to −11% on `select`; the pool −10% to −20% on `read_write`; the cache −3% to −13% on `f`) and lost nothing; Kafka and Redis read nothing, because every spend trial on Kafka and most on Redis were abandoned before they were judged: the rule ended a spend trial when the service turned calm, and a spend that works calms the service within seconds. Amendment 3 (all five preregistrations, 13:05 UTC) lets a spend trial run to its samples; the five stacks run again on it, and the result of record is that set. Where the brain's verdict stands today, knob by knob (the five stacks both ways; the cluster's machines, the card's clock, the robots' speed and the drones' cruise by the engine's own verdict; the grids, the districts and the 945 modelled muscles by the law alone), is the last table of `docs/WIRING_VERDICTS.md`. Inside the modelled realms the organism still takes one directive for


© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

every muscle; the verdict per muscle there is the next engine, Omni v4, ordered on 10 October and designed in `docs/OMNI_V4_PLAN.md` as the collective mechanism (one body, one brain); it is built next and every result is run again on it.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

# Measured on real systems (evidence class L), Omni v3

| Test, ten pairs × three runs | Work | Speed (p95) | Machines | Energy (declared model) | Source |
|---|---|---|---|---|---|
| **Newest: on top of KEDA, Kubernetes with its add-ons** (`docs/ADDONS_PREREGISTRATION.md`): KEDA on the CPU target and the requests in flight | service CPU +24% to +32% (closed-loop load: work shown, not judged); failed requests no difference beyond the noise | **−51% to −53%** (time over the line −97% to −100%) | no difference beyond the noise | idle-power model **+1.0% to +1.6%, WORSE in all three of our runs**; standby model inside the noise in 1 of 3 runs | `results/live/V3_KEDA_CPU_REQUESTS.md` |
| **Newest: on top of KEDA's HTTP add-on** (the requests in flight alone) | service CPU +45% to +54% (shown, not judged); failed requests no difference beyond the noise | **−73% to −76%** (time over the line −96% to −97%) | **−5.9% to −7.8%, better in all three of our runs** (replicas −14%) | idle-power model **+1.4% to +2.6%, WORSE in all three of our runs**; standby model −1.6% to −4.2%, inside the noise in 1 of 3 runs | `results/live/V3_KEDA_REQUESTS.md` |
| All four in one run | **+54% to +68%** | −57% to −63% | inside the noise | inside the noise | `results/live/V3_ALL_FOUR.md` |
| Steady work in steps | equal by design | −64% to −66% | **−1.7% to −2.0%** | −1.3% to −1.8% | `results/live/V3_STEADY.md` |
| Demand that wanders | failed requests −12% to −13% | −57% to −64% | inside the noise | idle-power model −0.1% to −0.4% | `results/live/V3_WANDERING.md` |
| A public day of demand: the Google cluster trace of 2011 replayed one step at a time (`docs/TRACES_PREREGISTRATION.md`) | failed requests **−11% to −18%**, better in all three of our runs; no pod ever without a machine | **−64% to −67%** (p99 −60% to −62%) | **−7% to −9%, better in all three of our runs** (replicas −4% to −8%) | standby model **−5% to −7%, better in all three of our runs**; idle-power model −0.3% to −0.4% | `results/live/V3_TRACE_GOOGLE2011.md` |
| Faults: machine down, spike, runaway pod, blind probe | failed requests **−14% to −18%**, better in all three of our runs | −58% to −63% | inside the noise | inside the noise | `results/live/V3_FAULTS.md` |
| A queue of batch jobs | queue finished no difference beyond the noise | mean response −11% to −13% | **−16% to −20%** (−26% to −33% after the queue) | **−11% to −14%** | `results/live/V3_BATCH.md` |
| Fairness, a noisy neighbour | inside the noise on every row | inside the noise | inside the noise | inside the noise | `results/live/V3_FAIRNESS.md` |
| PostgreSQL behind PgBouncer, three workloads (the set of 10 October, the brain's verdict on the pool) | inside the noise | inside the noise | connections held open **−6% to −11% on `select`, better in all three of our runs**; `simple_update` and `tpcb_hot` inside the noise (the add above the operator's 20 refused, as expected) | host CPU inside the noise on all three | `results/live/V3_PGBENCH.md` |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Test, ten pairs × three runs | Work | Speed (p95) | Machines | Energy (declared model) | Source |
|---|---|---|---|---|---|
| Apache Kafka, a consumer group's size, three workloads (the set of 10 October, the brain's verdict on the count) | inside the noise | inside the noise | consumers 2 → 1.98 to 1.99, better in all three of our runs by a fraction (the trial's reference seconds); every spend trial abandoned unjudged, the group left at the operator's two: amendment 3, the next set running | inside the noise | `results/live/V3_KAFKA.md`, `results/live/V3_KAFKA_SERVICE.md` |
| Redis, a cache's memory ceiling, three workloads (the set of 10 October, the brain's verdict on the ceiling) | inside the noise | inside the noise | the allowance one notch each way (56 to 72 MB), most trials abandoned unjudged, every gauge inside the noise: amendment 3, the next set running | inside the noise | `results/live/V3_REDIS.md`, `results/live/V3_REDIS_SERVICE.md` |
| MongoDB under YCSB, a database's storage-engine cache size, four workloads (the set of 10 October, the brain's verdict on the cache) | inside the noise on all four; no failed operation | inside the noise on all four | cache size held **−3% to −13% on `f`, better in all three of our runs**; `b`, `c` and `burst` inside the noise | inside the noise | `results/live/V3_YCSB.md` |
| MySQL under sysbench, a database's buffer pool size, four workloads (the set of 10 October, the brain's verdict on the pool) | inside the noise on all four; no error | inside the noise on all four | pool held **−10% to −20% on `read_write`, better in all three of our runs**, the pages holding data with it; `burst`, `read_only` and `update_index` inside the noise; handed back on every omni arm | inside the noise | `results/live/V3_SYSBENCH.md` |
| Robustness: the governor killed outright at 40% of the window, 10 pairs × 3 runs | **every setting back within the allowance in every repetition of every run**; a second governor to the end; failed requests inside the noise | the whole window p95 −27% to −48%, mean response −33% to −40%, time over the line −32% to −36%, better in all three of our runs | inside the noise | inside the noise | `results/live/V3_ROBUST_KILL.md` |
| Robustness: the long run, 7,200 s an arm, 3 pairs × 3 runs | decisions valid in every repetition; failed requests inside the noise | inside the noise in 2 of 3 runs (better in all three of our runs in B) | inside the noise; **no leak** (memory under its limit in every repetition); no slowing; every setting handed back | inside the noise | `results/live/V3_ROBUST_LONG.md` |
| The six organisms with the real cluster inside, 10 and 100 copies, 5 pairs a cell (the set of 10 October) | the organisms' work unchanged | the cluster's p95 lower in all 12 cells (−1% to −52%); failed requests lower in 11 | 6 in both arms (no autoscaler under kind) | the organisms' energy lower in every cell | `results/live/V3_SIX_KUBE.md` (v1 at 1 to 1,000 copies: `V1_SIX_KUBE.md`, `V1_BIG_ORGANISM.md`) |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

## Measured on a real cloud (Azure, its own bill), Omni v1

| Test | Reading | Source |
|---|---|---|
| Steady load, 4 workers, 5 pairs | no difference beyond the noise on any gauge; the bill −4.7% with its interval across zero | `results/live/V1_AKS_STEADY.md` |
| A burst sized to the cluster, 4 workers, 5 pairs | the bill +5.0% with its interval across zero; p99 −34% clear of the noise in this one run; the rest inside the noise | `results/live/V1_AKS_BURST.md` |
| The fleet that can show one machine (40 workers, nine families) on v3 | dispatched; two earlier dispatches refused by the subscription's family allowances before any arm ran, the third by Azure's own cluster capacity in eastus; every refusal cost cents and is recorded | `docs/K8S_COMPASS_PREREGISTRATION.md` |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

# Modelled (evidence class S: our own models, never counted in the headline), Omni v3

| What | Reading | Source | | | |
|---|---|---|---|---|---|
| The 945 muscles and six organisms, A/B/C | reproduced to the digit in 3 of 3; 0 muscles worse; every organism superior within guardrails (work per energy +0.1% to +0.3%) | `results/realms/REALMS.md` | | | |
| The whole tower at 1,000 copies (945,000 modelled muscles) with the real cluster inside, on a rented machine, 3 pairs (v3) | the organism's work unchanged (rounding-level) | the cluster's p95 **6.7 s → 160 ms**, p99 −98%, time over the line **76% → 0**, failed requests **4.8% → 0** | 6 machines in both arms | inside the noise (standby model); the organism's energy −0.4% (model) | `results/live/V3_BIG_ORGANISM.md` |
| The four stacked at 1,000 copies with the real cluster inside, on a rented machine, 3 pairs (v3) | the organism's work unchanged (rounding-level) | the cluster's p95 **2.9 s → 150 ms**, p99 −97%, time over the line **51% → 0**, failed requests **0.14% → 0** | 6 machines in both arms | inside the noise (standby model); the organism's energy −0.4% (model) | `results/live/V3_BIG_ORGANISM.md` |
| The organisms at 1, 10, 100 and 1,000 copies × 1 to 1,000 runs, 84 of 90 cells | every organism superior within guardrails in every cell of 10 runs or more; work per energy +0.07% to +0.37%, the same figure at every size; the six cells left are beyond the machines available | `results/scale/GRID.md` | | | |
| Power grid, 11 SimBench grids in pandapower, A/B/C (v1 and v3 identical) | with ZIP loads energy drawn and net import better in all 11; losses better in 7, worse in 4; tap operations fewer in 10, 4 → 8 a year in one (worse, the declared cost) | `results/live/V3_PANDAPOWER.md` | | | |
| Robot arms, MuJoCo Menagerie, A/B/C | Gen3 and Panda: peak torque −29% and −10%, tracking error −21%, energy per takt −0.8% and −0.5%; the Panda's copper +14% worse; UR5e and iiwa left native | `results/live/V3_MUJOCO.md`, `V1_MUJOCO_PANDA.md` | | | |
| CityLearn, every district, A/B/C | electricity bought, peak and unevenness better in all 11 battery districts, carbon in 8; the bill worse in 7, ramping worse in 7 | `results/live/V3_CITYLEARN.md` | | | |
| Drone swarms, gym-pybullet-drones, 20 drones in three cells, A/B/C | energy a mission −7% to −20% and missions a charge +8% to +25%, better in all three of our runs; no late mission, reserve breach, near miss or collision in any arm | `results/live/V3_SWARM.md` | | | |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## The card (evidence class P)

Every earlier card result ran on a controller since replaced and is obsolete. The one-card, card-inside-the-organisms and eight-card runs are the founder's, on rented cards, after the CPU and cloud work, at one named commit (`docs/GPU_RUN_GUIDE.md`, `docs/GPU_PREREGISTRATION.md`).

## Verified in code

| Property | Where |
|---|---|
| The canonical engine is `symmetric_verified`; the printed chart is a named variant, not benchmarked | `docs/CANONICAL_ENGINE.md` |
| Nine laws twinned in C++20 and proven equal to the Python; sealed by fingerprint | `results/SEAL.json`, `tools/seal.py` |
| The conveyance law conserves its budget and converges (proof and 20,000 random systems) | `docs/CONVEYANCE_LAW.md`, `tests/test_conveyance.py` |
| Safety shield: 2,000,000 adversarial cases, 0 violations; C++ engine: 100,000,000 decisions, no failures | `tests/test_shield_properties.py`, `results/SOAK.json` |

## Open

1. **Azure, the fleet that can show one machine**: steady and burst on 40 workers, v3; waiting on Azure's cluster capacity in eastus (the only region where this subscription has more than 10 cores). Then B and C.
2. **The four stacked and the tower at 1,000 copies with the real cluster inside, on v3: both done, 3 of 3 on the clock** (`results/live/V3_BIG_ORGANISM.md`). The stack: the cluster's p95 2.9 s → 150 ms, time over the line 51% → 0, failed requests 0.14% → 0, machines and energy unchanged, 22 s behind a 10,800 s window. The tower (945,000 modelled muscles, collected 9 October by run 37945866426 after 25 hours on its rented machine): the cluster's **p95 6.7 s → 160 ms (−98%)**, p99 9.1 s → 187 ms, **time over the line 76% → 0, failed requests 4.8% → 0**, all clear of zero; replicas, pods started and machines (6 in both arms) the same; energy inside the noise; 14 s behind the window in both arms; the organism's own model (shown, never in the index) energy −0.4% and work rounding-level worse by rule. Both machines deleted.
3. **The real card**: the founder's runs on Lambda, one exact commit.
4. **Robustness** (`docs/ROBUSTNESS_PREREGISTRATION.md`, no engine file changes): the kill scenario is done (`results/live/V3_ROBUST_KILL.md`: every setting back 7 to 11 s after the kill in 30 of 30 repetitions) and the governor's own CPU is tabulated (`results/live/V3_OWN_COST.md`); the long run is done (`results/live/V3_ROBUST_LONG.md`, 3 pairs × 3 runs, 7,200 s an arm: no leak, the decisions valid, no slowing, every setting handed back at the end; service inside the noise in two of three runs); the 24-hour run on three rented machines (scenario 2b, started 8 October 10:15 UTC) ends about 10:15 UTC on 10 October and is collected by the scheduled collect; the kill and long scenarios were dispatched again on 10 October (`docs/RERUN_2026-10-10.md`).
5. **YCSB on MongoDB, done** (`docs/YCSB_PREREGISTRATION.md`, `results/live/V3_YCSB.md`): the operator's WiredTiger cache as native, Omni on the cache size through the server's own console; four smoke runs recorded, then A, B and C on v3; the cache held given back by about half on two untouched workloads at no measurable cost in work, p95 or CPU, one loss in all three of our runs (the burst mean latency, +2% to +4%) that the second counted set of 9 October removed (item 6); the index spans six real categories at +20.5%. Cassandra and Redis under YCSB and HammerDB are next in row 24.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

6. **The costs in the database tables, traced and amended with the engine locked** (8 October; `docs/POSTGRES_PREREGISTRATION.md` amendment 2, `docs/MYSQL_PREREGISTRATION.md` amendment 2, `docs/YCSB_PREREGISTRATION.md` amendment 1; Omni v3 before and after, the harnesses are outside the fingerprint). PostgreSQL's +14% to +28% host CPU was our harness launching a `psql` process for every reading (measured: 52 ms of CPU a launch, 7,956 console logins in one workload's pooler log, +55.9 CPU-s on the launches against a +52.8 s host difference on a metered repetition; the pooler +1.2 s), and its +15% to +17% median latency was the pool shrunk into a queue the pooler itself reported: one console connection an arm and the queue line (a server taken back only while clients waited under 1% of the pooler's time, servers added back one per percent of waiting, up to the operator's 20). MySQL's read_write pool was bought for slow writes the pool cannot mend (36 of 82 grows at a miss share under 1%): the pool grows only while missing. MongoDB's memory saving on b, c and burst was a cold cache given back before its first eviction (four notches in the first five seconds of every arm) and held at the floor evicting: the give-back gate is the miss share, as MySQL's, and the saving may fall to the noise, said beforehand. The second (PostgreSQL, MongoDB) and third (MySQL) counted sets run on these rules, dispatched 23:16 UTC on commit `310cf318` (PostgreSQL 37858494179, 37858496620, 37858499033; MySQL 37858501997, 37858505059, 37858509109; MongoDB 37858512907, 37858515717, 37858519010). **Done, 9 October; the earlier tables are whole in** `docs/history`**.** PostgreSQL: CPU and median latency inside the noise on all three workloads, connections −36% to −38% on `select` better in all three of our runs, the runs disagree on `tpcb_hot`, and on `simple_update` the add rule of amendment 1 buys connections above the operator's 20 (most at once +72% to +80%, worse in all three of our runs, nothing bought): category +4.0% from +14.2%. MongoDB: the cache −21% to −36% on all four workloads, better in all three of our runs, the latency cost gone, 8 rows better and 0 worse: category +8.5% from +10.2%. MySQL: burst −49% to −56% better in all three of our runs; read_write memory +44% to +52% worse with CPU −4% to −6% better, a trade; read_only inside the noise (its earlier saving was in part the cold-start give-back); update_index disagree: category +5.2% from +12.2%. The index headline moves from +24.1% to **+20.5%**: smaller, and every number in it is now a gain without a harness cost under it.
7. **The queue** (`docs/REGISTER.md` section 4, `docs/PROOF_PROGRAM.md`): drone swarms on PX4 and ArduPilot (gym-pybullet-drones done), YCSB on Cassandra and Redis and HammerDB, Spark, OpenSearch, fio, Open-RMF, the 24-hour robustness run, Basilisk, Orekit and GMAT, RocketPy, Cantera (Kafka and Redis done); one or two at a time, each preregistered.
8. **The gaps the tree shows (the founder's reading, 10 October).** The index's six real categories are the cluster, PostgreSQL, Kafka, Redis, MongoDB and MySQL; Azure waits on its three-run close (item 1) and the card on its run on the current controller (item 3); the power grids and the districts are in as modelled results, never in the index by rule. Not yet a category: **a second and a third cloud**, AWS and Google Cloud under Azure's method (`docs/REGISTER.md` row 38; each needs its own account credential in a GitHub secret and an allowance of about 90 cores, about $12 a steady and $22 a burst run, nothing spent until the account exists); **CPU power through Linux's own governor with the RAPL meter**: **built and preregistered the same day** (`docs/CPU_POWER_PREREGISTRATION.md`; `tools/run_cpu_power.py`, one command `scripts/cpu_power_run.sh`, workflow `cpu-power`, the three-run table `tools/cpu_power_abc.py`): the kernel's governor stays native, Omni moves the frequency ceiling on top inside [half the top clock, the top], every notch down tried on the machine first by the brain's verdict, energy from the processor's own meter and a wall plug where fitted; it waits for a machine on the metal, the founder's own tower or laptop on Linux (free) or a rented bare-metal server (about $10), because GitHub's machines are virtual and expose neither the governor nor the meter (the workflow's probe says so on every dispatch); **the card's energy meter**, which the card harness already has (the integral of the device's power draw, the CPU package by RAPL, the wall plug where

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

fitted, `tools/gpu_reps.py`) and which reads on the founder's run. **The founder's sweep of the same day** (nothing with value left out: operating systems, television and telecom, satellites and signals, energy, oil and minerals, finance, the platforms and social media, science and physics, robotics to the top) added rows 39 to 57 to the register's queue and section 3.5 to the proof program, each an open benchmark with a shipped controller for native and one knob for Omni, each to be preregistered before its first run; aerospace and rockets were already rows 32 to 36. **The second check the same night** (cars, self-driving, batteries, nuclear, power plants, medicine, surgery named) is written as `docs/COVERAGE_MAP.md`: every domain in the world of controllers against the register, with the abbreviations (AKS, CPU, UPS, GPU, AWS and the rest) each given its place; it added rows 58 to 73 (self-driving stacks, the EV powertrain, mobility fleets, stream processing, storage clusters, AI batch admission, games, thermal plants with nuclear as the model only, fabs, surgical robot servos, the other Kubernetes scalers, distributed SQL, the other brokers, CDN caches, air traffic, observability pipelines) and wrote the exclusions by rule: safety reserves, clinical dosing, trading decisions, weapons, a person's command, solvers without a controller.

9. **Omni-Compass 1.0**: when the founder declares the engine final, v3 as it stands is published as 1.0 and the older fingerprints go to `docs/history` as the road to it.

## Where things are

| Path | What it is |
|---|---|
| `docs/OMNI_COMPASS_MANUAL.md` (PDF: `docs/OMNI_COMPASS_MANUAL.pdf`) | the manual: the governor, its mechanism, the wiring stack by stack, the frozen engines, the three-run rule, every result |
| `docs/RERUN_2026-10-10.md` | every benchmark dispatched again on 10 October 2026 on one commit: the record, run by run |
| `docs/OMNI_V3.md`, `docs/OMNI_V1.md` | what each engine is and every result read on it |
| `docs/REGISTER.md`, `docs/PROOF_PROGRAM.md` | every muscle, every benchmark run and still to run; the program to full size |
| `results/live/V3_*.md`, `V1_*.md`, `results/live/raw/` | the three-run tables and every archived run's files |
| `results/OMNI_INDEX.md` | the one combined number |
| `docs/BENEFIT_SHEET.md` | one number per benchmark, plus always good for Omni and minus always bad, whatever the gauge measures; yes, no, none or trade beside it |
| `docs/WIRING_VERDICTS.md`, `results/WIRING_VERDICTS.csv` | which knobs earn a wire in: write, watch or the operator's choice, per knob and per muscle, from every table, every loss with its cause |
| `docs/INTEGRATION_MANUAL.md`, `docs/WIRING_GUIDE.md` | wiring it in yourself |
| `docs/METRICS_CATALOG.md` | every gauge, and whether it is measured or modelled |
| `omnicompass/`, `omni_controller/`, `realms/`, `cpp/` | the engine, the controllers, the muscles, the C++20 twins |
| `docs/HISTORY.md` | earlier states of play, kept whole |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

PART EIGHT

# Value, License and History

*What a receipt is worth, how the license is priced against it, how the code is sealed, and how Omni-Compass came to be.*

CHAPTER 56

# Where the Value Comes From

## The problem we are attacking

Computing has run into a wall that is not made of silicon. Data centers cannot get the power they need, cannot get it fast enough, and are running out of room to put more machines. AI made it worse: every new model needs more chips, and every chip needs more electricity and more cooling. The answers on the table are expensive and slow: new chips that do more work per watt (a new generation, new hardware to buy), new power plants and power contracts (billions, and years), and point fixes at a single layer (an AI serving trick, a scheduler, a power cap on one chip).

We attack it from the other side. Every system already running wastes room it paid for:

- GPUs boost to the top of their clock range and are knocked back by their own power limiter many times a second;
- Kubernetes keeps replicas and machines sized for the worst minute of the day;
- cooling runs colder than the heat requires; batteries hold more reserve than the hour needs.

That room is paid for in energy, in machines and in floor space, and it produces nothing.

Why the room is there is worth a paragraph, because it decides what can be done about it. Each layer of a stack is sized by someone who cannot see the others. The autoscaler's author does not know what the power cap will do; the cooling engineer sizes for the hottest hour of the year; the database operator sets a buffer pool for the largest table and leaves it there; each of them, sensibly, leaves a margin against the layer next door, and the margins add up. None of these people is wrong. The waste is not a mistake anyone made; it is the price of layers that cannot talk to each other, and it is why a fix at one layer, however good, moves the margin rather than removing it. The benchmarks in this manual are each a measurement of one such margin on real software: the connections a pooler holds open that no client is using, the memory a cache keeps for data it will not be asked for, the machines a cluster keeps in service for a minute that has passed. Each is small on its own and each is paid for every hour of the year.

The attack, then, is not a better autoscaler or a better cache. It is a governor that reads every layer's own gauge, keeps each layer in the middle of its own band, and gives the margin back where the layer's own controller shows it is not needed, with the layer's own controller still underneath and the layer's own setting restored when the governor is gone. That is a different kind of thing from a point fix, and it is why it stacks on top of every point fix a data center has already made rather than competing with any of them.

## What Omni-Compass is, in one paragraph

Omni-Compass is software that sits **on top of** the controllers a system already has (the GPU's firmware, the Kubernetes autoscaler, the building's thermostats) and governs every layer under one law: a bounded push and pull that keeps each service in the middle of its band instead of far below its limit. It never replaces the native controller, never asks for more than the native settings allow, hands every knob back the moment it is switched off or loses a sense, and refuses any step that measures more than 2% worse on anything. No new hardware. It stacks on top of every other fix a data center makes.

## The benefit: more work from what is already paid for

The room that was spent on nothing becomes one of two things, and a company chooses which:

- **More work for the same cost:** the same machines and the same watts carry more work; or
- **The same work for less:** fewer machines in service, fewer watts, a smaller bill.

They are the same gain read from two sides (3.4). The health side comes with it: faster answers, faster recovery from failures, nothing waiting, every knob returned, no human babysitting the stack.

The results to date (Chapter 43) show both readings on real software, and show that which one a stack gets is decided by the stack, not by Omni-Compass. On the real Kubernetes cluster the gain was taken as **more work, faster**: the cluster carried more requests inside its response line on the same machines. On PostgreSQL behind its pooler, on MySQL's buffer pool and on MongoDB's storage-engine cache the gain was taken as **the same work for less**: the same transactions answered inside the same line with fewer connections open, a smaller pool or a smaller cache, each notch proved on the stack first and the resource handed back at the end of every run. On Apache Kafka the earlier set had taken the gain as more work, faster, at the cost of three to four times the consumers, and on Redis the compass had bought the hit rate with three to four times the memory; under the brain's own verdict on the knob (10 October) neither spend was judged on the first set, because the trial rule of the day ended a spend trial on the calm the spend itself produced, so both read exactly nothing until the amended set lands (section 16.4). The combined index (section 16.6) is +4.8% over the six real categories on the sets of 10 October, with no loss in all three of our runs in any category. An operator should expect the same honesty from their own paired run: the receipts will say which side of the gain their stack took, and whether the compass's own cost (its CPU, its reads) ate into it.

## How to read the benefit: the arithmetic

Every comparison is paired: the same system, the same work, the same moments, native alone against native with Omni-Compass on top. Let

- $W$ be the work served (requests answered, tokens generated), held equal in both arms;
- $C$ be a resource it used: machine-hours, CPU core-hours, or joules.

**Productivity** is work per resource, $P = W / C$. With the work held equal, the gain from Omni-Compass on top is

$$G = P_{omni} / P_{native} - 1 = C_{native} / C_{omni} - 1.$$

A gain $G$ means the same resources can carry $G$ more work. Read from the other side, the same work needs only

$$C_{omni} / C_{native} = 1 / (1 + G)$$

of the resources, a saving of $S = G / (1 + G)$. So **a third more work (G = 33%) is a quarter off the bill (S = 25%)**; 20% more work is 16.7% off; 50% more work is a third off. Both columns are the same measurement.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

Three rules for reading any receipt:

1. **The work must be equal.** Every receipt states that the load was fixed-rate (open loop): the same requests at the same moments in every arm. Without that, a lower resource count could just mean less work was done.
2. **Nothing may be worse.** Beside every gain the receipt prints response times, failures, waiting and the controller's own CPU, each with its 95% interval. Our one rule (`DISCLOSURES.md`, Chapter 56): no measure more than 2% worse, and only where energy or the bill is saved. A gain paid for with worse service is not a gain.
3. **Know the class and the comparator.** A machine-hour gain on kind (where idle machines stay powered and native has no node autoscaler) is not yet a bill; a modelled energy line is not a meter. Section 14 lists the classes; Chapter 43 says, for every result, what it is measured against.

**Three worked examples, from the tables.** In the all-four Kubernetes test on v3, the same ten machines carried 35% to 49% more requests inside the response line: *G* is 0.35 to 0.49, so the same work would need 1/1.35 to 1/1.49 of the machines, a saving *S* of 26% to 33%, had the fleet been free to shrink; on kind it was not, which is why the table reports the work column and not a bill. In the database test, the pooler held 61% to 72% fewer connections open for the same work: the resource there is connections, *C_omni / C_native* is 0.28 to 0.39, and the gain read as "more work for the same connections" would be $G = 1.6$ to 2.6, which the table reports as the machines-equivalent column, "fewer connections by". In the Kafka test the work inside the line rose 16% to 21% while the consumers running rose by a factor of 2.6 to 4: there the gain in work was bought with resources, and the table reports both, the one as better in all three of our runs and the other as worse in all three of our runs, because that is what happened.

## Where the evidence stands today, read honestly

Every number below is from a three-run table on the frozen engine Omni v3, with its evidence class and the question a referee is entitled to ask about it. The answers to those questions are given, where they exist, in the row; where they do not yet exist, the row says what will supply them.

| What | Measured, same work, three runs | Class | What a referee will ask, and the answer |
|---|---|---|---|
| Kubernetes: work inside the response line, all four in one run | **+35% to +49%** better in all three of our runs | L | "Is the cluster real?" Kubernetes is real (API server, scheduler, HPA); the seven nodes are containers on one 4-core GitHub runner, so the host's own busy share is recorded beside every run and the machines column is machine-hours in service, not a bill. The bill is Azure's test (below) |
| Kubernetes: 95th-percentile response time, steady, wandering, all four, faults | **47% to 66% faster** better in all three of our runs | L | "Did native have a fair chance?" Native is Kubernetes with its HPA at the operator's target of 50%; the cost-to-match runs on the earlier engine showed no native target tried (40, 30, 20) reached Omni's p95 (`docs/history/`). The runs are paired on one runner with the order rotated |



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

| What | Measured, same work, three runs | Class | What a referee will ask, and the answer |
|---|---|---|---|
| Kubernetes: machines in service | −1.5% to −2.9% at steady load, **−19% to −23%** on the batch queue, better in all three of our runs; inside the noise on the other tests | L | "Why so little?" Because the verdict kept every machine whose removal made requests slower in the paired trial. Earlier engines without that check parked 29% to 36% of the machines and paid for it in response time; the frozen engine puts service first. kind has no node autoscaler, so parked machines stay powered: the saving is machine-hours, not watts |
| Kubernetes: energy | the standby model, −11% to −14% on the batch queue and −5% to −7% under the Google trace, inside the noise elsewhere | L, modelled | "Is it a meter?" No. The machines are containers; energy is a declared formula (a parked worker at 25 W standby), stated as a model on every line |
| Kubernetes: fairness beside a noisy neighbour | no difference beyond the noise on every row, the neighbour's included | L | "Does it hurt the other tenant?" Not measurably, in three runs of ten pairs |
| Kubernetes under a public day of demand (the Google cluster trace of 2011): the 95th percentile; machines in service | **64% to 67% faster**; machines **−7% to −9%**, better in all three of our runs in three runs; failed requests −11% to −18% | L | "Whose demand shape?" Google's, from a published trace, turned into the schedule by a rule written before the runs and receipted with every part's SHA-256 |
| Database behind its pooler: connections held open | **−6% to −11%** on the read-only workload, better in all three of our runs (the set of 10 October, the brain's verdict on the pool); the slow write workload's add refused | L | "What did it cost?" Nothing the same in all three of our runs: work, latency and host CPU inside the noise on all three workloads; the earlier sets' larger savings, and their costs, are whole in the history |
| Message broker: work inside the 500 ms line; the 95th percentile | on the set of 10 October exactly nothing: every spend trial was abandoned before it was judged and the group stayed at the operator's two consumers; the earlier set's +16% to +21% and 1.6 s → 9 to 14 ms, bought with 6 to 8 consumers, is whole in the history | L | "Why nothing?" Our trial rule ended a spend trial on the calm the spend produced; amended as amendment 3, the next set running |
| Database's storage-engine cache (MongoDB under YCSB): the cache size held | **−3% to −13% on f**, better in all three of our runs (the set of 10 October, the brain's verdict on the cache); b, c and burst inside the noise | L | "What did it cost?" Nothing the same in all three of our runs: work, p95, p99, mean latency and host CPU inside the noise on all four workloads, no failed operation |
| Database's buffer pool (MySQL under sysbench): the pool held | **−10% to −20% on read_write**, better in all three of our runs, the pages holding data with it (the set of 10 October, the brain's verdict on the pool); burst, read_only and update_index inside the noise | L | "What did it cost?" Nothing the same in all three of our runs: work, p95, p99 and CPU inside the noise on every workload; the earlier set's memory bought for the written working set is gone |
| Cache: work inside the 2 ms line; the hit rate | on the set of 10 October exactly nothing: the allowance one notch each way, most trials abandoned before they were judged, every gauge inside the noise; the earlier set's +14% to +27%, bought with a 64 → 200 to 270 MB ceiling, is whole in the history | L | "Why nothing?" The same trial rule as the broker's; amendment 3, the next set running |
| The modelled realms: work per energy, 945 muscles, six organisms, 1 to 1,000 copies | **+0.07% to +0.37%**, every organism superior within guardrails, 0 muscles worse | S | "Why so small?" The native controllers in the models are well tuned and leave little room; the number is small and real within the model, the same at every size, and it is never counted in the headline |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| What | Measured, same work, three runs | Class | What a referee will ask, and the answer |
|---|---|---|---|
| Power grid, 11 SimBench grids in pandapower | energy drawn and net import better in all 11; losses better in 7, **worse in 4**; tap operations fewer in 10, 4 → 8 a year in one, worse | S | "Where does it lose?" In the rural and semi-urban grids with their own generation, where a lower voltage raises losses; the table shows it |
| Robot arms, MuJoCo Menagerie | peak torque −29% and −10%, tracking error −21% where Omni moved; the Panda's copper loss **+14% worse**; two arms left native | S | "What about the arms that gained nothing?" The paired physics trial left them native and the table says "nothing for Omni to move" |
| Buildings with batteries, CityLearn, 11 districts | electricity bought, daily peak and unevenness better in all 11; the bill **worse in 7**; ramping worse in 7 | S | "Is the bill a loss?" Yes, in the 2023 districts, and it is in the table as such |
| Drone swarms, gym-pybullet-drones, three 20-drone cells | energy a mission −7% to −20%, missions a charge +8% to +25%; no late mission, reserve breach, near miss or collision | S | "Is the energy a meter?" No, a declared model from the simulator's own motor constants; the tracking error rose from 0.07 to 0.13 m inside its 0.25 m band and is shown |
| The governor's own cost, 1 to 1,000 copies with the real cluster inside | **0.006 to 0.013 of one core** at every size, 0.1% to 0.3% of the host's cores | L | "Does the brain's cost grow with the body?" No: it governs the real cluster, and that cost does not grow with the organism around it; shown, not judged, each run under its own engine (`results/live/V3_OWN_COST.md`) |
| The governor killed outright mid-run, 10 pairs × 3 runs | every setting back at the operator's **7 to 11 s** after the kill in 30 of 30 repetitions (mean 9 s; allowance 60 s); a second governor to the end in every one; the 120 s after the kill inside the noise against native | L | "What if Omni-Compass dies?" The watchdog hands back from the lease the governor left, within seconds, and the service does not measurably notice; the whole window with a kill and a restart in it still read faster and with fewer failures than native (`results/live/V3_ROBUST_KILL.md`) |
| Azure's bill on a 4-worker fleet (v1) | no difference beyond the noise on any gauge | L, a real bill | "Then where is the saving?" A fleet of 4 cannot show one machine; the 40-worker fleet that can is preregistered and waits on Azure's own cluster capacity in its region |
| The real card (GPU) | **obsolete**: every earlier card result ran on a controller since replaced | P pending | "When?" The founder's runs on rented cards at one named commit, after the CPU and cloud work |

The upside the real-software data points toward is a third to a half more work from what is already installed on the Kubernetes side, with faster answers, and a smaller, resource-priced gain on the stacks where the knob is a pool, a consumer group or a ceiling. It becomes a claim about the bill, not a direction, when two tests land: the bill on a real cloud with a fleet big enough to show one machine, and the real card's own meter. We publish each of them as it comes, whatever it says.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

## One law, frozen; and what came before it

The engine in every result of this manual is **the compass law with the verdict** (Chapter 17 and 1.2), frozen as Omni v1 on 5 October 2026, grown to the 945-muscle catalog as v2 and given the slack gate on speed knobs as v3, with the law, the controllers and the runners the same bytes in all three (`docs/OMNI_V1.md`, `OMNI_V2.md`, `OMNI_V3.md`). Earlier engines, among them an allocation law that gave back about a third of the machines at the cost of response time and a compass law without the verdict, produced the numbered sets kept in `docs/history/`; their results are history, labelled as the engine that produced them, and are never read beside v1's or v3's. A buyer does not choose between modes: there is one law, and it is the one every table in Part VI was made on. Each new result therefore describes exactly the code that produced the old ones, and when the founder declares the engine final, the engine that stands then is published as Omni-Compass 1.0, with the older fingerprints kept as the road to it, never as a second product.

"Frozen" has a precise meaning here. The engine is a fixed set of 40 files, and `OMNI_V3.json` holds one SHA-256 for each of them and one digest over all of them; `OMNI_V2.json` and `OMNI_V1.json` do the same for the two earlier sets. `python3 tools/omni_version.py` hashes the files in a checkout and prints which fingerprint they match (for this edition, `omni-v3 (digest b53d05449ee04c4b, 40 files)`), or, when they match none, which files differ from the nearest; with `--commit <sha>` it asks the same of any pushed commit, which is how every three-run table checks that its runs A, B and C were on the same engine before it combines them, and refuses if they were not. The three versions differ in exactly the ways their records name: v2 added the 945-muscle catalog and thirteen presets to v1 without touching the law, the controllers or the runners, which are the same bytes in all three; v3 added the slack gate on speed knobs and the marine propulsion preset to v2 (`docs/OMNI_V3.md`). Nothing else changed, and a reader can check that from the fingerprints rather than from this sentence.

The rule that any change makes the next version is also the rule that keeps the results honest. A gain or a guard changed after a result is in would make that result unrepeatable on the engine that now stands, so the change moves the version, every result is run again on the new version, and the old tables keep their version in their name and move to history. The founder's standing order is that the engine is never changed without being told first; this manual records no change made otherwise.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 57

# The Economics of a Receipt

Run the stack native and print the receipt. Run the same stack with Omni-Compass and print the receipt. The difference is the gain, read from either side: more work for the same energy, or the same work for less energy and fewer machines.

The license fee is set against the measured gain on the customer's own receipt. On a stack that costs one hundred, a measured gain of ten is worth ten; the fee is a share of that ten, and the customer keeps the rest. Percentages from different parts of a stack are not added into one bill: the invoice is one gain, measured once.

The value a customer sees comes in three forms, each on its own line of the receipt:

1. **Work per energy** - more finished work per kilowatt-hour.
2. **Capacity** - the same work on fewer machines in service; it becomes a cost reduction when the released machines are returned to the cloud or switched off by the platform around Omni-Compass.
3. **Service** - faster responses and fewer breaches at the same load.

The babysitting tax - the people and tools kept on the clock to set caps, answer pages and turn knobs back after a run or a crash - is the cost Omni-Compass removes by holding the knobs and returning them itself.

**The receipt's arithmetic, and the one number.** With the work held equal, a gain G in productivity is the same measurement as a saving S = G / (1 + G) in resources: a third more work is a quarter off the bill (manual, section 3.4). Across every real stack the one number is the Omni index (`results/OMNI_INDEX.md`): each test's work, speed, machines and energy as ratios, combined by geometric means so that a gain and an equal loss cancel exactly, each real category weighed the same, and only rows the same in all three of our runs counted. The index is honest in both directions by construction. As this edition is written it reads +30.2%: real Kubernetes +25.5%, the database +14.2%, messaging +166.9%, and the cache −24.9%, the last negative because the memory the governor holds for a wide working set is the resource it trades and reads worse by rule, even as the cache's work and hit rate read better. A buyer reads the category that matches their stack, and reads its losses in the same table as its gains.

**What a receipt is not.** A receipt is not a forecast. It is the measured difference between two arms on one system at one time, with its interval. The only number that applies to a customer's system is the one their own paired runs produce, and the method of those runs is the manual's section 13, the same method every table in this book was made with.

CHAPTER 58

# The Buyer Edition

> **Superseded for live results.** The live evidence below predates sets 19 and 20. The current live table is `results/live/LIVE_PAIRED.md`; energy on kind is a declared model, not a meter. The first metered test is `scripts/gpu_paired.sh` (`docs/GPU_BENCH.md`).

Commit cdc8cd9. Every number below is generated from result files in the repository by `python tools/buyer_report.py`. Simulated results use the repository's fleet plant; live results come from real Kubernetes (kind) in GitHub Actions. Competitors are reproduced from their public documentation, not their binaries.

## What Omni-Compass is

- **The engine:** a six-state control engine, stable by construction (every mode decays), in Python and C++.
- **The law it applies:** the closure law of the owner's manuscript.
  - It does nothing in the safe interior.
  - It corrects inward when the forecast approaches capacity.
  - It releases a machine only after the peak.
  - It keeps released machines warm for instant reuse.
- **What it governs:** machines, pod counts and sizes, cold start, batch pacing, containment of runaway agent workloads, and cooling setpoints.
- **Two ways to deploy it:**
  - **B, on top of the platform a customer already runs.** Nothing is replaced.
  - **C, alone.** Kubernetes stays only as the muscle; the scaling layer is replaced (HPA, Cluster Autoscaler, Karpenter, and optimizers such as CAST AI, Spot Ocean and Turbonomic).

# B: Omni-Compass on top of each platform (held-out scenarios, settings frozen first)

| Workload | Platform | Omni-Compass on top vs the platform alone |
|---|---|---|
| Web services | Kubernetes | scale reversals 12% better, machine starts+stops 2% better |
| Web services | OpenShift | scale reversals 44% better, machine starts+stops 4% better |
| Web services | GKE | scale reversals 21% better, machine starts+stops 7% better, machine-hours 3% better |
| Web services | AKS NAP / Karpenter | scale reversals 8% better, machine starts+stops 3% better |
| Web services | Turbonomic | scale reversals 8% better, machine starts+stops 2% better |
| Web services | CAST AI | slowest responses (p99) 34% better, scale reversals 30% better, machine starts+stops 15% better |
| Web services | Spot Ocean | slowest responses (p99) 42% better, scale reversals 34% better, machine starts+stops 17% better |
| Four clusters, one site | Kubernetes | scale reversals 93% better, slowest responses (p99) 51% better, machine starts+stops 21% better |
| Four clusters, one site | OpenShift | scale reversals 100% better, slowest responses (p99) 39% better, machine starts+stops 16% better |
| Four clusters, one site | GKE | slowest responses (p99) 38% better, scale reversals 19% better, machine starts+stops 11% better |
| Four clusters, one site | AKS NAP / Karpenter | scale reversals 7% better, slowest responses (p99) 6% better, machine starts+stops 4% better |
| Four clusters, one site | Turbonomic | scale reversals 4% better, machine starts+stops 2% better |
| Four clusters, one site | CAST AI | equal |
| Four clusters, one site | Spot Ocean | scale reversals 9% better, slowest responses (p99) 5% better, machine starts+stops 5% better |
| Batch jobs | Kubernetes | slowest responses (p99) 91% better, scale reversals 82% better, average response 25% better |
| Batch jobs | OpenShift | slowest responses (p99) 92% better, scale reversals 84% better, average response 26% better |
| Batch jobs | GKE | time over backlog limit 100% better, slowest responses (p99) 89% better, scale reversals 66% better |
| Batch jobs | AKS NAP / Karpenter | typical response (p95) 60% better, slowest responses (p99) 34% better, time over backlog limit 25% better |
| Batch jobs | Turbonomic | time over backlog limit 44% better, scale reversals 22% better, machine starts+stops 15% better |
| Batch jobs | CAST AI | time over backlog limit 100% better, typical response (p95) 94% better, slowest responses (p99) 91% better |
| Batch jobs | Spot Ocean | time over backlog limit 83% better, scale reversals 59% better, machine starts+stops 47% better |
| GPU training | Kubernetes | scale reversals 68% better, machine starts+stops 35% better |
| GPU training | OpenShift | scale reversals 61% better, machine starts+stops 29% better, time healthy 1% better |
| GPU training | GKE | scale reversals 59% better, machine starts+stops 39% better, machine-hours 1% better |
| GPU training | AKS NAP / Karpenter | machine starts+stops 51% better, scale reversals 33% better, machine-hours 2% better |
| GPU training | Turbonomic | scale reversals 12% better, machine starts+stops 9% better |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Workload | Platform | Omni-Compass on top vs the platform alone |
|---|---|---|
| GPU training | CAST AI | machine starts+stops 45% better, scale reversals 32% better, machine-hours 2% better |
| GPU training | Spot Ocean | machine starts+stops 28% better, scale reversals 21% better |

## C: Omni-Compass alone, one global setting, confirmatory run

- **Setting:** one global setting for every workload, frozen by SHA-256 before the run (28cbb82607cb0009).
- **Scenarios:** 100 never-used scenarios per workload (seeds 710001-710100).
- **Comparisons:** 364 cells (4 workloads x 7 platforms x 13 gauges).
- **Rule:** a cell counts as better or worse only if it survives Holm-Bonferroni correction across all 364 cells (family alpha 0.05) and exceeds the 0.5% practical tolerance. Otherwise it is equal.

**Result:** 153 better, 174 equal, 37 worse.

| Workload | Better | Equal | Worse | The worse cells |
|---|---|---|---|---|
| Web services | 41 | 41 | 9 | Kubernetes typical response (p95) 20.7%; Kubernetes machine starts+stops 24.5%; OpenShift typical response (p95) 20.8%; OpenShift machine starts+stops 66.0%; OpenShift scale reversals 152.0%; GKE typical response (p95) 20.1%; AKS NAP / Karpenter typical response (p95) 19.7%; CAST AI typical response (p95) 19.2%; Spot Ocean typical response (p95) 19.7% |
| Four clusters, one site | 45 | 36 | 10 | Kubernetes typical response (p95) 20.4%; Kubernetes machine starts+stops 23.7%; OpenShift typical response (p95) 20.5%; OpenShift machine starts+stops 62.1%; OpenShift scale reversals 207.3%; GKE typical response (p95) 19.7%; AKS NAP / Karpenter typical response (p95) 19.3%; Turbonomic typical response (p95) 27.6%; CAST AI typical response (p95) 18.8%; Spot Ocean typical response (p95) 19.3% |
| Batch jobs | 35 | 50 | 6 | Kubernetes machine starts+stops 39.0%; OpenShift machine starts+stops 29.5%; AKS NAP / Karpenter energy 2.1%; CAST AI energy 1.2%; Spot Ocean energy 2.2%; Spot Ocean machine-hours 0.6% |
| GPU training | 32 | 47 | 12 | AKS NAP / Karpenter energy 1.0%; AKS NAP / Karpenter time over power limit 1.6%; AKS NAP / Karpenter time over heat limit 1.1%; CAST AI energy 1.0%; CAST AI time over power limit 3.2%; CAST AI time over heat limit 2.6%; CAST AI time healthy 1.6%; Spot Ocean energy 1.2%; Spot Ocean time over power limit 3.8%; Spot Ocean time over heat limit 1.8%; Spot Ocean machine-hours 1.4%; Spot Ocean time healthy 2.0% |

| Workload | Energy vs Kubernetes | Machine-hours | Slowest responses | Average response |
|---|---|---|---|---|
| Web services | +21% | +38% | +40% | +11% |
| Four clusters, one site | +25% | +40% | +41% | +13% |
| Batch jobs | +6% | +18% | +40% | +2% |
| GPU training | +2% | +11% | +0% | +0% |

Positive means Omni-Compass is better.

## Real demand: 1,052 recorded machines (PlanetLab)

- **Losing cells:** Omni-Compass alone, with the frozen web setting (never tuned on these traces), loses 0 of 91 against the seven platforms.
- **Whole-pod packing:** with whole-pod packing it loses 0 of 91.
- **Against Kubernetes:**

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- energy +24%
- machine-hours +33%
- typical response +97%
- slowest responses +85%

Positive means better.

## Four-cluster sites as one body (held-out)

- **The mechanism:** traffic shift between clusters, plus the law run on the site total with one warm reserve.
- **Omni-Compass alone:** zero losing cells against GKE, AKS NAP / Karpenter, CAST AI, Spot Ocean.
- **Omni-Compass on top:** zero losing cells on all seven platforms.
- **Assumption:** services are replicated across the site's clusters.

## Under failure: the runtime protocol

Machines dying, load spikes, crash-looping services and noisy neighbours were injected at five stress levels, with identical faults for every system. Totals over all levels:

| Workload | System | Runs conveyed | Pages to a human | Recovery (min) |
|---|---|---|---|---|
| Web services | CAST AI | 231 of 500 | 404 | 6.6 |
| Web services | Kubernetes | 218 of 500 | 338 | 8.3 |
| Web services | Omni-Compass alone | 240 of 500 | 246 | 5.1 |
| Four clusters, one site | CAST AI | 44 of 500 | 608 | 5.7 |
| Four clusters, one site | Kubernetes | 44 of 500 | 330 | 6.5 |
| Four clusters, one site | Omni-Compass alone | 46 of 500 | 197 | 3.6 |

## The supervisory nervous system

- **One state, every organ:** `omnicompass/nervous_system.py` turns the engine's state (convergence, basin health, stress against its equation-6 equilibrium, unmet need) into one calm value between 0 and 1.
- **Authority from calm:** calm grants each organ its authority:
  - pods and machines may give capacity back only above their reversibility thresholds (0.5 and 0.7);
  - CPU frequency, GPU power, routing and cooling get envelopes that widen with calm;
  - batch is admitted or paused;
  - rollback is authorised.
- **Holds and the shield:** a security hold stops every capacity organ from expanding. The shield stays downstream and can still veto.
- **Invariants tested:** 300,000 random engine states, with zero violations (`tests/test_nervous_system.py`).
- **Coordination:** pods move first, and a machine move opposite to the pod move is vetoed. Controller contradictions per day (fighting, or a reversal within one boot time):
  - web: 2.10 to 0.10;
  - four-cluster: 5.30 to 1.03, below Kubernetes' 2.10.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

**Coordination, confirmatory on fresh seeds 712001-712100** (frozen first in `tuning/COORD_PREREGISTRATION.json`, same 364 cells, same Holm rule): 151 better, 173 equal, 40 worse.

| Workload | Better | Equal | Worse | Contradictions per day: Omni-Compass / Kubernetes |
|---|---|---|---|---|
| Web services | 40 | 41 | 10 | 0.07 / 0.52 |
| Four clusters, one site | 45 | 37 | 9 | 0.56 / 1.77 |
| Batch jobs | 35 | 49 | 7 | 0.11 / 0.00 |
| GPU training | 31 | 46 | 14 | 0.11 / 0.04 |

**Mechanism ablation** (`tuning/ABLATION.json`, fresh seeds 711001-711030): each part of the law removed in turn, against the full law. Listed: the gauges that get significantly worse (95% interval excludes 0, more than 0.5%).

| Part removed | Web services | Four clusters, one site | Batch jobs | GPU training |
|---|---|---|---|---|
| engine release gate removed | no change | no change | no change | no change |
| turning point removed | no change | machine starts+stops +2%, scale reversals +6% | slowest responses (p99) +32%, average response +12%, time over backlog limit +333%, machine starts+stops +47%, scale reversals +193%, contradictions +1350% | machine starts+stops +18%, scale reversals +118%, contradictions +1000% |
| muscle tone removed | typical response (p95) +1%, slowest responses (p99) +110%, average response +4%, machine starts+stops +92%, scale reversals +458%, contradictions +22% | slowest responses (p99) +8%, machine starts+stops +29%, scale reversals +108% | typical response (p95) +65%, slowest responses (p99) +151%, average response +26%, machine starts+stops +239%, scale reversals +1265%, machine-hours +1%, contradictions +875% | time over backlog limit +2%, machine starts+stops +297%, scale reversals +829%, contradictions +1150% |
| trend term removed | no change | slowest responses (p99) +2% | slowest responses (p99) +75%, average response +24%, time over backlog limit +533%, machine starts+stops +65%, scale reversals +348%, contradictions +2350% | machine starts+stops +29%, scale reversals +129%, contradictions +1100% |

## Live Kubernetes evidence

Repeated live runs, 5 paired repetitions per arm, probe through the Service (`results/live/LIVE_REPS_3.md`):

## Live repetitions, set 3: GitHub Actions run 36289557446 (commit 9892270), 5 × native / omni / strict on kind

This is the first set with the probe through the Service (NodePort via kube-proxy) and the PodDisruptionBudget, and the first live set with the supervisory nervous system gating machine release.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

## B: Omni-Compass on top vs native, 5 paired repetitions

| Gauge | Native | Omni | Change | 95% interval of the difference | Significant |
|---|---|---|---|---|---|
| worker nodes in service, mean | 6 | 6 | +0.0% | -3.155e-16 to +6.708e-16 | no |
| node-hours | 1.517 | 1.516 | -0.1% | -0.02125 to +0.01859 | no |
| energy (Wh) | 168.1 | 165.7 | -1.4% | -7.697 to +2.854 | no |
| response time (ms), mean | 221.7 | 210.3 | -5.1% | -57.38 to +34.55 | no |
| response time (ms), 95th percentile | 431.4 | 440.2 | +2.0% | -47.09 to +64.69 | no |
| response time (ms), 99th percentile | 568.9 | 609 | +7.0% | -51.68 to +131.8 | no |
| failed requests (%) | 0 | 0 | +0 (native is 0) | +0 to +0 | no |
| pending pods, pod-minutes | 0.3433 | 0.3967 | +15.5% | -0.4106 to +0.5172 | no |
| utilisation (used / allocatable) | 0.07676 | 0.06667 | -13.1% | -0.02607 to +0.005892 | no |

## C: Omni-Compass decides (strict) vs native, 5 paired repetitions

| Gauge | Native | Omni | Change | 95% interval of the difference | Significant |
|---|---|---|---|---|---|
| worker nodes in service, mean | 6 | 6 | +0.0% | -7.797e-16 to +7.797e-16 | no |
| node-hours | 1.517 | 1.518 | +0.0% | -0.00974 to +0.01041 | no |
| energy (Wh) | 168.1 | 166.4 | -1.0% | -4.867 to +1.447 | no |
| response time (ms), mean | 221.7 | 217.8 | -1.8% | -89.74 to +81.98 | no |
| response time (ms), 95th percentile | 431.4 | 463.2 | +7.4% | -105.7 to +169.4 | no |
| response time (ms), 99th percentile | 568.9 | 629 | +10.6% | -155.4 to +275.7 | no |
| failed requests (%) | 0 | 0 | +0 (native is 0) | +0 to +0 | no |
| pending pods, pod-minutes | 0.3433 | 0.8033 | +134.0% | +0.05385 to +0.8661 | yes, worse |
| utilisation (used / allocatable) | 0.07676 | 0.06928 | -9.8% | -0.01814 to +0.003173 | no |

## Reading

**Failed requests.** They are 0% in every arm. The 16-66% of sets 1-2 was the probe's hung tunnel, as diagnosed.

**Machines.** Omni-Compass kept all 6 machines in service in every repetition. Controller log: 15 of 15 decisions, 0 errors, no node-scale command.

- **The likely cause.** With a working probe, p95 sits at 430-460 ms against the 500 ms SLO declared before the run. Every window above 500 ms is SLO pressure. That pressure blocks machine release for the next 3 decisions and lowers the engine's calm.
- **Why that is odd here.** Machines are at 7% utilisation with no waiting pods. The latency comes from the workload's per-request compute, not from a machine shortage.
- **Why the earlier sets differed.** Sets 1-2 released machines because the hung probe produced few or no successful samples, so the controller saw little latency pressure. Their machine savings were therefore measured with a blinded latency sense and are not claimed.
- **How this will be settled.** The decision trail (p95, SLO state, calm, node authority per decision) is now printed in every job log. Set 4 will show which gate held.

**Everything else.** No significant difference from native on any gauge, except that C has more pending pod-minutes (0.80 vs 0.34 pod-minutes).



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

- **Earlier live sets 1 and 2:** these carried a probe defect, now fixed. The probe's one-pod tunnel hung when a drain moved its pod, so the Omni arms logged false failed requests (`results/live/LIVE_REPS_PROBE_DEFECT.md`).
- **Their machine savings are withdrawn.** The hung probe left the controller's latency sense nearly blind, so it released machines it would not have released with a working probe. Set 3 shows that: with a working probe and latency near the declared 500 ms SLO, Omni-Compass kept all 6 machines. The decision trail per run is now in every job log (set 4).

**Live levers under the nervous system:**

- **What acted:** right-sizing, cold start, batch pacing, agent containment and the cooling connector.
- **Authority:** each lever acted on real Kubernetes only inside the authority the nervous system granted.
- **Kill switch:** it restored every lever, including from a fresh process.
- **Result:** 19 of 19 checks passed (`results/live/LIVE_LEVERS_2_NERVOUS.txt`, first pass `LIVE_LEVERS_1.txt`).
- **Identity:** least-privilege, with `kubectl auth can-i` receipts.

**Shadow pilot kit, live:** a read-only identity ran for 600 s and logged 40 decisions, with 0 writes (`results/live/LIVE_SHADOW_1.txt`). This is the kit a customer runs first.

## Safety and correctness

- **Safety shield:** tested on 2,000,000 random and adversarial inputs, with zero invariant violations, idempotent, never inventing an action, and intervening minimally (`tests/test_shield_properties.py`). The test found two real bugs, both fixed.
- **C++ twins:** the C++ shield and the C++ closure law match Python exactly, over 300,000 adversarial shield cases and every recorded closure decision.
- **Endurance:** the C++ engine ran 100,000,000 decisions with no failure, at about 2.3 microseconds per decision.
- **Fail-safe:** after repeated failed decisions, control returns to the native autoscalers.
- **Reproducibility:** a clean copy of the delivered zip reproduced every held-out result byte for byte.

## What is not claimed

- **No production or customer deployment yet.** The next step is the shadow pilot (`docs/PILOT_KIT.md`), which is read-only.
- **Live runs are small.** They use kind on CI machines, and power is modelled, not metered.
- **Hardware levers are not proven.** CPU-frequency and power caps need real servers, and cooling was exercised against a stand-in controller.
- **Parked machines in the public cloud.** A parked cloud machine still bills, so the warm-reserve energy saving applies to owned hardware.
- **Single-cluster energy against the tightest packers is roughly a tie.** There Omni-Compass wins on response time and stability.
- **Some gaps cannot be closed.** No controller, even one with perfect foresight, can match both the tightest packer's machine-hours and the calmest autoscaler's machine churn (`tuning/bound.py`).

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

- **Not modelled in the plant:** variable boot times and pod-eviction cost. Fragmentation is modelled (whole-pod packing) and is small.

## Reproduce

```
pip install -r requirements.txt &amp;&amp; python verify.py
python tuning/confirmatory.py        # C, one global setting, 100 scenarios per workload, Holm-corrected
python tuning/confirmatory.py --coord   # the same with nervous-system coordination, fresh seeds
python tuning/planetlab_league.py &lt;planetlab-workload-traces/20110303&gt;
python tuning/site_league.py --heldout
python tools/protocol_bench.py 100
live: push a commit whose message contains [reps], [levers] or [shadow]
```



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 59

# Due Diligence

Answers reference the Claims Register (C-numbers) and the Technical Manual.

## Engineering

**Does the mathematics hold?** Equations (1) to (8), Propositions 1 to 3 and their proofs are in Manual Chapters 2 and 3. The 500/500 CONVEY/CERT result is a property of the controller (Proposition 3), shown by counterfactuals: the controller aimed at the wrong basin also scores 500/500. **Is Python the same as C++?** Yes (C1, C2). The verifier also builds a deliberately mutated C++ governor and checks that the parity test rejects it. **Can every number be reproduced?** `python verify.py` rebuilds the C++, reruns parity, reruns the 100,000,000-decision soak and compares it with the recorded result, replays held-out scenarios and checks the pre-registered SHA-256 hashes (program fingerprints are additionally checked when the Python version matches the recorded one).

## Operations

**Better than what we run today?** Compared against a documented-behaviour reference model of Kubernetes autoscaling (HPA tolerance 0.1, 300 s scale-down stabilization, HPA targets 0.5 to 0.8; simplified Cluster Autoscaler with 10-minute unneeded time, 0.5 utilization threshold, 10-minute delay after scale-up): C8 to C12, including where Omni-Compass is worse. The reference model is not the upstream controllers (C12b); running the upstream controllers against the same scenarios is the next baseline step. **On real traffic?** Not yet. Results use a synthetic stack model. Replay of published production traces and the pilot protocol are the next evidence steps (C14). **What happens when it is wrong?** Observe mode changes nothing (C6). The reset returns control to the native managers at the next interval (omni_kill arm). The shield blocks actions that violate I1 to I5 (C7). **Will it wear hardware?** Machine start/stop cycles, power-cap travel and thermal travel are measured for every arm (C12, Manual Chapter 8).

## Security

**What authority does it hold?** Capacity, replicas, power caps, rollback authorization and routing, only in AUTOPILOT, only through the shield. **Can it expand capacity during a security block?** No: invariant I1 is enforced before execution. **Does it replace encryption, identity or policy engines?** No. Those components are retained (fleet model, security role).

## Finance

**What does it save?** Energy per run versus current autoscaling (C8). Fleet-scale figures are modeled (C13); the governance saving comes from reduced idle and padded capacity, not from removing the decision components' own consumption. **What does it cost to run?** C5.

## Adoption

**How is it introduced without risk?** Observe, then shadow on production telemetry, then one control loop at a time under the reset (docs/PILOT_PROTOCOL.md).

## Referees

**Was it tuned on the test data?** No. Law, shield and baselines were frozen and fingerprinted before the held-out seeds 346410161 and 360555127 (results/PREREGISTRATION.json). **Where does it fail?** Backlog violations against current autoscaling (C11); the engine-dynamics ablation (Manual Chapter 8); open obligations (Manual Chapter 10).



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 60

# License and Commercial Terms

The software and this manual are licensed under the Omni-Compass Evaluation License (`LICENSE`): evaluation and simulation use only. Everything else - commercial use, production use, operating any system beyond evaluation, redistribution, a hosted or managed service, incorporation into a product or service, or using the software or its results to build a competing product - requires a written Omni-Compass Enterprise License signed by The Omni-Compass LLC and paid for. All patents, copyrights and trademarks covering the Omni-Compass engine, its mathematics and its software have been filed in the United States by The Omni-Compass LLC. No patent or trademark license is granted for any other use. Contributions are accepted only on the terms in `.github/CONTRIBUTING.md`, which assign their rights to The Omni-Compass LLC.

CHAPTER 61

# Licensing: Questions and Answers

Plain answers to the questions buyers, engineers and their lawyers ask. The binding terms are in `LICENSE`; where this page and `LICENSE` differ, `LICENSE` governs. Every declaration is in `DISCLOSURES.md`.

**Is Omni-Compass open source?** No. The source is published so that it can be evaluated, reproduced and audited. It is licensed under the Omni-Compass Evaluation License 1.0 (`LicenseRef-OmniCompass-Evaluation-1.0`), which permits evaluation and simulation only.

**What may I do without a commercial license?** Evaluation and simulation, free of charge (`LICENSE`, sections 1.4, 1.5 and 2): read the code; run `verify.py`, the simulations and the benchmarks; reproduce the published results and publish your own, every row included (section 4); run Omni-Compass in watch, dry-run or shadow mode, or in a test environment, on systems you own or control, where it writes to nothing that serves production traffic or real users and to no physical process outside a test bench. A limited patent license comes with it, for evaluation only (section 2.2).

**What needs the Omni-Compass Enterprise License?** Every commercialization or monetization of any kind; production use; operating any system beyond evaluation; offering it, or its function, as a hosted, managed or software-as-a-service offering; redistributing it; incorporating it, or any part or derivative of it, into a product, service, appliance or firmware; using it or its results to build or train a competing product (`LICENSE`, sections 3 and 5). The Enterprise License is a written agreement signed by The Omni-Compass LLC and paid for; its standard terms are summarized in `ENTERPRISE_LICENSE.md` (production, hosted service, OEM and embedded, managed service provider, research).

**May I run it as a service for others?** Not without an Enterprise License with a hosted-service rider (`LICENSE`, section 3(c); `ENTERPRISE_LICENSE.md`, section 1).

**May I connect it to a grid, a plant, a vehicle or a medical device?** Not under the evaluation license: such systems may be studied only in simulation. A live connection to any physical system needs an Enterprise License and a written safety case accepted by the system's owner (`LICENSE`, section 10).

**How is the Enterprise License priced?** Against the measured gain on the customer's own paired receipts (native against Omni-Compass on the same system, the same load and the same clock). Terms are set in each signed agreement; no price stated anywhere in this repository is an offer.

**Are patents involved?** All patents, copyrights and trademarks covering the Omni-Compass engine, its mathematics and its software have been filed in the United States by The Omni-Compass LLC (`PATENTS.md`). The evaluation license grants no patent license beyond evaluation.

**May I use the name or the compass rose?** Only to refer to Omni-Compass accurately (`TRADEMARKS.md`).

**May I contribute?** Contributions require the contributor license agreement (`.github/CLA.md`): every contribution is assigned to The Omni-Compass LLC, with a patent license from the contributor.

**Does Omni-Compass send data anywhere?** No. The software contains no telemetry and makes no network connection of its own except to the systems an operator points it at (a Kubernetes API, `nvidia-smi`, a meter command).

**What do scanners report?** The license is declared in machine-readable form for Black Duck, FOSSA, Snyk and REUSE (`pyproject.toml`, `REUSE.toml`, `.fossa.yml`, `.snyk`, `sbom/`); third-party components are listed in `THIRD_PARTY_NOTICES.md`.

**Can the terms change?** Yes. The Omni-Compass LLC reserves the right to change the software, its documentation, its results and these terms at any time, without notice; a changed version applies to copies obtained and uses made after it is published (`LICENSE`, section 9). A signed Enterprise License governs its own term.

**Which law governs?** The laws of the State of Delaware and the federal laws of the United States, with exclusive venue in the Delaware state courts and the United States District Court for the District of Delaware (`LICENSE`, section 19). The Omni-Compass LLC is based in Florida.

**Who do I contact?** The Omni-Compass LLC, www.omni-compass.com.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 62

# Third-Party Notices

Omni-Compass uses the following third-party components. Each remains under its own license; nothing in the Omni-Compass Evaluation License changes those terms. Installed versions are those resolved from `requirements.txt`.

| Component | Use | License |
|---|---|---|
| NumPy | numerical arrays | BSD-3-Clause |
| Matplotlib | charts in the reports | Matplotlib License (PSF-based, BSD-compatible) |
| ReportLab (open-source edition) | the printable book and reports | BSD-3-Clause |
| Numba | compiled inner loops of the motion-axis plant | BSD-2-Clause |
| PyTorch (on GPU machines only, not installed by `requirements.txt`) | the GPU workload of the benchmark | BSD-3-Clause |
| `tests/third_party/hpa_independent.py` | an independently written HPA reference used unmodified in tests | its own terms, stated in the file |
| Liberation Serif and Liberation Sans fonts (embedded in the PDF book) | typesetting | SIL Open Font License 1.1 |
| DejaVu Sans and DejaVu Sans Mono fonts (embedded in the PDF book) | typesetting | Bitstream Vera / DejaVu license (free) |
| PlanetLab / CoMon VM CPU traces, March 2011 (`fleet/traces/`, `k8s_controlplane/traces/`; source in their `PROVENANCE.json`) | held-out demand for the modelled fleet | their own terms (public research data); no Omni-Compass copyright is claimed in them |
| Google cluster-usage traces 2011 (`clusterdata-2011-2`), from which `results/traces/google2011/` is derived (`docs/TRACES_PREREGISTRATION.md`) | the public demand trace on the real cluster | Google's published terms for the traces; no Omni-Compass copyright is claimed in the trace itself |

Kubernetes, kind, `kubectl`, the NVIDIA driver and `nvidia-smi` are not distributed with Omni-Compass; it calls them where an operator has installed them. Their names are the property of their owners (`DISCLOSURES.md`, section 1).

CHAPTER 63

# Repository Standards

The files the leading repositories carry, the open ones such as Kubernetes and the source-available, commercially licensed ones such as HashiCorp Terraform, Elastic, MongoDB, Redis, CockroachDB and Sentry, and where Omni-Compass carries each. Omni-Compass is published for evaluation and simulation; everything else is licensed (`LICENSE`, `LICENSING_FAQ.md`).

| What a leading repository carries | Omni-Compass |
|---|---|
| License, machine-readable and in full | `LICENSE`, `LICENSES/LicenseRef-OmniCompass-Evaluation-1.0.txt`, SPDX headers, `REUSE.toml` |
| Licensing questions in plain words | `LICENSING_FAQ.md` |
| Notices and third-party components | `NOTICE`, `THIRD_PARTY_NOTICES.md` |
| Disclosures and disclaimers in one place | `DISCLOSURES.md` |
| Patents and trademarks | `PATENTS.md`, `TRADEMARKS.md` |
| Contributor license agreement | `.github/CLA.md` |
| Contributing, code of conduct, governance, ownership | `.github/CONTRIBUTING.md`, `.github/CODE_OF_CONDUCT.md`, `.github/GOVERNANCE.md`, `.github/OWNERS`, `.github/CODEOWNERS` |
| Security policy and contacts | `SECURITY.md`, `.github/SECURITY_CONTACTS` |
| Support | `.github/SUPPORT.md` |
| Changes and roadmap | `CHANGELOG.md`, `ROADMAP.md` |
| Citation and software metadata for archives and crawlers | `CITATION.cff`, `codemeta.json`, `pyproject.toml` (keywords, URLs) |
| Software bill of materials (SPDX and CycloneDX) | `sbom/omni-compass.spdx.json`, `sbom/omni-compass.cdx.json` |
| License scanners (Black Duck, FOSSA, Snyk, ScanCode, REUSE) | `pyproject.toml`, `.fossa.yml`, `.snyk`, `REUSE.toml`, SPDX headers |
| Issue and pull-request templates, including a licensing template | `.github/ISSUE_TEMPLATE/`, `.github/PULL_REQUEST_TEMPLATE.md` |
| Dependency updates | `.github/dependabot.yml` |
| Continuous verification | `.github/workflows/verify.yml`, `python3 verify.py` |
| Reproducibility: fingerprinted release, sealed twins, checksummed results | `RELEASE_MANIFEST.json`, `results/SEAL.json`, `SHA256SUMS.txt` in every result folder |
| Rules written before every benchmark run | `docs/GPU_PREREGISTRATION.md`, `docs/REALMS_PREREGISTRATION.md`, `docs/K8S_COMPASS_PREREGISTRATION.md` |
| A manual and a printable book | `docs/OMNI_COMPASS_MANUAL.md`, `docs/OMNI_COMPASS_MANUAL.pdf` |
| Every result in one place, with charts | `docs/DOSSIER.md` |

| What a leading repository carries | Omni-Compass |
|---|---|
| A one-file archive of the repository | GitHub's Download ZIP, at any commit (the copyright deposit is `release/copyright/`) |

Files present at the commit this page describes; `python3 verify.py` checks the ones the results depend on.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 64

# Python, C++ and the Seal

The laws are twinned: each has a Python version and a C++20 version that give the same answers, proven by a parity test on every build (`cmake -S cpp -B cpp/build && cmake --build cpp/build`).

| Law | Python | C++ | Proven by |
|---|---|---|---|
| core engine | `omnicompass/core.py` | `cpp/src/core.cpp` | 500 frozen fixtures |
| governor | `omnicompass/adapter.py` | `cpp/src/governor.cpp` | `tests/test_cpp_governor_parity.py` |
| safety shield | `omnicompass/shield.py` | `cpp/src/shield.cpp` | `tests/test_cpp_shield_parity.py` |
| HPA replica law | `fleet/harness.py` | `cpp/src/hpa.cpp` | `tests/test_cpp_hpa_parity.py` |
| closure law | `omnicompass/closure.py` | `cpp/src/closure.cpp` | `tests/test_cpp_closure_parity.py` |
| conveyance law | `omnicompass/conveyance.py` | `cpp/src/conveyance.cpp` | `tests/test_cpp_conveyance_parity.py` |
| nervous system | `omnicompass/nervous_system.py` | `cpp/src/nervous_system.cpp` | `tests/test_cpp_twins_parity.py` |
| compass and ledger | `omnicompass/compass.py`, `storage.py` | `cpp/src/compass.cpp` | `tests/test_cpp_twins_parity.py` |
| GPU governor rules | `omni_controller/gpu_governor.py` | `cpp/src/gpu_rules.cpp` | `tests/test_cpp_twins_parity.py` |

The seal (`results/SEAL.json`) holds the SHA-256 fingerprint of every twinned file, written only after every parity test passes. `verify.py` fails, naming the file, if any sealed file changes afterwards. The compass law (`omnicompass/compass_law.py`) and the two-wire GPU governor are in Python today; their C++ twins are next.

**Why two languages.** A law that exists in one implementation can hide a bug that is also its behaviour: the tests pass because the tests were written against the same code. Two implementations written separately, in languages with different numerics and different defaults, that agree to fifteen decimal places on five hundred trajectories and on every fixture of the governor, the shield, the HPA law, the closure, the conveyance and the nervous system, are evidence that the text of the law is what both compute. The C++ twin is also the one a buyer would embed where Python does not run (a controller, a firmware, a PLC), and the soak (`results/SOAK.json`, one hundred million decisions, no failure) is run on it. The seal makes the twins' agreement a property of the repository rather than a claim: change a sealed byte and `verify.py` fails, naming the file.

**The three fingerprints.** The seal fingerprints the twinned laws; `OMNI_V1.json`, `OMNI_V2.json` and `OMNI_V3.json` fingerprint the engine as a whole (law, controllers, muscles, runners: 38 to 40 files); `RELEASE_MANIFEST.json` fingerprints the release (the commit, the engine, the twins, the GPU protocol, the live evidence and the verification receipt) and is rewritten by `tools/release_manifest.py` before every push, under the Python GitHub runs. A reader holding any one of the three can tell whether the files in front of them are the files the results were made on.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

CHAPTER 65

# The Founder's Working Notes

Where a sentence is a design aim rather than what the code does today, or a figure that is not yet measured, the check column says so. Nothing in this chapter adds a result: every number that is a result names the file it comes from.

## What Omni is

OmniCompass is the governor. It is the process running on the processor, and the chip executes it. The card, the replica count, the rack cap, the joint and the feeder are the levers: the muscles. Omni reads the meters, steps a bounded loop, and writes only the lever it is allowed to write. When it stops, it puts that lever back.

It is not the chip, not the card and not Kubernetes. It is the brain on the host, writing through a bolt.

*Governor* is the machine word. A governor on a steam engine did not build the engine and did not turn the shaft. It watched the speed and moved the throttle so the speed stayed in a band. The plant does the work: the chip, the card, the pods, the job. Omni reads a meter, writes one lever, and puts that lever back.

## The loop

| The chapter says | The code today | Check |
|---|---|---|
| Omni reads the plant's meters and maps them to the state E, U, S | `omnicompass/adapter.py`, `observe_vector` and `assimilate` | matches |
| It computes the drift with the command at zero, then raw = −drift + K_P·(σ − U), clipped to the authority before the step | `omnicompass/core.py`, `control_command`: `u = clip(−f_U(x,t) + KP·(σ − U), ±U_AUTHORITY)` | matches |
| It holds that same u through every RK4 stage and does not rewrite U afterwards | `core.py`, `macro_step`: one `u` per micro step, passed unchanged into all four RK4 stages; U is never overwritten | matches (proved: `docs/TRACKING_THEOREM.md`) |
| It writes the clipped command u to the lever | **Not today.** The frozen live governor runs the engine with u = 0 (`macro_step(..., target=None)`) and uses u only as its convergence signal (`push = u / U_AUTHORITY`). The lever is set by the allocation law and the nervous system's authority (`omnicompass/nervous_system.py`). | **design aim**, see "What to strengthen" |
| On exit it restores the lever | kill switch: `omni_controller/controller.py` restore; the GPU bench restores and reads back the start limit; the realm harness checks every knob is handed back | matches |

So the step the chapter describes is the engine's own step, and it is proved. Writing u itself onto a lever, through an output map with a gain in watts (or replicas) per unit of u, is a further mechanism. It is not what produced the results

below.

## Wiring

A wrist wire moves the wrist. It does not move a finger that has its own tendon unless that tendon is also wired.

The card's clock wire goes to the NVIDIA driver. The replica wire goes to the Kubernetes API. The scheduler's wire goes to the kubelet. Those wires already exist and do not pass through Omni. Omni is an added process, and a tendon is connected to it only if the output map writes that lever and the input map reads it back.

A power-limit write can make the driver drop clocks. That is a side effect on one finger, not the replica nerve, which keeps running on its own path unless a second line is connected. The cluster starts its own wires: the API server accepts a replica write, the scheduler places the pod, the kubelet starts it, the driver moves the clocks, and HPA, if installed, is its own process. Omni joins those nerves. It is a client of the plant, not its origin.

## Four realms, five organisms

1. Compute / AI / Cloud: GPU server, Kubernetes, the card, the job.
2. Physics / Robotics / Autonomous: joint, servo, PLC, car, line.
3. Energy / Facility / Industrial: rack, cooling, PDU, battery.
4. Distribution / Specialized: network, feeder, logistics.
5. The whole organism: all 656 canonical muscles, each once, one clock.

A muscle may sit in more than one realm. Every realm stands on the same spine (Kubernetes, machines, GPUs and CPUs, network, storage, observability, security, cooling, electrical distribution), and then has its own domain muscles. The fifth run holds each of the 656 once.

Each organism is shown native, then with Omni, on the same membership, the same seed and the same clock: two receipts. The core does not change when the lever changes.

| Check | |
|---|---|
| Built | `realms/` (round 3): spine of 190 muscles in all four realms; organisms of 345, 262, 282, 337 and 656 (`docs/REALM_MUSCLES.md`) |
| Result | Whole organism +0.1% work per energy, SUPERIOR WITHIN GUARDRAILS; the realms pay in service when they share the spine (`results/realms/REALMS.md`) |
| Limit | These are modelled plants (evidence S / rung E2). A catalog row is not a tendon. GitHub can host the compute organism for real (kind, and the GPU on a rented card); facility, machine and grid need their own hosts |
| Open | The 656 is a working catalog, not a census: `docs/realm_study/` checks it against the real systems' documentation and industry practice (waves 1 and 2 done; wave 3 maps every row) |



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

## Evidence rungs

| Rung (chapter) | Class (repository, `docs/EVIDENCE_LEDGER.md`) | Meaning |
|---|---|---|
| E0 | design | the written spec |
| E1 | T / V | deterministic tests and proofs: Python against the C++ twin, the tracking theorem |
| E2 | S | simulation on a made plant (the realms, the fleet and GPU models) |
| E3 | L | real software: Kubernetes on kind, no card (sets 22 and 23) |
| E4 | P | a physical meter: the card's own power reading (the GPU bench; first run on an NVIDIA A10, 2026-10-02, `results/gpu/run-20261002T082232Z/GPU_REPS.md`; the corrected governor not yet run on a card) |

A result does not climb a rung by itself. The four realms are plants; E1 to E4 are how hard the proof is on whichever plant is run.

## What blocked the card

The hash did not stop the card. A Python–C++ mismatch can fail a test after a machine has started the job; it cannot stop GitHub from handing out a machine, because the runner is chosen before the code runs. The verify job started and failed (fixed since: it is green), and separately the GPU job never got a machine.

The organisation's and enterprise's settings show no GPU runner and no option to create one (checked 2026-10-02), so no budget could start it. The run therefore moves to a rented card (`scripts/gpu_rented_run.sh`, one command).

## The bake-off that exists

Set 23 (`results/live/LIVE_REPS_23.md`, run 36940088922): real Kubernetes, ten pairs, fifteen minutes an arm, the same open-loop work, native against Omni, six kind workers, order rotated.

| Gauge | Native | Omni | Change |
|---|---|---|---|
| Response, 95th percentile (ms) | 327.3 | 123.8 | −62.2% |
| Response, 99th percentile (ms) | 503.4 | 169.3 | −66.4% |
| Response, mean (ms) | 146.6 | 80.1 | −45.4% |
| Failed requests | 0 | 0 | 0 |
| HPA replicas, mean | 8.51 | 5.40 | −36.6% |
| Workers in service | 6 | 4.28 | −28.7% |
| Pods started | 4.7 | 1.7 | −63.8% |
| Pod start wait (s) | 18.6 | 3.8 | −79.6% |
| CPU, service plus Omni (cores) | 0.903 | 0.894 | −1.0% (no difference) |
| Energy, workers still on (Wh, declared model) | 159.0 | 158.7 | −0.2% (no difference) |

The same requests were sent and none failed. They were answered faster, on fewer copies, at the same bill. Kind leaves every worker powered, so the watt line stays flat. Parking a worker and waking it again was tried; the churn ate the saving (`docs/CLAIMS_REGISTER.md`, C17), so that path is closed on this plant.

Arithmetic, not a result: if a run finishes 5% more work, nineteen runs do what twenty used to (20 / 1.05 = 19.05).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

## The chip

The card is the meter that can move the electric bill, and the meter that can move output.

Energy on the chip is the integral of power.draw. The same pile can be spent two ways:

- **Output:** more finished work on the same bill. The cap stays near where the native arm left it, the draw stays in the same band, and heat gets no new reason to rise.
- **The bill:** a lower enforced limit, the job still finishes, and joules fall.

A company short of cards spends it on output. A company paying for GPU hours with spare capacity spends it on the bill. A company against a building power cap splits it. The bench's primary outcome, work per joule, counts both, and its table prints requests and joules separately, so the receipt shows which way the pile went (`tools/gpu_reps.py`).

For the pile to be real:

- the cap has to bind;
- the write has to be enforced by the driver, and the enforced limit read back;
- the job has to finish;
- the limit has to be restored on exit, so the next native arm is not still capped.

All four are in the bench and its preregistration (`docs/GPU_PREREGISTRATION.md`, amendments 1–3).

## The babysitting tax

The electric bill is the small pile. The babysitting tax is the people and tools kept on the clock to watch the levers, reset a cap, and stop the muscles fighting. A card's real cost is usually the card and the hours it sat idle or late, not the electrons.

What fills the seat today is a person: someone sets the cap, someone gets the page when it is left down, someone turns it back after the run or the crash. The product takes that knob and that page.

## What we found nobody selling

We found no published product that does all of the following across compute, machines, facility and grid as one governor:

- reads all of their meters into one state;
- holds one clipped command through the step;
- puts each lever back when the run stops, and prints both receipts.

Borg, Twine and OpenShift govern compute. AWS is a cloud. Turbonomic covers the IT stack. Tesla runs the car, the battery and the factory as separate software. Each covers more than one thing as a company, not four realms as one governor.

Omni on such a stack does not replace it: Borg remains the muscle. This is a market observation from public material, not a claim tested here.

## The bill

Run the stack native and print the receipt. Run the same stack with Omni and print the receipt. The difference is the pile, and the fee is 20 percent of it.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

On a stack that costs $100, a 10 percent win is $10. The fee is $2, and the customer keeps $8. In general, the fee is 0.2 × (measured win) × (spend). A gain and a cut are the same difference, read from opposite sides. Do not add CPU, GPU and babysitting percentages into one bill: the invoice is one pile.

**Check:** the 10 percent is an example, not a measured win. What is measured so far:

- on Kubernetes, response time −62% at the same energy (set 23);
- in simulation, +0.1% work per energy for the whole organism (realm round 3);
- on a card, nothing yet: the GPU bench is the first real-meter number.

Revenue figures should be computed from the measured win on each customer's own receipt. The per-company table in the working draft (2% of each company's published infrastructure spend) assumed a 10% win everywhere, and several of its spend figures do not match the companies' published capital expenditure. It is left out of the repository until both are sourced.

## What to strengthen

1. **Wire the output map.** u becomes one lever, read back and restored, with the start value, the written value and the value after a kill printed on the receipt. The GPU bench already records the start limit, every write and its read-back, and the restored limit. A u-to-lever output map (a gain in watts or replicas per unit of u) is a new mechanism. It would be preregistered and tested as its own arm, first in the realms, then on a card, never mixed into the frozen confirmation run.
2. **Keep the step as it is:** clip before the stage, hold u through RK4, do not rewrite U (proved: `docs/TRACKING_THEOREM.md`).
3. **Run native against Omni on the same membership for each realm.** Compute is the one GitHub can host for real; the other three are modelled until their hosts exist.
4. **Aim the chip at about 5% more finished work on the same bill**, cap unchanged and temperature no worse, or at fewer joules for the same work. The buyer chooses, and the receipt prints which.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

CHAPTER 66

# History

Earlier states of play, kept whole. Nothing here is current status; the current state is `docs/STATE_OF_PLAY.md`.

## State of play as written on 27 September 2026 (with the update of 28 September)

**Superseded for live results.** The live evidence below predates sets 19 and 20. The current live table is `results/live/LIVE_PAIRED.md`; energy on kind is a declared model, not a meter. The first metered test is `scripts/gpu_paired.sh` (`docs/GPU_BENCH.md`).

**Update, 28 September 2026.** Added since this page was written, each with its evidence: - **Live, set 21** (conveyance only when response time needs it): p95 -37%, machines in service -21%, CPU used +26% (unchanged from set 20; its source is still open). `results/live/LIVE_REPS_21.md`. - **GPU governor bounded** (0.70 share floor, busy gate, 2 s decisions): on the MLPerf-calibrated modelled card, +5.1% and +1.3% work per kJ with p95 within +10% (the old governor failed that guardrail at +31% and +66%). Model only. `results/gpu/sim/`. - **Speed lock** (opt-in): speed won elsewhere spent on GPU watts, every gauge kept at least 1% faster than without Omni. Model only. `results/gpu/sim/pipeline/`. - **CPU and GPU on one conserved power budget** (conveyance law over CPU and GPU organs): +1.4% to +5.7% work served against today's fixed caps, never over the site budget. Model only. `docs/CONVEYANCE_LAW.md`, `results/hardware/NODE_EXCHANGE_*.json`. - **The manual in the box:** `docs/INTEGRATION_MANUAL.md`; every gauge: `docs/METRICS_CATALOG.md`; against what runs today: `docs/COMPARISON.md`.

This is the whole repository at the commit named in `STATE_OF_PLAY_COMMIT.txt`. Everything below can be rerun from it.

**Run it live on real Kubernetes yourself:** on any machine with Docker, kind, kubectl and Python, run `bash RUN_LIVE.sh 3`. It builds a fresh 7-node cluster per arm, runs native, Omni on top and Omni alone, and writes `live_runs/LIVE_REPS.md`. GitHub Actions runs the same comparison with a pushed commit whose message contains `[reps]`.

**Rerun the whole thing:** `pip install -r requirements.txt && python verify.py`. It must end `VERIFICATION: PASS`, and it does at this commit.

## Best measured results, and where each one comes from

| Claim | Evidence | Where |
|---|---|---|
| **B (Omni on top of each platform):** 0 losing cells on held-out scenarios; strictly better on 27 of 28 platform-workload pairs | simulation, pre-registered | `tuning/B_FIX.json`, `tuning/B_TONE_HELDOUT.json` |
| **Four-cluster sites:** B 0 losses on all 7 platforms; C 0 losses vs GKE, AKS/Karpenter, CAST AI, Spot | simulation, held-out | `tuning/SITE_LEAGUE.json` |
| **C (Omni alone), one global setting, 400 never-used scenarios:** 153 better / 174 equal / 37 worse of 364 cells (Holm-corrected) | simulation, held-out | `tuning/CONFIRMATORY.json` |
| **C with nervous-system coordination, 400 fresh scenarios:** 151 / 173 / 40 | simulation, held-out | `tuning/CONFIRMATORY_COORD.json` |
| **Controllers fighting each other:** lowest of all 8 systems on web (0.07 per day) and four-cluster sites (0.56 per day) | simulation, held-out | same file |
| **Real demand, 1,052 recorded PlanetLab machines:** 0 losing cells of 91 | simulation on real traces | `tuning/PLANETLAB_LEAGUE.json` |
| **Faults** (machines dying, spikes, crash loops, noisy neighbours): fewest pages to a human, fastest recovery | simulation, 500 runs per workload | `results/protocol/` |
| **GPU power law** (engine cap over the MLPerf-measured performance law), against native at equal work: energy -27% to -33%, time over the heat limit -73% to -78% | simulation, held-out | `results/hardware/SUMMARY_HELDOUT.json` |
| **Site power exchange (conveyance law, manuscript Ch. 29-31):** 4 GPU groups under one site budget: 0 minutes over the site limit at 70/60/50% budgets (native 10.5/45.6/106); least energy at every budget; 31-115 fewer backlog minutes than the static split | simulation, held-out; law proved (conservation, Lyapunov, exponential convergence) | `docs/CONVEYANCE_LAW.md`, `results/hardware/SITE_EXCHANGE_HELDOUT_*.json` |
| **Live levers under the nervous system:** 19 of 19 checks on real Kubernetes; reset restores everything | live, kind | `results/live/LIVE_LEVERS_2_NERVOUS.txt` |
| **Shadow pilot kit:** a read-only identity made 0 writes across 40 decisions | live, kind | `results/live/LIVE_SHADOW_1.txt` |
| **Safety shield:** 2,000,000 adversarial cases, 0 violations (the test found 2 real bugs, both fixed); C++ twin matches | test | `tests/test_shield_properties.py` |
| **C++ engine:** 100,000,000 decisions, no failures, about 2.3 µs per decision | test | `results/SOAK.json` |

## What still loses, measured

- **C, typical response time, web and four-cluster:** about 20% slower than every platform. Omni fills pods to 80%, where the others stop at 70%, and trades response time for energy.
- **C, machine starts and stops:** loses to plain Kubernetes and OpenShift, which hold machines steady and pay for it in machine-hours.
- **C, GPU and batch:** energy 1-2% worse and power and heat margins 1-4% worse than the tight packers (AKS/Karpenter, CAST AI, Spot).
- **Live, set 3, with a working probe:**

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

  - Omni kept all 6 machines. Latency near the 500 ms SLO blocks machine release.
  - There is no significant difference from native on anything else, except that C has more waiting pods.
  - The earlier live machine savings (sets 1-2) are withdrawn, because the broken probe had blinded the latency sense (`results/live/LIVE_REPS_PROBE_DEFECT.md`).
  - Set 4's decision trail showed why. Latency spikes from pod saturation at load steps kept the service from being clean, and the engine's calm stayed under the machine threshold (0.65 against 0.7). Both are addressed by the attribution gate and the two-way wiring above; the live test is pending (`results/live/LIVE_REPS_4_DIAGNOSIS.md`).
- **A bound, not a loss:** no controller, even one with perfect foresight, can match both the tightest packer's machine-hours and the calmest autoscaler's churn at once (`tuning/bound.py`).

## Two-way nervous system (new, live test pending)

Manuscript Appendix J names delay and dropout handling and feedback interpretation as the nervous system's job. The live controller now wires both directions (`docs/TWO_WAY_NERVOUS_SYSTEM.md`).

- **Blind senses are detected against the wall clock.** A hung probe was the root cause of the live set 1-2 error.
  - A blind sense feeds the engine's `stale` channel.
  - It blocks every give-back.
- **Commands are read back.** Drift feeds the engine's `drift_ratio`. No new machine release goes out before the last one has landed.
- **The machine organ has its own engine view and a release gate.** Pods move first, and the headroom proof uses the engine's rho.

**Tests:** 75,000 blind states, 200,000 gate states, 300,000 nervous-system states.

**Live sets 5 and 6 could not run.** From 03:51 UTC GitHub refused every job, including `verify`: runner ID 0, no steps, failed within about 2 seconds. That is GitHub declining to start runners, which on a private repository is most often exhausted Actions minutes or a spending limit. It is not a code failure. `python verify.py` passes locally at this commit.

## Map

| Path | What it is |
|---|---|
| `omnicompass/core.py`, `cpp/` | six-state engine; C++ twin |
| `omnicompass/closure.py` | closure law: forward projection, boundary correction, turning point, muscle tone |
| `omnicompass/nervous_system.py` | one engine state grants each organ its authority and envelope |
| `omnicompass/shield.py` | safety shield, downstream of everything |
| `omni_controller/` | live Kubernetes controller and levers |
| `fleet/`, `hardware/` | simulation plants |
| `tuning/` | benchmarks, pre-registrations, results |
| `results/live/` | every live run, including the failed and withdrawn ones |
| `docs/history/OMNICOMPASS_BUYER_EDITION.md` | the buyer-facing report |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

# The state of play as it stood on 2026-10-06 (moved here whole on 2026-10-07)

## In one paragraph (2026-10-06, Omni v1)

**Omni v1, the same in all three of our runs.** On real Kubernetes, every test ran three times as separate GitHub runs on the frozen engine, ten pairs each (`docs/OMNI_V1.md`, the six `results/live/V1_*.md` tables). Work inside the response line **+42% to +48%** in all three runs of the all-four test (18.6 → 27.6 requests a second in run A), better in all three of our runs; the 95th percentile **−57% to −69%** in every run of the steady, wandering, all-four and fault tests, better in all three of our runs; failed requests −12% to −13% where load swings, better in all three of our runs; machines −1.5% to −3.4% at steady load and −15% to −24% on the batch queue, better in all three of our runs, no difference beyond the noise elsewhere; energy a declared model (kind never powers a machine down); beside a noisy neighbour, no difference beyond the noise on every row. **The Omni index on v1: +26.2%** on real Kubernetes (`results/OMNI_INDEX.md`; only rows the same in all three of our runs count, a row inside the noise counts as zero; Azure and the card join when their v1 runs land). The paragraph below is the earlier engine's reading, kept as the record it was.

Omni-Compass sits on top of Kubernetes and hardware and gets more out of what is already there. On real Kubernetes, the engine before v1 (the live controller at rules 1-8, commit `353903683009`), ten pairs per test, native against compass with the same work sent to both: **41.7% more work handled inside the response line on the same machines** (capacity, all four in one run), responses **59-64% faster** at the 95th percentile (steady, wandering, all four, faults), 12% fewer failed requests where load swings, energy equal or lower, and not one pod left without a machine. Machines: up to 2% fewer. Omni gives a machine back only when a paired trial shows the service no slower without it, and on these clusters one machine fewer made requests 30-45% slower in most trials, so it kept them and spent them on speed and work (earlier engines without that check parked 29-36%: `docs/history/`). All of it together, real machines only: **the Omni index +12.9%** (`results/OMNI_INDEX.md`).

**Omni v1** (`docs/OMNI_V1.md`): the engine is frozen and fingerprinted (`OMNI_V1.json`), and every test above runs three times on it, as separate GitHub runs (A the result, B and C the replications); each table reads better in all three of our runs, worse in all three of our runs, no difference beyond the noise, or the runs disagree, and replaces the earlier engine's table as it lands. Running on v1 now: the Kubernetes suite three times, the batch test, the six organisms at 1, 10, 100 and 1,000 copies with the real cluster inside, the big organisms on rented Azure machines, the Azure steady and burst bill tests, CityLearn, and the power grid.

**What is shown and what is not.** Shown: more work inside the response line on the same machines (34-52% across the three capacity runs on earlier engines) and a faster tail, on real Kubernetes. Not yet shown: an energy or cloud-bill saving on real machines. On kind the machines are containers on one runner, so a machine out of service saves modelled watts, not a metered bill, and Azure's bill did not move on the earlier engine (`results/live/AKS_BILL.md`); the v1 Azure runs are the test of that. The current card governor has not run on a real GPU.

The card: the first real run (NVIDIA A10, 2026-10-02) used a card controller since replaced; it is not a result to stand on. In simulation the current controller (amendment 12) is clearly faster under an operator's power cap (p95 6-7% faster, time over the line 1 point lower) and saves 0.5-3.7% energy on the card's own firmware with p95 even (`results/sim/gpu_two_wire/`). The real card runs next: one card, the card inside the six organisms, then eight cards.

The 945 muscles (Omni v2; 656 in v1) are models of real control systems. With the real cluster or the real card inside, they show the mechanism (work the same, energy 0.1-0.2% lower, time over the line lower than native in every organism); they are never counted in the headline.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

| Test (real) | Work | Speed | Machines | Energy | Source |
|---|---|---|---|---|---|
| All four in one run: load up and down one step at a time, 10 pairs | **+29%** | p95 -62% | **-3.6%** | -0.3% | `results/live/ALL_FOUR.md` |
| Capacity, load rising, 10 pairs | **+48%** | p95 -50% | -1% | -0.3% | `results/live/AMENDMENT_3_RUNS.md` |
| Sets 22-27, same work, 10 pairs each | same, none failed | p95 -55% to -65% | **-29% to -36%** | -0.1% to -0.5% | `results/live/LIVE_REPS_22.md` to `results/live/LIVE_REPS_27.md` |
| Six organisms, cluster inside, 5 pairs each | requests served +0.5% to +6% | p95 -24% to -40% | same to -2% | -0.1% to -0.6% | `results/live/SIX_KUBE.md` |
| Azure AKS, steady load, 5 pairs | same | p95 -20% | same | (billed) same | `results/live/AKS_BILL.md` |

All four in one run is done: every one better and proven in the same run (`results/live/ALL_FOUR.md`). Running now: demand that wanders (up, spike, down, back up, idle, 10 pairs), the Azure burst bill test, and the 1,000-copy grid of the six organisms on Lambda.

### *The engine now, and what runs on it (2026-10-05)*

The live controller is frozen at rules 1-8 (`docs/K8S_COMPASS_PREREGISTRATION.md`, amendments 1-8): rules 5-8 were added today (a pinned gauge is not a steady demand; coasting; cruise; the emergency brake). Every result above was measured on an earlier version of the controller, and each names its commit. So that every number comes from one engine, the whole Kubernetes suite runs again on the frozen engine: all four in one run, demand that wanders, the steady same-work set, the fault and fairness tests, the batch test, and the six organisms at every size; the Azure burst bill test and the two big organisms on rented Azure machines too. The card harness changed only in how it judges (amendment 11 of the GPU preregistration), so the single card and the eight cards run once, on this engine, on Lambda.

Back on the frozen engine so far, ten pairs each:

- **Faults** (`results/live/FAULTS.md`): p95 −60%, mean response −35%, time over the line −23%, pending pods −77%. Recovery is faster after every fault: machine down 42 s against 70 s, a blind probe 54 s against 66 s, a runaway pod 98 s against 105 s, a spike the same.
- **Fairness** (`results/live/FAIRNESS.md`): a noisy neighbour on the same cluster. Mean response −15%, nothing worse, and the neighbour's own app no worse. The earlier engine scored −0.3% on the index here; this engine scores +5.6%.

Independent simulator: CityLearn round 1 split (the 2022 battery districts better on bill -4% to -11%, electricity -9% to -17%, carbon -7% to -14%; the 2020-2021 water-tank districts worse on peaks and ramping), the fix (only the electric batteries are steered), and round 2 on every untouched district (`docs/CITYLEARN_PREREGISTRATION.md`).

### *Measured on real systems: the newest set, Omni-Compass against Kubernetes as it runs today*

**Set 24 (2026-10-02, commit `c908054`) repeats it again: machines in service −31.6%, p95 −60.1%, p99 −64.1%, HPA replicas −38.6%, 0 failed requests, total CPU including Omni-Compass's own −1.8% (not significant)** (`results/live/LIVE_REPS_24.md`). Set 23 before it: p95 −62%, replicas −37%, pod starts −64%, 0 failed requests, no energy or total-CPU difference (`results/live/LIVE_REPS_23.md`). The set-22 table below stands as first measured.

Set 22 (`results/live/LIVE_REPS_22.md`): 10 paired repetitions on real Kubernetes (kind), each pair on one machine, Kubernetes with its autoscaler alone against the same Kubernetes with Omni-Compass on top. The load is



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

sent at a fixed rate, so both arms were given **the same work**.

| Result | Kubernetes alone | With Omni-Compass | Change (95% interval) |
|---|---|---|---|
| **Energy, parked machines still on at idle power** (declared model, no meter) | 160.3 Wh | 160.1 Wh | **−0.1%, no difference** |
| **Response time, 95th percentile** | 407.9 ms | 158.8 ms | **−61%** (proven) |
| Response time, 99th percentile | 639.4 ms | 245.1 ms | −62% (proven) |
| Response time, mean | 179.9 ms | 98.4 ms | −45% (proven) |
| Failed requests | 0 | 0 | equal |
| Pods waiting to start, pod-minutes | 0.265 | 0.025 | −91% (proven) |
| Replicas, mean | 8.93 | 6.91 | −23% (proven) |
| Machines in service, mean (all stayed powered) | 6 | 4.14 | −31% (proven) |
| CPU used by the service | 1.036 cores | 0.957 cores | −7.6% (proven) |
| Omni's own CPU (its controller and every command it ran) | 0 | 0.070 cores | +0.070 (proven) |
| **CPU used, service and Omni together** | 1.036 cores | 1.026 cores | **−0.9%, no difference** |

- **Same work, much faster answers**, with no failed requests and far less waiting.
- **No energy saving is shown on kind.** Every machine stays powered; energy is a declared model, and counted at the idle power a parked machine really draws it is unchanged.
- **No CPU saving once Omni's own cost is counted.** The service used 7.6% less CPU; the controller spent almost all of it. Cutting the controller's cost is the next improvement.
- The reset restored every setting in every run.

### *Kubernetes sets 25 and 26 (2026-10-02, 10 paired repetitions each, equal work)*

| Run | Machines in service | p95 | Failed | Total CPU incl. Omni's own | Receipt |
|---|---|---|---|---|---|
| Set 25, the engine's allocation law | **−32.3%** | **−57.3%** | 0 / 0 | −0.6% (not significant) | `results/live/LIVE_REPS_25.md` |
| Set 26, the engine's allocation law | **−35.8%** | **−55.4%** | 0 / 0 | −1.5% (not significant) | `results/live/LIVE_REPS_26.md` |
| Set 26, **the compass law in the live controller** | **−17.2%** | **−64.8%** | 0 / 0 | +1.0% (not significant) | same; label by the preregistered rule: **better on machines within the band** |
| Set 27, the engine's allocation law | **−36.6%** | **−53.1%** | 0 / 0 | −0.0% (not significant) | `results/live/LIVE_REPS_27.md` |
| Set 27, **the compass law aligned with the GPU governor** | **−15.9%** | **−65.5%** | 0 / 0 | +0.2% (not significant) | same; label by the preregistered rule: **better on machines within the band** |

Set 27 (running): the compass in the live controller reads the service as the corrected GPU compass does (mean response time, center 0.4), against native and the allocation law (`docs/K8S_COMPASS_PREREGISTRATION.md`).

### *Measured on a real GPU: the card's own meter (evidence class P)*

**First counted run, NVIDIA A10 on Lambda, 2026-10-02** (`results/gpu/run-20261002T082232Z/GPU_REPS.md`, 10 paired repetitions × native / watch / Omni, 600 s each, frozen at commit `c908054`, checksums verified). Wire check 7 of 7; 2,144 writes, none refused, every one read back, every arm ended at the start limit; watch equal to native.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

| Gauge | Native | Omni | Change (95% interval) |
|---|---|---|---|
| **Work per energy** (requests per kJ) | 50.79 | 52.62 | **+3.6% (+2.7% to +4.5%), proven** |
| GPU energy | 69,180 J | 66,790 J | −3.5%, proven |
| Requests served / not served | 3,514 / 0 | 3,514 / 0 | equal |
| **Response time, 95th percentile** | 510 ms | 809 ms | **+58.5%, worse, proven** |

**Result, by rule: ENERGY IMPROVEMENT WITH SERVICE TRADEOFF** (the p95 guardrail of +10% failed). The six organisms with the same card inside (`results/hil/run-20261002T082232Z/HIL.md`, 3 repetitions each): the card's work per energy +1.6% to +2.8% in every organism, the same requests, its p95 500 to about 600-935 ms. The cause, from the card's own samples, was wiring in the governor (amendment 6 of `docs/GPU_PREREGISTRATION.md`): busy bursts served at 736-768 MHz against 861-889 MHz on its own. Corrected (amendments 6 and 7); the corrected governor has not yet run on a card.

### *Simulated (models: they show the mechanism, not a measurement)*

| Result | Where |
|---|---|
| GPU governor with share floor and busy gate (one wire, the power limit), MLPerf-calibrated card: +5.1% and +1.3% work per kJ, p95 within +10% | `results/gpu/sim/after` |
| **Two-wire GPU card under the compass law, corrected governor** (amendments 6-7), 10 seeds and 10 fresh seeds, geometric means: **service** profile work per energy **+6.9% / +3.8%**, energy −6.4% / −3.7%, p95 **−5.9% / −2.3%** (faster), time over the line −0.03 / −0.04 pp; **batch** profile +8.1% / +4.2%, p95 +7.0% / −2.3%; the one-wire governor +0.1%; both wires restored every seed | `results/sim/gpu_two_wire/RESULT.md`, `fresh/` |
| **The six organisms** (Compute 345, Physics 262, Energy 282, Distribution 337, the four stacked 1,226, the whole tower 656), native against the compass law on every muscle, 1,000 paired runs at 1× and at 10× size: work per energy +0.20% to +0.30%, energy −0.21% to −0.32%, work −0.01% to −0.02%, time over the service line **+0.19 to +0.27 pp in every cell (band first not held)**, every knob handed back. 100× and 1,000× are running | `results/scale/GRID.md` |
| Speed lock (speed won elsewhere spent on GPU watts) | `results/gpu/sim/pipeline/` |
| CPU and GPU on one conserved power budget: +1.4% to +5.7% work served against a fixed cap, never over the budget | `results/hardware/NODE_EXCHANGE_*.json`, `docs/CONVEYANCE_LAW.md` |
| GPU groups sharing a site budget: 0 minutes over the budget | `results/hardware/SITE_EXCHANGE_HELDOUT_*.json` |
| Platform leagues, faults, PlanetLab traces, stack benchmark | `tuning/`, `results/protocol/`, `results/` (see `docs/history/BENCHMARK_REPORT.md`) |
| **The 656-muscle tower as organisms**, round 3 (preregistered, 10 seeds; every realm carries the shared spine; Omni as the shipped controller commands): the whole tower native against one governor on top, work per energy **+0.1%, SUPERIOR WITHIN GUARDRAILS**; inside the realms the spine costs service: Energy +0.2% with +1.9 pp violations (tradeoff), Compute 0.0% (+2.1 pp, not established), Distribution −0.1% and Physics −0.7% (**WORSE**). Rounds 1 and 2 kept, superseded | `results/realms/REALMS.md`, `docs/REALM_MUSCLES.md` |


*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

# Back Matter

BACK MATTER

# Glossary

| Term | Meaning |
|---|---|
| Antagonist pair | the up side and the down side of a muscle's control, each with its own gain |
| Authority | how far the nervous system lets an organ move this decision |
| Band | a reading's or a knob's safe range |
| Compass | the band seen as a position from 0 (calm) to 1 (the line), with its bottom in the middle |
| Better in all three of our runs / worse in all three of our runs | the same sign in all three runs (A, B, C), every 95% interval clear of zero |
| Cover | a knob's hard range; every write is clipped to it |
| Cushion | the 5% at each edge of the compass, 10% in all |
| Fail up | the return to full capacity the moment service crosses the wall or a sense goes blind |
| Governor | the process that reads, decides and writes; Omni-Compass |
| Muscle | any machine, service or controller Omni-Compass reads and sets |
| Native | the system as it runs without Omni-Compass |
| No difference beyond the noise | an interval over zero in at least one run: native and omni could not be told apart; the result, stated as such |
| Off the clock | a repetition in which either arm ended more than 5% of its window late; marked, the pairing stands |
| Organism | a set of muscles run together on one clock |
| Omni index | the one combined number: the geometric mean of every real category's work, speed, machines and energy ratios, each read by the three-run rule |
| Omni v1, v2, v3 | the frozen engines, fingerprinted file by file; a result belongs to the engine its commit carries |
| Plug | the two-way connection to one muscle: read, write, read back, restore |
| Pool (Azure) | one machine family under Azure's autoscaler with its own ceiling; several pools make the fleet |
| Profile | a named set of the compass's settings for one kind of work: service (the default) or batch |
| Race | full speed at once while work waits, so a burst is never served slowly |
| Speed floor | the clock a card reaches on its own while busy, learned from its own meter; the governor never sets the ceiling under it |
| Receipt | the paired record of native against Omni-Compass for one run |
| Snapshot | a knob's value read once before the first write; the restore point |
| The runs disagree | runs clear of the noise point different ways; the test is unstable on that measure and is looked into |

| Term | Meaning |
|---|---|
| Wire check | the test that proves every wire follows, reads back and returns before anything runs |
| Work per energy | work done divided by energy used; the primary outcome |
| Work inside the line | the requests, transactions or messages a second answered within the response line; the product number on every real stack |
| Pedals | idle, gas, brake and reset: the governor's four ways of moving capacity under autopilot |
| Cruise | every machine in service while work waits for a place, held without second-guessing until the line has been empty for two decisions |
| Emergency brake | the work done and demand at zero: straight to the floor in one move, every safety check still holding |
| Kill switch | security only: one switch in a human hand that turns every governor on the machine off at once; never the reset |
| Lease | the record every governor renews each decision; a lease older than its limit, or a dead process, makes the watchdog hand back for it |
| Watchdog | the service beside the governors that hands back for any governor that died or hung without doing it itself |
| Verdict | the paired trial on the muscle itself that allows a slow knob to move only where the cost is within 2%; "left native" where no step passes |
| Slack gate | v3's rule: a motion axis busy more than half the time at full speed keeps its speed native |
| Tuning case | the one workload, cell or swarm the rules were fitted on; shown in every table and never counted |
| Untouched case | a workload, cell or swarm the rules were applied to unchanged; the judged rows |
| Knee | nine tenths of a system's measured capacity, where its queue begins to grow; the broker's native was set there by design |
| Amendment | a change to a preregistration after it was written, with its time and reason; one made after a result was seen says so |
| Dwell | the seconds after a knob moves during which the law will not move it back |
| Notch | the smallest step a knob moves in one decision (one consumer, 8 MB of cache, one tap) |
| Evidence class P, L, S | a physical meter; live software with no meter; a model of ours |
| Handed back | the knob read back at the operator's value at the end of an arm; every table says whether every arm was |
| Off the clock | see above; a repetition whose arm ended more than 5% of its window late |
| Own cost | the governor's CPU (its process and every command it ran) as a share of one core over the window, from its audit; a few thousandths of a core at every size |
| Smoke run | one repetition that exercises a harness end to end before any counted run; never counted, always recorded |
| L + S | an organism run with a real cluster inside: the cluster is L, the organism around it S |
| Working set | the records a workload actually touches over a window; a cache benchmark measures nothing unless the working set reaches the cache |
| Key space | the records a benchmark may ask for at one notch; in the MongoDB test the knob's load, stepped one notch at a time |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Term | Meaning |
|---|---|
| Uniform draw | every record in the key space equally likely, so the working set is the key space; YCSB's shipped zipfian draw concentrates on a few thousand hot records whatever the key space |
| Ordered keys | records named in load order, so a run that asks for record n finds record n; a run with hashed keys against an ordered load asks for records that do not exist |
| Full-cache gate | the MongoDB and Redis rule: slow reads grow the cache only while the cache is full (nine tenths used), because a miss in a cache with room to spare is not the cache's to mend |
| Buffer pool | InnoDB's cache of table and index pages, the one memory setting every MySQL operator sizes; resized online by the server in whole chunks |
| Chunk | the unit in which MySQL resizes its buffer pool (128 MB as shipped); the knob's notch on that stack |
| Page cache | the operating system's own cache of file contents; where the data fits in it, a storage-engine miss is a read from memory and a decompression, not a disk read, and the gain available to the cache knob is smaller |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

APPENDIX

# Appendix A. Command Reference

| Task | Command | | | |
|---|---|---|---|---|
| Verify everything | `python3 verify.py` | | | |
| Six organisms, modelled | `python3 tools/run_scale.py --runs N --scale K --out <dir>` | | | |
| The full ladder | `bash scripts/scale_ladder.sh` | | | |
| Two-wire card, modelled | `python3 tools/run_gpu_card.py <dir> [fresh]` | | | |
| GPU wire check | `sudo python3 tools/gpu_wire_check.py --gpu 0` | | | |
| GPU, whole benchmark in one command | `sudo nohup bash scripts/gpu_rented_run.sh > run.log 2>&1 &` | | | |
| The six organisms with the real card inside | `python3 tools/run_hil.py --out <dir>` | | | |
| GPU governor, two wires | `sudo python3 -m omni_controller.gpu_compass --mode watch | cap ...` | | | |
| Kubernetes, watch | `kubectl apply -f deploy/install/omni-compass.yaml` | | | |
| Kubernetes paired runs | `scripts/kind_paired.sh`, then `tools/live_reps.py` | | | |
| OFF (Kubernetes) | `touch /tmp/omni.kill` | | | |
| OFF (GPU) | `sudo touch /tmp/omni-gpu-kill` | | | |
| The whole harness OFF, ON, status, watchdog | `python3 tools/omni_switch.py off | on | status | watchdog` | | | |
| Seal check | `python3 tools/seal.py --check` | | | |
| Which engine a checkout or a commit carries | `python3 tools/omni_version.py [--commit <sha>]` | | | |
| The three-run table (Kubernetes, Azure) | `python3 tools/confirm_abc.py "<title>" raw/run-<A> raw/run-<B> raw/run-<C> --out results/live/V3_<TEST>.md` | | | |
| The three-run table (database, messaging, cache, swarms, robots, grid, buildings) | `tools/pgbench_abc.py`, `tools/kafka_abc.py`, `tools/redis_abc.py`, `tools/swarm_abc.py`, `tools/mujoco_abc.py`, `tools/pandapower_abc.py`, `tools/citylearn_abc.py`, the same arguments | | | |

| Task | Command | | | |
|---|---|---|---|---|
| The six organisms with the cluster inside | `python3 tools/six_kube_report.py <raw dirs> --out results/live/V3_SIX_KUBE.md` | | | |
| The grid of copies and runs | `python3 tools/grid.py` (reads `results/scale/receipts/`) | | | |
| The Omni index | `python3 tools/omni_index.py` | | | |
| Which knobs earn a wire in (write, watch, operator's choice) | `python3 tools/wiring_verdicts.py` (`--check` compares the page with the tables; `verify.py` runs it) | | | |
| One number per benchmark, plus always good for Omni | `python3 tools/benefit_sheet.py` (`--check` the same way; `verify.py` runs it) | | | |
| The database benchmark, one run | `python3 tools/run_pgbench.py --out <dir>` (workflow `pgbench`) | | | |
| The message broker, one run | `python3 tools/run_kafka.py --setup`, then `--workloads light,heavy,burst --out <dir>` (workflow `kafka`) | | | |
| The cache, one run | `python3 tools/run_redis.py --setup`, then `--workloads small,large,burst --out <dir>` (workflow `redis`) | | | |
| The database's storage-engine cache under YCSB, one run | `python3 tools/run_ycsb.py --setup`, then `--workloads b,c,f,burst --out <dir>` (workflow `ycsb`) | | | |
| The database's buffer pool under sysbench, one run | `python3 tools/run_sysbench.py --setup`, then `--workloads read_only,read_write,update_index,burst --out <dir>` (workflow `sysbench`) | | | |
| The robustness test | workflow `robustness`: `scenario` kill or long, `reps`, `duration_s`; the own-cost table `python3 tools/own_cost.py <raw run dirs> --out results/live/V3_OWN_COST.md` | | | |
| The drone swarms, one cell | `python3 tools/run_swarm.py --cell short | mixed | long | all --out <dir> (workflow swarm`) |
| The independent simulators | `python3 tools/run_citylearn.py`, `run_pandapower.py`, `run_mujoco.py` (workflows `citylearn`, `pandapower`, `mujoco`) | | | |
| Azure, the bill | workflow `aks-metered`: `reps`, `arms`, `duration_s`, `load_steps`, `worker_pools`, `pool_prices`, `hpa_max` | | | |
| The big organisms, detached | workflow `big-organism-detached`: `mode` start / collect / survey, `organisms`, `size`, `duration_s`, `commit`, `max_hours` | | | |
| Archive a finished run with the code | add its id to `.github/archive_request.txt` and push (workflow `archive-run`) | | | |



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

| Task | Command | | | |
|---|---|---|---|---|
| Verify before a push, as GitHub runs it | `python3 tools/release_manifest.py` under Python 3.12: 0 FAIL | | | |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

APPENDIX

# Appendix B. File Map

| Path | Contents |
|---|---|
| `omnicompass/` | the engine, governor, nervous system, shield, compass, conveyance law, the compass |
| `omni_controller/` | the Kubernetes controller and muscles; the GPU governors (one and two wires) |
| `realms/` | the 945-muscle catalog, the plant models, the realm harness, the compass on every muscle, the modelled card |
| `tools/` | benchmarks, receipts, the wire check, the scale ladder, manifests, seals |
| `scripts/` | one-command runs (GPU, Kubernetes, ladder) |
| `deploy/` | container image, install and permission files |
| `cpp/` | the C++20 twins |
| `results/` | every published result, with its raw files and checksums |
| `results/live/V1_*.md`, `V3_*.md` (+ `.json`) | the three-run tables, one per test, named by the engine they ran on |
| `results/live/raw/run-<id>/` | every archived run's files, one SHA-256 manifest per repetition |
| `results/scale/GRID.md`, `results/realms/REALMS.md`, `results/OMNI_INDEX.md` | the grid of copies and runs, the modelled realms, the one combined number |
| `OMNI_V1.json`, `OMNI_V2.json`, `OMNI_V3.json` | the frozen engines' fingerprints |
| `docs/` | this manual, the preregistrations, the evidence ledger, the theorem, the realm study |
| `docs/OMNI_V1.md`, `OMNI_V2.md`, `OMNI_V3.md` | what each engine is and every result read on it |
| `docs/REGISTER.md`, `docs/PROOF_PROGRAM.md` | every muscle, every benchmark run and every benchmark still to run; the program to full size |
| `.github/workflows/` | every benchmark as GitHub runs it: `benchmark-reps`, `six`, `six-kube`, `big-organism`, `big-organism-detached`, `aks-metered`, `pgbench`, `kafka`, `redis`, `ycsb`, `sysbench`, `robustness`, `swarm`, `citylearn`, `pandapower`, `mujoco`, `archive-run`, `verify` |
| `tests/`, `verify.py` | every unit and property test, the shield's adversarial cases, the Python-against-C++ agreement; `verify.py` runs them all and must end with its pass line |
| `tools/omni_version.py`, `tools/release_manifest.py`, `tools/layout_check.py` | which engine a checkout or commit carries; the release manifest and verification as GitHub runs it; the check that every path the documents name exists |
| `tools/confirm_abc.py` and its siblings (`pgbench_abc.py`, `kafka_abc.py`, `redis_abc.py`, `ycsb_abc.py`, `sysbench_abc.py`, `swarm_abc.py`, `mujoco_abc.py`, `pandapower_abc.py`, `citylearn_abc.py`) | the three-run tables, one tool per kind of raw record, each checking the engine of every run it reads |
| `tools/omni_index.py`, `tools/dossier.py`, `tools/own_cost.py` | the one number from the tables; the dossier from the tables; the governor's own cost from the archived audits |

| Path | Contents |
|---|---|
| `tools/wiring_verdicts.py`, `docs/WIRING_VERDICTS.md`, `results/WIRING_VERDICTS.csv` | the verdict per knob from every table: write, watch or the operator's choice, every loss with its cause; one row per knob and per muscle |
| `tools/knob_verdict.py` | the brain's own verdict on a live knob, around the engine's `omnicompass/verdict.py`: watch first, one notch a trial, the allowance, the two objectives; used by the five live harnesses |
| `tools/benefit_sheet.py`, `docs/BENEFIT_SHEET.md`, `results/BENEFIT_SHEET.json` | one number per benchmark from the tables, plus always good for Omni, with yes, no, none or trade beside it |
| `tools/legal.py` | the legal notice every generated report carries at its head and foot |
| `docs/book/` | the builder of this manual's PDF (`build_book.py`) and the theory chapters bound into it |
| `release/` | the copyright deposit and the release notes, printed at named commits |
| `CHANGELOG.md`, `docs/STATE_OF_PLAY.md` | what changed, by date; where everything stands today |
| `docs/*_PREREGISTRATION.md` | the rules of every benchmark, written before it ran, with every amendment and its time |
| `docs/DOSSIER.md`, `docs/dossier/` | every result in one place, with its charts, built by `tools/dossier.py` from the tables |
| `docs/HISTORY.md`, `docs/history/` | earlier states of play, earlier engines' sets and the pre-v1 index, kept whole |
| `LICENSE`, `NOTICE`, `LICENSES/` | the license and notices |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

APPENDIX

# Appendix C. The Equations in Full

```
(1) dE/dt    = -alpha_E E + beta_int + beta_ext + v_eff
(2) dU/dt    = mu U (1 - U^2) - (dE/dt)/E_max - lambda_U U + u,   |u| &lt;= U_AUTHORITY = 25
(3) dI_U/dt  = (1 - U) - sigma_1 E - delta S - lambda_I I_U
(4) v_eff    = cos(omega_B t / 2) * c * tanh(lambda_0 + lambda_1 (U - 0.5) + lambda_2 S)
(5) Phi(S)   = alpha_s S^2 / 2 + beta_s S^3 / 4 - delta S
(6) dS/dt    = -dPhi/dS = delta - alpha_s S - (3/4) beta_s S^2
(7) dB/dt    = B_dot ;  dB_dot/dt = gamma_c delta S - (omega_B / Q_B) B_dot - omega_B^2 B
(8) R_B[n]   = finite-difference audit of (7); never fed back into the state

Control (one micro step, zero-order hold across all four RK4 stages):
    u = clip(-f_U(x, t) + K_P (sigma - U), -U_AUTHORITY, +U_AUTHORITY),  K_P = 12,  sigma in {-1, +1}

The compass (every muscle):
    p = position of the service reading in its band (0 calm, 1 the line)
    F = A tanh((K_P (p - center) + K_D v) / A),   K_D &gt;= critical damping
    p &gt;= 0.95  =&gt;  full up force; down side held
    knob &lt;- clip(knob + g_side F span, cover)

The GPU governor (two wires, omni_controller/gpu_compass.py; one law, amendments 8 to 10):
    p = (mean response of the last 5 s - S) / (SLO - S),  S = SLO / 10;  p95 &gt;= SLO, a failure, blind  =
    ↪ &gt;  fail up
    fail up / race (utilization &gt;= 0.95)  =&gt;  ceiling = top clock, lid = start limit (the card's own set
    ↪ tings)
    steady under the limit (saturated and drawing &gt;= 0.97 of the limit)  =&gt;  ceiling = the card's own bu
    ↪ sy clock,
        lid = start limit (the same watts, no knock-backs)
    otherwise:
        ceiling &lt;- clip(ceiling + round(g_side F f_top / 15 MHz) 15 MHz, c_floor, f_top)
```

```
        c_floor = max(f_busy_own x floor, f_top - k* 15 MHz)       (never under the card's own busy cloc
    ↪ k,
                                                                     never past the verdict's deepest st
    ↪ ep k*)
        lid = clip(ceil(1.10 P_busy_own), envelope floor, start limit)
    g_up 0.10, g_down 0.0125, center 0.4, floor 1.00

The verdict (omnicompass/verdict.py; every slow knob, the GPU ceiling and the Kubernetes machine count):
    reference r = the median cost per request at the deepest allowed step, measured fresh, n &gt;= 30 reque
    ↪ sts
    trial t     = the median cost at one step past it, n &gt;= 30 requests, under the same traffic
    allow the step  iff  t &lt;= r (1 + a)  and  t &lt;= c_native (1 + a),   a = 0.02
    a refused step is not tried again for R decisions (GPU R = 900; Kubernetes 120); the knob never goes
    ↪  past the
    deepest allowed step, so steps cannot add up beyond the allowance

Kubernetes, the compass on the HPA target (omni_controller/controller.py; sets 30 and 31):
    target x in [0.6 x_op, x_op] (times the conveyed gain); the up force lowers x (more pods)
    p &gt;= 0.95 from load (line breached, every sense live, nothing pending)  =&gt;  x = 0.6 x_op at once
    p &gt;= 0.95 from a blind sense or pods waiting for a lost machine           =&gt;  x = x_op (native's own
    ↪ )
    fault over (p &lt; center, line clean, nothing pending)                     =&gt;  x = x_op at once, neve
    ↪ r held
    machines: one more at once past the wall; one back only when the verdict and the release gate agree
    g_up 0.10, g_down 0.02, release force &lt; -0.2

The productivity arithmetic (every receipt; manual section 3.4):
    work W equal in both arms;  G = C_native / C_omni - 1;  the same work needs 1/(1 + G) of the resourc

    ↪ es;
    saving S = G / (1 + G):  G = 1/3  &lt;=&gt;  S = 1/4
```

Parameter ranges, defaults and the proof of convergence: `omnicompass/core.py`, `docs/TRACKING_THEOREM.md`, `docs/CANONICAL_ENGINE.md`.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

APPENDIX

# Appendix D. Metrics

Every gauge, where it comes from, and whether it is measured or modelled: `docs/METRICS_CATALOG.md`. The gauges that appear in the three-run tables of this manual, with their source and their direction, are these:

| Gauge | Source | Measured or modelled | Direction |
|---|---|---|---|
| work inside the response line (requests, transactions or messages a second answered within the line) | the probe's own records, pgbench's log, the consumers' records, the application's records | measured | higher is better; the product number |
| response time, mean, median, 95th and 99th percentile | the same | measured | lower is better |
| time over the response line (share of samples) | the same, against the preregistered line | measured | lower is better |
| failed requests, lost messages | the same | measured | any increase is WORSE |
| worker machines in service, mean; node-hours | the cluster's own node records every 15 s | measured (machine-hours on kind; Azure's own count on AKS) | lower is better |
| compute bill at list price | Azure's own machine count × list price per family | measured (a real bill) | lower is better |
| energy (Wh), the standby model | a declared formula over the machines in service (kind) | **modelled**, said so on every line | lower is better |
| host CPU busy; host CPU-seconds; CPU-seconds per 1,000 units inside the line | `/proc/stat` of the real machine under the test | measured | lower is better |
| Omni's own CPU (cores), mean | the governor's audit (its process and every command it ran) | measured | shown, not judged in the Kubernetes tables; a loss in all three of our runs where it is one |
| connections held open, consumers running, memory ceiling held | the pooler's, the group's, the cache's own reports | measured | lower is better (the resource held) |
| cache hit rate; consumer lag | Redis's INFO; the consumers' records | measured | higher is better; lower is better |
| HPA replicas, pods started, pod start wait | the API server's own records | measured | shown, not judged except pod start wait |
| organism work, energy, time over its line | the organism's plant models | **modelled** | by direction; S class |
| energy a mission, missions a charge (drones) | the simulator's motor constants through a declared model | **modelled** | lower; higher |
| cache size held, mean; bytes in the cache, mean; pages read into the cache (MongoDB) | the server's own `serverStatus` (WiredTiger cache) | measured | lower is better (the resource held); shown, not judged (pages read) |
| seconds from the kill to every setting back at the operator's (robustness) | the harness polling the cluster once a second from the kill | measured | lower is better; past 60 s WORSE; never, INVALID |

| Gauge | Source | Measured or modelled | Direction |
|---|---|---|---|
| the governor's resident memory, last ten minutes over the first ten (robustness, long run) | the process table, sampled every 15 s by the harness | measured | a growth over a quarter is WORSE |
| decisions made of expected; failed decisions (robustness, long run) | the governor's audit | measured | under 95% INVALID; any failed decision shown with its reason |
| mission time, position error, collisions (drones) | the simulator's own state at every step | measured in the simulator (S class) | lower; under the safe error; any collision voids the cell |
| handed back (every arm) | the knob read back at the end of the arm | measured | yes in every arm, or the run is invalid |
| off the clock | the organism's own clock against its window | measured | shown; marks the cell |

**Three rules for reading any gauge in this appendix.** A measured gauge is one the system under test, or the host's own kernel, reported about itself; a modelled gauge is one computed from a declared formula over measured inputs, and every table says which it is on the line itself, never only here. A resource gauge (machines, connections, consumers, memory, cache) is always judged lower-is-better, even where the governor's purpose was to add the resource to buy service, so that a gain bought with a resource is paid for in the same table and in the index. And a gauge marked "shown, not judged" never enters a reading or the index; it is there so that a reader can see what the governor did (how often it moved, how many pages it read, how many pods it started), not to be counted for or against it.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

APPENDIX

# Appendix E. Troubleshooting

| Symptom | Cause | Fix |
|---|---|---|
| Wire check `FAIL 3 up wire (lock)` | the driver or VM refuses clock locking | use bare metal or full passthrough; or run the one-wire governor |
| Wire check `FAIL 3 up wire (follows down)` | the card ignores the ceiling | update the driver; check for another management agent holding clocks |
| Wire check `FAIL 4 down wire` | the power limit is refused or capped by the board | check `nvidia-smi -q -d POWER`; run as root |
| `another copy of this test is already running` | a second copy was started | wait for the first or reboot; start once |
| `something else is using the GPU` | another process holds the card | stop it; the benchmark must run alone |
| Governor exit 5 | another writer changed a knob | find the other controller; Omni-Compass left its value alone |
| Governor exit 3 | a restore did not read back | restore by hand (`nvidia-smi -rgc`, `-pl <start>`); investigate before rerunning |
| `decision failed (n in a row)` | the cluster API is unreachable | turn it OFF; native runs on |
| Energy saved but p95 slower than native | the card served bursts below its own clock, or the lid sat under its own draw | read section 8.5; in the audit compare `telemetry.clock_mhz` while busy with `native_busy_clock_mhz`, and `want_w` with `native_busy_draw_w` |
| Fail-up in most decisions | `--slo-ms` set too low for the workload, or a stale feed read as blind | set `--slo-ms` from the workload's own target; check the feed's age |
| No saving at all, service unchanged | the card is saturated almost all the time (it races), or the service sits above the compass's center | expected on a card with no slack; the saving comes from the quiet stretches |
| Kubernetes: no machine ever given back | the release gate refuses (its reason is in the audit), or the service is above the compass's center | read `node_gate.reason` in the audit; check `--slo-ms` and the latency feed |
| Kubernetes: a repetition marked OFF THE CLOCK | the machine could not step the organism inside its window | a longer window for that size on that machine; never a faster reading of the same run |
| Broker: consumers climb to the cover and stay | messages wait at every step (the producer is past the group's capacity) | the benchmark's capacity step sets the peak at nine tenths of the operator's group; check the measured capacity in the run's log |
| Cache: the ceiling grows while little memory is used | cold misses read as pressure | the full-cache gate is the rule (used ≥ 90% of the ceiling); check `used_mb` beside `maxmemory_mb` in the run's log |
| Cache: the ceiling never grows | the working set fits the operator's ceiling | expected: a cache that fits has nothing for the ceiling to mend, and the row reads inside the noise |

| Symptom | Cause | Fix |
|---|---|---|
| Storage-engine cache: the cache never fills, whatever the key space, and omni reads as a free memory saving | the benchmark's requests concentrate on a few hot records (YCSB's shipped zipfian draw), or the run asks for records the load did not write (hashed keys over an ordered load), so the reads are served from the index | a result that flatters the governor is the first thing to suspect: check "in cache" against the cache size in the per-notch lines; draw requests uniformly over the key space and name the same key order in load and run; a knob with nothing to do is not a saving |
| Storage-engine cache: the compass only ever reads calm | the line is set far above the stack's own latencies (a 2 ms line over 0.15 to 0.45 ms reads) | set the line on the tuning workload's own figures so that a hit is calm and a full-cache miss is slow, and write the figures in the preregistration before any counted run |
| Swarm: a cell reads "void" | two drones came within two collision radii in either arm | a task or simulator setting (keep-out, altitude stagger, depot spacing); fixed before any counted run and said in the preregistration |
| Three-run table heads itself with a warning | a run's commit is not the engine it claims, or the three are not separate runs | run `tools/omni_version.py --commit <sha>` on each; make the table only from runs on one frozen engine |
| Archive bot skips a run | the run was not finished when the bot passed | leave the id in `.github/archive_request.txt`; the next pass copies it |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

APPENDIX

# Appendix F. Evidence Map

`docs/EVIDENCE_LEDGER.md` (every claim and its class), `docs/CLAIMS_REGISTER.md` (what is claimed and what is not), `docs/GPU_PREREGISTRATION.md`, `docs/REALMS_PREREGISTRATION.md`, `docs/K8S_COMPASS_PREREGISTRATION.md`, `docs/POSTGRES_PREREGISTRATION.md`, `docs/KAFKA_PREREGISTRATION.md`, `docs/REDIS_PREREGISTRATION.md`, `docs/SWARM_PREREGISTRATION.md`, `docs/CITYLEARN_PREREGISTRATION.md`, `docs/PANDAPOWER_PREREGISTRATION.md`, `docs/ROBOTICS_PREREGISTRATION.md`, `docs/ROBUSTNESS_PREREGISTRATION.md`, `docs/YCSB_PREREGISTRATION.md` and `docs/MYSQL_PREREGISTRATION.md` (the rules written before each run), `docs/OMNI_V1.md` to `OMNI_V3.md` (every result by engine), `docs/REGISTER.md` (every benchmark and its file), `docs/PROOF_PROGRAM.md` (the program to full size with its costs), `docs/DOSSIER.md` (every result in one place), `results/OMNI_INDEX.md` (the one number), `docs/WIRING_VERDICTS.md` (which knobs earn a wire in, and every loss with its cause), `docs/STATE_OF_PLAY.md` (where everything stands), `docs/HANDOFF.md` (every command in one page).

**The map, claim by claim.** Each claim this manual makes is listed here with the class of evidence behind it and the document that carries it, so that a reader can go from any sentence of the executive summary to the file that would have to be wrong for the sentence to be wrong.

| Claim | Class | Where the evidence is | What would falsify it |
|---|---|---|---|
| The engine's six states stay inside their walls and settle | T / V (proof and test) | `docs/FORMAL_STATUS.md`, `docs/TRACKING_THEOREM.md`, `tests/`, `results/SOAK.json` | a trajectory leaving its wall, or the C++ twin disagreeing with the Python |
| No write ever leaves a knob's cover; a blind sense holds; fail-up is immediate | T / V | `tests/test_shield_properties.py` (two million cases), the plug tests per stack | one case of a write past the cover or a give-back while blind |
| On a real cluster the governor does more work inside the line, faster, on no more machines, at no more energy | L | the six Kubernetes tables in `results/live/`, A/B/C each, `docs/K8S_COMPASS_PREREGISTRATION.md` | a row worse in all three of our runs on work, speed, machines or energy in any of the six, which would stand in the table |
| Killed outright, the governor's settings are back at the operator's within seconds | L | `results/live/V3_ROBUST_KILL.md`, `docs/ROBUSTNESS_PREREGISTRATION.md` | a repetition handed back after 60 s, or not at all |
| Over two hours the governor neither leaks, slows nor drifts, and hands everything back at the end | L | `results/live/V3_ROBUST_LONG.md`, `docs/ROBUSTNESS_PREREGISTRATION.md` | memory growing past a quarter, the decision time growing past a half, fewer than 95% of expected decisions, or a setting not handed back |
| Governing costs a few thousandths of a core at every size | L | `results/live/V3_OWN_COST.md` | an own-cost figure growing with the organism |

| Claim | Class | Where the evidence is | What would falsify it |
|---|---|---|---|
| The pooler, the buffer pool and the storage-engine cache each give memory back on an untouched workload with nothing worse; the broker and the cache read nothing on the first set under the brain's verdict | L | `V3_PGBENCH.md`, `V3_KAFKA.md`, `V3_REDIS.md`, each preregistration | a product row worse in all three of our runs on an untouched workload; the memory row of the cache is such a loss and stands |
| The one number is +4.8% across the real categories on the sets of 10 October, losses included | L, by rule | `results/OMNI_INDEX.md`, `tools/omni_index.py` | a category omitted, a loss not entered, a tuning row counted |
| A day of demand measured by someone else reads the same way as our own schedules | L | `results/live/V3_TRACE_GOOGLE2011.md`, `results/traces/google2011/schedule.json`, `docs/TRACES_PREREGISTRATION.md` | a schedule derived after the runs; a rule fitted to the governor; a trace part whose SHA-256 does not match |
| The modelled realms, grids, arms, buildings and swarms gain under their own native controllers | S | `results/realms/`, `V3_PANDAPOWER.md`, `V3_MUJOCO.md`, `V3_CITYLEARN.md`, `V3_SWARM.md` | a run of the same simulator at the same version and seed giving other digits |
| A physical meter shows less energy for the same work | P | **no current result**; `docs/GPU_PREREGISTRATION.md` names the run | the rerun on the current card controller reading no difference or worse |
| Omni-Compass never changes the engine between a rule and its result | by construction | `OMNI_V3.json`, `tools/omni_version.py --commit <sha>` on every table's commits | a table whose runs' commits carry different fingerprints |

**What a referee asks of each document.** Of a preregistration: is it dated before the first counted run, does it name the tuning case and the untouched cases, is every amendment dated and does any amendment made after a result was seen say so? Of a three-run table: are the three run ids separate runs, does the head name the engine at each commit, is every gauge of the preregistration present including the ones that went against the governor, and is the reading one of the three the rule allows? Of the index: is every real category in it, is every tuning row out of it, and does a loss in all three of our runs enter as a loss? Of the raw folder: does the SHA-256 manifest in it match the files, does the governor's audit show every decision with its reading and its write, and does the hand-back record read the operator's value? Every one of those questions has a yes in this program or a disclosure naming the exception; the disclosures are in section 16.7.

**How to audit one result from this manual to its raw files.** Take any row of Chapter 43. Its source column names a three-run table in `results/live/`; the table's head names the three GitHub run ids, the commit each ran on and the engine `tools/omni_version.py` reads at that commit. Each run id names a folder `results/live/raw/run-<id>/` holding every repetition's files (the capture, the response times, the governor's audit and log, the per-request records, one SHA-256 manifest per repetition). The table tool named in the table's text (`tools/confirm_abc.py` or its siblings) rebuilds the table from those folders; `tools/omni_index.py` rebuilds the index from the tables; `tools/dossier.py` rebuilds the dossier. The preregistration named in the table's text holds the rules, written before the first counted run, with every amendment dated. Nothing in that chain is by hand.

**What one repetition leaves behind.** Take the first repetition of the first kill-scenario run, archived at `results/live/raw/run-37716845219/paired-1/`. It holds two folders, one per arm, and the omni arm's folder holds twenty-nine files, every one of which a reader can open: `preflight.txt` (the UTC time, the arm, the git commit the arm ran at, the versions of Docker, kind, kubectl, the API server and Python, the metrics-server version with its SHA-256, the worker count and the probe's address); `rbac_omni.txt` (twenty lines of `kubectl auth can-i`, each verb the governor needs reading yes and each it must not have reading no); `window_start.txt` and

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

`window_end.txt` (the window's clock); `load_schedule.log` (every load step and when it was applied); `latency.csv` (the probe's response times, one row a second); `capture.csv` (the cluster's state every fifteen seconds: replicas, workers, pods pending, the HPA's target); `host_cpu.csv` (the runner's own `/proc/stat`); `pod_watch.json` and `pods_end.json`, `nodes_end.txt`, `hpa_end.json` (the API server's own records at the end); `audit.jsonl` (one line per decision: the compass's position, heading and letter, the engine's state, every write with its value sent and its value read back); `controller.log` and `controller2.log` (the first governor's log, and the second one's after the kill); `audit_kill.jsonl` (the watchdog's restore writes, each with its command and its reason, such as "reset: remove record"); `watchdog.log` (one record: which governor, its process id, whether it was hung or dead, every command run and each exit code); `robust.log` (the kill at the 360th second with the settings as they stood, "target=50 range=1,10 limits=1850m 3700m workers=5/6", and the hand-back "every setting at the operator's after 8 s, target=50 limits=500m workers=6/6 left=none"); `rss.csv` (the governor's resident memory every fifteen seconds); `omni_writes.txt` (how many writes the governor made in the window: 42); `kill_switch.txt` (the reset check: target restored, workers in service, no record left); `omni2.pid` (the second governor's process id) and `kill` (the marker the harness wrote at the kill); `robust.out` (the same robustness log as the job printed it); `end_reads.err`, `pod_watch.err` and `capture.csv.errors` (empty, as they should be); `metrics-server-components.yaml` (the pinned manifest as applied); and `SHA256SUMS.txt`, one line per file. A reader who doubts a number in the kill table can find the second it was taken.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

APPENDIX

# Appendix G. The Muscles: What Each Is For, and How It Is Wired

A **muscle** is one setting on one machine that already has its own control: a replica target, a node pool's size, a GPU's clock ceiling, a chiller's setpoint, a battery's reserve, a joint's effort. Omni-Compass does not replace that control. It reads the machine's meters, computes one bounded force with the compass law, and moves the setting the machine already accepts, through a plug that reads the setting once before the first write, reads back every write, steps aside if another controller moves it, and puts it back at the end.

This chapter lists all 945 muscles of the catalog (`realms/catalog.csv`). For each one it says what kind of machine it is, which of the four kinds of knob it is, what Omni-Compass reads and does with it, how such a muscle is reached in a real stack, and which organisms it belongs to. The plants behind the benchmark numbers are models of these machines (evidence class **S**); a muscle in this list is wired on a real system only through the levels and the checks of the manual (chapters 8 and 9). See `DISCLOSURES.md`.

## How to read an entry

**The four kinds of knob.**

- **capacity**: how much of the machine is in service (replicas, instances, units, speed). Omni-Compass writes it.
- **admission**: what is let in (queues, rate limits, admission control). Omni-Compass reads it and leaves it to its own controller in the current law; it is counted, never written.
- **power**: the power share or limit the machine may draw. Omni-Compass writes it inside its cover.
- **setpoint**: the target the machine's own loop holds (a temperature, a reserve, a process value). Omni-Compass moves it inside its safe band.

**The five kinds of plant.** Every muscle is modelled on one of five plants, each with its own service reading and lever:

| Plant | What it is | What Omni-Compass reads | What Omni-Compass moves |
|---|---|---|---|
| `compute_pool` | a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs | how far its queue and its load sit toward the service line (queue or lateness, and load above half) | the capacity it may use: its replica or instance count (through its autoscaler's target or floor), its power share, or one unit given back through the release gate |
| `thermal_zone` | a data hall or building zone cooled by chillers, air handlers or fans | its temperature against its limit | its supply setpoint (colder is more cooling), its power share, or one cooling unit given back when the room has margin |
| `energy_storage` | a battery, UPS or microgrid store with a load and a grid connection | its draw on the grid connection and its reserve | its reserve setpoint (how much charge it holds back); its power limit is left native |

| Plant | What it is | What Omni-Compass reads | What Omni-Compass moves |
|---|---|---|---|
| `motion_axis` | a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor | its tracking error and its load | its speed and effort share, or its power share |
| `process_loop` | a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint | how far the process sits from its band | its setpoint within its safe band, its actuator range, or its power share |

**The organisms.** The 945 muscles build six organisms: each of the four realms (every muscle whose realm list includes it), the four stacked with every duplicate kept (1,716), and the whole tower with every muscle once (945). A muscle of the shared spine sits in all four realms.

| Organism | Muscles |
|---|---|
| Compute / AI / Cloud | 430 |
| Physics / Robotics / Autonomous | 376 |
| Energy / Facility / Industrial | 470 |
| Distribution / Specialized | 440 |
| The four stacked, every duplicate kept | 1,716 |
| The whole tower, every muscle once | 945 |

**How a muscle is reached in a real stack.** The class of machine decides the wire:

| Class of machine | What it is | The wire in a real stack |
|---|---|---|
| `batch` | batch and queued jobs | the job queue (Kubernetes Jobs, Slurm, Spark) |
| `batch_reactor` | a GMP batch reactor, fermenter or food process | the batch control system (ISA-88, OPC UA) |
| `building` | a building zone | the building management system |
| `chamber` | a controlled chamber (clean room, kiln, reactor) | the PLC or DCS |
| `clinical` | clinical systems: imaging archives, records, interface engines, monitoring gateways | the application's own scaling and admission settings (Kubernetes, the vendor console) |
| `commerce` | a customer-facing service with bursts | the Kubernetes API |
| `cpu_host` | a CPU host whose clock and power can be set | Linux cpufreq and RAPL |
| `data_hall` | a data hall cooling plant | the building management system (BACnet, Modbus) or its command line |
| `database` | a database cluster | the database operator (replicas, connection pools) |
| `district_heat` | a district heating or cooling network | the network control system and substation controllers (Modbus, M-Bus) |
| `elevator_hoist` | an elevator or escalator drive and its group control | the lift controller and destination dispatch (modelled only) |
| `ev_traction` | an electric vehicle traction motor | the vehicle's motor controller (CAN; modelled only) |
| `fabric` | a high-speed fabric (RDMA, DPU, SmartNIC) | the fabric manager's API |
| `facility` | a facility battery behind the meter | the energy management system |
| `feeder_voltage` | a distribution feeder's voltage | the distribution management system (IEC 61850, DNP3; modelled only) |
| `flight_axis` | a flight-control axis | the flight controller (modelled only) |
| `gpu` | GPU serving with a response-time target | nvidia-smi (clock ceiling and power limit) or the serving autoscaler |
| `gpu_batch` | GPU training and batch work | nvidia-smi, the job scheduler's pause and resume |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| Class of machine | What it is | The wire in a real stack |
|---|---|---|
| `hospital` | a hospital's critical rooms (operating rooms, isolation, pharmacy, imaging suites) | the hospital's building management system (BACnet) |
| `inverter` | a plant of grid-support inverters or wind turbines | the plant controller and inverter settings (SunSpec, IEC 61850, IEEE 1547) |
| `irrigation` | pumps, pressure and climate on a farm or in a greenhouse | the pump station controller, the pivot panel or the climate computer (Modbus, the vendor cloud) |
| `marine_propulsion` | a ship's propulsion shaft and power plant | the vessel automation and power management system (modelled only) |
| `microgrid` | a microgrid battery and loads | the inverter or energy management system (IEEE 2030.5, SunSpec, OpenADR) |
| `mill` | a grinding, flotation or materials-handling circuit | the plant DCS or PLC (OPC UA) |
| `network` | network routing and switching | the network controller (SDN, routing API) |
| `node` | a fleet of machines or VMs that boot in minutes | the node pool's size command (Karpenter, Cluster Autoscaler, MachineSet, cloud ASG) |
| `pipeline` | a pipeline segment with its compressors, pumps and valves | pipeline SCADA (DNP3, Modbus, OPC UA) |
| `port` | a container terminal's cranes and vehicles taking moves | the terminal operating system and fleet controllers |
| `process` | an industrial process loop | the PLC or DCS (OPC UA, EtherNet/IP, PROFINET) |
| `qpu` | a quantum or specialised accelerator queue | the accelerator's job queue |
| `rail_traction` | a train's traction and auxiliary systems | the train control and management system and ATO (modelled only) |
| `ran` | a radio access network cell or site | the RAN controller (O-RAN interfaces) |
| `reaction_wheel` | a spacecraft reaction wheel | the attitude controller (modelled only) |
| `robot_fleet` | a fleet of robots or vehicles taking tasks | the fleet manager's dispatch API |
| `robot_joint` | a robot joint servo | the robot controller (ROS 2, EtherCAT, the drive's fieldbus) |
| `server` | an application service on servers or pods | the Kubernetes API (HPA target and floor, pod resize) |
| `storage` | block, file or object storage | the storage system's QoS and tiering API |
| `turbine` | a generating unit's governor, boiler or excitation loop | the plant DCS and governor (OPC UA, IEC 61850; modelled only) |
| `ups` | an uninterruptible power supply | the UPS and PDU management interface (SNMP, Modbus) |
| `water` | a water or pumping process | the SCADA system (Modbus, DNP3) |
| `workflow` | a workflow or pipeline engine | the workflow engine's concurrency settings |

Wires marked *modelled only* exist as plants in the benchmark and are not built for live use.

## The shared spine: 257 muscles in all four realms

The machines every realm stands on: servers, machines, GPUs and CPUs, network, storage, observability, security, cooling and electrical distribution. Each spine muscle is part of every realm's organism, counted once in the tower and four times in the stack.

### *Cloud VM & Capacity (21 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `node` (a fleet of machines or VMs that boot in minutes), reached through the node pool's size command (Karpenter, Cluster Autoscaler, MachineSet, cloud ASG). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 97 | vm count | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 98 | vm size | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 99 | vm start stop | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 100 | vm migrate | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 101 | instance family | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 102 | asg min | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 103 | asg max | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 104 | spot mix | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 105 | reservation mix | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 106 | zone selection | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 107 | region selection | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 108 | architecture selection | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 109 | accelerator selection | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 110 | boot disk class | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 111 | placement group | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 112 | interruption response | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1038 | target tracking policy | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | all four realms; stack; tower |
| 1039 | scaling cooldown warmup | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 1114 | vm cpu cap by priority | admission | reads it; left to its own controller | all four realms; stack; tower |
| 1120 | predictive scaling | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1121 | instance refresh | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |

### *Container Resources (23 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 33 | cpu request | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 34 | cpu limit | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 35 | memory request | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 36 | memory limit | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 37 | ephemeral storage | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 38 | hugepages | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 39 | io weight | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | all four realms; stack; tower |
| 40 | pids limit | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 41 | memory high | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 42 | memory reclaim | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 43 | swap limit | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 44 | cgroup io max | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 45 | cpu weight | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | all four realms; stack; tower |
| 46 | cpu quota | admission | reads it; left to its own controller | all four realms; stack; tower |
| 47 | cpuset | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 48 | runtime class | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1017 | in place resize | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1019 | namespace quota | admission | reads it; left to its own controller | all four realms; stack; tower |
| 1020 | default limits | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1022 | memory hard limit | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1023 | memory protection | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1024 | io latency target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | all four realms; stack; tower |
| 1025 | freeze | admission | reads it; left to its own controller | all four realms; stack; tower |

### *Host CPU & Memory (23 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `cpu_host` (a CPU host whose clock and power can be set), reached through Linux cpufreq and RAPL. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 113 | cpufreq min | power | holds the power share inside its cover; full at once past the wall | all four realms; stack; tower |
| 114 | cpufreq max | power | holds the power share inside its cover; full at once past the wall | all four realms; stack; tower |
| 115 | rapl package power | power | holds the power share inside its cover; full at once past the wall | all four realms; stack; tower |
| 116 | uncore frequency | power | holds the power share inside its cover; full at once past the wall | all four realms; stack; tower |
| 117 | energy perf preference | power | holds the power share inside its cover; full at once past the wall | all four realms; stack; tower |
| 118 | cpu idle policy | power | holds the power share inside its cover; full at once past the wall | all four realms; stack; tower |
| 119 | memory bandwidth | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 120 | numa balance | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 121 | irq affinity | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 122 | llc allocation | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 123 | memory pressure gate | admissi on | reads it; left to its own controller | all four realms; stack; tower |
| 124 | page reclaim rate | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 125 | transparent hugepages | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 126 | core online offline | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 127 | thermal throttle policy | power | holds the power share inside its cover; full at once past the wall | all four realms; stack; tower |
| 128 | host power profile | power | holds the power share inside its cover; full at once past the wall | all four realms; stack; tower |
| 1040 | frequency governor | power | holds the power share inside its cover; full at once past the wall | all four realms; stack; tower |
| 1041 | turbo boost | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1042 | dram power limit | power | holds the power share inside its cover; full at once past the wall | all four realms; stack; tower |
| 1043 | swappiness reclaim | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1112 | core packing | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1115 | per pod cpu frequency range and c state access | power | holds the power share inside its cover; full at once past the wall | all four realms; stack; tower |
| 1116 | uncore frequency per node | power | holds the power share inside its cover; full at once past the wall | all four realms; stack; tower |

### *Kubernetes Placement & Scheduling (20 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 49 | node selector | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 50 | node affinity | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 51 | pod anti affinity | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 52 | topology spread | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 53 | numa placement | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 54 | gpu topology | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 55 | storage locality | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 56 | network locality | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 57 | taint toleration | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | all four realms; stack; tower |
| 58 | priority class | admissio n | reads it; left to its own controller | all four realms; stack; tower |
| 59 | preemption policy | admissio n | reads it; left to its own controller | all four realms; stack; tower |
| 60 | device claim | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 61 | failure domain spread | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 62 | scheduler backoff | admissio n | reads it; left to its own controller | all four realms; stack; tower |
| 63 | gang admission | admissio n | reads it; left to its own controller | all four realms; stack; tower |
| 64 | deschedule | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1026 | scheduler scoring strategy | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1027 | scheduler plugin weights | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | all four realms; stack; tower |
| 1028 | scheduling gates | admissio n | reads it; left to its own controller | all four realms; stack; tower |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 1029 | cpu manager | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |

### *Kubernetes Workload Scaling (31 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 17 | replicas | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 18 | hpa cpu target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | all four realms; stack; tower |
| 19 | hpa memory target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | all four realms; stack; tower |
| 20 | hpa custom target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | all four realms; stack; tower |
| 21 | vpa apply | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 22 | scale to zero | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 23 | keda threshold | admission | reads it; left to its own controller | all four realms; stack; tower |
| 24 | rollout rate | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 25 | max surge | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 26 | max unavailable | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 27 | deployment pause resume | admission | reads it; left to its own controller | all four realms; stack; tower |
| 28 | rollout abort | admission | reads it; left to its own controller | all four realms; stack; tower |
| 29 | pod eviction | admission | reads it; left to its own controller | all four realms; stack; tower |
| 30 | pdb policy | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 31 | scheduler queue priority | admission | reads it; left to its own controller | all four realms; stack; tower |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 32 | api priority fairness | admissio n | reads it; left to its own controller | all four realms; stack; tower |
| 1001 | hpa min replicas | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1002 | hpa max replicas | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1003 | hpa scale down stabilisation window | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | all four realms; stack; tower |
| 1004 | hpa scale up stabilisation window | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | all four realms; stack; tower |
| 1005 | hpa scaling rate policies | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1006 | hpa policy selection | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1007 | hpa tolerance | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1008 | hpa sync period | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1009 | vpa minmax allowed resources | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1010 | keda polling interval | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | all four realms; stack; tower |
| 1011 | rollout progress deadline | admissio n | reads it; left to its own controller | all four realms; stack; tower |
| 1012 | rollout readiness delay | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1014 | unhealthy pod eviction policy | admissio n | reads it; left to its own controller | all four realms; stack; tower |
| 1015 | graceful termination | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1016 | probes | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |

### *NVIDIA GPU Hardware (19 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `gpu` (GPU serving with a response-time target), reached through nvidia-smi (clock ceiling and power limit) or the serving autoscaler. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 129 | gpu allocate | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 130 | gpu power limit | power | holds the power share inside its cover; full at once past the wall | all four realms; stack; tower |
| 131 | gpu sm clock | power | holds the power share inside its cover; full at once past the wall | all four realms; stack; tower |
| 132 | gpu memory clock | power | holds the power share inside its cover; full at once past the wall | all four realms; stack; tower |
| 133 | gpu persistence mode | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 134 | gpu compute mode | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 135 | mig mode | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 136 | mig geometry | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 137 | gpu timeslice | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 138 | gpu mps | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 139 | gpu quarantine | admission | reads it; left to its own controller | all four realms; stack; tower |
| 140 | gpu reset | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 141 | gpu thermal limit | power | holds the power share inside its cover; full at once past the wall | all four realms; stack; tower |
| 142 | gpu ecc response | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 143 | gpu job power budget | admission | reads it; left to its own controller | all four realms; stack; tower |
| 1044 | application clocks | power | holds the power share inside its cover; full at once past the wall | all four realms; stack; tower |
| 1045 | gpu temperature target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | all four realms; stack; tower |
| 1117 | sync boost | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1118 | target clocks | power | holds the power share inside its cover; full at once past the wall | all four realms; stack; tower |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

### *Node Fleet & Karpenter-Class Control (24 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `node` (a fleet of machines or VMs that boot in minutes), reached through the node pool's size command (Karpenter, Cluster Autoscaler, MachineSet, cloud ASG). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 65 | node desired | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 66 | node pool min | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 67 | node pool max | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 68 | node provision | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 69 | node cordon | admissio n | reads it; left to its own controller | all four realms; stack; tower |
| 70 | node drain | admissio n | reads it; left to its own controller | all four realms; stack; tower |
| 71 | node consolidate | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 72 | node replace | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 73 | node shutdown | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 74 | node power on | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 75 | nodepool weight | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | all four realms; stack; tower |
| 76 | disruption budget | admissio n | reads it; left to its own controller | all four realms; stack; tower |
| 77 | consolidation policy | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 78 | consolidate after | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 79 | expire after | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 80 | capacity class | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 1030 | scale down utilisation threshold | admissio n | reads it; left to its own controller | all four realms; stack; tower |
| 1031 | scale down unneeded time | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1032 | scale down delay after add | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1033 | expander | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1034 | node pool limits | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1035 | kubelet eviction thresholds | admissio n | reads it; left to its own controller | all four realms; stack; tower |
| 1036 | kubelet reserved resources | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1037 | max pods per node | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |

### *Network Routing & Switching (19 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `network` (network routing and switching), reached through the network controller (SDN, routing API). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 225 | lb weight | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | all four realms; stack; tower |
| 226 | route weight | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | all four realms; stack; tower |
| 227 | rate limit | admissio n | reads it; left to its own controller | all four realms; stack; tower |
| 228 | bandwidth limit | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 229 | qos class | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 230 | connection limit | admissio n | reads it; left to its own controller | all four realms; stack; tower |
| 231 | failover route | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 232 | nic rate limit | admissio n | reads it; left to its own controller | all four realms; stack; tower |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 233 | nic queue count | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 234 | queue discipline | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 235 | congestion control | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 236 | egress budget | admission | reads it; left to its own controller | all four realms; stack; tower |
| 237 | ingress budget | admission | reads it; left to its own controller | all four realms; stack; tower |
| 238 | ecmp weight | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | all four realms; stack; tower |
| 239 | path selection | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 240 | network isolation | admission | reads it; left to its own controller | all four realms; stack; tower |
| 1046 | nic interrupt coalescing | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1047 | nic ring size | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1048 | port link power | power | holds the power share inside its cover; full at once past the wall | all four realms; stack; tower |

### *Observability & Telemetry (9 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 373 | collector memory limit | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 374 | export concurrency | admission | reads it; left to its own controller | all four realms; stack; tower |
| 376 | cardinality budget | admission | reads it; left to its own controller | all four realms; stack; tower |
| 377 | retention window | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | all four realms; stack; tower |
| 378 | remote write queue | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 379 | telemetry shed | admissio n | reads it; left to its own controller | all four realms; stack; tower |
| 1053 | metric scrape interval | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | all four realms; stack; tower |
| 1054 | batching | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1119 | per pod energy attribution | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |

### *Reliability, Security & Recovery (12 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 385 | restart | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 386 | rollback | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 388 | traffic divert recovery | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 389 | degraded mode | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 390 | actuator freeze | admissio n | reads it; left to its own controller | all four realms; stack; tower |
| 391 | workload isolate | admissio n | reads it; left to its own controller | all four realms; stack; tower |
| 393 | credential rotation gate | admissio n | reads it; left to its own controller | all four realms; stack; tower |
| 394 | policy enforcement | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 395 | rate abuse gate | admissio n | reads it; left to its own controller | all four realms; stack; tower |
| 396 | fault domain isolate | admissio n | reads it; left to its own controller | all four realms; stack; tower |
| 397 | backup trigger | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 400 | kill switch | admissio n | reads it; left to its own controller | all four realms; stack; tower |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

### *Storage Block/File/Object (20 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `storage` (block, file or object storage), reached through the storage system's QoS and tiering API. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 257 | volume size | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 258 | iops limit | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 259 | throughput limit | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 260 | replica count | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 261 | storage tier | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 262 | volume placement | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 263 | snapshot trigger | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 264 | rebalance | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 265 | recovery rate | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 266 | backfill rate | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 267 | compaction pressure | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 268 | cache allocation | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 269 | object replication | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 270 | erasure code profile | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 271 | storage admission | admission | reads it; left to its own controller | all four realms; stack; tower |
| 272 | degraded storage gate | admission | reads it; left to its own controller | all four realms; stack; tower |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 1049 | scrub schedule | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1050 | io scheduler | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1051 | disk power spin down | power | holds the power share inside its cover; full at once past the wall | all four realms; stack; tower |
| 1052 | lifecycle tiering | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |

### *Cooling, Chillers & Thermodynamics (18 muscles)*

Each is a data hall or building zone cooled by chillers, air handlers or fans; its class is `data_hall` (a data hall cooling plant), reached through the building management system (BACnet, Modbus) or its command line. Omni-Compass reads its temperature against its limit. It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 545 | supply air temperature | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | all four realms; stack; tower |
| 546 | return air target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | all four realms; stack; tower |
| 547 | coolant supply temperature | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | all four realms; stack; tower |
| 548 | coolant flow | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 549 | pump speed | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 550 | fan speed | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 551 | chiller setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | all four realms; stack; tower |
| 552 | compressor authority | power | holds the power share inside its cover; full at once past the wall | all four realms; stack; tower |
| 553 | cooling tower fan | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 554 | cooling capacity | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 555 | rack thermal budget | power | holds the power share inside its cover; full at once past the wall | all four realms; stack; tower |
| 556 | gpu thermal envelope | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | all four realms; stack; tower |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 557 | cpu thermal envelope | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | all four realms; stack; tower |
| 558 | thermal workload migrate | admission | reads it; left to its own controller | all four realms; stack; tower |
| 559 | thermal load shed | admission | reads it; left to its own controller | all four realms; stack; tower |
| 1055 | server fan speed | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1056 | economiser | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1122 | chiller plant supervisory control rl with safety layer | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |

### *PDU, UPS & Electrical Distribution (18 muscles)*

Each is a battery, UPS or microgrid store with a load and a grid connection; its class is `ups` (an uninterruptible power supply), reached through the UPS and PDU management interface (SNMP, Modbus). Omni-Compass reads its draw on the grid connection and its reserve. It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 529 | server power cap | power | holds the power share inside its cover; full at once past the wall | all four realms; stack; tower |
| 530 | rack power cap | power | holds the power share inside its cover; full at once past the wall | all four realms; stack; tower |
| 531 | pdu branch power limit | power | holds the power share inside its cover; full at once past the wall | all four realms; stack; tower |
| 532 | pdu outlet control | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 534 | ups operating mode | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 535 | ups charge rate | power | holds the power share inside its cover; full at once past the wall | all four realms; stack; tower |
| 536 | ups discharge rate | power | holds the power share inside its cover; full at once past the wall | all four realms; stack; tower |
| 537 | phase balance | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 538 | load transfer | admission | reads it; left to its own controller | all four realms; stack; tower |
| 539 | reactive power target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | all four realms; stack; tower |
| 540 | voltage target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | all four realms; stack; tower |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 541 | generator dispatch | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 542 | electrical isolation | admission | reads it; left to its own controller | all four realms; stack; tower |
| 543 | breaker trip gate | admission | reads it; left to its own controller | all four realms; stack; tower |
| 1057 | ups shutdown battery test | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | all four realms; stack; tower |
| 1110 | hierarchical power budget row pdu switchboard controllers | power | holds the power share inside its cover; full at once past the wall | all four realms; stack; tower |
| 1111 | priority aware capping throttle low priority first | power | holds the power share inside its cover; full at once past the wall | all four realms; stack; tower |
| 1113 | power oversubscription with prediction | power | holds the power share inside its cover; full at once past the wall | all four realms; stack; tower |

# Realm: Compute / AI / Cloud (173 muscles of its own, 430 in its organism with the spine)

## *AI Inference Serving (24 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `gpu` (GPU serving with a response-time target), reached through nvidia-smi (clock ceiling and power limit) or the serving autoscaler. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 161 | model replicas | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 162 | model route weight | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Compute; stack; tower |
| 163 | model load | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 164 | model unload | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 165 | model instance count | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 166 | continuous batching | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 167 | max batch size | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 168 | batch queue delay | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 169 | inference concurrency | admissio n | reads it; left to its own controller | Compute; stack; tower |
| 170 | inference max tokens | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 171 | kv cache budget | admissio n | reads it; left to its own controller | Compute; stack; tower |
| 172 | prefix cache budget | admissio n | reads it; left to its own controller | Compute; stack; tower |
| 173 | speculative decode budget | admissio n | reads it; left to its own controller | Compute; stack; tower |
| 174 | model precision | power | holds the power share inside its cover; full at once past the wall | Compute; stack; tower |
| 175 | inference priority | admissio n | reads it; left to its own controller | Compute; stack; tower |
| 176 | inference slo gate | admissio n | reads it; left to its own controller | Compute; stack; tower |
| 1058 | autoscaling target concurrency qps | admissio n | reads it; left to its own controller | Compute; stack; tower |
| 1059 | scale to zero panic window | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Compute; stack; tower |
| 1060 | request priority levels | admissio n | reads it; left to its own controller | Compute; stack; tower |
| 1061 | kv cache swap space | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 1062 | chunked prefill | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 1063 | tensor parallel degree | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 1064 | disaggregated prefill decode | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 1065 | lora adapter load | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |

### *AI Training (18 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `gpu_batch` (GPU training and batch work), reached through nvidia-smi, the job scheduler's pause and resume. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 177 | training workers | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 178 | global batch size | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 179 | microbatch size | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 180 | gradient accumulation | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 181 | data parallelism | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 182 | tensor parallelism | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 183 | pipeline parallelism | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 184 | expert parallelism | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 185 | checkpoint interval | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Compute; stack; tower |
| 186 | checkpoint trigger | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 187 | training preempt | admission | reads it; left to its own controller | Compute; stack; tower |
| 188 | training gang size | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 189 | elastic worker count | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 190 | straggler mitigation | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 191 | training precision | power | holds the power share inside its cover; full at once past the wall | Compute; stack; tower |
| 192 | compute comm overlap | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 1066 | activation checkpointing | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 1067 | dataloader workers | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

### *Cross-Cluster, Multi-Region & Edge (15 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `node` (a fleet of machines or VMs that boot in minutes), reached through the node pool's size command (Karpenter, Cluster Autoscaler, MachineSet, cloud ASG). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 609 | multi cluster dispatch | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 610 | region dispatch | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 611 | zone dispatch | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 612 | edge dispatch | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 613 | cloud capacity class | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 614 | workload migrate region | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 615 | data residency gate | admission | reads it; left to its own controller | Compute; stack; tower |
| 616 | latency region gate | admission | reads it; left to its own controller | Compute; stack; tower |
| 617 | cost region gate | admission | reads it; left to its own controller | Compute; stack; tower |
| 618 | carbon region gate | admission | reads it; left to its own controller | Compute; stack; tower |
| 619 | global failover | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 620 | federation quota | admission | reads it; left to its own controller | Compute; stack; tower |
| 621 | cross cluster replication | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 622 | edge offload | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 623 | global admission | admission | reads it; left to its own controller | Compute; stack; tower |

### *DPU SmartNIC & Programmable IO (12 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `fabric` (a high-speed fabric (RDMA, DPU, SmartNIC)), reached through the fabric manager's API. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| AUDIT-0021 | dpu pf tx rate | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| AUDIT-0022 | dpu vf tx rate | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| AUDIT-0023 | dpu sf tx rate | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| AUDIT-0024 | dpu bandwidth share | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| AUDIT-0025 | dpu qos group | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| AUDIT-0026 | sriov vf count | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| AUDIT-0027 | smartnic flow steering | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| AUDIT-0028 | smartnic offload enable | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| AUDIT-0029 | dpu cpu budget | admission | reads it; left to its own controller | Compute; stack; tower |
| AUDIT-0030 | dpu memory budget | admission | reads it; left to its own controller | Compute; stack; tower |
| AUDIT-0031 | dpu service placement | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| AUDIT-0032 | dpu failover | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |

### *Distributed Cluster Managers (15 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `batch` (batch and queued jobs), reached through the job queue (Kubernetes Jobs, Slurm, Spark). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 209 | task admission | admission | reads it; left to its own controller | Compute; stack; tower |
| 210 | task priority | admission | reads it; left to its own controller | Compute; stack; tower |
| 211 | resource reservation | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 212 | cluster quota | admission | reads it; left to its own controller | Compute; stack; tower |
| 213 | task preemption | admission | reads it; left to its own controller | Compute; stack; tower |
| 214 | binpack pressure | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 215 | spread pressure | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 216 | gang schedule | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 217 | worker allocation | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 218 | maintenance evacuation | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 219 | oversubscription | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 220 | resource reclaim | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 221 | queue fairness | admission | reads it; left to its own controller | Compute; stack; tower |
| 222 | deadline pressure | admission | reads it; left to its own controller | Compute; stack; tower |
| 223 | scheduler retry | admission | reads it; left to its own controller | Compute; stack; tower |

### *GPU Fabric & RDMA (18 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `fabric` (a high-speed fabric (RDMA, DPU, SmartNIC)), reached through the fabric manager's API. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 145 | nvlink placement | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 146 | nvswitch route | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 147 | gpu fabric quarantine | admission | reads it; left to its own controller | Compute; stack; tower |
| 148 | rdma bandwidth | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 149 | rdma route | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 150 | nic affinity | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 151 | collective concurrency | admission | reads it; left to its own controller | Compute; stack; tower |
| 152 | collective algorithm | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 153 | collective chunk size | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 154 | rank placement | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 155 | gpudirect policy | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 156 | congestion response | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 157 | rail selection | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 158 | fabric failover | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 159 | communication priority | admission | reads it; left to its own controller | Compute; stack; tower |
| 160 | fabric isolation | admission | reads it; left to its own controller | Compute; stack; tower |
| 1068 | collective protocol | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 1069 | nic hca selection | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |

### *HPC & Distributed Compute (19 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `batch` (batch and queued jobs), reached through the job queue (Kubernetes Jobs, Slurm, Spark). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 193 | job slots | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 194 | mpi ranks | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 195 | rank mapping | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 196 | node allocation | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 197 | job walltime | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 198 | job priority | admission | reads it; left to its own controller | Compute; stack; tower |
| 199 | job preemption | admission | reads it; left to its own controller | Compute; stack; tower |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 200 | checkpoint restart | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 201 | parallel io budget | admission | reads it; left to its own controller | Compute; stack; tower |
| 202 | collective budget | admission | reads it; left to its own controller | Compute; stack; tower |
| 203 | accelerator share | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 204 | cpu gpu ratio | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Compute; stack; tower |
| 205 | memory per rank | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 206 | scratch budget | admission | reads it; left to its own controller | Compute; stack; tower |
| 207 | scheduler fair share | admission | reads it; left to its own controller | Compute; stack; tower |
| 208 | backfill policy | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 1070 | node power saving | power | holds the power share inside its cover; full at once past the wall | Compute; stack; tower |
| 1071 | cpu frequency per job | power | holds the power share inside its cover; full at once past the wall | Compute; stack; tower |
| 1072 | partition limits | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |

### *Kubernetes Dynamic Device Allocation (8 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `gpu` (GPU serving with a response-time target), reached through nvidia-smi (clock ceiling and power limit) or the serving autoscaler. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| AUDIT-0001 | dra device class selection | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| AUDIT-0002 | dra claim capacity | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| AUDIT-0003 | dra claim sharing | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| AUDIT-0004 | dra device taint | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| AUDIT-0005 | dra device eviction | admission | reads it; left to its own controller | Compute; stack; tower |
| AUDIT-0006 | dra binding readiness | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| AUDIT-0007 | dra binding failure response | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| AUDIT-0008 | dra device configuration | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Compute; stack; tower |

### *OpenShift & Machine API (9 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `node` (a fleet of machines or VMs that boot in minutes), reached through the node pool's size command (Karpenter, Cluster Autoscaler, MachineSet, cloud ASG). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 85 | machine remediation | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 88 | machine health gate | admission | reads it; left to its own controller | Compute; stack; tower |
| 89 | mcp pause | admission | reads it; left to its own controller | Compute; stack; tower |
| 90 | mcp max unavailable | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 91 | node config rollout | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 92 | operator reconcile budget | admission | reads it; left to its own controller | Compute; stack; tower |
| 93 | cluster version pacing | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 94 | infra machine admission | admission | reads it; left to its own controller | Compute; stack; tower |
| 95 | machine failure domain | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |

### *Quantum Computing Control Simulation (16 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `qpu` (a quantum or specialised accelerator queue), reached through the accelerator's job queue. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 593 | qubit mapping | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 594 | circuit admission | admission | reads it; left to its own controller | Compute; stack; tower |
| 595 | shot allocation | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 596 | circuit scheduling | admission | reads it; left to its own controller | Compute; stack; tower |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 597 | gate scheduling | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 598 | pulse amplitude | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 599 | pulse duration | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Compute; stack; tower |
| 600 | pulse phase | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 601 | pulse frequency | power | holds the power share inside its cover; full at once past the wall | Compute; stack; tower |
| 602 | coupling control | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 603 | reset scheduling | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 604 | measurement scheduling | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 605 | dynamical decoupling | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 606 | noise aware routing | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Compute; stack; tower |
| 607 | error mitigation budget | admission | reads it; left to its own controller | Compute; stack; tower |
| 608 | quantum queue priority | admission | reads it; left to its own controller | Compute; stack; tower |

### *Work Admission & Demand Shaping (19 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 1 | api concurrency | admission | reads it; left to its own controller | Compute; stack; tower |
| 2 | queue concurrency | admission | reads it; left to its own controller | Compute; stack; tower |
| 3 | queue backpressure | admission | reads it; left to its own controller | Compute; stack; tower |
| 4 | job admission | admission | reads it; left to its own controller | Compute; stack; tower |
| 5 | batch admission | admission | reads it; left to its own controller | Compute; stack; tower |
| 6 | inference admission | admission | reads it; left to its own controller | Compute; stack; tower |
| 7 | load shed | admission | reads it; left to its own controller | Compute; stack; tower |
| 8 | priority gate | admission | reads it; left to its own controller | Compute; stack; tower |
| 9 | tenant admission | admission | reads it; left to its own controller | Compute; stack; tower |
| 10 | burst limit | admission | reads it; left to its own controller | Compute; stack; tower |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 11 | deadline admission | admission | reads it; left to its own controller | Compute; stack; tower |
| 12 | work budget | admission | reads it; left to its own controller | Compute; stack; tower |
| 13 | request queue limit | admission | reads it; left to its own controller | Compute; stack; tower |
| 14 | retry admission | admission | reads it; left to its own controller | Compute; stack; tower |
| 15 | background work gate | admission | reads it; left to its own controller | Compute; stack; tower |
| 16 | maintenance work gate | admission | reads it; left to its own controller | Compute; stack; tower |
| 1074 | request concurrency limit | admission | reads it; left to its own controller | Compute; stack; tower |
| 1123 | adaptive concurrency limit | admission | reads it; left to its own controller | Compute; stack; tower |
| 1124 | priority request queue under limit | admission | reads it; left to its own controller | Compute; stack; tower |

# Realm: Physics / Robotics / Autonomous (119 muscles of its own, 376 in its organism with the spine)

## *Automotive EV & Mobile Powertrain (14 muscles)*

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `ev_traction` (an electric vehicle traction motor), reached through the vehicle's motor controller (CAN; modelled only). Omni-Compass reads its tracking error and its load. Its home realm is Physics / Robotics / Autonomous.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 1083 | state of charge limits | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| 1084 | smart charging schedule | admission | reads it; left to its own controller | Physics; stack; tower |
| AUDIT-0045 | traction torque limit | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| AUDIT-0046 | regen braking level | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| AUDIT-0047 | battery charge limit | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| AUDIT-0048 | battery discharge limit | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| AUDIT-0049 | battery thermal target | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| AUDIT-0050 | motor thermal limit | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| AUDIT-0051 | vehicle speed envelope | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| AUDIT-0052 | energy recovery target | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| AUDIT-0053 | auxiliary power budget | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| AUDIT-0054 | fast charge current | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| AUDIT-0055 | fast charge voltage | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| AUDIT-0056 | vehicle safe state | admission | reads it; left to its own controller | Physics; stack; tower |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

### *Aviation & Autonomous Flight (20 muscles)*

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `flight_axis` (a flight-control axis), reached through the flight controller (modelled only). Omni-Compass reads its tracking error and its load. Its home realm is Physics / Robotics / Autonomous.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 481 | throttle envelope | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| 482 | attitude target | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 483 | attitude rate target | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 484 | velocity target | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 485 | altitude target | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 486 | waypoint authority | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 487 | flight hold | admission | reads it; left to its own controller | Physics; stack; tower |
| 488 | return to home | admission | reads it; left to its own controller | Physics; stack; tower |
| 489 | land action | admission | reads it; left to its own controller | Physics; stack; tower |
| 490 | mission admission | admission | reads it; left to its own controller | Physics; stack; tower |
| 491 | geofence response | admission | reads it; left to its own controller | Physics; stack; tower |
| 492 | failsafe selection | admission | reads it; left to its own controller | Physics; stack; tower |
| 493 | battery reserve threshold | admission | reads it; left to its own controller | Physics; stack; tower |
| 494 | actuator saturation envelope | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 495 | flight mode transition | admission | reads it; left to its own controller | Physics; stack; tower |
| 496 | flight termination safe state | admission | reads it; left to its own controller | Physics; stack; tower |
| 1079 | attitude rate gains | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 1080 | horizontal speed limit | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| 1081 | vertical speed limits | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| 1082 | acceleration jerk limits | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

### *Robotics Fleet & Warehouse Automation (15 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `robot_fleet` (a fleet of robots or vehicles taking tasks), reached through the fleet manager's dispatch API. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Physics / Robotics / Autonomous.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 449 | robot dispatch | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 450 | task assignment | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 451 | traffic reservation | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 452 | robot route | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 453 | charging dispatch | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 454 | battery reserve | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 455 | elevator request | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 456 | door request | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 457 | conveyor speed | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 458 | agv speed | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 459 | warehouse zone admission | admission | reads it; left to its own controller | Physics; stack; tower |
| 460 | robot quarantine | admission | reads it; left to its own controller | Physics; stack; tower |
| 461 | fleet failover | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 462 | human safe stop | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 463 | fleet concurrency | admission | reads it; left to its own controller | Physics; stack; tower |

### *Robotics Motion Control (18 muscles)*

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `robot_joint` (a robot joint servo), reached through the robot controller (ROS 2, EtherCAT, the drive's fieldbus). Omni-Compass reads its tracking error and its load. Its home realm is Physics / Robotics / Autonomous.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 433 | joint position | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 434 | joint velocity | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 435 | joint acceleration | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 436 | joint effort | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| 437 | cartesian velocity | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 438 | trajectory speed | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 439 | trajectory acceleration | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 440 | jerk limit | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 441 | collision margin | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 442 | force limit | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| 443 | gripper force | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| 444 | locomotion speed | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 445 | steering angle | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 446 | braking force | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| 447 | balance correction | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 1076 | trajectory tolerances | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 1077 | velocity acceleration scaling | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 1078 | controller update rate | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

### *Spacecraft & Flight Software (14 muscles)*

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `reaction_wheel` (a spacecraft reaction wheel), reached through the attitude controller (modelled only). Omni-Compass reads its tracking error and its load. Its home realm is Physics / Robotics / Autonomous.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 497 | space command admission | admission | reads it; left to its own controller | Physics; stack; tower |
| 499 | flight task schedule | admission | reads it; left to its own controller | Physics; stack; tower |
| 500 | space mode transition | admission | reads it; left to its own controller | Physics; stack; tower |
| 501 | payload duty cycle | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| 502 | communication allocation | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 503 | space power budget | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| 504 | space thermal command | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| 505 | attitude command envelope | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 506 | reaction wheel allocation | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 507 | rcs authority | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| 508 | safe mode transition | admission | reads it; left to its own controller | Physics; stack; tower |
| 509 | watchdog recovery | admission | reads it; left to its own controller | Physics; stack; tower |
| 510 | instrument activation | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 511 | fault isolation | admission | reads it; left to its own controller | Physics; stack; tower |

### *Rail Traction & Train Control (14 muscles)*

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `rail_traction` (a train's traction and auxiliary systems), reached through the train control and management system and ATO (modelled only). Omni-Compass reads its tracking error and its load. Its home realm is Physics / Robotics / Autonomous.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 2054 | traction effort limit | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| 2055 | regenerative braking share | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| 2056 | coasting speed target | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 2057 | dwell time schedule | admission | reads it; left to its own controller | Physics; stack; tower |
| 2058 | hvac duty cycle | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| 2059 | train auxiliary power budget | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 2060 | platform approach speed | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 2061 | acceleration rate setpoint | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 2062 | wheel slip protection mode | admission | reads it; left to its own controller | Physics; stack; tower |
| 2063 | traction motor thermal derate | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| 2064 | catenary voltage limit | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| 2065 | timetable recovery margin | admission | reads it; left to its own controller | Physics; stack; tower |
| 2066 | headway target | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 2067 | door release hold | admission | reads it; left to its own controller | Physics; stack; tower |

### *Marine Propulsion & Vessel Automation (12 muscles)*

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `marine_propulsion` (a ship's propulsion shaft and power plant), reached through the vessel automation and power management system (modelled only). Omni-Compass reads its tracking error and its load. Its home realm is Physics / Robotics / Autonomous.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 2068 | shaft speed target | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 2069 | propeller pitch limit | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| 2070 | shaft power limit | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| 2071 | bow thruster duty | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| 2072 | ballast pump mode | admission | reads it; left to its own controller | Physics; stack; tower |
| 2073 | engine load sharing setpoint | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 2074 | slow steaming speed target | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 2075 | auxiliary engine staging | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 2076 | shore power mode | admission | reads it; left to its own controller | Physics; stack; tower |
| 2077 | hotel load budget | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| 2078 | trim target | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 2079 | rudder rate limit | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |

### *Elevators & Vertical Transport (12 muscles)*

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `elevator_hoist` (an elevator or escalator drive and its group control), reached through the lift controller and destination dispatch (modelled only). Omni-Compass reads its tracking error and its load. Its home realm is Physics / Robotics / Autonomous.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 2146 | hoist speed target | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 2147 | acceleration limit | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 2148 | regen drive mode | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| 2149 | standby power mode | admission | reads it; left to its own controller | Physics; stack; tower |
| 2150 | destination dispatch schedule | admission | reads it; left to its own controller | Physics; stack; tower |
| 2151 | car parking mode | admission | reads it; left to its own controller | Physics; stack; tower |
| 2152 | door dwell hold | admission | reads it; left to its own controller | Physics; stack; tower |
| 2153 | escalator speed target | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Physics; stack; tower |
| 2154 | group capacity mode | admission | reads it; left to its own controller | Physics; stack; tower |
| 2155 | motor thermal derate | power | holds the power share inside its cover; full at once past the wall | Physics; stack; tower |
| 2156 | brake test schedule | admission | reads it; left to its own controller | Physics; stack; tower |
| 2157 | fire recall mode | admission | reads it; left to its own controller | Physics; stack; tower |

## Realm: Energy / Facility / Industrial (213 muscles of its own, 470 in its organism with the spine)

### *Building & Critical Environment HVAC (15 muscles)*

Each is a data hall or building zone cooled by chillers, air handlers or fans; its class is `building` (a building zone), reached through the building management system. Omni-Compass reads its temperature against its limit. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 1094 | setpoint reset trim and respond | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 1095 | duct static pressure setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 1096 | optimal start stop | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| AUDIT-0081 | zone temperature target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| AUDIT-0082 | zone airflow | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| AUDIT-0083 | ahu fan speed | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| AUDIT-0084 | damper position | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| AUDIT-0085 | economizer position | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| AUDIT-0086 | boiler setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| AUDIT-0087 | heat pump mode | admission | reads it; left to its own controller | Energy; stack; tower |
| AUDIT-0088 | humidity target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| AUDIT-0089 | occupancy ventilation | admission | reads it; left to its own controller | Energy; stack; tower |
| AUDIT-0090 | building demand limit | power | holds the power share inside its cover; full at once past the wall | Energy; stack; tower |
| AUDIT-0091 | thermal storage dispatch | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| AUDIT-0092 | hvac emergency mode | admission | reads it; left to its own controller | Energy; stack; tower |

### *Energy Storage & Microgrid (19 muscles)*

Each is a battery, UPS or microgrid store with a load and a grid connection; its class is `microgrid` (a microgrid battery and loads), reached through the inverter or energy management system (IEEE 2030.5, SunSpec, OpenADR). Omni-Compass reads its draw on the grid connection and its reserve. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 513 | battery charge power | power | holds the power share inside its cover; full at once past the wall | Energy; stack; tower |
| 514 | battery discharge power | power | holds the power share inside its cover; full at once past the wall | Energy; stack; tower |
| 515 | battery soc reserve | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 516 | grid import limit | power | holds the power share inside its cover; full at once past the wall | Energy; stack; tower |
| 517 | grid export limit | power | holds the power share inside its cover; full at once past the wall | Energy; stack; tower |
| 518 | pv curtailment | power | holds the power share inside its cover; full at once past the wall | Energy; stack; tower |
| 519 | ev charge power | power | holds the power share inside its cover; full at once past the wall | Energy; stack; tower |
| 520 | heat pump power | power | holds the power share inside its cover; full at once past the wall | Energy; stack; tower |
| 521 | electrolyzer power | power | holds the power share inside its cover; full at once past the wall | Energy; stack; tower |
| 522 | microgrid demand limit | power | holds the power share inside its cover; full at once past the wall | Energy; stack; tower |
| 523 | peak shaving | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 524 | time of use schedule | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 525 | energy load shed | admission | reads it; left to its own controller | Energy; stack; tower |
| 526 | flex load admission | admission | reads it; left to its own controller | Energy; stack; tower |
| 527 | storage dispatch | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 528 | microgrid emergency reserve | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 1085 | volt var | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 1086 | volt watt | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 1087 | constant power factor reactive power | power | holds the power share inside its cover; full at once past the wall | Energy; stack; tower |

### *Facility & Grid Optimization (15 muscles)*

Each is a battery, UPS or microgrid store with a load and a grid connection; its class is `facility` (a facility battery behind the meter), reached through the energy management system. Omni-Compass reads its draw on the grid connection and its reserve. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 561 | facility power budget | power | holds the power share inside its cover; full at once past the wall | Energy; stack; tower |
| 562 | utility demand limit | power | holds the power share inside its cover; full at once past the wall | Energy; stack; tower |
| 563 | demand response | admission | reads it; left to its own controller | Energy; stack; tower |
| 564 | electricity price gate | admission | reads it; left to its own controller | Energy; stack; tower |
| 565 | carbon intensity gate | admission | reads it; left to its own controller | Energy; stack; tower |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 566 | renewable dispatch | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 567 | generator start stop | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 568 | site battery dispatch | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 569 | pue target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 570 | cooling power budget | power | holds the power share inside its cover; full at once past the wall | Energy; stack; tower |
| 571 | it power budget | power | holds the power share inside its cover; full at once past the wall | Energy; stack; tower |
| 572 | rack power allocation | power | holds the power share inside its cover; full at once past the wall | Energy; stack; tower |
| 573 | facility peak guard | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 574 | grid frequency response | admission | reads it; left to its own controller | Energy; stack; tower |
| 575 | facility islanding | admission | reads it; left to its own controller | Energy; stack; tower |

### *Grid Transmission & Distribution (12 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `feeder_voltage` (a distribution feeder's voltage), reached through the distribution management system (IEC 61850, DNP3; modelled only). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| AUDIT-0033 | capacitor bank switch | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| AUDIT-0034 | voltage regulator tap | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| AUDIT-0035 | transformer tap | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| AUDIT-0036 | inverter real power | power | holds the power share inside its cover; full at once past the wall | Energy; stack; tower |
| AUDIT-0037 | inverter reactive power | power | holds the power share inside its cover; full at once past the wall | Energy; stack; tower |
| AUDIT-0038 | feeder voltage target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| AUDIT-0039 | feeder load transfer | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| AUDIT-0040 | distribution storage dispatch | admission | reads it; left to its own controller | Energy; stack; tower |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| AUDIT-0041 | demand response dispatch | admission | reads it; left to its own controller | Energy; stack; tower |
| AUDIT-0042 | frequency droop setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| AUDIT-0043 | grid protection mode | admission | reads it; left to its own controller | Energy; stack; tower |
| AUDIT-0044 | grid restoration sequence | admission | reads it; left to its own controller | Energy; stack; tower |

### *Industrial PLC & Process Automation (18 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `process` (an industrial process loop), reached through the PLC or DCS (OPC UA, EtherNet/IP, PROFINET). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 465 | plc cycle authority | admission | reads it; left to its own controller | Energy; stack; tower |
| 466 | machine cell admission | admission | reads it; left to its own controller | Energy; stack; tower |
| 467 | valve position | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 468 | pump flow | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 469 | compressor speed | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 470 | heater power | power | holds the power share inside its cover; full at once past the wall | Energy; stack; tower |
| 471 | furnace setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 472 | pressure setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 473 | temperature setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 474 | mass flow setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 475 | tank level target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 476 | conveyor rate | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 477 | feed rate | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 478 | purge vent action | admission | reads it; left to its own controller | Energy; stack; tower |
| 1088 | controller mode | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 1089 | alarm limits | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 1090 | safety interlock trip | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 1091 | opc ua writes | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |

### *Semiconductor Fab & Precision Manufacturing (13 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `chamber` (a controlled chamber (clean room, kiln, reactor)), reached through the PLC or DCS. Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 1092 | run to run control | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 1093 | idle sleep mode | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| AUDIT-0057 | tool job dispatch | admission | reads it; left to its own controller | Energy; stack; tower |
| AUDIT-0058 | wafer route | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| AUDIT-0059 | chamber recipe selection | admission | reads it; left to its own controller | Energy; stack; tower |
| AUDIT-0060 | chamber temperature | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| AUDIT-0061 | chamber pressure | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| AUDIT-0062 | gas flow | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| AUDIT-0063 | rf power | power | holds the power share inside its cover; full at once past the wall | Energy; stack; tower |
| AUDIT-0064 | vacuum pump speed | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| AUDIT-0065 | robot transfer rate | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| AUDIT-0066 | lot priority | admission | reads it; left to its own controller | Energy; stack; tower |
| AUDIT-0068 | tool quarantine | admission | reads it; left to its own controller | Energy; stack; tower |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

### *Water Wastewater & Pumping (12 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `water` (a water or pumping process), reached through the SCADA system (Modbus, DNP3). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| AUDIT-0069 | pump speed water | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| AUDIT-0070 | valve position water | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| AUDIT-0071 | reservoir level target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| AUDIT-0072 | line pressure target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| AUDIT-0073 | flow target water | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| AUDIT-0074 | aeration rate | power | holds the power share inside its cover; full at once past the wall | Energy; stack; tower |
| AUDIT-0075 | chemical dose rate | power | holds the power share inside its cover; full at once past the wall | Energy; stack; tower |
| AUDIT-0076 | filtration backwash | admission | reads it; left to its own controller | Energy; stack; tower |
| AUDIT-0077 | lift station dispatch | admission | reads it; left to its own controller | Energy; stack; tower |
| AUDIT-0078 | leak isolation | admission | reads it; left to its own controller | Energy; stack; tower |
| AUDIT-0079 | water demand shed | admission | reads it; left to its own controller | Energy; stack; tower |
| AUDIT-0080 | water emergency shutdown | admission | reads it; left to its own controller | Energy; stack; tower |

### *Healthcare Critical Environments (13 muscles)*

Each is a data hall or building zone cooled by chillers, air handlers or fans; its class is `hospital` (a hospital's critical rooms (operating rooms, isolation, pharmacy, imaging suites)), reached through the hospital's building management system (BACnet). Omni-Compass reads its temperature against its limit. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 2001 | operating room air change setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2002 | isolation room pressure target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2003 | patient room temperature setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 2004 | surgical suite humidity target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2005 | ahu supply air temperature | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2006 | pharmacy cold room setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2007 | sterile storage humidity setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2008 | imaging suite cooling capacity | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 2009 | chiller plant staging | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 2010 | exhaust fan capacity | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 2011 | ward night setback mode | admissio n | reads it; left to its own controller | Energy; stack; tower |
| 2012 | medical gas plant demand limit | power | holds the power share inside its cover; full at once past the wall | Energy; stack; tower |
| 2013 | emergency power load shed | admissio n | reads it; left to its own controller | Energy; stack; tower |

### *Agriculture & Irrigation (14 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `irrigation` (pumps, pressure and climate on a farm or in a greenhouse), reached through the pump station controller, the pivot panel or the climate computer (Modbus, the vendor cloud). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 2026 | irrigation pump speed | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 2027 | mainline pressure setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2028 | soil moisture target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2029 | pivot speed setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2030 | fertigation dose | power | holds the power share inside its cover; full at once past the wall | Energy; stack; tower |
| 2031 | greenhouse temperature setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 2032 | greenhouse co2 target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2033 | vent position setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2034 | grain dryer temperature setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2035 | cold storage temperature setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2036 | barn ventilation rate | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 2037 | milking vacuum level setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2038 | well drawdown level target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2039 | drip zone dispatch | admissio n | reads it; left to its own controller | Energy; stack; tower |

### *Oil & Gas Pipelines (14 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `pipeline` (a pipeline segment with its compressors, pumps and valves), reached through pipeline SCADA (DNP3, Modbus, OPC UA). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 2040 | compressor discharge pressure setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2041 | pump station suction pressure target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2042 | line pack target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2043 | pipeline flow setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2044 | compressor unit staging | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 2045 | vfd pump speed | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 2046 | terminal tank level target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 2047 | leak detection shutdown | admissio n | reads it; left to its own controller | Energy; stack; tower |
| 2048 | batch interface dispatch | admissio n | reads it; left to its own controller | Energy; stack; tower |
| 2049 | heater outlet temperature setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2050 | drag reducing agent dose | power | holds the power share inside its cover; full at once past the wall | Energy; stack; tower |
| 2051 | valve position target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2052 | cathodic protection voltage | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2053 | pressure protection priority | admissio n | reads it; left to its own controller | Energy; stack; tower |

### *Mining & Mineral Processing (14 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `mill` (a grinding, flotation or materials-handling circuit), reached through the plant DCS or PLC (OPC UA). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 2092 | sag mill load setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2093 | mill speed target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2094 | crusher gap setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2095 | flotation aeration rate | power | holds the power share inside its cover; full at once past the wall | Energy; stack; tower |
| 2096 | cyclone feed pressure target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2097 | thickener underflow density target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2098 | reagent dose | power | holds the power share inside its cover; full at once past the wall | Energy; stack; tower |
| 2099 | conveyor speed setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2100 | dewatering pump level target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2101 | ventilation on demand airflow | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 2102 | stockpile level target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2103 | slurry pump speed | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 2104 | tailings discharge shutdown | admissio n | reads it; left to its own controller | Energy; stack; tower |
| 2105 | ore blend dispatch | admissio n | reads it; left to its own controller | Energy; stack; tower |

### *District Heating & Cooling (13 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `district_heat` (a district heating or cooling network), reached through the network control system and substation controllers (Modbus, M-Bus). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 2106 | supply temperature setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2107 | return temperature target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2108 | differential pressure setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2109 | network pump speed | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 2110 | heat pump staging | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 2111 | chp dispatch | admissio n | reads it; left to its own controller | Energy; stack; tower |
| 2112 | thermal storage level target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2113 | peak boiler heater output | power | holds the power share inside its cover; full at once past the wall | Energy; stack; tower |
| 2114 | substation flow limit | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 2115 | peak demand shed | admissio n | reads it; left to its own controller | Energy; stack; tower |
| 2116 | outdoor reset setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2117 | cooling network supply temperature | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 2118 | cooling storage level target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |

### *Power Generation & Turbine Control (14 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `turbine` (a generating unit's governor, boiler or excitation loop), reached through the plant DCS and governor (OPC UA, IEC 61850; modelled only). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 2119 | turbine speed droop setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2120 | unit load setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2121 | boiler steam pressure setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2122 | feedwater level target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2123 | agc participation limit | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 2124 | excitation voltage setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2125 | hydro gate position target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2126 | combustion air ratio setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2127 | cooling water flow setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2128 | inlet guide vane position | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2129 | reserve dispatch priority | admission | reads it; left to its own controller | Energy; stack; tower |
| 2130 | turbine ramp rate limit | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 2131 | emissions shutdown | admission | reads it; left to its own controller | Energy; stack; tower |
| 2132 | nuclear rod position target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

### *Renewable Generation & Inverter Control (13 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `inverter` (a plant of grid-support inverters or wind turbines), reached through the plant controller and inverter settings (SunSpec, IEC 61850, IEEE 1547). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 2133 | inverter volt var setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2134 | inverter volt watt setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2135 | active power curtailment | power | holds the power share inside its cover; full at once past the wall | Energy; stack; tower |
| 2136 | plant reactive power target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2137 | wind turbine yaw offset | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 2138 | pitch angle limit | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 2139 | rotor speed setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2140 | inverter frequency droop setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2141 | ramp rate limit | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Energy; stack; tower |
| 2142 | tracker stow mode | admission | reads it; left to its own controller | Energy; stack; tower |
| 2143 | string mppt voltage setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2144 | noise mode schedule | admission | reads it; left to its own controller | Energy; stack; tower |
| 2145 | ice detection shutdown | admission | reads it; left to its own controller | Energy; stack; tower |

### *Pharmaceutical & Food Manufacturing (14 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `batch_reactor` (a GMP batch reactor, fermenter or food process), reached through the batch control system (ISA-88, OPC UA). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 2158 | reactor temperature setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2159 | agitator speed setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2160 | fermenter dissolved oxygen target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2161 | ph setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2162 | pasteurizer holding temperature setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2163 | freezer tunnel temperature setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2164 | cip cycle dispatch | admissio n | reads it; left to its own controller | Energy; stack; tower |
| 2165 | cleanroom pressure cascade setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2166 | lyophilizer shelf temperature setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2167 | chromatography flow setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2168 | steam sterilizer cycle priority | admissio n | reads it; left to its own controller | Energy; stack; tower |
| 2169 | oven zone temperature setpoint | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Energy; stack; tower |
| 2170 | refrigerant compressor authority | admissio n | reads it; left to its own controller | Energy; stack; tower |
| 2171 | batch hold quarantine | admissio n | reads it; left to its own controller | Energy; stack; tower |

# Realm: Distribution / Specialized (183 muscles of its own, 440 in its organism with the spine)

## *Cache & Memory Services (16 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 305 | cache size | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 306 | cache ttl | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Distribution; stack; tower |
| 307 | cache eviction policy | admission | reads it; left to its own controller | Distribution; stack; tower |
| 308 | cache replicas | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 309 | cache sharding | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 310 | cache prefetch | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 311 | cache writeback rate | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 312 | cache admission | admission | reads it; left to its own controller | Distribution; stack; tower |
| 313 | hot key isolation | admission | reads it; left to its own controller | Distribution; stack; tower |
| 314 | cache connection limit | admission | reads it; left to its own controller | Distribution; stack; tower |
| 315 | cache memory limit | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 316 | cache compression | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 317 | cache warmup | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 318 | cache failover | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 319 | cache flush rate | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 1106 | memory size threads | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |

### *Commerce & Payment Systems (15 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `commerce` (a customer-facing service with bursts), reached through the Kubernetes API. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 401 | payment admission | admission | reads it; left to its own controller | Distribution; stack; tower |
| 402 | payment concurrency | admission | reads it; left to its own controller | Distribution; stack; tower |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 403 | payment retry | admission | reads it; left to its own controller | Distribution; stack; tower |
| 404 | payment timeout | admission | reads it; left to its own controller | Distribution; stack; tower |
| 405 | fraud review gate | admission | reads it; left to its own controller | Distribution; stack; tower |
| 406 | authorization route | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 407 | processor route weight | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Distribution; stack; tower |
| 408 | transaction queue limit | admission | reads it; left to its own controller | Distribution; stack; tower |
| 409 | idempotency window | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Distribution; stack; tower |
| 410 | order reservation | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 411 | inventory hold | admission | reads it; left to its own controller | Distribution; stack; tower |
| 412 | checkout load shed | admission | reads it; left to its own controller | Distribution; stack; tower |
| 413 | refund queue rate | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 414 | settlement batch | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 415 | payment failover | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |

### *Data Analytics & ETL (15 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `batch` (batch and queued jobs), reached through the job queue (Kubernetes Jobs, Slurm, Spark). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 338 | executor size | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 339 | dynamic allocation min | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 340 | dynamic allocation max | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 341 | shuffle partitions | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 342 | shuffle bandwidth | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 343 | etl concurrency | admission | reads it; left to its own controller | Distribution; stack; tower |
| 344 | stage parallelism | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 345 | query slots | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 346 | spill threshold | admission | reads it; left to its own controller | Distribution; stack; tower |
| 347 | cache fraction | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Distribution; stack; tower |
| 348 | batch interval | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Distribution; stack; tower |
| 349 | stream backpressure | admission | reads it; left to its own controller | Distribution; stack; tower |
| 350 | data locality wait | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 351 | speculation policy | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 352 | analytics admission | admission | reads it; left to its own controller | Distribution; stack; tower |

### *Database & Transactions (19 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `database` (a database cluster), reached through the database operator (replicas, connection pools). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 273 | db replicas | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 275 | db memory | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 276 | db cache | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 277 | query concurrency | admission | reads it; left to its own controller | Distribution; stack; tower |
| 278 | read route | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 279 | shard placement | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 280 | db failover | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 281 | replication lag gate | admission | reads it; left to its own controller | Distribution; stack; tower |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 282 | db write throttle | admission | reads it; left to its own controller | Distribution; stack; tower |
| 283 | db pool resize | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 284 | transaction concurrency | admission | reads it; left to its own controller | Distribution; stack; tower |
| 285 | lock timeout | admission | reads it; left to its own controller | Distribution; stack; tower |
| 286 | checkpoint rate | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 287 | vacuum compaction rate | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 1099 | parallel workers | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 1100 | background writer | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 1101 | synchronous replication | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 1102 | buffer pool | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 1103 | io capacity | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |

### *Messaging & Streaming (18 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 289 | partition count | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 290 | partition placement | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 291 | producer quota | admission | reads it; left to its own controller | Distribution; stack; tower |
| 292 | consumer quota | admission | reads it; left to its own controller | Distribution; stack; tower |
| 293 | broker io quota | admission | reads it; left to its own controller | Distribution; stack; tower |
| 294 | message retention | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 295 | queue depth limit | admission | reads it; left to its own controller | Distribution; stack; tower |
| 296 | consumer concurrency | admission | reads it; left to its own controller | Distribution; stack; tower |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 297 | producer batch size | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 298 | fetch batch size | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 299 | rebalance rate | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 300 | replication factor | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 301 | retry backoff | admission | reads it; left to its own controller | Distribution; stack; tower |
| 302 | dead letter divert | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 303 | stream priority | admission | reads it; left to its own controller | Distribution; stack; tower |
| 304 | broker failover | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 1104 | prefetch | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 1105 | memory disk alarm | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |

### *Runtime & Application (17 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 353 | worker count | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 354 | thread pool | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 355 | jvm heap | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 356 | gc budget | admission | reads it; left to its own controller | Distribution; stack; tower |
| 357 | connection pool runtime | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 358 | application cache size | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 359 | runtime memory | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 360 | async concurrency | admission | reads it; left to its own controller | Distribution; stack; tower |
| 361 | event loop workers | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 362 | process count | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 363 | request timeout | admission | reads it; left to its own controller | Distribution; stack; tower |
| 364 | background workers | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 365 | runtime cpu budget | admission | reads it; left to its own controller | Distribution; stack; tower |
| 366 | runtime io budget | admission | reads it; left to its own controller | Distribution; stack; tower |
| 367 | runtime restart | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 1108 | go runtime | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 1109 | keepalive connection reuse | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |

### *Search, Indexing & Vector DB (16 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 321 | index workers | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 322 | index refresh rate | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 323 | segment merge rate | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 324 | search concurrency | admission | reads it; left to its own controller | Distribution; stack; tower |
| 325 | search timeout | admission | reads it; left to its own controller | Distribution; stack; tower |
| 326 | shard count | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 327 | shard replication | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 328 | shard rebalance | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 329 | vector search k | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 330 | vector batch size | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 331 | embedding workers | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 332 | index memory budget | admission | reads it; left to its own controller | Distribution; stack; tower |
| 333 | query route | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 334 | hot shard isolation | admission | reads it; left to its own controller | Distribution; stack; tower |
| 335 | search admission | admission | reads it; left to its own controller | Distribution; stack; tower |
| 1107 | circuit breakers | admission | reads it; left to its own controller | Distribution; stack; tower |

### *Service Mesh & API Reliability (16 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 241 | circuit breaker | admission | reads it; left to its own controller | Distribution; stack; tower |
| 242 | retry budget | admission | reads it; left to its own controller | Distribution; stack; tower |
| 243 | service timeout | admission | reads it; left to its own controller | Distribution; stack; tower |
| 244 | service concurrency | admission | reads it; left to its own controller | Distribution; stack; tower |
| 245 | connection pool | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 246 | traffic divert | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 247 | traffic mirror | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 248 | canary weight | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Distribution; stack; tower |
| 249 | outlier ejection | admission | reads it; left to its own controller | Distribution; stack; tower |
| 250 | health threshold | admission | reads it; left to its own controller | Distribution; stack; tower |
| 251 | dns traffic weight | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Distribution; stack; tower |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 252 | session affinity | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 253 | request hedging | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 254 | fault injection gate | admission | reads it; left to its own controller | Distribution; stack; tower |
| 255 | service failover | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 1097 | load balancing policy | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |

### *Telecom RAN & Edge Radio (12 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `ran` (a radio access network cell or site), reached through the RAN controller (O-RAN interfaces). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| AUDIT-0009 | ran connection admission | admission | reads it; left to its own controller | Distribution; stack; tower |
| AUDIT-0010 | ran ue handover | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| AUDIT-0011 | ran cell traffic steering | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| AUDIT-0012 | ran slice resource budget | admission | reads it; left to its own controller | Distribution; stack; tower |
| AUDIT-0013 | ran prb allocation | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| AUDIT-0014 | ran scheduler weight | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Distribution; stack; tower |
| AUDIT-0015 | ran tx power | power | holds the power share inside its cover; full at once past the wall | Distribution; stack; tower |
| AUDIT-0016 | ran antenna tilt | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| AUDIT-0017 | ran carrier enable | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| AUDIT-0018 | ran cell sleep | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| AUDIT-0019 | ran du cu placement | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| AUDIT-0020 | ran fronthaul budget | admission | reads it; left to its own controller | Distribution; stack; tower |

### *Workflow, Logistics & Fulfillment (15 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `workflow` (a workflow or pipeline engine), reached through the workflow engine's concurrency settings. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 417 | workflow admission | admission | reads it; left to its own controller | Distribution; stack; tower |
| 418 | workflow worker rate | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 419 | task queue rate | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 420 | workflow retry | admission | reads it; left to its own controller | Distribution; stack; tower |
| 421 | workflow backoff | admission | reads it; left to its own controller | Distribution; stack; tower |
| 422 | workflow timeout | admission | reads it; left to its own controller | Distribution; stack; tower |
| 423 | inventory allocation | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 424 | fulfillment route | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 425 | warehouse queue | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 426 | carrier selection | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 427 | dispatch priority | admission | reads it; left to its own controller | Distribution; stack; tower |
| 428 | shipment batch | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 429 | route replan | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 430 | sla escalation | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 431 | compensation action | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

### *Medical Imaging & Clinical Systems (12 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `clinical` (clinical systems: imaging archives, records, interface engines, monitoring gateways), reached through the application's own scaling and admission settings (Kubernetes, the vendor console). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 2014 | pacs archive tier target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Distribution; stack; tower |
| 2015 | dicom router concurrency | admission | reads it; left to its own controller | Distribution; stack; tower |
| 2016 | ehr application replicas | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 2017 | hl7 interface queue limit | admission | reads it; left to its own controller | Distribution; stack; tower |
| 2018 | fhir api rate limit | admission | reads it; left to its own controller | Distribution; stack; tower |
| 2019 | reconstruction gpu workers | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 2020 | modality worklist timeout | admission | reads it; left to its own controller | Distribution; stack; tower |
| 2021 | patient monitoring gateway capacity | admission | reads it; left to its own controller | Distribution; stack; tower |
| 2022 | lab analyzer batch window | admission | reads it; left to its own controller | Distribution; stack; tower |
| 2023 | telehealth session admission | admission | reads it; left to its own controller | Distribution; stack; tower |
| 2024 | clinical backup window | admission | reads it; left to its own controller | Distribution; stack; tower |
| 2025 | imaging prefetch priority | admission | reads it; left to its own controller | Distribution; stack; tower |

### *Ports & Maritime Logistics (12 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `port` (a container terminal's cranes and vehicles taking moves), reached through the terminal operating system and fleet controllers. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 2080 | quay crane allocation | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 2081 | yard crane fleet size | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 2082 | berth window admission | admission | reads it; left to its own controller | Distribution; stack; tower |



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms |
|---|---|---|---|---|
| 2083 | truck gate rate limit | admission | reads it; left to its own controller | Distribution; stack; tower |
| 2084 | horizontal transport fleet size | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 2085 | reefer plug power budget | power | holds the power share inside its cover; full at once past the wall | Distribution; stack; tower |
| 2086 | shore power connection capacity | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 2087 | rail mounted gantry dispatch | capacity | holds it so the service sits in the middle of its band; more at once past the wall; gives back one unit at a time through the release gate | Distribution; stack; tower |
| 2088 | container dwell priority | admission | reads it; left to its own controller | Distribution; stack; tower |
| 2089 | vessel arrival pacing | admission | reads it; left to its own controller | Distribution; stack; tower |
| 2090 | stacking height target | setpoint | moves the setpoint inside its safe band toward the calm end while the service has room; back toward stress at once when it does not | Distribution; stack; tower |
| 2091 | equipment charging window | admission | reads it; left to its own controller | Distribution; stack; tower |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

APPENDIX

# Appendix H. Source of the Engine

The engine, the compass law, the compass on every muscle, the two-wire GPU governor, the plug adapter and the nervous system, printed in full from the repository so that the book and the code can be read side by side. The engine file is sealed: its fingerprint is in `results/SEAL.json`, and `verify.py` fails if a byte of it changes.

`omnicompass/core.py`

*240 lines.*

```
"""Omni-Compass core: six-state ODE, RK4 integrator, bounded feedforward-plus-proportional controller.

State x = (E, U, I_U, S, B, B_dot).

  (1) dE/dt    = -alpha_E E + beta_int + beta_ext + v_eff
  (2) dU/dt    = mu U (1 - U^2) - (dE/dt)/E_max - lambda_U U + u,   |u| &lt;= U_AUTHORITY
  (3) dI_U/dt  = (1 - U) - sigma_1 E - delta S - lambda_I I_U
  (4) v_eff    = cos(omega_B t / 2) * c * tanh(lambda_0 + lambda_1 (U - 0.5) + lambda_2 S)
  (5) Phi(S)   = alpha_s S^2 / 2 + beta_s S^3 / 4 - delta S
  (6) dS/dt    = -dPhi/dS = delta - alpha_s S - (3/4) beta_s S^2
  (7) dB/dt    = B_dot ;  dB_dot/dt = gamma_c delta S - (omega_B/Q_B) B_dot - omega_B^2 B
  (8) R_B[n]   = finite-difference audit of (7); never fed back into the state.

Controller (one micro step, zero-order hold across all four RK4 stages):
  u = clip(-f_U(x, t) + KP (sigma - U), -U_AUTHORITY, +U_AUTHORITY)
where f_U is the uncontrolled U drift and sigma in {-1, +1} is the locked target.
"""
from __future__ import annotations

import math
from dataclasses import dataclass, asdict, fields
from typing import Dict, List, Optional, Tuple

EPS = 1e-12
KP = 12.0
U_AUTHORITY = 25.0
MACRO_STEPS = 20
MICRO_STEPS = 10
```

```
MACRO_DT = 0.1
MICRO_DT = MACRO_DT / MICRO_STEPS
BASIN_TOL = 0.10
CONVEY_WINDOW = 5
CERT_WINDOW = 10

PARAMETER_RANGES: Dict[str, Tuple[float, float]] = {
    "alpha": (4.2, 4.2), "beta_int": (0.0, 1.0), "beta_ext": (0.0, 1.0), "k": (1.7, 1.7),
    "sigma_1": (0.38, 0.38), "delta": (0.01, 1.0), "gamma_1": (1.35, 1.35), "gamma_c": (1.0, 1.0),
    "lambda_0": (-2.0, 2.0), "lambda_1": (0.1, 3.0), "lambda_2": (0.0, 2.0), "c": (0.5, 5.0),
    "E_max": (1.0, 10.0), "omega_B": (0.1, 5.0), "Q_B": (0.5, 10.0), "alpha_s": (0.05, 0.25),
    "beta_s": (0.05, 0.25), "mu": (2.0, 6.0), "alpha_E": (4.2, 4.2), "alpha_U": (4.2, 4.2),
    "lambda_I": (0.5, 2.0), "lambda_U": (0.0, 0.5),
}
# alpha, k, gamma_1, alpha_U do not enter equations (1)-(8).


@dataclass
class State:
    E: float
    U: float
    I_U: float
    S: float
    B: float
    B_dot: float

    def as_tuple(self) -&gt; Tuple[float, ...]:
        return (self.E, self.U, self.I_U, self.S, self.B, self.B_dot)

@dataclass
class Params:
    alpha: float = 4.2
    beta_int: float = 0.0
    beta_ext: float = 0.0
    k: float = 1.7
    sigma_1: float = 0.38
    delta: float = 0.5
    gamma_1: float = 1.35
    gamma_c: float = 1.0
    lambda_0: float = 0.0
    lambda_1: float = 1.0
    lambda_2: float = 0.5
    c: float = 1.0
    E_max: float = 2.0
    omega_B: float = 1.0
    Q_B: float = 2.5
    alpha_s: float = 0.12
    beta_s: float = 0.10
    mu: float = 4.0
    alpha_E: float = 4.2
    alpha_U: float = 4.2
    lambda_I: float = 1.0
    lambda_U: float = 0.1

# ---------------------------------------------------------------- equations
def v_eff(x: State, p: Params, t: float = 0.0) -&gt; float:
    z = p.lambda_0 + p.lambda_1 * (x.U - 0.5) + p.lambda_2 * x.S
    return math.cos(0.5 * p.omega_B * t) * (p.c * math.tanh(z))


def phi(S: float, p: Params) -&gt; float:
    return 0.5 * p.alpha_s * S * S + 0.25 * p.beta_s * S ** 3 - p.delta * S


def s_flow(S: float, p: Params) -&gt; float:
    return p.delta - p.alpha_s * S - 0.75 * p.beta_s * S * S


def derivatives(x: State, p: Params, t: float, u: float = 0.0) -&gt; State:
    v = v_eff(x, p, t)
    dE = -p.alpha_E * x.E + p.beta_int + p.beta_ext + v
    dU = p.mu * x.U * (1.0 - x.U * x.U) - dE / max(p.E_max, 1e-6) - p.lambda_U * x.U + u
    dI = (1.0 - x.U) - p.sigma_1 * x.E - p.delta * x.S - p.lambda_I * x.I_U
    dS = s_flow(x.S, p)
    dBd = p.gamma_c * p.delta * x.S - (p.omega_B / max(p.Q_B, EPS)) * x.B_dot - p.omega_B ** 2 * x.B
    return State(dE, dU, dI, dS, x.B_dot, dBd)


def _axpy(x: State, k: State, h: float) -&gt; State:
    return State(x.E + h * k.E, x.U + h * k.U, x.I_U + h * k.I_U, x.S + h * k.S, x.B + h * k.B, x.B_dot + h * k.B_dot)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

```
def rk4_step(x: State, p: Params, t: float, h: float, u: float = 0.0) -&gt; State:
    k1 = derivatives(x, p, t, u)
    k2 = derivatives(_axpy(x, k1, 0.5 * h), p, t + 0.5 * h, u)
    k3 = derivatives(_axpy(x, k2, 0.5 * h), p, t + 0.5 * h, u)
    k4 = derivatives(_axpy(x, k3, h), p, t + h, u)
    c = h / 6.0
    return State(*(a + c * (b1 + 2 * b2 + 2 * b3 + b4) for a, b1, b2, b3, b4 in
                   zip(x.as_tuple(), k1.as_tuple(), k2.as_tuple(), k3.as_tuple(), k4.as_tuple())))


def bath_residual(prev: State, cur: State, nxt: State, p: Params, h: float) -&gt; float:
    d2 = (nxt.B - 2.0 * cur.B + prev.B) / (h * h)
    dm = (cur.B - prev.B) / h
    return d2 + (p.omega_B / max(p.Q_B, EPS)) * dm + p.omega_B ** 2 * cur.B - p.gamma_c * p.delta * cur.S


# ------------------------------------------------------------------ controller
def control_command(x: State, p: Params, t: float, target: int) -&gt; Tuple[float, float]:
    """Return (u, raw) for one micro step. raw is the unclipped command."""
    drift = derivatives(x, p, t, 0.0).U
    raw = -drift + KP * (float(target) - x.U)
    return max(-U_AUTHORITY, min(U_AUTHORITY, raw)), raw


def basin_sign(U: float) -&gt; int:
    if abs(U - 1.0) &lt;= BASIN_TOL:
        return 1
    if abs(U + 1.0) &lt;= BASIN_TOL:

        return -1
    return 0


def nearest_basin_sign(U: float) -&gt; int:
    return 1 if abs(U - 1.0) &lt;= abs(U + 1.0) else -1


def macro_step(x: State, p: Params, t0: float, target: Optional[int],
               micro_steps: int = MICRO_STEPS, macro_dt: float = MACRO_DT) -&gt; Tuple[State, Dict[str, float]]:
    """Advance one macro interval. target=None runs the plant with u=0 (observe)."""
    h = macro_dt / micro_steps
    t = t0
    u_abs = u_sq = u_peak = 0.0
    sat = 0
    for _ in range(micro_steps):
        if target is None:
            u = raw = 0.0
        else:
            u, raw = control_command(x, p, t, target)
        u_abs += abs(u) * h
        u_sq += u * u * h
        u_peak = max(u_peak, abs(u))
        sat += int(abs(raw) &gt; U_AUTHORITY + EPS)
        x = rk4_step(x, p, t, h, u)
        t += h
    return x, {"u_abs": u_abs, "u_sq": u_sq, "u_peak": u_peak, "saturated": sat}
```



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

```
# ------------------------------------------------------------------ trajectory
@dataclass
class Result:
    final: State
    U_hist: List[float]
    target_sign: int
    born: int
    convey: int
    convey_start: int
    convey_confirm: int
    cert: int
    cert_start: int
    cert_complete: int
    max_streak: int
    actuator_abs_integral: float
    actuator_sq_integral: float
    actuator_peak: float
    actuator_saturated_periods: int


def simulate(x0: State, p: Params, target: Optional[int] = None, controlled: bool = True) -&gt; Result:
    """Canonical 20-macro-step trajectory with CONVEY-5 / CERT-10 predicates.

    target defaults to the nearest basin of U0. controlled=False runs u=0.
    """
    tgt = nearest_basin_sign(x0.U) if target is None else int(target)
    b0 = basin_sign(x0.U)

    born = int(b0 != 0)
    streak = 1 if born else 0
    ssign = b0 if born else 0
    best = streak
    convey = cert = 0
    cs = cc = ts = tc = -1
    ua = usq = upk = 0.0
    nsat = 0
    x = x0
    hist = [x0.U]
    for m in range(1, MACRO_STEPS + 1):
        x, a = macro_step(x, p, (m - 1) * MACRO_DT, tgt if controlled else None)
        ua += a["u_abs"]; usq += a["u_sq"]; upk = max(upk, a["u_peak"]); nsat += a["saturated"]
        hist.append(x.U)
        b = basin_sign(x.U)
        if b:
            if b == ssign:
                streak += 1
            else:
                ssign, streak = b, 1
        else:
            ssign, streak = 0, 0
        best = max(best, streak)
        if not convey and streak &gt;= CONVEY_WINDOW:
            convey, cs, cc = 1, m - CONVEY_WINDOW + 1, m
        if not cert and streak &gt;= CERT_WINDOW:
            cert, ts, tc = 1, m - CERT_WINDOW + 1, m
    return Result(x, hist, tgt, born, convey, cs, cc, cert, ts, tc, best, ua, usq, upk, nsat)

def sample_state(rng) -&gt; State:
    return State(float(rng.uniform(0.0, 1.0)), float(rng.uniform(-1.0, 1.0)), float(rng.uniform(0.0, 1.0)),
                 float(rng.uniform(-1.0, 1.0)), float(rng.uniform(-1.0, 1.0)), float(rng.uniform(-1.0, 1.0)))


def sample_params(rng) -&gt; Params:
    return Params(**{k: float(rng.uniform(lo, hi)) for k, (lo, hi) in PARAMETER_RANGES.items()})


def s_roots(p: Params) -&gt; Tuple[float, float]:
    """Roots of equation (6): S_minus &lt; 0 &lt; S_plus."""
    a = 0.75 * p.beta_s
    d = math.sqrt(p.alpha_s ** 2 + 4.0 * a * p.delta)
    return (-p.alpha_s - d) / (2.0 * a), (-p.alpha_s + d) / (2.0 * a)
```

omnicompass/compass_law.py

*174 lines.*

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. Evaluation and simulation use only; any other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE.
"""The compass: one smooth law for every muscle, and the plug every muscle is wired through.

A muscle is a plug with one wire in and one wire out. The plug reads its meters and writes its lever; all the
thinking is here, in the brain, written once for every muscle.

The plug (Plug)
  read      the service reading (response time, temperature, queue) and the lever as it stands
  write     one value; the plug clips it to the cover, sends it, and reads back what the device took
  restore   the lever back to the snapshot, the value read once before the first write; it never moves after that
  one writer if the lever is found at a value the brain did not write, someone else owns it: the brain stops writing
            and leaves that value alone (restoring over it would fight the new owner)

Two bands
  cover     the outer band, the lever's hard range (the device's or the buyer's lowest and highest setting); every
            write is clipped to it, so nothing the math does can set the lever outside
  compass      the service reading as a position in its band, 0 to 1 (0: calm, 1: the service line); the brain pulls i
    ↪ t
            to the bottom of the compass, the center (0.5 by default); the walls run to 0.05 and 0.95, and the last 5%
    ↪  on
            each side is cushion (10% in all) before the line

The force (CompassLaw.force), the physics of a ball in a compass with the right friction
  pull      K_P x (position - center): gentle near the bottom, harder up the walls
  push      K_D x velocity: whatever is shoving the position (a load rising, heat building) is met by an equal and
            opposite push; it is also the friction that stops the ball sloshing past the bottom (with K_D chosen for

            critical damping it glides to the center and stops, no overshoot, no ringing)
  smooth    the force is A x tanh(raw / A): a spring near the center that bends over and flattens into its maximum A,
            never a corner, never a hammer; the same shape the engine already uses for v_eff (omnicompass/core.py)
  fail up   past the 0.95 wall the up side goes to its full force at once, and the down side may not act until the
            position is back inside the compass

Two forces (antagonist pairs)
  up        a positive force adds capacity, power, cooling or speed (scale out, raise a clock, start a chiller)
  down      a negative force takes it back (scale in, lower a limit, warm a setpoint)
  each side has its own gain (lever units per unit of force), because adding and taking back do not cost the same;
  a muscle with two wires (a GPU's clock range and power limit) gives each side its own lever; a one-wire muscle
  carries both directions on its one lever

The frozen engine (omnicompass/core.py) and its proofs are untouched; this is a separate law, run as its own arm.
"""
from __future__ import annotations

import math
from dataclasses import dataclass, field
from typing import Optional


def clamp(v, lo, hi):
    return lo if v &lt; lo else hi if v &gt; hi else v


@dataclass
class Band:
```



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

```
    """A reading's band: lo is calm (position 0), hi is the line (position 1)."""
    lo: float
    hi: float
    center: float = 0.5
    cushion: float = 0.05

    def position(self, x: float) -&gt; float:
        return (x - self.lo) / (self.hi - self.lo)

    @property
    def wall_low(self) -&gt; float:
        return self.cushion

    @property
    def wall_high(self) -&gt; float:
        return 1.0 - self.cushion


@dataclass
class CompassLaw:
    """The brain's law for one muscle. dt: seconds between decisions; tau: the muscle's response time in seconds
    (how long the service reading takes to follow the lever); kd defaults to critical damping for that response."""
    band: Band
    dt: float
    tau: float
    kp: float = 1.0
    kd: Optional[float] = None
    authority: float = 1.0

    smooth: float = 0.5          # the velocity estimate's smoothing (0: raw difference, near 1: heavily smoothed)
    p: Optional[float] = field(default=None, init=False)
    v: float = field(default=0.0, init=False)

    def __post_init__(self):
        if self.kd is None:
            # a first-order muscle with response tau under a P-D pull: critical damping when kd = 2 sqrt(kp tau) - 1
            # scaled to the decision period; never negative
            self.kd = max(0.0, 2.0 * math.sqrt(self.kp * self.tau / self.dt) - 1.0) * self.dt

    def sense(self, reading: float) -&gt; float:
        p = self.band.position(reading)
        if self.p is not None:
            self.v = self.smooth * self.v + (1.0 - self.smooth) * (p - self.p) / self.dt
        self.p = p
        return p

    def force(self, reading: float) -&gt; float:
        """Positive: the up side (add capacity); negative: the down side (take it back). Bounded by the authority."""
        p = self.sense(reading)
        if p &gt;= self.band.wall_high:
            return self.authority                                   # fail up: past the wall, full force up at once
        raw = self.kp * (p - self.band.center) + self.kd * self.v
        f = self.authority * math.tanh(raw / self.authority)
        if f &lt; 0.0 and self.v &gt; 0.0 and p &gt; self.band.center:
            f = 0.0                                                 # never take back while it climbs above the center
        return f
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

```
class ForeignWriter(Exception):
    pass


class Plug:
    """One wire in, one wire out. Subclasses implement _read_service, _read_lever and _send; everything else is here."
    ↪ ""

    def __init__(self, lo: float, hi: float, tolerance: float = 1e-6):
        self.lo, self.hi, self.tol = lo, hi, tolerance
        self.snapshot: Optional[float] = None
        self.last_written: Optional[float] = None
        self.owned = True
        self.writes = 0
        self.clipped = 0

    # the wires ------------------------------------------------------------------------------------------------
    def _read_service(self) -&gt; float:
        raise NotImplementedError

    def _read_lever(self) -&gt; float:
        raise NotImplementedError

    def _send(self, value: float) -&gt; None:
        raise NotImplementedError

    # the contract ---------------------------------------------------------------------------------------------

    def attach(self) -&gt; float:
        """Read the lever once, before any write: the restore point. It never moves after this."""
        self.snapshot = self._read_lever()
        return self.snapshot

    def read(self) -&gt; float:
        return self._read_service()

    def lever(self) -&gt; float:
        return self._read_lever()

    def write(self, value: float) -&gt; Optional[float]:
        """Clip to the cover, check nobody else moved the lever, send, read back. None: not written (not owned)."""
        if not self.owned:
            return None
        now = self._read_lever()
        expected = self.last_written if self.last_written is not None else self.snapshot
        if expected is not None and abs(now - expected) &gt; self.tol:
            self.owned = False                                  # someone else moved it: it is theirs now
            raise ForeignWriter(f"lever at {now}, last written {expected}")
        v = clamp(value, self.lo, self.hi)
        self.clipped += int(v != value)
        if abs(v - now) &lt;= self.tol:
            return now
        self._send(v)
        self.last_written = self._read_lever()
        self.writes += 1
        return self.last_written

    def restore(self) -&gt; bool:
        """The lever back to the snapshot. Left alone when someone else owns it. True: at the snapshot (or not ours)."
    ↪ ""
        if not self.owned or self.snapshot is None:
            return True
        if abs(self._read_lever() - self.snapshot) &gt; self.tol:
            self._send(self.snapshot)
        self.last_written = None
        return abs(self._read_lever() - self.snapshot) &lt;= self.tol
```

`realms/compass_arm.py`

*184 lines.*



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. Evaluation and simulation use only; any other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE.
"""The compass law on every realm muscle (omnicompass/compass_law.py): each plant's knob held by its own compass.

Each plant's service is read as one position in its compass (0 calm, 1 the line): the worst of its queue or lateness,
    ↪ its
load above half, and, for a storage site, its draw on the grid connection above half. The last period in violation
reads as past the wall (fail up). The compass's force moves the knob: a positive force adds capacity, power, cooling o
    ↪ r
protection, a negative force gives it back, each knob inside its own cover. At the kill every knob returns to native.

Knob by plant (the override key the plant already obeys, its native value, its cover, and which way is "more"):
  compute_pool   power: limit share 0.4-1; replicas-type: the HPA target held at native (loosening it past the
                 operator's own costs the service time, the same rule as the card's speed floor); machine pools: one
                 machine released while the force is clearly down and the machines left cover the recent peak with
                 margin (RELEASE_MARGIN); admission: native
  thermal_zone   setpoint between the band's stress (colder, more cooling) and calm ends; units: one released while th
    ↪ e
                 force is clearly down; power: native (a cooling cap moves heat later, never away)
  energy_storage reserve between the band's ends (a lower reserve gives the battery to the site); power: native
  motion_axis    speed and effort: 0.4-1
  process_loop   setpoint from native toward the band's calm end; actuator range 0.3-1; power 0.4-1
"""
from __future__ import annotations

from omnicompass.compass_law import Band, CompassLaw, clamp

UP, DOWN, RELEASE = 0.10, 0.02, -0.2
RELEASE_MARGIN = 0.3     # a machine goes back only when the rest covers the recent peak at 0.3 of the plant's own rel
    ↪ ease level
# 0.3: the largest margin at which no node pool is later than native (results/realms/RELEASE_MARGIN_SWEEP.md, the
# per-muscle sweep of 2026-10-05: 0.6 saved 2.0% energy with 32 of 42 pools later, 0.4 left 4 later, 0.3 none). A mach
    ↪ ine
# boots in minutes and a burst that comes in the meantime is served late, so speed first holds the machines (the same
# rule as the verdict on real Kubernetes: a machine goes back only where it costs no speed)


def position(plant) -&gt; float:
    if plant.vhist and plant.vhist[-1]:
        return 1.0
    if plant.template == "thermal_zone":                        # a zone's service is its temperature against its limit
        P = plant.P
        return (plant.T - min(P["stress"], P["calm"])) / (P["t_limit"] - min(P["stress"], P["calm"]))
    o = plant.observe()
    p = max(o["queue_ratio"], (o["load_ratio"] - 0.5) / 0.5)
    if plant.template == "energy_storage":
        p = max(p, (o["power_stress"] - 0.5) / 0.5)
    return p


def release_safe(plant) -&gt; bool:
    """A discrete unit goes back only when what is left covers the recent peak (the plant's own release window), with
    RELEASE_MARGIN of headroom on a machine pool: a machine boots in minutes, so a burst that arrives after a release
    is served late until it is back."""
    P = plant.P
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

```
    if plant.template == "compute_pool":
        from .plants import RELEASE_WINDOW, RELEASE_FRAC
        n = plant.n
        return (n &gt; P["n_min"] and not plant.pending and plant.Q &lt;= 0.0
                and len(plant.ahist) == RELEASE_WINDOW * P["startup_steps"]
                and max(plant.ahist) / ((n - 1) * plant.mu(plant.cap)) &lt;= RELEASE_FRAC * 0.95 * RELEASE_MARGIN)
    u = plant.units_on
    return u &gt; 1 and plant.T &lt;= plant.setpoint() and plant.qc_cmd_avg / ((u - 1) * P["q_unit_w"]) &lt;= 0.8


def capping_saves(plant) -&gt; bool:
    """Whether a lower power cap saves energy on this pool at all (race to idle, MECHANISM_OF_ACTION 9.6).

    With throughput mu(c) ~ c^eps at power cap c and the autoscaler holding utilisation near its target u*, the energy
    per unit of work is (p_idle / u* + p_dyn c) / mu(c). Its slope at c = 1 is positive (a lower cap saves) only if
    eps &lt; p_dyn / (p_idle / u* + p_dyn). Where idle power dominates (a quantum computer's cryostat, a network switch),
    running slower keeps the machines on longer and costs more than it saves: the cap is left native."""
    import math
    P = plant.P
    eps = math.log(plant.mu(1.0) / plant.mu(0.99)) / math.log(1.0 / 0.99)   # the machine's own speed curve at full po
    ↪ wer
    u = P.get("target", 0.7)
    return eps &lt; P["p_dyn"] / (P["p_idle"] / u + P["p_dyn"])


def effort_cap_saves(plant) -&gt; bool:
    """Whether a lower effort cap saves energy on this axis at all (MECHANISM_OF_ACTION 9.6).

    A cap on effort lowers the copper loss of a full-acceleration move, (J a_max / kt)^2 R, and stretches the move, du
    ↪ ring
    which the axis keeps drawing its idle power and turning against viscous friction, p_idle + b v_max^2. It pays only
    where the first exceeds the second (a reaction wheel); elsewhere (traction, a flight axis, a robot joint) a slower
    move costs more than the current it saves, and the cap is left native."""
    P = plant.P
    copper = (P["J"] * P["a_max"] / P["kt"]) ** 2 * P["R"]
    standing = P["p_idle"] + P["b"] * P["v_max"] ** 2
    return copper &gt; standing


SLACK = 0.5             # a speed knob is offered only where the axis at full speed is busy at most half the time (Om
    ↪ ni v3)


def speed_slack(plant) -&gt; bool:
    """Whether the axis has slack to spend on a slower, gentler move at all (Omni v3, docs/REALMS_PREREGISTRATION.md).

    The compass may ease a motion axis down to 0.4 of its speed, which stretches every move by up to 2.5 times. An axi
    ↪ s
    whose duty at full speed, task rate x (move time + dwell), is already above one half has no room for that: tasks
    arrive in bunches, and a stretched move pushes the next ones past their deadline (a train on its timetable, a lift
    ↪  at
    rush hour). There the speed knob is left native, as the robot benchmark's paired trial leaves it
    (docs/ROBOTICS_PREREGISTRATION.md). Computed from the plant's own figures before any decision."""
    import math
    P = plant.P
    v, a, D = P["v_max"], P["a_max"], P["D"]
```



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

```
    move = 2.0 * math.sqrt(D / a) if D &lt; v * v / a else D / v + v / a
    return P["task_rate"] * (move + P["dwell"]) &lt;= SLACK


def lever(plant, knob):
    """(override key, native value, low, high, sign): sign +1 when a higher value is more capacity."""
    P, t = plant.P, plant.template
    if knob == "admission":
        return None
    if t == "compute_pool":
        if knob == "power":
            return ("power", 1.0, 0.4, 1.0, 1) if capping_saves(plant) else None
        if P.get("ca"):
            return ("release", None, 0, 0, 0)
        return None                                          # the HPA target is held at the operator's own
    if t == "thermal_zone":
        if knob == "setpoint":
            return ("setpoint", P["t_set"], min(P["stress"], P["calm"]), max(P["stress"], P["calm"]), -1)
        if knob == "capacity":
            return ("release", None, 0, 0, 0)
        # the cooling power cap gives nothing back: the heat the zone makes must leave it either way, so a lower cap o
    ↳ nly
        # lets the temperature drift up while the PI command grows and stages more units (and their fans). Native
        # (MECHANISM_OF_ACTION 9.6)
        return None
    if t == "energy_storage":
        if knob == "setpoint" and P.get("backup"):
            return None                                      # a backup reserve (a UPS) is kept for an outage: nativ

    ↳ e
        if knob == "setpoint":
            # the reserve may go down to the stress end (the battery given to the site under strain), never above the
            # operator's own: a kWh held back in a calm battery is a kWh bought from the grid (MECHANISM_OF_ACTION 9.6
    ↳ )
            return ("setpoint", P["reserve"], min(P["stress"], P["reserve"]), P["reserve"], -1)
        return None                                          # a battery's power limit gives nothing back: native
    if t == "motion_axis":
        if knob == "power":
            return ("power", 1.0, 0.4, 1.0, 1) if effort_cap_saves(plant) else None
        return ("capacity", 1.0, 0.4, 1.0, 1) if speed_slack(plant) else None
    if t == "process_loop":
        if knob == "setpoint":
            lo, hi = min(P["sp"], P["calm"]), max(P["sp"], P["calm"])
            return ("setpoint", P["sp"], lo, hi, 1 if P["sp"] &gt; P["calm"] else -1)
        if knob == "power":
            return ("power", 1.0, 0.4, 1.0, 1)
        return ("capacity", 1.0, 0.3, 1.0, 1)
    return None


def compass_apply(plant, knob, dt=1.0, tau=3.0):
    """One decision: the plant's compass reads its position and moves its knob. Returns the override written."""
    lv = lever(plant, knob)
    if lv is None:
        plant.override = {}
        return {}
    if not hasattr(plant, "_compass_law"):
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

```
        plant._compass_law = CompassLaw(Band(0.0, 1.0), dt=dt, tau=tau, kp=1.0, smooth=0.3)
        plant._compass_law.kd *= 3.0
        plant._compass_law_x = lv[1]
    b = plant._compass_law
    F = b.force(position(plant))
    key, native, lo, hi, sign = lv
    if key == "release":
        ok = F &lt; RELEASE and b.p &lt; b.band.center and release_safe(plant)
        plant.override = {"release": 1.0} if ok else {}
        return plant.override
    span = hi - lo
    if key == "setpoint" and plant.template == "energy_storage":
        # a battery's reserve is spent only where it buys service: at the connection's wall the reserve drops to the
        # stress end at once; anywhere else the operator's own reserve, so no round trip is paid for nothing (9.6)
        P = plant.P
        excess = plant.imp - P["p_lim_w"]                       # what the connection is over its limit by, now
        coverable = 0.0 &lt; excess &lt;= P["p_batt_w"] * plant.override.get("power", 1.0)
        x = lo if (b.p &gt;= b.band.wall_high and (coverable or plant.imp &gt;= 0.95 * P["p_lim_w"] and excess &lt;= 0.0)) else
    ↪  native
        plant._compass_law_x = x
        plant.override = {} if abs(x - native) &lt; 1e-9 else {key: x}
        return plant.override
    if b.p &gt;= b.band.wall_high:
        x = hi if sign &gt; 0 else lo                                # fail up: full capacity at once
    else:
        g = UP if F &gt; 0 else DOWN
        x = clamp(plant._compass_law_x + sign * g * F * span, lo, hi)
    plant._compass_law_x = x

    plant.override = {} if abs(x - native) &lt; 1e-9 else {key: x}
    return plant.override
```

## omni_controller/gpu_compass.py

*432 lines.*

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. Evaluation and simulation use only; any other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE.
"""Omni-Compass on one GPU through two wires: the compass law (omnicompass/compass_law.py) on a real card.

The card keeps its own control: NVIDIA's firmware still boosts and still protects the chip. Omni holds two settings th
    ↪ e
card already accepts, and nothing else:
  up wire    the clock ceiling: how high boost may climb (nvidia-smi -lgc MIN,MAX; -rgc resets it)
  down wire  the power limit: the lid above what that ceiling draws (nvidia-smi -pl W)

Every decision (--interval seconds):
  read      nvidia-smi: power.draw, temperature, utilization, power.limit, clocks.sm; the workload's response times
  position  the service as one place in its compass, 0 calm to 1 the line: response time only (the mean over the last
            --latency-window-s, between a tenth of the line, the bare service time, and the line; the 95th percentile
    ↪ at
            or past the line, or a failed request, is past the wall). A card that is busy is doing its work; being bus
    ↪ y
            is not a breach and is not read as one
  native    what the card does on its own, learned from its own meter before the compass may lower anything: while the
            ceiling is at the top and the card is busy, its clock and its draw (the clock its own power limit holds it
            at, and what that costs). Until --learn-samples busy readings are in, the ceiling stays at the top
  race      while the card is saturated (utilization at or over --race-util: work is waiting), the ceiling goes to the
            top and the lid to the start limit, so a burst is served at full speed; the compass paces only the slack
  force     the compass: pull to the center, push against what is rising, tanh-bounded; past the 0.95 wall: fail up
  verdict   how far the ceiling may go (omnicompass/verdict.py): while the service is calm, a paired trial: the ceilin
    ↪ g
            at the top until --verdict-samples requests are measured, then one 15 MHz step past the deepest step
```


© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

```
            already allowed until as many again are measured; the card's own time on each request (the workload's
            service_ms, the wait in the queue left out) is compared: at most --allow slower and the step is allowed,
            slower than that and it is refused and not tried again for --verdict-recheck decisions. Where no step
            passes, the ceiling stays at the top: the card runs as it does alone
  law       down gain 0.0125, the compass's center at the v5 edge (omnicompass/edge.py: 1 - (climb x reaction + 3.09 x
            wobble) of the card's own response-time position, never below 0.4 unless a line crossed while the ceiling
    ↪ was
            held under the top has taught it otherwise; the wall moves out with the reaction part), the speed floor at
    ↪  the
            card's own busy clock (amendment 8)
  steady    while the card is saturated against its own power limit (work waiting and the draw at the limit), the
            firmware boosts a step, hits the limit and is knocked back: a sawtooth. The ceiling is then held at the
            card's own busy clock under that limit (what the sawtooth averages to), so the same watts serve the work
            without the knock-backs (amendment 9). Only under an operator's cap (the start limit under the card's
            factory limit, amendment 12): on the card's own limit the firmware's boost is left to serve the burst. It
            never holds under that clock, and blind always fails up
  write     up wire: the ceiling moves by the force (fast up, gently down), inside its cover: from the card's own busy
            clock (never slower than native while there is work) to the top; down wire: the lid at the card's own busy
            draw plus --lid-headroom, never under it and never over the start limit, inside the declared envelope;
            fail up: ceiling to the top and the lid to the start limit at once
  guards    blind (meters unreadable, response times stale): fail up; heat (the card reports a thermal or hardware
            slowdown): never tighten; one writer: a power limit neither mine nor the start means another writer, so
            observe only from then on and exit 5; read-back: every write is read back
  restore   on exit, kill file or SIGTERM: clocks reset (-rgc) and the power limit back to the start, read back

The audit (--audit) carries the same records the bench reads from the one-wire governor (snapshot, decision with
telemetry and decided_by, write and actuator for each power-limit write, restored), plus clock_write records.
Exit: 0 clean, 3 restore failed, 4 a write refused, 5 another writer.

"""
from __future__ import annotations

import argparse
import json
import math
import os
import shlex
import signal
import subprocess
import sys
import time

from omnicompass import master
from omnicompass.compass_law import Band, CompassLaw, clamp
from omnicompass.verdict import Verdict
from omni_controller.gpu_governor import query, snapshot, throttle, slowed, WriteFailed, SNAPSHOT
from omni_controller.muscles import latency_sense, latency_window


# the law (docs/GPU_PREREGISTRATION.md, amendment 8): the down gain (share of the top clock per unit of force), the co
    ↪ mpass's
# center (where it holds the response time, 0 the bare service time and 1 the line) and the speed floor (a share of th
    ↪ e
# card's own busy clock); the same in realms/gpu_card.py
LAW = {"down": 0.0125, "center": 0.4, "floor": 1.0}
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

```
def smi_run(smi, args):
    try:
        p = subprocess.run(shlex.split(smi) + args, capture_output=True, text=True, timeout=20)
        return p.returncode, p.stderr.strip()[:300]
    except (subprocess.SubprocessError, OSError) as e:
        return -1, f"{type(e).__name__}: {e}"[:300]


def query_top_clock(smi, g):
    try:
        out = subprocess.run(shlex.split(smi) + ["-i", str(g), "--query-gpu=clocks.max.sm", "--format=csv,noheader,nou
    ↪ nits"],
                             capture_output=True, text=True, timeout=10, check=True).stdout
        return float(out.strip().splitlines()[0])
    except (subprocess.SubprocessError, OSError, ValueError, IndexError):
        return None


def query_default_limit(smi, g):
    """The card's factory power limit (watts), or None if the driver will not say. A start limit under it means an
    operator's cap is underneath (amendment 12)."""
    try:
        out = subprocess.run(shlex.split(smi) + ["-i", str(g), "--query-gpu=power.default_limit", "--format=csv,nohead
    ↪ er,nounits"],
                             capture_output=True, text=True, timeout=10, check=True).stdout
        return float(out.strip().splitlines()[0])
    except (subprocess.SubprocessError, OSError, ValueError, IndexError):
        return None

def query_min_clock(smi, g):
    """The lowest graphics clock the card supports (the floor of a -lgc range); 300 MHz if the driver will not list th
    ↪ em."""
    try:
        out = subprocess.run(shlex.split(smi) + ["-i", str(g), "--query-supported-clocks=graphics", "--format=csv,nohe
    ↪ ader,nounits"],
                             capture_output=True, text=True, timeout=10, check=True).stdout
        return int(min(float(x) for x in out.split() if x.strip()))
    except (subprocess.SubprocessError, OSError, ValueError):
        return 300


class GpuCompass:
    def __init__(self, a):
        self.a = a
        self.g = int(str(a.gpus).split(",")[0])
        self.log = open(a.audit, "a")
        self.writes = 0
        self.foreign = False
        self.clock_set = False
        self.enforced_ok = query(a.smi, [self.g], enforced=True) is not None
        s = query(a.smi, [self.g], self.enforced_ok)
        if s is None:
            raise SystemExit("nvidia-smi unreadable at start: no snapshot, so I take no authority")
        snap = snapshot(a.smi, [self.g])
        self.start = s[self.g]["limit"]
```



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

```
        # an operator's cap underneath: the start limit under the card's factory limit (unknown: treated as no cap)
        dflt = query_default_limit(a.smi, self.g)
        self.capped = dflt is not None and self.start &lt; dflt - 1.0
        self.expect = self.start
        self.top = query_top_clock(a.smi, self.g) or s[self.g]["clock_mhz"]
        self.floor_w = max(float(a.floor_w or 0.0), s[self.g]["min"])
        self.c_lo = a.clock_min_share * self.top
        self.c_floor = query_min_clock(a.smi, self.g)
        self.ceiling = self.top           # where the compass holds the ceiling (continuous)
        self.written_ceiling = self.top    # what the card was last told
        self.busy_clk, self.busy_draw = [], []    # the card on its own: busy clock and busy draw, ceiling at the top
        self.prof = dict(LAW)
        if a.down_gain is not None:
            self.prof["down"] = a.down_gain
        self.verdict = Verdict(tolerance=a.allow, min_samples=a.verdict_samples, probe_every=a.verdict_every,
                               recheck=a.verdict_recheck)
        self.svc_t = None                 # elapsed_seconds of the last request whose service time the verdict has se
    ↪ en
        self.brain = CompassLaw(Band(0.0, 1.0, center=self.prof["center"]), dt=a.interval, tau=2.0 * a.interval, kp=1.
    ↪ 0, smooth=0.3)
        self.brain.kd *= 3.0
        # the v5 edge: the clock ceiling answers within the compass's own damping time (2 decisions)
        from omnicompass.edge import Edge
        self.edge = Edge(react=2.0)
        self.audit({"snapshot": {str(self.g): {"limit_w": self.start, "min_limit_w": s[self.g]["min"],
                                               "enforced_w": s[self.g]["enforced"] if self.enforced_ok else None,
                                               "clock_top_mhz": self.top,
                                               **{f: snap[f][str(self.g)] for f in SNAPSHOT}}},

                    "engine": "compass, two wires", "mode": a.mode, "law": self.prof,
                    "verdict": {"allow": a.allow, "samples": a.verdict_samples, "every": a.verdict_every,
                                "recheck": a.verdict_recheck},
                    "covers": {"clock_mhz": [round(self.c_lo), round(self.top)], "power_w": [self.floor_w, self.start]
    ↪ }})
        if a.mode == "cap" and snap["power.management"][str(self.g)].lower() != "enabled":
            self.audit({"refused": "power management not Enabled: a written limit would not bind"})
            raise SystemExit("power management not Enabled: cap mode refused, no authority taken")

    def audit(self, rec):
        self.log.write(json.dumps({"time": time.time(), **rec}) + "\n"); self.log.flush()

    def position(self, util):
        a = self.a
        ls = latency_sense(a.latency_file, a.latency_window_s) if a.latency_file and a.slo_ms else None
        if ls is not None and ls["blind"]:
            return None, ls
        resp = 0.0
        if ls is not None:
            # the position is the mean response time of the window between the bare service time and the line (what
            # the model reads); the 95th percentile at or past the line, or any failed request, is past the wall
            bare = a.slo_ms / 10.0
            w = latency_window(a.latency_file, a.latency_window_s)["ms"]
            mean = sum(w) / len(w) if w else ls["p95"]
            resp = (mean - bare) / (a.slo_ms - bare)
            if ls["fail"] or ls["p95"] &gt;= a.slo_ms:
                resp = max(resp, 1.0)
        return resp, ls
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

```
    def service_costs(self):
        """The card's own time (ms) on each request finished since the last decision (the workload's service_ms)."""
        import csv
        try:
            rows = list(csv.DictReader(open(self.a.latency_file)))
        except (OSError, TypeError):
            return []
        out, last = [], self.svc_t
        for row in rows:
            t, svc = row.get("elapsed_seconds"), row.get("service_ms")
            if row.get("ok") != "1" or not t or not svc:
                continue
            t = float(t)
            if last is None or t &gt; last:
                out.append(float(svc))
                self.svc_t = t if self.svc_t is None else max(self.svc_t, t)
        if last is None:                  # the first read only finds where the file stands
            return []
        return out

    def learn(self, r):
        """The card on its own: while the ceiling is at the top and the card is busy, its clock and draw are native's.
    ↪ """
        if r is not None and self.written_ceiling &gt;= self.top and r["limit"] &gt;= self.start - 1.0 and r["util"] &gt;= 0.9:
            self.busy_clk = (self.busy_clk + [r["clock_mhz"]])[-200:]
            self.busy_draw = (self.busy_draw + [r["draw"]])[-200:]

    def native(self):
        """(busy clock, busy draw) of the card on its own, or (None, None) until enough busy readings are in."""
        if len(self.busy_clk) &lt; self.a.learn_samples:
            return None, None
        c, d = sorted(self.busy_clk), sorted(self.busy_draw)
        return c[len(c) // 2], d[int(0.9 * (len(d) - 1))]

    def write_clock(self, mhz, why):
        mhz = int(round(mhz))
        cmd = ["-i", str(self.g), "-lgc", f"{self.c_floor},{mhz}"]
        if self.a.mode != "cap" or self.foreign:
            self.audit({"would_clock_write": cmd, "why": why}); return
        rc, err = smi_run(self.a.smi, cmd)
        self.audit({"clock_write": cmd, "why": why, "rc": rc, "stderr": err})
        if rc != 0:
            raise WriteFailed(f"nvidia-smi -lgc {self.c_floor},{mhz} returned {rc}: {err}")
        self.clock_set = True
        self.writes += 1

    def write_limit(self, w, why):
        w = int(w)
        cmd = ["nvidia-smi", "-i", str(self.g), "-pl", str(w)]
        if self.a.mode != "cap" or self.foreign:
            self.audit({"would_write": cmd, "why": why}); return
        self.audit({"write": cmd, "why": why})
        t0 = time.time()
        rc, err = smi_run(self.a.smi, ["-i", str(self.g), "-pl", str(w)])
        act = {"gpu": self.g, "requested_w": w, "t_requested": t0, "rc": rc, "stderr": err}
```



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

```
        if rc != 0:
            self.audit({"actuator": act, "write_failed": cmd})
            raise WriteFailed(f"nvidia-smi -pl {w} returned {rc}: {err}")
        self.writes += 1
        s = query(self.a.smi, [self.g], self.enforced_ok)
        if s is not None:
            back = s[self.g]
            act.update({"readback_w": back["limit"], "enforced_w": back["enforced"], "t_readback": time.time(),
                        "realized": abs(back["limit"] - w) &lt; 1.0, "override": back["enforced"] &lt; back["limit"] - 1.0})
            if act["realized"]:
                act["delay_s"] = round(act["t_readback"] - t0, 3)
                self.expect = w
        self.audit({"actuator": act})

    def step(self):
        a, g = self.a, self.g
        s = query(a.smi, [g], self.enforced_ok)
        if s is None:
            p, ls, r = None, None, None
        else:
            r = s[g]
            if abs(r["limit"] - self.expect) &gt;= 1.0 and not self.foreign:
                self.foreign = True
                self.audit({"foreign_writer": {"gpu": g, "reads_w": r["limit"], "expected_w": self.expect},
                            "action": "observe only from now on; both wires left to the other writer"})
            p, ls = self.position(r["util"])
        self.learn(r)
        n_clk, n_draw = self.native()

        thr = throttle(a.smi, [g]) if r is not None else None
        heat = slowed((thr or {}).get(g))
        saturated = r is not None and r["util"] &gt;= a.race_util
        # the verdict: the card's own time on the requests since the last decision, at the step the ceiling stood at;
        # then the deepest step the compass may use now, or the step a trial needs
        self.verdict.observe(self.service_costs())
        if p is not None:
            # the v5 edge: past the line while the ceiling was held under the top teaches the cushion a floor; then th
    ↳ e
            # balance point and the wall move with the card's own climb and wobble
            if p &gt;= 1.0 and self.written_ceiling &lt; self.top:
                self.edge.learn_fall(0.1)
            center = self.edge.update(p)
            base = self.prof["center"] if self.edge.floor &lt;= 0.0 else 0.0
            self.brain.band.center = min(0.995, max(0.0, max(base, center)))
            self.brain.band.cushion = min(0.05, max(0.005, self.edge.react_part))
        calm = p is not None and p &lt; self.brain.band.wall_high and not saturated
        deepest, trial, ev = self.verdict.tick(calm)
        if ev:
            self.audit({"verdict": ev, "state": self.verdict.state, "deepest_step": self.verdict.allowed})
        at_limit = r is not None and r["draw"] &gt;= 0.97 * r["limit"]
        if p is not None and saturated and at_limit and n_clk is not None and self.capped:
            # saturated against the card's own limit: hold the ceiling at the card's own busy clock under that limit,
            # no knock-backs; the lid stays at the start limit
            self.brain.force(p)
            ceiling = clamp(round(n_clk / a.min_change_mhz) * a.min_change_mhz, self.c_lo, self.top)
            lid = self.start
            who = "steady_under_limit"
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

```
        elif p is None or p &gt;= self.brain.band.wall_high or saturated:
            # fail up past the wall or blind; and race while work waits (the card saturated: a queue is forming), so a
            # burst is always served at full speed and the compass paces only the slack between bursts
            if p is not None:
                self.brain.force(p)
            ceiling, lid = self.top, self.start
            who = "blind_fail_up" if p is None else "fail_up" if p &gt;= self.brain.band.wall_high else "race"
        elif trial:
            self.brain.force(p)
            ceiling = self.top - deepest * a.min_change_mhz
            lid = self.start
            who = "verdict_trial"
        else:
            F = self.brain.force(p)
            # the speed floor: never under the clock the card reaches on its own while busy (until that is learned,
            # the top), so work waiting on the card is never served slower than native
            c_floor = self.top if n_clk is None else max(self.c_lo, min(self.top, n_clk * self.prof["floor"]))
            # and never past the deepest step the verdict has allowed (none allowed: the ceiling stays at the top)
            c_floor = max(c_floor, self.top - deepest * a.min_change_mhz)
            # the ceiling moves in whole clock steps, as the card's own clock does: the force times the gain (a share
            # of the top clock per unit of force) rounded to whole --min-change-mhz steps; a pull under half a step
            # moves nothing and is not stored up, so a calm card is paced only when the pull is clearly down
            down = self.prof["down"]
            step = (a.up_gain if F &gt; 0 else down) * F * self.top
            step = round(step / a.min_change_mhz) * a.min_change_mhz
            ceiling = clamp(self.ceiling + step, c_floor, self.top)
            if heat and ceiling &lt; self.ceiling:
                ceiling, who = self.ceiling, "thermal_hold"

            else:
                who = "compass"
            # the lid: the start limit scaled to what a fully busy card draws at this ceiling (a fifth of the draw doe
    ↪ s not
            # scale with the clock; the rest goes as clock^2.5, clock times voltage squared), plus headroom; at the to
    ↪ p
            # clock the lid is the start limit, so the lid never adds a hammer of its own
            # the lid: never under what the card draws on its own while busy (its own meter, not a curve), plus
            # headroom; the start limit until that is learned
            lid = self.start if n_draw is None else clamp(math.ceil(n_draw * (1.0 + a.lid_headroom)), self.floor_w, se
    ↪ lf.start)
        rec = {"decision": {str(g): {"telemetry": None if r is None else {
                    "util": r["util"], "draw_w": r["draw"], "temp_c": r["temp"], "limit_w": r["limit"],
                    "enforced_w": r["enforced"], "clock_mhz": r["clock_mhz"], "throttle": (thr or {}).get(g)},
               "position": None if p is None else round(p, 4), "velocity": round(self.brain.v, 4),
               "center": round(self.brain.band.center, 4), "wall": round(self.brain.band.wall_high, 4),
               "latency_p95_ms": None if not ls else ls.get("p95"),
               "ceiling_mhz": round(ceiling), "want_w": int(lid), "decided_by": who,
               "native_busy_clock_mhz": n_clk, "native_busy_draw_w": n_draw,
               "verdict_state": self.verdict.state, "verdict_deepest_step": self.verdict.allowed}}}
        self.audit(rec)
        self.ceiling = ceiling
        if abs(ceiling - self.written_ceiling) &gt;= a.min_change_mhz or (ceiling &gt;= self.top and self.written_ceiling &lt;
    ↪ self.top):
            if ceiling &gt;= self.top and self.clock_set and a.mode == "cap" and not self.foreign:
                rc, err = smi_run(a.smi, ["-i", str(g), "-rgc"])
                self.audit({"clock_write": ["-i", str(g), "-rgc"], "why": who, "rc": rc, "stderr": err})
                if rc != 0:
```



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

```
                    raise WriteFailed(f"nvidia-smi -rgc returned {rc}: {err}")
                self.clock_set = False
            elif ceiling &lt; self.top:
                self.write_clock(ceiling, who)
            self.written_ceiling = ceiling
        cur = r["limit"] if r is not None else self.expect
        if abs(lid - cur) &gt;= a.min_change_w or (lid == self.start and cur != lid):
            self.write_limit(lid, who)

    def restore(self):
        failed = []
        if self.a.mode == "cap" and not self.foreign:
            rc, err = smi_run(self.a.smi, ["-i", str(self.g), "-rgc"])
            self.audit({"clock_write": ["-i", str(self.g), "-rgc"], "why": "restore", "rc": rc, "stderr": err})
            if rc != 0:
                failed.append(f"-rgc returned {rc}: {err}")
            s = query(self.a.smi, [self.g], self.enforced_ok)
            if s is None or abs(s[self.g]["limit"] - self.start) &gt;= 1.0:
                try:
                    self.write_limit(self.start, "reset: the limit read at start")
                except WriteFailed as e:
                    failed.append(str(e))
        s = query(self.a.smi, [self.g], self.enforced_ok) or {}
        back = s[self.g]["limit"] if self.g in s else None
        ok = self.foreign or (not failed and back is not None and abs(back - self.start) &lt; 1.0)
        self.audit({"restored": {str(self.g): back}, "start": {str(self.g): self.start}, "ok": ok,
                    "writes": self.writes, "foreign_writer": self.foreign, "restore_write_failed": failed})
        return ok

def parser():
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--mode", choices=["watch", "cap"], default="watch")
    ap.add_argument("--gpus", default="0")
    ap.add_argument("--smi", default=os.environ.get("NVIDIA_SMI", "nvidia-smi"))
    ap.add_argument("--interval", type=float, default=2.0)
    ap.add_argument("--duration", type=float, default=0.0)
    ap.add_argument("--audit", default="gpu_compass_audit.jsonl")
    ap.add_argument("--kill-file", default="/tmp/omni-gpu-kill")
    ap.add_argument("--latency-file", default="")
    ap.add_argument("--slo-ms", type=float, default=0.0)
    ap.add_argument("--latency-window-s", type=float, default=5.0, help="seconds of response times the position reads"
    ↪ )
    ap.add_argument("--race-util", type=float, default=0.95, help="utilization at which work is waiting: race, never p
    ↪ ace")
    ap.add_argument("--learn-samples", type=int, default=15, help="busy readings of the card on its own before the com
    ↪ pass may lower anything")
    ap.add_argument("--floor-w", type=float, default=0.0, help="the declared envelope's lowest watts")
    ap.add_argument("--clock-min-share", type=float, default=0.35, help="the clock ceiling's cover: lowest share of th
    ↪ e top")
    ap.add_argument("--allow", type=float, default=0.02, help="the most a ceiling step may add to the card's own time
    ↪ on a request")
    ap.add_argument("--verdict-samples", type=int, default=30, help="requests measured at native and at the trial step
    ↪ ")
    ap.add_argument("--verdict-every", type=int, default=8, help="decisions between trials")
    ap.add_argument("--verdict-recheck", type=int, default=900, help="decisions before a refused step is tried again")
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

```
    ap.add_argument("--up-gain", type=float, default=0.10, help="share of the top clock moved per unit of force, up")
    ap.add_argument("--down-gain", type=float, default=None, help="share of the top clock moved per unit of force, dow
    ↪ n (default: the law's)")
    ap.add_argument("--lid-headroom", type=float, default=0.10, help="the lid above the draw the ceiling takes")
    ap.add_argument("--min-change-mhz", type=float, default=15.0)
    ap.add_argument("--min-change-w", type=float, default=3.0)
    return ap


def main(argv=None):
    a = parser().parse_args(argv)
    master.refuse_if_off("GPU governor (two wires)")
    gov = GpuCompass(a)
    # what puts the card back if this process dies without doing it itself (the watchdog runs it)
    back = ([[a.smi, "-i", str(gov.g), "-rgc"], [a.smi, "-i", str(gov.g), "-pl", str(int(round(gov.start)))]]
            if a.mode == "cap" else [])
    master.register("GPU governor (two wires)", restore=back, stale_s=max(60.0, 5 * a.interval))
    stop = {"now": False}
    signal.signal(signal.SIGTERM, lambda *_: stop.__setitem__("now", True))
    t0 = time.time()
    failed = False
    try:
        while not stop["now"] and not os.path.exists(a.kill_file) and not master.is_off() and (a.duration &lt;= 0 or time
    ↪ .time() - t0 &lt; a.duration):
            try:
                master.heartbeat(); gov.step()
            except WriteFailed as e:
                gov.audit({"fatal": f"write failed: {e}"}); failed = True

                break
            except Exception as e:  # noqa: BLE001  a failed decision is logged; the kill path still runs
                gov.audit({"error": f"{type(e).__name__}: {e}"})
            end = time.time() + a.interval
            while time.time() &lt; end and not stop["now"] and not os.path.exists(a.kill_file) and not master.is_off():
                time.sleep(0.2)
    finally:
        ok = gov.restore()
    return 5 if gov.foreign else 3 if not ok else 4 if failed else 0


if __name__ == "__main__":
    sys.exit(main())
```

## omnicompass/adapter.py

*309 lines.*

```
"""Omni-Compass stack governor.

  sense      : stack telemetry -&gt; normalized observation vector
  assimilate : observation blended into the six-state engine state
  evolve     : one macro step of equations (1)-(7) with u = 0
  allocate   : engine state -&gt; directive (capacity, change gating, power, rollback, routing, alerting, paging)
  authority  : OBSERVE | AUTOPILOT; kill() revokes authority and returns control to the native managers
"""
from __future__ import annotations

import math
from collections import deque
from dataclasses import dataclass, asdict, field
from typing import Any, Dict, List, Optional

from .core import State, Params, macro_step, v_eff, control_command, U_AUTHORITY

OBSERVE = "observe"
AUTOPILOT = "autopilot"

# Engine parameters used for stack governance (fixed, not sampled).
STACK_PARAMS = Params(alpha=4.2, beta_int=0.0, beta_ext=0.0, k=1.7, sigma_1=0.38, delta=0.705, gamma_1=1.35,
                      gamma_c=1.0, lambda_0=-0.40, lambda_1=1.55, lambda_2=1.15, c=1.0, E_max=2.0,
                      omega_B=1.0, Q_B=2.5, alpha_s=0.12, beta_s=0.10, mu=3.782, alpha_E=3.487, alpha_U=4.2,
                      lambda_I=0.719, lambda_U=0.0)
INITIAL_STATE = State(E=0.12, U=0.82, I_U=0.0, S=0.0, B=0.0, B_dot=0.0)
ASSIMILATION = 0.339
```



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

```
def clamp(v: float, lo: float, hi: float) -&gt; float:
    return lo if v &lt; lo else hi if v &gt; hi else v


def observe_vector(obs: Dict[str, float], conflicts: int) -&gt; Dict[str, float]:
    """Normalize telemetry. Inputs are the stack's delayed observations."""
    return {
        "q": clamp(float(obs.get("queue_ratio", 0.0)), 0.0, 2.0),
        "load": clamp(float(obs.get("load_ratio", 0.0)), 0.0, 2.0),
        "power": clamp(float(obs.get("power_stress", 0.0)), 0.0, 1.5),
        "thermal": clamp(float(obs.get("thermal", 0.0)), 0.0, 1.5),
        "network": clamp(float(obs.get("network_stress", 0.0)), 0.0, 1.5),
        "drift": clamp(float(obs.get("drift_ratio", 0.0)), 0.0, 1.5),
        "stale": clamp(float(obs.get("stale", 0.0)), 0.0, 1.0),
        "security": clamp(float(obs.get("security_block", 0.0)), 0.0, 1.0),
        "conflict": clamp(conflicts / 4.0, 0.0, 1.0),
    }


def assimilate(x: State, o: Dict[str, float], p: Params) -&gt; State:
    """Blend the observation into the engine state and set the forcing terms."""
    q, load, pw, th, nw, dr, st, sec, cf = (o[k] for k in
        ("q", "load", "power", "thermal", "network", "drift", "stale", "security", "conflict"))
    e_obs = clamp(0.25 * q + 0.18 * max(0.0, load - 0.85) + 0.16 * pw + 0.13 * th + 0.10 * nw + 0.08 * dr
                  + 0.06 * cf + 0.04 * st, 0.0, 1.5)
    u_obs = clamp(1.0 - (0.27 * q + 0.18 * pw + 0.16 * th + 0.12 * nw + 0.12 * dr + 0.10 * cf + 0.05 * st), 0.0, 1.0)
    s_obs = clamp(0.36 * th + 0.28 * pw + 0.18 * nw + 0.10 * sec + 0.08 * st - 0.20 * q, -1.0, 1.0)

    b_obs = clamp(0.52 * pw + 0.26 * th + 0.22 * dr, -1.0, 1.5)
    i_obs = clamp(q + dr + cf, 0.0, 2.0)
    a = ASSIMILATION
    p.beta_int = clamp(0.58 * q + 0.42 * max(0.0, load - 0.75), 0.0, 1.0)
    p.beta_ext = clamp(0.33 * pw + 0.27 * th + 0.20 * nw + 0.12 * st + 0.08 * sec, 0.0, 1.0)
    return State(E=(1 - a) * x.E + a * e_obs, U=(1 - a) * x.U + a * u_obs, I_U=(1 - a) * x.I_U + a * i_obs,
                 S=(1 - a) * x.S + a * s_obs, B=(1 - a) * x.B + a * b_obs, B_dot=(1 - a) * x.B_dot + a * (b_obs - x.B)
    ↪ )


@dataclass(frozen=True)
class AllocationLaw:
    """Engine state -&gt; governing directive.

    release    capacity removed only while push &lt;= push_release (the engine's controller reports convergence)
    add        without backlog, capacity added only while push &gt;= push_add (the engine's controller reports need)
    push       equation (2) control law evaluated on the evolved state with target sigma = +1:
               u = clip(-f_U(x) + KP (1 - U), -U_AUTHORITY, U_AUTHORITY);  push = u / U_AUTHORITY
               (f_U contains the double-well drift and -(dE/dt)/E_max from equation (1))
    headroom   rho* = clamp(rho0 - kI*I_U - kE*E, rho_min, rho0)
    sizing     load_s = load * cap_now if size_at_full_cap else load
    capacity   n_req = ceil(n * load / rho*) + ceil(kq * q * n);  delta = clamp(n_req - n, -down_max, +up_max)
    guard      no capacity removal while q &gt;= guard_queue
    band       capacity removed only when n_req &lt;= n - down_band
    dwell      capacity removed only after surplus persists for down_dwell intervals and no earlier than
               down_after_add intervals after the last addition
    envelope   no capacity addition while observed power &gt;= power_ceiling (power_ceiling_backlog while q &gt;= guard_queu
    ↪ e)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

```
    security   no capacity addition while a security block is observed
    change     Terraform plans only while U &gt;= U_gate, only in the direction of the live capacity decision
    power      q &lt; guard_queue and U &gt;= U_gate: cap = clamp(cap_now * load * (1 + margin), cap_min, 1)   (trim to dema
    ↪ nd)
               q &gt;= guard_queue at the power envelope: cap = max(cap_min, cap_now - spread_step)      (spread)
               q &gt;= guard_queue below the envelope: cap held (capacity added as nodes; 1 at max_nodes)
               otherwise cap = 1; always cap &lt;= 1 - cap_gain * max(0, B - B_cap)
    rollback   a proposed rollback is authorised if S &gt;= S_rollback, U is falling, or security blocks change
    replicas   direct actuation: desired = ceil(replicas * load / rho*) (the HPA formula with the engine-set target),
               increases unless capacity is being reduced, decreases only while reducing;
               otherwise autoscaler increases approved unless reducing, decreases only while reducing
    consistency a step that expands capacity never tightens the power cap
    paging     none: the governor holds every action a paged operator would take
    """
    rho0: float = 0.914
    rho_min: float = 0.627
    kI: float = 0.252
    kE: float = 0.297
    kq: float = 0.795
    up_max: int = 2
    down_max: int = 1
    guard_queue: float = 0.05
    power_ceiling: float = 0.95
    power_ceiling_backlog: float = 1.00
    spread_step: float = 0.05
    backlog_release: bool = False
    size_at_full_cap: bool = False
    push_release: float = 0.005

    push_add: float = -9.0
    U_gate: float = 0.481
    margin: float = 0.006
    cap_min: float = 0.706
    B_cap: float = 0.40
    cap_gain: float = 0.60
    S_rollback: float = 0.55
    route_network: float = 0.20
    down_band: int = 4
    down_dwell: int = 4
    down_after_add: int = 4
    min_nodes: int = 8
    max_nodes: int = 40


@dataclass
class Governor:
    law: AllocationLaw = field(default_factory=AllocationLaw)
    mode: str = AUTOPILOT
    killed: bool = False
    x: State = field(default_factory=lambda: State(**asdict(INITIAL_STATE)))
    p: Params = field(default_factory=lambda: Params(**asdict(STACK_PARAMS)))
    t: float = 0.0
    current_cap: float = 1.0
    nodes: int = 20
    surplus_streak: int = 0
    since_add: int = 99
    last_push: float = 0.0
```



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

```
    log: deque = field(default_factory=lambda: deque(maxlen=4096))
    evolve: bool = True

    def kill(self) -&gt; None:
        """Revoke authority; the engine continues to compute and log directives."""
        self.killed = True

    def set_mode(self, mode: str) -&gt; None:
        if mode not in (OBSERVE, AUTOPILOT):
            raise ValueError(mode)
        self.mode = mode

    @property
    def has_authority(self) -&gt; bool:
        return self.mode == AUTOPILOT and not self.killed

    def step(self, obs: Dict[str, float], conflicts: int) -&gt; Dict[str, Any]:
        """Sense, assimilate, evolve, allocate."""
        o = observe_vector(obs, conflicts)
        U_before = self.x.U
        xa = assimilate(self.x, o, self.p)
        if self.evolve:
            xn, _ = macro_step(xa, self.p, self.t, target=None)
        else:
            xn = xa
        self.x = xn
        self.t += 0.1
        L = self.law

        n = int(self.nodes)
        u_push, _ = control_command(xn, self.p, self.t, +1)
        push = u_push / U_AUTHORITY
        self.last_push = push
        rho = clamp(L.rho0 - L.kI * xn.I_U - L.kE * xn.E, L.rho_min, L.rho0)
        converged = push &lt;= L.push_release
        load_sized = o["load"] * self.current_cap if L.size_at_full_cap else o["load"]
        n_req = math.ceil(n * load_sized / rho) + math.ceil(L.kq * o["q"] * n)
        delta = max(-L.down_max, min(L.up_max, n_req - n))
        if delta &lt; 0 and not converged:
            delta = 0
        if delta &gt; 0 and o["q"] &lt; L.guard_queue and push &lt; L.push_add:
            delta = 0
        if delta &lt; 0 and (o["q"] &gt;= L.guard_queue or n_req &gt; n - L.down_band):
            delta = 0
        self.surplus_streak = self.surplus_streak + 1 if delta &lt; 0 else 0
        if delta &lt; 0 and (self.surplus_streak &lt; L.down_dwell or self.since_add &lt; L.down_after_add):
            delta = 0
        ceiling = L.power_ceiling_backlog if o["q"] &gt;= L.guard_queue else L.power_ceiling
        if delta &gt; 0 and (o["power"] &gt;= ceiling or n &gt;= L.max_nodes):
            delta = 0
        if o["security"] &gt; 0.5:
            delta = min(delta, 0)
        self.since_add = 0 if delta &gt; 0 else self.since_add + 1
        if delta &lt; 0:
            self.surplus_streak = 0
        at_envelope = o["power"] &gt;= ceiling
        if o["q"] &lt; L.guard_queue and xn.U &gt;= L.U_gate:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

```
            cap = clamp(self.current_cap * o["load"] * (1.0 + L.margin), L.cap_min, 1.0)
        elif o["q"] &gt;= L.guard_queue and L.spread_step &gt; 0.0:
            if at_envelope:
                cap = clamp(self.current_cap - L.spread_step, L.cap_min, 1.0)
            else:
                cap = 1.0 if L.backlog_release else (self.current_cap if n &lt; L.max_nodes else 1.0)
        elif o["q"] &gt;= L.guard_queue and L.backlog_release:
            cap = 1.0
        else:
            cap = 1.0
        cap = min(cap, clamp(1.0 - L.cap_gain * max(0.0, xn.B - L.B_cap), L.cap_min, 1.0))
        if delta &gt; 0:
            cap = max(cap, self.current_cap)
        command = clamp(0.5 * (v_eff(xn, self.p) / max(self.p.c, 1e-9) + 1.0), 0.0, 1.0)
        d = {
            "node_delta": int(delta),
            "change_permitted": xn.U &gt;= L.U_gate and o["security"] &lt;= 0.5,
            "power_cap": cap,
            "rollback_authorized": xn.S &gt;= L.S_rollback or xn.U &lt; U_before or o["security"] &gt; 0.5,
            "route_shift": o["network"] &gt; L.route_network,
            "state": {"E": xn.E, "U": xn.U, "I_U": xn.I_U, "S": xn.S, "B": xn.B},
            "demand": rho,
            "load": o["load"],
            "command": command,
        }
        self.log.append({"mode": self.mode, "killed": self.killed, **{k: v for k, v in d.items() if k != "state"},
                         **d["state"]})
        return d

def directive_to_actions(d: Dict[str, Any], stack_state: Dict[str, Any], proposals: List[Dict[str, Any]],
                         cfg, direct_actuation: bool = False) -&gt; List[Dict[str, Any]]:
    """Directive -&gt; executable stack actions. direct_actuation: capacity is owned by the governor; no Terraform plans.
    ↪ """
    tag = "Omni-Compass"
    out: List[Dict[str, Any]] = []
    cur = int(stack_state["actual_nodes"])
    target = int(max(cfg.minimum_nodes, min(cfg.maximum_nodes, cur + int(d["node_delta"]))))
    if target != cur:
        out.append({"manager": tag, "action": "nodes", "target": target, "direction": 1 if target &gt; cur else -1})
    if not direct_actuation and d["change_permitted"] and any(p["action"] == "terraform_plan" for p in proposals):
        tf_dir = 1 if target &gt; stack_state["terraform_desired_nodes"] else -1
        live_dir = 1 if target &gt; cur else -1 if target &lt; cur else 0
        if abs(target - stack_state["terraform_desired_nodes"]) &gt;= 2 and live_dir in (0, tf_dir):
            out.append({"manager": tag, "action": "terraform_plan", "target": target,
                        "direction": 1 if target &gt; stack_state["terraform_desired_nodes"] else -1})
    reps = [int(p["target"]) for p in proposals if p["action"] == "replicas"]
    if direct_actuation:
        cur_r = int(stack_state["replicas"])
        rt = max(20, min(500, math.ceil(cur_r * float(d["load"]) / max(float(d["demand"]), 1e-9))))
        if (rt &gt; cur_r and d["node_delta"] &gt;= 0) or (rt &lt; cur_r and d["node_delta"] &lt; 0):
            out.append({"manager": tag, "action": "replicas", "target": rt, "direction": 1 if rt &gt; cur_r else -1})
        reps = []
    if reps:
        cur_r = int(stack_state["replicas"])
        rt = None
```



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

```
        if d["node_delta"] &gt;= 0 and max(reps) &gt; cur_r:
            rt = max(reps)
        elif d["node_delta"] &lt; 0 and min(reps) &lt; cur_r:
            rt = min(reps)
        if rt is not None:
            out.append({"manager": tag, "action": "replicas", "target": rt,
                        "direction": 1 if rt &gt; cur_r else -1})
    cap = float(d["power_cap"])
    if any(a["direction"] &gt; 0 and a["action"] in ("nodes", "terraform_plan", "replicas") for a in out):
        cap = max(cap, float(stack_state["power_cap"]))
    if abs(cap - stack_state["power_cap"]) &gt; 1e-9:
        out.append({"manager": tag, "action": "power_cap", "target": cap,
                    "direction": -1 if cap &lt; stack_state["power_cap"] else 1})
    if d["route_shift"]:
        out.append({"manager": tag, "action": "route_shift", "target": 1.0, "direction": 1})
    if any(p["action"] == "rollback" for p in proposals) and d["rollback_authorized"]:
        out.append({"manager": tag, "action": "rollback", "target": 1.0, "direction": -1})
    alerts = [p for p in proposals if p["action"] == "alert"]
    if alerts:
        out.append({"manager": tag, "action": "alert", "target": max(int(p["target"]) for p in alerts), "direction": 1
    ↪ })
    return out


THROUGHPUT_MODE = dict(backlog_release=True, power_ceiling=9.0, power_ceiling_backlog=9.0, spread_step=0.0, B_cap=9.0,
                       size_at_full_cap=True, kI=0.05, kE=0.40, cap_min=0.65, down_band=1, down_dwell=10,
                       down_after_add=3, kq=0.30, U_gate=0.45, push_release=0.01)

FLEET_MODE = dict(THROUGHPUT_MODE, rho0=0.829, rho_min=0.769, kI=0.072, kE=0.487, kq=0.635, down_band=3, down_dwell=1,
    ↪  down_after_add=5, push_release=0.053, cap_min=0.695, margin=0.131, U_gate=0.85, up_max=8)
FLEET_BALANCED_MODE = dict(FLEET_MODE, push_release=0.0098, push_add=0.0098)
FLEET_WEAR_MODE = dict(FLEET_MODE, push_release=0.0035, push_add=0.0035)
FLEET_DECISION_SECONDS = 60


def mode_law(mode: str, base: "AllocationLaw" = None) -&gt; "AllocationLaw":
    """Operating modes.

    power_protect  the site power limit is enforced (envelope, spread-and-throttle, bath cap; shield I4 active)
    fleet          throughput mode with constants selected on the 15-second fleet harness (fleet/), decisions every 60
    ↪  s;
                   energy-first: the equation (2) release gate is lenient and the add gate is off
    fleet_balanced fleet with the equation (2) gate active in both directions at 0.0098 (release and add)
    fleet_wear     fleet with the equation (2) gate active in both directions at 0.0035 (release and add)
    throughput     the power envelope is not enforced (same power rules as the native stack); caps released on backlog
    ↪ ;
                   capacity sized at full power (load * cap_now), so power trimming does not trigger machine starts;
                   constants selected by constrained search on development seeds (minimum energy subject to backlog,
                   health, recovery, reversal and invariant constraints)
    """
    from dataclasses import replace
    base = base or AllocationLaw()
    if mode == "power_protect":
        return base
    if mode == "throughput":

        return replace(base, **THROUGHPUT_MODE)
    if mode == "fleet":
        return replace(base, **FLEET_MODE)
    if mode == "fleet_balanced":
        return replace(base, **FLEET_BALANCED_MODE)
    if mode == "fleet_wear":
        return replace(base, **FLEET_WEAR_MODE)
    raise ValueError(mode)
```

omnicompass/nervous_system.py

*158 lines.*

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

```
"""The supervisory nervous system: one evolving engine state grants authority to every organ family.

Omni-Compass does not replace the specialist controllers. Each keeps its own mechanism:
- the HPA and VPA size pods;
- the Cluster Autoscaler and Karpenter size node pools;
- Linux schedutil picks CPU frequency;
- NVIDIA DCGM manages GPUs;
- rollout controllers and routers do their own work.

The nervous system decides, from one shared state, the envelope inside which each of them may act.

Inputs, all from the six-state engine (omnicompass/core.py, via the governor), and nothing tuned per organ:
  kappa  convergence   = 1 if push &lt;= push_release, else push_release / push. This is the equation-(2) gate made
                         continuous: the further the engine is from its basin, the less it may give back.
  h      basin health  = clamp((U - U_gate) / (1 - U_gate), 0, 1)
  sigma  stress ratio  = S / S*, where S* solves delta - alpha_s S - (3/4) beta_s S^2 = 0 (equation 6)
  nu     unmet need    = max(0, I_U) (equation 3)
  calm                 = kappa * h * (1 - clamp(sigma - 1, 0, 1)) * (1 - clamp(nu, 0, 1))
                         in [0, 1]; 1 means fully settled

Authority per organ:
  expand     may the organ add capacity or performance. Always yes, except during a security hold.
  contract   may it give capacity or performance back. Only when calm &gt;= the organ's reversibility threshold
             (theta_pods 0.5, theta_nodes 0.7: a node costs a boot to reverse) and the SLO is clean.
  step       fraction of the organ's surplus it may shed this decision. Equals calm.
  envelope   for continuous organs, the allowed range:
               cpufreq ceiling (fraction of cpuinfo_max) in [1 - 0.35 calm, 1];
               GPU power limit (fraction of max) in [1 - 0.30 calm, 1];

               site power cap in [0.65, 1];
               traffic-shift fraction in [0, 0.5 calm];
               cooling supply-air setpoint (C) in [18, 18 + 9 calm].
             Heat above 0.96 forces the frequency and GPU ceilings down to 1 - 0.35 x excess.
  batch      admit held work only if calm &gt;= 0.5 and power stress &lt; 0.9; pause pausable work if sigma &gt; 1, power
             stress &gt;= 0.95, or heat &gt;= 0.96.
  rollback   the engine's rollback authorisation (S high, U falling, or a security hold).

Global rules, applied last:
  - observe: every authority is computed and logged; execute is False everywhere.
  - kill: no authority at all.
  - security hold: no capacity organ may expand. Cooling is the one protective organ: "expand" there means more
    cooling, which adds protection and no capacity, so it stays allowed.
The safety shield (omnicompass/shield.py) stays downstream and can still veto any action.
"""
from __future__ import annotations

from dataclasses import dataclass
from typing import Any, Dict

from omnicompass.closure import stress_equilibrium

ORGANS = ("pods", "nodes", "cpufreq", "gpu", "power", "batch", "routing", "rollback")
# The living band (the founder's rule): every level the nervous system hands out lives between 5% and 95% of its range
    ↪ .
# Nothing is driven to zero (a part with no work idles down to its floor, alive and ready) and nothing is driven to it
    ↪ s
# absolute top (the last 5% is never spent). The band is applied last, to every envelope, so no organ can leave it.
```



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

```
BAND = (0.05, 0.95)


def in_band(lo: float, hi: float) -&gt; list:
    """Clip an envelope [lo, hi] (fractions of the organ's range) into the living band, keeping lo &lt;= hi."""
    b0, b1 = BAND
    h = min(b1, max(b0, hi))
    return [min(h, max(b0, lo)), h]


THETA = {"pods": 0.5, "nodes": 0.7, "cpufreq": 0.3, "gpu": 0.3, "power": 0.3, "routing": 0.5}


def clamp(x, lo, hi):
    return lo if x &lt; lo else hi if x &gt; hi else x


@dataclass(frozen=True)
class NervousInputs:
    E: float; U: float; I_U: float; S: float
    push: float
    push_release: float = 0.2
    U_gate: float = 0.5
    s_eq: float = 1.0
    security_block: float = 0.0
    slo_clean: bool = True
    power_stress: float = 0.0
    thermal: float = 0.0

    rollback: bool = False
    stale: float = 0.0              # fraction of declared senses that are blind (dropout, frozen, unreadable)
    mode: str = "autopilot"         # "observe" or "autopilot"
    killed: bool = False


def scalars(i: NervousInputs) -&gt; Dict[str, float]:
    kappa = 1.0 if i.push &lt;= i.push_release else i.push_release / max(i.push, 1e-12)
    h = clamp((i.U - i.U_gate) / max(1.0 - i.U_gate, 1e-12), 0.0, 1.0)
    sigma = i.S / i.s_eq if i.s_eq &gt; 0 else 0.0
    nu = max(0.0, i.I_U)
    calm = kappa * h * (1.0 - clamp(sigma - 1.0, 0.0, 1.0)) * (1.0 - clamp(nu, 0.0, 1.0))
    return {"kappa": kappa, "h": h, "sigma": sigma, "nu": nu, "calm": clamp(calm, 0.0, 1.0)}


def authority(i: NervousInputs) -&gt; Dict[str, Any]:
    if i.killed:
        return {"execute": False, "killed": True, "organs": {}}
    sc = scalars(i); calm = sc["calm"]
    sec = i.security_block &gt; 0.5
    hot = i.thermal &gt;= 0.96
    excess = clamp(i.thermal - 0.96, 0.0, 1.0)
    org: Dict[str, Dict[str, Any]] = {}
    seeing = i.stale &lt;= 0.0          # never give capacity back on a blind sense; expanding stays allowed (the safe si
    ↪ de)
    for o, th in THETA.items():
        ok = calm &gt;= th and i.slo_clean and seeing
        org[o] = {"expand": not sec, "contract": ok, "step": calm if ok else 0.0}
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

```
    cf_lo = 1.0 - 0.35 * calm; gp_lo = 1.0 - 0.30 * calm
    cf_hi = 1.0 - 0.35 * excess if hot else 1.0; gp_hi = 1.0 - 0.35 * excess if hot else 1.0
    org["cpufreq"]["envelope"] = in_band(min(cf_lo, cf_hi), cf_hi)
    org["gpu"]["envelope"] = in_band(min(gp_lo, gp_hi), gp_hi)
    org["power"]["envelope"] = in_band(0.65, 1.0)
    # routing is an amount moved, not a level: it may move nothing, never more than the band's top
    org["routing"]["envelope"] = [0.0, min(BAND[1], 0.5 * calm) if not sec else 0.0]
    org["cooling"] = {"expand": True, "protective": True, "contract": calm &gt;= 0.3 and seeing, "step": calm, "envelope"
    ↪ : [18.0, 18.0 + 9.0 * calm]}
    org["batch"] = {"expand": (not sec) and seeing and calm &gt;= 0.5 and i.power_stress &lt; 0.9, "contract": True, "protec
    ↪ tive_contract": True,
                    "admit": (not sec) and seeing and calm &gt;= 0.5 and i.power_stress &lt; 0.9,
                    "pause": sc["sigma"] &gt; 1.0 or i.power_stress &gt;= 0.95 or hot, "step": calm}
    org["rollback"] = {"expand": False, "contract": False, "authorized": bool(i.rollback or sec), "step": 0.0}
    return {"execute": i.mode == "autopilot", "killed": False, "security_hold": sec, "scalars": sc, "organs": org}


def from_governor(g, obs: Dict[str, Any], d: Dict[str, Any] = None, mode: str = None) -&gt; Dict[str, Any]:
    """Authority from a live omnicompass.adapter.Governor after its step (d = the step's directive)."""
    x = g.x; p = g.p; L = g.law
    return authority(NervousInputs(
        E=x.E, U=x.U, I_U=x.I_U, S=x.S, push=g.last_push, push_release=getattr(L, "push_release", 0.2),
        U_gate=getattr(L, "U_gate", 0.5), s_eq=stress_equilibrium(p.delta, p.alpha_s, p.beta_s),
        security_block=float(obs.get("security_block", 0.0)), slo_clean=bool(obs.get("slo_clean", True)),
        power_stress=float(obs.get("power_stress", 0.0)), thermal=float(obs.get("thermal", 0.0)),
        rollback=bool((d or {}).get("rollback_authorized", False)), stale=float(obs.get("stale", 0.0)),
        mode=mode or g.mode, killed=g.killed))

def node_release_gate(n: int, per_node_m: float, used_m: float, pending: int, pods_scaling_up: bool,
                      latency_breach_now: bool, rho: float, node_auth: Dict[str, Any],
                      senses_live: bool = True, last_command_landed: bool = True) -&gt; Dict[str, Any]:
    """May the machine organ give one machine back now? Attribution: an organ is held back only by stress it can cause
    or cure. The node organ reads its own engine view (fed with machine-attributable pressure: pods waiting for a
    place), and the release must also pass:
      coordination  pods are not scaling up and latency is not breached now (pods move first; machines never move
                    against them, the rule of the benchmarked coordination)
      headroom      nothing is pending, and after the release the remaining machines run at or below the engine's own
                    utilisation target rho: used / ((n - 1) x per_node) &lt;= rho
      authority     the node organ's own calm, security and stress gates (authority() above) grant contraction
      senses        every declared sense is live (afferent integrity)
      proprioception the node organ's last command landed (efferent feedback: no new order to a muscle that did not
                    carry out the previous one)
    Returns {"ok": bool, "reason": str, "util_after": float}."""
    util_after = used_m / max((n - 1) * per_node_m, 1e-9) if n &gt; 1 else float("inf")
    checks = [("one machine left", n &gt; 1), ("pods waiting", pending == 0), ("pods scaling up", not pods_scaling_up),
              ("latency breached now", not latency_breach_now), (f"util after {util_after:.2f} &gt; rho {rho:.2f}", util_
    ↪ after &lt;= rho),
              ("node organ has no contraction authority", bool(node_auth.get("organs", {}).get("nodes", {}).get("contr
    ↪ act", False))),
              ("a sense is blind", senses_live), ("last node command did not land", last_command_landed)]
    failed = [name for name, ok in checks if not ok]
    return {"ok": not failed, "reason": "; ".join(failed) or "release permitted", "util_after": util_after}
```



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

APPENDIX

# Appendix I. The License

The license under which this book and the software are made available, as it stands in the file `LICENSE` at the root of the repository.

```
Omni-Compass Evaluation License, version 1.0 (revised 11 October 2026)
SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0

Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

THIS IS NOT AN OPEN-SOURCE LICENSE. The Software is proprietary. It is published so that it can be evalu
    ↪ ated, reproduced
and audited, and for no other purpose. It is licensed, not sold. Evaluation and simulation are free of c
    ↪ harge; every
other use, and in particular every commercialization or monetization of any kind, requires a signed, pai
    ↪ d Omni-Compass
Enterprise License.

ACCEPTANCE. By downloading, copying, forking, installing, running, scanning or otherwise using any part
    ↪ of the Software,
You accept this license. If You do not accept it, You may not use the Software. If You use the Software
    ↪ on behalf of an
organization, You represent that You have authority to bind it, and "You" includes that organization and
    ↪  its
Affiliates.

1. DEFINITIONS

   1.1 "Licensor" means The Omni-Compass LLC.
   1.2 "Software" means everything Licensor publishes or makes available under this license, in any form
    ↪ : the source
       code and any object code; the Omni-Compass engine, the compass law, the governors, controllers, b
```

```
 ↪ rains, shields,
    plugs and wiring; the harnesses, tools, scripts, workflows, configurations, presets, catalogs, or
 ↪ ganisms and realms;
    the documentation, the manual, the book, the specifications, the preregistrations and the receipt
 ↪ s; the benchmarks,
    data, tables, reports and Results published by Licensor; and every update, version and copy of an
 ↪ y of them.
1.3 "Mechanism" means the methods, processes, control laws, algorithms, architectures and systems emb
 ↪ odied in or
    described by the Software, including everything described in the patent applications Licensor has
 ↪  filed.
1.4 "Evaluation" means internal use by You, free of charge, solely to test, study and assess the Soft
 ↪ ware in
    contemplation of an Enterprise License: reading it; running its tests, verifiers, benchmarks and
 ↪ simulations; and
    running it in observe, watch, dry-run or shadow mode, or in a test environment, on systems You ow
 ↪ n or control,
    where it writes to no system that serves production traffic, real users, customers, patients or p
 ↪ assengers, and to
    no power system, vehicle, machine or physical process outside a test bench.
1.5 "Simulation" means running the Software against modelled, recorded or synthetic systems, includin
 ↪ g the organisms,
    realms, traces and simulators published with the Software.
1.6 "Production Use" means any use that is not Evaluation or Simulation, including any use in which t
 ↪ he Software, the
    Mechanism or any output of either controls, governs, tunes, sizes, schedules or otherwise influen
 ↪ ces a system that
    serves production workloads or real users, or any physical process outside a test bench.

1.7 "Commercial Use" means any use for, or in connection with, commercial advantage, revenue, cost re
 ↪ duction in
    operations, or monetary or other compensation, including consulting, integration, managed, outsou
 ↪ rcing, training,
    benchmarking-for-hire and advisory services, and any use by or for a for-profit operation beyond
 ↪ Evaluation.
1.8 "Hosted Service" means making the Software, the Mechanism or any of their functionality available
 ↪  to any third
    party over a network or as a service, including software-as-a-service, platform-as-a-service, a m
 ↪ anaged or
    outsourced service, a cloud marketplace offering, or a feature of another product or service.
1.9 "Derivative" means any modification, adaptation, translation, port, extension or re-implementatio
 ↪ n of the Software
    or of the Mechanism, and any work that incorporates, is based on, or is built using the Software
 ↪ as a reference.
1.10 "Results" means the measurements, tables, receipts, logs, reports and other outputs produced by
 ↪ running the
    Software.
1.11 "Enterprise License" means a written agreement for the Software signed by Licensor and a license
 ↪ e, under which
    the licensee pays the fees it sets out; it may include order forms, riders and statements of work
 ↪ .
1.12 "Marks" means the names "Omni-Compass" and "OmniCompass", the Omni-Compass compass rose with its
 ↪  ring of Greek
    letters, and every related name, logo and trade dress of Licensor.
1.13 "Affiliate" means any entity that controls, is controlled by, or is under common control with Yo
 ↪ u, where control
    means ownership of more than half of the voting interests or the power to direct management.
```



© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

```
   1.14 "You" means the individual or entity exercising rights under this license, with its Affiliates w
    ↪ here the
       Acceptance paragraph above includes them.

2. WHAT THIS LICENSE ALLOWS

   2.1 Copyright license. Subject to Your compliance with this license, Licensor grants You a personal,
    ↪ worldwide,
       royalty-free, non-exclusive, non-transferable, non-sublicensable and revocable license to downloa
    ↪ d, copy, read,
       run and modify the Software solely for Evaluation and Simulation, and to reproduce and publish Re
    ↪ sults as section
       4 allows.
   2.2 Patent license. Subject to the same conditions, Licensor grants You a license of the same scope,
    ↪ under the patent
       rights Licensor holds or will hold in the filed applications and any patent issuing from them, so
    ↪ lely to the
       extent needed to carry out Evaluation and Simulation with the Software. No patent license is gran
    ↪ ted for any
       Production Use, Commercial Use or Hosted Service, for any Derivative used outside Evaluation or S
    ↪ imulation, or for
       any implementation of the Mechanism other than the Software itself.
   2.3 Forks. You may keep a copy or a fork of the Software on a code-hosting service, publicly or priva
    ↪ tely, only for
       Evaluation and Simulation, with this license and every notice intact. A fork confers no right bey
    ↪ ond this license.
   2.4 Reservation. Every right not expressly granted is reserved by Licensor. No right, title, license
    ↪ or interest is

       granted by implication, estoppel, exhaustion or otherwise.

3. WHAT THIS LICENSE DOES NOT ALLOW

   Unless an Enterprise License expressly permits it, You may not, and may not let or help anyone else:
   (a) make any Production Use or Commercial Use of the Software, the Mechanism or any Derivative;
   (b) commercialize or monetize the Software, the Mechanism, any Derivative or any Result in any way, d
    ↪ irectly or
       indirectly, including by charging for access, services, support, results, savings or outcomes the
    ↪ y produce;
   (c) offer the Software, the Mechanism or any Derivative as, or as part of, a Hosted Service;
   (d) sell, resell, license, sublicense, lease, rent, lend, distribute, publish as a package or binary,
    ↪  or otherwise
       transfer the Software or any Derivative, except forks under section 2.3;
   (e) incorporate the Software, any part of it or any Derivative into any product, service, platform, a
    ↪ ppliance, image,
       firmware or offering;
   (f) use the Software, the Mechanism, the documentation or the Results to design, build, train, tune,
    ↪ benchmark for
       sale, or improve any product or service that competes with Omni-Compass, or to re-implement the M
    ↪ echanism;
   (g) use the Software or the Results to train, fine-tune or evaluate any machine-learning model intend
    ↪ ed to reproduce,
       imitate or substitute for the Mechanism;
   (h) remove, alter or obscure any notice, the SPDX identifiers, the patent and trademark notices, this
    ↪  license, the
       NOTICE or DISCLOSURES.md, or circumvent, disable or work around any limit, switch or safeguard in
    ↪  the Software,
```

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

```
       including the kill switch and the hand-back to native settings;
   (i) connect the Software so that it writes to a safety-critical system, as section 10 describes;
   (j) use the Marks, except to refer to Omni-Compass accurately as TRADEMARKS.md allows; or
   (k) use the Software in violation of any law, or in breach of section 11.

4. REPRODUCING AND PUBLISHING RESULTS

   You may reproduce the benchmarks published with the Software and publish Your own Results, provided t
    ↪ hat You: (a)
   name the Software's engine version and the exact commit You ran; (b) report every measured row, the u
    ↪ nfavourable ones
   included, with the evidence class the Software assigns to it; (c) make Your complete raw files and co
    ↪ nfiguration
   available to Licensor on request; (d) do not state or imply any endorsement, certification or affilia
    ↪ tion by
   Licensor; and (e) carry the notice at the top of README.md. Publishing Results is not Commercial Use
    ↪ so long as no
   compensation is received for it, and it grants no right beyond this license.

5. THE OMNI-COMPASS ENTERPRISE LICENSE

   5.1 Every Production Use, Commercial Use, Hosted Service, distribution, incorporation, OEM or embedde
    ↪ d use, and every
       other commercialization or monetization of the Software, the Mechanism or any Derivative requires
    ↪  an Enterprise
       License, signed before the use begins and paid as it provides. Moving from Evaluation to Producti
    ↪ on Use requires
       an Enterprise License before the first write to any production system.

   5.2 An Enterprise License sets its own scope, fees, term, support, warranties, indemnities and limits
       (ENTERPRISE_LICENSE.md summarizes the standard terms; the signed agreement alone is binding). For
    ↪  its licensee and
       its scope, a signed Enterprise License governs where it differs from this license.
   5.3 Enterprise licensing: The Omni-Compass LLC, www.omni-compass.com.

6. OWNERSHIP AND INTELLECTUAL PROPERTY

   6.1 Licensor and its licensors own the Software, the Mechanism, the Marks and all intellectual proper
    ↪ ty rights in
       them, including in every Derivative to the extent it embodies them. Nothing in this license trans
    ↪ fers ownership.
   6.2 All patents, copyrights and trademarks covering the Omni-Compass engine, its mathematics and its
    ↪ software have
       been filed in the United States by The Omni-Compass LLC. No filing number is stated here, and no
    ↪ grant,
       registration or approval is claimed; the status of every filing is held by the office where it wa
    ↪ s filed. Every
       person who downloads, copies, forks, scans or runs the Software is on notice of these filings and
    ↪  of this
       license (PATENTS.md, TRADEMARKS.md).
   6.3 Any use outside this license, without an Enterprise License, is unlicensed and infringes Licensor
    ↪ 's copyright and
       other rights, in addition to breaching this license.

7. PATENT AND RIGHTS DEFENSE

   If You or any of Your Affiliates bring or join any claim, counterclaim or proceeding alleging that th
```



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

```
    ↪ e Software or
   the Mechanism infringes, misappropriates or violates any patent, copyright or other intellectual prop
    ↪ erty right, or
   challenge the validity or enforceability of any of Licensor's rights in them, every license granted t
    ↪ o You under this
   license ends automatically on the date the claim or challenge is filed.

8. FEEDBACK AND CONTRIBUTIONS

   8.1 If You give Licensor comments, suggestions, ideas, reports or other feedback about the Software,
    ↪ You grant
       Licensor a perpetual, irrevocable, worldwide, royalty-free right to use, disclose and exploit it
    ↪ for any purpose,
       without obligation of any kind.
   8.2 Contributions are accepted only under the contributor license agreement (.github/CLA.md).

9. CHANGES

   Licensor reserves the right to change the Software, its documentation, its results and these terms at
    ↪  any time,
   without notice. A changed version of these terms applies to every copy of the Software obtained, and
    ↪ to every use
   made, after it is published in the Software's repository; continued use after publication is acceptan
    ↪ ce of the
   changed terms. Licensor may discontinue the Software, or this license for future copies, at any time.

10. SAFETY

   The Software is not designed, tested or certified for safety-critical use. Under this license it may
    ↪ not write to
   any system whose failure could cause death, injury, environmental damage or significant property dama
    ↪ ge, including
   vehicles and aircraft, medical devices, power-grid protection and dispatch, industrial safety systems
    ↪  and life-safety
   systems; such systems may be studied only in Simulation. Any live connection to a physical system req
    ↪ uires an
   Enterprise License, a written safety case accepted by the system's owner, and every review that syste
    ↪ m's regulator or
   owner requires. The machines' own controls and protections stay in place; the kill switch must remain
    ↪  operable at all
   times.

11. EXPORT CONTROL, SANCTIONS AND LAWFUL USE

   You will comply with every export-control, sanctions, anti-corruption and other law that applies to t
    ↪ he Software and
   to Your use of it, including the U.S. Export Administration Regulations and the sanctions administere
    ↪ d by the U.S.
   Treasury's Office of Foreign Assets Control. You represent that You are not located in, organized und
    ↪ er the laws of,
   or ordinarily resident in a country or region subject to comprehensive U.S. sanctions, and that You a
    ↪ re not on, or
   owned or controlled by a party on, any U.S. restricted-party list. You will not use the Software for
    ↪ any end use
   prohibited by those laws, including the design, development or production of weapons of mass destruct
    ↪ ion.
```

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

```
12. DATA AND PRIVACY

   The Software contains no telemetry. It sends nothing to Licensor and makes no network connection of i
    ↪ ts own except to
   the systems You point it at. Your data and Your Results stay on Your systems and are Yours, subject t
    ↪ o section 4. If
   You send Licensor any material (for example in an issue or an e-mail), You represent that You may law
    ↪ fully do so and
   that it contains no personal data or confidential information of any third party.

13. THIRD-PARTY COMPONENTS

   Files that carry a different license notice (for example tests/third_party/) and third-party data set
    ↪ s (for example
   the PlanetLab traces under fleet/traces/ and k8s_controlplane/traces/) remain under their own terms
   (THIRD_PARTY_NOTICES.md). Third-party software the Software runs on or connects to (Kubernetes, KEDA,
    ↪  Karpenter,
   NVIDIA drivers, databases and others) is licensed by its owners under their own terms.

14. TERM AND TERMINATION

   14.1 This license lasts until it ends under this section or section 7.
   14.2 It ends automatically, without notice, if You breach it. If Your breach is Your first, was inadv
    ↪ ertent, and is
        not a breach of sections 3(a) to 3(g) or section 10, Your license is reinstated if You cure the
    ↪ breach within
        thirty days after You learn of it or after Licensor notifies You, whichever comes first.

   14.3 Licensor may end this license for any person at any time by written notice, and for all persons
    ↪ for future
        copies under section 9.
   14.4 When this license ends, You must stop all use of the Software, delete or destroy every copy and
    ↪ Derivative in Your
        possession or control, and, on request, certify in writing that You have done so.
   14.5 Sections 1, 3, 4(b) to 4(e), 6, 7, 8, 11, 12, 15 to 18 and 19 survive the end of this license.

15. NO WARRANTY

   TO THE FULLEST EXTENT PERMITTED BY LAW, THE SOFTWARE, THE MECHANISM AND THE RESULTS ARE PROVIDED "AS
    ↪ IS" AND "AS
   AVAILABLE", WITH ALL FAULTS AND WITHOUT WARRANTY OR CONDITION OF ANY KIND, EXPRESS, IMPLIED OR STATUT
    ↪ ORY, INCLUDING ANY
   WARRANTY OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE, NON-INFRINGEMENT, ACCURACY, QUI
    ↪ ET ENJOYMENT, OR
   THAT THE SOFTWARE WILL BE ERROR-FREE OR UNINTERRUPTED. NO RESULT IS A PROMISE OF ANY OUTCOME ON ANY O
    ↪ THER SYSTEM;
   MODELLED RESULTS (EVIDENCE CLASS S) ARE STATEMENTS ABOUT THE MODEL, NOT MEASUREMENTS (DISCLOSURES.md)
    ↪ . YOU USE THE
   SOFTWARE AT YOUR OWN RISK AND ARE RESPONSIBLE FOR ITS USE ON YOUR SYSTEMS.

16. LIMITATION OF LIABILITY

   TO THE FULLEST EXTENT PERMITTED BY LAW, LICENSOR, ITS MEMBERS, MANAGERS, EMPLOYEES, CONTRACTORS AND L
    ↪ ICENSORS SHALL NOT
   BE LIABLE FOR ANY INDIRECT, INCIDENTAL, SPECIAL, CONSEQUENTIAL, EXEMPLARY OR PUNITIVE DAMAGES, OR FOR
    ↪  ANY LOSS OF
```



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

```
   PROFITS, REVENUE, SAVINGS, DATA, USE OR GOODWILL, OR ANY COST OF SUBSTITUTE GOODS OR SERVICES, ARISIN
    ↪ G OUT OF OR
   RELATING TO THE SOFTWARE, THE MECHANISM, THE RESULTS OR THIS LICENSE, UNDER ANY THEORY OF LIABILITY,
    ↪ EVEN IF ADVISED OF
   THEIR POSSIBILITY. LICENSOR'S TOTAL LIABILITY FOR ALL CLAIMS SHALL NOT EXCEED ONE HUNDRED U.S. DOLLAR
    ↪ S (USD 100). THESE
   LIMITS APPLY EVEN IF A REMEDY FAILS OF ITS ESSENTIAL PURPOSE.

17. INDEMNITY

   You will defend, indemnify and hold harmless Licensor and its members, managers, employees and contra
    ↪ ctors from and
   against every claim, loss, liability, damage, cost and expense (including reasonable attorneys' fees)
    ↪  arising out of
   or relating to (a) Your use of the Software outside this license, (b) any Production Use, Commercial
    ↪ Use or Hosted
   Service without an Enterprise License, (c) Your Derivatives, (d) Your Results as You publish them, or
    ↪  (e) Your
   violation of any law or third-party right.

18. U.S. GOVERNMENT END USERS

   The Software is "commercial computer software" and "commercial computer software documentation" as th
    ↪ ose terms are
   used in 48 C.F.R. 2.101, 12.212 and 227.7202. A U.S. Government end user acquires only the rights set
    ↪  out in this
   license, consistent with 48 C.F.R. 12.212 and 227.7202-1 through 227.7202-4.

19. GENERAL

   19.1 Governing law. This license is governed by the laws of the State of Delaware and the federal law
    ↪ s of the United
        States, without regard to conflict-of-laws rules. The United Nations Convention on Contracts for
    ↪  the
        International Sale of Goods does not apply.
   19.2 Venue. The state courts of the State of Delaware and the United States District Court for the Di
    ↪ strict of
        Delaware have exclusive jurisdiction over any dispute arising out of or relating to this license
    ↪ , and You consent
        to their personal jurisdiction. Licensor may seek
        injunctive or other equitable relief in any court of competent jurisdiction, without posting a b
    ↪ ond, to protect
        its intellectual property or confidential information; a breach of sections 3 or 10 would cause
    ↪ Licensor
        irreparable harm.
   19.3 Enforcement costs. In any action to enforce this license, the prevailing party may recover its r
    ↪ easonable
        attorneys' fees and costs.
   19.4 Assignment. You may not assign or transfer this license or any right under it, by operation of l
    ↪ aw or otherwise;
        any attempt is void. Licensor may assign this license freely.
   19.5 Notices. Notices to Licensor go to The Omni-Compass LLC through www.omni-compass.com. Licensor m
    ↪ ay give notice to
        You through the Software's repository or at any address You have given it.
   19.6 Severability and waiver. If any provision is held unenforceable, it is enforced to the maximum e
    ↪ xtent permitted and
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

```
        the rest of this license remains in effect. No failure or delay in enforcing any provision is a
    ↪ waiver of it.
   19.7 Entire license. This license, with the NOTICE and DISCLOSURES.md, is the entire agreement betwee
    ↪ n You and
        Licensor about its subject; for a licensee and its scope, a signed Enterprise License governs wh
    ↪ ere it differs.
   19.8 Interpretation. "Including" means "including without limitation". Headings are for convenience o
    ↪ nly. The
        English text of this license controls over any translation.
   19.9 Copies. Every copy, fork, export, report, archive or printout of any part of the Software must c
    ↪ arry this
        LICENSE, the NOTICE and DISCLOSURES.md unchanged.

All rights reserved. Revision history: version 1.0 (evaluation and simulation only; first published 2026
    ↪ ); revised
10 October 2026 (the notice in one wording everywhere); revised 11 October 2026 (definitions, patent lic
    ↪ ense for
evaluation, hosted-service and machine-learning restrictions, results publication, patent and rights def
    ↪ ense, safety,
export control, data, U.S. Government end users, term, cure, indemnity, Delaware governing law and venue
    ↪ ).

Enterprise licensing: The Omni-Compass LLC, www.omni-compass.com
```



*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

BACK MATTER

# Contact

Licensing, pilots, evaluations and the Omni-Compass Enterprise License:

**The Omni-Compass LLC**

Owner and developer: AJ Dubra

www.omni-compass.com

Repository: github.com/The-Omni-Compass-LLC/The-Omni-Compass

Any use beyond evaluation and simulation requires a signed, paid Omni-Compass Enterprise License. Write to The Omni-Compass LLC before any production use.

# OMNI-COMPASS
## UNIFIED CONTROL AUTHORITY

EVALUATING ADMISSIBLE EVOLUTION ACROSS OPERATIONAL SYSTEMS




Unified, Transparent Decision Oversight Across Strategic Infrastructure

**THE OMNI-COMPASS LLC · www.omni-compass.com**

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only; all other use requires a signed, paid Omni-Compass Enterprise License. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com